COMP0411 중간고사 · 10월 21일(수)

컴퓨터구조, 처음부터 끝까지

시험 범위: Ch.1 ~ Ch.3 전체 + Ch.4 단일 사이클(p.1–29). Closed Book, 한글 시험, T/F + 약술·주관식(빈칸·계산·코드) 10문제 안팎. 중간고사 비중 40% [Ch0 p.5]

중간고사 공지	
시간	10월 21일(수) 오전 10:30 ~ 오후 12:00 (90분)
장소	IT5호관(융복합관) B101호
형식	Closed Book 필기시험, 단답형 위주, 한글 시험
범위	Chapter 1 ~ Chapter 4 Single-Cycle Implementation

이 노트로 공부하는 법

1. **기초 다지기부터.** 진수 변환·단위·논리·C 읽기를 모르면 Ch2부터 막힙니다. 기초 1~7을 먼저 끝내세요.
2. **전체 지도**로 무엇이 얼마나 중요한지(상·중·하) 보고 시작합니다.
3. 개념 단원은 **쉽게 말하면** → 본문 → 예제 순서로 읽고, **예제와 스스로 확인은 답을 펼치기 전에 종이에** 씁니다.
4. 단원 끝 **한 문장 요약** 칸에 노트를 덮고 핵심을 써 봅니다. Claude 세션에 붙여 넣으면 피드백을 받습니다.
5. 장을 마칠 때마다 연결된 **실전 연습**을 풉니다(기출 유형).
6. 시험 이틀 전부터 **회상 점검**과 **모의고사**(답을 다 쓴 뒤 버튼을 눌러야 정답이 보임)를 보고, 틀린 것은 **불안한 개념 목록**에 적어 그 목록부터 다시 봅니다.
7. 시험 직전에는 **한눈에 정리**만 봅니다. 쓴 답·요약·체크는 이 기기에만 저장됩니다.

상자 읽는 법

쉽게 말하면 비유로 감 잡기. 비유가 어디서 안 맞는지도 함께 적었습니다.

핵심 그대로 외울 정의·공식

함정 시험에서 자주 틀리는 지점

시험 포인트 단답형으로 나오기 쉬운 것

[Ch2 p.14] 근거 슬라이드: 2장 PDF 14쪽

PDF 외 보충 강의 PDF에 없지만 이해를 돕는 내용

교재 기준 슬라이드에 없어 교재(RISC-V 2판)로 보충한 세부

장별 진도

남은 일정 (하루 5시간)

배경 지식이 없는 상태 기준입니다. 하루 분량을 못 끝내면 다음 날로 넘기되, 10/20 모의고사 날은 지킵니다.

날짜	할 것
10/13 (화)	전체 지도 훑기 → 기초 1 ~ 4 (진법, 진수 변환, 2진 계산, 비트 논리)
10/14 (수)	기초 5 ~ 7 (단위, 디지털 논리, C·어셈블리 읽기) → Ch1 · 1 ~ 5
10/15 (목)	Ch1 · 6 ~ 10 → 실전 1 → Ch2 · 1 ~ 4
10/16 (금)	Ch2 · 5 ~ 10 → 실전 3
10/17 (토)	Ch2 · 11 ~ 14 → 실전 2, 4 → Ch3 · 1 ~ 3
10/18 (일)	Ch3 · 4 ~ 9 → 실전 5, 6
10/19 (월)	Ch4 전체 → 실전 7 → 회상 점검
10/20 (화)	**모의고사 날**: 모의고사 1회(90분) → 채점 → 불안 목록 복습 → 모의고사 2회 → 실전 8(T/F·빈칸)
10/21 (수)	아침: 한눈에 정리 + 불안 목록만. 시험 10:30

같은 교수님 · 같은 과목 수강 후기 2건

기출 경향과 대비

에브리타임 시험 정보(21년 2학기, 26년 1학기)를 정리했습니다. 문제 유형마다 어느 단원과 어느 실전 연습을 보면 되는지 연결해 두었습니다.

시험의 모양

- **형식**: T/F형, 약술형·주관식(빈칸, 계산, 코드 쓰기). 10문제 정도이고 한 문제에 소문항(가지 문제)이 붙습니다.
- **출처**: 강의 PPT 중심. 21-2 수강생은 "PPT만 잘 보면 된다"고 했고, T/F는 "교수님이 수업 시간에 말씀하신 것"에서 나왔습니다.
- **난이도 감각**: 26-1 수강생은 "족보와 내용은 비슷한데 실제로 마주치면 문제가 새롭게 느껴졌다"고 했습니다. 같은 개념을 숫자·방향·빈칸 위치를 바꿔 묻는다고 보고, 실전 연습도 그렇게 변형했습니다.
- **시험지에 주는 것**: 26-1에는 encoding 표(opcode·funct 값)를 줬습니다. 다만 "계속 주실 거라고 장담은 못 한다"고 했으니, 형식 이름과 필드 위치(비트 범위)는 외워 가세요.

핵심

계산 문제(MIPS·개선율·부동소수점·인코딩)는 **손이 기억할 때까지** 직접 풀고, 나머지는 슬라이드 문장 그대로의 정의·비교를 T/F와 빈칸으로 연습합니다. Ch1의 사소해 보이는 한두 줄(임베디드 정의 등)도 T/F로 나왔습니다.

후기에 나온 문제 → 어디서 공부하나

나온 문제	학기	개념 단원	실전 연습
컴퓨터 성능을 결정하는 3가지	21-2	Ch1 · 8	실전 1
MIPS 계산 (CPI 등 값을 줌, 단위 주의)	21-2, 26-1	Ch1 · 10	실전 1
실행 명령어·시간 표 → 성능 개선율	26-1	Ch1 · 8, 10	실전 1
C → 어셈블리 2문제, 어셈블리 → C 1문제	26-1	Ch2 · 3, 7	실전 2
레지스터 번호 주고 스택 할당 → 덧셈 → 원래 주소로 복귀	21-2	Ch2 · 8~10	실전 2
코드 실행 후 레지스터 결과값, little/big endian	26-1	Ch2 · 3, 4, 6	실전 3
어셈블리 → 기계어 → little endian으로 쓰기 (encoding 표 줌)	26-1	Ch2 · 5, 12, 13	실전 4
빈칸: branch offset, encoding 이름	26-1	Ch2 · 5, 12	실전 4, 8
부호 없는/있는/부호 다른 곱셈 upper·lower 빈칸	21-2	Ch3 · 3	실전 5
십진수 → 이진수, A+B·A×B (A = −0.5, B = 2.625)	26-1	Ch3 · 6~8	실전 6
16비트(1-5-10)로 표현	26-1	Ch3 · 7	실전 6
단일 사이클 데이터패스 그림 빈칸(ALU, adder 등)	21-2	Ch4 · 3, 4	실전 7
ALUSrc·Branch·MemtoReg 등 0/1/X 표 (ld, sd, beq, add)	21-2	Ch4 · 6	실전 7
datapath·control line, 라인별 참조 레지스터 번호	26-1	Ch4 · 4, 6, 7	실전 7
프로세서를 구성하는 두 가지, 레지스터가 있는 하드웨어 이름	21-2	Ch1 · 5, Ch4 · 1, 3	실전 7
빈칸: 캐시 메모리	26-1	Ch1 · 5	실전 8
T/F: 임베디드 등 Ch1 한두 줄, 수업 중 언급	21-2, 26-1	Ch1 전체	실전 8

헷갈리기 쉬운 두 가지

함정

ld, sd가 나와도 당황하지 않기. 21-2 시험은 64비트판 RISC-V 이름(ld = load doubleword, sd = store doubleword)을 썼습니다. 지금 강의 슬라이드는 lw, sw입니다. 제어 신호 값은 ld = lw, sd = sw로 똑같습니다. PDF 외 보충

16비트 부동소수점은 PDF에 없습니다. 그래도 26-1에 나왔습니다. IEEE 754와 규칙이 같고 칸 수만 1-5-10, bias 15입니다. 실전 6에서 연습합니다.

시험 전까지 이렇게

1. 개념 단원을 일정대로 읽되, 각 장을 마치면 해당 실전 연습을 바로 풉니다.
2. 실전 연습은 답을 펼치기 전에 종이에 끝까지 씁니다. 틀린 문제는 번호를 적어 두고 다음 날 다시 풉니다.
3. 10/20 모의고사 날에는 실전 8(T/F·빈칸)을 처음부터 끝까지 풀고, 틀린 문장의 단원만 다시 읽습니다.

1단계 · 무엇이 얼마나 나오나

전체 지도

시험 범위 전체를 주제별로 묶고 출제 가능성(상·중·하)을 매겼습니다. 공부를 시작하기 전에 한 번, 각 장을 끝낸 뒤 다시 한 번 훑어보며 "상" 주제를 설명할 수 있는지 확인하세요. 표 오른쪽 끝은 좌우로 밀어서 봅니다.

읽는 법

- 상 단답·계산으로 거의 확실히 나오는 것. 기출 후기에도 나온 것이 많습니다.
- 중 빈칸·T/F로 나올 수 있는 것.
- 하 한 줄 정의 정도만 알면 되는 것. 다만 21-2 후기처럼 Ch1의 한두 줄이 T/F로 나오기도 했습니다.

1. 핵심 개념 지도

Ch1 · 컴퓨터 추상화와 기술

주제	핵심 개념	슬라이드	출제	이유
컴퓨터 분류	PC(범용, 비용·성능 균형) · 서버(네트워크로 접근, 대용량·고성능·고신뢰) · 임베디드(다른 기기 안, 정해진 응용, 전력·성능·비용 제약이 엄격). Post-PC: PMD(배터리·무선 인터넷), 클라우드 = WSC, SaaS. 자동차 ECU = 차량 전장 제어용 임베디드	Ch1 p.3–7	하	정의 짝짓기 정도
구조 정의·추상화	컴퓨터 구조 = ISA + Machine Organization. ISA = 하드웨어와 최하위 소프트웨어 사이의 추상 인터페이스(= architecture), ABI = ISA + OS 인터페이스, 구현 = 아키텍처를 따르는 하드웨어	Ch1 p.9–11, 17	중	ISA·ABI 정의는 단답 단골
8대 위대한 아이디어	무어의 법칙을 고려한 설계(Design for Moore's Law: 집적 자원 18~24개월마다 2배), 추상화로 설계 단순화(Abstraction), 자주 생기는 경우를 빠르게(Common case fast), 병렬성(Parallelism), 파이프라이닝(병렬성의 특정 형태), 예측(기다리기보다 추측해 먼저 시작하는 게 평균적으로 빠름), 메모리 계층(Hierarchy of memories), 중복을 통한 신뢰성(Dependability via redundancy)	Ch1 p.12	상	나열·짝짓기 단답에 최적, 영어 명칭도 함께
프로그램 아래 계층	응용SW → 시스템SW → HW. OS 역할 = 입출력 처리, 저장공간·메모리 할당, 자원 공유·작업 스케줄링. 컴파일러 = 고급어 → 명령어, 고급어 장점 = 생산성·이식성, 어셈블리 = 기계어의 기호 표현, 어셈블러 = 기호 → 2진	Ch1 p.13–14	중	OS 역할·도구 역할 목록형
5대 구성 요소	입력·출력·메모리·데이터패스·제어, 프로세서 = 데이터패스 + 제어. 입력은 메모리에 쓰고 출력은 메모리에서 읽음, 제어는 데이터패스·메모리·입출력에 신호를 보냄, 모든 종류의 컴퓨터에 동일. iPhone: I/O가 대부분, A12 = 큰 코어 2 + 작은 코어 4, 2.5 GHz	Ch1 p.15–16	상	Ch4와 직결, 단답 단골
메모리 종류	DRAM = 주기억, SRAM = 캐시(빠르지만 집적도가 낮아 비쌈), 둘 다 메모리 계층의 층. 휘발성 = 1차(주) 기억장치(DRAM·SRAM), 비휘발성 = 2차 기억장치(하드디스크·플래시·CD/DVD), 더 크고 느림	Ch1 p.17–18	중	비교형 단답
네트워크	장점 3: 통신, 자원 공유(I/O 장치), 원격 접근(가까이 있을 필요 NOT). LAN = 한 건물 안(이더넷), WAN = 수백 km~대륙(인터넷), 무선 = Wi-Fi·블루투스	Ch1 p.19	중	목록형 단답
반도체·IC	트랜지스터 = 전기로 제어되는 on/off 스위치, IC = 수십~수백 개, VLSI = 수백~수백만 개. 실리콘 = 반도체(전기가 잘 통하지 않음 → 화학 공정으로 도체·절연체·스위치로)	Ch1 p.20–21	하	정의 단답
칩 비용·수율	다이당 비용 = 웨이퍼당 비용 / (웨이퍼당 다이 수 × 수율), 웨이퍼당 다이 수 ≈ 웨이퍼 면적 / 다이 면적, 수율 = 1 / (1 + 결함밀도 × 다이면적 / 2)^N(슬라이드도 지수 N, 교재 예제는 N = 2). 면적·결함률과 비선형. 웨이퍼 비용·면적 고정, 결함률 = 제조 공정, 다이 면적 = 구조·회로 설계. 예: Ice Lake 300 mm, 506개, 10 nm	Ch1 p.22	중	공식 빈칸·결정 요인 단답
성능 고려 요소	클럭·코어 수, 버스 속도, 캐시/메모리 크기, 디스크, GPU, 전력, 가격 등	Ch1 p.23	하	부수 내용
성능 정의	응답 시간(= 실행 시간) vs 처리량(= 대역폭). 더 빠른 프로세서로 교체 → 둘 다 개선, 프로세서 추가 → 처리량만 개선. 성능 = 1/실행시간, 성능X/성능Y = 실행시간Y/실행시간X = n → 'X가 n배 빠르다'	Ch1 p.24	상	O/X·계산 모두 출제
시간 측정	경과 시간(wall clock = 응답 시간) = 디스크·메모리·I/O·OS 오버헤드 전부 포함. CPU 시간 = CPU가 그 작업에 쓴 시간, I/O·다른 프로그램 시간 포함 NOT, user CPU 시간 + system CPU 시간	Ch1 p.25	중	NOT 강조, 용어 구분

주제	핵심 개념	슬라이드	출제	이유
클럭	클럭 주기(예 250 ps) ↔ 클럭 속도(예 4 GHz)는 역수. CPU 시간 = 사이클 수 × 주기 = 사이클 수 / 클럭 속도. 성능 개선 = 사이클 수 줄이기 or 주기 줄이기, 둘은 트레이드오프	Ch1 p.26–27	상	단위 환산 계산
CPU 성능 공식	CPU 시간 = IC × CPI × 클럭 주기 = IC × CPI / 클럭 속도. IC는 프로그램·ISA·컴파일러가, CPI(명령어당 평균 사이클)는 CPU 하드웨어가 정함, CPI는 평균. 요소별 영향 표는 '한눈에 정리' 표 1	Ch1 p.28–29	상	계산 문제 1순위
교재 지정 예제	슬라이드가 교재 pp.34–37(성능 공식), pp.41–42(전력) 예제를 직접 지정 → '한눈에 정리' 표 2	Ch1 p.27, 28, 31	상	교수가 직접 짚은 예제
전력과 전력 장벽	에너지 ∝ 용량성 부하 × 전압², 전력 ∝ 용량성 부하 × 전압² × 주파수. 20년간 클럭 1000배인데 전력 30배뿐 = 전압 5 V → 1 V(세대당 약 15%). 전력 장벽: ① 전압을 더 낮출 수 NOT(누설) ② 열을 더 뺄 수 NOT(냉각 비용) → 2006년부터 멀티코어(응답 시간보다 처리량에 이득)	Ch1 p.30–32	상	NOT 강조 두 줄, 이유 단답
암달의 법칙	T개선 = T영향/n + T비영향. 100초 중 곱셈 80초, 전체 2.5배 → 40 = 80/n + 20 → n = 4. 곱셈 외만 개선 → 불가능(최대 1.25배). 교훈 = 자주 생기는 경우를 빠르게 (교재 원문은 5배, 슬라이드 수치 2.5배로 외울 것)	Ch1 p.33	상	계산 단골
MIPS의 함정	MIPS = IC / (실행시간 × 10^6) = 클럭 속도 / (CPI × 10^6). 빠른 컴퓨터가 MIPS도 높지만 ① ISA 차이 ② 명령어 복잡도 차이를 반영 NOT ③ 같은 CPU에서도 프로그램마다 CPI가 다름	Ch1 p.34	중	'왜 부적절한가' 단답

Ch2 · 명령어: 컴퓨터의 언어

주제	핵심 개념	슬라이드	출제	이유
명령어와 ISA	명령어 = opcode(무엇을) + operand(대상), 명령어 집합 = 어휘. 마이크로아키텍처 = ISA 구현, SW에 드러나지 않음(파이프라이닝, 추측 실행, 메모리 접근 스케줄링, 산술 장치). 예: x86 ISA의 구현 286 … Core i	Ch2 p.2–5	중	정의 단답
CISC vs RISC	CISC(x86): 복잡한 가변 길이, 어셈블리·컴파일러 쉬움, 명령어 메모리 절약 / CPU 설계 어려움(마이크로코드). RISC(ARM·MIPS·RISC-V): 단순 고정 길이, 설계 쉬움·클럭 높음 / 코드 길어짐·메모리 많이 씀. 현대 x86은 내부적으로 RISC 기법 사용	Ch2 p.6–7	상	비교형 단답 최적
RISC-V 배경	공개 표준 ISA, UC Berkeley의 다섯 번째 RISC 설계, RISC-V Foundation이 관리, 임베디드 시장	Ch2 p.8	하	정의 단답
설계 원칙 3가지	① 단순성은 규칙성을 선호(모든 산술 = 소스 2 + 목적지 1, a=b+c+d+e는 명령어 3개) ② 작을수록 빠르다(레지스터가 많으면 클럭이 늘어날 수 있음) ③ 좋은 설계는 좋은 타협(형식 여러 개지만 모두 32비트, 최대한 비슷하게)	Ch2 p.9–10, 23	상	문장 그대로 나오기 쉬움
레지스터	32 × 32비트 레지스터 파일 x0~x31, 32비트 = word, 64비트 = doubleword. x0 = 항상 0 → sub x1, x0, x1 = 부호 반전, beq x0, x0, L = 무조건 분기. 역할표는 '한눈에 정리' 표 4	Ch2 p.10, 16, 29	상	x0 활용 단답 단골
메모리 피연산자	산술은 레지스터에서만 → 메모리는 load/store. 바이트 주소, 연속 word 주소는 4씩 차이. A[12] = h + A[8] → lw x9, 32(x22) · add x9, x21, x9 · sw x9, 48(x22). 레지스터가 훨씬 빠름 → 덜 쓰는 변수만 메모리로 spill	Ch2 p.11–15	상	오프셋 계산
엔디언·정렬	RISC-V = 리틀 엔디언(최하위 바이트가 가장 낮은 주소). 0x00112233 → 빅: 00 11 22 33, 리틀: 33 22 11 00. 정렬 제한 없음(word가 4의 배수 주소에서 시작할 필요 NOT)	Ch2 p.13	상	배치 문제
즉시값	addi(가장 많이 쓰는 명령 중 하나), 상수를 메모리에서 읽는 것보다 빠르고 에너지 적음	Ch2 p.16	중	addi 사용
수 표현	MSB = 가장 왼쪽, LSB = 가장 오른쪽. 무부호 0 ~ $2^{32} - 1$. 부호 표현 3가지: 부호-크기, 1의 보수, 2의 보수(최종 채택, -2^{31} ~ $2^{31} - 1$, MSB = 부호). 부정 = ~x + 1, 부호 확장 = 부호 비트를 왼쪽에 복제(−2: 1111 1110 → 1111…1110)	Ch2 p.17–20	상	변환 계산 필수
명령어 형식	모두 32비트(규칙성). R = 즉시값 없는 산술·논리, I = load·즉시값 산술, S = store(12비트 즉시값을 둘로 쪼개 rs1·rs2 위치 유지). 비트 폭은 '한눈에 정리' 표 3. 주의: p.21 텍스트 'add x0, x20, x21'의 2진수는 rd = x9	Ch2 p.21–24	상	인코딩·디코딩 핵심
논리·시프트 연산	and = 마스크(일부만 남기고 0), or = 일부를 1로, xor = 다르면 1 → NOT 명령 없이 xor 111…1로 대신. sll/srl/slli/srli, sra = 오른쪽 시프트하며 부호 비트로 채움(1100 0011 → 1110 0001). 예제 값은 '한눈에 정리' 표 9	Ch2 p.25–27	중	결과값 계산
조건 분기	beq/bne(조건 분기), blt/bge(부호), bltu/bgeu(무부호). 0xFFFFFFFF vs 1: blt 참(−1<1), bltu 거짓. if-else: bne로 Else, beq x0,x0,Exit. while: slli → add → lw → bne → addi → beq. switch = 분기 주소 표(branch table)	Ch2 p.28–31	상	C → 어셈블리 번역
프로시저 호출	caller(호출자)·callee(피호출자). 6단계: ① 인자 배치 ② 제어 이동 ③ 저장 공간 확보 ④ 작업 수행 ⑤ 결과 배치 ⑥ 원래 위치로 복귀. 레지스터 9개 = x10~x17(인자, x10·x11 반환값) + x1(복귀 주소). jal x1, 라벨 = 점프하며 다음 주소를 x1에 / jalr x0, 0(x1) = x1 주소로 복귀. PC = 다음 실행할 명령어 주소	Ch2 p.32–36	상	단계 나열·레지스터 역할

주제	핵심 개념	슬라이드	출제	이유
스택	인자 8개·반환 2개를 넘거나 레지스터 spilling 때 사용. LIFO, push/pop 명령 없음(lw/sw로 접근), sp = x2, 높은 → 낮은 주소로 자람, 레지스터 하나당 sp 4 조정. leaf 예: `addi sp, sp, -12` (3개 저장)	Ch2 p.37–38	상	sp 조정량 계산
레지스터 보존 규약	p.39: 19개를 두 그룹 = caller-saved 임시 x5~7·x28~31 / callee-saved 저장 x8~9·x18~27 → leaf 예제에서 x5·x6 저장은 불필요, x20만 필요. p.41·43 호출 규약: caller 쪽에 x10~17 추가, callee 쪽에 x1 추가	Ch2 p.39–43	상	누가 저장하나, '19개' 함정
재귀 프로시저	nonleaf는 x1·인자를 스택에 저장. fact: `addi sp, sp, -8` (x1·x10 두 개), bge x5, x0, L1, fact(3) = 6	Ch2 p.41–42	중	코드 빈칸형
메모리 배치	procedure frame = activation record, fp(x8) = 프로시저 안에서 변하지 않는 기준. 영역: text(기계어) · static data(상수·정적 변수) · heap(동적 할당, malloc/free, 연결 리스트) · stack(지역 변수·레지스터 저장), 스택과 힙은 서로를 향해 자람	Ch2 p.44–45	중	용어 단답
큰 상수	lui(U형식): 20비트 상수를 31~12번 비트에, 하위 12비트는 0. 이어서 addi로 하위 12비트. 아주 먼 점프 = lui + jalr	Ch2 p.46, 48	중	lui + addi 조합
분기 주소·형식	SB 형식: −4096~4094바이트, 2의 배수(짝수 주소만), 예 bne opcode 1100011·funct3 001. jal = UJ 형식, 20비트 즉시값(opcode 1101111). PC 상대 주소: 대부분의 분기 목표가 가까움 → PC를 기준 레지스터로, 분기 $\pm 2^{10}$ word, jal $\pm 2^{18}$ word	Ch2 p.47–48	상	범위·계산
주소 지정 방식 4가지	즉시값(addi·xori·srai), 레지스터(add·sub·xor), 베이스/변위(lw·sw·jalr), PC 상대(beq·bne·jal). 한 연산이 여러 방식을 쓸 수 있음(add/addi) → '한눈에 정리' 표 5	Ch2 p.49	상	jalr = 베이스 방식 함정
기계어 디코딩	① 2진 변환 ② opcode로 형식 결정 ③ 형식대로 필드 분리 ④ 어셈블리로. 0x00578833 → '한눈에 정리' 표 9	Ch2 p.50	상	손으로 푸는 단답
프로그램 번역 4단계	컴파일러 → 어셈블러 → 링커 → 로더	Ch2 p.51	중	슬라이드는 그림뿐 (교재 기준)

Ch3 · 컴퓨터 연산

주제	핵심 개념	슬라이드	출제	이유
덧셈·뺄셈	오른쪽부터 비트별로 더하며 자리올림. 뺄셈 = 두 번째 피연산자를 부정(2의 보수)해서 더하기	Ch3 p.2–3	중	기본기
오버플로	결과를 하드웨어로 표현할 수 없음. 부호 다른 덧셈·부호 같은 뺄셈에선 생기지 않음. 생기는 경우: + + + = −, − + − = +, + − − = −, − − + = +. 무부호(주소, 무시): 덧셈은 합 < 피가수, 뺄셈은 차 > 피감수. C/C++·Java는 무시, Ada·Fortran은 프로그램에 통보	Ch3 p.4–6	상	조건 단답 단골
곱셈 하드웨어	곱 길이 = 피승수 길이 + 승수 길이. 1판: 32비트 승수, 64비트 피승수·곱 레지스터, 64비트 ALU, Test-Add-Shift(피승수 왼쪽·승수 오른쪽 시프트). 개선판: 32비트 피승수·ALU, 승수 레지스터 없음(64비트 곱 레지스터 오른쪽 절반), 곱 레지스터를 오른쪽 시프트. 예 $0010_2 \times 0011_2 = 0000\ 0110_2$	Ch3 p.7–11	상	반복 표 채우기
부호 있는 곱셈·명령어	양수로 바꿔 31회 반복 후 부호가 다르면 부정, 여러 덧셈기로 빠른 곱셈(트리). mul(하위 32비트), mulh(둘 다 부호), mulhu(둘 다 무부호), mulhsu(부호 × 무부호) → 상위 32비트. 3비트 예 $101_2 \times 101_2$: 무부호 25(011 001), 부호 9(001 001), 부호-무부호 −15(110 001), 하위는 같고 상위만 다름	Ch3 p.12–13	중	mulh가 왜 셋인가
나눗셈 하드웨어	피제수 = 몫 × 제수 + 나머지, 제수 ≠ 0. 1판: 곱셈기 1판과 같은 HW, 시프트 방향 반대, 덧셈 대신 뺄셈(Subtract-Test-Shift), 제수는 왼쪽 절반에서 시작. 빼고 → 나머지 부호 검사 → 음수면 복원(제수 다시 더함)·몫 0, 아니면 몫 1 → 제수 오른쪽 시프트. n+1회(4비트 예 5회). 개선판: 개선 곱셈기와 같은 HW, 좌우 시프트. 예 $0111_2 \div 0010_2$ = 몫 0011_2, 나머지 0001_2	Ch3 p.14–18	상	반복 표 채우기
부호 있는 나눗셈	절댓값으로 계산 → 부호가 다르면 몫 부정 → 나머지 부호 = 피제수 부호. −7÷2 → 몫 −3·나머지 −1, 7÷−2 → 몫 −3·나머지 1, −7÷−2 → 몫 3·나머지 −1. C의 %도 같은 규칙	Ch3 p.19	상	함정 단답
빠른 나눗셈·명령어	나눗셈은 뺄셈이 나머지 부호에 따라 달라져 곱셈처럼 병렬화할 수 없음 → SRT(표로 몫 여러 비트 예측). div·divu·rem·remu. 2의 거듭제곱 곱셈 = 왼쪽 시프트, 나눗셈 = 오른쪽 시프트(음수는 부호 확장)	Ch3 p.20	중	이유·명령어 이름 단답
정규화·정밀도 vs 범위	과학적 표기 = 소수점 왼쪽 한 자리, 앞에 0이 없으면 정규화(1.0×10^{-9} O, 10.0×10^{-10}·0.1×10^{-8} X), binary point. 정규화 표기의 장점 3: ① 데이터 교환 단순화 ② 알고리즘 단순화 ③ word 안 정확도 증가. 가수↑ = 정밀도, 지수↑ = 범위	Ch3 p.23–25	중	나열·O/X 단답
FP 오버/언더플로	오버플로 = 지수가 너무 커서 지수 칸에 안 들어감, 언더플로 = 음의 지수가 너무 커서 안 들어감. double(64비트)로 가능성 줄임	Ch3 p.26	중	정의 구분
IEEE 754 형식	IEEE 754-1985, 이식성 문제 해결, 설계자 William Kahan(튜링상). 단정도 1/8/23(bias 127), 배정도 1/11/52(bias 1023), 숨은 1(24비트 정밀도), 지수 저장값 1~254, 지수 칸 0과 255는 예약(NaN = 0/0 등). 정리는 '한눈에 정리' 표 6	Ch3 p.27–30	상	변환 계산 1순위
bias를 쓰는 이유	정수 비교로 정렬하려고: ① 부호가 MSB → 빠른 비교 ② 지수가 가수보다 앞 → 정렬 단순 ③ 음수 지수가 크게 보이는 문제 → bias로 해결	Ch3 p.29	중	이유 단답
IEEE 754 변환 예	$-0.75 = -1.1_2 \times 2^{-1}$ → S 1, 지수 126(01111110), F 100…0 → 0xBF400000	Ch3 p.31	상	거의 그대로 출제 가능
FP 덧셈·곱셈 절차	덧셈 4단계: ① 작은 지수 쪽 정렬(오른쪽 시프트) ② 가수 더하기 ③ 정규화·오버/언더플로 검사 ④ 반올림(정규화가 깨지면 ③으로). 예 0.5 + (−0.4375) = $1.000_2 \times 2^{-4} = 0.0625$. 곱셈 5단계: ① 지수 더하기(bias 한 번 빼기) ② 가수 곱 ③ 정규화·	Ch3 p.32–36	상	순서·계산

주제	핵심 개념	슬라이드	출제	이유
	검사 ④ 반올림 ⑤ 부호 결정. 예 0.5 × (−0.4375) = $-1.110_2 \times 2^{-3} = -0.21875$. FP 덧셈기는 정수 덧셈기보다 훨씬 복잡			
RISC-V FP 명령어	fadd/fsub/fmul/fdiv/fsqrt .s/.d, feq/flt/fle는 정수 레지스터에 0/1 → beq/bne로 분기. 별도 레지스터 f0~f31(f0은 0 고정 아님), flw/fsw·fld/fsd	Ch3 p.37	하	f0 함정
반올림	FP는 근삿값. 4가지 모드: 0 쪽(버림), +∞ 쪽, −∞ 쪽, 가장 가까운 짝수(기본). guard·round 2비트 유지: $2.56 \times 10^0 + 2.34 \times 10^2$ → 없으면 2.36, 있으면 2.37. G = LSB 바로 다음, R = 그다음, S(sticky) = 나머지 전체 OR. GRS 000~011 내림, 101~111 올림, 100 = LSB가 0인 쪽	Ch3 p.38–40	중	GRS 판정

Ch4 · 프로세서 (단일 사이클까지, p.1–29)

주제	핵심 개념	슬라이드	출제	이유
성능 3요소의 결정 주체	IC = ISA·컴파일러, 클럭 주기·CPI = CPU 하드웨어(마이크로아키텍처, 공정 기술)	Ch4 p.2	중	Ch1과 연결
구현 대상 명령어	lw, sw / add, sub, and, or / beq. 공통 두 단계: ① PC로 명령어 인출 ② 레지스터 1~2개 읽기. 이후는 명령어 종류에 따라 다름	Ch4 p.3	상	목록 단답
명령어 실행 흐름	add x5,x6,x7: 인출 → x6·x7 읽기 → 덧셈 → x5에 쓰기. lw x5,40(x6): 인출 → x6 읽기 → x6+40 주소 계산 → 메모리 읽어 x5에 쓰기	Ch4 p.5–6	중	순서 서술
MUX·제어 유닛	데이터 선은 그냥 합칠 수 없음 → MUX 필요, 데이터패스 제어 → 제어 유닛 필요. 기본 구현 = MUX 3개 + 제어 유닛, 프로세서 = 데이터패스 + 제어	Ch4 p.7–8	중	이유 단답
조합 소자 vs 상태 소자	조합 = AND 게이트·ALU(출력은 입력만의 함수). 상태 = 레지스터·메모리(정보 저장), 입력 최소 2개(쓸 데이터, 클럭), 출력 1개(이전 클럭에 쓴 값)	Ch4 p.9	상	분류 단답
클로킹 방식	클로킹 방식 = 신호를 언제 읽고 쓰는지 정의(하드웨어를 예측 가능하게). 엣지 트리거 = 상태 소자 값은 클럭 엣지에서만 갱신, 조합 논리는 엣지 사이에 동작, 가장 긴 지연이 클럭 주기 결정	Ch4 p.10	상	정의 단답
데이터패스 구성 요소	명령어 메모리, PC(현재 명령어 주소), 가산기(PC+4), 레지스터 파일(32개, 번호로 읽고 씀), ALU(32비트 입력 2개, 4비트 제어, 결과가 0이면 Zero = 1), ImmGen(12비트 → 32비트 부호 확장), 데이터 메모리	Ch4 p.11–13	상	요소 ↔ 역할
분기 데이터패스	beq 피연산자 3개(레지스터 2개 + 12비트 오프셋). 목표 주소 = PC + (부호 확장 오프셋 << 1, 반 word 단위 → 짝수 주소만). ① ImmGen + 가산기로 목표 계산 ② ALU 뺄셈 후 Zero 검사. 분기하면 목표 주소, 안 하면 PC+4	Ch4 p.14–15	상	계산·이유
단일 사이클 데이터패스	모든 명령어를 한 클럭에 → 자원은 명령어당 한 번만 → 명령어 메모리와 데이터 메모리 분리, 여러 입력 중 고를 곳에 MUX	Ch4 p.16–17	상	'왜 메모리가 둘?'
ALU 제어	ALU 동작 4가지: lw/sw = 덧셈, beq = 뺄셈, R형 = funct7·funct3에 따라 AND·OR·add·sub. ALUOp(2비트) + funct → 4비트 ALU 제어, 다단 디코딩, don't care(2^{12} = 4096 조합 압축) → '한눈에 정리' 표 7	Ch4 p.18–20	상	진리표 채우기
주 제어 유닛 설계	opcode [6:0], funct3 [14:12]·funct7 [31:25] = 확장 opcode. rs1 = R형·분기의 첫 피연산자이자 load/store의 베이스, rs2 = R형·분기의 둘째 피연산자이자 store에서 메모리로 복사될 레지스터, rd = R형·load의 목적지	Ch4 p.21	상	필드 위치 단답
ImmGen 단순화	SB·UJ의 즉시값 비트 뒤섞기 → 즉시값 19~12번 MUX 3:1 → 2:1, 10~1번 5/4:1 → 3/2:1. 대가는 어셈블러만 부담	Ch4 p.22–23	하	이유 한 줄
제어 신호	RegWrite, ALUSrc, MemRead, MemWrite, MemtoReg, Branch, ALUOp. PCSrc만 opcode로 정할 수 없음: beq이고 ALU Zero일 때 1 (교재 식 Branch AND Zero) → '한눈에 정리' 표 8	Ch4 p.24–25	상	명령어별 신호 값 표
명령어별 데이터패스 동작	R형(add x1,x2,x3): 레지스터 2개 → ALU → rd. lw: x2 + 오프셋 → 메모리 → rd. beq: x1 − x2의 Zero로 PC+4 또는 PC+오프셋	Ch4 p.26–28	상	경로 서술·신호 값
단일 사이클을 안 쓰는 이유	모든 명령어의 클럭 주기가 같아야 함 → 최장 경로 = lw(명령어 메모리 → 레지스터 파일 → ALU → 데이터 메모리 → 레지스터 파일, 5개 직렬) → 명령어마다 주기를 다르게 할 수 없음 → '자주 생기는 경우를 빠르게' 위반 → 파이프라이닝으로 개선	Ch4 p.29	상	서술 단골

2. 가장 흔히 헷갈리는 오개념 3가지

오개념 1

틀린 생각 클럭 속도(GHz)가 높거나 MIPS가 높으면 그 컴퓨터가 더 빠르다.

올바른 이해 성능은 **CPU 시간 = 명령어 수 × CPI × 클럭 주기** 세 가지를 모두 곱해서 판단한다 [Ch1 p.28–29]. 클럭이 빨라도 CPI가 크거나 명령어 수가 많으면 더 느릴 수 있다 ('한눈에 정리' 표 2의 교재 예제 3: 명령어가 더 많은 코드가 더 빠름). MIPS = 클럭 속도 / (CPI × 10^6). 슬라이드도 "빠른 컴퓨터가 MIPS가 높다"고는 말하지만, ① ISA가 다른 컴퓨터끼리 비교할 수 없고 ② 명령어 복잡도 차이를 반영하지 못하며 ③ 같은 CPU에서도 프로그램마다 CPI가 달라 MIPS가 하나로 정해지지 않는다 [Ch1 p.34]. O/X 문제에서는 이 세 가지 한계를 기준으로 판단할 것.

오개념 2

틀린 생각 `A[8]`을 읽으려면 `lw x9, 8(x22)`라고 쓴다. 분기 오프셋도 명령어 개수 그대로다.

올바른 이해 RISC-V 메모리는 **바이트 주소**라서 4바이트 word 배열의 8번째는 8 × 4 = **32** 바이트 떨어져 있다 → `lw x9, 32(x22)` [Ch2 p.12, p.14]. 분기 오프셋도 **바이트 단위**라 명령어 k개 뒤로 가려면 4k다. 다만 항상 짝수라서 0번 비트는 저장하지 않고 imm[12:1]만 넣으며, 하드웨어가 1비트 왼쪽 시프트해서 PC에 더한다 [Ch4 p.14]. 범위: 분기 −4096~4094바이트(= $\pm 2^{10}$ word), jal $\pm 2^{18}$ word [Ch2 p.47–48]. 슬라이드 p.48은 범위를 word 단위로 적었으니 단위를 확인할 것.

오개념 3

틀린 생각 IEEE 754의 지수 칸에는 실제 지수가 그대로 들어가고, 가수 칸에 맨 앞 1도 저장된다.

올바른 이해 지수 칸 = **실제 지수 + bias**(단정도 127, 배정도 1023). 정규화된 수의 맨 앞 1은 **저장하지 않는 숨은 비트**라서 값 = $(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$ [Ch3 p.28]. 예: −0.75 = $-1.1_2 \times 2^{-1}$ → 지수 칸 126 = `01111110`, Fraction = `10 0…0` [Ch3 p.31].

주의: [Ch3 p.25]는 IEEE 754 이전의 일반형을 설명하면서 지수를 "2의 보수"라고 적었다. 시험에서 IEEE 754 지수를 물으면 항상 **bias 방식** [Ch3 p.28]으로 답한다. bias를 쓰는 이유 = 정수 비교만으로 크기 정렬이 되게 하려고 [Ch3 p.29].

3. 서로 연결되는 개념 3쌍

① 명령어 형식 (Ch2) ↔ 단일 사이클 데이터패스 (Ch4)

R·I·S·SB 형식은 모두 32비트이고, **rs1·rs2·opcode·funct3 위치가 (그 필드가 있는) 모든 형식에서 같다** [Ch2 p.21, p.24]. 그래서 Ch4 데이터패스는 명령어 비트 [19:15]·[24:20]을 해독이 끝나기 전에 레지스터 파일 읽기 포트에 **바로** 연결할 수 있다 [Ch4 p.21]. S형식에서 12비트 즉시값이 두 조각으로 쪼개진 것도 이 고정 위치를 지키려는 타협(설계 원칙 3)이다 [Ch2 p.23–24].

> 예: `add x9, x20, x21`과 `sw x9, 8(x20)` 모두 x20이 rs1 칸에 있으므로 같은 전선으로 읽힌다.

② CPU 시간 공식 (Ch1) ↔ 단일 사이클의 한계 (Ch4)

단일 사이클은 모든 명령어가 1사이클이라 **CPI = 1**이다. 하지만 클럭 주기가 **가장 긴 경로(lw: 명령어 메모리 → 레지스터 파일 → ALU → 데이터 메모리 → 레지스터 파일)**에 맞춰지므로 짧은 명령어도 그만큼 기다린다 [Ch4 p.29]. CPU 시간 = IC × CPI × 클럭 주기에서 CPI는 최소인데 클럭 주기가 커지는 구조이고, 명령어마다 클럭 주기를 다르게 할 수 없으므로 "자주 쓰는 경우를 빠르게"를 어긴다 [Ch4 p.29] (8대 아이디어 [Ch1 p.12], 공식 [Ch1 p.28]).

> (PDF 외 보충) 예: lw 경로 800 ps, add 경로 600 ps라면 단일 사이클 클럭은 800 ps. add만 100만 개여도 100만 × 800 ps.

③ 2의 보수·부호 확장 (Ch2) ↔ 뺄셈·오버플로 (Ch3) ↔ ImmGen (Ch4)

음수 = `~x + 1`, 비트를 늘릴 때는 부호 비트를 왼쪽에 복제한다 [Ch2 p.20]. 덕분에 뺄셈은 "두 번째 피연산자를 부정해서 더하기"가 되고 [Ch3 p.3], 오버플로는 **부호가 같은 두 수의 덧셈**(+ + + = −, − + − = +) 또는 **부호가 다른 두 수의 뺄셈**(+ − − = −, − − + = +)에서 결과 부호가 뒤집힐 때만 생긴다 [Ch3 p.4–5]. Ch4의 ImmGen은 12비트 오프셋을 바로 이 부호 확장으로 32비트로 만든다 [Ch4 p.13].

> 예: `lw x5, -4(x6)`의 오프셋 `1111 1111 1100`(−4) → ImmGen → `0xFFFFFFFC`(여전히 −4) → ALU가 x6 + (−4).

기초 · 1 이 기초가 쓰이는 곳: Ch2 · 4, 5 / Ch3 · 6

수를 쓰는 방법: 진법과 자릿값

이 단원을 마치면 10진수·2진수·16진수가 같은 원리(자릿값)로 쓰인다는 것을 설명하고, 0~15의 세 표기를 바로 떠올리며, 2의 거듭제곱 값과 비트·니블·바이트, MSB·LSB를 쓸 수 있다. PDF 외 보충

쉽게 말하면

돈을 셀 때 1원짜리가 10개 모이면 10원짜리 하나로 바꾸고, 10원짜리가 10개 모이면 100원짜리 하나로 바꾼다. 그래서 "123원"은 100원짜리 1개, 10원짜리 2개, 1원짜리 3개다. 이것이 10진법이다.

2진법은 "2개 모이면 다음 동전으로 바꾸는" 나라의 돈이다. 동전 종류가 1원, 2원, 4원, 8원, 16원, ...이고, 각 동전은 0개 아니면 1개만 가질 수 있다. 16진법은 "16개 모이면 바꾸는" 나라로, 동전이 1원, 16원, 256원, ...이다.

비유가 안 맞는 지점 실제 돈은 동전을 몇 개든 늘어놓을 수 있지만, 컴퓨터는 자리 수(비트 수)가 정해진 칸에 수를 담는다. 칸이 모자라면 담을 수 없다. 이 문제는 기초 · 3에서 다룬다.

10진수의 자릿값

우리가 쓰는 수는 **10진법(decimal)**이다. 쓰는 숫자는 0~9의 열 개이고, 자리가 왼쪽으로 하나 갈 때마다 그 자리의 무게가 10배가 된다. 이 무게를 **자릿값(place value)**이라 한다.

$$123 = 1\times100 + 2\times10 + 3\times1 = 1\times10^2 + 2\times10^1 + 3\times10^0$$

지수를 잠깐 상기하자. $10^2 = 10\times10 = 100$, $10^1 = 10$, 그리고 어떤 수든 0제곱은 1이다($10^0 = 1$, $2^0 = 1$). 그래서 맨 오른쪽 자리의 자릿값은 항상 1이다.

자리에는 번호를 붙인다. **맨 오른쪽이 0번**이고 왼쪽으로 갈수록 1, 2, 3, ...이다. 번호가 i인 자리의 자릿값은 (진법)^{i}이다.

2진수와 16진수도 같은 원리

진법 b(밑, base)로 쓴 수에서 i번 자리에 숫자 d가 있으면, 그 자리가 나타내는 값은 $d \times b^i$이다. 모든 자리의 값을 더하면 그 수가 된다. [Ch2 p.17]

$$\text{i번째 자리의 값} = d \times b^i$$

진법	쓰는 숫자	자릿값 (오른쪽부터)	예
10진(decimal)	0~9	1, 10, 100, 1000, ...	123 = 1×100 + 2×10 + 3×1
2진(binary)	0, 1	1, 2, 4, 8, 16, 32, ...	1101_2 = 1×8 + 1×4 + 0×2 + 1×1 = 13
16진(hexadecimal)	0~9, A~F	1, 16, 256, 4096, ...	0x2A = 2×16 + 10×1 = 42

16진법은 한 자리에 0~15까지 들어가야 하는데 숫자는 0~9밖에 없다. 그래서 10~15를 알파벳 **A, B, C, D, E, F**로 쓴다. A = 10, B = 11, C = 12, D = 13, E = 14, F = 15. 소문자(a~f)로 써도 같은 뜻이지만 이 노트는 대문자로 통일한다.

컴퓨터는 모든 정보를 0과 1로 저장하므로 2진수가 기본이다. 2진수 한 자리를 **비트(bit, binary digit)**라 한다. 슬라이드도 123_{10} = 1111011_2를 예로 든다. [Ch2 p.17] 16진수는 2진수를 짧게 적으려고 쓴다. 16진 한 자리가 2진 네 자리와 정확히 맞아떨어지기 때문이다(기초 · 2).

0~15 대응표

아래 표는 진법 변환의 기본 재료다. 시험장에서 매번 계산하지 않도록 외워 두면 좋다. 2진은 네 자리로 맞춰 앞에 0을 채웠다.

10진	2진(4자리)	16진	10진	2진(4자리)	16진
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

외우는 요령: 4자리 2진의 자릿값은 8, 4, 2, 1이다. 예를 들어 1011_2은 8 + 2 + 1 = 11 = B. 거꾸로 C(12)는 8 + 4 → 1100_2. 0~7은 맨 앞 비트가 0, 8~15는 맨 앞 비트가 1이다.

2의 거듭제곱 표

2진수의 자릿값은 모두 2의 거듭제곱이다. 2^n은 2를 n번 곱한 값이고, 한 칸 올라갈 때마다 2배가 된다.

n	2^n	n	2^n	n	2^n
0	1	7	128	14	16,384
1	2	8	256	15	32,768
2	4	9	512	16	65,536
3	8	10	1,024	20	1,048,576
4	16	11	2,048	24	16,777,216
5	32	12	4,096	30	1,073,741,824
6	64	13	8,192	31	2,147,483,648
				32	4,294,967,296

- 2^{10} = 1,024 ≈ 1천(킬로), 2^{20} ≈ 1백만(메가), 2^{30} ≈ 10억(기가). 큰 거듭제곱을 어림할 때 쓴다.

- 2^{31} = 2,147,483,648과 2^{32} = 4,294,967,296은 32비트 수의 범위에 나온다. 32비트 무부호 범위는 0 ~ 2^{32} − 1, 2의 보수 범위는 -2^{31} ~ 2^{31} − 1이다. [Ch2 p.18–19]
- n비트로 만들 수 있는 서로 다른 0·1 패턴의 개수는 2^n개다. 칸마다 경우가 2가지이고, 칸이 n개이므로 2를 n번 곱한다.

표기법

같은 "10"이라는 글자도 10진이면 열, 2진이면 둘, 16진이면 열여섯이다. 그래서 진법을 표시해야 한다.

표기	뜻	예
아래첨자 $_{2\ 10\ 16}$	수학·교재식 표기	1101_2, 13_{10}, D_{16}
`0b` 접두사	프로그래밍식 2진	`0b1101` = 13
`0x` 접두사	프로그래밍식 16진 (슬라이드도 사용)	`0x2A` = 42, `0xFFFFFFFF`
표시 없음	보통 10진	42

긴 2진수는 읽기 쉽게 4자리마다 띄어 쓴다(예: `0010 0101`). 띄어쓰기는 값에 영향이 없다.

비트·니블·바이트, MSB·LSB

비트(bit) 2진수 한 자리. 0 또는 1. 정보의 가장 작은 단위다. [Ch2 p.17]

니블(nibble) 4비트 묶음. 16진 한 자리와 같다.

바이트(byte) 8비트 묶음. 16진 두 자리와 같다. 메모리 주소는 바이트 단위로 붙는다(Ch2 · 3).

word(워드) RISC-V에서 32비트 = 4바이트 = 16진 8자리. [Ch2 p.18]

MSB(most significant bit, 최상위 비트) 가장 왼쪽 비트. 자릿값이 가장 크다. 32비트 word에서는 31번 비트. [Ch2 p.17]

LSB(least significant bit, 최하위 비트) 가장 오른쪽 비트. 0번 비트. 자릿값이 1이다.

비트 번호는 오른쪽 끝(LSB)이 0번이고, n비트 수의 MSB는 n−1번이다. 8비트면 7번, 32비트면 31번이 MSB다.

예제

(1) 10110_2를 자릿값으로 풀어 10진으로 쓰시오. (2) 0x1F를 10진으로 쓰시오. (3) 8비트 `1001 0110`의 MSB와 LSB는?

풀이 보기

(1) 오른쪽부터 자리 번호 0, 1, 2, 3, 4. 숫자는 오른쪽부터 0, 1, 1, 0, 1. $1\times2^4 + 0\times2^3 + 1\times2^2 + 1\times2^1 + 0\times2^0 = 16 + 0 + 4 + 2 + 0 =$ **22**.

(2) $1\times16^1 + F\times16^0 = 1\times16 + 15\times1 =$ **31**.

(3) MSB = 맨 왼쪽(7번) = **1**, LSB = 맨 오른쪽(0번) = **0**.

함정

자리 번호를 1부터 세면 모든 값이 2배로 틀린다. 맨 오른쪽은 **0번**, 자릿값은 $2^0 = 1$이다.

"0x10"을 10으로 읽는 실수. 0x10 = 1×16 + 0 = **16**이다. 마찬가지로 0b10 = 2.

n비트의 최댓값은 2^n이 아니라 $2^n - 1$이다. 패턴은 2^n개지만 0부터 세기 때문이다. 4비트 최댓값 = $1111_2 = 15$.

스스로 확인

1. 4,507을 자릿값의 합(×1000, ×100 …)으로 풀어 쓰시오.

   답 보기

   $4\times1000 + 5\times100 + 0\times10 + 7\times1 = 4\times10^3 + 5\times10^2 + 0\times10^1 + 7\times10^0$.

2. 16진수에서 B, D, F는 10진으로 각각 얼마인가?

   답 보기

   B = 11, D = 13, F = 15. A부터 10, 11, 12, … 순서.

3. 빈칸: 2진 1001 = 10진 ( ) = 16진 ( )

   답 보기

   9, 9. 8 + 1 = 9이고, 9는 16진에서도 9.

4. 빈칸: 10진 12 = 2진 4자리 ( ) = 16진 ( )

   답 보기

   1100_2, C. 12 = 8 + 4.

5. 2^8, 2^{10}, 2^{16}의 값은?

   답 보기

   256, 1,024, 65,536.

6. 2진 111011_2를 10진으로 쓰시오.

   답 보기

   32 + 16 + 8 + 0 + 2 + 1 = **59**.

7. 16진 0x3F를 10진으로 쓰시오.

   답 보기

   3×16 + 15 = **63**.

8. 1바이트는 몇 비트이고 16진 몇 자리인가? 32비트 word는 16진 몇 자리인가?

   답 보기

   8비트, 16진 2자리. word는 16진 8자리(4비트 = 16진 1자리).

9. 8비트로 만들 수 있는 서로 다른 패턴 수와 무부호 최댓값은?

   답 보기

   2^8 = 256개, 최댓값 255(1111 1111). 0도 한 패턴이라 최댓값은 256 − 1.

10. 32비트 word에서 MSB는 몇 번 비트이고 그 자릿값(무부호)은?

    답 보기

    31번 비트, 2^{31} = 2,147,483,648.

11. O/X: `0x100`과 `0b100`과 100_{10}은 모두 같은 값이다.

    답 보기

    X. 0x100 = 256, 0b100 = 4, 100_{10} = 100.

12. 2^{32}의 값을 쓰고, 32비트 무부호 수의 최댓값을 쓰시오.

    답 보기

    2^{32} = 4,294,967,296. 최댓값 $2^{32} - 1$ = 4,294,967,295. [Ch2 p.18]

이 기초가 시험에서 쓰이는 곳

- Ch2 · 4 2진수와 2의 보수: 자릿값 공식, MSB·LSB, 32비트 범위(2^{31}, 2^{32}). [Ch2 p.17–19]
- Ch2 · 5 명령어 인코딩, 실전 · 4: 비트 번호([31:25] 등)와 16진 8자리 기계어.
- Ch3 · 6~7 부동소수점: 2의 거듭제곱(bias 127 = $2^7 - 1$ 등).

기초 · 2 이 기초가 쓰이는 곳: Ch2 · 4, 5 / Ch3 · 6, 7 / 실전 · 4, 6

진법 변환 완전 정복

이 단원을 마치면 2진·10진·16진 사이를 어느 방향으로든 바꾸고, 소수(0.625 같은 수)를 2진으로 바꾸어 $1.xxx \times 2^n$ 꼴로 정규화할 수 있다.

PDF 외 보충

쉽게 말하면

진법 변환은 같은 금액을 다른 나라 동전으로 다시 세는 일이다. 2진 → 10진은 "동전 값을 다 더하기", 10진 → 2진은 "큰 동전부터 거슬러 주기"다. 2진과 16진 사이는 2진 동전 4개를 16진 동전 1개로 묶기만 하면 되어 가장 쉽다.

비유가 안 맞는 지점 소수점 아래로 가면 어떤 금액은 2진 동전으로 끝까지 맞출 수 없다(예: 0.1). 이런 수는 컴퓨터에서 근삿값으로만 저장된다.

1. 2진 → 10진: 자릿값 더하기

오른쪽 끝부터 자릿값 1, 2, 4, 8, …을 적고, 숫자가 1인 자리의 자릿값만 더한다. [Ch2 p.17]

자릿값	64	32	16	8	4	2	1
1111011_2	1	1	1	1	0	1	1

64 + 32 + 16 + 8 + 2 + 1 = 123. 슬라이드의 $123_{10} = 1111011_2$와 맞는다. [Ch2 p.17]

2. 10진 → 2진: 방법 두 가지

방법 A: 2로 나누고 나머지를 거꾸로 읽기. 몫이 0이 될 때까지 2로 계속 나눈다. 나머지(0 또는 1)를 아래에서 위로 읽으면 2진수다. 처음 나온 나머지가 LSB(1의 자리)이기 때문이다.

나눗셈	몫	나머지
45 ÷ 2	22	1 (LSB)
22 ÷ 2	11	0
11 ÷ 2	5	1
5 ÷ 2	2	1
2 ÷ 2	1	0
1 ÷ 2	0	1 (MSB)

아래에서 위로 읽으면 45 = **101101_2**. 검산: 32 + 8 + 4 + 1 = 45.

방법 B: 큰 2의 거듭제곱부터 빼기. 그 수를 넘지 않는 가장 큰 2의 거듭제곱을 빼고, 남은 수로 반복한다. 뺀 자리에 1, 건너뛴 자리에 0을 쓴다. 156으로 해 보자.

1. 156 이하에서 가장 큰 2의 거듭제곱은 128(2^7). 156 − 128 = 28. 7번 자리 = 1.
2. 64(2^6)와 32(2^5)는 28보다 크다. 6번, 5번 자리 = 0.

3. $16(2^4) \le 28$. $28 - 16 = 12$. 4번 자리 = 1.

4. $8(2^3) \le 12$. $12 - 8 = 4$. 3번 자리 = 1.

5. $4(2^2) \le 4$. $4 - 4 = 0$. 2번 자리 = 1. 남은 1번, 0번 자리 = 0.

156 = **$1001\ 1100_2$**. 방법 B는 2의 거듭제곱 표를 외웠을 때 빠르고, 방법 A는 기계적으로 실수가 적다. 둘 중 편한 것을 쓰고 다른 방법으로 검산하면 좋다.

3. 2진 ↔ 16진: 4비트씩 묶기

$16 = 2^4$이므로 16진 한 자리는 2진 정확히 4자리(니블)다. 그래서 변환에 곱셈·나눗셈이 필요 없다.

- **2진 → 16진**: **오른쪽 끝부터** 4자리씩 끊는다. 맨 왼쪽 묶음이 4자리보다 짧으면 **앞에 0을 채운다**. 각 묶음을 0~15 대응표로 바꾼다.
- **16진 → 2진**: 16진 한 자리마다 4자리 2진으로 바꾸어 이어 붙인다. 0도 반드시 0000 네 자리로 쓴다.

예: 1111011_2 → 오른쪽부터 끊으면 111 | 1011 → 앞에 0 채움 0111 | 1011 → 7 | B → **0x7B**. 검산: 7×16 + 11 = 123.

예: 0xDEAD → D = 1101, E = 1110, A = 1010, D = 1101 → **1101 1110 1010 1101**.

4. 10진 ↔ 16진

2진을 거쳐 가기. 10진 → 2진(방법 A 또는 B) → 4비트 묶기 → 16진. 반대 방향은 16진 → 2진 → 자릿값 더하기, 또는 16진 자릿값으로 바로 더한다.

16으로 나누기. 2로 나누던 방법 A를 16으로 바꾼 것이다. 나머지는 0~15이고 10 이상은 A~F로 쓴다. 2026으로 해 보자.

나눗셈	몫	나머지
2026 ÷ 16	126	10 → A
126 ÷ 16	7	14 → E
7 ÷ 16	0	7

아래에서 위로 읽으면 2026 = **0x7EA**. 검산: 7×256 + 14×16 + 10 = 1792 + 224 + 10 = 2026.

16진 → 10진은 자릿값 1, 16, 256, 4096, …을 곱해 더한다. 예: 0x1F4 = 1×256 + 15×16 + 4×1 = 256 + 240 + 4 = 500.

5. 32비트를 16진 8자리로

32비트 = 4비트 × 8묶음이므로 32비트 값은 16진 정확히 8자리다. 기계어(Ch2 · 5)와 IEEE 754(Ch3 · 7)의 답은 보통 8자리 16진으로 쓴다. [Ch2 p.21–24]

- 16진 → 32비트 2진: 앞자리 0까지 8자리를 모두 2진 4자리로 바꾼다. 결과는 반드시 32자리여야 한다.
- 32비트 2진 → 16진: 왼쪽 31번 비트부터 4개씩 8묶음. 앞쪽 0000 묶음도 0으로 적어 8자리를 맞춘다(`0x0000002A`).

예제 (시험 수준)

`0x00578833` 을 32비트 2진수로 쓰고, 맨 오른쪽 7비트를 쓰시오.

풀이 보기

자리마다 바꾼다: 0 → 0000, 0 → 0000, 5 → 0101, 7 → 0111, 8 → 1000, 8 → 1000, 3 → 0011, 3 → 0011.

결과: **0000 0000 0101 0111 1000 1000 0011 0011** (32자리 확인).

맨 오른쪽 7비트(6~0번): **0110011**. 기계어에서 이 7비트가 opcode 칸이고, 0110011은 R형(add 등)의 opcode다. 기계어를 해독할 때 이렇게 16진 → 2진 → 칸 자르기를 한다(실전 · 4). [Ch2 p.50]

참고로 이 값을 10진으로 읽으면 5,736,499지만, 기계어 문제에서는 10진 값이 필요 없다.

6. 소수: 2진 소수 → 10진

소수점 오른쪽 자리도 같은 규칙이다. 소수점 바로 오른쪽이 −1번 자리, 그다음이 −2번 자리다. 지수가 음수이면 분수가 된다: 2^{-1} = 1/2, 2^{-2} = 1/4.

자리	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}
분수	1/2	1/4	1/8	1/16	1/32
소수	0.5	0.25	0.125	0.0625	0.03125

예: 0.1011_2 = 0.5 + 0 + 0.125 + 0.0625 = 0.6875. 예: 110.01_2 = 4 + 2 + 0 + 0 + 0.25 = 6.25.

2진에서 소수점은 **이진 소수점(binary point)**이라 부른다. [Ch3 p.23]

7. 소수: 10진 소수 → 2진 (×2 반복)

소수 부분에 2를 곱하고, 결과의 **정수 부분(0 또는 1)을 떼어 적는다**. 남은 소수 부분에 다시 2를 곱한다. 소수 부분이 0이 되면 멈춘다. 떼어 낸 비트를 **위에서 아래로** 읽는다(정수 쪽 나눗셈과 방향이 반대다).

계산	결과	떼어 낸 비트	남은 소수
0.625 × 2	1.25	1 (2^{-1} 자리)	0.25
0.25 × 2	0.5	0	0.5
0.5 × 2	1.0	1	0 (끝)

0.625 = **0.101_2**. 검산: 0.5 + 0.125 = 0.625.

왜 이렇게 되는가: 소수에 2를 곱하면 이진 소수점이 오른쪽으로 한 칸 이동한다. 그때 소수점 왼쪽으로 넘어온 비트가 바로 2^{-1} 자리의 숫자다.

끝나지 않는 경우: 0.1.

계산	결과	비트
0.1 × 2	0.2	0
0.2 × 2	0.4	0
0.4 × 2	0.8	0
0.8 × 2	1.6	1
0.6 × 2	1.2	1
0.2 × 2	0.4	0 (두 번째 줄로 돌아옴, 반복 시작)

0.2가 다시 나왔으므로 0011이 영원히 반복된다: 0.1 = 0.0 0011 0011 0011 …$_2$. 정해진 비트 수에 담으면 중간에서 잘리므로 컴퓨터의 0.1은 정확히 0.1이 아니다. 10진에서 1/3 = 0.333…이 끝나지 않는 것과 같은 현상이다. 분모가 2의 거듭제곱인 분수(1/2, 3/8, 5/16 …)만 2진 소수가 끝난다.

8. 정수 + 소수 합치기

정수 부분과 소수 부분을 **따로** 바꾼 뒤 이진 소수점으로 잇는다.

- 2.625: 정수 2 = 10_2, 소수 0.625 = 0.101_2 → **10.101_2**. 검산: 2 + 0.5 + 0.125.
- 5.75: 정수 5 = 101_2, 소수 0.75 → 0.75×2 = 1.5(1), 0.5×2 = 1.0(1) → 0.11_2 → **101.11_2**.
- −0.5: 부호는 따로 둔다. 0.5×2 = 1.0(1) → 0.1_2 → **-0.1_2**.

9. 정규화 맛보기: 1.xxx × 2^n

10진 과학적 표기 3.15576×10^9처럼, 2진수도 소수점 왼쪽에 **0이 아닌 숫자 하나**만 두는 꼴로 쓴다. 2진에서 0이 아닌 숫자는 1뿐이므로 항상 **$1.xxx_2 \times 2^n$** 꼴이 된다. 이것을 **정규화(normalized)** 표기라 한다. [Ch3 p.23–24]

- 이진 소수점을 **왼쪽으로 k칸** 옮기면 n = **+k** (수가 작아진 만큼 2^k를 곱해 보충).
- 이진 소수점을 **오른쪽으로 k칸** 옮기면 n = **−k**.

표기법

앞의 가수에는 $_2$를 붙이고(2진수라는 뜻), 뒤의 **2^n**는 그대로 둔다. 밑 2와 지수 n은 10진수로 쓰는 것이 관례다. 예: 5.75 = 101.11_2 = $1.0111_2 \times 2^2$. $_2$를 빼고 1.0111×2^2라고만 쓰면 10진 $1.0111 \times 4 = 4.0444$로 읽힐 수 있다. 슬라이드도 처음 소개할 때는 $1.0_{(2)} \times 2^{-1}$처럼 표시하고 [Ch3 p.23–24], 계산 과정에서는 2진임이 분명하면 생략하기도 한다 [Ch3 p.33]. 답안에는 $_2$를 붙이는 쪽이 안전하다.

10진	2진	정규화
2.625	10.101	$1.0101_2 \times 2^1$ (왼쪽 1칸)
5.75	101.11	$1.0111_2 \times 2^2$ (왼쪽 2칸)
−0.5	−0.1	$-1.0_2 \times 2^{-1}$ (오른쪽 1칸)
0.0625	0.0001	$1.0_2 \times 2^{-4}$ (오른쪽 4칸)

이 정규화 결과에서 부호, 지수 n, 소수점 뒤의 xxx(가수, fraction)를 떼어 IEEE 754 비트 칸에 넣는 것이 Ch3 · 7이다. 이 단원은 그 직전까지다.

예제 (시험 수준)

26-1 시험에는 "A = −0.5, B = 2.625를 2진수로 바꿔 부동소수점 계산"이 나왔다. 두 수를 2진으로 바꾸고 정규화하시오.

풀이 보기

A: 0.5 × 2 = 1.0 → 비트 1, 남은 소수 0 → 0.1_2. 부호를 붙이면 -0.1_2. 이진 소수점을 오른쪽으로 1칸 옮겨 1.0_2 → $\mathbf{-1.0_2 \times 2^{-1}}$.

B: 정수 2 = 10_2. 소수 0.625 → 1.25(1), 0.5(0), 1.0(1) → 0.101_2. 합쳐서 10.101_2. 왼쪽으로 1칸 → $\mathbf{1.0101_2 \times 2^1}$.

검산: 1.0101_2 = 1 + 0.25 + 0.0625 = 1.3125, × 2 = 2.625. 이어지는 계산은 실전 · 6.

연습 예제 (단계별)

1. 200을 2진으로(방법 A).
2. 100을 2진으로(방법 B).
3. 0xB4를 10진으로.
4. 1000을 16진으로(16으로 나누기).
5. 13.375를 2진으로 쓰고 정규화하시오.

풀이 보기

1. 200→100 r0, 100→50 r0, 50→25 r0, 25→12 r1, 12→6 r0, 6→3 r0, 3→1 r1, 1→0 r1. 거꾸로 $\mathbf{1100\ 1000_2}$. 검산 128 + 64 + 8 = 200.

2. 100 − 64 = 36 (6번 1), 32 ≤ 36 → 36 − 32 = 4 (5번 1), 16·8은 큼 (4·3번 0), 4 − 4 = 0 (2번 1), 1·0번 0 → $\mathbf{110\ 0100_2}$ = 0110 0100.

3. B = 11 → 11×16 + 4 = **180**. (2진 1011 0100 = 128 + 32 + 16 + 4 = 180.)

4. 1000 ÷ 16 = 62 r8, 62 ÷ 16 = 3 r14(E), 3 ÷ 16 = 0 r3 → **0x3E8**. 검산 3×256 + 14×16 + 8 = 768 + 224 + 8 = 1000.

5. 13 = 1101_2, 0.375 → 0.75(0), 1.5(1), 1.0(1) → 0.011_2. 합쳐 $\mathbf{1101.011_2}$, 왼쪽 3칸 → $\mathbf{1.101011_2 \times 2^3}$.

함정

정수 부분(÷2)은 나머지를 **아래에서 위로**, 소수 부분(×2)은 떼어 낸 비트를 **위에서 아래로** 읽는다. 방향을 바꾸면 틀린다.

2진 → 16진에서 왼쪽부터 4자리씩 끊으면 틀린다. 1111011_2을 왼쪽부터 끊으면 1111 | 011 → F3이 되어 틀린다. **오른쪽부터** 끊고 왼쪽에 0을 채운다(0x7B).

16진 → 2진에서 0을 "0" 한 자리로 쓰면 비트 위치가 다 밀린다. 0은 **0000**, 1은 **0001**로 4자리를 지킨다.

정규화 지수의 부호: 소수점을 왼쪽으로 옮기면 +, 오른쪽으로 옮기면 −. $0.0625 = 1.0_2 \times 2^{-4}$이다($2^4$ 아님).

0.1(10진)은 2진으로 끝나지 않는다. "$0.1_{10} = 0.1_2$"는 틀렸다($0.1_2 = 0.5$).

스스로 확인

1. 1101_2, 100101_2를 10진으로 쓰시오.

   답 보기

   8 + 4 + 1 = 13. 32 + 4 + 1 = 37.

2. 19를 2진으로 쓰시오.

   답 보기

   10011_2 (16 + 2 + 1).

3. 77을 8비트 2진으로 쓰시오.

   답 보기

   0100 1101 (64 + 8 + 4 + 1).

4. 1011 0100_2를 16진으로 쓰시오.

   답 보기

   1011 = B, 0100 = 4 → 0xB4.

5. 11 1110 1000_2를 16진으로 쓰시오.

   답 보기

   오른쪽부터 1000 | 1110 | 11 → 0011 | 1110 | 1000 → 0x3E8 (= 1000).

6. 0x7C를 2진과 10진으로 쓰시오.

   답 보기

   0111 1100, 7×16 + 12 = 124.

7. 255를 16진으로, 256을 16진으로 쓰시오.

   답 보기

   0xFF, 0x100. 255 = 15×16 + 15, 256 = 1×256.

8. 42를 32비트 값으로 보고 16진 8자리로 쓰시오.

   답 보기

   42 = 2×16 + 10 → 0x2A → **0x0000002A**.

9. `0x00578833`을 32비트 2진으로 쓰시오.

   답 보기

   0000 0000 0101 0111 1000 1000 0011 0011.

10. 0.101_2, 0.0001_2를 10진으로 쓰시오.

    답 보기

    0.5 + 0.125 = 0.625. 2^{-4} = 0.0625.

11. 0.0625를 2진으로 바꾸고 정규화하시오.

    답 보기

    ×2 → 0.125(0), 0.25(0), 0.5(0), 1.0(1) → 0.0001_2 = $\mathbf{1.0_2 \times 2^{-4}}$.

12. 5.75를 2진으로 바꾸고 정규화하시오.

    답 보기

    101.11_2 = $\mathbf{1.0111_2 \times 2^2}$. 검산 4 + 1 + 0.5 + 0.25.

13. −0.5와 2.625를 정규화된 2진 표기로 쓰시오.

    답 보기

    $-1.0_2 \times 2^{-1}$, $1.0101_2 \times 2^{1}$.

14. 0.1을 2진으로 바꿀 때 소수점 아래 8자리까지 쓰고, 끝나는지 답하시오.

    답 보기

    $0.00011001\ldots_2$ 0.2 → 0.4 → 0.8 → 0.6 → 0.2로 되돌아와 0011이 무한 반복. 끝나지 않는다.

15. O/X: $0.75 = 0.11_2 = 1.1_2 \times 2^{1}$이다.

    답 보기

    X. 0.11_2는 소수점을 오른쪽으로 1칸 옮겨 $1.1_2 \times 2^{-1}$이다. [Ch3 p.23–24]

이 기초가 시험에서 쓰이는 곳

- Ch2 · 4: 10진 ↔ 2진, 음수를 2의 보수로 만들기 전 단계. [Ch2 p.17–20]
- Ch2 · 5와 실전 · 4: 레지스터 번호·즉시값 → 2진 칸 → 16진 8자리 기계어, 그 반대.
- Ch3 · 6~8과 실전 · 6: 10진 소수 → 2진 → 정규화 → IEEE 754 32비트·16비트 16진. 26-1 기출(−0.5, 2.625). [Ch3 p.23–28]

기초 · 3 이 기초가 쓰이는 곳: Ch2 · 4 / Ch3 · 1, 2

2진수 계산과 비트 폭

이 단원을 마치면 2진수 덧셈·뺄셈·곱셈과 16진 덧셈을 손으로 하고, 정해진 비트 폭에서 자리올림이 넘치는 것(carry out)과 같은 비트 패턴의 무부호·부호 해석 차이를 설명할 수 있다. PDF 외 보충

쉽게 말하면

2진 덧셈은 초등학교 세로셈과 같다. 다만 10이 아니라 **2가 되면** 올린다. 1 + 1은 "2"인데 2진에서 2는 10_2이므로, 0을 쓰고 1을 왼쪽으로 올린다.

비트 폭은 계산기 화면의 칸 수다. 4칸 화면에 15 + 1을 하면 16을 쓸 칸이 없어서 맨 앞 자리가 화면 밖으로 사라진다.

비유가 안 맞는 지점 계산기는 칸이 모자라면 오류를 띄우지만, 컴퓨터의 정수 덧셈 회로는 넘친 비트를 버리고 남은 비트를 결과로 내놓는다. 그것을 오류로 볼지는 해석(무부호/부호)과 명령어에 달렸다(Ch3 · 1).

2진 덧셈

한 자리끼리 더할 때 나올 수 있는 경우는 아래가 전부다. 아래에서 오는 **자리올림(carry)**까지 셋을 더하는 경우가 있다.

더하는 것	합(10진)	2진 결과	이 자리에 쓸 비트	왼쪽으로 올림
0 + 0	0	0	0	0
0 + 1, 1 + 0	1	1	1	0
1 + 1	2	10	0	1
1 + 1 + 1 (올림 포함)	3	11	1	1

예제

8비트 0110 1101 + 0011 0111을 계산하시오. (109 + 55)

풀이 보기

오른쪽(0번)부터 한 자리씩. "올림"은 그 자리로 들어오는 자리올림이다.

자리	7	6	5	4	3	2	1	0
올림	1	1	1	1	1	1	1	0
A	0	1	1	0	1	1	0	1
B	0	0	1	1	0	1	1	1
합	1	0	1	0	0	1	0	0

0번: $1 + 1 = 10_2$ → 0 쓰고 1 올림. 1번: $0 + 1 + 1 = 10_2$ → 0, 올림. 2번: $1 + 1 + 1 = 11_2$ → 1, 올림. 3번: $1 + 0 + 1 = 10_2$ → 0, 올림. 4번: $0 + 1 + 1 = 10_2$ → 0, 올림. 5번: $1 + 1 + 1 = 11_2$ → 1, 올림. 6번: $1 + 0 + 1 = 10_2$ → 0, 올림. 7번: 0 + 0 + 1 = 1 → 1, 올림 없음.

결과 **1010 0100** = 128 + 32 + 4 = 164 = 109 + 55. 맞다.

2진 뺄셈과 빌림

위 자리 숫자가 아래보다 작으면(0 − 1) 왼쪽 자리에서 1을 **빌려(borrow)** 온다. 왼쪽의 1은 이 자리에서 2(= 10_2)의 가치이므로 0 − 1은 2 − 1 = 1이 되고, 빌려 준 자리는 1이 줄어든다.

예제

$1101_2 - 0110_2$를 계산하시오. (13 − 6)

풀이 보기

0번: 1 − 0 = 1.

1번: 0 − 1 → 2번 자리에서 빌림. 2번 자리 1 → 0, 1번 자리는 2. 2 − 1 = 1.

2번: (빌려 주고 남은) 0 − 1 → 3번 자리에서 빌림. 3번 자리 1 → 0, 2번 자리는 2. 2 − 1 = 1.

3번: 0 − 0 = 0.

결과 **0111_2** = 7. 맞다.

컴퓨터는 실제로는 빼기 회로를 따로 두지 않고 a − b = a + (−b)로 계산한다. −b는 2의 보수(비트 반전 + 1)로 만든다. 이것은 Ch2 · 4와 Ch3 · 1에서 다룬다. [Ch3 p.3]

2진 곱셈 = 시프트 + 덧셈

2진 곱셈의 각 자리 곱은 0 아니면 1이다. 그래서 곱하는 수(승수)의 비트가 1이면 피승수를 그 자리만큼 왼쪽으로 민(시프트) 것을 적고, 0이면 0을 적는다. 그다음 다 더한다. 왼쪽으로 한 칸 미는 것은 ×2다.

```
  1011      (11, 피승수)
×  101      (5, 승수)
------
  1011      승수 0번 비트 = 1 → 1011
 0000       승수 1번 비트 = 0 → 0
1011        승수 2번 비트 = 1 → 1011을 2칸 왼쪽
------
110111      = 55
```

검산: 32 + 16 + 4 + 2 + 1 = 55 = 11 × 5. Ch3 · 2의 곱셈 하드웨어는 이 "1이면 더하고, 한 칸씩 민다"를 그대로 회로로 만든 것이다. [Ch3 p.7–8]

비트 폭과 넘침(carry out)

컴퓨터 안의 수는 칸 수가 정해져 있다. 4비트, 8비트(바이트), 32비트(RISC-V 레지스터) 같은 폭을 **비트 폭(bit width)**이라 한다. 덧셈 결과가 그 폭보다 길어지면 맨 왼쪽 밖으로 나가는 자리올림을 **carry out(최상위 자리올림)**이라 하고, 결과 칸에는 담기지 않는다.

예제

4비트 칸에서 $1011_2 + 0110_2$을 계산하시오. (11 + 6)

풀이 보기

세로셈 결과는 10001_2(= 17)로 5자리다. 4비트 칸에는 아래 4자리 **0001_2**만 남고 맨 앞 1은 carry out으로 나간다.

무부호로 보면 4비트 최댓값은 15라 17을 담을 수 없다. 남은 값 1은 17 − 16과 같다. 이처럼 n비트 무부호 덧셈에서 carry out이 1이면 결과가 2^n만큼 모자라다.

8비트에서 200 + 100 = 300은 1 0010 1100_2이 되어 carry out이 나고 칸에는 0010 1100 = 44(= 300 − 256)가 남는다. 결과를 비트 수로 표현할 수 없는 상황을 Ch3에서는 **오버플로(overflow)**라 부른다. 부호 있는 수의 오버플로 판정은 carry out과 같지 않으며 Ch3 · 1에서 따로 배운다. [Ch3 p.4–5]

같은 비트, 다른 해석 (맛보기)

4비트 `1111`은 무부호로 읽으면 8 + 4 + 2 + 1 = 15다. 그런데 음수도 표현하려고 "MSB가 1이면 음수"로 약속한 **2의 보수**로 읽으면 −1이다. 비트는 같고 읽는 규칙만 다르다. [Ch2 p.19]

4비트 패턴	무부호로 읽기	2의 보수로 읽기
0111	7	7
1000	8	−8
1010	10	−6
1111	15	−1

2의 보수로 읽는 법: MSB 자릿값만 음수로 본다. 1010_2 = −8 + 2 = −6. 1111_2 = −8 + 4 + 2 + 1 = −1. 자세한 규칙과 범위는 Ch2 · 4에서 다룬다. 재미있는 점은 1111_2 + 0001_2 = 1 0000_2에서 carry out을 버리면 0000_2이 되어, −1 + 1 = 0이 저절로 맞는다는 것이다. 그래서 컴퓨터는 무부호·부호 덧셈에 같은 회로를 쓴다.

16진 덧셈

한 자리 합이 **16 이상**이면 16을 빼고 쓴 뒤 1을 올린다. 숫자를 10진으로 바꿔 더하면 실수가 줄어든다.

- 0x9 + 0x8: 9 + 8 = 17 = 16 + 1 → 1 쓰고 1 올림 → **0x11** (= 17).
- 0x3A + 0x2C: 1의 자리 A + C = 10 + 12 = 22 = 16 + 6 → 6 쓰고 올림. 16의 자리 3 + 2 + 1 = 6 → **0x66** (58 + 44 = 102).
- 0x7E + 0x45: E + 5 = 19 = 16 + 3 → 3, 올림. 7 + 4 + 1 = 12 = C → **0xC3** (126 + 69 = 195).
- 0xFF + 0x01: F + 1 = 16 → 0, 올림. F + 0 + 1 = 16 → 0, 올림 → **0x100**. 8비트 칸이라면 0x00이 남고 carry out.

16진 뺄셈에서 빌리면 그 자리에 16을 더한다. 0x1C − 0x0F: C − F는 모자라므로 빌려서 16 + 12 − 15 = 13 = D, 16의 자리 1 − 1 − 0 = 0 → 0x0D.

함정

2진 1 + 1을 "2"로 쓰는 실수. 2진에는 숫자 2가 없다. **0 쓰고 1 올림**.

16진에서 9 + 8을 "17"로 쓰면 0x17(= 23)이 되어 틀린다. 17은 16진으로 **0x11**이다.

carry out이 났다고 항상 "틀린 결과"인 것은 아니다. 2의 보수로 −1 + 1을 하면 carry out이 나지만 결과 0은 맞다. 부호 있는 오버플로 판정은 Ch3 · 1의 규칙(같은 부호끼리 더했는데 결과 부호가 다름)을 쓴다. [Ch3 p.4–5]

"1111_2은 15다"라고 단정하면 안 된다. 무부호면 15, 2의 보수면 −1. 문제가 어떤 해석인지 먼저 확인한다.

스스로 확인

1. 2진 0101 + 0011을 계산하시오.

   답 보기

   1000_2 (5 + 3 = 8).

2. 2진 1 + 1 + 1의 결과 비트와 올림은?

답 보기

합 3 = 11_2. 이 자리 1, 올림 1.

---

3. 2진 0111 + 0001을 계산하시오.

답 보기

1000_2. 올림이 0번부터 3번까지 연달아 이어진다(7 + 1 = 8).

---

4. 2진 1000 − 0011을 계산하시오.

답 보기

0101_2 (8 − 3 = 5). 0번 자리에서 3번 자리까지 빌림이 이어진다.

---

5. 8비트 1100 1010 − 0101 0111을 계산하시오.

답 보기

0111 0011 (202 − 87 = 115).

---

6. 2진 1101 × 110을 시프트 + 덧셈으로 계산하시오.

답 보기

승수 비트 0번 0, 1번 1 → 11010_2, 2번 1 → 110100_2. 11010_2 + 110100_2 = **1001110_2** = 78 (13 × 6).

---

7. 4비트 칸에서 1001_2 + 1010_2의 결과 칸 값과 carry out은?

답 보기

10011_2 → 칸에는 **0011_2**, carry out 1. (9 + 10 = 19, 19 − 16 = 3.)

---

8. 8비트 무부호 200 + 100의 결과 칸 값은?

   답 보기

   칸 값 **44**, carry out 1.

   ① 8비트 칸에는 2^8 = 256가지, 곧 0 ~ 255만 담긴다.

   ② 2진으로 세로 덧셈: 1100 1000(200) + 0110 0100(100) = 1 0010 1100(300). 결과가 9비트가 된다.

   ③ 맨 왼쪽 9번째 비트(자릿값 2^8 = 256)는 들어갈 칸이 없어 밖으로 나간다 = carry out 1. 칸에는 아래 8비트 $0010\ 1100_2$만 남는다.

   ④ $0010\ 1100_2$ = 32 + 8 + 4 = 44. 10진으로 줄이면 "300 − 256 = 44"(나간 비트의 값 256을 뺀 것)와 같다.

   무부호 덧셈에서 결과(44)가 더한 수(200)보다 작아졌으므로 오버플로다 [Ch3 p.6].

9. 16진 0x9 + 0x8, 0x2B + 0x19를 계산하시오.

   답 보기

   0x11. B + 9 = 20 = 16 + 4 → 4 올림, 2 + 1 + 1 = 4 → 0x44.

10. 16진 0x0000BEEF + 0x00001111을 계산하시오.

    답 보기

    F+1 = 16 → 0 올림, E+1+1 = 16 → 0 올림, E+1+1 = 16 → 0 올림, B+1+1 = 13 = D → **0x0000D000**.

11. 4비트 1000_2을 무부호와 2의 보수로 각각 읽으시오.

    답 보기

    무부호 8, 2의 보수 −8. [Ch2 p.19]

12. 32비트 0xFFFFFFFF + 0x00000001의 결과 칸 값과 carry out은? 이 패턴을 2의 보수로 읽으면 무슨 계산인가?

    답 보기

    0x00000000, carry out 1. 0xFFFFFFFF는 2의 보수로 −1이므로 −1 + 1 = 0이라는 맞는 계산이다. 무부호로는 2^{32} − 1 + 1이 넘친 것.

13. O/X: 2진수를 왼쪽으로 한 칸 밀면(오른쪽에 0을 붙이면) 값이 2배가 된다.

    답 보기

    O. 모든 자리의 자릿값이 2배가 된다(칸이 넘치지 않을 때). 곱셈의 시프트가 이것이다.

이 기초가 시험에서 쓰이는 곳

- Ch2 · 4: 2의 보수 부정(반전 + 1)의 "+1" 계산, 무부호/부호 해석. [Ch2 p.18–20]
- Ch3 · 1: 2진 덧셈·뺄셈 예제와 오버플로 판정. [Ch3 p.3–5]
- Ch3 · 2, 실전 · 5: 곱셈 하드웨어 단계(시프트 + 덧셈), 곱의 upper/lower 32비트.
- Ch3 · 8, 실전 · 6: 부동소수점 덧셈에서 유효숫자(2진 소수) 더하기와 빼기.

기초 · 4 이 기초가 쓰이는 곳: Ch2 · 6 / Ch3 · 2

비트 논리와 시프트

이 단원을 마치면 AND·OR·XOR·NOT의 진리표를 쓰고, 두 수를 비트별로 계산하며(16진 상태에서도), 마스크로 비트를 남기기·켜기·뒤집기를 하고, 왼쪽·오른쪽 시프트의 결과와 의미를 쓸 수 있다. PDF 외 보충

쉽게 말하면

32비트 수를 전구 32개가 한 줄로 늘어선 것으로 보자. 1은 켜짐, 0은 꺼짐이다. 논리 연산은 두 줄의 **같은 자리 전구끼리만** 짝지어 규칙대로 새 전구를 켠다. 옆 자리로 올림이 넘어가지 않는다는 점이 덧셈과 다르다.

시프트는 전구 줄 전체를 왼쪽이나 오른쪽으로 몇 칸 미는 것이다. 끝에서 밀려난 전구는 버리고, 새로 생긴 빈 칸은 정해진 값으로 채운다.

비유가 안 맞는 지점 오른쪽으로 밀 때 왼쪽 빈 칸을 0으로 채울지, 원래 맨 왼쪽 비트(부호 비트)로 채울지는 명령어마다 다르다. 이 차이로 음수의 나눗셈 결과가 달라진다.

진리표

각 연산이 비트 하나와 하나를 받아 무엇을 내는지 모든 경우를 적은 표를 **진리표(truth table)**라 한다. [Ch2 p.25–27]

a	b	a AND b	a OR b	a XOR b
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

AND(논리곱, C의 &) 둘 다 1일 때만 1. 하나라도 0이면 0.

OR(논리합, C의 |) 하나라도 1이면 1. 둘 다 0일 때만 0.

XOR(배타적 논리합, C의 ^) 두 비트가 다르면 1, 같으면 0.

NOT(부정, C의 ~) 입력 하나. 0 → 1, 1 → 0. 모든 비트를 뒤집는 것을 비트 반전이라 한다.

비트별(bitwise) 계산: 세로로 맞추기

두 수를 2진으로 쓰고 **오른쪽 끝을 맞춰** 위아래로 놓는다. 같은 열(같은 자리)끼리만 진리표를 적용한다. 올림·빌림은 없다. [Ch2 p.25]

```
a       = 1100 1010   (0xCA)
b       = 1010 0110   (0xA6)
a AND b = 1000 0010   (0x82)   둘 다 1인 열만 1
a OR  b = 1110 1110   (0xEE)   하나라도 1인 열은 1
a XOR b = 0110 1100   (0x6C)   다른 열만 1
NOT a   = 0011 0101   (0x35)   a를 모두 뒤집음
```

16진 상태에서 니블별로 계산하는 요령

논리 연산은 자리끼리 독립이므로 16진 한 자리(4비트)씩 따로 계산해도 된다. 특히 마스크에 자주 나오는 **0과 F**는 외워 두면 2진으로 풀지 않아도 된다.

니블 x와	AND	OR	XOR
0 (0000)	0 (지움)	x (그대로)	x (그대로)
F (1111)	x (그대로)	F (모두 켬)	x를 뒤집음 (15 − x)

예: 0x12345678 AND 0x0000FF00 → 니블별로 F 자리는 그대로, 0 자리는 0 → **0x00005600**. 0x5A XOR 0xFF → 5 뒤집기 = A($0101_2 \rightarrow 1010_2$), A 뒤집기 = 5 → **0xA5**. 0과 F가 아닌 니블은 그 자리만 2진 4자리로 풀어 계산한다.

마스크: 남기기·켜기·뒤집기

원하는 비트 자리에 1을 둔 수를 **마스크(mask)**라 한다. 어떤 연산과 함께 쓰느냐에 따라 효과가 다르다. [Ch2 p.26–27]

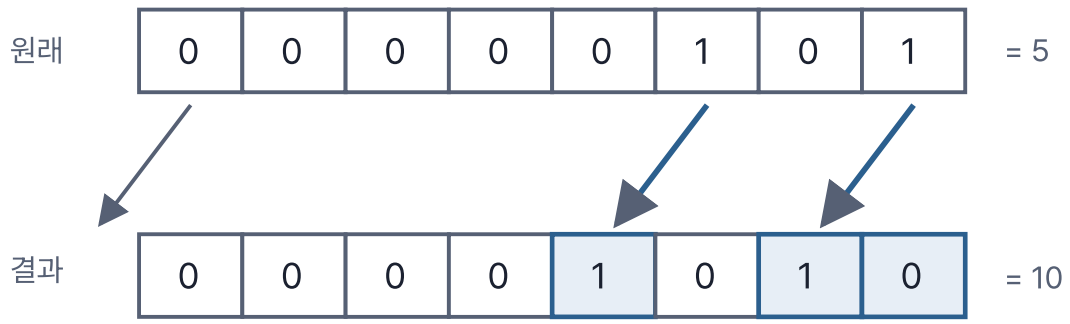
하고 싶은 일	연산	마스크의 1 자리에서	마스크의 0 자리에서	예 (8비트)
특정 비트만 남기기(나머지 지우기)	AND	원래 값 유지	0으로 지움	0x3C AND 0xF0 = 0x30
특정 비트 켜기	OR	1로 켬	원래 값 유지	0x5A OR 0x0F = 0x5F
특정 비트 뒤집기	XOR	뒤집음	원래 값 유지	0xA5 XOR 0x0F = 0xAA

n번 비트 하나만 1인 마스크는 2^n이다. 예: 3번 비트 = 0000 1000 = 0x08, 7번 비트 = 0x80. 그래서 0x80 OR 0x01 = 0x81은 7번과 0번 비트를 켠 값이다.

RISC-V에는 NOT 명령어가 없고 모든 비트가 1인 수와 XOR해서 NOT을 만든다. 위 표의 "F와 XOR = 뒤집기"를 32비트 전체에 한 것이다. [Ch2 p.27]

시프트

시프트(shift)는 모든 비트를 정해진 칸 수만큼 한쪽으로 옮기는 연산이다. 밀려난 비트는 버려지고, 반대쪽 빈 칸이 채워진다. [Ch2 p.25]



왼쪽 1칸 시프트: 0000 0101(5) → 0000 1010(10). 모든 비트의 자릿값이 2배가 된다.

- **왼쪽 시프트(shift left)**: 왼쪽으로 밀고 오른쪽 빈 칸은 0. k칸 밀면 **× 2^k**(넘쳐 버려지는 1이 없을 때). 예: 5 << 3 = 0010 1000 = 40 = 5 × 8.
- **오른쪽 시프트(shift right)**: 오른쪽으로 밀고 맨 오른쪽 비트들은 버려진다. k칸 밀면 **÷ 2^k의 몫**(나머지는 버려진 비트). 예: 23 = 0001 0111 → 1칸 → 0000 1011 = 11. 23 ÷ 2 = 11 나머지 1, 버려진 비트가 그 나머지 1이다.
- 16진에서 4칸 시프트 = 16진 한 자리 이동. 0x00000DC0을 왼쪽 8칸 → 0x000DC000(두 자리 이동).

> x << k = x × 2^k · x >> k = x ÷ 2^k의 몫

밀려나는 비트 주의. 8비트 1100 0000(192)을 왼쪽 1칸 밀면 맨 왼쪽 1이 버려져 1000 0000(128)이 된다. 384는 8비트에 담기지 않으므로 ×2가 성립하지 않는다.

논리 시프트와 산술 시프트 (맛보기)

오른쪽 시프트는 왼쪽에 생긴 빈 칸을 무엇으로 채우는지에 따라 둘로 나뉜다. 왼쪽 시프트는 둘 다 오른쪽을 0으로 채우므로 하나뿐이다(`sll`). [Ch2 p.25]

종류	RISC-V	왼쪽 빈 칸	8비트 1111 0000을 2칸	의미
논리 오른쪽 시프트(logical)	`srl`, `srli`	0	0011 1100	무부호 240 ÷ 4 = 60
산술 오른쪽 시프트(arithmetic)	`sra`, `srai`	부호 비트(MSB) 복제	1111 1100	2의 보수 −16 ÷ 4 = −4

산술 시프트는 음수의 부호를 지키려고 MSB를 복제한다. 슬라이드 예: 1100 0011을 sra 1칸 → 1110 0001. [Ch2 p.25] 32비트 예: 0x87654321을 4칸 오른쪽으로 밀면 srli는 0x08765432, srai는 MSB가 1이므로 왼쪽 빈 칸이 1111로 채워져 **0xF8765432**다.

예제

x = 0x00000DC0일 때 (1) x의 4~7번 비트만 꺼내 맨 오른쪽으로 옮긴 값 (2) x를 왼쪽 8칸 시프트한 값을 16진으로 쓰시오.

풀이 보기

0xDC0 = 1101 1100 0000. 4~7번 비트는 오른쪽에서 두 번째 니블 C(1100)이다.

(1) 오른쪽으로 4칸 밀면 0xDC. 여기에 마스크 0xF를 AND하면 마지막 니블만 남는다 → **0xC**. 시프트로 자리를 옮기고, AND로 나머지를 지운다.

(2) 8칸 = 16진 두 자리 → **0x000DC000**.

함정

비트별 연산을 덧셈처럼 계산해 올림을 넘기는 실수. 1 OR 1 = 1이고 1 XOR 1 = 0이며, 어느 쪽도 왼쪽으로 올림이 없다.

세로로 맞출 때 왼쪽 끝을 맞추면 틀린다. 길이가 다르면 짧은 수 앞에 0을 채워 **오른쪽 끝**을 맞춘다.

"오른쪽 시프트 = 나누기"는 몫만 남는다. 23 >> 1 = 11이지 11.5가 아니다. 음수를 sra하면 몫이 아래쪽(더 작은 쪽)으로 내려간다: 8비트 −7(1111 1001)을 sra 1칸 → 1111 1100 = −4.

음수(MSB가 1)를 srl하면 왼쪽에 0이 들어와 큰 양수가 된다. 부호 있는 수를 2로 나누려면 sra를 쓴다. [Ch2 p.25]

스스로 확인

1. 빈칸: 1 AND 0 = ( ), 1 OR 0 = ( ), 1 XOR 1 = ( ), NOT 0 = ( )

   답 보기

   0, 1, 0, 1.

2. 두 비트가 다를 때만 1이 되는 연산은?

   답 보기

   XOR(배타적 논리합). [Ch2 p.27]

3. 1100_2 AND 1010_2, 1100_2 OR 1010_2, 1100_2 XOR 1010_2를 쓰시오.

   답 보기

   1000_2, 1110_2, 0110_2.

4. 8비트 NOT 1100 1010을 2진과 16진으로 쓰시오.

   답 보기

   0011 0101 = 0x35.

5. 0x0F0F AND 0x00FF는?

   답 보기

   니블별로 0&0, F&0, 0&F, F&F → **0x000F**.

6. 0x12345678 OR 0x000000FF는?

   답 보기

   마지막 두 니블이 F로 켜짐 → **0x123456FF**.

7. 0x12345678 XOR 0xFFFF0000은?

   답 보기

   앞 네 니블 뒤집기: 1→E, 2→D, 3→C, 4→B → **0xEDCB5678**.

8. 8비트 값의 3번 비트만 1로 켜려면 어떤 연산과 어떤 마스크(16진)를 쓰는가?

   답 보기

   OR, 마스크 0x08 (2^3 = 0000 1000).

9. 8비트 0000 0101을 왼쪽 3칸 시프트한 값(2진, 10진)은?

   답 보기

   0010 1000 = 40 (5×2^3).

10. 108(0110 1100)을 논리 오른쪽 2칸 시프트한 값은?

    답 보기

    0001 1011 = 27 (108 ÷ 4 = 27).

11. 8비트 1111 0000을 2칸 srl, sra 한 결과를 각각 쓰시오.

    답 보기

    srl 0011 1100(왼쪽 0), sra 1111 1100(부호 비트 1 복제).

12. 32비트 0x87654321을 srai 4칸 한 결과는?

    답 보기

    MSB가 1이므로 왼쪽 4칸을 1로 채움 → **0xF8765432**. srli라면 0x08765432.

13. O/X: 8비트 1100 0000을 왼쪽 1칸 시프트하면 값이 2배(384)가 된다.

    답 보기

    X. 맨 왼쪽 1이 밀려나 버려져 1000 0000 = 128. 8비트에 384를 담을 수 없다.

14. NOT 명령어가 없는 RISC-V에서 x의 모든 비트를 뒤집는 방법은?

    답 보기

    모든 비트가 1인 수(−1)와 XOR한다. `xori rd, rs1, -1`. [Ch2 p.27]

이 기초가 시험에서 쓰이는 곳

- Ch2 · 6 논리 연산과 시프트: and·or·xor 예제(0x00000DC0, 0x00003C00), NOT = XOR, sll·srl·sra 결과 쓰기. [Ch2 p.25–27]
- Ch2 · 7: 배열 인덱스 × 4를 `slli` 2칸으로 계산.
- Ch3 · 2: 곱셈 하드웨어의 피승수 왼쪽 시프트·승수 오른쪽 시프트.
- Ch2 · 5와 실전 · 4: 기계어에서 칸(필드)을 잘라 내는 것은 시프트 + AND 마스크와 같은 생각이다.

기초 · 5 이 기초가 쓰이는 곳: Ch1 · 7~10, Ch4 · 2, 실전 · 1

단위와 지수 계산

이 단원을 마치면 10의 거듭제곱을 계산기 없이 곱하고 나누며, ns·ps·GHz·MHz를 서로 바꾸고, 클럭 주기와 클럭 속도를 환산하며, "몇 배 빠르다", "몇 % 줄었다", 가중 평균을 구할 수 있다. PDF 외 보충

쉽게 말하면

컴퓨터 성능 문제의 숫자는 아주 크거나(초당 40억 번) 아주 작다(0.00000000025초). 이런 수를 0을 줄줄이 써서 계산하면 반드시 자리를 틀린다. 그래서 "4×10^{9}", "250×10^{-12}"처럼 **앞의 숫자**와 **10의 몇 제곱**을 나눠 적는다. 앞의 숫자끼리 계산하고, 10의 지수끼리 계산한 뒤 합치면 된다.

단위 접두어(k, M, G, m, μ, n, p)는 "10의 몇 제곱"에 붙인 별명이다. G를 보면 10^{9}, n을 보면 10^{-9}로 바꿔 적는 것이 계산의 첫 단계다.

비유가 안 맞는 지점 메모리 크기에서는 같은 K, M, G가 10^{3}이 아닌 $2^{10} = 1024$로 쓰이기도 한다. 시간·주파수와 메모리 크기는 기준이 다를 수 있으니 따로 기억해야 한다.

1. 지수 법칙 다시 보기

10^{a}는 10을 a번 곱한 수다. $10^{3} = 10 \times 10 \times 10 = 1000$. 지수가 음수면 그만큼 나눈 수다. $10^{-3} = 1 / 10^{3} = 0.001$. $10^{0} = 1$이다.

$10^{a} \times 10^{b} = 10^{a+b}$ (곱하면 지수를 더한다)

$10^{a} / 10^{b} = 10^{a-b}$ (나누면 지수를 뺀다)

$1 / 10^{b} = 10^{-b}$ (분모의 10^{b}를 위로 올리면 부호가 바뀐다)

예를 하나씩 보자.

- $10^{3} \times 10^{6} = 10^{3+6} = 10^{9}$
- $10^{9} \times 10^{-12} = 10^{9+(-12)} = 10^{-3}$
- $10^{9} / 10^{6} = 10^{9-6} = 10^{3}$
- $1 / 10^{-9} = 10^{9}$ (음수 지수를 분모에서 위로 올리면 양수가 된다)
- $10^{6} / 10^{-3} = 10^{6-(-3)} = 10^{9}$ (빼는 수가 음수이면 더하는 것과 같다)

앞에 숫자가 붙어 있으면 **숫자끼리, 지수끼리 따로** 계산한다.

- $(2 \times 10^{9}) \times (3 \times 10^{-9}) = (2 \times 3) \times 10^{9-9} = 6 \times 10^{0} = 6$
- $(8 \times 10^{6}) / (2 \times 10^{9}) = (8 / 2) \times 10^{6-9} = 4 \times 10^{-3}$
- $1 / (4 \times 10^{9}) = (1 / 4) \times 10^{-9} = 0.25 \times 10^{-9} = 250 \times 10^{-12}$

마지막 줄처럼 0.25×10^{-9}를 250×10^{-12}로 바꾸는 것을 **자리 옮기기**라 한다. 앞의 숫자에 1000(10^{3})을 곱했으면 지수에서 3을 빼서 균형을 맞춘다. 앞 숫자를 1000배 하면 지수는 −3, 앞 숫자를 1/1000로 하면 지수는 +3이다.

2. 단위 접두어

국제 단위계(SI)의 접두어는 10의 거듭제곱을 뜻한다. 3제곱씩 건너뛰는 것만 외우면 된다.

큰 쪽 접두어	값	작은 쪽 접두어	값
k (킬로, kilo)	10^3 (천)	m (밀리, milli)	10^{-3} (천분의 1)
M (메가, mega)	10^6 (백만)	μ (마이크로, micro)	10^{-6} (백만분의 1)
G (기가, giga)	10^9 (십억)	n (나노, nano)	10^{-9} (십억분의 1)
T (테라, tera)	10^{12} (조)	p (피코, pico)	10^{-12} (조분의 1)

대문자 M(10^6)과 소문자 m(10^{-3})은 완전히 다른 값이다. MHz는 메가헤르츠, ms는 밀리초다. 한국어 큰 수와 맞춰 보면 1억 = 10^8, 10억 = 10^9이다. 문제에서 "명령어 6억 개"라고 하면 6×10^8개로 바꿔 쓴다.

3. 시간과 주파수

초(s, second) 시간의 기본 단위. ms = 10^{-3} s, μs = 10^{-6} s, ns = 10^{-9} s, ps = 10^{-12} s.

헤르츠(Hz, hertz) 1초에 몇 번 일어나는지 나타내는 단위. 1 Hz = 1초에 1번. 1 GHz = 1초에 10^9번.

주파수(frequency) 1초당 반복 횟수. 클럭에서는 클럭 속도(clock rate)라 부른다.

주기(period) 한 번 반복하는 데 걸리는 시간. 클럭에서는 클럭 주기(clock cycle time)라 부른다.

"1초에 4번 반복한다"면 한 번은 1/4초 걸린다. 반대로 "한 번에 1/4초 걸린다"면 1초에 4번 반복한다. 그래서 주기와 주파수는 서로 역수(1을 나눈 값)다. [Ch1 p.26]

```
클럭 주기 = 1 / 클럭 속도      클럭 속도 = 1 / 클럭 주기
```

짝이 되는 단위를 표로 기억하면 환산이 빨라진다. 지수의 부호만 바뀐다.

주파수	Hz로	짝이 되는 주기	초로
1 kHz	10^3 Hz	1 ms	10^{-3} s
1 MHz	10^6 Hz	1 μs	10^{-6} s
1 GHz	10^9 Hz	1 ns	10^{-9} s
1 THz	10^{12} Hz	1 ps	10^{-12} s

환산 순서는 늘 같다. ① 접두어를 10의 거듭제곱으로 바꾼다. ② 1을 그 값으로 나눈다(앞 숫자는 1 / 숫자, 지수는 부호를 바꾼다). ③ 보기 좋은 접두어로 자리를 옮긴다.

클럭 속도	계산	클럭 주기
4 GHz	$1 / (4 \times 10^9) = 0.25 \times 10^{-9}$ s	0.25 ns = 250 ps
2 GHz	$1 / (2 \times 10^9) = 0.5 \times 10^{-9}$ s	0.5 ns = 500 ps
2.5 GHz	$1 / (2.5 \times 10^9) = 0.4 \times 10^{-9}$ s	0.4 ns = 400 ps
5 GHz	$1 / (5 \times 10^9) = 0.2 \times 10^{-9}$ s	0.2 ns = 200 ps
800 MHz	$1 / (8 \times 10^8) = 0.125 \times 10^{-8}$ s	1.25 ns
500 MHz	$1 / (5 \times 10^8) = 0.2 \times 10^{-8}$ s	2 ns

800 MHz = $800 \times 10^6 = 8 \times 10^8$ Hz다. $1 / 8 = 0.125$이고 10^8의 역수는 10^{-8}이므로 $0.125 \times 10^{-8} = 1.25 \times 10^{-9}$ s = 1.25 ns다. 이 표의 250 ps ↔ 4 GHz는 슬라이드의 예 그대로다. [Ch1 p.26]

예제

(1) 클럭 주기가 250 ps인 CPU의 클럭 속도는? (2) 클럭 속도가 3.2 GHz인 CPU의 클럭 주기는 몇 ps인가?

풀이 보기

(1) 250 ps = 250×10^{-12} s = 2.5×10^{-10} s. 클럭 속도 = $1 / (2.5 \times 10^{-10}) = (1 / 2.5) \times 10^{10} = 0.4 \times 10^{10} = 4 \times 10^9$ Hz = **4 GHz**.

(2) 3.2 GHz = 3.2×10^9 Hz. 클럭 주기 = $1 / (3.2 \times 10^9) = (1 / 3.2) \times 10^{-9} = 0.3125 \times 10^{-9}$ s = 0.3125 ns = **312.5 ps**.

검산: $312.5 \times 10^{-12} \times 3.2 \times 10^9 = 1000 \times 10^{-3} = 1$. 주기 × 속도 = 1이면 맞다.

4. 메모리 크기: KB와 KiB

메모리 크기는 바이트(byte, 8비트) 단위로 센다. 주소가 2진수라서 메모리 크기는 2의 거듭제곱이 되기 쉽고, $2^{10} = 1024$가 1000과 비슷하므로 K를 1024로 쓰는 관습이 생겼다. 둘을 구별하려고 2의 거듭제곱 쪽에는 Ki, Mi, Gi(키비, 메비, 기비)를 쓴다.

10진 단위	바이트 수	2진 단위	바이트 수
KB (킬로바이트)	10^3 = 1,000	KiB (키비바이트)	2^{10} = 1,024
MB (메가바이트)	10^6	MiB (메비바이트)	2^{20} = 1,048,576
GB (기가바이트)	10^9	GiB (기비바이트)	2^{30} = 1,073,741,824
TB (테라바이트)	10^{12}	TiB (테비바이트)	2^{40}

2의 지수도 10의 지수와 같은 법칙을 따른다. $2^{10} \times 2^{22} = 2^{32} = 4 \times 2^{30}$이므로 2^{32}바이트 = 4 GiB다. 32비트 주소로 가리킬 수 있는 바이트 수가 이것이다. 현실에서는 GiB를 GB라고 적는 경우가 많으므로, 문제에 나오면 어느 뜻인지 확인한다.

5. 배수, 백분율, 가중 평균

n배 빠르다. 성능은 실행 시간의 역수다. 그래서 "X가 Y보다 n배 빠르다"의 n은 **느린 쪽 시간 / 빠른 쪽 시간**이다. A가 10초, B가 15초면 A가 15 / 10 = 1.5배 빠르다. [Ch1 p.24]

```
n = 실행 시간_Y / 실행 시간_X  (X가 더 빠를 때 n > 1)
```

백분율(%)은 100을 기준으로 한 비율이다. 25% = 25 / 100 = 0.25. "몇 % 줄었다"는 **줄어든 양 / 원래 양**이다. 100초가 80초로 줄면 줄어든 양 20초를 원래 100초로 나눠 20% 감소다.

```
감소율(%) = (원래 - 나중) / 원래 × 100    증가율(%) = (나중 - 원래) / 원래 × 100
```

같은 100초 → 80초를 배수로 말하면 100 / 80 = 1.25배 빨라졌다. 성능(시간의 역수)으로는 1.25 − 1 = 0.25, 즉 25% 향상이다. 시간 20% 감소와 성능 25% 향상은 같은 사건을 다르게 말한 것이다. 기준(분모)이 다르기 때문에 숫자가 다르다.

가중 평균(weighted average)은 개수나 비율이 다른 값들의 평균이다. 각 값에 자기 비율(가중치)을 곱해 더한다. 비율을 다 더하면 1(100%)이 되어야 한다.

```
가중 평균 = 값_1 × 비율_1 + 값_2 × 비율_2 + …
```

```
개수로 주어지면: 가중 평균 = (값_1 × 개수_1 + 값_2 × 개수_2 + …) / 전체 개수
```

명령어의 50%가 CPI 1, 30%가 CPI 2, 20%가 CPI 3이면 평균 CPI = 1 × 0.5 + 2 × 0.3 + 3 × 0.2 = 0.5 + 0.6 + 0.6 = 1.7이다. (1 + 2 + 3) / 3 = 2처럼 단순 평균을 내면 틀린다. CPU 성능 공식의 평균 CPI가 바로 이 가중 평균이다. [Ch1 p.28]

6. 분수·소수와 계산기 없이 하는 요령

- **자주 나오는 분수**: 1/2 = 0.5, 1/4 = 0.25, 1/5 = 0.2, 1/8 = 0.125, 1/3 ≈ 0.333, 2/3 ≈ 0.667, 1/1.25 = 0.8, 1/1.6 = 0.625, 1/2.5 = 0.4.
- **소수로 나누기**: 분자와 분모에 같은 수를 곱해 소수점을 없앤다. 2 / 1.6 = 20 / 16 = 1.25. 1 / 0.25 = 100 / 25 = 4.
- **10^6으로 묶기**: MIPS는 백만(10^6) 단위다. 클럭 2 GHz = 2000 × 10^6 Hz로 적으면 2 × 10^9 / (1.6 × 10^6) = 2000 / 1.6 = 1250이 바로 나온다. [Ch1 p.34]
- **약분 먼저**: 곱셈과 나눗셈만 있는 식은 분자와 분모에 같은 수가 있으면 먼저 지운다. 명령어 수 I가 양쪽에 있으면 몰라도 비율이 나온다.
- **단위도 약분된다**: (명령어) × (사이클 / 명령어) × (초 / 사이클) = 초. 답의 단위가 맞는지로 식을 검산할 수 있다. [Ch1 p.29]

예제 (CPU 시간과 MIPS의 형태)

클럭 속도 2 GHz, 명령어 수 5×10^8개, 평균 CPI 1.6인 프로그램이 있다. (1) CPU 시간 (2) MIPS를 구하시오.

풀이 보기

1단계: 단위를 바꾼다. 2 GHz = 2×10^9 Hz.

2단계: 사이클 수 = 명령어 수 × CPI = $5 \times 10^8 \times 1.6 = 8 \times 10^8$ 사이클.

3단계: CPU 시간 = 사이클 수 / 클럭 속도 = $8 \times 10^8 / (2 \times 10^9) = 4 \times 10^{8-9} = 4 \times 10^{-1}$ = **0.4초** (= 400 ms).

4단계: MIPS = 클럭 속도 / (CPI × 10^6) = $2000 \times 10^6 / (1.6 \times 10^6) = 2000 / 1.6$ = **1250**.

검산: MIPS = 명령어 수 / (시간 × 10^6) = $5 \times 10^8 / (0.4 \times 10^6) = 12.5 \times 10^2 = 1250$. 같다. [Ch1 p.28, 34]

함정

클럭 **주기**(ns, ps)가 주어지면 곱하고, 클럭 **속도**(GHz, MHz)가 주어지면 나눈다. 둘을 섞어 "명령어 수 × CPI × 4 GHz"처럼 쓰면 답이 엉뚱하게 커진다. 계산 전에 어느 쪽인지 단위로 확인한다.

M(10^6)과 m(10^{-3})을 헷갈리면 10^9배가 틀린다. $1/10^{-9}$에서 지수 부호를 안 바꾸는 실수도 흔하다.

"20% 줄었다"와 "1.2배 빨라졌다"는 다르다. 100 → 80은 1.25배다. 백분율은 무엇을 분모로 했는지가 정해져야 의미가 있다.

스스로 확인

1. $10^5 \times 10^{-2}$와 $10^9 / 10^{-3}$을 각각 10의 거듭제곱으로 쓰시오.

   답 보기

   10^3, 10^{12}. 곱하면 5 + (−2) = 3, 나누면 9 − (−3) = 12.

2. $(6 \times 10^8) / (3 \times 10^9)$를 계산하시오.

   답 보기

   2×10^{-1} = 0.2. 숫자 6 / 3 = 2, 지수 8 − 9 = −1.

3. 3 ns를 초(s)로, 0.5 ns를 ps로 바꾸시오.

   답 보기

   3×10^{-9} s, 500 ps. 1 ns = 1000 ps이므로 0.5 × 1000 = 500.

4. "명령어 6억 개"와 "명령어 32억 개"를 10의 거듭제곱 꼴로 쓰시오.

   답 보기

   6×10^8, 3.2×10^9. 1억 = 10^8, 32억 = $32 \times 10^8 = 3.2 \times 10^9$.

5. 클럭 속도가 2 GHz인 CPU의 클럭 주기를 ns와 ps로 쓰시오.

   답 보기

   0.5 ns = 500 ps. $1 / (2 \times 10^9) = 0.5 \times 10^{-9}$ s.

6. 클럭 주기가 200 ps인 CPU의 클럭 속도는?

   답 보기

   5 GHz. $1 / (200 \times 10^{-12}) = 1 / (2 \times 10^{-10}) = 0.5 \times 10^{10} = 5 \times 10^9$ Hz.

7. 클럭 주기 1.25 ns ↔ 클럭 속도 ? 그리고 클럭 속도 3 GHz ↔ 클럭 주기 약 몇 ps?

   답 보기

   800 MHz, 약 333 ps. $1 / (1.25 \times 10^{-9}) = 0.8 \times 10^9 = 8 \times 10^8$ Hz. $1 / (3 \times 10^9) \approx 0.333 \times 10^{-9}$ s ≈ 333 ps.

8. 1 KB와 1 KiB는 각각 몇 바이트인가? 64 KiB는 몇 바이트인가?

   답 보기

   1,000바이트, 1,024바이트. 64 KiB = 64 × 1024 = 65,536바이트(= 2^{16}).

9. 2^{32}바이트는 몇 GiB인가?

   답 보기

   4 GiB. $2^{32} = 2^2 \times 2^{30} = 4 \times 2^{30}$.

10. 같은 프로그램이 컴퓨터 A에서 12초, B에서 8초 걸린다. 어느 쪽이 몇 배 빠른가?

    답 보기

    B가 1.5배 빠르다. 느린 쪽 시간 / 빠른 쪽 시간 = 12 / 8 = 1.5.

11. 실행 시간이 200초에서 150초로 줄었다. (1) 실행 시간은 몇 % 줄었나? (2) 몇 배 빨라졌나?

답 보기

(1) 25%. (200 − 150) / 200 = 50 / 200 = 0.25. (2) 200 / 150 ≈ 1.33배(성능 약 33% 향상).

12. 명령어 비율이 40%(CPI 1), 40%(CPI 2), 20%(CPI 4)일 때 평균 CPI는?

답 보기

2.0. 1 × 0.4 + 2 × 0.4 + 4 × 0.2 = 0.4 + 0.8 + 0.8 = 2.0. 단순 평균 (1 + 2 + 4) / 3 ≈ 2.33은 틀린 값이다.

13. 명령어 10^9개, 평균 CPI 2, 클럭 4 GHz일 때 CPU 시간은?

답 보기

0.5초. $10^9 \times 2 / (4 \times 10^9) = 2 / 4 = 0.5$.

14. 클럭 3 GHz, 평균 CPI 1.5인 컴퓨터의 MIPS는?

답 보기

2000. $3000 \times 10^6 / (1.5 \times 10^6) = 3000 / 1.5 = 2000$.

15. 명령어 4×10^8개를 0.2초에 실행했다. MIPS는?

답 보기

2000. $4 \times 10^8 / (0.2 \times 10^6) = 20 \times 10^2 = 2000$.

16. 클럭 주기 0.4 ns, 명령어 2×10^9개, 평균 CPI 1.5일 때 CPU 시간은?

답 보기

1.2초. $2 \times 10^9 \times 1.5 \times 0.4 \times 10^{-9} = 2 \times 1.5 \times 0.4 = 1.2$. 주기가 주어졌으므로 곱한다.

17. 400 ms 걸리는 프로그램에서 240 ms가 곱셈이다. 곱셈만 3배 빨라지면 실행 시간과 속도 향상은? (Ch1 · 10 암달의 법칙 형태)

답 보기

240 ms. 240 / 3 + (400 − 240) = 80 + 160 = 240 ms. 속도 향상 = 400 / 240 ≈ 1.67배. 3배는 곱셈 부분에만 나눈다.

[Ch1 p.33]

이 기초가 시험에서 쓰이는 곳

- Ch1 · 7 "n배 빠르다", Ch1 · 8 클럭 주기 ↔ 클럭 속도 환산과 CPU 시간 공식 [Ch1 p.24, 26–29]
- Ch1 · 10 암달의 법칙(일부 개선 후 시간)과 MIPS 계산(10^6 묶기) [Ch1 p.33–34]
- Ch4 · 2, Ch4 · 8 가장 긴 지연으로 클럭 주기·주파수 구하기 [Ch4 p.10, 29]
- 실전 · 1 성능 계산 전체(단위 환산, 가중 평균 CPI, 개선율)

기초 · 6 이 기초가 쓰이는 곳: Ch4 · 1~4

디지털 논리 기초

이 단원을 마치면 AND·OR·NOT·XOR 게이트의 진리표를 쓰고, 멀티플렉서·디코더·가산기·ALU가 하는 일을 말하며, 조합 회로와 순차 회로를 구별하고, 클럭 엣지에서 레지스터가 갱신되는 과정을 따라갈 수 있다. PDF 외 보충

쉽게 말하면

컴퓨터 안의 모든 정보는 전선에 전기가 "높다(1)"와 "낮다(0)" 두 상태뿐이다. 이 0과 1을 받아 정해진 규칙대로 새 0과 1을 내보내는 작은 부품이 게이트다. 게이트를 레고처럼 쌓으면 덧셈기, 선택기가 되고, 이것을 더 쌓으면 CPU가 된다.

부품은 두 종류다. 하나는 계산기처럼 입력을 넣으면 바로 답이 나오고 아무것도 기억하지 않는다(조합 회로). 다른 하나는 메모지처럼 값을 기억해 두었다가 정해진 순간에만 고쳐 쓴다(순차 회로). 그 "정해진 순간"을 알려 주는 신호가 클럭이다.

비유가 안 맞는 지점 레고는 붙이는 순간 완성되지만, 실제 게이트는 신호가 통과하는 데 아주 짧은 시간(지연)이 걸린다. 이 지연이 쌓여 클럭 주기의 최소값을 정한다.

1. 0과 1은 전압이다

디지털 회로는 전압(전기의 높이)을 두 범위로만 나눠 읽는다. 높은 전압은 1(참, true), 낮은 전압은 0(거짓, false)이다. 하나의 0 또는 1을 비트(bit)라 하고, 전선 한 가닥이 비트 하나를 나른다. 32비트 값은 전선 32가닥이 나란히 가는 것이다. 회로 그림에서 굵은 선 하나가 여러 가닥을 묶어 나타내기도 한다.

진리표(truth table) 가능한 모든 입력 조합과 그때의 출력을 적은 표. 입력이 n개면 줄이 2^n개다.

활성(asserted) 신호가 1인 상태. 0이면 비활성(deasserted). Ch4에서 "제어 신호를 활성한다"고 쓴다. [Ch4 p.9]

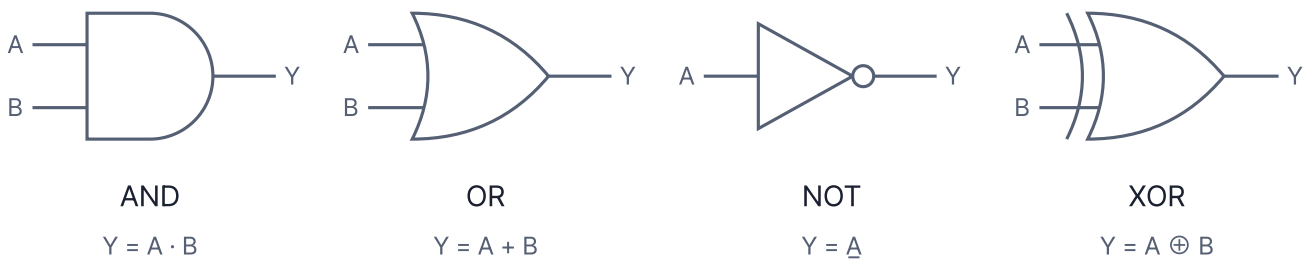
2. 논리 게이트 4가지

AND (논리곱) 입력이 **모두** 1일 때만 출력 1. 식으로 Y = A · B.

OR (논리합) 입력 중 **하나라도** 1이면 출력 1. Y = A + B (여기서 +는 덧셈이 아닌 OR).

NOT (부정, 인버터) 입력 하나를 뒤집는다. 0 → 1, 1 → 0. Y = $\underline{A}$ 또는 ¬A.

XOR (배타적 논리합, exclusive OR) 두 입력이 **서로 다를 때** 출력 1. Y = A ⊕ B.



게이트 기호. AND는 반달 모양, OR은 화살촉 모양, NOT은 삼각형과 작은 원(원이 "뒤집기"를 뜻한다), XOR은 OR 뒤에 곡선이 하나 더 붙은 모양이다.

A	B	A AND B	A OR B	A XOR B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

A	NOT A
0	1
1	0

입력이 2개이므로 줄은 2^2 = 4개다. 입력 순서는 00, 01, 10, 11처럼 2진수로 0, 1, 2, 3을 세는 순서로 쓰면 빠뜨리지 않는다. 32비트 값끼리의 AND·OR·XOR은 같은 자리 비트끼리 이 표를 32번 적용한 것이다(Ch2 · 6 논리 연산). [Ch2 p.25]

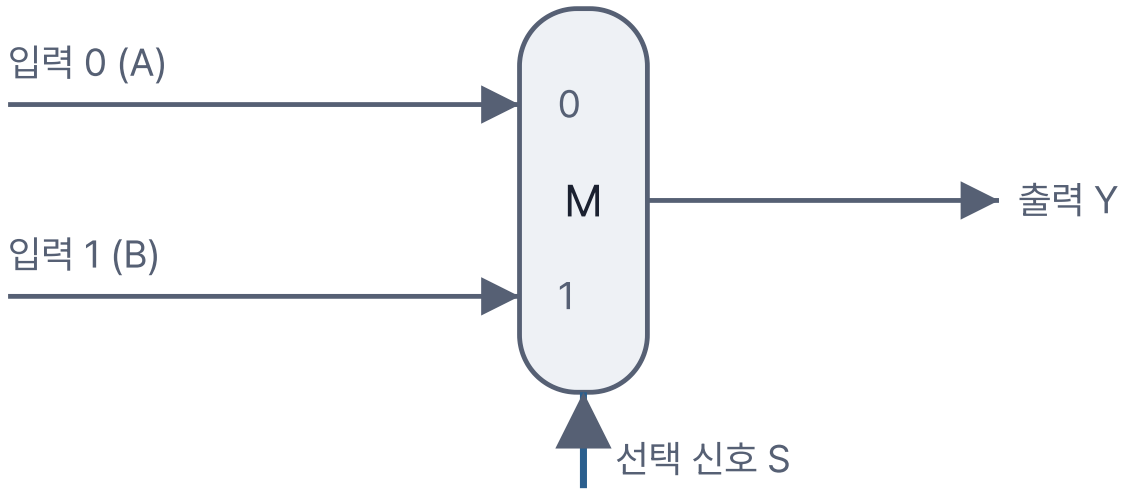
3. 조합 회로

게이트를 이어 만든 회로 중에서 **출력이 지금 들어온 입력만으로 정해지는** 회로를 조합 회로(combinational circuit)라 한다. 같은 입력이면 언제나 같은 출력이 나온다. 기억하는 값이 없다. Ch4의 조합 소자(AND 게이트, ALU, 가산기, MUX)가 여기 속한다. [Ch4 p.9]

예: Y = (A AND B) OR C에 A = 1, B = 0, C = 1을 넣으면 A AND B = 0, 0 OR 1 = 1이므로 Y = 1이다. 괄호 안부터 계산하는 것은 수학과 같다.

4. 멀티플렉서(MUX)

멀티플렉서(MUX, multiplexer)는 여러 입력 중 하나를 **선택 신호(select)**에 따라 골라 출력으로 내보내는 조합 회로다. 데이터 선택기(data selector)라고도 한다. 철도의 선로 전환기처럼 어느 길의 기차를 통과시킬지 고른다. [Ch4 p.7]



2:1 MUX. 입력 칸 옆의 숫자 0, 1은 선택 신호가 그 값일 때 고르는 입력이다. S = 0이면 Y = A, S = 1이면 Y = B.

입력이 2개면 2:1 MUX(2입력 MUX)라 한다. 진리표를 모든 입력에 대해 쓰면 다음과 같다. 입력이 S, A, B 세 개이므로 2^3 = 8줄이다.

S	A (입력 0)	B (입력 1)	Y	설명
0	0	0	0	S = 0 → Y는 A를 따라간다
0	0	1	0	
0	1	0	1	
0	1	1	1	
1	0	0	0	S = 1 → Y는 B를 따라간다
1	0	1	1	
1	1	0	0	
1	1	1	1	

게이트로 쓰면 Y = (A AND NOT S) OR (B AND S)다. A, B가 32비트 값이면 이 회로를 32개 나란히 두고 같은 S를 연결한다. 입력이 4개면 선택 신호가 2비트(00, 01, 10, 11), 8개면 3비트가 필요하다. 입력 2^k개를 고르려면 선택 신호 k비트다.

Ch4에서는 "레지스터에 쓸 값이 ALU 결과인가, 메모리 값인가"처럼 같은 자리에 들어올 후보가 둘 이상일 때 MUX를 둔다. 선을 그냥 한 점에 이으면 두 신호가 충돌하기 때문이다. 단일 사이클 데이터패스에는 MUX가 세 곳 있다. [Ch4 p.7, 16]

5. 디코더

디코더(decoder)는 n비트 입력을 받아 출력 2^n개 중 **입력 번호에 해당하는 하나만** 1로 만드는 조합 회로다. 번호를 "어느 줄을 켤지"로 바꿔 준다.

입력 I_1 I_0	출력 0	출력 1	출력 2	출력 3
00	1	0	0	0
01	0	1	0	0
10	0	0	1	0
11	0	0	0	1

위는 2비트 입력, 출력 4개인 2:4 디코더다. 레지스터 파일이 5비트 레지스터 번호(rd)를 받아 32개 레지스터 중 하나를 골라 쓸 때 5:32 디코더 같은 회로가 쓰인다. 명령어의 opcode를 보고 어떤 명령어인지 알아내는 것도 넓은 뜻의 디코딩이다(Ch4 · 5, 6). 교재 기준

6. 가산기와 ALU

2진수 한 자리 덧셈은 0 + 0 = 0, 0 + 1 = 1, 1 + 0 = 1, 1 + 1 = 10_2(= 2)이다. 1 + 1일 때 그 자리는 0이 되고 윗자리로 1이 올라간다. 올라가는 1을 자리올림(carry)이라 한다.

반가산기(half adder) 1비트 두 개(A, B)를 더해 합(S, sum)과 자리올림(C, carry)을 낸다. S = A XOR B, C = A AND B.

전가산기(full adder) 1비트 세 개(A, B, 아랫자리에서 올라온 자리올림 C_{in})를 더해 합 S와 윗자리로 보낼 자리올림 C_{out}을 낸다.

A	B	반가산기 S	반가산기 C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

A	B	C_{in}	1의 개수	S	C_{out}
0	0	0	0	0	0
0	0	1	1	1	0
0	1	0	1	1	0
0	1	1	2	0	1
1	0	0	1	1	0
1	0	1	2	0	1
1	1	0	2	0	1
1	1	1	3	1	1

전가산기 표는 "1의 개수"를 2진수로 쓴 것이다. 개수가 2면 10_2(S = 0, C_{out} = 1), 3이면 11_2(S = 1, C_{out} = 1)이다.

여러 비트 덧셈기: 전가산기 32개를 한 줄로 잇고, 각 자리의 C_{out}을 바로 윗자리의 C_{in}으로 연결하면 32비트 덧셈기가 된다. 맨 아랫자리 C_{in}은 0이다. 자리올림이 아래에서 위로 차례로 전해지므로 이것을 리플 캐리 가산기(ripple-carry adder)라 한다. 교재 기준

예제

4비트 덧셈기로 $0110_2 + 0111_2$을 계산하시오. 자리마다 S와 C_{out}을 쓰시오.

풀이 보기

맨 오른쪽(0번 자리)부터 계산한다. C_{in}은 처음에 0.

0번 자리: 0 + 1 + 0 = 1 → S = 1, C_{out} = 0

1번 자리: 1 + 1 + 0 = 2 → S = 0, C_{out} = 1

2번 자리: 1 + 1 + 1 = 3 → S = 1, C_{out} = 1

3번 자리: 0 + 0 + 1 = 1 → S = 1, C_{out} = 0

결과 **1101_2**. 10진수로 6 + 7 = 13이고 1101_2 = 8 + 4 + 1 = 13이므로 맞다.

ALU(산술 논리 장치, Arithmetic Logic Unit)는 덧셈·뺄셈 같은 산술 연산과 AND·OR 같은 논리 연산을 하는 조합 회로다. 안을 들여다보면 AND 회로, OR 회로, 덧셈기가 모두 동시에 계산하고, 마지막에 MUX가 **ALU 제어 신호**에 따라 그중 하나의 결과를 고른다. 그래서 "ALU =

여러 연산 회로 + 결과를 고르는 MUX"로 생각하면 된다. RISC-V ALU는 32비트 입력 2개, 32비트 결과, 결과가 0이면 1이 되는 Zero 출력, 4비트 ALU 동작 제어 입력을 가진다. [Ch4 p.12] 뺄셈은 2의 보수를 이용해 덧셈기로 한다(Ch2 · 4).

7. 순차 회로: 플립플롭과 레지스터

출력이 지금 입력뿐 아니라 **기억해 둔 값(상태)**에도 달린 회로를 순차 회로(sequential circuit)라 한다. Ch4의 상태 소자(state element)가 이것이다. [Ch4 p.9]

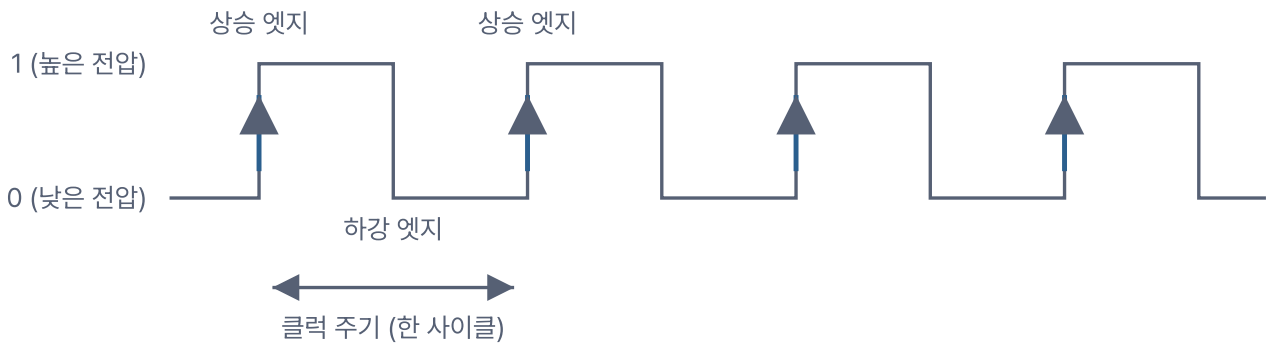
플립플롭(flip-flop) 1비트를 기억하는 가장 작은 상태 소자. D 플립플롭은 입력 D(데이터), Clk(클럭), 출력 Q를 가진다. 클럭 엣지 순간의 D 값을 저장해 다음 엣지까지 Q로 내보낸다.

레지스터(register) 플립플롭 여러 개를 묶어 여러 비트를 기억하는 장치. 32비트 레지스터 = 플립플롭 32개. PC도 레지스터다.

쓰기 제어(Write) Write 입력이 있는 레지스터는 Write = 1이고 클럭 엣지가 올 때만 값을 바꾼다. Write = 0이면 엣지가 와도 값을 유지한다. [Ch4 p.9]

8. 클럭 파형과 엣지

클럭(clock)은 0과 1을 일정한 간격으로 되풀이하는 신호다. 한 번 되풀이하는 시간이 클럭 주기, 1초에 되풀이하는 횟수가 클럭 속도다(b5). 0에서 1로 올라가는 순간을 **상승 엣지(rising edge)**, 1에서 0으로 내려가는 순간을 **하강 엣지(falling edge)**라 한다. [Ch1 p.26]



클럭 파형. 상승 엣지에서 다음 상승 엣지까지가 한 클럭 주기다. 레지스터는 엣지(이 노트는 상승 엣지로 그린다)에서만 값을 바꾼다.

Ch4는 **엣지 트리거 클로킹(edge-triggered clocking)**을 쓴다. 상태 소자의 값은 클럭 엣지에서만 바뀐다. 엣지와 엣지 사이에는 조합 회로가 다음에 저장할 값을 계산한다. 그래서 가장 느린 조합 경로의 지연이 클럭 주기보다 짧아야 한다. [Ch4 p.10]

예제

Write = 1인 레지스터 R의 값이 처음에 7이다. 입력 D가 아래처럼 바뀔 때 각 시점의 출력 Q를 쓰시오. (1) 첫 상승 엣지 직전, D = 3 (2) 첫 상승 엣지 직후 (3) 두 엣지 사이에 D가 9로 바뀐 직후 (4) 두 번째 상승 엣지 직후

풀이 보기

(1) Q = 7. 엣지 전이므로 아직 이전 값이다.

(2) Q = 3. 엣지 순간의 D = 3이 저장된다.

(3) Q = 3. 엣지 사이에 D가 바뀌어도 Q는 바뀌지 않는다.

(4) Q = 9. 두 번째 엣지 순간의 D = 9가 저장된다. 만약 Write = 0이었다면 끝까지 7이다.

9. 메모리와 레지스터 파일 미리보기

메모리(memory)는 **주소(address)가 붙은 칸들의 배열**이다. RISC-V에서는 한 칸이 1바이트이고 주소는 0부터 시작한다. [Ch2 p.12]

- **읽기(read)**: 주소를 주면 그 칸의 값이 나온다. 칸의 값은 그대로 남는다.
- **쓰기(write)**: 주소와 쓸 데이터를 주고 쓰기 신호를 활성하면 그 칸의 값이 바뀐다. Ch4의 데이터 메모리는 MemRead, MemWrite 신호로 읽기·쓰기를 정한다. [Ch4 p.13]

주소	0	4	8	12
값 (word)	15	−2	40	7

위 표는 4바이트(word)씩 묶어 그린 메모리다. word 하나가 4칸을 차지하므로 주소가 4씩 늘어난다. 주소 8을 읽으면 40이 나온다. 주소 4에 100을 쓰면 −2가 100으로 바뀌고 나머지는 그대로다.

레지스터 파일(register file)은 레지스터 32개(x0~x31)를 모아 번호로 고르게 만든 작은 메모리다. 번호 2개를 주면 두 레지스터 값을 동시에 읽고(읽기 포트 2개), 번호 1개와 데이터를 주고 RegWrite = 1이면 다음 클럭 엣지에 그 레지스터에 쓴다(쓰기 포트 1개). 읽는 쪽은 MUX(번호로 32개 중 하나 고르기), 쓰는 쪽은 디코더(번호로 32개 중 하나만 Write 켜기)로 생각하면 된다. [Ch4 p.12] 교재 기준

함정

OR에서 1 OR 1 = 1이다. "둘 다 1이면 0"은 XOR이다. 회로식의 A + B는 덧셈이 아닌 OR이므로 1 + 1 = 1로 읽는다.

"메모리·레지스터는 계산을 안 하니 조합 소자"는 틀렸다. 값을 저장하면 상태 소자(순차)다. 반대로 MUX와 가산기는 저장하지 않으므로 조합 소자다. [Ch4 p.9]

레지스터의 출력은 "지금 D에 들어온 값"이 아니다. 마지막 클럭 엣지에서 저장한 값이다.

MUX 선택 신호 0이 "입력 없음"을 뜻하지 않는다. 0번 입력을 고른다는 뜻이다.

스스로 확인

1. 1 AND 0, 1 OR 0, NOT 1, 1 XOR 1의 값을 쓰시오.

답 보기

0, 1, 0, 0. XOR은 두 입력이 다를 때만 1.

2. 입력이 3개인 회로의 진리표는 몇 줄인가?

답 보기

8줄. $2^3 = 8$.

3. A = 1, B = 0, C = 1일 때 Y = (A AND B) OR C와 Y = NOT(A OR B)의 값은?

답 보기

1, 0. (1 AND 0) OR 1 = 0 OR 1 = 1. A OR B = 1이므로 NOT 1 = 0.

4. 2:1 MUX에 입력 0 = 5, 입력 1 = 9가 들어온다. 선택 신호가 1일 때 출력은? 0일 때는?

답 보기

9, 5. 선택 신호 값과 같은 번호의 입력이 나간다.

5. 입력 8개 중 하나를 고르는 MUX의 선택 신호는 몇 비트인가?

답 보기

3비트. $2^3 = 8$.

6. 3비트 입력 디코더의 출력은 몇 개이고, 입력이 101_2이면 몇 번 출력이 1인가?

답 보기

8개, 5번 출력. $101_2 = 4 + 1 = 5$. 나머지 출력은 모두 0.

7. 반가산기에 A = 1, B = 1을 넣으면 S와 C는? 전가산기에 1, 1, 1을 넣으면 S와 C_{out}은?

답 보기

반가산기 S = 0, C = 1. 전가산기 S = 1, C_{out} = 1(1의 개수 3 = 2진수 11).

8. 4비트 덧셈 $0101_2 + 0011_2$을 계산하시오.

   답 보기

   1000_2. 자리올림이 0번, 1번, 2번 자리에서 연달아 생긴다. 10진수 5 + 3 = 8 = 1000_2.

9. 다음을 조합(C)과 순차(S)로 분류: MUX, 레지스터, 가산기, 데이터 메모리, 디코더, PC, ALU.

   답 보기

   MUX C, 레지스터 S, 가산기 C, 데이터 메모리 S, 디코더 C, PC S, ALU C. 값을 저장하면 순차(상태 소자)다.

10. 빈칸: ALU는 여러 연산을 동시에 계산하고 ( ) 신호에 따라 ( )가 그중 하나를 고르는 회로로 볼 수 있다.

    답 보기

    ALU 제어, MUX.

11. 클럭이 2 GHz다. 상승 엣지 사이 간격은? 1 μs 동안 상승 엣지는 몇 번 오는가?

    답 보기

    0.5 ns, 2000번. 1 μs / 0.5 ns = 10^{-6} / (0.5×10^{-9}) = 2000.

12. Write = 0인 레지스터의 값이 12이고, D = 4인 상태에서 상승 엣지가 왔다. 엣지 뒤 출력은?

    답 보기

    12. Write가 0이면 엣지가 와도 값을 유지한다. [Ch4 p.9]

13. 위 메모리 표(주소 0, 4, 8, 12에 15, −2, 40, 7)에서 주소 12를 읽은 값은? 주소 0에 3을 쓴 뒤 주소 0과 4를 읽으면?

    답 보기

    7. 쓰기 뒤 주소 0 = 3, 주소 4 = −2(바뀌지 않음).

14. 두 상태 소자 사이 조합 경로의 지연이 400 ps, 250 ps다. 가능한 최대 클럭 속도는?

    답 보기

    2.5 GHz. 가장 긴 지연 400 ps가 최소 주기이고 1 / (400×10^{-12}) = 2.5×10^{9} Hz. [Ch4 p.10]

15. O/X: 레지스터 파일은 읽기를 할 때도 클럭 엣지와 RegWrite = 1이 필요하다.

답 보기

X. 읽기는 번호만 주면 나온다. 쓰기만 RegWrite = 1과 클럭 엣지가 필요하다. [Ch4 p.12]

이 기초가 시험에서 쓰이는 곳

- Ch4 · 1 MUX와 제어 유닛이 필요한 이유 [Ch4 p.7]
- Ch4 · 2 조합 소자와 상태 소자 분류, 엣지 트리거 클로킹, 가장 긴 지연과 클럭 주기 [Ch4 p.9–10]
- Ch4 · 3 PC·가산기·레지스터 파일·ALU(Zero)·데이터 메모리 [Ch4 p.11–15]
- Ch4 · 4 데이터패스의 MUX 세 곳과 선택 신호 0·1의 의미 [Ch4 p.16–17]

기초 · 7 이 기초가 쓰이는 곳: Ch2 · 2, 3, 7~10, 실전 · 2

C 코드와 어셈블리 읽기 기초

이 단원을 마치면 변수·배열·if·while·for·함수로 된 짧은 C 코드의 실행 결과를 손으로 구하고, RISC-V 어셈블리 한 줄의 뜻(목적지, 출처, 즉시값, 괄호 주소, 분기)을 말로 쓸 수 있다. PDF 외 보충

쉽게 말하면

C 코드는 사람이 읽기 좋게 쓴 요리법이고, 어셈블리는 그 요리법을 "냄비 1에 물을 붓는다", "냄비 1과 냄비 2를 섞어 냄비 3에 담는다"처럼 아주 작은 동작으로 쪼갠 지시서다. 어셈블리 한 줄은 CPU가 한 번에 하는 동작 하나다.

읽는 요령은 한 가지다. 줄마다 "어느 칸의 값이 어떻게 바뀌는가"를 표에 적으며 위에서 아래로 따라간다.

비유가 안 맞는 지점 요리 지시서는 사람이 알아서 순서를 바꾸지만 CPU는 적힌 순서대로만 한다. 순서를 바꾸는 것은 분기 명령어뿐이다.

1. 변수와 대입

변수(variable)는 값을 담는 이름 붙은 칸이다. `int x;`는 정수(integer)를 담는 칸 x를 만든다는 뜻이다. C에서 **int는 4바이트(32비트)** 정수다. RISC-V의 word(32비트)와 크기가 같다. [Ch2 p.10]

`=`는 "같다"가 아닌 **대입(assignment)**이다. 오른쪽을 먼저 계산하고 그 값을 왼쪽 칸에 넣는다. 그래서 `x = x + 1;`은 "x의 지금 값에 1을 더해 다시 x에 넣어라"로 읽는다. 문장 끝의 `;`는 문장이 끝났다는 표시다.

```
int x = 10;     // x 칸을 만들고 10을 넣는다
x = x + 3;      // x = 13
x = x * 2;      // x = 26
```

`//` 뒤는 주석(comment)이다. 사람에게 설명하는 글이고 실행되지 않는다. C와 RISC-V 어셈블리 모두 `//`를 쓴다. `x += 1;`은 `x = x + 1;`, `i++`는 i에 1을 더하라는 줄임말이다.

2. 산술과 비교

연산자	뜻	예 (a = 17, b = 5)
`+` `-` `*`	더하기, 빼기, 곱하기	22, 12, 85
`/`	정수 나눗셈의 몫(소수점 아래 버림)	17 / 5 = 3
`%`	나머지	17 % 5 = 2
`==` `!=`	같다, 다르다	a == b → 0, a != b → 1
`<` `<=` `>` `>=`	작다, 작거나 같다, 크다, 크거나 같다	a >= b → 1

C에서 비교의 결과는 참이면 1, 거짓이면 0인 정수다. `=`(대입)과 `==`(같은지 비교)는 다른 기호다. 정수끼리 나누면 몫만 남는다. 17 / 5는 3.4가 아닌 3이다.

3. 배열

배열(array)은 같은 종류의 칸을 메모리에 **연속으로** 붙여 놓고 번호로 부르는 것이다. `int A[5];`는 int 칸 5개, A[0]부터 A[4]까지다. 번호(인덱스, index)는 0부터 시작하므로 마지막 칸은 A[4]다.

int 하나가 4바이트이고 메모리 주소는 바이트마다 붙으므로, 다음 원소는 주소가 4 큰 곳에 있다. [Ch2 p.12]

```
A[i]의 주소 = A의 시작 주소 + i × 4
```

원소	A[0]	A[1]	A[2]	A[3]	A[4]
주소 (시작 1000)	1000	1004	1008	1012	1016

4. if/else, while, for

```
if (x >= 5)        // 조건이 참(1)이면
    y = 1;         //   이 줄을 실행
else               // 거짓(0)이면
    y = 2;         //   이 줄을 실행
```

```
int n = 1;
while (n < 20)     // 조건이 참인 동안 반복
    n = n * 3;     // n: 1 → 3 → 9 → 27, 27 < 20은 거짓이라 멈춤
```

```
int s = 0;
for (int i = 0; i < 4; i++)   // 초기; 조건; 증가
    s = s + i;                // s = 0 + 0 + 1 + 2 + 3 = 6
```

for의 괄호 안은 세 부분이다. ① 초기(맨 처음 한 번 실행) ② 조건(매 반복 전에 검사, 거짓이면 끝) ③ 증가(몸체를 한 번 실행한 뒤 실행). 위 for는 다음 while과 같다.

```
int i = 0;           // ① 초기
while (i < 4) {      // ② 조건
    s = s + i;       // 몸체
    i++;             // ③ 증가
}
```

중괄호 `{ }`는 여러 문장을 하나로 묶는다. 반복문을 따라갈 때는 아래처럼 표를 그려 한 줄씩 채우면 틀리지 않는다.

회차	검사할 때 i	i < 4 ?	몸체 뒤 s
1	0	참	0
2	1	참	1
3	2	참	3
4	3	참	6
5	4	거짓 → 끝	6

5. 함수

함수(function)는 이름을 붙여 따로 떼어 둔 코드 묶음이다. 어셈블리 쪽 말로는 프로시저(procedure)다. [Ch2 p.32]

```
int sq(int v)        // 이름 sq, 인자 v (int), 돌려주는 값도 int
{
    return v * v;    // 결과를 돌려주고 함수를 끝낸다
}

int r = sq(3) + sq(4);   // 호출: sq(3) = 9, sq(4) = 16, r = 25
```

인자(argument) 함수를 부를 때 넘기는 값. 위에서 3과 4. 함수 안에서는 v라는 이름으로 받는다.

반환값(return value) `return` 뒤의 값. 호출한 자리가 이 값으로 바뀐다고 생각하면 된다.

호출(call) 함수 이름과 괄호로 함수를 실행시키는 것. 끝나면 호출한 줄로 돌아와 이어서 실행한다.

RISC-V에서는 인자를 x10~x17에 넣어 넘기고, 결과를 x10에 놓아 돌려주며, 돌아올 주소를 x1에 둔다(Ch2 · 8). [Ch2 p.34]

6. 레지스터: CPU 안의 작은 변수 32개

CPU는 메모리의 변수로 직접 계산하지 않는다. CPU 안에 있는 32개의 작은 칸, 레지스터(register) x0~x31에 값을 옮겨 놓고 계산한다. 각 레지스터는 32비트다. x0은 언제나 0이다. 컴파일러는 C 변수 f, g, h를 x19, x20, x21 같은 레지스터에 배정한다. 시험 문제는 보통 "f는 x19에 있다"처럼 대응을 알려 준다. [Ch2 p.10, 16]

7. 어셈블리 한 줄 읽는 법

RISC-V 산술 명령어는 **연산 목적지, 출처1, 출처2** 형식이다. 맨 앞이 무엇을 할지(연산), 쉼표 뒤 첫 칸이 결과를 쓸 곳(목적지, destination), 나머지 둘이 읽을 값(출처, source)이다. [Ch2 p.9]

```
add  x5, x6, x7      // x5 ← x6 + x7
sub  x5, x6, x7      // x5 ← x6 - x7  (순서 중요: 출처1 - 출처2)
addi x5, x6, 10      // x5 ← x6 + 10  (10은 즉시값)
addi x5, x0, 7       // x5 ← 0 + 7 = 7 (상수 넣기)
```

화살표 ←는 "오른쪽 값을 계산해 왼쪽에 넣는다"는 뜻이다. C의 `=`와 같다. `add x5, x6, x7`은 C로 `x5 = x6 + x7;`이다.

즉시값(immediate) 명령어 안에 직접 적힌 상수. `addi`의 i가 immediate다. 빼기용 subi는 없고 음수를 더한다: `addi x5, x5, -3`. [Ch2 p.16]

메모리 읽기·쓰기의 괄호. `lw x9, 32(x22)`는 "x22에 든 값에 32를 더한 주소에서 word를 읽어 x9에 넣어라"이다. 괄호 안 레지스터가 시작 주소(베이스), 괄호 앞 숫자가 그 주소에서 떨어진 바이트 수(오프셋)다. [Ch2 p.14]

```
lw  x9, 32(x22)      // x9 ← 메모리[x22 + 32]   (load word)
sw  x9, 48(x22)      // 메모리[x22 + 48] ← x9   (store word, 방향이 반대)
```

x22가 배열 A의 시작 주소라면 32 / 4 = 8이므로 `lw x9, 32(x22)`는 `x9 = A[8];`, 48 / 4 = 12이므로 `sw x9, 48(x22)`는 `A[12] = x9;`다. sw는 맨 앞 레지스터가 목적지가 아닌 **저장할 값**이라는 점이 다른 명령어와 다르다.

레이블과 분기. 어셈블리는 위에서 아래로 한 줄씩 실행된다. 줄 앞에 `Loop:`처럼 이름(레이블, label)을 붙여 두면, 분기 명령어가 조건이 맞을 때 그 줄로 건너뛴다. 조건이 맞지 않으면 바로 다음 줄로 간다. [Ch2 p.28]

```
beq x5, x6, L1       // x5 == x6 이면 L1로 점프, 아니면 다음 줄
bne x5, x6, L1       // x5 != x6 이면 L1로 점프
blt x5, x6, L1       // x5 < x6 이면 L1로 점프
bge x5, x6, L1       // x5 >= x6 이면 L1로 점프
beq x0, x0, L1       // 0 == 0 은 항상 참: 무조건 L1로
```

8. C ↔ 어셈블리 아주 작은 예

변수 대응: a = x19, b = x20, c = x21, d = x22. 중간값은 x5.

C	RISC-V
`a = b + c;`	`add x19, x20, x21`
`a = b - 1;`	`addi x19, x20, -1`
`a = 0;`	`addi x19, x0, 0`
`a = b + c - d;`	`add x5, x20, x21` 다음 `sub x19, x5, x22`
`a = A[3];` (A 시작 주소 x23)	`lw x19, 12(x23)`

한 명령어는 연산 하나만 하므로 C 한 줄이 어셈블리 여러 줄이 될 수 있다. 출처가 둘뿐이라 세 값을 더하려면 두 줄이 필요하다. [Ch2 p.9]

예제 (어셈블리 실행 따라가기)

다음 코드를 실행한 뒤 x5의 값은? 이 코드를 C로 쓰면?

```
      addi x5, x0, 0       // x5 = 0
      addi x6, x0, 3       // x6 = 3
Loop: beq  x6, x0, Exit    // x6 == 0 이면 Exit로
      add  x5, x5, x6      // x5 = x5 + x6
      addi x6, x6, -1      // x6 = x6 - 1
      beq  x0, x0, Loop    // 무조건 Loop로
Exit:
```

풀이 보기

표를 그려 Loop 줄에 올 때마다 기록한다.

Loop 도착	x6	x6 == 0 ?	add 뒤 x5	addi 뒤 x6
1번째	3	아니오	3	2
2번째	2	아니오	5	1
3번째	1	아니오	6	0
4번째	0	예 → Exit	6	0

x5 = **6**. C로는 `s = 0; n = 3; while (n != 0) { s = s + n; n = n - 1; }`이다. 분기 조건 `x6 == 0`은 "끝내는 조건"이므로 C의 반복 조건은 뒤집은 `n != 0`이다. [Ch2 p.30]

함정

A[3]의 오프셋은 3이 아닌 12다. 오프셋은 바이트 단위라서 인덱스 × 4를 쓴다. [Ch2 p.14]

`sub x5, x6, x7`은 x6 − x7이다. 출처 순서를 바꾸면 부호가 바뀐다.

sw는 맨 앞 레지스터가 읽히는 값이다. `sw x9, 12(x22)`에서 x9는 바뀌지 않고 메모리가 바뀐다.

for의 증가 부분은 몸체 **뒤에** 실행된다. 첫 회에는 초기값 그대로 몸체가 실행된다.

정수 나눗셈은 몫만 남긴다. 7 / 2 = 3, 7 % 2 = 1.

스스로 확인

1. `int x = 10; x = x + 3; x = x * 2;` 실행 뒤 x는?

   답 보기

   26. 10 + 3 = 13, 13 × 2 = 26.

2. C에서 `17 / 5`와 `17 % 5`의 값은?

답 보기

3, 2. 17 = 5 × 3 + 2.

---

3. a = 4, b = 9일 때 `a < b`, `a == b`, `a != b`의 값은?

답 보기

1, 0, 1. 참은 1, 거짓은 0.

---

4. `int s = 0; for (int i = 1; i <= 5; i++) s = s + i;` 실행 뒤 s는?

답 보기

15. 1 + 2 + 3 + 4 + 5. 조건이 <=이므로 5도 포함.

---

5. `int n = 2; while (n < 50) n = n * 3;` 실행 뒤 n은? 몸체는 몇 번 실행되는가?

답 보기

54, 3번. 2 → 6 → 18 → 54, 54 < 50이 거짓이라 멈춤.

---

6. `int y; int x = 4; if (x >= 5) y = 1; else y = 2;` 실행 뒤 y는?

답 보기

2. 4 >= 5가 거짓이므로 else.

---

7. int 배열 A의 시작 주소가 2000이다. A[6]의 주소는? 주소 2036은 몇 번 원소인가?

답 보기

2024, A[9]. 2000 + 6 × 4 = 2024. (2036 − 2000) / 4 = 9.

---

8. `int A[5] = {3, 1, 4, 1, 5}; int t = A[2] + A[4];`에서 t는?

답 보기

9. A[2] = 4, A[4] = 5. 인덱스는 0부터 센다.

---

9. `int f(int a, int b) { return a * 2 + b; }` 일 때 `f(3, f(1, 1))` 의 값은?

답 보기

9. 안쪽 f(1, 1) = 2 + 1 = 3, 바깥 f(3, 3) = 6 + 3 = 9.

---

10. `add x5, x6, x7` 과 `addi x5, x5, -3` 의 뜻을 C 문장 꼴로 쓰시오.

답 보기

x5 = x6 + x7; / x5 = x5 − 3;. 첫 칸이 목적지, −3은 즉시값.

---

11. x5 = 8, x6 = 3일 때 `sub x7, x5, x6` 과 `sub x7, x6, x5` 의 x7은?

답 보기

5, −5. 출처1 − 출처2.

---

12. x22 = 1000일 때 `lw x9, 32(x22)` 는 몇 번지를 읽는가? x22가 int 배열 A의 시작 주소라면 C로 무엇인가?

답 보기

1032번지, x9 = A[8];. 32 / 4 = 8. [Ch2 p.14]

---

13. `sw x9, 12(x22)` 를 C로 쓰시오(x22 = A의 시작 주소). 이 명령어 뒤 x9는 바뀌는가?

답 보기

A[3] = x9;. x9는 바뀌지 않는다. 메모리 A[3]이 바뀐다.

---

14. x5 = 4, x6 = 4일 때 `bne x5, x6, L` 다음에 실행되는 줄은? `beq x5, x6, L` 이라면?

답 보기

bne: 바로 다음 줄(같으므로 분기 안 함). beq: 레이블 L이 붙은 줄.

---

15. a = x19, b = x20, c = x21, d = x22일 때 `a = b + c - d;` 를 RISC-V로 쓰시오(중간값 x5).

답 보기

`add x5, x20, x21`, `sub x19, x5, x22`. 출처가 둘뿐이라 두 줄.

---

16. 다음 실행 뒤 x5는? `addi x5, x0, 1` / `addi x6, x0, 4` / `Loop: beq x6, x0, Exit` / `add x5, x5, x5` / `addi x6, x6, -1` / `beq x0, x0, Loop` / `Exit:`

    답 보기

    16. x5를 4번 두 배 한다: 1 → 2 → 4 → 8 → 16. x6이 4, 3, 2, 1일 때 몸체 실행, 0이면 Exit.

이 기초가 시험에서 쓰이는 곳

- Ch2 · 2 3피연산자 규칙, 즉시값, x0 [Ch2 p.9–10, 16]
- Ch2 · 3 배열 원소의 lw/sw 오프셋(인덱스 × 4) [Ch2 p.12–14]
- Ch2 · 7 if-else·while의 분기 번역(조건 뒤집기, 레이블) [Ch2 p.28–31]
- Ch2 · 8~10 함수 호출의 인자 x10~x17, 반환값 x10, 복귀 주소 x1 [Ch2 p.32–34]
- 실전 · 2 C ↔ 어셈블리 변환, 실전 · 3 실행 결과 추적

Ch1 · 1 슬라이드 p.2–8

컴퓨터의 종류와 이 과목의 목표

이 단원을 마치면 PC·서버·임베디드 컴퓨터를 특징으로 구별하고, PMD·클라우드(WSC)·SaaS·ECU가 무엇인지 한 줄로 말할 수 있다. 이 과목이 무엇을 배우는지도 정리한다.

쉽게 말하면

컴퓨터는 쓰임새에 따라 모양과 우선순위가 다르다. 집에서 쓰는 노트북은 "이것저것 다 하는 만능 도구", 회사 서버실의 서버는 "여러 사람이 동시에 쓰는 큰 공용 시설", 세탁기나 자동차 안의 칩은 "한 가지 일만 하도록 붙박이로 넣은 부품"에 가깝다.

비유가 안 맞는 지점 셋의 경계가 딱 나뉘지는 않는다. 스마트폰은 다양한 앱을 돌리지만 슬라이드는 임베디드의 예로도 들고, Post-PC 시대의 대표인 PMD로도 다룬다. 분류는 "주된 특징"으로 기억하면 된다.

컴퓨터 혁명

컴퓨터의 발전은 계속되고 있고, 사회의 거의 모든 분야에 영향을 준다. 슬라이드가 든 예는 자동차 속 컴퓨터, 스마트폰, 인간 게놈 프로젝트, 월드 와이드 웹(WWW), 검색 엔진이다. [Ch1 p.2]

컴퓨터의 세 가지 분류(Classes of Computers)

개인용 컴퓨터(PC, Personal Computer) 범용(general purpose)으로 다양한 소프트웨어를 돌린다. 비용과 성능 사이의 균형(trade-off)을 맞추는 것이 중요하다. 예: 데스크톱, 노트북. [Ch1 p.3]

서버(Server) 보통 네트워크를 통해 접근한다. 대용량·고성능·고신뢰성(reliability)이 요구된다. 작은 서버부터 건물 하나 크기까지 범위가 넓다. 저가형은 소규모 사업이나 웹 서비스용, 고가형은 슈퍼컴퓨터나 데이터센터로 수백~수천 개 프로세서, 테라바이트(TB) 단위 메모리, 페타바이트(PB) 단위 저장장치를 갖는다. [Ch1 p.3]

임베디드 컴퓨터(Embedded computer) 다른 기기 안에 들어가서 미리 정해진 응용(predetermined applications)을 실행하는 컴퓨터. 전력·성능·비용 제약이 엄격하다(stringent). 슬라이드 예: 스마트폰, 태블릿. [Ch1 p.3]

분류	핵심 특징	예
PC	범용, 다양한 SW, 비용·성능 균형	데스크톱, 노트북
서버	네트워크로 접근, 대용량·고성능·고신뢰	웹 서버, 슈퍼컴퓨터, 데이터센터
임베디드	다른 기기 안, 정해진 응용, 전력·성능·비용 제약 엄격	스마트폰, 태블릿, 자동차 ECU

Post-PC 시대

PC의 자리를 **개인 휴대 기기(PMD, Personal Mobile Device)**가 대신하고 있다. PMD는 배터리로 동작하고 인터넷에 무선으로 연결된다. 보통 수백 달러이고 키보드·마우스가 없다. 예: 스마트폰, 태블릿, 전자 안경. [Ch1 p.4]

기존 서버의 자리는 **클라우드 컴퓨팅(cloud computing)**이 넘겨받고 있다. 클라우드를 이루는 거대한 시설을 **창고 규모 컴퓨터(WSC, Warehouse Scale Computer)**라고 한다. **서비스형 소프트웨어(SaaS, Software as a Service)**는 클라우드를 통해 배포되는

소프트웨어로, 사용자가 프로그램을 설치하지 않고 인터넷으로 서비스를 받는다. 예: Amazon, Google. [Ch1 p.4] 슬라이드 p.5는 다양한 Post-PC 기기 사진이다. [Ch1 p.5]

자동차 전장과 ECU

자동차 안에는 여러 전자 시스템이 있다. 슬라이드 그림의 이름표: 첨단 운전자 보조 시스템(ADAS, Advanced Driver Assistance System), 파워트레인 관리(Powertrain Management, 엔진·변속기 쪽), 인포테인먼트 시스템(Infotainment System, 내비게이션·음악 등), 차체 제어기(Body Controllers, 문·창문·조명 등). [Ch1 p.6]

전자 제어 장치(ECU, Electronic Control Unit)는 자동차 전장에 들어가는 임베디드 시스템으로, 차량의 전기 시스템 하나 이상을 제어한다. 슬라이드는 자동차 전장을 소프트웨어로 다루려면 먼저 컴퓨터 구조를 배워야 한다고 강조한다. 그림에는 AUTOSAR 플랫폼 구조와 Xilinx Zynq UltraScale+ MPSoC 칩이 나온다. [Ch1 p.7]

이 과목에서 배우는 것

1. 프로그램이 어떻게 기계어로 번역되는가, 그리고 하드웨어가 그것을 어떻게 실행하는가
2. 하드웨어와 소프트웨어의 접점(interface), 즉 **명령어 집합 구조(ISA, Instruction Set Architecture)**
3. 무엇이 프로그램 성능을 결정하는가, 그리고 어떻게 개선하는가
4. 하드웨어 설계자는 성능을 어떻게 높이는가
5. 프로그램 성능 이해하기

[Ch1 p.8] 이 목록이 Ch2(명령어, ISA), Ch3(산술), Ch4(프로세서) 순서와 이어진다. PDF 외 보충

함정

임베디드 컴퓨터는 "성능이 낮은 컴퓨터"라는 뜻이 아니다. 정의의 핵심은 **다른 기기 안에 들어가 정해진 응용을 실행**한다는 점과 **전력·성능·비용 제약이 엄격**하다는 점이다. 또 서버는 "네트워크를 통해 접근"한다는 특징을 빠뜨리기 쉽다.

시험 포인트

- PC·서버·임베디드의 특징 짝짓기
- PMD의 특징(배터리, 무선 인터넷, 키보드·마우스 없음), 클라우드 = WSC, SaaS의 뜻
- ECU 정의: 자동차 전장의 임베디드 시스템, 차량 전기 시스템 하나 이상 제어

스스로 확인

1. "다른 기기 안에 들어가 미리 정해진 응용을 실행하며 전력·성능·비용 제약이 엄격한 컴퓨터"를 무엇이라 하는가?

   답 보기

   임베디드 컴퓨터(Embedded computer). [Ch1 p.3]

2. 보통 네트워크를 통해 접근하고 대용량·고성능·고신뢰성을 갖는 컴퓨터 분류는?

답 보기

서버. 저가형은 소규모 사업·웹 서비스, 고가형은 슈퍼컴퓨터·데이터센터.

3. Post-PC 시대에 PC를 대체하는 것과 기존 서버를 대체하는 것을 각각 쓰시오.

답 보기

PC → 개인 휴대 기기(PMD), 서버 → 클라우드 컴퓨팅(창고 규모 컴퓨터, WSC). [Ch1 p.4]

4. SaaS의 영어 전체 이름과 뜻을 쓰시오.

답 보기

Software as a Service. 클라우드 컴퓨팅을 통해 배포되는 소프트웨어 서비스.

5. O/X: PMD는 배터리로 동작하고 무선으로 인터넷에 연결되며, 보통 키보드와 마우스가 없다.

답 보기

O. 슬라이드의 PMD 특징 그대로다. [Ch1 p.4]

6. ECU의 영어 전체 이름은? 무엇을 하는가?

답 보기

Electronic Control Unit. 자동차 전장에 들어간 임베디드 시스템으로, 차량의 전기 시스템 하나 이상을 제어한다. [Ch1 p.7]

Ch1 · 2 슬라이드 p.9–11, 17

컴퓨터 구조와 추상화

이 단원을 마치면 "컴퓨터 구조 = ISA + 조직"을 설명하고, 추상화의 뜻과 ISA·ABI·구현의 정의를 한 줄로 쓸 수 있다.

쉽게 말하면

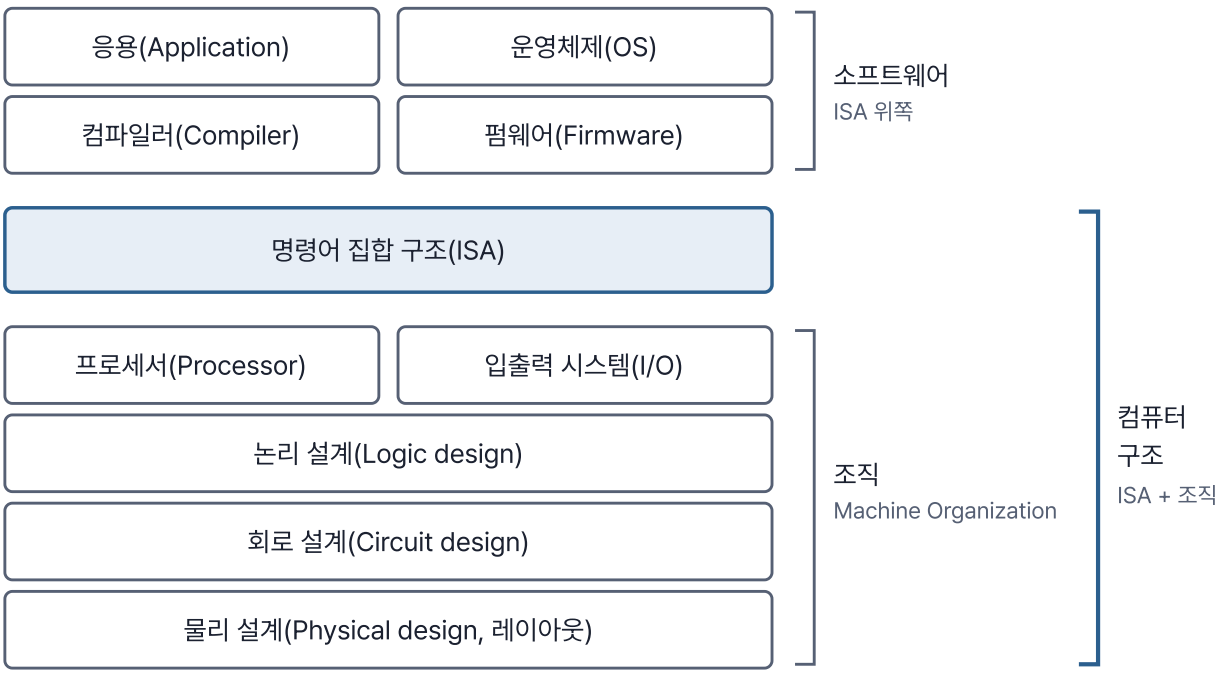
운전자는 핸들과 페달만 알면 차를 몬다. 엔진 안에서 연료가 어떻게 타는지, 브레이크 유압이 어떻게 전달되는지는 몰라도 된다. 핸들·페달이 "추상화 계층"이고, 엔진·브레이크 장치가 "기계의 세부"다. 소프트웨어 입장에서 ISA는 이 핸들·페달에 해당한다. 프로그램은 ISA가 정한 명령어만 알면 되고, 그 아래 회로가 어떻게 생겼는지는 몰라도 된다.

비유가 안 맞는 지점 자동차 운전자는 엔진을 전혀 몰라도 되지만, 컴퓨터에서는 아래 계층을 알면 더 빠른 프로그램을 쓸 수 있다. 성능을 이해하려면 추상화 아래를 들여다봐야 하고, 이 과목이 그 일을 한다.

컴퓨터 구조의 정의

```
컴퓨터 구조(Computer Architecture) = ISA + 조직(Machine Organization)
```

명령어 집합 구조(ISA, Instruction Set Architecture)는 소프트웨어가 보는 하드웨어의 모습이다. 어떤 명령어가 있고, 레지스터가 몇 개이고, 메모리를 어떻게 가리키는지 같은 약속이다. **조직(Machine Organization)**은 그 약속을 실제 하드웨어로 어떻게 만드는지에 해당한다. 프로세서와 입출력 시스템의 구성, 논리 설계, 회로 설계, 물리 설계(레이아웃)가 여기에 들어간다. [Ch1 p.9]



슬라이드 p.9 그림을 다시 그린 것. 조직(Machine Organization) 괄호는 프로세서·입출력 시스템, 논리 설계, 회로 설계, 물리 설계(레이아웃)를 덮고, 컴퓨터 구조(Computer Architecture) 괄호는 ISA부터 그 아래 전부를 덮는다. ISA 위에는 응용, OS, 컴파일러, 펌웨어가 있다. [Ch1 p.9]

추상화(Abstraction)

추상화는 아래 계층의 세부를 감추고 위 계층에는 간단한 사용법만 보여 주는 것이다. 복잡한 시스템을 다룰 수 있게 해 준다. [Ch1 p.17]

슬라이드의 비유는 두 가지다. [Ch1 p.10]

- 운전자(Driver) → 추상화 계층(핸들·페달) → 기계 세부(자동차의 연소 엔진, 브레이크 장치)
- 손님(Customer) → 추상화 계층(버튼·투입구) → 기계 세부(자판기 안의 하드웨어 보드)

스마트폰도 계층으로 나뉜다. 위에서부터 **API를 쓰는 응용 프로그래밍 → 운영체제 → ISA → 하드웨어 구현(ARM)**. 앱 개발자는 API만 알면 되고, 운영체제는 ISA만 알면 된다. [Ch1 p.11] API(Application Programming Interface)는 다른 프로그램(여기서는 운영체제)이 제공하는 기능을 부르는 약속된 방법이다. PDF 외 보충

ISA, ABI, 구현

명령어 집합 구조(ISA)	하드웨어와 가장 낮은 수준의 소프트웨어 사이의 추상 인터페이스. 간단히 **아키텍처(architecture)**라고도 부른다. [Ch1 p.17]
응용 이진 인터페이스(ABI, Application Binary Interface)	**ISA + 운영체제 인터페이스**. 응용 프로그램이 기계어 수준에서 지켜야 하는 약속 전체로, ISA 중 사용자 프로그램이 쓰는 부분과 운영체제 호출 방식을 합친 것이다. [Ch1 p.17]
구현(Implementation)	아키텍처라는 추상화를 따르는(obey) 실제 하드웨어. [Ch1 p.17]

같은 ISA에 구현은 여러 개일 수 있다. 예를 들어 같은 ARM ISA를 따르는 칩이 여러 회사에서 나오고, 같은 프로그램이 그 칩들에서 모두 돈다. 구현이 달라도 ISA가 같으면 소프트웨어를 다시 쓰지 않아도 된다는 것이 추상화의 이점이다. PDF 외 보충

핵심

컴퓨터 구조 = ISA + 조직. ISA = HW와 최하위 SW 사이의 추상 인터페이스(= architecture). ABI = ISA + OS 인터페이스. 구현 = 아키텍처를 따르는 하드웨어.

함정

ABI를 ISA와 같은 것으로 쓰면 틀린다. ABI는 ISA에 **운영체제 인터페이스**를 더한 것이다. 또 "구현"은 소프트웨어 구현이 아닌 **하드웨어**다.

시험 포인트

- "컴퓨터 구조 = ( ) + ( )" 빈칸
- ISA·ABI·구현의 한 줄 정의는 단답 단골
- 추상화의 목적: 아래 계층 세부를 감춰 복잡성을 다룬다

스스로 확인

1. 컴퓨터 구조 = ( ) + ( )

   답 보기

   ISA + 조직(Machine Organization). [Ch1 p.9]

2. 하드웨어와 가장 낮은 수준의 소프트웨어 사이의 추상 인터페이스를 무엇이라 하는가? 다른 이름 하나도 쓰시오.

   답 보기

   명령어 집합 구조(ISA). 간단히 아키텍처(architecture). [Ch1 p.17]

3. ABI의 영어 전체 이름과 구성을 쓰시오.

   답 보기

   Application Binary Interface = ISA + OS 인터페이스.

4. O/X: 구현(Implementation)이란 아키텍처 추상화를 따르는 소프트웨어를 말한다.

   답 보기

   X. 구현은 아키텍처를 따르는 **하드웨어**다.

5. 추상화가 하는 일을 한 문장으로 쓰시오.

   답 보기

   아래 계층의 세부를 감춰 복잡성을 다룰 수 있게 한다.

6. 스마트폰의 추상화 계층을 위에서부터 4개 쓰시오.

   답 보기

   API를 쓰는 응용 프로그래밍 → 운영체제 → ISA → 하드웨어 구현(ARM). [Ch1 p.11]

8대 위대한 아이디어

이 단원을 마치면 컴퓨터 구조의 8대 아이디어를 한국어·영어로 모두 쓰고, 각각의 뜻과 실제 예를 말할 수 있다.

쉽게 말하면

컴퓨터 설계자들이 지난 수십 년 동안 반복해서 써 온 "성공 공식" 8개다. 요리로 치면 "자주 쓰는 재료는 손 닿는 곳에(메모리 계층)", "여러 명이 동시에 썰기(병렬성)", "물 끓는 동안 채소 손질(파이프라이닝)" 같은 요령이다.

비유가 안 맞는 지점 요리 요령은 경험칙이지만 이 8개는 앞으로 배울 하드웨어 구조에 실제로 들어가 있는 설계 원리다. 각 아이디어가 뒤 단원의 어떤 내용과 이어지는지까지 연결해 두면 좋다.

8개 아이디어

슬라이드의 순서대로 정리한다. [Ch1 p.12]

#	아이디어(영어)	슬라이드 설명	실제 예
1	무어의 법칙을 고려한 설계 (Design for Moore's Law)	집적회로 자원이 18 ~ 24개월마다 2배가 된다	칩 설계에는 몇 년이 걸리므로, 시작 시점이 아닌 **완성 시점**의 기술 수준을 예상하고 설계한다 PDF 외 보충
2	추상화로 설계 단순화 (Use abstraction to simplify design)	설계를 여러 수준에서 추상화로 표현한다	ISA, 소프트웨어 계층(앞 단원)
3	자주 생기는 경우를 빠르게 (Make the common case fast)	드문 경우를 최적화하는 것보다 성능 향상이 크다	암달의 법칙(c1-10). 자주 쓰는 명령어를 빠르게 처리
4	병렬성을 통한 성능 (Performance via parallelism)	연산을 동시에 계산해 성능을 얻는다	멀티코어(c1-9), 여러 코어가 각자 일을 나눠 처리
5	파이프라이닝을 통한 성능 (Performance via pipelining)	병렬성의 특정한 한 형태	세탁: 첫 빨래가 건조기에 있는 동안 둘째 빨래를 세탁기에 넣는다 PDF 외 보충
6	예측을 통한 성능 (Performance via prediction)	기다리기보다 추측해 먼저 시작하는 것이 평균적으로 빠르다	분기 예측: 조건 결과가 나오기 전에 갈 방향을 짐작해 미리 실행 PDF 외 보충
7	메모리 계층 (Hierarchy of memories)	메모리의 크기와 속도가 충돌하는 문제를 계층으로 해결	캐시(SRAM) → 주기억장치(DRAM) → 디스크(c1-5)
8	중복을 통한 신뢰성 (Dependability via redundancy)	여분 부품을 넣어 시스템을 믿을 만하게 만든다	디스크 여러 개에 같은 데이터 저장, 서버의 이중 전원 PDF 외 보충

조금 더 풀어서

- **무어의 법칙**: 같은 크기 칩에 넣을 수 있는 트랜지스터 수가 18 ~ 24개월마다 약 2배가 된다는 관찰. 설계자는 이 성장을 미리 계산에 넣는다. [Ch1 p.12]
- **메모리 계층**: 빠른 메모리는 비싸고 작으며, 크고 싼 메모리는 느리다. 그래서 가장 빠르고 작은 메모리를 맨 위에, 크고 느린 메모리를 아래에 두어, 평균적으로 "빠르면서 큰" 메모리처럼 보이게 한다. [Ch1 p.12]
- **예측**: 추측이 틀렸을 때 되돌리는 비용이 크지 않고 추측이 꽤 자주 맞을 때 이득이다. PDF 외 보충
- **파이프라이닝**은 병렬성의 한 종류라서 4번과 5번이 따로 있다. 4번은 일반적인 동시 처리, 5번은 일을 단계로 나눠 단계마다 다른 일을 겹쳐 처리하는 방식이다. [Ch1 p.12]

외우는 요령

앞 글자로 **"무·추·자·병·파·예·계·중"**. 영어 키워드로는 **Moore, Abstraction, Common case, Parallelism, Pipelining, Prediction, Hierarchy, Redundancy**. "P"로 시작하는 성능 3형제(Parallelism, Pipelining, Prediction)는 모두 "Performance via ~" 형태다.

함정

"드문 경우를 빠르게"로 바꿔 내는 O/X에 주의. 정답은 **자주 생기는 경우(common case)**를 빠르게다. 또 8번은 "중복으로 **성능**"이 아니라 "중복으로 **신뢰성(dependability)**"이다. 무어의 법칙 주기는 슬라이드 기준 **18 ~ 24개월**이다.

시험 포인트

- 8개 나열(한국어 또는 영어), 설명과 짝짓기
- 무어의 법칙 = 집적회로 자원 18 ~ 24개월마다 2배
- 파이프라이닝 = 병렬성의 특정 형태, 예측 = 기다리기보다 추측해 시작

스스로 확인

1. 8대 아이디어를 모두 쓰시오.

   답 보기

   무어의 법칙을 고려한 설계, 추상화로 설계 단순화, 자주 생기는 경우를 빠르게, 병렬성을 통한 성능, 파이프라이닝을 통한 성능, 예측을 통한 성능, 메모리 계층, 중복을 통한 신뢰성.

2. 무어의 법칙에서 집적회로 자원은 몇 개월마다 몇 배가 되는가?

   답 보기

   18 ~ 24개월마다 2배. [Ch1 p.12]

3. "기다리기보다 추측해서 먼저 시작하는 것이 평균적으로 빠르다"에 해당하는 아이디어는?

답 보기

예측을 통한 성능(Performance via prediction).

4. "크기와 속도의 충돌"을 해결하는 아이디어는?

답 보기

메모리 계층(Hierarchy of memories).

5. O/X: 파이프라이닝은 병렬성과 무관한 별개의 기법이다.

답 보기

X. 슬라이드는 파이프라이닝을 병렬성의 특정한 형태(a particular pattern of parallelism)라고 한다.

6. Dependability via redundancy를 한국어로 옮기고 뜻을 쓰시오.

답 보기

중복을 통한 신뢰성. 여분(중복) 부품을 넣어 고장이 나도 시스템을 믿을 만하게 만든다.

Ch1 · 4 슬라이드 p.13–14

프로그램 아래의 세계

이 단원을 마치면 응용 소프트웨어·시스템 소프트웨어·하드웨어의 계층과 운영체제의 역할을 쓰고, 고급 언어·어셈블리어·기계어와 컴파일러·어셈블러의 관계를 설명할 수 있다.

쉽게 말하면

한국어로 쓴 요리책(고급 언어 프로그램)을 외국인 요리사(하드웨어)가 읽으려면 번역이 필요하다. 번역가(컴파일러)가 먼저 짧은 약어 메모(어셈블리어)로 옮기고, 그 약어를 다시 요리사만 아는 숫자 암호(기계어)로 바꾼다(어셈블러). 주방 관리인(운영체제)은 여러 요리사가 화구와 냉장고를 나눠 쓰도록 순서를 정한다.

비유가 안 맞는 지점 사람 번역은 뜻을 풀어 옮기지만, 어셈블러의 번역은 기호와 2진수가 거의 1:1로 대응하는 기계적 변환이다. 또 많은 컴파일러는 어셈블리를 거치지 않고 바로 기계어를 만들기도 한다. PDF 외 보충

소프트웨어 계층

소프트웨어는 주로 계층 구조로 짜여 있다. [Ch1 p.13]

- **응용 소프트웨어(Application software)**: 고급 언어로 작성된 프로그램. 사용자가 직접 쓰는 프로그램.
- **시스템 소프트웨어(System software)**: 하드웨어와 응용 소프트웨어 사이에 있다. 대표가 운영체제와 컴파일러.
- **하드웨어(Hardware)**: 프로세서, 메모리, 입출력(I/O) 제어기 등.

[Ch1 p.13]

운영체제(OS, Operating System)의 역할

운영체제는 사용자 프로그램과 하드웨어 사이의 인터페이스다. 하는 일은 다음 세 가지다. [Ch1 p.13]

1. 기본적인 입출력 처리(Handling basic input and output operation)
2. 저장 공간과 메모리 할당(Allocating storage and memory)
3. 자원 공유와 작업 스케줄링(Resource sharing and task scheduling): 여러 프로그램이 컴퓨터를 함께 쓰도록 순서와 몫을 정함

예: Linux, Windows, iOS, Android. [Ch1 p.13]

컴파일러와 어셈블러

컴파일러(Compiler) 고급 언어로 쓴 프로그램을 하드웨어가 실행하는 명령어로 번역하는 시스템 소프트웨어. [Ch1 p.13–14]

어셈블러(Assembler) 기호로 쓴 명령어(어셈블리어)를 2진수 버전(기계어)으로 번역한다. [Ch1 p.14]

세 가지 언어 수준

고급 언어(High-level language) 문제 영역에 가까운 추상화 수준의 언어. **생산성(productivity)**과 **이식성(portability)**을 준다. 예: C++. 이식성은 같은 프로그램을 다른 컴퓨터에서도 다시 컴파일만 하면 쓸 수 있다는 뜻이다. [Ch1 p.14]

어셈블리어(Assembly language) 기계 명령어의 **기호 표현(symbolic representation)**. 사람이 읽을 수 있게 `add` 같은 이름을 쓴다. [Ch1 p.14]

기계어(Machine language) 기계 명령어의 **하드웨어 표현**. 2진수 숫자(비트, bit)로 표현되며, 명령어와 데이터를 부호화(encoded)한 것이다. [Ch1 p.14]

예

C 문장 `a = b + c;`에서 a, b, c가 각각 레지스터 x5, x6, x7에 있다고 하자. 세 수준으로 쓰면 다음과 같다. PDF 외 보충 (형식의 자세한 뜻은 Ch2에서 배운다)

```
고급 언어 (C)      a = b + c;
        ↓ 컴파일러
어셈블리어 (RISC-V)  add  x5, x6, x7     // x5 = x6 + x7
        ↓ 어셈블러
기계어 (2진수)       0000000 00111 00110 000 00101 0110011
                    (16진수로 0x007302B3)
```

함정

컴파일러와 어셈블러의 입력·출력을 바꿔 쓰지 말 것. **컴파일러: 고급 언어 → 명령어(어셈블리 또는 기계어)**, **어셈블러: 어셈블리어(기호) → 기계어(2진)**. 또 컴파일러는 응용 소프트웨어가 아닌 **시스템 소프트웨어**다.

시험 포인트

- OS의 역할 3가지 목록
- 고급 언어의 장점 2가지: 생산성, 이식성
- 어셈블리어 = 기계 명령어의 기호 표현, 기계어 = 하드웨어 표현(비트)

스스로 확인

1. 하드웨어와 응용 소프트웨어 사이에 있는 소프트웨어를 무엇이라 하는가? 예 두 가지는?

   답 보기

   시스템 소프트웨어. 운영체제, 컴파일러. [Ch1 p.13]

2. 운영체제의 역할 세 가지를 쓰시오.

   답 보기

   기본 입출력 처리, 저장 공간·메모리 할당, 자원 공유와 작업 스케줄링.

3. 고급 언어가 주는 이점 두 가지는?

답 보기

생산성(productivity), 이식성(portability). [Ch1 p.14]

4. "기호로 쓴 명령어를 2진 버전으로 번역하는 프로그램"은?

답 보기

어셈블러(Assembler).

5. O/X: 어셈블리어는 기계 명령어의 하드웨어 표현이다.

답 보기

X. 어셈블리어는 **기호** 표현, 하드웨어 표현(2진수)은 기계어다.

6. 짝짓기: (가) 고급 언어 (나) 어셈블리어 (다) 기계어 / ① 비트로 표현 ② 문제 영역에 가까운 추상화 ③ 기호 표현

답 보기

(가)-②, (나)-③, (다)-①.

Ch1 · 5 슬라이드 p.15–19

컴퓨터의 5대 구성 요소와 메모리

이 단원을 마치면 컴퓨터의 5대 구성 요소와 그 사이의 데이터 흐름을 그리고, DRAM·SRAM·캐시, 휘발성·비휘발성 메모리, LAN·WAN과 네트워크의 장점을 구별할 수 있다.

쉽게 말하면

컴퓨터를 식당 주방에 비유하면, 재료 반입구가 입력, 서빙 창구가 출력, 냉장고가 메모리, 요리사가 데이터패스, 주방장이 제어다. 주방장은 직접 요리하지 않고 "이걸 꺼내", "이걸 볶아"처럼 신호만 보낸다. 요리사와 주방장을 합친 것이 프로세서다.

비유가 안 맞는 지점 식당에서는 재료가 반입구에서 바로 요리사에게 갈 수도 있지만, 슬라이드의 모델에서는 **입력은 메모리에 쓰고, 출력은 메모리에서 읽는다**. 그리고 냉장고(메모리)에는 재료(데이터)뿐 아니라 요리법(명령어)도 같이 들어 있다.

5대 구성 요소(five classic components)

입력(input), 출력(output), 메모리(memory), 데이터패스(datapath), 제어(control). [Ch1 p.15]

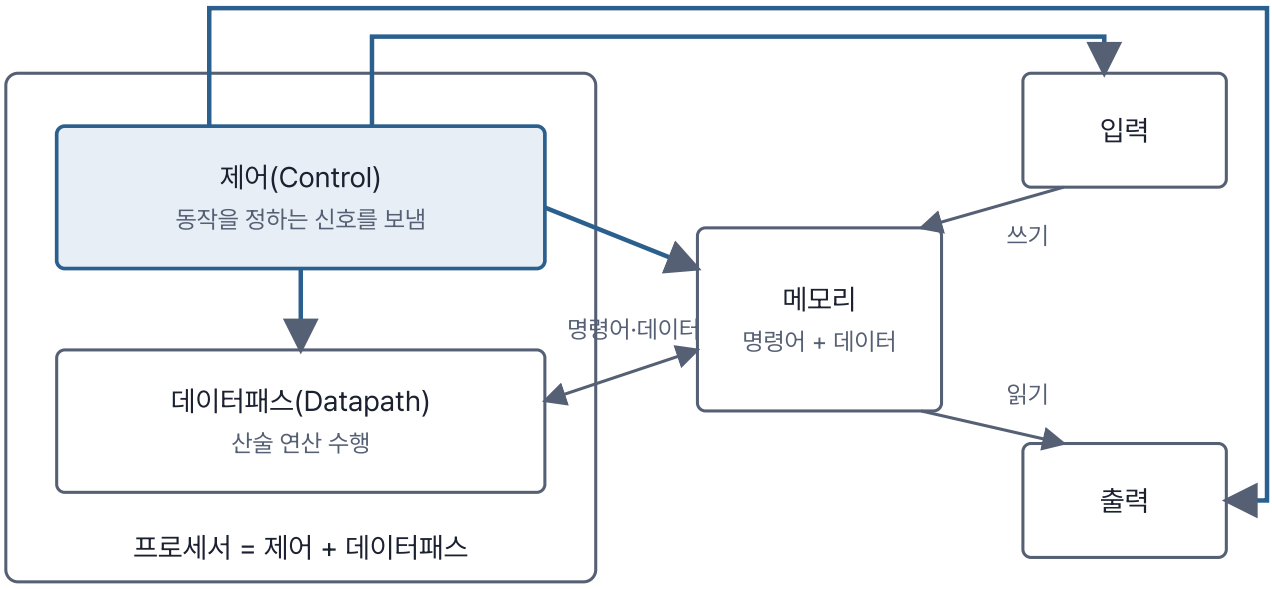
프로세서(Processor) 데이터패스 + 제어. 메모리에서 명령어와 데이터를 가져온다. CPU(Central Processing Unit, 중앙 처리 장치)라고도 한다. [Ch1 p.15–16]

데이터패스(Datapath) 산술 연산을 수행하는 부분. [Ch1 p.16]

제어(Control) 데이터패스·메모리·입력·출력의 동작을 결정하는 신호를 보낸다. 데이터패스·메모리·I/O의 순서를 정한다(sequences). [Ch1 p.15–16]

입력(Input) / 출력(Output) 입력은 데이터를 메모리에 **쓰고**, 출력은 메모리에서 데이터를 **읽는다**. [Ch1 p.15]

데스크톱, 서버, 임베디드 등 **모든 종류의 컴퓨터가 같은 구성 요소**를 갖는다. [Ch1 p.15]



강조 선 = 제어 신호(제어 → 데이터패스·메모리·입력·출력), 보통 선 = 데이터 흐름

5대 구성 요소와 흐름. 입력은 메모리에 쓰고, 출력은 메모리에서 읽고, 프로세서는 메모리에서 명령어와 데이터를 가져온다. [Ch1 p.15]

상자를 열어 보면: iPhone XS Max

- 5대 요소 중 **I/O가 대부분**을 차지한다. 정전식 멀티터치 LCD, 앞·뒤 카메라, 마이크, 스피커, Wi-Fi·블루투스 등. 데이터패스·제어·메모리는 작은 부분이다. [Ch1 p.16]
- 보드에는 집적회로(IC, Integrated Circuit), 즉 칩이 많다. **A12 프로세서**는 큰 ARM 프로세서 2개와 작은 ARM 프로세서 4개를 갖고, **2.5 GHz** 클럭으로 동작한다. [Ch1 p.16]
- 패키지에는 **32 Gb**(기가비트) 메모리 칩이 들어 있다. 32 Gb = 4 GB(기가바이트)다(1바이트 = 8비트). [Ch1 p.17]

메모리: DRAM, SRAM, 캐시

메모리(Memory) 프로그램이 실행되는 동안 프로그램과 데이터가 보관되는 저장 영역. [Ch1 p.17]

DRAM(Dynamic Random Access Memory) 메모리(주기억장치)를 만드는 칩. 프로그램의 명령어와 데이터를 담는다. [Ch1 p.17]

캐시 메모리(Cache memory) DRAM의 **버퍼** 역할을 하는 작고 빠른 메모리. [Ch1 p.17]

SRAM(Static Random Access Memory) 캐시를 만드는 메모리. DRAM보다 **빠르지만 집적도가 낮아(less dense) 더 비싸다**. [Ch1 p.17]

SRAM과 DRAM은 **메모리 계층의 두 층**이다(8대 아이디어의 "메모리 계층"). [Ch1 p.17] 집적도가 낮다는 것은 같은 면적에 담을 수 있는 비트 수가 적다는 뜻이다. 그래서 같은 용량을 만들려면 칩 면적이 더 들어 비싸진다. PDF 외 보충

데이터의 안전한 장소: 휘발성과 비휘발성

	휘발성 메모리(Volatile)	비휘발성 메모리(Nonvolatile)
전원이 꺼지면	명령어와 데이터를 **잃는다**	프로그램과 데이터를 **유지한다**
다른 이름	주기억장치(main memory), 1차 기억장치(primary memory)	2차 기억장치(secondary memory)
크기·속도	보통 작고 빠름	주기억장치보다 크고 느림
예	DRAM, SRAM	자기(하드) 디스크, 플래시 메모리, 광디스크(CD, DVD)

[Ch1 p.18]

다른 컴퓨터와 통신하기: 네트워크

네트워크는 컴퓨터 전체를 서로 연결한다. 네트워크로 연결된 컴퓨터의 주요 장점은 세 가지다. [Ch1 p.19]

1. **통신(Communication)**: 컴퓨터 사이에 정보를 빠르게 주고받는다.
2. **자원 공유(Resource sharing)**: 네트워크의 컴퓨터들이 I/O 장치를 함께 쓴다(예: 프린터 하나를 여러 PC가 공유).
3. **원격 접근(Nonlocal access)**: 사용자가 쓰는 컴퓨터 가까이에 있을 필요가 **없다(NOT)**.

근거리 통신망(LAN, Local Area Network) 지리적으로 제한된 구역, 보통 건물 하나 안에서 데이터를 전달하도록 설계된 네트워크. 예: 이더넷(Ethernet). [Ch1 p.19]

광역 통신망(WAN, Wide Area Network) 수백 km에 걸쳐 대륙까지 이를 수 있는 네트워크. 예: 인터넷. [Ch1 p.19]

무선 네트워크 Wi-Fi, 블루투스. [Ch1 p.19]

함정

- 입력은 메모리에서 "읽는" 것이 아니라 메모리에 **쓴다**. 출력은 메모리에서 **읽는다**. 방향을 바꿔 내는 O/X가 나오기 쉽다.
- SRAM이 DRAM보다 빠르다. 그러나 집적도는 **낮고** 값은 **비싸다**. "SRAM이 더 싸다"는 X.
- 캐시는 SRAM, 주기억장치는 DRAM. 둘 다 **휘발성**이다.
- 프로세서 = 데이터패스 + 제어. 메모리는 프로세서에 포함되지 않는다.

시험 포인트

- 5대 구성 요소 나열, 프로세서 = 데이터패스 + 제어
- 입력·출력과 메모리의 관계(쓰기/읽기), 제어가 신호를 보내는 대상
- DRAM vs SRAM 비교, 휘발성 vs 비휘발성의 예
- 네트워크 장점 3가지, LAN·WAN 정의와 예

스스로 확인

1. 컴퓨터의 5대 구성 요소를 쓰고, 프로세서는 그중 무엇으로 이루어지는지 쓰시오.

   답 보기

   입력, 출력, 메모리, 데이터패스, 제어. 프로세서 = 데이터패스 + 제어. [Ch1 p.15]

2. 빈칸: 입력은 데이터를 메모리에 ( ), 출력은 메모리에서 데이터를 ( ).

   답 보기

   쓰고(writes), 읽는다(reads).

3. 데이터패스·메모리·입력·출력의 동작을 결정하는 신호를 보내는 구성 요소는?

   답 보기

   제어(Control).

4. 캐시를 만드는 메모리와, 그것이 DRAM보다 비싼 이유를 쓰시오.

   답 보기

   SRAM. 더 빠르지만 집적도가 낮아(less dense) 비싸다. [Ch1 p.17]

5. O/X: 플래시 메모리는 휘발성 메모리이다.

답 보기

X. 플래시는 전원이 꺼져도 데이터를 유지하는 비휘발성(2차 기억장치)이다.

---

6. 네트워크로 연결된 컴퓨터의 장점 세 가지를 쓰시오.

답 보기

통신, 자원 공유(I/O 장치), 원격 접근(가까이 있을 필요 없음). [Ch1 p.19]

---

7. 짝짓기: (가) LAN (나) WAN / ① 인터넷, 수백 km~대륙 ② 이더넷, 건물 하나

답 보기

(가)-②, (나)-①.

---

8. iPhone XS Max의 A12 프로세서 구성과 클럭 속도는?

답 보기

큰 ARM 코어 2개 + 작은 ARM 코어 4개, 2.5 GHz. [Ch1 p.16]

Ch1 · 6 슬라이드 p.20–22

반도체와 칩 비용

이 단원을 마치면 트랜지스터·IC·VLSI·반도체의 뜻을 쓰고, 다이당 비용과 수율 공식을 이용해 칩 비용을 계산하며, 각 요소를 누가 결정하는지 말할 수 있다.

쉽게 말하면

둥근 피자 도우(웨이퍼)에 네모난 쿠키 틀(다이)을 찍어 낸다고 하자. 도우 한 판 값은 정해져 있다. 쿠키를 크게 찍으면 한 판에서 나오는 개수가 줄고, 도우 군데군데 탄 자국(결함)이 있으면 그 자리에 걸린 쿠키는 버린다. 쿠키가 클수록 탄 자국에 걸릴 확률도 커진다. 그래서 큰 칩은 개수도 줄고 불량도 늘어 값이 급격히 오른다.

비유가 안 맞는 지점 실제 칩은 탄 자국처럼 눈에 보이는 결함만 있는 것이 아니고, 검사를 거쳐야 불량을 안다. 공식은 결함이 무작위로 퍼져 있다고 보고 만든 근사식이다.

트랜지스터와 집적회로

프로세서와 메모리는 놀라운 속도로 발전해 왔다. [Ch1 p.20]

트랜지스터(Transistor) 전기로 제어되는 단순한 켜짐/꺼짐(on/off) 스위치. [Ch1 p.20]

집적회로(IC, Integrated Circuit) 칩 하나에 트랜지스터 수십~수백 개를 담은 것. [Ch1 p.20]

초대규모 집적회로(VLSI, Very Large-Scale Integrated circuit) 트랜지스터 수백~수백만 개를 담은 회로. [Ch1 p.20]

집적도가 높아지는 속도는 놀랄 만큼 꾸준했다. 슬라이드의 그래프는 DRAM 용량이 꾸준히 늘어난 모습을 보여 준다. [Ch1 p.20] 이 꾸준한 증가가 8대 아이디어의 "무어의 법칙"이다.

칩 제조 과정

칩은 모래에 있는 물질인 **실리콘(silicon)**에서 시작한다. 실리콘은 전기가 잘 통하지 **않기(NOT)** 때문에 **반도체(semiconductor)**라고 부른다. 특수한 화학 공정을 거치면 실리콘의 일부 영역을 다음 셋 중 하나로 만들 수 있다. [Ch1 p.21]

1. 뛰어난 도체(전기가 잘 통함)
2. 뛰어난 절연체(전기가 통하지 않음)
3. 특정 조건에서 전기가 통하거나 막히는 영역(스위치, 즉 트랜지스터)

슬라이드 그림은 Intel Core i7 칩들이 찍힌 12인치 웨이퍼다. [Ch1 p.21]

웨이퍼(Wafer) 실리콘 덩어리를 얇게 썬 원판. 그 위에 칩 여러 개를 한꺼번에 만든다. PDF 외 보충

다이(Die) 웨이퍼를 잘라 낸 칩 하나하나의 네모난 조각. PDF 외 보충

수율(Yield) 웨이퍼의 다이 중 정상 동작하는 다이의 비율. PDF 외 보충

집적회로의 비용

예: 10세대 Intel Core 프로세서(Ice Lake) 웨이퍼는 12인치(300 mm), 10 nm 공정, 칩 크기 11.4 × 10.7 mm^2이고, 수율이 100%라면 칩 506개가 나온다. [Ch1 p.22]

```
다이당 비용 = 웨이퍼당 비용 / (웨이퍼당 다이 수 × 수율)
```

```
웨이퍼당 다이 수 ≈ 웨이퍼 면적 / 다이 면적
```

```
수율 = 1 / (1 + 면적당 결함 수 × 다이 면적 / 2)^N
```

[Ch1 p.22] 슬라이드는 지수를 N으로 쓴다. 교재(Patterson–Hennessy) 예제는 N = 2를 쓴다. 문제에 N이 주어지지 않으면 N = 2로 두고 계산한다.

이 식들은 면적과 결함률에 대해 **비선형** 관계다. 다이 면적이 커지면 웨이퍼당 다이 수가 줄고 동시에 수율도 떨어지므로, 비용이 면적에 비례하는 것보다 빠르게 오른다. [Ch1 p.22]

요소	누가 정하나
웨이퍼 비용, 웨이퍼 면적	고정되어 있다
결함률(면적당 결함 수)	제조 공정(manufacturing process)
다이 면적	구조(architecture)와 회로 설계(circuit design)

[Ch1 p.22]

예제

웨이퍼 한 장이 5,000달러, 다이 면적 1 cm^2일 때 웨이퍼당 다이가 500개 나온다. 면적당 결함 수는 0.2개/cm^2이다. (1) 수율과 다이당 비용은? (2) 설계를 바꿔 다이 면적이 2 cm^2가 되면(다이 250개)? 수율 식의 지수는 N = 2라고 둔다(교재 예제의 값). PDF 외 보충

풀이 보기

(1) 수율 = $1 / (1 + 0.2 \times 1 / 2)^2 = 1 / 1.1^2 = 1 / 1.21 \approx 0.826$

다이당 비용 = $5000 / (500 \times 0.826) = 5000 / 413.2 \approx$ **12.10달러**

(2) 수율 = $1 / (1 + 0.2 \times 2 / 2)^2 = 1 / 1.2^2 = 1 / 1.44 \approx 0.694$

다이당 비용 = $5000 / (250 \times 0.694) = 5000 / 173.6 \approx$ **28.80달러**

면적은 2배인데 비용은 28.80 / 12.10 ≈ 2.38배가 되었다. 다이 수가 절반이 되고 수율도 떨어졌기 때문이다. 이것이 "비선형"의 뜻이다.

예제

Ice Lake 웨이퍼(지름 300 mm)에서 "웨이퍼 면적 / 다이 면적"으로 다이 수를 어림하면 몇 개인가? 슬라이드의 506개와 왜 다른가?

PDF 외 보충

풀이 보기

웨이퍼 면적 = $\pi \times 150^2 \approx 70{,}686\ mm^2$, 다이 면적 = $11.4 \times 10.7 = 121.98\ mm^2$

70,686 / 121.98 ≈ 579개. 실제는 506개로 더 적다. 웨이퍼는 둥글고 다이는 네모라서 가장자리에 온전한 다이를 놓을 수 없는 부분이 생기기 때문이다. 그래서 식에 "≈"(근사)가 붙는다.

함정

- 실리콘이 반도체인 이유는 전기가 "잘 통하기" 때문이 아니라 잘 통하지 **않기** 때문이다. 슬라이드의 NOT 강조.
- 다이당 비용 식에서 수율은 **분모**에 곱한다. 수율이 낮을수록 비용이 올라간다.
- 결함률은 공정이, 다이 면적은 구조·회로 설계가 정한다. 둘을 바꿔 쓰지 말 것.

시험 포인트

- 트랜지스터·IC·VLSI·반도체 정의
- 다이당 비용·웨이퍼당 다이 수·수율 공식 빈칸
- 웨이퍼 비용·면적 = 고정, 결함률 = 제조 공정, 다이 면적 = 구조·회로 설계

스스로 확인

1. "전기로 제어되는 단순한 on/off 스위치"는 무엇인가?

   답 보기

   트랜지스터. [Ch1 p.20]

2. 실리콘을 반도체라고 부르는 이유는?

   답 보기

   전기가 잘 통하지 않기 때문. 화학 공정으로 도체·절연체·스위치 영역으로 만들 수 있다. [Ch1 p.21]

3. 다이당 비용 = ( ) / (× 수율)

   답 보기

   웨이퍼당 비용 / (웨이퍼당 다이 수 × 수율).

4. 칩 비용 요소 중 제조 공정이 결정하는 것과 구조·회로 설계가 결정하는 것을 쓰시오.

답 보기

제조 공정 → 결함률(면적당 결함 수), 구조·회로 설계 → 다이 면적. [Ch1 p.22]

5. O/X: 다이 면적이 2배가 되면 다이당 비용도 정확히 2배가 된다.

답 보기

X. 다이 수가 줄고 수율도 떨어지므로 2배보다 더 오른다(비선형).

6. 면적당 결함 0.4개/cm^2, 다이 면적 1 cm^2일 때 수율은? (N = 2라고 둔다)

답 보기

$1 / (1 + 0.4 \times 1 / 2)^2 = 1 / 1.2^2 = 1 / 1.44 \approx 0.69$.

Ch1 · 7 슬라이드 p.23–25

성능이란 무엇인가

이 단원을 마치면 응답 시간과 처리량을 구별하고, 성능 = 1/실행 시간으로 "n배 빠르다"를 계산하며, 경과 시간과 CPU 시간의 차이를 말할 수 있다.

쉽게 말하면

택배 회사를 생각하자. 고객 한 명은 "내 택배가 얼마나 빨리 오나"(응답 시간)를 따진다. 회사 사장은 "하루에 몇 개를 배달했나"(처리량)를 따진다. 더 빠른 트럭으로 바꾸면 둘 다 좋아진다. 트럭을 한 대 더 사면 하루 배달 개수는 늘지만, 내 택배 한 개가 오는 시간은 그대로다.

비유가 안 맞는 지점 택배가 밀려 줄을 서 있는 상황이라면 트럭을 늘렸을 때 기다리는 시간이 줄어 응답 시간도 좋아질 수 있다. 교재도 이 경우를 언급한다. 시험에서는 슬라이드대로 "추가 = 처리량만"으로 답한다. PDF 외 보충

성능을 정의하기

"어느 비행기가 성능이 가장 좋은가?"라는 질문은 기준에 따라 답이 달라진다. 속도로 보면 가장 빠른 비행기가, 한 번에 나르는 승객 수로 보면 가장 큰 비행기가 이긴다. [Ch1 p.23] 컴퓨터 성능도 무엇을 기준으로 삼느냐가 먼저다.

컴퓨터 성능에 영향을 주는 요소: 프로세서 특성(클럭 주파수, 코어 수), 시스템 버스 속도, 캐시·메모리 크기, 디스크 저장장치(SSD 또는 HDD), 그래픽 카드(GPU), 전력 소비, 가격·무게·폼 팩터(크기·모양) 등. [Ch1 p.23]

응답 시간과 처리량

응답 시간(Response time) 작업의 시작부터 완료까지 걸린 시간. **실행 시간(execution time)**이라고도 한다. 개인 사용자는 응답 시간을 줄이는 데 관심이 있다. [Ch1 p.24]

처리량(Throughput) 주어진 시간 동안 처리한 일의 총량. **대역폭(bandwidth)**이라고도 한다. 데이터센터 관리자는 처리량을 늘리는 데 관심이 있다. [Ch1 p.24]

변경	응답 시간	처리량
프로세서를 더 빠른 것으로 **교체**	개선	개선
프로세서를 **추가**	변화 없음	개선

[Ch1 p.24]

성능의 정의와 "n배 빠르다"

이 과목은 주로 응답 시간에 관심을 둔다. 컴퓨터 X의 성능은 다음과 같이 정의한다. [Ch1 p.24]

$$\text{성능}_X = 1 \,/\, \text{실행 시간}_X$$

$$\text{"X가 Y보다 n배 빠르다"} \Leftrightarrow \text{성능}_X \,/\, \text{성능}_Y = \text{실행 시간}_Y \,/\, \text{실행 시간}_X = n$$

실행 시간이 짧을수록 성능이 높다. 그래서 비율을 구할 때 성능은 X가 분자, 실행 시간은 **Y가 분자**로 뒤집힌다. [Ch1 p.24]

예제

같은 프로그램이 컴퓨터 A에서 10초, B에서 15초 걸린다. 어느 쪽이 몇 배 빠른가? 교재 기준

풀이 보기

$성능_A$ / $성능_B$ = $실행\ 시간_B$ / $실행\ 시간_A$ = 15 / 10 = 1.5

A가 B보다 1.5배 빠르다. 같은 말로 "B가 A보다 1.5배 느리다"고 할 수 있다.

성능 측정: 경과 시간과 CPU 시간

시간이 컴퓨터 성능의 척도다. 프로그램 실행 시간은 프로그램당 초(seconds per program)로 잰다. [Ch1 p.25]

경과 시간(Elapsed time)

벽시계 시간(wall clock time), 응답 시간이라고도 한다. 작업을 끝내는 데 걸린 **전체** 시간으로, 디스크·메모리 접근, 입출력(I/O) 활동, 운영체제 오버헤드까지 **모두** 포함한다. [Ch1 p.25]

CPU 실행 시간(CPU execution time, CPU time)

CPU가 그 작업을 위해 계산하는 데 쓴 시간. I/O에 쓴 시간이나 다른 프로그램이 쓴 시간은 포함하지 **않는다(NOT)**. [Ch1 p.25]

사용자 CPU 시간(User CPU time) / 시스템 CPU 시간(System CPU time)

CPU 시간은 다시 둘로 나뉜다. 사용자 CPU 시간은 프로그램 자체의 코드를 실행한 시간, 시스템 CPU 시간은 그 프로그램을 위해 운영체제가 일한 시간이다. [Ch1 p.25] (세부 정의는 PDF 외 보충)

응용 프로그램마다 민감한 성능 측면이 다르다. 그래서 프로그램 성능을 높이려면 먼저 어떤 성능 지표가 중요한지 명확히 정하고, 프로그램 실행을 측정해 성능 병목(bottleneck)을 찾는다. [Ch1 p.25]

예제

어떤 프로그램을 실행했더니 시작부터 끝까지 10초가 걸렸다. 그동안 CPU는 이 프로그램의 코드를 6초, 이 프로그램을 위한 운영체제 일을 1초 실행했고, 나머지는 디스크를 기다리거나 다른 프로그램이 CPU를 쓴 시간이다. 경과 시간, CPU 시간, 사용자 CPU 시간은? PDF 외 보충

풀이 보기

경과 시간 = 10초(전부 포함). CPU 시간 = 6 + 1 = 7초. 사용자 CPU 시간 = 6초(시스템 CPU 시간 = 1초). 나머지 3초는 CPU 시간에 들어가지 않는다.

함정

- "프로세서를 추가하면 응답 시간도 줄어든다"는 슬라이드 기준 X. 추가는 처리량만 개선한다.
- CPU 시간은 I/O 시간과 다른 프로그램의 시간을 포함하지 **않는다**. 경과 시간은 모두 포함한다.
- "X가 n배 빠르다"에서 n = 실행 시간$_Y$ / 실행 시간$_X$. 실행 시간 비율의 분자·분모를 뒤집어 쓰면 1보다 작은 값이 나온다.

시험 포인트

- 응답 시간 = 실행 시간, 처리량 = 대역폭. 교체 vs 추가의 영향(O/X 단골)
- 성능 = 1/실행 시간, n배 계산
- 경과 시간(wall clock, 응답 시간)과 CPU 시간의 포함 범위, user/system CPU 시간

스스로 확인

1. 개인 사용자가 줄이고 싶어 하는 것과 데이터센터 관리자가 늘리고 싶어 하는 것을 쓰시오.

   답 보기

   응답 시간(실행 시간), 처리량(대역폭). [Ch1 p.24]

2. O/X: 컴퓨터에 프로세서를 추가하면 응답 시간과 처리량이 모두 개선된다.

   답 보기

   X. 추가는 처리량만 개선. 더 빠른 프로세서로 교체해야 둘 다 개선된다.

3. 성능$_X$를 실행 시간으로 나타내시오.

   답 보기

   성능$_X$ = 1 / 실행 시간$_X$.

4. 프로그램이 X에서 4초, Y에서 12초 걸린다. X는 Y보다 몇 배 빠른가?

   답 보기

   12 / 4 = 3배.

5. 디스크·메모리 접근, I/O, OS 오버헤드까지 모두 포함한 시간을 무엇이라 하는가? 다른 이름 두 가지는?

   답 보기

   경과 시간(elapsed time). 벽시계 시간(wall clock time), 응답 시간(response time). [Ch1 p.25]

6. CPU 시간은 무엇과 무엇으로 나뉘는가?

답 보기

사용자 CPU 시간(user CPU time)과 시스템 CPU 시간(system CPU time).

Ch1 · 8 슬라이드 p.26–29

클럭과 CPU 성능 공식

이 단원을 마치면 클럭 주기와 클럭 속도를 서로 환산하고, CPU 시간 = 명령어 수 × CPI × 클럭 주기로 성능을 계산하며, 각 요소에 무엇이 영향을 주는지 표로 말할 수 있다.

쉽게 말하면

조정 경기에서 북 치는 사람이 "둥, 둥" 박자를 맞추면 노 젓는 사람들이 그 박자마다 한 번씩 움직인다. 이 북소리가 클럭이다. 일을 끝내는 데 걸리는 시간은 (해야 할 동작 수) × (동작 하나에 필요한 박자 수) × (박자 하나의 길이)로 정해진다. 이것이 CPU 시간 = 명령어 수 × CPI × 클럭 주기다.

비유가 안 맞는 지점 조정에서는 모든 동작이 박자 하나씩이지만, CPU에서는 명령어마다 필요한 클럭 수가 다를 수 있다. 그래서 CPI는 **평균**이다.

CPU 클럭

거의 모든 컴퓨터는 하드웨어에서 일이 언제 일어날지를 정하는 **클럭(clock)**을 써서 만든다. 클럭이 나누는 일정한 시간 간격을 **클럭 사이클(clock cycle)**이라 하고, tick, clock tick, clock period, clock, cycle이라고도 부른다. 한 사이클 동안 데이터 전송과 계산이 일어나고, 사이클 경계에서 상태가 갱신된다. [Ch1 p.26]

클럭 주기(Clock period, clock cycle time) 클럭 사이클 하나의 길이(시간). 예: 250 ps. [Ch1 p.26]

클럭 속도(Clock rate, clock frequency) 초당 사이클 수. 예: 4 GHz. [Ch1 p.26]

```
클럭 속도 = 1 / 클럭 주기
```

시간 단위	값	주파수 단위	값
ms (밀리초)	10^{-3} 초	kHz	10^{3} Hz
μs (마이크로초)	10^{-6} 초	MHz	10^{6} Hz
ns (나노초)	10^{-9} 초	GHz	10^{9} Hz
ps (피코초)	10^{-12} 초	THz	10^{12} Hz

Hz(헤르츠) = 초당 횟수. 예: 250 ps = 250 × 10^{-12} 초 → 1 / (250 × 10^{-12}) = 4 × 10^{9} Hz = **4 GHz**. 2 GHz ↔ 0.5 ns = 500 ps, 2.5 GHz ↔ 0.4 ns = 400 ps. PDF 외 보충

CPU 성능 공식 유도

최종 성능 척도는 CPU 실행 시간이다. [Ch1 p.27]

```
CPU 실행 시간 = CPU 클럭 사이클 수 × 클럭 주기 = CPU 클럭 사이클 수 / 클럭 속도
```

따라서 하드웨어 설계자는 두 방법으로 성능을 높인다. ① 프로그램에 필요한 클럭 사이클 수를 줄인다. ② 클럭 주기를 줄인다(클럭 속도를 높인다). 설계자는 흔히 사이클 수와 클럭 속도 사이의 **트레이드오프**에 부딪힌다. 한쪽을 좋게 하면 다른 쪽이 나빠지기 쉽다는 뜻이다. [Ch1 p.27]

실행 시간은 프로그램의 명령어 수에도 달려 있다. 사이클 수를 명령어 수로 쪼개면 다음과 같다. [Ch1 p.28]

```
클럭 사이클 수 = 명령어 수 × CPI
```

```
CPU 실행 시간 = 명령어 수 × CPI × 클럭 주기 = 명령어 수 × CPI / 클럭 속도
```

명령어 수(IC, Instruction Count) 프로그램이 실행하는 명령어의 개수. **프로그램, ISA, 컴파일러**가 정한다. [Ch1 p.28]

CPI(Clock cycles Per Instruction) 한 프로그램에서 명령어 하나가 실행되는 데 걸리는 **평균** 클럭 사이클 수. CPU 하드웨어가 정한다. 명령어마다 걸리는 사이클이 다를 수 있으므로 CPI는 평균이다. [Ch1 p.28]

명령어 종류가 여러 가지일 때, 종류 i의 CPI를 CPI_i, 개수를 C_i라 하면 다음과 같다. 교재 기준

$$\text{클럭 사이클 수} = \Sigma\,(CPI_i \times C_i),\quad \text{평균 CPI} = \text{클럭 사이클 수} / \text{명령어 수}$$

성능 요약

모든 것을 고려하면 프로그램당 초로 잰 실행 시간은 다음과 같다. 단위가 차례로 지워지는 것을 보면 외우기 쉽다. [Ch1 p.29]

```
CPU 시간 = 초 / 프로그램 = (명령어 / 프로그램) × (클럭 사이클 / 명령어) × (초 / 클럭 사이클)
```

공식의 각 요소와 측정 단위는 다음과 같다. [Ch1 p.29]

성능의 구성 요소(Components of performance)	측정 단위(Units of measure)
프로그램의 CPU 실행 시간(CPU execution time for a program)	프로그램에 걸린 초(Seconds for the program)
명령어 수(Instruction count)	프로그램에서 실행된 명령어 수(Instructions executed for the program)
CPI(Clock cycles per instruction)	명령어당 평균 클럭 사이클 수(Average number of clock cycles per instruction)
클럭 주기(Clock cycle time)	클럭 사이클당 초(Seconds per clock cycle)

프로그램 성능에 각 요소가 주는 영향은 다음과 같다(○ = 영향을 줌). [Ch1 p.29]

요소	명령어 수	CPI	클럭 속도	이유
알고리즘(Algorithm)	○	○		실행할 명령어 수와, CPI가 다른 명령어를 어떤 비율로 쓰는지 정함
프로그래밍 언어(Programming Language)	○	○		문장이 명령어로 바뀌는 방식
컴파일러(Compiler)	○	○		어떤 명령어를 몇 개 고르는지
ISA	○	○	○	명령어 종류·사이클·구현 모두에 영향
기술(Technology)			○	공정이 회로 속도를 정함

기술(Technology) 행은 교재 표에는 없고 슬라이드에 추가된 행이다. 슬라이드에서 이 행은 클럭 속도 칸에만 표시가 있다. [Ch1 p.29] "CPI는 CPU 하드웨어가 정한다"는 말과 표에서 알고리즘·언어·컴파일러가 CPI에 영향을 주는 것이 어긋나 보일 수 있다. 명령어 하나하나의 사이클은 하드웨어가 정하지만, 그 명령어들을 어떤 비율로 섞어 쓸지는 소프트웨어가 정하기 때문에 평균 CPI가 바뀐다.

교재 예제 1 교재 기준 [Ch1 p.27 → 교재 pp.34–35]

어떤 프로그램이 클럭 2 GHz인 컴퓨터 A에서 10초 걸린다. 같은 프로그램을 6초에 실행하는 컴퓨터 B를 만들려 한다. 클럭 속도를 크게 올릴 수 있지만 그 영향으로 B는 이 프로그램에 A보다 1.2배 많은 클럭 사이클이 필요하다. B의 클럭 속도는 얼마여야 하는가?

풀이 보기

1단계: A의 사이클 수 = 실행 시간 × 클럭 속도 = 10초 × 2×10^9 사이클/초 = 20×10^9 사이클

2단계: B의 사이클 수 = $1.2 \times 20 \times 10^9 = 24 \times 10^9$ 사이클

3단계: B의 클럭 속도 = 사이클 수 / 실행 시간 = 24×10^9 / 6초 = 4×10^9 Hz = **4 GHz**

검산: B의 시간 = 24×10^9 / (4×10^9) = 6초. 클럭을 2배로 올렸지만 사이클이 1.2배 늘어 시간은 10/6 ≈ 1.67배만 줄었다 (트레이드오프).

교재 예제 2 교재 기준 [Ch1 p.28 → 교재 pp.35–36]

같은 ISA를 구현한 두 컴퓨터가 있다. A는 클럭 주기 250 ps, 어떤 프로그램의 CPI 2.0이고, B는 클럭 주기 500 ps, 같은 프로그램의 CPI 1.2다. 어느 쪽이 몇 배 빠른가?

풀이 보기

1단계: ISA가 같으므로 명령어 수는 같다. 이를 I라 하자.

2단계: A의 CPU 시간 = I × 2.0 × 250 ps = 500 × I ps

3단계: B의 CPU 시간 = I × 1.2 × 500 ps = 600 × I ps

4단계: 성능$_A$ / 성능$_B$ = 실행 시간$_B$ / 실행 시간$_A$ = 600 I / 500 I = 1.2

A가 B보다 1.2배 빠르다. I가 약분되므로 명령어 수를 몰라도 답이 나온다.

교재 예제 3 교재 기준 [Ch1 p.28 → 교재 pp.36–37]

명령어 클래스 A, B, C의 CPI가 각각 1, 2, 3이다. 같은 일을 하는 두 코드가 있다. 코드 1은 A 2개, B 1개, C 2개, 코드 2는 A 4개, B 1개, C 1개를 실행한다. 명령어가 더 많은 코드는? 더 빠른 코드는? 각 코드의 CPI는?

풀이 보기

1단계 (명령어 수): 코드 1 = 2 + 1 + 2 = 5개, 코드 2 = 4 + 1 + 1 = 6개 → 코드 2가 더 많다.

2단계 (사이클 수 = Σ CPI$_i$ × C$_i$): 코드 1 = 2×1 + 1×2 + 2×3 = 2 + 2 + 6 = 10사이클, 코드 2 = 4×1 + 1×2 + 1×3 = 4 + 2 + 3 = 9사이클 → **코드 2가 더 빠르다**(같은 컴퓨터이므로 클럭 주기가 같다).

3단계 (CPI = 사이클 / 명령어 수): 코드 1 = 10 / 5 = **2.0**, 코드 2 = 9 / 6 = **1.5**

교훈: 명령어 수가 더 많아도 CPI가 낮으면 더 빠를 수 있다. 명령어 수 하나만으로 성능을 판단하면 안 된다.

핵심

CPU 시간 = 명령어 수 × CPI × 클럭 주기 = 명령어 수 × CPI / 클럭 속도. 클럭 주기와 클럭 속도는 역수. 성능은 세 요소를 **모두** 곱해서 판단한다.

함정

- 클럭 속도가 높다고 무조건 빠른 것은 아니다. 교재 예제 2에서 클럭이 2배 빠른 A가 이긴 것은 CPI 차이가 그보다 작았기 때문이고, CPI가 충분히 크면 결과가 뒤집힌다.
- 명령어 수가 적다고 무조건 빠른 것도 아니다(교재 예제 3).
- 클럭 **주기**를 곱할 때와 클럭 **속도**로 나눌 때를 헷갈리지 말 것. 단위(초/사이클 vs 사이클/초)를 적어 보면 실수가 줄어든다.
- CPI는 "명령어당 사이클"이다. "사이클당 명령어"(IPC)와 뒤집지 말 것. PDF 외 보충

시험 포인트

- ps·ns ↔ GHz 단위 환산
- CPU 시간 공식 빈칸, 세 교재 예제 유형의 계산(계산 문제 1순위)
- IC는 프로그램·ISA·컴파일러가, CPI는 CPU 하드웨어가 정함. CPI는 평균
- 성능 요소 영향 표(알고리즘·언어·컴파일러 = IC·CPI, ISA = 셋 다)

스스로 확인

1. 클럭 주기가 250 ps인 CPU의 클럭 속도는?

   답 보기

   $1 / (250 \times 10^{-12}) = 4 \times 10^{9}$ Hz = 4 GHz.

2. 클럭 속도가 2.5 GHz이면 클럭 주기는 몇 ps인가?

   답 보기

   $1 / (2.5 \times 10^{9}) = 0.4 \times 10^{-9}$ 초 = 0.4 ns = 400 ps.

3. CPU 실행 시간 = 명령어 수 × ( ) × ( )

   답 보기

   CPI × 클럭 주기 (또는 명령어 수 × CPI / 클럭 속도). [Ch1 p.28]

4. 명령어 수를 결정하는 것 세 가지와 CPI를 결정하는 것을 쓰시오.

답 보기

명령어 수: 프로그램, ISA, 컴파일러. CPI: CPU 하드웨어. [Ch1 p.28]

5. 명령어 수 10^9개, CPI 2, 클럭 4 GHz일 때 CPU 시간은?

답 보기

$10^9 \times 2 / (4 \times 10^9) = 0.5$초.

6. O/X: 같은 ISA라면 클럭 속도가 높은 컴퓨터가 항상 더 빠르다.

답 보기

X. CPI가 다르면 결과가 달라진다. CPU 시간은 IC × CPI × 클럭 주기로 판단한다.

7. 성능 요소 표에서 명령어 수·CPI·클럭 속도 셋 모두에 영향을 주는 요소는?

답 보기

ISA. [Ch1 p.29]

8. CPI가 1, 2, 3인 클래스 A, B, C를 각각 3, 2, 1개 실행하는 코드의 평균 CPI는?

답 보기

사이클 = 3×1 + 2×2 + 1×3 = 10, 명령어 = 6 → CPI = 10/6 ≈ 1.67.

Ch1 · 9 슬라이드 p.30–32

전력과 멀티코어

이 단원을 마치면 동적 에너지·전력 식을 쓰고, 전압을 낮춰 전력 증가를 억제한 원리와 전력 장벽의 두 이유, 멀티코어로 바뀐 까닭을 설명하며 교재 전력 예제를 계산할 수 있다.

쉽게 말하면

자전거 페달을 더 빨리 밟을수록(클럭 속도) 힘이 더 든다(전력). 오랫동안은 기어를 바꿔(전압을 낮춰) 같은 힘으로 더 빨리 밟을 수 있었다. 그런데 기어를 더 낮출 수 없게 되었고, 몸에서 나는 열을 식히는 것도 한계에 왔다. 그래서 한 사람이 더 빨리 밟는 대신 자전거를 여러 대 두고 여러 사람이 나눠 타기로 했다. 이것이 멀티코어다.

비유가 안 맞는 지점 자전거 여러 대는 짐을 나누면 바로 이득이지만, 멀티코어는 프로그램이 일을 나눌 수 있게 짜여 있어야 이득이 난다. 그래서 슬라이드도 이득이 응답 시간보다 처리량 쪽에 크다고 한다.

전력 추세

클럭 속도와 전력은 수십 년 동안 함께 빠르게 늘다가, 최근에는 평평해지거나 떨어졌다. [Ch1 p.30] Post-PC 시대에 정말 귀한 자원은 **에너지**다. [Ch1 p.30]

집적회로의 주류 기술인 **CMOS(Complementary Metal Oxide Semiconductor, 상보형 금속 산화막 반도체)**에서 동적 에너지는 다음과 같다. **용량성 부하(capacitive load)**는 트랜지스터 출력에 연결된 도선과 다른 트랜지스터들이 전하를 담아 두는 정도로, 한 번 전환할 때 채우고 비워야 하는 전기의 양을 결정한다. [Ch1 p.30] (용량성 부하 설명은 PDF 외 보충)

$$\text{에너지} \propto \text{용량성 부하} \times \text{전압}^2$$

동적 전력은 에너지와 주파수의 곱이다. 1초에 전환이 몇 번 일어나느냐가 주파수이기 때문이다. [Ch1 p.31]

$$\text{전력} \propto \text{용량성 부하} \times \text{전압}^2 \times \text{주파수}$$

교재는 이 식에 1/2을 붙여 쓰기도 한다(0→1 또는 1→0 한 번의 전환 기준). 비례식(∝)이라 비율 계산에는 영향이 없다. 교재 기준

클럭 1000배, 전력 30배

지난 20년 동안 클럭 속도는 1000배 늘었는데 전력은 30배밖에 늘지 않았다. 어떻게 가능했을까? 전압이 **5 V에서 1 V로** 내려갔기 때문이다. 전압은 보통 **세대마다 약 15%**씩 낮아졌다. [Ch1 p.31]

전력 식에서 전압은 **제곱**으로 들어간다. 전압이 1/5이 되면 전압^{2}은 1/25이 된다. 그래서 주파수가 크게 늘어도 전력 증가를 크게 억제할 수 있었다. PDF 외 보충

전력 장벽(Power Wall)

전력 장벽이란 다음 두 가지 때문에 더 이상 클럭을 올리기 어려워진 상황이다. [Ch1 p.31]

1. **전압을 더 낮출 수 없다(NOT)** → 누설(leakage) 때문. 전압을 너무 낮추면 트랜지스터가 꺼져 있어도 전류가 새어 나간다.
2. **열을 더 제거할 수 없다(NOT)** → 냉각 비용(cooling cost) 때문.

(누설에 대한 한 줄 설명은 PDF 외 보충)

교재 예제 교재 기준 [Ch1 p.31 → 교재 pp.41–42]

더 단순한 새 프로세서를 만들었다. 용량성 부하는 기존 프로세서의 85%이고, 전압을 15% 낮출 수 있으며 그 결과 주파수도 15% 줄어든다. 동적 전력은 기존의 몇 배인가?

풀이 보기

1단계: 새 값을 기존 값의 비율로 쓴다. 용량성 부하 0.85, 전압 (1 − 0.15) = 0.85, 주파수 0.85.

2단계: 전력$_{새}$ / 전력$_{기존}$ = (0.85 × C × (0.85 V)^2 × 0.85 f) / (C × V^2 × f)

3단계: = 0.85 × 0.85^2 × 0.85 = 0.85^4

4단계: 0.85^2 = 0.7225, 0.85^4 = 0.7225^2 ≈ 0.522

새 프로세서의 전력은 기존의 **약 0.52배**(약 52%)다. 전압이 제곱으로 들어가므로 0.85가 네 번 곱해진다는 점이 핵심이다.

단일 프로세서에서 멀티프로세서로

전력 한계가 마이크로프로세서 설계를 크게 바꿨다. 2006년 무렵부터 모든 데스크톱·서버 회사가 **칩 하나에 코어 여러 개(multiple cores per chip)**를 넣은 마이크로프로세서를 내놓고 있다. 이 방식의 이득은 응답 시간보다 **처리량** 쪽에 더 크다. [Ch1 p.32]

코어(Core) 칩 안에 들어 있는 프로세서 하나. 멀티코어 칩은 프로세서 여러 개를 한 칩에 담은 것이다. PDF 외 보충

코어를 늘리면 여러 프로그램(또는 한 프로그램의 여러 부분)을 동시에 돌릴 수 있어 처리량이 는다. 한 프로그램 하나가 빨라지려면 프로그래머가 일을 나눠 병렬로 실행되도록 프로그램을 짜야 한다. 이는 앞 단원 표의 "프로세서 추가 = 처리량 개선"과도 맞는다. PDF 외 보충

함정

- 전압은 **제곱**, 용량성 부하와 주파수는 **1제곱**으로 들어간다. 예제에서 0.85^{3}으로 끝내면 틀린다.
- 에너지 식에는 주파수가 없다. 주파수가 곱해지는 것은 **전력** 식이다.
- 전력 장벽의 두 이유: 전압을 더 낮출 수 없음(누설), 열을 더 뺄 수 없음(냉각 비용). "전압을 더 높일 수 없다"로 쓰면 틀린다.
- 멀티코어의 이득은 응답 시간보다 처리량 쪽에 크다.

시험 포인트

- 에너지·전력 비례식 빈칸
- 클럭 1000배·전력 30배의 이유: 전압 5 V → 1 V(세대당 약 15%)
- 전력 장벽 두 이유(NOT 강조 두 줄), 2006년 멀티코어 전환
- 교재 예제: 0.85^4 ≈ 0.52

스스로 확인

1. CMOS에서 동적 전력 $\propto$ ( ) × ( )^2 × ( )

답 보기

용량성 부하 × 전압^2 × 주파수. [Ch1 p.31]

2. 20년간 클럭은 1000배 늘었는데 전력은 30배만 는 이유는?

답 보기

전압이 5 V에서 1 V로 낮아졌기 때문(세대당 약 15%씩 감소). 전력은 전압의 제곱에 비례한다.

3. 전력 장벽의 두 원인을 쓰시오.

답 보기

① 누설 때문에 전압을 더 낮출 수 없다. ② 냉각 비용 때문에 열을 더 제거할 수 없다. [Ch1 p.31]

4. 용량성 부하 85%, 전압 15% 감소, 주파수 15% 감소 시 전력은 기존의 몇 배인가?

답 보기

$0.85^4 \approx 0.52$배.

5. 전압만 절반으로 줄이고 나머지는 그대로일 때 동적 전력은 몇 배가 되는가?

답 보기

$0.5^2 = 0.25$배.

6. O/X: 멀티코어 프로세서의 이득은 처리량보다 응답 시간 쪽에 더 크다.

답 보기

X. 슬라이드는 응답 시간보다 처리량 쪽 이득이 크다고 한다. [Ch1 p.32]

Ch1 · 10 슬라이드 p.33–34

암달의 법칙과 MIPS의 함정

이 단원을 마치면 암달의 법칙으로 일부 개선 후의 실행 시간과 최대 속도 향상을 계산하고, MIPS 식을 쓰고 MIPS가 성능 지표로 부적절한 이유를 말할 수 있다.

쉽게 말하면

집에서 학교까지 100분 걸리는데 그중 80분이 버스, 20분이 걸어가는 시간이라 하자. 버스를 아무리 빠르게 해도 걷는 20분은 그대로라서 100분이 20분 아래로 줄지는 않는다. 반대로 걷는 시간만 줄이면 아무리 노력해도 80분 아래로 못 내려간다. 그러니 시간을 많이 차지하는 쪽(자주 생기는 경우)을 빠르게 해야 효과가 크다.

비유가 안 맞는 지점 출퇴근은 버스와 걷기가 확실히 나뉘지만, 프로그램에서는 어느 부분이 몇 초를 차지하는지 측정해야 안다. 앞 단원의 "측정으로 병목 찾기"가 그래서 필요하다.

암달의 법칙(Amdahl's Law)

컴퓨터의 한 부분을 개선하면서 전체 성능이 그에 비례해 좋아지리라 기대하는 것은 함정이다. 개선 후 실행 시간은 다음과 같다. [Ch1 p.33]

$$T_{\text{개선 후}} = T_{\text{영향 받는 부분}} / \text{개선 배수} + T_{\text{영향 받지 않는 부분}}$$

영향 받지 않는 부분은 그대로 남는다. 그래서 개선 배수를 아무리 키워도 전체 실행 시간은 $T_{\text{영향 받지 않는 부분}}$보다 작아질 수 없다. [Ch1 p.33]

예제 (슬라이드) [Ch1 p.33]

어떤 프로그램이 100초 걸리고, 그중 곱셈 연산이 80초를 차지한다. (1) 전체를 2.5배 빠르게 하려면 곱셈 성능을 몇 배 개선해야 하는가? (2) 곱셈이 아닌 연산만 개선해서 전체 2.5배를 얻을 수 있는가?

풀이 보기

(1) 목표 실행 시간 = 100 / 2.5 = 40초

40 = 80 / n + (100 − 80) = 80 / n + 20

20 = 80 / n → 20n = 80 → **n = 4**

검산: 80 / 4 + 20 = 20 + 20 = 40초, 100 / 40 = 2.5배.

(2) 곱셈 외 부분(20초)만 개선하면 40 = 80 + 20 / n이 되어야 한다. 그러나 80초는 그대로 남아 실행 시간이 80초 아래로 내려갈 수 없다. → **불가능**. 곱셈 외 부분을 0초로 만들어도 최대 속도 향상은 100 / 80 = **1.25배**다.

교재 원문은 같은 상황에서 5배를 묻지만, 슬라이드 수치는 2.5배다. 슬라이드 수치로 외울 것. 교재 기준

극한으로 생각하기

곱셈을 무한히 빠르게 하면(n → ∞) 80 / n → 0이 되어 실행 시간은 20초에 다가간다. 이때 최대 속도 향상은 100 / 20 = **5배**다. 즉 이 프로그램은 곱셈만 개선해서는 5배 이상 빨라질 수 없다. 2.5배는 5배보다 작으므로 (1)은 가능했다. PDF 외 보충

```
최대 속도 향상 = 전체 시간 / 영향 받지 않는 부분의 시간
```

그래서 전체 성능을 높이려면 **자주 생기는 경우를 빠르게(make the common case fast)** 해야 한다. 8대 아이디어의 3번이 여기서 나온다. [Ch1 p.33]

MIPS를 성능 지표로 쓸 때의 함정

MIPS(Million Instructions Per Second)는 초당 백만 개 단위로 센 명령어 실행 수다. [Ch1 p.34]

$$\text{MIPS} = \text{명령어 수} / (\text{실행 시간} \times 10^6)$$

$$= \text{명령어 수} / ((\text{명령어 수} \times \text{CPI} / \text{클럭 속도}) \times 10^6) = \text{클럭 속도} / (\text{CPI} \times 10^6)$$

빠른 컴퓨터일수록 MIPS 값이 높다. 하지만 다음 문제가 있다. [Ch1 p.34]

1. MIPS는 컴퓨터 사이의 **ISA 차이를 반영하지 못한다(NOT)**. ISA가 다르면 같은 일을 하는 데 필요한 명령어 수가 달라, 명령어 개수로 비교하는 것이 의미가 없다.
2. MIPS는 **명령어 사이의 복잡도 차이를 반영하지 못한다(NOT)**. 단순한 명령어 하나와 복잡한 명령어 하나를 똑같이 1개로 센다.
3. 같은 CPU에서도 **프로그램마다 CPI가 달라** MIPS가 하나로 정해지지 않는다.

예제

클럭 2 GHz인 같은 컴퓨터에서 같은 프로그램을 두 컴파일러로 컴파일했다. 컴파일러 1의 결과는 명령어 10^9개, CPI 1.0이고, 컴파일러 2의 결과는 명령어 0.6×10^9개, CPI 1.5다. 각각의 실행 시간과 MIPS를 구하고, 어느 쪽이 더 빠른지 말하시오. PDF 외 보충

풀이 보기

컴파일러 1: 시간 = $10^9 \times 1.0 / (2 \times 10^9) = 0.5$초, MIPS = $2 \times 10^9 / (1.0 \times 10^6) = 2000$

컴파일러 2: 시간 = $0.6 \times 10^9 \times 1.5 / (2 \times 10^9) = 0.45$초, MIPS = $2 \times 10^9 / (1.5 \times 10^6) \approx 1333$

검산: $0.6 \times 10^9 / (0.45 \times 10^6) \approx 1333$. MIPS는 컴파일러 1이 높지만, 실제로는 **컴파일러 2가 더 빠르다**(0.45초 < 0.5초). MIPS가 높다고 빠른 것이 아님을 보여 준다.

핵심

$T_{\text{개선 후}} = T_{\text{영향}} / n + T_{\text{비영향}}$. 100초 중 80초 곱셈, 2.5배 → n = 4, 곱셈 외만 개선 → 불가능(최대 1.25배). MIPS = 클럭 속도 / (CPI × 10^6), ISA 차이·명령어 복잡도 차이를 반영하지 못하고 프로그램마다 CPI가 다르다.

함정

- 암달 식에서 개선 배수 n은 **영향 받는 부분에만** 나눈다. 전체 시간을 n으로 나누면 틀린다.
- "곱셈 외 부분만 개선해도 충분히 키우면 2.5배가 된다"는 X. 최대 1.25배.
- MIPS 식 분모의 10^6을 빠뜨리거나 CPI를 분자에 두지 말 것. MIPS = 클럭 속도 / (CPI × 10^6).
- "MIPS가 높으면 항상 더 빠르다"는 X. ISA·명령어 수가 다르면 뒤집힌다(위 예제).

시험 포인트

- 암달 공식 빈칸, 슬라이드 예제 2.5배 → n = 4, 곱셈 외만 → 불가능(최대 1.25배)
- 교훈: 자주 생기는 경우를 빠르게
- MIPS 식 두 형태, MIPS가 부적절한 이유(ISA 차이, 명령어 복잡도, 프로그램마다 CPI 다름)

스스로 확인

1. 암달의 법칙: $T_{\text{개선 후}}$ = ( ) / 개선 배수 + ( )

   답 보기

   $T_{\text{영향 받는 부분}}$ / 개선 배수 + $T_{\text{영향 받지 않는 부분}}$. [Ch1 p.33]

2. 100초 중 곱셈이 80초인 프로그램을 전체 2.5배 빠르게 하려면 곱셈을 몇 배 개선해야 하는가?

   답 보기

   4배. 40 = 80/n + 20 → n = 4.

3. 위 프로그램에서 곱셈 외 부분만 개선할 때 최대 속도 향상은?

   답 보기

   1.25배(100/80). 2.5배는 불가능.

4. 100초 중 60초를 차지하는 부분을 3배 빠르게 하면 전체 실행 시간과 속도 향상은?

   답 보기

   60/3 + 40 = 60초, 100/60 ≈ 1.67배.

5. 클럭 4 GHz, CPI 2인 컴퓨터의 MIPS는?

답 보기

$4 \times 10^9 / (2 \times 10^6) = 2000$ MIPS.

6. MIPS가 성능 지표로 부적절한 이유를 두 가지 이상 쓰시오.

답 보기

컴퓨터 사이의 ISA 차이를 반영하지 못함, 명령어 사이의 복잡도 차이를 반영하지 못함, 같은 CPU에서도 프로그램마다 CPI가 달라 MIPS가 하나로 정해지지 않음. [Ch1 p.34]

7. 암달의 법칙에서 얻는 설계 교훈(8대 아이디어 중 하나)은?

답 보기

자주 생기는 경우를 빠르게(Make the common case fast).

Ch2 · 1 슬라이드 p.2–8

명령어와 ISA, CISC와 RISC

이 단원을 마치면 명령어·opcode·operand·ISA·마이크로아키텍처를 정의하고, CISC와 RISC의 특징과 장단점을 비교해 쓸 수 있다.

쉽게 말하면

컴퓨터에게 일을 시키려면 컴퓨터가 알아듣는 말로 해야 한다. 그 말의 단어 하나하나가 **명령어**이고, 컴퓨터가 아는 단어 전체를 모은 사전이 **명령어 집합**이다. 사람은 C 같은 고급 언어로 "두 수를 바꿔 줘"라고 쓰고, 이것이 어셈블리어를 거쳐 0과 1로 된 기계어로 바뀐 뒤에야 컴퓨터가 알아듣는다.

ISA는 "이 사전에 어떤 단어가 있고 각 단어가 무슨 뜻인지"를 정한 약속이고, 마이크로아키텍처는 그 약속을 지키며 실제로 기계를 만드는 방법이다. 같은 요리법(ISA)을 보고도 주방(마이크로아키텍처)은 여러 가지로 꾸밀 수 있는 것과 비슷하다.

비유가 안 맞는 지점 사람의 언어는 단어가 수십만 개지만 명령어 집합은 훨씬 작고, 뜻이 모호한 단어가 하나도 없다. 또 명령어는 결국 정해진 길이의 비트 묶음이라서 "단어"보다 "규격이 정해진 양식"에 더 가깝다.

명령어와 명령어 집합

컴퓨터 언어의 단어를 **명령어(instruction)**라 하고, 그 어휘 전체를 **명령어 집합(instruction set)**이라 한다. 프로그램은 고급 언어 프로그램 → 어셈블리 언어 프로그램 → 2진 기계어 프로그램 순서로 바뀐다. [Ch2 p.2]

명령어는 프로세서가 할 동작 하나를 나타내는 가장 기본적인 명령이며, 하드웨어와 소프트웨어 사이를 잇는 접점이다. 명령어 하나는 두 부분으로 이루어진다. [Ch2 p.3]

명령어(instruction)	프로세서의 동작 하나를 기술하는 기본 명령. 하드웨어와 소프트웨어의 인터페이스. [Ch2 p.3]
연산 코드(opcode, operation code)	명령어가 **무엇을 하는지**(더하기, 빼기, 읽기 등)를 나타내는 부분.
피연산자(operand)	연산의 **대상**. 레지스터, 상수, 메모리 주소 등이 될 수 있다.
어셈블리 언어(assembly language)	명령어를 사람이 읽을 수 있는 기호로 적은 것. 예 `add $2, $4, $2`에서 `add`가 opcode, 나머지가 operand. [Ch2 p.3]
기계어(machine language)	같은 명령어를 0과 1의 2진수로 적은 것. 컴퓨터가 실제로 읽는 형태.

슬라이드 p.3의 `add $2, $4, $2`는 레지스터를 `$`로 적는 MIPS 식 표기다. 이 강의에서 쓰는 RISC-V는 `x2`처럼 적는다. 개념(opcode + operand)만 보면 된다. PDF 외 보충

명령어 집합 구조(ISA)

명령어 집합 구조(ISA, Instruction Set Architecture)는 프로세서가 이해하는 명령어들의 집합이다. 프로세서마다 ISA가 다르지만 서로 꽤 비슷하다. 이유는 두 가지다. [Ch2 p.3]

1. 비슷한 원리에 기반한 비슷한 하드웨어 기술을 쓰기 때문
2. 모든 컴퓨터가 제공해야 하는 기본 연산이 몇 가지로 공통이기 때문

ISA의 예: ARM(ARMv7, ARMv8, ARMv9), Sun SPARC, IBM POWER, MIPS, **RISC-V**(이 강의에서 다루는 ISA), Intel x86(IA-16, IA-32, x64). 하나를 배우면 다른 것도 쉽게 이해할 수 있다. [Ch2 p.4]

마이크로아키텍처

마이크로아키텍처(microarchitecture)는 정해진 설계 제약과 목표 아래에서 ISA를 실제로 구현한 것이다. 명세(ISA)만 만족하면 구현은 여러 가지일 수 있다. 예를 들어 x86 ISA 하나에 286, 386, ..., Pentium, Core i 시리즈 같은 많은 구현이 있다. [Ch2 p.5]

마이크로아키텍처는 **소프트웨어에 드러나지 않고 하드웨어 안에서 이루어지는 모든 것**이다. 슬라이드의 예 네 가지: [Ch2 p.5]

- 파이프라이닝(pipelining)
- 추측 실행(speculative execution)
- 메모리 접근 스케줄링(memory access scheduling)
- 산술 장치(arithmetic units)

> **핵심**
>
> ISA = 소프트웨어가 보는 약속(어떤 명령어가 있는가). 마이크로아키텍처 = 그 약속의 하드웨어 구현(어떻게 실행하는가). 같은 ISA에 여러 마이크로아키텍처가 있을 수 있다.

CISC와 RISC

CISC(Complex Instruction Set Computer, 복합 명령어 집합 컴퓨터)는 더 오래된 설계 생각이다. Intel x86이 CISC다. 명령어가 복잡하고 길이가 제각각(가변 길이)이며, 강력한 명령어를 ISA 안에 많이 둔다. CISC 명령어는 내부에서 **마이크로코드(microcode)** 변환을 거쳐 더 작은 **마이크로 명령어(microinstruction)**로 바뀐 뒤 실행된다. [Ch2 p.6]

RISC(Reduced Instruction Set Computer, 축소 명령어 집합 컴퓨터)는 CISC보다 새로운 개념이다. 명령어가 단순하고 규격화되어 있으며 명령어 수가 적다. CISC의 명령어 하나가 하던 일은 RISC 명령어 여러 개를 이어서 한다. 예: ARM, MIPS, SPARC, RISC-V. RISC 명령어는 변환 없이 바로 실행된다. [Ch2 p.7]

구분	CISC	RISC
시기	오래된 설계	CISC보다 새로운 개념
명령어	복잡, **가변 길이**, 강력한 명령어 많음	단순, 규격화, 명령어 수 적음
실행	명령어 → 마이크로코드 변환 → 마이크로 명령어 → 실행	명령어 → 실행
장점	어셈블리 프로그래밍이 쉬움, 컴파일러도 단순, 명령어 메모리 사용이 적음	CPU 설계가 쉬움, 명령어 집합이 작아 클럭 속도를 높이기 좋음
단점	CPU 설계가 훨씬 어려움	어셈블리 코드가 대체로 길어짐(컴파일러 설계 문제), 메모리를 많이 씀
예	Intel x86	ARM, MIPS, SPARC, RISC-V

요즘 x86(CISC) 프로세서 대부분은 내부를 **RISC 기법**으로 구현한다. 겉으로는 CISC 명령어를 받지만 안에서는 RISC 같은 단순한 동작으로 쪼개 실행한다는 뜻이다. [Ch2 p.7]

RISC-V

- RISC 원칙에 기반한 **공개 표준(open standard) ISA**. 학계와 산업계가 자유롭게 쓸 수 있다. [Ch2 p.8]
- UC Berkeley에서 개발한 **다섯 번째** RISC ISA 설계("V"가 5). 지금은 RISC-V 재단(riscv.org)이 관리한다.

- 현대 ISA의 전형적인 모습. 명령어 목록은 교재 그림 2.1(pp.70–71).
- 비슷한 ISA들이 임베디드 코어 시장에서 큰 비중을 차지한다(가전, 네트워크·저장 장치, 카메라, 프린터 등). 슬라이드는 "ARM의 대안?"이라는 물음을 던진다.

예제

다음 항목을 CISC와 RISC 중 어디의 특징인지 고르시오. (가) 가변 길이 명령어 (나) 높은 클럭 속도에 유리 (다) 명령어 메모리 사용 감소 (라) 어셈블리 코드가 길어짐 (마) CPU 설계가 어려움

풀이 보기

(가) CISC (나) RISC (다) CISC (라) RISC (마) CISC. [Ch2 p.6–7]

요령: CISC는 명령어 하나가 일을 많이 하므로 코드가 짧고 메모리를 덜 쓰지만 하드웨어가 복잡하다. RISC는 반대로 하드웨어가 단순하고 빠르지만 같은 일에 명령어가 더 많이 필요하다.

함정

"파이프라이닝은 ISA의 일부다" (X). 파이프라이닝, 추측 실행 같은 기법은 소프트웨어에 드러나지 않으므로 **마이크로아키텍처**에 속한다. [Ch2 p.5]

"x86은 CISC이므로 내부도 전부 CISC 방식으로 실행한다" (X). 현대 x86은 대부분 RISC 기법으로 구현된다. [Ch2 p.7]

RISC의 단점에 "메모리를 많이 씀(heavy use of memory)"이 있다. 장점만 외우지 말 것.

시험 포인트

- 명령어 = opcode(무엇을) + operand(대상) 정의
- ISA와 마이크로아키텍처 구분, 마이크로아키텍처 예 4가지
- CISC·RISC 비교표의 장단점, 예시 ISA
- RISC-V: 공개 표준, UC Berkeley 다섯 번째 RISC, RISC-V 재단 관리

스스로 확인

1. 명령어를 이루는 두 부분의 이름과 뜻을 쓰시오.

   답 보기

   opcode(연산 코드, 무엇을 하는지)와 operand(피연산자, 연산 대상). [Ch2 p.3]

2. 프로세서마다 ISA가 다른데도 서로 비슷한 이유 두 가지를 쓰시오.

   답 보기

   ① 비슷한 원리의 비슷한 하드웨어 기술 ② 모든 컴퓨터가 제공해야 하는 공통 기본 연산이 몇 가지 있음. [Ch2 p.3]

3. 특정 설계 제약과 목표 아래에서 ISA를 구현한 것을 무엇이라 하는가?

답 보기

마이크로아키텍처(microarchitecture). [Ch2 p.5]

4. O/X: 추측 실행은 소프트웨어에 드러나는 ISA의 기능이다.

답 보기

X. 소프트웨어에 드러나지 않는 하드웨어 기법이므로 마이크로아키텍처에 속한다.

5. CISC의 장점 두 가지와 단점 한 가지를 쓰시오.

답 보기

장점: 어셈블리 프로그래밍이 쉬움(컴파일러도 단순), 명령어 메모리 사용 감소. 단점: CPU 설계가 훨씬 어려움. [Ch2 p.6]

6. RISC가 클럭 속도를 높이기 좋은 이유를 한 줄로 쓰시오.

답 보기

명령어 집합이 작고 단순해서 하드웨어가 단순해지기 때문. [Ch2 p.7]

7. RISC-V는 어디에서 개발된 몇 번째 RISC 설계이며, 지금 누가 관리하는가?

답 보기

UC Berkeley의 다섯 번째 RISC ISA, RISC-V 재단(riscv.org)이 관리. [Ch2 p.8]

Ch2 · 2 슬라이드 p.9–10, 16

레지스터와 설계 원칙

이 단원을 마치면 RISC-V 산술 명령어의 3피연산자 규칙으로 식을 번역하고, 레지스터 파일·word·doubleword·즉시값·x0의 쓰임을 설명하며, 설계 원칙 1·2를 쓸 수 있다.

쉽게 말하면

레지스터는 CPU 책상 위에 놓인 작은 메모지 32장이다. 계산은 반드시 이 메모지에 적힌 값으로만 한다. 책장(메모리)에 있는 자료는 먼저 메모지로 옮겨 와야 계산할 수 있다(다음 단원).

RISC-V의 덧셈은 언제나 "두 메모지의 값을 더해 한 메모지에 적어라" 한 가지 모양뿐이다. 그래서 네 수를 더하려면 이 동작을 세 번 해야 한다.

비유가 안 맞는 지점 메모지 중 `x0` 한 장은 항상 0이 인쇄되어 있어 무엇을 써도 지워지지 않는다. 또 메모지를 더 많이 두면 편할 것 같지만, 하드웨어에서는 레지스터가 많아질수록 느려질 수 있다.

3피연산자 규칙

RISC-V 어셈블리 `add a, b, c`는 "두 변수 b와 c를 더해 그 합을 a에 넣어라"라는 뜻이다. RISC-V의 모든 산술 연산은 이 모양을 따른다. 변수가 정확히 **세 개**이고, 그중 **둘은 소스(source, 읽는 쪽)**, **하나는 목적지(destination, 쓰는 쪽)**다. [Ch2 p.9]

```
add a, b, c      // a = b + c
sub a, b, c      // a = b - c
```

그래서 네 변수 b, c, d, e의 합을 a에 넣으려면 명령어가 **세 개** 필요하다. [Ch2 p.9]

```
add a, b, c      // a = b + c
add a, a, d      // a = b + c + d
add a, a, e      // a = b + c + d + e
```

일반적으로 n개 변수를 더하려면 덧셈 명령어가 n−1개 필요하다. PDF 외 보충

핵심

설계 원칙 1: 단순성은 규칙성을 선호한다(Simplicity favors regularity). 규칙성은 구현을 단순하게 하고, 단순성은 더 낮은 비용으로 더 높은 성능을 가능하게 한다. 모든 산술 명령어의 피연산자를 3개로 고정한 것이 그 예다. [Ch2 p.9]

레지스터와 레지스터 파일

산술 명령어의 피연산자는 **레지스터(register)**여야 한다. 레지스터는 하드웨어 설계에서 쓰는 기본 저장 장치로, **프로세서 안에 있는** 저장 공간이다. [Ch2 p.10]

레지스터 파일(register file)	레지스터들을 모아 놓은 장치. RISC-V는 **32비트 레지스터 32개**(32 × 32비트)를 가지며 이름은 `x0` ~ `x31`이다. 자주 접근하는 데이터에 쓴다. [Ch2 p.10]
워드(word, w)	32비트 데이터. RISC-V 레지스터 하나의 크기와 같다.

더블워드(doubleword, dw) 64비트 데이터.

레지스터 번호가 0 ~ 31이므로 번호를 나타내는 데 5비트(2^5 = 32)가 필요하다. 이 사실은 명령어 형식 단원에서 레지스터 필드가 5비트인 이유가 된다. PDF 외 보충

> 핵심
>
> **설계 원칙 2: 작을수록 빠르다(Smaller is faster).** 레지스터가 많으면 클럭 주기가 길어질 수 있다. 레지스터 개수와 성능 사이에는 절충(tradeoff)이 있다. [Ch2 p.10]

설계 원칙 3(좋은 설계에는 좋은 타협이 필요하다)은 명령어 형식 단원(c2-5)에서 다룬다. [Ch2 p.23]

> 예제 (교재 pp.71–74)
>
> C 문장 `f = (g + h) - (i + j);`를 RISC-V로 번역하시오. 변수 f, g, h, i, j는 각각 레지스터 `x19`, `x20`, `x21`, `x22`, `x23`에 있다. 중간 결과는 임시 레지스터 `x5`, `x6`을 쓴다. 교재 기준
>
> 풀이 보기
>
> 명령어 하나는 연산 하나만 하므로 괄호 안부터 하나씩 계산한다.
>
> ```
> add x5, x20, x21     // x5 = g + h
> add x6, x22, x23     // x6 = i + j
> sub x19, x5, x6      // f = x5 - x6
> ```
>
> 슬라이드는 이 예제(pp.71–72의 변수 이름 판, pp.73–74의 레지스터 판)를 보라고 지정한다. [Ch2 p.9–10]

상수(즉시값) 피연산자와 addi

프로그램은 연산에 상수를 자주 쓴다(예: 배열 인덱스 1 증가). RISC-V는 상수를 명령어 안에 직접 넣을 수 있다. 이런 상수를 **즉시값(immediate value)**이라 하고, 즉시값을 더하는 명령어가 `addi`(add immediate)다. [Ch2 p.16]

```
addi x22, x22, 4     // x22 = x22 + 4  (4는 즉시값)
addi x9, x9, -1      // x9 = x9 - 1   (음수 즉시값으로 빼기)
```

- 상수 피연산자는 자주 나오므로 `addi`는 대부분의 RISC-V 프로그램에서 가장 많이 쓰이는 명령어 중 하나다. [Ch2 p.16]
- 즉시값을 쓰는 명령어는 상수를 메모리에서 읽어 오는 것보다 **훨씬 빠르고 에너지도 적게** 든다. [Ch2 p.16]
- 즉시값 뺄셈 명령어(subi)는 따로 없다. 음수 즉시값을 `addi`로 더하면 된다. PDF 외 보충

x0: 항상 0인 레지스터

RISC-V는 레지스터 `x0`을 값 0에 **배선으로 고정(hardwired)**하여 상수 0을 제공한다. `x0`에 무엇을 쓰더라도 값은 계속 0이다. 상수 0은 흔한 연산에 유용하다. 예를 들어 값의 부호를 바꿀 때 쓴다. [Ch2 p.16]

```
sub x1, x0, x1       // x1 = 0 - x1 = -x1  (부호 반전)
```

다른 활용 예(뒤 단원에서 다시 나옴): `beq x0, x0, L`은 0 == 0이 항상 참이므로 무조건 분기 [Ch2 p.29]. `add x5, x6, x0`은 x6의 값을 x5로 복사 PDF 외 보충.

함정

"a = b + c + d + e는 add 명령어 하나로 된다" (X). 산술 명령어는 소스 2개, 목적지 1개뿐이라 **3개**가 필요하다. [Ch2 p.9]

"레지스터가 많을수록 무조건 빠르다" (X). 설계 원칙 2에 따라 레지스터가 많으면 클럭 주기가 늘어날 수 있다. [Ch2 p.10]

RISC-V 레지스터는 32개이고 각각 32비트다. "32개"와 "32비트"를 섞어 쓰지 말 것. word = 32비트, doubleword = 64비트.

시험 포인트

- 설계 원칙 1·2의 영어 문장과 뜻(문장 그대로 묻기 쉬움)
- a = b + c + d + e에 필요한 명령어 수(3개)와 코드
- 레지스터 파일 32 × 32비트, x0~x31, word 32비트, doubleword 64비트
- addi의 장점(빠르고 에너지 적음), x0 = 항상 0, `sub x1, x0, x1` = 부호 반전

스스로 확인

1. 설계 원칙 1을 영어와 한국어로 쓰시오.

   답 보기

   Simplicity favors regularity, 단순성은 규칙성을 선호한다. [Ch2 p.9]

2. 설계 원칙 2를 쓰고, 레지스터 수와 관련해 그 이유를 한 줄로 쓰시오.

   답 보기

   Smaller is faster(작을수록 빠르다). 레지스터가 많으면 클럭 주기가 늘어날 수 있다. [Ch2 p.10]

3. a = b + c + d + e + f를 RISC-V 덧셈으로 계산하려면 add 명령어가 최소 몇 개 필요한가?

   답 보기

   4개. 변수 5개를 소스 2개짜리 덧셈으로 묶으면 5 − 1 = 4번.

4. RISC-V 레지스터 파일의 레지스터 개수와 각 크기, 64비트 데이터의 이름을 쓰시오.

   답 보기

   32개, 각 32비트(x0~x31). 64비트 = doubleword. [Ch2 p.10]

5. 빈칸: `sub x1, ___, x1`은 x1의 부호를 바꾼다.

   답 보기

   `x0`. x0은 항상 0이므로 0 − x1 = −x1. [Ch2 p.16]

6. 즉시값 명령어가 상수를 메모리에서 읽는 방식보다 좋은 점 두 가지를 쓰시오.

   답 보기

   훨씬 빠르고, 에너지를 덜 쓴다. [Ch2 p.16]

7. x20에서 5를 빼서 x20에 넣는 명령어 하나를 쓰시오.

   답 보기

   `addi x20, x20, -5`. 즉시값 빼기 명령어가 따로 없으므로 음수를 더한다.

Ch2 · 3 슬라이드 p.11–15

메모리 피연산자와 엔디언

이 단원을 마치면 배열 원소를 읽고 쓰는 RISC-V 코드를 쓰고, 바이트 오프셋을 계산하며, 빅·리틀 엔디언 배치를 그릴 수 있다.

쉽게 말하면

메모리는 번호가 붙은 아주 긴 사물함 줄이다. 사물함 한 칸에는 1바이트(8비트)가 들어간다. 정수 하나(word, 4바이트)는 사물함 네 칸을 차지하므로, 정수 배열에서 다음 원소는 번호가 4 큰 사물함에서 시작한다.

계산은 책상 위 메모지(레지스터)에서만 할 수 있으므로, 사물함 값을 메모지로 가져오는 것이 **load**, 메모지 값을 사물함에 넣는 것이 **store**다.

비유가 안 맞는 지점 진짜 사물함은 물건을 꺼내면 비지만, load는 값을 **복사**할 뿐이라 메모리의 값은 그대로 남는다. 또 4칸짜리 정수를 어느 순서로 넣느냐(엔디언)는 사물함 비유에는 없는 문제다.

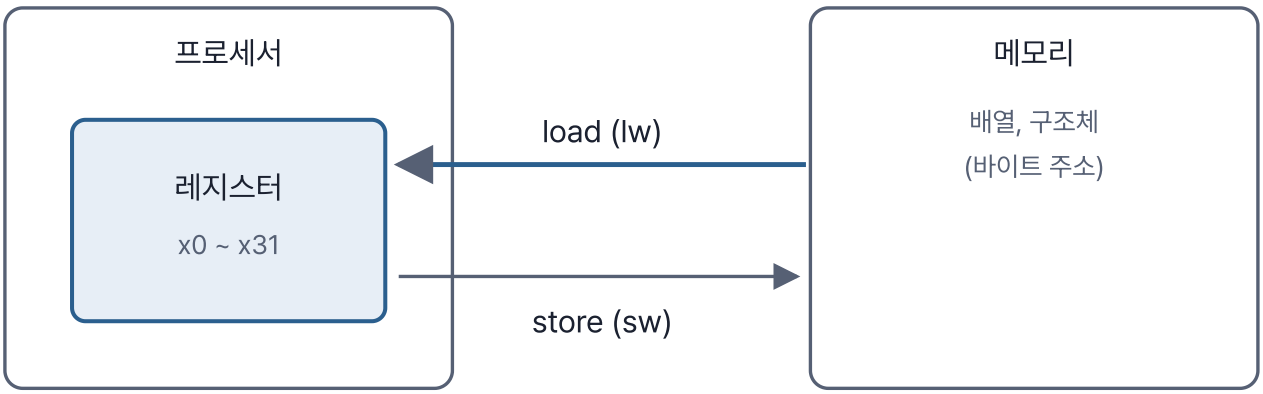
왜 load와 store가 필요한가

배열이나 구조체 같은 **복합 데이터(composite data)**는 보통 주 메모리(main memory)에 둔다. 레지스터는 개수가 적어서 이런 데이터를 담을 수 **없다(NOT)**. [Ch2 p.11]

산술 연산은 레지스터에서만 일어나므로, 메모리에 있는 데이터로 계산하려면 두 단계를 거친다. [Ch2 p.11]

1. 메모리에서 값을 레지스터로 **읽어 온다(load)**.
2. 계산 결과를 레지스터에서 메모리로 **저장한다(store)**.

레지스터와 메모리 사이에 데이터를 옮기는 명령어를 **데이터 전송 명령어(data transfer instruction)**라 한다. [Ch2 p.11]



load는 메모리 → 레지스터, store는 레지스터 → 메모리. 산술은 레지스터에서만 한다. [Ch2 p.11]

바이트 주소

메모리의 word에 접근하려면 명령어가 **메모리 주소(address)**를 알려 줘야 한다. 메모리는 크고 1차원인 배열이고, 주소는 그 배열의 인덱스이며 0부터 시작한다. [Ch2 p.12]

- 8비트(바이트)는 많은 프로그램에서 유용하므로, 거의 모든 아키텍처가 **바이트 하나하나에 주소를 붙인다(byte addressing)**.
- word(32비트 = 4바이트)의 주소는 그 word를 이루는 4바이트 중 하나의 주소와 같다.
- 그래서 RISC-V에서 **연속한 word의 주소는 4씩 차이** 난다(0, 4, 8, 12, ...). [Ch2 p.12]

```
A[i]의 주소 = A의 시작 주소(base) + i × 4 (4바이트 정수 배열)
```

엔디언

4바이트짜리 word를 주소 4개에 어떤 순서로 놓을지 정하는 방식이 **엔디언(endian)**이다. [Ch2 p.13]

빅 엔디언(big-endian) 데이터의 **최상위 바이트(most significant byte)**를 가장 낮은 주소에 놓는다.

리틀 엔디언(little-endian) 데이터의 **최하위 바이트(least significant byte)**를 가장 낮은 주소에 놓는다. **RISC-V는 리틀 엔디언**이다.

예: `0x00112233`을 주소 0부터 저장할 때. 최상위 바이트는 `00`, 최하위 바이트는 `33`이다. [Ch2 p.13]

주소	0	1	2	3
빅 엔디언	00	11	22	33
리틀 엔디언 (RISC-V)	33	22	11	00

16진수 두 자리가 1바이트다. 바이트 단위로 순서를 바꿀 뿐, 바이트 안의 숫자 순서(예: `33`을 `33`으로)는 바꾸지 않는다. PDF 외 보충

정렬 제한

RISC-V는 word가 메모리에서 **정렬(aligned)될 것을 요구하지 않는다(NOT)**. 즉 RISC-V에는 정렬 제한(alignment restriction)이 없어서, word가 4의 배수 주소에서 시작하지 **않아도 된다(NOT)**. [Ch2 p.13]

정렬 제한이 있는 ISA(예: MIPS)에서는 word가 4의 배수 주소에서만 시작해야 한다. RISC-V는 허용하지만, 정렬되지 않은 접근은 느릴 수 있다고 교재는 설명한다. PDF 외 보충

배열 예제: A[12] = h + A[8]

조건: h는 레지스터 `x21`, A는 정수(4바이트) 배열이고 시작 주소(base address)는 `x22`에 있다. [Ch2 p.14]

```
lw  x9, 32(x22)      // x9 = A[8]       (8 × 4 = 32)
add x9, x21, x9      // x9 = h + A[8]
sw  x9, 48(x22)      // A[12] = x9      (12 × 4 = 48)
```

lw (load word) `lw rd, 오프셋(rs1)`: 주소 = rs1 + 오프셋인 곳의 word를 rd로 읽는다.

sw (store word) `sw rs2, 오프셋(rs1)`: rs2의 값을 주소 = rs1 + 오프셋인 곳에 쓴다.

베이스 레지스터와 오프셋 괄호 안 레지스터(`x22`)가 시작 주소를 담은 **베이스 레지스터(base register)**, 괄호 앞 상수(`32`)가 **오프셋(offset)**이다. 오프셋은 **바이트** 단위다.

핵심은 오프셋 계산이다. A[8]은 인덱스가 8이지만 원소 하나가 4바이트이므로 시작 주소에서 8 × 4 = **32바이트** 떨어져 있다. A[12]는 12 × 4 = **48**이다. 데이터 전송 명령어 목록은 교재 p.70. [Ch2 p.14]

예제

(1) 배열 A의 시작 주소가 `x22`에 있을 때 A[5]를 `x10`에 읽는 명령어는? (2) `x10`의 값을 A[0]에 저장하는 명령어는? (3) 시작 주소가 1000일 때 A[3]의 주소는?

풀이 보기

(1) 5 × 4 = 20 → `lw x10, 20(x22)`

(2) 0 × 4 = 0 → `sw x10, 0(x22)`

(3) 1000 + 3 × 4 = 1012

예제

값 `0xA1B2C3D4`를 주소 100부터 저장했다. RISC-V(리틀 엔디언)에서 주소 100, 101, 102, 103에 들어가는 바이트를 쓰시오. 빅 엔디언이라면?

풀이 보기

바이트로 나누면 A1, B2, C3, D4이고 최하위 바이트는 D4다.

리틀 엔디언: 100 = D4, 101 = C3, 102 = B2, 103 = A1.

빅 엔디언: 100 = A1, 101 = B2, 102 = C3, 103 = D4.

레지스터와 메모리 비교, spill

- 레지스터는 메모리보다 **훨씬 빠르게** 접근할 수 있다. [Ch2 p.15]
- RISC-V에서는 산술 명령어가 메모리의 데이터에 직접 접근할 수 **없다(NOT)**.
- 메모리 데이터로 연산하려면 load와 store가 필요하므로 실행할 명령어가 늘어난다.
- 그래서 컴파일러는 변수를 최대한 레지스터에 두어야 한다. 자주 쓰지 않는 변수만 메모리로 내보낸다(**spill**). 레지스터 최적화가 중요하다. [Ch2 p.15]

스필(spill, spilling registers) 레지스터가 모자라서 덜 자주 쓰는 변수를 메모리에 내려 두는 것.

바이트·하프워드 load와 store PDF 외 보충

슬라이드 인코딩 표(Ch2 · 13)에는 lw·sw 말고도 `lb`, `lh`, `lbu`, `lhu`, `sb`, `sh`가 나온다. [Ch2 p.50] word(4바이트) 대신 바이트(1바이트)나 하프워드(halfword, 2바이트)를 옮기는 명령어다. 쓰는 법은 lw·sw와 같다(`lb rd, 오프셋(rs1)`, `sb rs2, 오프셋(rs1)`).

lb, lh (load byte, load halfword)	1바이트·2바이트를 읽어 32비트 레지스터에 넣을 때 **부호 확장(sign extension)**한다. 읽은 값의 맨 위 비트를 남은 위쪽 비트에 모두 복사한다는 뜻이다(자세한 것은 Ch2 · 4).
lbu, lhu (load byte unsigned, load halfword unsigned)	같은 크기를 읽되 위쪽 비트를 모두 0으로 채운다(**0 확장**).
sb, sh (store byte, store halfword)	레지스터의 **하위 1바이트·하위 2바이트만** 메모리에 쓴다. 나머지 메모리 바이트는 그대로다.

리틀 엔디언에서는 word의 **주소 +0 바이트가 최하위 바이트**다. 그래서 word가 있는 주소에 lb를 하면 그 word의 최하위 바이트를 읽고, sb는 레지스터의 최하위 바이트를 그 주소에 쓴다.

예제

word `0x1234ABF0`이 주소 200부터 저장되어 있고 `x5` = 200이다. 다음 각 명령어 뒤 `x6`의 값을 16진수 8자리로 쓰시오. (1) `lb x6, 0(x5)` (2) `lbu x6, 0(x5)` (3) `lh x6, 0(x5)` (4) `lb x6, 3(x5)` (5) `x7` = 0x11223344일 때 `sb x7, 1(x5)` 뒤 주소 200의 word는?

풀이 보기

주소	200	201	202	203
바이트(리틀 엔디언)	F0	AB	34	12

(1) 주소 200의 바이트 0xF0 = 1111 0000, 맨 위 비트가 1 → 부호 확장 → **0xFFFFFFF0** (−16)

(2) 같은 바이트를 0 확장 → **0x000000F0** (240)

(3) 주소 200 ~ 201의 하프워드 = 0xABF0(201이 위 바이트), 맨 위 비트 1 → **0xFFFFABF0**

(4) 주소 203의 바이트 0x12, 맨 위 비트 0 → **0x00000012** (lb여도 위쪽이 0)

(5) x7의 최하위 바이트 0x44만 주소 201에 쓴다. 바이트는 F0, 44, 34, 12 → word = **0x123444F0**

함정

A[8]을 읽을 때 `lw x9, 8(x22)` 라고 쓰면 틀린다. 주소는 바이트 단위이므로 인덱스 × 4 = **32**를 써야 한다. [Ch2 p.12, 14]

"RISC-V는 빅 엔디언이다" (X). **리틀 엔디언**이다. 리틀 엔디언은 **최하위** 바이트가 **가장 낮은 주소**. [Ch2 p.13]

"RISC-V에서 word는 반드시 4의 배수 주소에서 시작해야 한다" (X). 정렬 제한이 없다. [Ch2 p.13]

sw는 첫 번째 레지스터가 목적지가 아니다. `sw x9, 48(x22)`에서 x9은 **저장할 값**이고, 쓰이는 곳은 메모리다.

시험 포인트

- A[12] = h + A[8]의 세 줄 코드와 32, 48 계산
- 0x00112233의 빅·리틀 엔디언 배치, RISC-V = 리틀 엔디언
- 정렬 제한 없음(NOT 문장), 산술 명령어는 메모리에 직접 접근 못 함(NOT 문장)
- spill의 뜻, 레지스터가 메모리보다 빠름

스스로 확인

1. RISC-V에서 연속한 두 word의 주소 차이는?

   답 보기

   4. 바이트 주소이고 word는 4바이트. [Ch2 p.12]

2. h는 x21, 정수 배열 A의 시작 주소는 x22일 때 A[12] = h + A[8]을 RISC-V로 쓰시오.

   답 보기

   `lw x9, 32(x22)` / `add x9, x21, x9` / `sw x9, 48(x22)`. [Ch2 p.14]

3. 0x00112233을 리틀 엔디언으로 주소 0 ~ 3에 저장했을 때 주소 0의 바이트는?

   답 보기

   0x33. 최하위 바이트가 가장 낮은 주소. [Ch2 p.13]

4. 빅 엔디언의 정의를 쓰시오.

   답 보기

   데이터의 최상위 바이트를 가장 낮은 주소에 놓는 방식. [Ch2 p.13]

5. O/X: RISC-V에서 word는 4의 배수 주소에서 시작해야 한다.

   답 보기

   X. RISC-V는 정렬 제한이 없다. [Ch2 p.13]

6. 레지스터와 메모리 사이에 데이터를 옮기는 명령어를 무엇이라 하는가? 예 두 개도 쓰시오.

   답 보기

   데이터 전송 명령어. lw(load word), sw(store word). [Ch2 p.11, 14]

7. 컴파일러가 자주 쓰지 않는 변수를 메모리로 내보내는 것을 무엇이라 하는가?

   답 보기

   spill(스필). [Ch2 p.15]

8. 주소 0 ~ 3에 바이트 `02`, `01`, `FF`, `80`이 차례로 들어 있고 `x5` = 0이다. `lb x6, 2(x5)`와 `lbu x7, 2(x5)` 뒤 x6, x7의 값을 16진수 8자리로 쓰시오.

   답 보기

   주소 2의 바이트는 0xFF(맨 위 비트 1). lb는 부호 확장 → x6 = **0xFFFFFFFF**(−1). lbu는 0 확장 → x7 = **0x000000FF**(255). PDF 외 보충

9. `x8` = 0xCAFEBEEF일 때 `sh x8, 0(x5)`는 메모리 몇 바이트를 바꾸고, 주소 x5+0과 x5+1에 각각 어떤 바이트를 쓰는가? (리틀 엔디언)

   답 보기

   2바이트. 하위 하프워드 0xBEEF만 저장하고, 최하위 바이트가 낮은 주소로 가므로 x5+0 = **EF**, x5+1 = **BE**. 위 2바이트(CA, FE)는 저장하지 않는다. PDF 외 보충

Ch2 · 4 슬라이드 p.17–20

2진수와 2의 보수

이 단원을 마치면 2진수를 10진수로 바꾸고, 무부호·2의 보수 범위를 쓰며, ~x + 1로 음수를 만들고, 부호 확장을 할 수 있다.

쉽게 말하면

10진수는 자리마다 1, 10, 100, …을 곱하고, 2진수는 자리마다 1, 2, 4, 8, …을 곱한다. 컴퓨터는 모든 것을 0과 1로 저장하므로 2진수가 기본이다.

음수는 자동차 주행 거리계로 생각하면 쉽다. 0000에서 한 칸 뒤로 돌리면 9999가 된다. 2진수 4비트에서는 0000_2에서 1을 빼면 1111_2이 되므로 1111_2을 −1로 정한다. 이것이 2의 보수다.

비유가 안 맞는 지점 주행 거리계는 9999를 −1로 해석하지 않는다. 2의 보수는 "맨 왼쪽 비트가 1이면 음수"라는 약속을 더한 것이고, 같은 비트 묶음도 무부호로 보면 큰 양수다.

2진수와 자릿값

수는 어떤 진법(base)으로든 나타낼 수 있다. 예: $123_{10} = 1111011_2$. 2진수 한 자리가 컴퓨팅의 "원자"이며, 모든 정보는 2진 숫자, 즉 **비트(bit)**로 이루어진다. [Ch2 p.17]

진법 b에서 i번째 자리 숫자 d의 값은 다음과 같다. i는 0부터 시작하고 오른쪽에서 왼쪽으로 커진다. [Ch2 p.17]

$$i\text{번째 자리의 값} = d \times b^i$$

검산: $1111011_2 = 64 + 32 + 16 + 8 + 0 + 2 + 1 = 123$.

MSB(most significant bit, 최상위 비트) 가장 왼쪽 비트. 32비트 word에서는 31번 비트. [Ch2 p.17]

LSB(least significant bit, 최하위 비트) 가장 오른쪽 비트. 0번 비트.

무부호 수

RISC-V word는 32비트이므로 서로 다른 비트 패턴 2^{32}개를 나타낸다. 이것을 0 ~ 2^{32} − 1의 양수로만 해석한 것이 **무부호 수(unsigned number)**다. 각 비트 값에 2의 거듭제곱을 곱해 더한다. [Ch2 p.18]

$$\text{값} = x_{31}\times 2^{31} + x_{30}\times 2^{30} + \dots + x_1\times 2^1 + x_0\times 2^0$$

$$32\text{비트 무부호 범위: } 0 \sim 2^{32} - 1 \ (= 4{,}294{,}967{,}295)$$

부호 있는 수의 세 가지 표현

프로그램은 양수와 음수를 모두 계산하므로 둘을 구분하는 표현이 필요하다. 표현 방식은 ① 부호-크기 ② 1의 보수 ③ 2의 보수가 있고, 최종 해법은 **2의 보수**다. [Ch2 p.19]

표현	음수 만드는 법	−3의 4비트 2진	특징
부호-크기(sign and magnitude)	MSB만 1로 바꿈	1011	$+0(0000_2)$과 $-0(1000_2)$ 두 개
1의 보수(1's complement)	모든 비트 반전	1100	$+0(0000_2)$과 $-0(1111_2)$ 두 개
2의 보수(2's complement)	모든 비트 반전 후 +1	1101	0이 하나, 덧셈 회로를 그대로 씀 **(채택)**

표의 4비트 예와 "0이 두 개" 같은 특징 설명은 슬라이드에 없는 보충이다. PDF 외 보충

2의 보수

2의 보수에서 **MSB는 부호 비트(sign bit)**다. 1이면 음수, 0이면 양수(또는 0). 32비트의 범위는 $-2^{31} \sim 2^{31} - 1$이다. [Ch2 p.19]

$$\text{32비트 2의 보수 범위: } -2^{31} \sim 2^{31} - 1 \ (= -2{,}147{,}483{,}648 \sim 2{,}147{,}483{,}647)$$

$$\text{n비트 2의 보수 값} = -x_{n-1}\times 2^{n-1} + x_{n-2}\times 2^{n-2} + \dots + x_0\times 2^0$$

두 번째 식은 MSB의 자릿값만 음수로 보는 계산법이다. 예: 8비트 1111 1010 = −128 + 64 + 32 + 16 + 8 + 2 = −6. PDF 외 보충

음수 쪽이 하나 더 많다(-2^{31}에 대응하는 양수 $+2^{31}$은 없다). 0이 양수 쪽 패턴 하나를 쓰기 때문이다. PDF 외 보충

부정: ~x + 1 = −x

2의 보수에서 수의 부호를 바꾸려면 **모든 비트를 반전한 뒤 1을 더한다**. "~"는 보수 연산자로, 모든 비트를 0→1, 1→0으로 바꾼다. 예: $\sim(0001_2) = 1110_2$. [Ch2 p.20]

$$-x = \sim x + 1$$

예제 (단계별)

8비트로 +37을 −37로 바꾸시오. 다시 부정하면 +37이 되는지 확인하시오.

풀이 보기

① 37 = 32 + 4 + 1 → `0010 0101`

② 모든 비트 반전 → `1101 1010`

③ 1 더하기 → `1101 1011` = −37

검산: −128 + 64 + 16 + 8 + 2 + 1 = −37.

되돌리기: ~(1101 1011) = 0010 0100, +1 → `0010 0101` = 37. 같은 방법이 양방향으로 통한다.

예제 (교재 pp.84–86 유형)

32비트 2의 보수 `1111 1111 1111 1111 1111 1111 1111 1100`은 10진수로 얼마인가?

풀이 보기

MSB가 1이므로 음수. 부정해서 크기를 구한다. ~x = 0000 … 0000 0011, +1 → 0000 … 0000 0100 = 4. 따라서 원래 값은 **−4**.

부호 확장

부호 확장(sign extension)은 수를 더 많은 비트로 나타내는 것이다. 수의 값은 그대로 유지해야 하며, 방법은 **부호 비트를 왼쪽으로 복제**하는 것이다. [Ch2 p.20]

값	8비트	16비트로 부호 확장
+2	0000 0010	0000 0000 0000 0010
−2	1111 1110	1111 1111 1111 1110

−2를 0으로 채우면 0000 0000 1111 1110 = 254가 되어 값이 바뀐다. 그래서 음수는 1로 채워야 한다. 교재 예제는 pp.84–86. [Ch2 p.20]

RISC-V에서 12비트 즉시값(`addi`, `lw`의 오프셋 등)을 32비트 레지스터 값과 계산할 때 이 부호 확장을 한다. 그래서 `addi x9, x9, -1`의 즉시값 1111 1111 1111은 32비트 0xFFFFFFFF(−1)로 늘어난다. PDF 외 보충

연습 예제

1. 8비트 2의 보수 `1000 0000`의 값은?
2. −6을 8비트 2의 보수로 쓰시오.
3. −100을 16비트 2의 보수로 쓰시오.
4. 4비트 `1010`을 8비트로 부호 확장하고 값을 쓰시오.
5. 4비트 2의 보수의 범위는?

풀이 보기

1. −128 (MSB 자릿값 -2^7만 있음). 8비트의 가장 작은 수다.
2. 6 = 0000 0110 → 반전 1111 1001 → +1 → **1111 1010**.
3. 100 = 0000 0000 0110 0100 → 반전 1111 1111 1001 1011 → +1 → **1111 1111 1001 1100**.
4. MSB가 1이므로 1로 채움 → **1111 1010** = −6 (원래 1010_2 = −8 + 2 = −6, 값 유지).
5. $-2^3 \sim 2^3 - 1$ = **−8 ~ 7**.

함정

부정을 "비트 반전만"으로 끝내면 1의 보수다. 2의 보수는 반드시 **+1**까지 한다.

32비트 2의 보수 범위를 $-2^{31} - 1 \sim 2^{31}$로 쓰는 실수. 정답은 **$-2^{31} \sim 2^{31} - 1$**. 무부호는 $0 \sim 2^{32} - 1$. [Ch2 p.18–19]

부호 확장에서 무조건 0을 채우면 음수 값이 바뀐다. 부호 비트(MSB)를 복제한다. [Ch2 p.20]

같은 비트 패턴이라도 해석에 따라 값이 다르다. 0xFFFFFFFF는 2의 보수로 −1, 무부호로 $2^{32} - 1$. 이 차이가 c2-7의 blt·bltu에서 다시 나온다.

시험 포인트

- MSB·LSB 정의, 무부호 범위 $0 \sim 2^{32} - 1$, 2의 보수 범위 $-2^{31} \sim 2^{31} - 1$
- 부호 표현 3가지 이름과 최종 채택(2의 보수), MSB = 부호 비트(1 음수, 0 양수)
- ~x + 1 = −x로 음수 변환, 10진수 ↔ 2진수 변환
- 부호 확장 = 부호 비트를 왼쪽으로 복제, +2·−2 예

스스로 확인

1. 123을 2진수로 쓰시오.

   답 보기

   1111011_2 (64+32+16+8+2+1). [Ch2 p.17]

2. 부호 있는 수의 표현 방식 세 가지를 쓰고, 최종 채택된 것을 고르시오.

   답 보기

   부호-크기, 1의 보수, 2의 보수. 채택: 2의 보수. [Ch2 p.19]

3. 32비트 2의 보수 수의 범위를 2의 거듭제곱으로 쓰시오.

   답 보기

   $-2^{31} \sim 2^{31} - 1$. [Ch2 p.19]

4. 8비트 0000 1100(12)을 2의 보수로 부정한 결과를 쓰시오.

   답 보기

   반전 1111 0011, +1 → 1111 0100 (= −12).

5. 8비트 −2(1111 1110)를 16비트로 부호 확장하시오.

답 보기

1111 1111 1111 1110. 부호 비트 1을 왼쪽으로 복제. [Ch2 p.20]

6. O/X: 2의 보수에서 MSB가 0이면 그 수는 음수다.

답 보기

X. MSB 1이 음수, 0이 양수(또는 0). [Ch2 p.19]

7. ~(0001_2)의 결과를 쓰시오.

답 보기

1110_2. 모든 비트 반전. [Ch2 p.20]

8. 8비트 2의 보수 1111 1111의 10진수 값은?

답 보기

−1. 부정하면 0000 0000 + 1 = 1이므로.

Ch2 · 5 슬라이드 p.21–24

명령어 형식과 인코딩

이 단원을 마치면 R·I·S 형식의 필드와 비트 폭을 그리고, add·lw·addi·sw를 32비트 기계어로 인코딩하거나 거꾸로 해독할 수 있으며, 설계 원칙 3을 설명할 수 있다.

쉽게 말하면

명령어를 0과 1로 바꿀 때는 정해진 양식지에 칸을 채우는 것과 같다. 양식지는 모두 길이가 32칸으로 같고, "어떤 일인지" 칸, "어느 레지스터를 읽는지" 칸, "어디에 쓰는지" 칸이 정해진 자리에 있다. 하는 일에 따라 양식지가 몇 종류(R, I, S …) 있다.

비유가 안 맞는 지점 양식지 종류가 달라도 같은 이름의 칸(rs1, rs2, opcode, funct3)은 일부러 같은 위치에 둔다. 또 S형은 상수 칸 하나를 두 조각으로 나눠 떨어진 곳에 적는데, 보통 양식지에서는 하지 않는 일이다.

기계어와 명령어 형식

명령어는 2진수로 인코딩되며 이것을 **기계 코드(machine code)**라 한다. 명령어를 수로 나타낸 것이 **기계어(machine language)**다. [Ch2 p.21]

- RISC-V 명령어는 모두 **32비트 명령어 워드**로 인코딩된다. 규칙성(설계 원칙 1)의 예다.
- 형식, opcode, 레지스터 번호 등의 종류가 적다.
- 명령어의 배치(layout)를 **명령어 형식(instruction format)**이라 한다. [Ch2 p.21]

형식	쓰임	예
R형(R-type)	대부분의 산술·논리 명령어(즉시값 쓰는 것 제외)	add, sub, and, or, xor, sll
I형(I-type)	데이터 전송 중 load, 즉시값 산술	lw, addi, andi, slli
S형(S-type)	데이터 전송 중 store	sw

[Ch2 p.21]

필드 정의

필드 비트 폭과 뜻은 슬라이드 형식 그림에 있다. [Ch2 p.22–24]

opcode (7비트) 명령어의 기본 동작과 형식을 나타낸다.

rd (5비트) 목적지(destination) 레지스터 번호. 결과를 쓸 곳.

funct3 (3비트) 추가 opcode 필드. 같은 opcode 안에서 연산을 구분.

rs1 (5비트) 첫 번째 소스 레지스터 번호.

rs2 (5비트) 두 번째 소스 레지스터 번호.

funct7 (7비트) 추가 opcode 필드. 예: add와 sub 구분.

immediate (12비트) 즉시값 또는 주소 오프셋. 2의 보수로 해석하여 −2048 ~ 2047.

레지스터 필드가 5비트인 이유는 레지스터가 32개(2^5)이기 때문이다.

R형

funct7 31–25	rs2 24–20	rs1 19–15	funct3 14–12	rd 11–7	opcode 6–0

비트 수: 7 + 5 + 5 + 3 + 5 + 7 = 32. [Ch2 p.22–24] 칸 아래 숫자 31–25는 비트 범위 [31:25]와 같은 뜻이다(슬라이드 그림 일부는 [31-25]로 쓴다).

예제 (단계별 인코딩)

`add x9, x20, x21`을 32비트 기계어로 바꾸시오. [Ch2 p.22]

풀이 보기

① 형식: add는 즉시값이 없는 산술 → R형.

② 필드 값(10진): funct7 = 0, rs2 = 21, rs1 = 20, funct3 = 0, rd = 9, opcode = 51 (0110011). 어셈블리의 순서는 `add rd, rs1, rs2`이므로 rd = x9, rs1 = x20, rs2 = x21. 슬라이드는 이 단계를 십진수 칸 `0 | 21 | 20 | 0 | 9 | 51`로 보여 준다. [Ch2 p.22]

③ 각 필드를 정해진 비트 수의 2진수로: 21 = 10101_2, 20 = 10100_2, 9 = 01001_2.

0000000 funct7	10101 rs2 = 21	10100 rs1 = 20	000 funct3	01001 rd = 9	0110011 opcode

④ 이어 붙이기: `0000 0001 0101 1010 0000 0100 1011 0011` = **0x015A04B3**.

슬라이드 p.21의 2진수 `00000001010110100000010010110011`과 같다.

함정 (슬라이드 원문 표기 오류)

슬라이드 p.21 텍스트는 이 2진수를 `add x0, x20, x21`이라고 적었다. 그런데 rd 칸(비트 [11:7])이 `01001` = 9이므로 실제로는 `add x9, x20, x21`이고, p.22의 예제와도 일치한다. p.21의 "x0"은 슬라이드 원문 표기 오류다(2진수는 x9). [Ch2 p.21–22]

I형

immediate[11:0] 31–20	rs1 19–15	funct3 14–12	rd 11–7	opcode 6–0

R형의 funct7과 rs2 자리(7 + 5 = 12비트)를 12비트 즉시값 하나로 쓴다. load에서는 rs1이 베이스 레지스터, 즉시값이 오프셋, rd가 읽은 값을 받을 레지스터다. [Ch2 p.23]

예제 (I형 인코딩)

(1) `lw x9, 32(x22)` (2) `addi x22, x22, 1`을 인코딩하시오. lw: opcode 0000011, funct3 010. addi: opcode 0010011, funct3 000. [Ch2 p.23]

풀이 보기

(1) imm = 32 = 0000 0010 0000, rs1 = 22 = 10110, rd = 9 = 01001.

000000100000	10110	010	01001	0000011
imm = 32	rs1 = 22	funct3	rd = 9	opcode

→ `0000 0010 0000 1011 0010 0100 1000 0011` = **0x020B2483**

(2) imm = 1, rs1 = rd = 22.

000000000001	10110	000	10110	0010011
imm = 1	rs1 = 22	funct3	rd = 22	opcode

→ `0000 0000 0001 1011 0000 1011 0001 0011` = **0x001B0B13**

참고: 음수 즉시값은 12비트 2의 보수로 넣는다. `addi x9, x9, -1`의 imm 칸은 1111 1111 1111이다.

핵심

설계 원칙 3: 좋은 설계에는 좋은 타협이 필요하다(Good design demands good compromises). 형식이 여러 개면 해독(decoding)이 복잡해지지만, 대신 모든 명령어를 똑같이 32비트로 유지할 수 있다. 형식들은 가능한 한 비슷하게 만든다. [Ch2 p.23]

예를 들어 lw에 R형을 쓰면 오프셋을 5비트 칸에 넣어야 해서 0 ~ 31밖에 표현하지 못한다. 그래서 형식을 하나 더 만들어 12비트 즉시값을 얻는 대신 형식 종류가 늘어나는 것을 받아들였다. PDF 외 보충

S형

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
31–25	24–20	19–15	14–12	11–7	6–0

store는 결과를 레지스터에 쓰지 않으므로 rd가 필요 없다. 대신 레지스터를 두 개 읽는다: rs1 = 베이스 주소, rs2 = 저장할 값. [Ch2 p.24]

S형은 **12비트 즉시값을 두 필드로 나눈다**(위 7비트 imm[11:5], 아래 5비트 imm[4:0]). 이유: [Ch2 p.24]

- **rs1과 rs2 필드를 모든 형식에서 같은 자리**에 두기 위해서다.
- 마찬가지로 **opcode와 funct3** 필드도 모든 형식에서 크기와 위치가 같다.

덕분에 하드웨어는 명령어 종류를 다 해독하기 전에 비트 [19:15], [24:20]으로 레지스터를 미리 읽을 수 있다. 이 점은 Ch4 데이터패스에서 다시 나온다. PDF 외 보충

예제 (S형 인코딩)

`sw x9, 48(x22)`를 인코딩하시오. sw: opcode 0100011, funct3 010. [Ch2 p.24]

풀이 보기

① 48을 12비트로: 0000 0011 0000.

② 나누기: imm[11:5] = 위 7비트 `0000001`, imm[4:0] = 아래 5비트 `10000`. 검산: $0000001_2 \times 32 + 10000_2(16) = 32 + 16 = 48$.

③ rs2 = 저장할 값 x9 = 01001, rs1 = 베이스 x22 = 10110.

0000001 imm[11:5]	01001 rs2 = 9	10110 rs1 = 22	010 funct3	10000 imm[4:0]	0100011 opcode

→ `0000 0010 1001 1011 0010 1000 0010 0011` = **0x029B2823**

명령어	형식	opcode	funct3	funct7
add	R	0110011	000	0000000
sub	R	0110011	000	0100000
addi	I	0010011	000	없음
lw	I	0000011	010	없음
sw	S	0100011	010	없음

opcode 값표 [Ch2 p.22–24, p.50]. 전체 인코딩 표는 Ch2 · 13에 있다. 시험에서는 보통 표를 주고 칸을 채우게 하므로, 값 자체보다 필드 순서와 비트 폭을 확실히 외운다.

예제 (교재 pp.91–92)

A[30] = h + A[30] + 1; 에서 A의 시작 주소는 `x10`, h는 `x21`이다. RISC-V 코드를 쓰고 lw와 sw의 필드 값을 10진수로 쓰시오. 교재 기준

풀이 보기

```
lw   x9, 240(x10)    // x9 = A[30]   (30 × 4 = 240)
add  x9, x21, x9     // x9 = h + A[30]
addi x9, x9, 1       // x9 = h + A[30] + 1
sw   x9, 240(x10)    // A[30] = x9
```

lw (I형): imm 240, rs1 10, funct3 2, rd 9, opcode 3 → 0x0F052483.

add (R형): funct7 0, rs2 9, rs1 21, funct3 0, rd 9, opcode 51 → 0x009A84B3.

addi (I형): imm 1, rs1 9, funct3 0, rd 9, opcode 19 → 0x00148493.

sw (S형): 240 = 0000 1111 0000 → imm[11:5] = 0000111 (7), rs2 9, rs1 10, funct3 2, imm[4:0] = 10000 (16), opcode 35 → 0x0E952823.

함정

어셈블리 순서와 비트 순서가 다르다. `add rd, rs1, rs2`로 쓰지만 기계어는 왼쪽부터 funct7, **rs2**, rs1, funct3, rd, opcode다. rs2가 rs1보다 왼쪽(높은 비트)에 있다.

sw에는 rd가 없다. `sw x9, 48(x22)`의 x9는 **rs2**(저장할 값)로 들어간다.

S형 즉시값을 나눌 때 위 7비트가 비트 [31:25], 아래 5비트가 비트 [11:7]에 들어간다. 반대로 넣지 말 것.

시험 포인트

- RISC-V 명령어는 모두 32비트(규칙성), 형식 이름과 용도(R·I·S)
- R·I·S 필드 순서와 비트 폭, 각 필드의 의미
- add x9, x20, x21 ↔ 0000000 10101 10100 000 01001 0110011 인코딩·해독
- S형 즉시값을 둘로 나누는 이유(rs1·rs2·opcode·funct3 위치 통일)
- 설계 원칙 3 문장과 뜻

스스로 확인

1. R형의 필드를 31번 비트 쪽부터 순서대로 쓰고 각 비트 수를 쓰시오.

   답 보기

   funct7(7), rs2(5), rs1(5), funct3(3), rd(5), opcode(7). [Ch2 p.22]

2. `0000000 10101 10100 000 01001 0110011`을 어셈블리로 쓰시오.

   답 보기

   `add x9, x20, x21`. rs2 = 21, rs1 = 20, rd = 9, R형 add. 슬라이드 p.21의 "x0"은 원문 표기 오류(2진수는 x9). [Ch2 p.21–22]

3. 설계 원칙 3을 영어와 한국어로 쓰시오.

   답 보기

   Good design demands good compromises, 좋은 설계에는 좋은 타협이 필요하다. [Ch2 p.23]

4. S형이 12비트 즉시값을 두 필드로 나누는 이유를 쓰시오.

   답 보기

   rs1·rs2 필드(그리고 opcode·funct3)를 모든 형식에서 같은 위치에 두기 위해. [Ch2 p.24]

5. lw와 sw는 각각 어떤 형식인가?

   답 보기

   lw = I형, sw = S형. [Ch2 p.21]

6. I형 즉시값의 비트 수와 표현 범위(2의 보수)는?

   답 보기

   12비트, −2048 ~ 2047(-2^{11} ~ $2^{11}-1$).

7. `sw x5, 20(x6)`에서 imm[11:5]와 imm[4:0]의 값을 2진수로 쓰시오.

   답 보기

   20 = 0000 0001 0100 → imm[11:5] = 0000000, imm[4:0] = 10100. rs2 = 5, rs1 = 6.

Ch2 · 6 슬라이드 p.25–27

논리 연산과 시프트

이 단원을 마치면 and·or·xor와 시프트 명령어의 결과를 비트 단위로 계산하고, 마스크·NOT 만들기·srl과 sra의 차이를 설명할 수 있다.

쉽게 말하면

논리 연산은 두 수를 숫자로 보지 않고 32개의 전구 줄로 본다. 같은 자리 전구끼리 짝지어 규칙대로 켜고 끈다. AND는 "둘 다 켜졌을 때만 켜기", OR는 "하나라도 켜졌으면 켜기", XOR는 "서로 다를 때만 켜기"다.

시프트는 전구 줄 전체를 왼쪽이나 오른쪽으로 몇 칸 미는 것이다. 밀려 나간 칸은 버리고 빈 칸은 채운다.

비유가 안 맞는 지점 오른쪽 시프트에서 빈 칸을 0으로 채울지 부호 비트로 채울지는 명령어(srl과 sra)에 따라 다르다. 전구 비유만으로는 이 차이가 드러나지 않는다.

비트 단위 논리 연산

RISC-V는 **비트 단위 논리 연산(bitwise logical operation)** 명령어를 제공한다. word 안에서 비트 묶음을 꺼내거나 끼워 넣는 데 유용하다. [Ch2 p.25]

연산	C 연산자	레지스터형(R)	즉시값형(I)	규칙
AND	&	and	andi	둘 다 1이면 1
OR	|	or	ori	하나라도 1이면 1
XOR	^	xor	xori	다르면 1, 같으면 0
왼쪽 시프트	<<	sll	slli	왼쪽으로 밀고 오른쪽은 0
논리 오른쪽 시프트	>> (무부호)	srl	srli	오른쪽으로 밀고 왼쪽은 0
산술 오른쪽 시프트	>> (부호)	sra	srai	오른쪽으로 밀고 왼쪽은 부호 비트

C 연산자 열은 교재 표를 따랐다. 교재 기준

AND: 마스크

AND는 word의 비트를 **가리는(mask)** 데 유용하다. 일부 비트만 골라 남기고 나머지는 0으로 지운다. 남기고 싶은 자리에 1, 지우고 싶은 자리에 0을 둔 수를 **마스크(mask)**라 한다. [Ch2 p.26]

```
and x9, x10, x11
x10 = 00000000 00000000 00001101 11000000   (0x00000DC0)
x11 = 00000000 00000000 00111100 00000000   (0x00003C00)
x9  = 00000000 00000000 00001100 00000000   (0x00000C00)
```

OR: 비트 켜기

OR는 word에 비트를 **포함시키는(include)** 데 유용하다. 일부 비트를 1로 만들고 나머지는 그대로 둔다. [Ch2 p.26]

```
or  x9, x10, x11
x10 = 00000000 00000000 00001101 11000000   (0x00000DC0)
x11 = 00000000 00000000 00111100 00000000   (0x00003C00)
x9  = 00000000 00000000 00111101 11000000   (0x00003DC0)
```

XOR와 NOT

XOR는 **차이를 구하는(differencing)** 연산이다. 두 비트가 같으면 0, 다르면 1이 된다. [Ch2 p.27]

```
xor x9, x10, x12
x10 = 00000000 00000000 00001101 11000000   (0x00000DC0)
x12 = 11111111 11111111 11111111 11111111   (0xFFFFFFFF)
x9  = 11111111 11111111 11110010 00111111   (0xFFFFF23F)
```

RISC-V에는 NOT 명령어가 따로 없고, **NOT은 XOR로 만든다**. 모든 비트가 1인 수($111\ldots1_2$)와 XOR하면 각 비트가 1과 비교되어, 0이던 비트는 1이 되고 1이던 비트는 0이 된다. 위 예에서 x9는 x10을 반전한 값이다. [Ch2 p.27]

NOT x = x XOR $111\ldots1_2$ (즉 `xori rd, rs1, -1`)

즉시값 –1은 12비트로 1111 1111 1111이고 부호 확장하면 32비트 전부 1이므로 `xori` 하나로 NOT을 만들 수 있다. PDF 외 보충

시프트

시프트 명령어는 word의 모든 비트를 왼쪽이나 오른쪽으로 옮긴다. 예: sll/slli, srl/srli, sra/srai. 즉시값을 쓰는 시프트(slli 등)는 I형이며, 시프트할 칸 수가 즉시값 imm[11:0] 자리에 들어간다. [Ch2 p.25]

32비트 값은 0 ~ 31칸만 의미가 있으므로 실제로는 즉시값의 아래 5비트가 칸 수이고, 위 7비트는 srli(0000000)와 srai(0100000)를 구분하는 데 쓴다. 교재 기준

sll (shift left logical, 논리 왼쪽 시프트) — 왼쪽으로 밀고 오른쪽 빈 칸은 0.

srl (shift right logical, 논리 오른쪽 시프트) — 오른쪽으로 밀고 왼쪽 빈 칸은 0.

sra (shift right arithmetic, 산술 오른쪽 시프트) — 오른쪽으로 밀고 왼쪽 빈 칸은 **부호 비트**로 채운다. 예: `1100 0011` → (1칸) `1110 0001`. [Ch2 p.25]

8비트 `1100 0011`을 1칸	결과	왼쪽 빈 칸
srl	0110 0001	0
sra	1110 0001	부호 비트(1)

sra 예는 슬라이드, srl 비교는 보충이다. PDF 외 보충

시프트와 2의 거듭제곱

왼쪽으로 i칸 시프트하면 2^i를 곱한 것과 같다(넘치지 않을 때). 오른쪽 시프트는 2^i로 나눈 몫과 같다(무부호는 srl, 부호 있는 수는 sra). 그래서 컴파일러는 배열 인덱스 × 4를 곱셈 대신 `slli x, i, 2`로 계산한다(c2-7의 while 예제). PDF 외 보충

x << i = x × 2^i · x >> i = x ÷ 2^i의 몫

예제

x10 = 0x00000DC0일 때 다음 결과를 16진수로 쓰시오. (1) `slli x9, x10, 4` (2) `srli x9, x10, 4` (3) `andi x9, x10, 0xFF` (4) `ori x9, x10, 0x00F`

풀이 보기

0xDC0 = 1101 1100 0000 (10진 3520).

(1) 4칸 왼쪽 = 16진 한 자리 왼쪽 → **0x0000DC00** (3520 × 16 = 56320).

(2) 4칸 오른쪽 = 16진 한 자리 오른쪽 → **0x000000DC** (3520 ÷ 16 = 220).

(3) 아래 8비트만 남김: 1100 0000 → **0x000000C0**.

(4) 아래 4비트를 1로: 0xDC0 | 0x00F → **0x00000DCF**.

예제

8비트 `1000 0100` (2의 보수 −124)을 2칸 sra, 2칸 srl한 결과와 값을 쓰시오.

풀이 보기

sra: 왼쪽에 1을 채움 → `1110 0001` = −128 + 64 + 32 + 1 = −31 (−124 ÷ 4 = −31).

srl: 왼쪽에 0을 채움 → `0010 0001` = 33 (무부호 132 ÷ 4 = 33).

부호 있는 수를 나눌 때 sra를 써야 부호가 유지된다.

함정

"RISC-V에는 NOT 명령어가 있다" (X). NOT은 $111\ldots1_2$과 XOR해서 만든다. [Ch2 p.27]

srl과 sra를 혼동하지 말 것. 둘 다 오른쪽 시프트지만 sra만 부호 비트로 채운다. 왼쪽 시프트는 sll 하나뿐이다(산술 왼쪽 시프트는 따로 없음).

AND는 비트를 0으로 지우는 데(마스크), OR는 비트를 1로 켜는 데 쓴다. 용도를 거꾸로 쓰지 말 것. [Ch2 p.26]

시험 포인트

- 0x00000DC0과 0x00003C00의 and = 0x00000C00, or = 0x00003DC0, xor 0xFFFFFFFF = 0xFFFFF23F
- AND = 마스크(선택, 나머지 0), OR = 일부를 1로, XOR = 같으면 0 다르면 1
- NOT 대신 XOR $111\ldots1_2$
- sra = 부호 비트로 채움(1100 0011 → 1110 0001), 시프트 즉시값형은 I형

스스로 확인

1. x10 = 0x00000DC0, x11 = 0x00003C00일 때 `and x9, x10, x11`의 결과는?

   답 보기

   0x00000C00. [Ch2 p.26]

2. 같은 값으로 `or x9, x10, x11`의 결과는?

   답 보기

   0x00003DC0. [Ch2 p.26]

3. RISC-V에서 NOT 연산은 어떻게 하는가?

   답 보기

   모든 비트가 1인 값과 XOR한다. 예 0x00000DC0 XOR 0xFFFFFFFF = 0xFFFFF23F. [Ch2 p.27]

4. 일부 비트만 남기고 나머지를 0으로 지우는 데 쓰는 연산과 그 기능의 이름은?

   답 보기

   AND, 마스크(mask). [Ch2 p.26]

5. 8비트 1100 0011을 sra로 1칸 시프트한 결과는?

   답 보기

   1110 0001. 왼쪽을 부호 비트 1로 채움. [Ch2 p.25]

6. x5 = 3일 때 `slli x6, x5, 3` 후 x6의 값은?

   답 보기

   24. 3칸 왼쪽 = × 2^3.

7. O/X: slli는 R형 명령어다.

   답 보기

   X. 즉시값을 쓰므로 I형. 레지스터로 칸 수를 주는 sll이 R형. [Ch2 p.25]

Ch2 · 7 슬라이드 p.28–31

분기와 조건문·반복문

이 단원을 마치면 beq·bne·blt·bge·bltu·bgeu의 뜻을 쓰고, C의 if-else와 while을 RISC-V 코드로 번역하며, 부호·무부호 비교 결과를 판단할 수 있다.

쉽게 말하면

프로그램은 보통 위에서 아래로 한 줄씩 실행된다. 분기 명령어는 "조건이 맞으면 저기 표시해 둔 줄로 건너뛰어라"라는 표지판이다. 표시해 둔 줄의 이름을 **레이블**이라 한다.

if-else와 while 같은 C 문장은 모두 이 표지판 몇 개로 바뀐다. 반복문은 아래에서 위로 되돌아가는 표지판을 쓴 것이다.

비유가 안 맞는 지점 표지판은 보통 "같으면 가라"처럼 C 조건 그대로 쓰지 않는다. 컴파일러는 흔히 조건을 **뒤집어** "다르면 else로 건너뛰어라"로 쓴다. 이 점이 번역에서 가장 헷갈린다.

조건 분기 beq와 bne

컴퓨터가 단순한 계산기와 다른 점은 **판단(decision)**을 할 수 있다는 것이다. 판단은 보통 if 문으로 나타낸다. 조건이 참이면 레이블이 붙은 명령어로 **분기(branch)**하고, 거짓이면 다음 명령어를 순서대로 실행한다. [Ch2 p.28]

```
beq rs1, rs2, L1     // if (rs1 == rs2) L1로 분기  (branch if equal)
bne rs1, rs2, L1     // if (rs1 != rs2) L1로 분기  (branch if not equal)
```

이 두 명령어를 **조건 분기(conditional branch)**라 한다. [Ch2 p.28]

레이블(label) 명령어 앞에 `Else:`처럼 붙인 이름. 분기 목표를 가리킨다. 어셈블러가 실제 주소(거리)로 바꾼다.

무조건 분기(unconditional branch) 조건 없이 항상 건너뛰는 것. RISC-V에서는 `beq x0, x0, L`로 만든다. x0 == x0(0 == 0)은 항상 참이기 때문이다. [Ch2 p.29]

if-else 번역

C 코드 (f, g, h, i, j는 각각 `x19`, `x20`, `x21`, `x22`, `x23`): [Ch2 p.29]

```
if (i == j) f = g + h;
else        f = g - h;
```

번역 단계:

1. 조건을 **뒤집는다**: i == j가 아니면(i != j) else 부분으로 건너뛴다 → `bne x22, x23, Else`.
2. 건너뛰지 않았다면 i == j이므로 then 부분을 실행 → `add x19, x20, x21`.
3. then이 끝나면 else 부분을 건너뛰어 끝으로 가야 한다 → 무조건 분기 `beq x0, x0, Exit`.
4. `Else:` 레이블에 else 부분 → `sub x19, x20, x21`.
5. `Exit:` 레이블은 if-else 다음 문장 위치.

```
      bne x22, x23, Else   // i != j 이면 Else로
      add x19, x20, x21    // f = g + h  (i != j 이면 건너뜀)
      beq x0, x0, Exit     // 0 == 0 이므로 항상 Exit로 (무조건 분기)
Else: sub x19, x20, x21    // f = g - h  (i == j 이면 건너뜀)
Exit:
```

[Ch2 p.29]

조건을 뒤집는 이유: 조건이 참일 때 바로 아래의 then 부분이 실행되도록 하면, 분기를 하나 덜 쓰는 자연스러운 배치가 되기 때문이다.
PDF 외 보충

while 번역

C 코드 (i, k, 배열 save의 시작 주소는 각각 `x22`, `x24`, `x25`): [Ch2 p.30]

```
while (save[i] == k)
    i += 1;
```

번역 단계:

1. save[i]의 주소 = 시작 주소 + i × 4. i × 4는 2칸 왼쪽 시프트로 구한다 → `slli x10, x22, 2`.
2. 시작 주소를 더한다 → `add x10, x10, x25`.
3. save[i]를 읽는다 → `lw x9, 0(x10)`. 주소를 이미 x10에 다 계산했으므로 오프셋은 0.
4. 조건을 뒤집어 반복 탈출: save[i] != k이면 Exit로 → `bne x9, x24, Exit`.
5. 반복 몸체 i += 1 → `addi x22, x22, 1`.
6. 처음으로 돌아가 다시 검사 → `beq x0, x0, Loop`.

```
Loop: slli x10, x22, 2     // 임시 x10 = i * 4
      add  x10, x10, x25   // x10 = save[i]의 주소
      lw   x9, 0(x10)      // 임시 x9 = save[i]
      bne  x9, x24, Exit   // save[i] != k 이면 Exit로
      addi x22, x22, 1     // i = i + 1
      beq  x0, x0, Loop    // Loop로 (무조건 분기)
Exit:
```

[Ch2 p.30]

여기서 lw의 오프셋에 i × 4를 직접 쓸 수 없는 이유: 오프셋은 명령어에 고정된 상수인데 i는 실행 중에 바뀌는 변수이기 때문이다. 그래서 주소를 레지스터로 계산한다. PDF 외 보충

크기 비교 분기: blt, bge, bltu, bgeu

같다·다르다 외에 크기 비교로 분기하는 명령어도 있다. [Ch2 p.31]

명령어	뜻	비교 방식
`blt rs1, rs2, L1`	rs1 < rs2이면 분기 (branch if less than)	부호 있음(2의 보수)
`bge rs1, rs2, L1`	rs1 >= rs2이면 분기 (branch if greater or equal)	부호 있음
`bltu rs1, rs2, L1`	rs1 < rs2이면 분기	무부호(unsigned)
`bgeu rs1, rs2, L1`	rs1 >= rs2이면 분기	무부호

같은 비트 패턴이라도 부호 비교와 무부호 비교의 결과가 다를 수 있다. 슬라이드 예: [Ch2 p.31]

```
rs1 = 11111111 11111111 11111111 11111111   (0xFFFFFFFF)
rs2 = 00000000 00000000 00000000 00000001   (1)

blt  rs1, rs2, L1   // 부호: -1 < 1 → 참 → L1로 분기
bltu rs1, rs2, L1   // 무부호: 2^32 - 1 < 1 → 거짓 → 분기 안 함
```

blt는 rs1을 2의 보수로 보아 −1로 읽고, bltu는 무부호로 보아 $2^{32} - 1$(가장 큰 수)로 읽는다. [Ch2 p.31]

>와 <=용 명령어는 따로 없다. 피연산자 순서를 바꿔 쓴다. 예: `if (a > b)` 는 `blt b, a, L`. PDF 외 보충

switch 문과 분기 주소 표

switch/case 문은 if-else를 여러 개 이어서 만들 수도 있지만, **분기 주소 표(branch address table, 줄여서 branch table)**를 쓰면 더 효율적이다. 프로그램은 표에서 해당 번호의 칸을 찾아(index) 거기에 적힌 주소로 분기하기만 하면 된다. [Ch2 p.31]

분기 주소 표는 각 case 코드의 시작 주소를 배열로 모아 둔 것이다. 프로그램은 case 번호로 표에서 주소를 읽어 레지스터에 넣고, 레지스터에 든 주소로 점프하는 명령어(`jalr` , 프로시저 단원에서 다룸)로 이동한다. PDF 외 보충

예제

`if (a < b) c = 1; else c = 0;` 을 RISC-V로 번역하시오. a, b, c는 x5, x6, x7이고 부호 있는 정수다.

풀이 보기

조건을 뒤집으면 a >= b일 때 Else로 간다 → bge.

```
      bge  x5, x6, Else    // a >= b 이면 Else로
      addi x7, x0, 1       // c = 1
      beq  x0, x0, Exit    // 무조건 Exit로
Else: addi x7, x0, 0       // c = 0
Exit:
```

`addi x7, x0, 1` 은 0 + 1을 x7에 넣으므로 상수 1을 넣는 방법이다.

예제

x5 = 0xFFFFFFFE, x6 = 0x00000003일 때 다음 중 분기하는 것을 고르시오. (가) `blt x5, x6, L` (나) `bltu x5, x6, L` (다) `bge x5, x6, L` (라) `bgeu x5, x6, L`

풀이 보기

부호로 보면 x5 = −2, x6 = 3. 무부호로 보면 x5 = 2^{32} − 2(아주 큼), x6 = 3.

(가) −2 < 3 참 → 분기. (나) 2^{32} − 2 < 3 거짓. (다) −2 >= 3 거짓. (라) 2^{32} − 2 >= 3 참 → 분기.

정답: (가), (라).

함정

if (i == j)를 번역할 때 beq 대신 **bne**로 Else에 건너뛴다(조건 뒤집기). while (save[i] == k)도 **bne**로 Exit에 간다. [Ch2 p.29–30]

if-else에서 then 부분 뒤의 `beq x0, x0, Exit`를 빠뜨리면 then을 실행한 뒤 else까지 실행된다.

while의 맨 끝은 `beq x0, x0, Loop`로 **처음(Loop)**에 돌아가 조건을 다시 검사한다. Exit로 가는 것은 bne 쪽이다. [Ch2 p.30]

0xFFFFFFFF와 1을 비교하면 blt는 분기하고 bltu는 분기하지 않는다. 부호(blt)와 무부호(bltu)를 거꾸로 외우지 말 것. [Ch2 p.31]

시험 포인트

- beq·bne 뜻, 조건 분기 정의, 무조건 분기 = `beq x0, x0, 레이블`
- if-else 코드 4줄(bne → add → beq → Else: sub)과 빈칸 채우기
- while 코드 6줄 순서: slli → add → lw → bne → addi → beq
- blt·bge(부호), bltu·bgeu(무부호), 0xFFFFFFFF vs 1 결과
- switch = 분기 주소 표(branch table)

스스로 확인

1. RISC-V에서 무조건 분기를 beq로 어떻게 만드는가?

   답 보기

   `beq x0, x0, 레이블`. 0 == 0은 항상 참. [Ch2 p.29]

2. 빈칸: `___ x22, x23, Else` / `add x19, x20, x21` / `beq x0, x0, Exit` / `Else: sub x19, x20, x21` (C: if (i == j) f = g + h; else f = g − h;)

   답 보기

   `bne`. i != j일 때 Else로 건너뛴다. [Ch2 p.29]

3. while (save[i] == k) i += 1;의 RISC-V 코드에서 i × 4를 계산하는 명령어는?

답 보기

`slli x10, x22, 2`. 2칸 왼쪽 시프트 = × 4. [Ch2 p.30]

4. 위 while 코드의 여섯 명령어를 순서대로 쓰시오(이름만).

답 보기

slli, add, lw, bne, addi, beq. [Ch2 p.30]

5. rs1 = 0xFFFFFFFF, rs2 = 1일 때 blt와 bltu는 각각 분기하는가?

답 보기

blt: 분기함(−1 < 1). bltu: 분기 안 함($2^{32} - 1 < 1$ 거짓). [Ch2 p.31]

6. switch 문을 효율적으로 구현하기 위해 case별 코드 주소를 모아 둔 표를 무엇이라 하는가?

답 보기

분기 주소 표(branch address table, branch table). [Ch2 p.31]

7. 컴퓨터를 단순한 계산기와 구별하는 능력은 무엇이며, 보통 어떤 문장으로 나타내는가?

답 보기

판단(decision)하는 능력, if 문. [Ch2 p.28]

Ch2 · 8 슬라이드 p.32–36

프로시저 호출과 jal·jalr

이 단원을 마치면 프로시저 호출의 6단계를 순서대로 쓰고, 호출에 쓰는 레지스터 9개의 역할과 `jal`·`jalr`이 PC와 `x1`을 어떻게 바꾸는지 주소로 따라갈 수 있다.

쉽게 말하면

책을 읽다가 각주 번호를 보고 맨 뒤 각주 쪽으로 넘어간다고 해 보자. 넘어가기 전에 지금 읽던 쪽에 손가락을 끼워 둔다. 각주를 다 읽으면 손가락이 끼워진 쪽으로 돌아와 다음 줄부터 읽는다. 프로시저 호출도 같다. "각주로 넘어가기"가 점프(jump), "손가락 끼우기"가 링크(link)이고, 손가락 역할을 하는 레지스터가 `x1` 이다.

비유가 안 맞는 지점 손가락은 하나뿐이다. 각주 안에서 또 다른 각주로 넘어가면 손가락을 옮겨야 해서 처음 자리를 잃는다. 실제 CPU도 `x1` 이 하나라서 같은 문제가 생기고, 이것은 c2-10에서 스택으로 해결한다.

프로시저, 호출자, 피호출자

프로시저(procedure)는 이름을 붙여 따로 떼어 둔 코드 묶음이다. C의 함수가 여기에 해당한다. 프로그래머가 큰 일을 작은 조각으로 나눠 한 번에 한 부분에만 집중할 수 있게 해 준다. [Ch2 p.32]

호출자(caller) 프로시저를 부르는 쪽. 예: `main()` 안의 `b = procedure(g, h);`

피호출자(callee) 불려 가는 프로시저. 예: `procedure(x, y)`

인자(argument) 호출자가 피호출자에게 넘겨주는 데이터. 예: `g`, `h`

반환값(return value) 피호출자가 일을 마치고 호출자에게 돌려주는 결과. 예: `return a;`의 `a`

슬라이드의 예에서 `main`은 `if`와 `else`에서 같은 계산 `(x+y)*(x+2)`를 두 번 쓰던 것을 `procedure` 하나로 묶고, 인자만 바꿔 `procedure(g, h)`, `procedure(k, i)`로 부른다. 호출자는 인자를 주고, 피호출자는 결과를 돌려준다. [Ch2 p.32]

프로시저 실행의 6단계

프로그램은 프로시저를 실행할 때 다음 여섯 단계를 지켜야 한다. [Ch2 p.33]

1. **인자 배치**: 프로시저가 읽을 수 있는 곳에 인자를 놓는다. (호출자)
2. **제어 이동**: 프로시저로 제어를 넘긴다. (호출자의 `jal`)
3. **저장 공간 확보**: 프로시저에 필요한 저장 자원을 얻는다. (피호출자)
4. **작업 수행**: 원하는 일을 한다. (피호출자)
5. **결과 배치**: 호출한 프로그램이 읽을 수 있는 곳에 결과값을 놓는다. (피호출자)
6. **복귀**: 호출이 일어났던 지점으로 제어를 돌려준다. (피호출자의 `jalr`)

슬라이드 그림은 시간 순서로 표시한다. PC가 t일 때 `r = procedure(g, h)`를 만나 ①로 g, h를 넘기고, t+1에 ②③(프로시저로 넘어가 공간 확보), t+2·t+3에 ④(계산), ⑤로 결과 a를 넘긴 뒤, ⑥으로 t+4, 즉 호출 다음 줄로 돌아온다. [Ch2 p.33]

프로그램 카운터(PC, Program Counter) 다음에 실행할 명령어의 주소를 담고 있는 레지스터. 보통은 명령어 하나를 실행할 때마다 4(명령어 하나 = 4바이트)씩 커지고, 분기나 점프가 일어나면 목표 주소로 바뀐다. [Ch2 p.35]
x0~x31과 별개의 특수 레지스터라서 `add` 같은 명령의 피연산자로 쓸 수 없다. PDF 외 보충

호출에 쓰는 레지스터 9개

RISC-V는 프로시저 호출용으로 레지스터 **9개**를 정해 둔다. [Ch2 p.34]

레지스터	개수	역할	6단계 중
`x10` ~ `x17`	8	인자(parameter) 레지스터. 인자를 넘기거나 값을 돌려줄 때 사용	①
`x10`, `x11`	(위에 포함)	결과(반환값)를 놓는 곳	⑤
`x1`	1	복귀 주소(return address) 레지스터. 원래 지점으로 돌아갈 주소	②·⑥

8 + 1 = 9개다. `x10`·`x11`은 인자 레지스터 8개 안에 들어 있으므로 따로 세지 않는다. 앞의 예에서 g는 `x10`, h는 `x11`로 들어가고, 피호출자는 결과 a를 `x10`에 놓는다. [Ch2 p.34]

jal: 점프하면서 링크하기

RISC-V에는 프로시저 호출 전용 명령어 **jal(jump-and-link)**이 있다. 어떤 주소로 분기하면서 **동시에** 그다음 명령어의 주소를 목적지 레지스터 rd에 저장한다. [Ch2 p.35]

```
jal  x1, ProcedureAddress   // x1 = PC + 4 (다음 명령어 주소), PC = ProcedureAddress
```

- **Jump(점프)**: 프로시저로 제어를 넘긴다(2단계).
- **Link(링크)**: 나중에 호출 지점으로 돌아올 수 있게 연결 고리를 만든다. 이 연결 고리를 **복귀 주소(return address)**라 하고 `x1`에 저장한다. [Ch2 p.35]

저장되는 값은 jal 자신의 주소가 아니고 **jal 다음 명령어의 주소(PC + 4)**다. 돌아왔을 때 호출을 다시 하지 않고 다음 일을 이어 가야 하기 때문이다.

jalr: 레지스터에 든 주소로 복귀

복귀에는 **간접 점프(indirect jump)**를 쓴다. 간접 점프는 점프할 주소를 명령어에 직접 적지 않고 레지스터에서 꺼내 쓰는 점프다. RISC-V의 간접 점프는 **jalr(jump-and-link register)**이다. [Ch2 p.36]

```
jalr x0, 0(x1)   // PC = x1 + 0, 링크 결과는 x0에 쓰므로 버려짐
```

- `x1`에 저장된 주소(정확히는 0 + `x1`의 값)로 분기한다.
- 목적지 레지스터로 `x0`을 쓴다. `x0`은 항상 0이라 쓰기가 무시되므로, 결과적으로 "다음 주소를 저장"하는 링크 동작을 버리는 효과가 난다. [Ch2 p.36]

jalr도 jal처럼 "점프 + 링크"를 하는 명령어다. 복귀할 때는 링크가 필요 없어서 rd 자리에 `x0`을 넣어 링크를 없앤 것이다.

핵심

호출 = `jal x1, 라벨` (x1 ← 다음 명령어 주소, PC ← 라벨). 복귀 = `jalr x0, 0(x1)` (PC ← x1, 링크는 x0에 버림). 호출용 레지스터 9개 = 인자 x10~x17(8개, 그중 x10·x11은 반환값) + 복귀 주소 x1.

흐름을 주소로 따라가기

슬라이드 p.35·36의 그림은 주소를 줄 번호처럼 1씩 늘려 그렸다(10: `r = procedure(g,h);`, 11: `print(r);`, 12: `procedure(x,y):`). 흐름은 ① PC가 10을 가리킴 → ② jal로 PC가 12(프로시저)로 점프 → ③ 동시에 x1에 11(호출 지점의 다음)을 링크 → 프로시저 실행 → jalr로 PC = x1 = 11로 복귀, 11이 "the point of origin"이다. [Ch2 p.35–36] 실제 RISC-V에서는 명령어가 4바이트라 주소가 4씩 커진다. 아래는 바이트 주소로 다시 쓴 예다.

```
// 주소   명령어
1000    jal  x1, proc       // main: 호출
1004    ...                 // 호출 다음 명령어 (돌아올 곳)
...
2000    proc: add x10, x10, x11
2004          jalr x0, 0(x1)  // 복귀
```

순서	실행하는 명령어	실행 후 PC	실행 후 x1
1	1000의 `jal x1, proc`	2000	1004
2	2000의 `add x10, x10, x11`	2004	1004
3	2004의 `jalr x0, 0(x1)`	1004	1004 (x0에 쓴 값은 버려짐)

예제

주소 400에 `jal x1, Sum`이 있고 Sum은 주소 800에서 시작한다. jal을 실행한 직후 PC와 x1의 값은? Sum의 마지막 명령이 `jalr x0, 0(x1)`일 때 그 실행 직후 PC는?

풀이 보기

jal 직후 PC = 800(점프), x1 = 400 + 4 = **404**(링크).

Sum 안에서 x1을 바꾸지 않았다면 jalr 직후 PC = 0 + x1 = **404**. jal 다음 명령어부터 이어서 실행한다.

함정

① x1에 저장되는 값은 jal의 주소가 아니고 **다음 명령어 주소(PC + 4)**다. ② 호출용 레지스터는 9개다. x10·x11을 따로 세어 11개라고 쓰면 틀린다. ③ `jalr x0, 0(x1)`의 x0은 "0번지로 간다"는 뜻이 아니다. x0은 목적지 레지스터(링크를 버리는 곳)이고, 가는 곳은 0 + x1이다. ④ 슬라이드 그림의 주소 10, 11, 12는 설명용 줄 번호다. 실제 명령어 주소는 4씩 차이 난다.

시험 포인트

- 6단계를 순서대로 쓰기(인자 배치 → 제어 이동 → 저장 공간 확보 → 작업 수행 → 결과 배치 → 복귀).
- x10~x17, x10·x11, x1의 역할 짝짓기, "9개" 개수.
- jal의 jump와 link가 각각 무엇을 하는지, 복귀 주소가 저장되는 레지스터(x1).
- `jalr x0, 0(x1)`에서 x0을 쓰는 이유(복귀 주소를 버림).
- PC의 정의.

스스로 확인

1. 프로시저를 부르는 쪽과 불려 가는 쪽을 각각 무엇이라 하는가? (영어로도)

   답 보기

   호출자(caller), 피호출자(callee).

2. 프로시저 실행 6단계 중 3번째와 5번째 단계는?

   답 보기

   3: 프로시저에 필요한 저장 공간 확보, 5: 호출자가 읽을 수 있는 곳에 결과값 배치.

3. RISC-V가 프로시저 호출에 할당한 레지스터 수와 그 구성은?

   답 보기

   9개. 인자 레지스터 x10~x17(8개, 그중 x10·x11이 반환값) + 복귀 주소 레지스터 x1.

4. 빈칸: jal은 목표 주소로 분기하면서 (㉠)의 주소를 (㉡)에 저장한다. 이 주소를 (㉢)라 한다.

   답 보기

   ㉠ 다음 명령어(PC + 4), ㉡ 목적지 레지스터 rd(호출 때는 x1), ㉢ 복귀 주소(return address).

5. O/X: `jalr x0, 0(x1)`을 실행하면 PC가 0이 된다.

   답 보기

   X. PC = 0 + x1의 값. x0은 링크(다음 주소)를 버리기 위한 목적지 레지스터다.

6. 주소 2000의 `jal x1, F`를 실행한 직후 x1의 값은?

   답 보기

   2004. 명령어가 4바이트라 다음 명령어 주소는 2000 + 4.

7. PC란 무엇인가?

   답 보기

   다음에 실행할 명령어의 주소를 담은 레지스터(프로그램 카운터).

Ch2 · 9 슬라이드 p.37–38

스택과 leaf 프로시저

이 단원을 마치면 스택이 왜 필요하고 어느 방향으로 자라는지 설명하고, `leaf_example` 코드를 한 줄씩 해석하며 sp 조정량과 저장 오프셋을 계산할 수 있다.

쉽게 말하면

식당 주방에 접시를 쌓아 둔다고 하자. 새 접시는 맨 위에 올리고, 꺼낼 때도 맨 위부터 꺼낸다. 마지막에 올린 것이 먼저 나온다. 레지스터가 모자라면 지금 레지스터에 든 값을 이런 접시 더미(스택)에 잠시 올려 두고, 일이 끝나면 거꾸로 꺼내 되돌린다.

비유가 안 맞는 지점 접시 더미는 위로 쌓이지만 RISC-V 스택은 메모리의 **높은 주소에서 낮은 주소 쪽으로** 자란다. 또 "올리기·꺼내기" 전용 버튼(push/pop 명령어)이 없어서, sp를 직접 빼고 더하며 sw/lw로 넣고 꺼낸다.

레지스터가 모자랄 때

다음 두 경우에는 메모리가 필요하다. [Ch2 p.37]

- 프로시저의 인자가 8개보다 많거나 반환값이 2개보다 많을 때: 넘치는 인자·반환값을 메모리에 둔다.
- 피호출자가 레지스터를 써야 하는데 그 레지스터에 호출자의 값이 들어 있을 때: 레지스터 수가 한정돼 있으므로 호출자의 값을 메모리로 옮겨 두었다가 되돌려야 한다.

레지스터 스필링(register spilling) 레지스터의 값을 메모리에 저장해 두는 일. [Ch2 p.37]

스택(stack) 마지막에 넣은 것이 먼저 나오는 **후입선출(LIFO, last-in-first-out)** 자료 구조. 스필링에 가장 알맞은 자료 구조다. [Ch2 p.37]

푸시(push) / 팝(pop) 스택에 값을 넣기 / 스택에서 값을 꺼내기.

스택 포인터(sp, stack pointer) 스택의 맨 위(top of stack), 즉 가장 최근에 넣은 값의 주소를 가리키는 레지스터. RISC-V에서는 **x2**이고 이름이 sp다. [Ch2 p.37]

RISC-V 스택의 규칙

- **push·pop 전용 명령어가 없다.** 메모리에 있는 스택은 load·store 명령어(lw, sw)로 접근한다. [Ch2 p.37]
- 스택은 **높은 주소에서 낮은 주소로** 자란다. 그래서 push는 sp를 **줄이고**, pop은 sp를 **늘린다.** [Ch2 p.37]
- 레지스터 하나를 저장(push)하거나 복원(pop)할 때마다 sp를 **한 word(4바이트)**씩 조정한다. [Ch2 p.37]

레지스터 n개 저장: `addi sp, sp, -4n` → 복원 후: `addi sp, sp, 4n`

leaf 프로시저 컴파일하기

leaf 프로시저(leaf procedure)는 다른 프로시저를 호출하지 않는 프로시저다. 호출 관계를 나무로 그리면 잎(leaf)에 해당해서 이렇게 부른다. [Ch2 p.38]

```
int leaf_example(int g, int h, int i, int j)
{
    int f;
    f = (g + h) - (i + j);
    return f;
}
// g → x10, h → x11, i → x12, j → x13, f → x20
```

인자 4개는 인자 레지스터 x10~x13으로 들어오고, f는 x20을 쓰기로 했다. 계산 중간값(g+h, i+j)은 x5, x6에 둔다. 컴파일 결과는 다음과 같다. [Ch2 p.38]

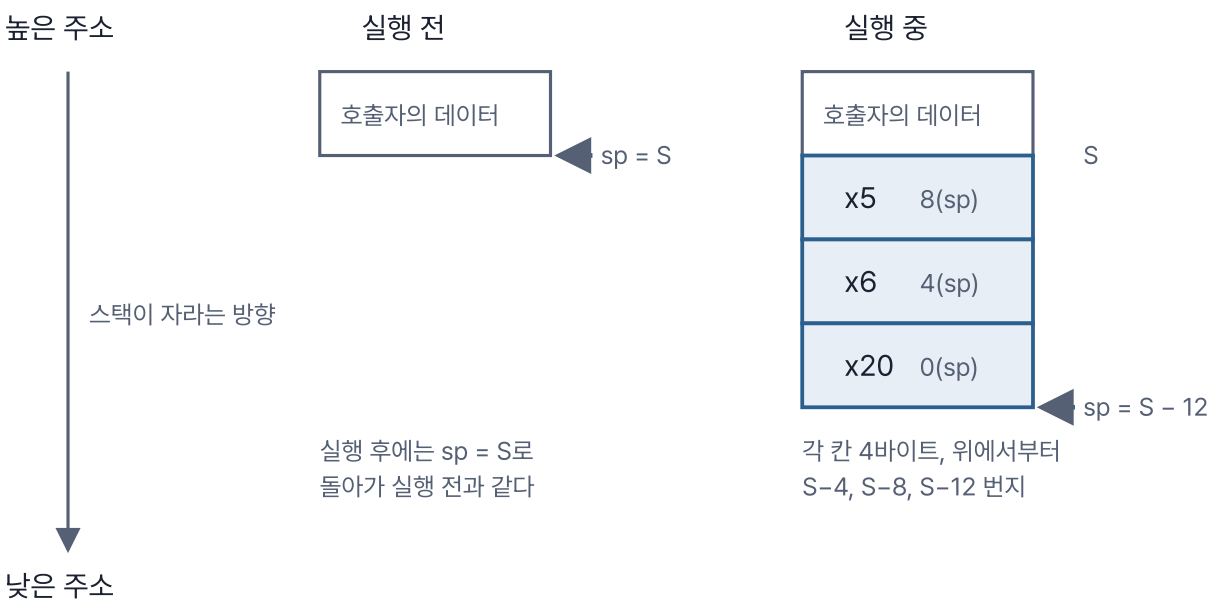
```
leaf_example:
    addi sp, sp, -12     // 스택에 3개 넣을 자리 만들기
    sw   x5, 8(sp)       // 나중을 위해 x5 저장
    sw   x6, 4(sp)       // x6 저장
    sw   x20, 0(sp)      // x20 저장
    add  x5, x10, x11    // x5 = g + h
    add  x6, x12, x13    // x6 = i + j
    sub  x20, x5, x6     // f = x5 - x6 = (g+h)-(i+j)
    addi x10, x20, 0     // f를 반환 (x10 = x20 + 0)
    lw   x20, 0(sp)      // 호출자를 위해 x20 복원
    lw   x6, 4(sp)       // x6 복원
    lw   x5, 8(sp)       // x5 복원
    addi sp, sp, 12      // 스택에서 3개 지우기
    jalr x0, 0(x1)       // 호출한 곳으로 복귀
```

한 줄씩 해설

처음 sp 값을 S라고 하자.

줄	하는 일	결과
`addi sp, sp, -12`	레지스터 3개 × 4바이트 = 12바이트 자리 확보(push 준비). 스택이 아래로 자라므로 뺀다.	sp = S − 12
`sw x5, 8(sp)`	x5를 sp + 8에 저장	주소 S − 4
`sw x6, 4(sp)`	x6을 sp + 4에 저장	주소 S − 8
`sw x20, 0(sp)`	x20을 sp + 0에 저장	주소 S − 12
`add x5, x10, x11`	g + h	x5 = g + h
`add x6, x12, x13`	i + j	x6 = i + j
`sub x20, x5, x6`	f 계산	x20 = f
`addi x10, x20, 0`	반환값은 x10에 놓아야 하므로 복사(5단계). 0을 더해 값 옮기기.	x10 = f
`lw x20, 0(sp)` 등 3줄	저장했던 값을 같은 오프셋에서 다시 읽어 원래 값으로 복원(pop). 순서는 넣은 반대로.	x5, x6, x20 = 호출 전 값
`addi sp, sp, 12`	확보했던 12바이트 반납	sp = S
`jalr x0, 0(x1)`	복귀(6단계)	PC = x1

복원 순서를 저장의 반대로 쓴 것은 LIFO 모양을 따른 것이다. 다만 각 값의 위치를 오프셋으로 정확히 지정하므로, 오프셋만 맞으면 순서를 바꿔도 결과는 같다. PDF 외 보충



leaf_example의 스택. 그림 위쪽이 높은 주소다. 실행 후 그림은 실행 전과 같으므로 생략했다. [Ch2 p.37–38]

핵심

스택 = LIFO, sp = x2, 높은 주소 → 낮은 주소로 자람, push/pop 명령어 없음(sw/lw 사용), 레지스터 하나에 sp 4바이트. leaf_example은 3개를 저장하므로 `addi sp, sp, -12` 로 시작하고 `addi sp, sp, 12` 로 끝난다.

예제

어떤 leaf 프로시저가 x9, x18, x19, x20, x21 다섯 개를 저장해야 한다. 처음 sp = 0x7FFFFF00이다. 자리를 만든 뒤 sp 값과, x9를 가장 높은 주소에 저장할 때 x9의 저장 주소는?

풀이 보기

5개 × 4 = 20바이트 → `addi sp, sp, -20`. 0x7FFFFF00 − 20(= 0x14) = **0x7FFFFEEC**.

가장 높은 칸은 16(sp)이다(오프셋 0, 4, 8, 12, 16). 주소 = 0x7FFFFEEC + 16 = **0x7FFFFEFC** = 원래 sp − 4. 끝날 때는 `addi sp, sp, 20`.

함정

① 스택에 자리를 만들 때 sp를 **빼야** 한다(`addi sp, sp, -12`). 더하면 호출자의 데이터를 덮는다. ② 조정량은 레지스터 개수가 아니고 **바이트 수**(개수 × 4)다. ③ RISC-V에 push, pop 명령어는 없다. ④ 저장(sw)과 복원(lw)의 오프셋이 같은 레지스터끼리 짝이 맞아야 한다. ⑤ 반환값은 x20에 있어도 소용없다. 반드시 x10에 옮겨야 호출자가 읽는다.

시험 포인트

- spilling의 정의, 스택이 LIFO라는 것, sp = x2.
- 스택이 자라는 방향(높은 → 낮은 주소)과 push/pop 명령어가 없다는 점.
- 저장할 레지스터 수 → sp 조정량 계산, sw 오프셋 빈칸 채우기.
- leaf 프로시저의 정의.

스스로 확인

1. 레지스터 값을 메모리에 저장해 두는 일을 무엇이라 하는가?

   답 보기

   레지스터 스필링(register spilling).

2. RISC-V에서 스택 포인터로 쓰는 레지스터 번호는?

   답 보기

   x2 (이름 sp).

3. O/X: RISC-V 스택은 낮은 주소에서 높은 주소로 자라며 push 명령어로 값을 넣는다.

   답 보기

   X. 높은 주소에서 낮은 주소로 자라고, push/pop 명령어 없이 sw/lw로 접근한다.

4. leaf 프로시저란?

   답 보기

   다른 프로시저를 호출하지 않는 프로시저.

5. 레지스터 4개를 저장하려 할 때 첫 명령어를 쓰시오.

   답 보기

   `addi sp, sp, -16` (4개 × 4바이트).

6. leaf_example에서 `addi x10, x20, 0`이 필요한 이유는?

   답 보기

   반환값은 x10(또는 x11)에 놓아야 호출자가 읽을 수 있는데, f가 x20에 있으므로 x10으로 옮긴다.

Ch2 · 10 슬라이드 p.39–43

레지스터 보존 규약, 중첩·재귀 호출

이 단원을 마치면 어떤 레지스터를 호출자와 피호출자 중 누가 저장하는지 말하고, 중첩 호출에서 x1·x10이 덮이는 문제를 설명하며, 재귀 `fact` 코드를 추적할 수 있다.

쉽게 말하면

룸메이트와 책상 하나를 나눠 쓴다고 하자. "왼쪽 서랍은 아무나 막 써도 되니 중요한 건 각자 치워 둬라, 오른쪽 서랍은 쓰고 나면 반드시 원래대로 돌려놔라"라고 약속해 두면, 매번 서랍 전체를 비우고 다시 채우는 수고가 줄어든다. 레지스터 보존 규약이 이런 약속이다. 왼쪽 서랍이 caller-saved(임시) 레지스터, 오른쪽 서랍이 callee-saved(저장) 레지스터다.

비유가 안 맞는 지점 실제로는 "중요한 것"을 판단하는 주체가 정해져 있다. 임시 레지스터는 호출 뒤에도 그 값이 필요할 때만 호출자가 저장하고, 저장 레지스터는 피호출자가 그 레지스터를 쓸 때만 저장한다. 필요 없으면 아무도 저장하지 않는다.

왜 규약이 필요한가

c2-9의 leaf_example은 x5, x6, x20 셋을 모두 저장했다. 하지만 호출자가 x5에 든 값을 다시 쓰지 않는다면 그 저장·복원은 낭비다. 이렇게 값을 쓰지도 않을 레지스터를 저장·복원하는 일을 피하려고, RISC-V는 레지스터 **19개**를 두 그룹으로 나눈다. [Ch2 p.39]

caller-saved 레지스터(호출자 저장) — **x5~x7, x28~x31**(7개). 임시(temporary) 레지스터. 피호출자는 이 값을 보존해 주지 **않는다** (NOT preserved). 그래서 호출 뒤에도 값이 필요하면 호출자가 호출 전에 스택에 저장한다. 호출 결과로 이 레지스터의 내용은 바뀔 수 있다. [Ch2 p.39]

callee-saved 레지스터(피호출자 저장) — **x8~x9, x18~x27**(12개). 저장(save) 레지스터. 호출 전후로 값이 보존되어야 한다. 피호출자가 이 레지스터를 쓰려면 쓰기 전에 스택에 저장하고, 호출자에게 돌아가기 전에 복원한다. 그래서 이 레지스터의 내용은 호출을 거쳐도 그대로다. [Ch2 p.39]

```
p.39: 7개(x5~x7, x28~x31) + 12개(x8~x9, x18~x27) = 19개
```

leaf_example에 적용하기

규약을 따르면 앞의 예제에서 **x5와 x6의 sw/lw는 필요 없고, x20의 sw/lw는 필요하다.** [Ch2 p.39] x5·x6은 caller-saved라 피호출자가 마음대로 써도 되고, x20은 callee-saved라 쓰려면 피호출자가 보존해야 하기 때문이다. 줄이면 다음과 같다.

```
leaf_example:
    addi sp, sp, -4      // 1개만 저장
    sw   x20, 0(sp)
    add  x5, x10, x11
    add  x6, x12, x13
    sub  x20, x5, x6
    addi x10, x20, 0
    lw   x20, 0(sp)
    addi sp, sp, 4
    jalr x0, 0(x1)
```

명령어가 13개에서 9개로 줄었다. PDF 외 보충

호출 규약(calling convention) 1차 정리

p.40은 호출자와 피호출자가 할 일을 순서대로 묶는다. [Ch2 p.40]

호출자(caller)	피호출자(callee)
① (살아 있는) 레지스터 저장 ← caller saved	① 스택 프레임 할당
② 인자 설정	② 피호출자에서 바뀔 수 있는 레지스터 저장 ← callee saved
③ 복귀 주소 저장하며 피호출자로 점프	③ 반환값 배치
④ (돌아온 뒤) 저장했던 레지스터 복원	④ 저장했던 레지스터 복원
	⑤ 스택 프레임 해제
	⑥ 복귀 주소로 점프

여기서 "살아 있는(live)" 레지스터는 호출이 끝난 뒤에도 그 값을 다시 쓸 레지스터를 말한다. 스택 프레임은 프로시저 하나가 스택에서 차지하는 영역이다(c2-11).

중첩 호출의 문제

leaf가 아닌(nonleaf) 프로시저, 즉 다른 프로시저를 호출하는 프로시저에서는 문제가 생긴다. 슬라이드 예를 보자. 주소는 설명용 줄 번호다. [Ch2 p.41]

```
main():           A(a):             B(b):
  10: a = a+1;      40: s = B(7);     60: m = b*10;
  11: r = A(3);     41: s = s*a;      61: m--;
  12: return r;     42: return s;     62: return m;
```

1. main이 3을 인자로 A를 부른다: x10 = 3, `jal x1, A` → x1 = 12.
2. A가 7을 인자로 B를 부른다: x10 = 7, `jal x1, B` → x1 = 41.
3. 이 순간 **x10의 3과 x1의 12가 덮어써졌다.**
4. B는 결과 m을 x10에 넣고 x1 = 41로 돌아간다. 여기까지는 문제없다.
5. A의 41번 줄 `s = s*a`는 자기 인자 a(= 3)가 필요한데 x10은 이미 B의 결과로 바뀌었다.
6. A가 42번 줄에서 `jalr x0, 0(x1)`로 복귀하려 하면 x1 = 41이다. 12(main)로 돌아가지 못하고 41로 다시 가서 같은 곳을 맴돈다.

해결책: 중첩 호출에서는 호출하는 쪽이 **복귀 주소**와 **호출 뒤에 필요한 인자·임시값**을 스택에 저장하고, 호출이 끝난 뒤 스택에서 복원한다. [Ch2 p.41] A는 B를 부르기 전에 x1(=12)과 x10(=3)을 스택에 넣어 두고, B에서 돌아온 뒤 꺼내면 된다.

일반화하면, 보존해야 하는 나머지 레지스터를 모두 스택에 push한다. [Ch2 p.41]

- 호출자(caller)가 push: **x10~x17**(인자 레지스터), x5~x7, x28~x31
- 피호출자(callee)가 push: **x1**(복귀 주소 레지스터), x8~x9, x18~x27

함정: "19개"와 확장된 목록

p.39의 정의는 **19개**(x5~x7·x28~x31 = 7개, x8~x9·x18~x27 = 12개)이고 x10~x17과 x1은 들어 있지 않다. p.41과 p.43의 호출 규약에서 caller 쪽에 x10~x17, callee 쪽에 x1이 **추가**된다. "레지스터 19개를 두 그룹으로 나눈다"를 물으면 p.39 기준으로, "호출 규약에서 피호출자가 저장하는 레지스터"를 물으면 x1을 포함해 답한다. [Ch2 p.39, 41, 43]

x1이 callee 쪽에 있는 것은 이렇게 이해하면 된다. x1은 다른 프로시저를 호출하는 순간 jal이 덮어쓴다. 그래서 다른 프로시저를 부를 nonleaf 프로시저가 자기 시작 부분에서 자기 x1을 저장했다가 복귀 직전에 복원한다. 다음 fact가 그 예다. 교재 표도 x1(복귀 주소), sp,

x8~x9·x18~x27을 "보존됨"으로 분류한다. 교재 기준

재귀 프로시저: 팩토리얼

재귀(recursive) 프로시저는 자기 자신을 호출하는 프로시저다. 팩토리얼 C 코드와 RISC-V 코드는 다음과 같다. 인자 n은 x10이다. [Ch2 p.42]

```
int fact(int n)
{
    if (n < 1) return 1;
    else return n * fact(n - 1);
}
```

```
fact:
    addi sp, sp, -8      // 스택에 2개 넣을 자리
    sw   x1, 4(sp)       // 복귀 주소 저장
    sw   x10, 0(sp)      // 인자 n 저장
    addi x5, x10, -1     // x5 = n - 1
    bge  x5, x0, L1      // n - 1 >= 0 (즉 n >= 1)이면 L1로
    addi x10, x0, 1      // 반환값 1
    addi sp, sp, 8       // 스택에서 2개 빼기
    jalr x0, 0(x1)       // 호출자에게 복귀
L1: addi x10, x10, -1    // n >= 1: 인자를 n - 1로
    jal  x1, fact        // fact(n - 1) 호출
    addi x6, x10, 0      // 돌아옴: fact(n-1)의 결과를 x6으로
    lw   x10, 0(sp)      // 인자 n 복원
    lw   x1, 4(sp)       // 복귀 주소 복원
    addi sp, sp, 8       // 스택에서 2개 빼기
    mul  x10, x10, x6    // n * fact(n-1)을 반환값으로
    jalr x0, 0(x1)       // 호출자에게 복귀
```

한 줄씩 보면 다음과 같다.

- `addi sp, sp, -8` · `sw x1, 4(sp)` · `sw x10, 0(sp)` : fact는 자기 자신을 부르므로 jal이 x1을, 새 인자가 x10을 덮어쓴다. 그래서 둘을 먼저 저장한다. 2개 × 4 = 8바이트.
- `addi x5, x10, -1` · `bge x5, x0, L1` : `n < 1`을 직접 비교하지 않고 n − 1 ≥ 0, 즉 n ≥ 1인지 보고 그러면 L1(else 부분)로 간다. x5는 임시 레지스터라 저장하지 않았다.
- `addi x10, x0, 1` · `addi sp, sp, 8` · `jalr x0, 0(x1)` : n < 1이면 1을 반환. x1과 x10을 아직 바꾸지 않았으므로 lw로 복원할 필요 없이 sp만 되돌린다. x10은 반환값 1로 바뀌는데, 반환값 자리라서 괜찮다.
- `L1: addi x10, x10, -1` · `jal x1, fact` : 인자를 n − 1로 바꿔 자신을 호출. 이때 x1이 jal 다음 줄 주소로 덮인다.
- `addi x6, x10, 0` : 돌아오면 x10에 fact(n−1)이 들어 있다. 곧 x10에 n을 복원할 것이므로 결과를 x6으로 옮긴다.
- `lw x10, 0(sp)` · `lw x1, 4(sp)` · `addi sp, sp, 8` : 이번 호출에서 저장했던 n과 복귀 주소를 꺼내고 자리를 반납한다.
- `mul x10, x10, x6` · `jalr x0, 0(x1)` : n × fact(n−1)을 x10에 놓고 복귀한다. `mul`은 곱셈 명령어다(Ch3에서 다룸).

fact(3) 추적

fact(3) = 3 × fact(2) = 3 × 2 × fact(1) = 3 × 2 × 1 × fact(0) = 3 × 2 × 1 × 1 = 6. [Ch2 p.42] main이 처음 호출할 때 sp = S라 하고, main의 jal 다음 주소를 RA_main, fact 안 `jal x1, fact` 다음 줄(`addi x6, x10, 0`)의 주소를 RA_fact라 하자.

단계	상황	sp	스택에 저장된 것 [x1, n]	x10
1	fact(3) 시작, 저장 후 L1로	S − 8	[RA_main, 3]	3 → 2
2	fact(2) 시작, 저장 후 L1로	S − 16	[RA_fact, 2]	2 → 1
3	fact(1) 시작, 저장 후 L1로	S − 24	[RA_fact, 1]	1 → 0
4	fact(0): x5 = −1 < 0이라 분기 안 함, 1 반환	S − 32 → S − 24	[RA_fact, 0] 저장 후 바로 버림	1
5	fact(1)로 복귀: x6 = 1, n = 1 복원, 1 × 1	S − 16		1
6	fact(2)로 복귀: x6 = 1, n = 2 복원, 2 × 1	S − 8		2
7	fact(3)로 복귀: x6 = 2, n = 3 복원, 3 × 2, x1 = RA_main으로 복귀	S		**6**

스택이 가장 깊을 때 프레임 4개 × 8바이트 = 32바이트를 쓴다. 각 단계는 자기 프레임에 저장한 n을 꺼내므로, 아래 단계에서 x10이 바뀌어도 자기 n을 잃지 않는다. 이것이 중첩 호출 문제의 해결이다.

호출 규약 다시 정리 (p.43)

	호출자(caller)	피호출자(callee)
호출 전 / 시작	살아 있는 caller-saved 레지스터 저장: **x10~x17, x5~x7, x28~x31**	스택 프레임 할당
	인자 설정: 처음 8개는 **x10~x17**, 나머지는 스택	피호출자에서 바뀔 수 있는 callee-saved 레지스터 저장: **x1, x8~x9, x18~x27**
	복귀 주소(x1) 저장하며 피호출자로 점프 (`jal x1, …`)	반환값 배치: **x10~x11**
호출 후 / 끝	caller-saved 레지스터 복원	callee-saved 레지스터 복원 → 스택 프레임 해제 → 복귀 주소로 점프

[Ch2 p.43]

핵심

caller-saved = 임시 x5~x7, x28~x31 (+ 규약상 x10~x17), 피호출자가 보존 안 함. callee-saved = 저장 x8~x9, x18~x27 (+ 규약상 x1), 피호출자가 쓰면 저장·복원. p.39의 두 그룹은 19개. nonleaf 프로시저는 x1과 호출 뒤 필요한 인자를 스택에 저장한다.

예제

프로시저 P는 x18과 x5를 쓰고, 다른 프로시저를 호출하지 않는다. 호출 규약(p.39)에 따라 P가 시작할 때 저장해야 하는 레지스터와 sp 조정량은? P가 다른 프로시저 Q를 호출한다면 무엇이 더 필요한가?

풀이 보기

x18은 callee-saved라 P가 저장해야 한다. x5는 caller-saved라 P는 저장하지 않아도 된다. → x18 하나, `addi sp, sp, -4`.

Q를 호출하면 jal이 x1을 덮으므로 x1도 저장해야 한다(2개, −8). 또 Q 호출 뒤에도 x5나 인자 레지스터 값이 필요하다면, 이번에는 P가 호출자이므로 그것들도 Q 호출 전에 저장한다.

함정

① caller-saved는 "호출자가 항상 저장"이 아니다. 호출 뒤에도 그 값이 필요할 때만 저장한다. callee-saved도 피호출자가 그 레지스터를 쓸 때만 저장한다. ② fact의 기저 사례(n < 1)에서는 lw 없이 sp만 8 늘린다. x1, x10을 바꾸지 않았기 때문이다. ③ fact의 `addi x6, x10, 0`을 빼면, 바로 다음 `lw x10, 0(sp)`가 fact(n−1)의 결과를 지워 버린다. ④ `bge x5, x0, L1`은 n − 1 ≥ 0을 검사한다. C의 조건 `n < 1`과 반대 방향이다(조건이 거짓일 때 else로 분기).

시험 포인트

- caller-saved / callee-saved 레지스터 번호 쓰기, "19개" 구성.
- leaf_example에서 x5·x6 저장이 불필요하고 x20 저장이 필요한 이유.
- 중첩 호출에서 x1과 x10이 덮이는 문제와 해결책(스택에 저장).
- fact 코드 빈칸(`addi sp, sp, -8` , `sw x1, 4(sp)` , `bge` , `jal x1, fact` , `mul`), fact(3) = 6.
- p.43 호출 규약에서 호출자·피호출자가 하는 일.

스스로 확인

1. p.39 기준 caller-saved 레지스터와 callee-saved 레지스터를 쓰고, 합계 개수를 쓰시오.

   답 보기

   caller-saved: x5~x7, x28~x31(7개). callee-saved: x8~x9, x18~x27(12개). 합계 19개.

2. O/X: 피호출자는 x6을 사용하기 전에 반드시 스택에 저장해야 한다.

   답 보기

   X. x6은 caller-saved(임시)라 피호출자가 보존하지 않는다.

3. main → A → B 중첩 호출에서 A가 B를 jal로 부를 때 덮어써지는 두 레지스터는?

   답 보기

   x1(복귀 주소)과 x10(인자). A는 호출 전에 둘을 스택에 저장해야 한다.

4. fact 코드의 첫 줄이 `addi sp, sp, -8`인 이유는?

   답 보기

   x1(복귀 주소)과 x10(인자 n) 두 개를 저장하므로 2 × 4 = 8바이트.

5. p.43 호출 규약에서 피호출자가 저장하는 레지스터를 모두 쓰시오.

답 보기

x1, x8~x9, x18~x27 (바뀔 수 있는 것만).

6. fact(3)을 호출했을 때 스택이 가장 깊을 때 사용한 바이트 수는?

답 보기

32바이트. fact(3), fact(2), fact(1), fact(0) 네 프레임 × 8바이트.

7. 인자가 10개인 프로시저를 호출할 때 9번째, 10번째 인자는 어디에 두는가?

답 보기

스택. 처음 8개만 x10~x17에 둔다.

Ch2 · 11 슬라이드 p.44–45

메모리 배치: 프레임·스택·힙

이 단원을 마치면 프로시저 프레임과 프레임 포인터를 정의하고, 메모리의 text·static data·heap·stack 네 영역이 무엇을 담고 어디에 놓이는지 말할 수 있다.

쉽게 말하면

기숙사 방 하나를 메모리라 하자. 바닥 쪽 선반에는 교과서(프로그램 코드)와 늘 쓰는 사전(전역 변수)을 둔다. 그 위로는 필요할 때마다 상자를 쌓는 짐 더미(힙)가 바닥에서 위로 자라고, 천장에는 그날그날 쓰는 메모지(스택)가 천장에서 아래로 매달려 내려온다. 둘이 가운데 빈 공간에서 만나기 전까지는 공간을 낭비 없이 쓴다.

비유가 안 맞는 지점 방의 짐은 손으로 치우지만, 스택은 프로시저가 끝나면 sp를 되돌리는 것만으로 자동 정리된다. 힙은 프로그래머가 malloc으로 받고 free로 직접 돌려줘야 한다.

스택에 지역 데이터 두기

스택은 레지스터 저장뿐 아니라 프로시저의 **지역 변수**(local variable, 그 프로시저 안에서만 쓰는 변수)를 두는 데도 쓴다. 레지스터에 들어가지 않는 지역 배열이나 구조체가 대표적이다. [Ch2 p.44]

프로시저 프레임(procedure frame) = 활성 레코드(activation record) — 스택에서 한 프로시저가 쓰는 구간. 저장한 레지스터와 지역 변수가 들어 있다. 두 이름은 같은 것이다. [Ch2 p.44]

프레임 포인터(fp, frame pointer) — 프로시저 프레임의 첫 word를 가리키는 레지스터. RISC-V에서는 **x8**이다. [Ch2 p.44]

sp가 있는데 fp가 왜 필요할까? 프로시저 실행 중에 sp가 바뀔 수 있기 때문이다(예: 중간에 값을 더 push). sp를 기준으로 지역 변수를 찾으면 sp가 바뀔 때마다 오프셋도 달라진다. fp는 프로시저 안에서 변하지 않는 **안정된 기준 레지스터**라서, 지역 변수를 항상 같은 오프셋으로 찾을 수 있다. [Ch2 p.44] x8은 callee-saved 레지스터이므로 fp로 쓰는 프로시저는 원래 값을 저장했다가 복원한다.

p.44 그림은 호출 전·중·후의 스택을 보여 준다. 교재 기준

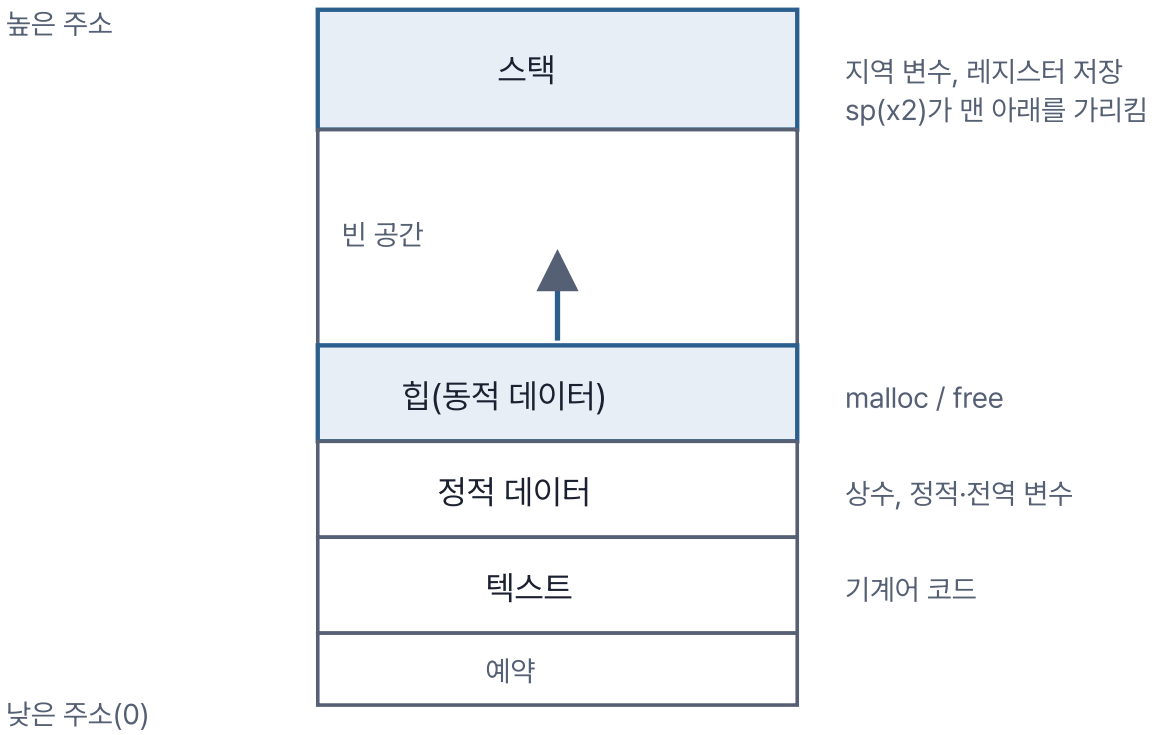
시점	스택 모습
호출 전 (Before)	호출자의 데이터까지만 있고 sp가 그 끝을 가리킨다.
호출 중 (During)	fp가 새 프레임의 첫 word를 가리키고, 그 아래로 저장한 인자 레지스터(있으면), 복귀 주소, 저장 레지스터, 지역 배열·구조체가 쌓이며, sp가 맨 아래(가장 낮은 주소)를 가리킨다.
호출 후(After)	프레임이 해제되어 sp(와 fp)가 호출 전 값으로 돌아간다.

메모리 영역 4가지

RISC-V는 메모리를 다음과 같이 나눠 쓰는 규약을 둔다. [Ch2 p.45]

영역	담는 것	자라는 방향
텍스트(text)	RISC-V 기계어 코드	고정
정적 데이터(static data)	상수와 기타 정적 변수(프로그램 내내 존재하는 변수, 예: 전역 변수)	고정
힙(heap) = 동적 데이터(dynamic data)	실행 중에 동적으로 할당하는 메모리. 예: 연결 리스트, 트리. C에서는 `malloc()`으로 받고 `free()`로 반납	낮은 → 높은 주소 (위로)
스택(stack)	지역 변수와 레지스터 저장	높은 → 낮은 주소 (아래로)

스택과 힙은 **서로를 향해** 자란다. 두 영역이 부딪치기 전까지 남은 저장 공간을 최대한 쓸 수 있게 하려는 배치다. [Ch2 p.45]



메모리 배치. 스택은 아래로, 힙은 위로 자라 가운데 빈 공간을 함께 쓴다. 맨 아래 예약 영역과 각 영역의 정확한 순서·주소는 교재 그림 기준이다. 교재 기준 [Ch2 p.45]

교재는 정적 데이터에 접근하기 쉽도록 전역 포인터(gp, global pointer) x3을 둔다고 설명한다. PDF 외 보충

핵심

procedure frame = activation record(저장한 레지스터 + 지역 변수). fp = x8, 프로시저 안에서 변하지 않는 기준. 메모리 영역: text(기계어) · static data(상수·정적 변수) · heap(동적 할당, malloc/free) · stack(지역 변수·레지스터 저장). 스택과 힙은 서로를 향해 자란다.

예제

다음 C 프로그램의 각 데이터는 어느 영역에 있는가? (a) 전역 배열 `int table[100];` (b) 함수 안의 지역 배열 `int buf[10];` (c) `malloc(sizeof(Node))`로 만든 연결 리스트 노드 (d) 함수 `main`의 기계어

풀이 보기

(a) 정적 데이터 (b) 스택(프로시저 프레임 안) (c) 힙 (d) 텍스트.

함정

① 힙은 위로(높은 주소 쪽으로), 스택은 아래로(낮은 주소 쪽으로) 자란다. 방향을 바꿔 쓰기 쉽다. ② fp는 x8, sp는 x2다. ③ procedure frame과 activation record는 같은 것을 가리키는 두 이름이다. ④ fp가 필요한 이유는 "sp가 프로시저 실행 중에 바뀔 수 있어서"다.

시험 포인트

- procedure frame의 다른 이름(activation record)과 담는 것.
- fp 레지스터 번호(x8)와 필요한 이유.
- 네 영역 이름과 각각 담는 것, malloc/free가 다루는 영역(힙).
- 스택과 힙이 서로를 향해 자라는 이유.

스스로 확인

1. 스택에서 저장된 레지스터와 지역 변수를 담은 구간을 무엇이라 하는가? (두 가지)

   답 보기

   프로시저 프레임(procedure frame), 활성 레코드(activation record).

2. 프레임 포인터는 몇 번 레지스터이며 무엇을 가리키는가?

   답 보기

   x8. 프로시저 프레임의 첫 word.

3. sp가 있는데도 fp를 쓰는 이유는?

   답 보기

   sp는 프로시저 실행 중에 바뀔 수 있어서, 지역 변수 접근에 쓸 변하지 않는 기준 레지스터가 필요하다.

4. C의 malloc()으로 할당한 메모리가 있는 영역은?

   답 보기

   힙(동적 데이터).

5. RISC-V 기계어 코드가 있는 영역의 이름은?

   답 보기

   텍스트(text).

6. 스택과 힙이 서로를 향해 자라도록 배치한 이유는?

   답 보기

   둘이 충돌하기 전까지 남은 저장 공간을 최대한 쓰기 위해.

Ch2 · 12 슬라이드 p.46–48

큰 상수와 분기 주소

이 단원을 마치면 lui와 addi로 32비트 상수를 만들고, SB·UJ 형식의 즉시값 범위를 계산하며, PC 상대 주소 지정이 무엇이고 왜 쓰는지 설명할 수 있다.

쉽게 말하면

계산기 화면이 12자리뿐이라 32자리 수를 한 번에 못 넣는다고 하자. 그러면 앞쪽 20자리를 먼저 넣어 왼쪽으로 밀어 두고, 뒤쪽 12자리를 더한다. lui와 addi가 이렇게 큰 상수를 두 번에 나눠 만든다. 분기 주소도 마찬가지로 칸이 작아서 "몇 번지로 가라" 대신 "여기서 몇 칸 앞(뒤)으로 가라"고 적는다. 이것이 PC 상대 주소다.

비유가 안 맞는 지점 계산기와 달리 addi의 12비트는 부호가 있는 수로 취급된다. 하위 12비트의 맨 앞 비트가 1이면 음수로 더해지므로 위쪽 값을 1 늘려 보정해야 한다.

큰 상수: lui

상수는 대개 작아서 12비트 즉시값 칸에 들어가지만, 가끔 더 큰 상수가 필요하다. 그래서 RISC-V에는 **lui(load upper immediate)**가 있다. lui는 **20비트 상수를 레지스터의 비트 [31:12]에** 넣고, 오른쪽 12비트(비트 [11:0])는 **0으로** 채운다. lui는 **U형식(U-type)**을 쓴다. [Ch2 p.46]

imm[31:12]	rd	opcode
31–12	11–7	6–0

U형식 비트 배치 교재 기준

예: 0x003D0500을 x19에 넣기

목표 값: `00000000 00111101 00000101 00000000` (= 0x003D0500) [Ch2 p.46]

1. 상위 20비트 = `0000 0000 0011 1101 0000` = 0x003D0 = 3 × 256 + 13 × 16 = **976**. 하위 12비트 = `0101 0000 0000` = 0x500 = **1280**.
2. `lui x19, 976` → x19 = `00000000 00111101 00000000 00000000` (하위 12비트는 0)
3. `addi x19, x19, 1280` → x19 = `00000000 00111101 00000101 00000000`

```
lui  x19, 976          // x19 = 0x003D0000
addi x19, x19, 1280    // x19 = 0x003D0000 + 0x500 = 0x003D0500
```

검산: 976 × 4096 = 3,997,696, 여기에 1280을 더하면 3,998,976. 0x003D0500 = 3,998,976. 일치한다. 참고로 `lui x19, 976`의 기계어는 0x003D09B7이다(imm 0x003D0, rd 10011, opcode 0110111). 교재 기준

함정: addi의 부호 확장 PDF 외 보충

addi의 12비트 즉시값은 부호 있는 수(−2048 ~ 2047)로 부호 확장된다. 하위 12비트의 최상위 비트(11번 비트)가 1이면 addi가 음수를 더하게 된다. 예: 0x00001800을 만들려고 `lui x5, 1` (0x1000) + `addi x5, x5, 0x800`을 쓰면, 0x800은 12비트로 −2048이라 결과가 0x1000 − 2048 = 0x0800이 된다. 올바르게는 lui 값을 1 늘려 `lui x5, 2` (0x2000) + `addi x5, x5, -2048` → 0x2000 − 0x800 = **0x1800**. 0x003D0500은 하위 12비트 0x500의 11번 비트가 0이라 이 문제가 없다.

분기 주소: SB 형식

조건 분기(beq, bne 등)는 **SB 형식(SB-type)**을 쓴다. 이 형식은 분기 주소를 **−4096 ~ 4094**(바이트)까지, **2의 배수로만** 나타낼 수 있다. 그래서 **짝수 주소로만** 분기할 수 있다. [Ch2 p.47]

왜 이 범위일까? 즉시값 칸은 12비트지만, 주소가 항상 짝수라 맨 아래 0번 비트(늘 0)는 저장하지 않고 imm[12:1]만 저장한다. 실제 오프셋은 13비트 부호 있는 수 imm[12:0]이 되어 -2^{12} ~ $2^{12} - 2$ = −4096 ~ 4094다. 교재 기준

imm[12] 31	imm[10:5] 30–25	rs2 24–20	rs1 19–15	funct3 14–12	imm[4:1] 11–8	imm[11] 7	opcode 6–0

rs1, rs2, funct3, opcode는 R·I·S 형식과 같은 자리에 있다. 즉시값 비트가 흩어진 것은 이 고정 위치를 지키기 위해서다. [Ch2 p.47, Ch4 p.21] Ch4 p.21 그림은 같은 배치를 `immed[12,10:5] | rs2 | rs1 | funct3 | immed[4:1,11] | opcode`로 적는다.

bne 인코딩 예

슬라이드 예: `bne x10, x11, 2000` (x10 ≠ x11이면 2000으로 간다). bne의 opcode = `1100011`, funct3 = `001`, 오프셋 2000 = `01 11 1101 0000`. [Ch2 p.47] 슬라이드 아래 칸 표의 opcode 칸에는 `1100111`이 적혀 있는데, 같은 슬라이드 위 문장은 "opcode: 1100011 for bne"다. `1100011`이 맞고 칸 표의 `1100111`은 오타다(`1100111`은 jalr의 opcode).

1. 2000 = 1024 + 512 + 256 + 128 + 64 + 16 → 13비트 imm[12:0] = `0 0111 1101 0000`
2. imm[12] = 0, imm[11] = 0, imm[10:5] = `111110`, imm[4:1] = `1000` (imm[0] = 0은 저장 안 함)
3. rs2 = x11 = `01011`, rs1 = x10 = `01010`

0 imm[12]	111110 imm[10:5]	01011 rs2	01010 rs1	001 funct3	1000 imm[4:1]	0 imm[11]	1100011 opcode

4비트씩 묶으면 `0111 1100 1011 0101 0001 1000 0110 0011` = **0x7CB51863**. 슬라이드는 이 비트열을 "imm[12:1] = 0 0111 1101 0000"으로 적었는데, 13자리라 실제로는 0번 비트까지 포함한 imm[12:0]이다. 저장되는 12비트 imm[12:1]은 `0 0111 1101 000`이다.

jal: UJ 형식

무조건 점프-링크 명령어 jal은 **UJ 형식(UJ-type)**을 쓴다. 즉시값이 **20비트**라서 SB보다 범위가 넓다. opcode = `1101111`. [Ch2 p.47]

imm[20] 31	imm[10:1] 30–21	imm[11] 20	imm[19:12] 19–12	rd 11–7	opcode 6–0

슬라이드 예: `jal x0, 2000`. imm[20:1] = `0 0000 0000 0111 1101 0000` (2000을 21비트로 쓴 것)을 나눠 imm[20] = 0, imm[10:1] = `1111101000`, imm[11] = 0, imm[19:12] = `00000000`, rd = `00000`, opcode = `1101111`. [Ch2 p.47] 이어 붙이면

0x7D00006F. 실제 오프셋은 21비트 부호 있는 수(0번 비트는 0)라 $\pm 2^{20}$바이트 = ±1 MiB 범위다.

PC 상대 주소 지정

주소 자체를 20비트 칸에 넣어야 한다면, 어떤 프로그램도 2^{20}바이트(1 MiB)보다 클 수 없다. 오늘날 현실적이지 않다. 대안은 분기 오프셋에 항상 어떤 레지스터를 더하도록 정하는 것이다. [Ch2 p.48]

```
분기 목표 주소 = 레지스터 + 분기 오프셋
```

어떤 레지스터를 고를까? 대부분의 분기 목표는 분기 명령어 **근처**에 있다(앞쪽이든 뒤쪽이든). if, for, while 문의 분기가 그렇다. 그래서 현재 명령어 위치를 담은 **PC가 분기의 기준 레지스터로 가장 알맞다.** 이 방식을 **PC 상대 주소 지정(PC-relative addressing)**이라 한다. [Ch2 p.48]

```
분기 목표 주소 = PC + 오프셋 (PC = 분기 명령어 자신의 주소)
```

RISC-V에서 기준이 되는 PC는 분기 명령어 **자신의** 주소다. 교재 기준

PC 상대 방식 덕분에 분기는 현재 명령어에서 **$\pm 2^{10}$ word**, 점프(jal)는 **$\pm 2^{18}$ word** 범위로 갈 수 있다. [Ch2 p.48]

명령어(형식)	바이트 단위 범위	word 단위 범위
조건 분기(SB)	−4096 ~ 4094 바이트 (p.47)	$\pm 2^{10}$ word (p.48)
jal(UJ)	$\pm 2^{20}$ 바이트 = ±1 MiB	$\pm 2^{18}$ word (p.48)

두 단위는 같은 범위다: 2^{10} word × 4바이트 = 4096바이트, 2^{18} word × 4 = 2^{20}바이트. 오프셋 값 자체는 **바이트 단위**라서 명령어 k개 앞으로 가려면 4k를 쓴다.

아주 먼 점프: lui + jalr

RISC-V는 어떤 32비트 주소로든 멀리 점프할 수 있다. [Ch2 p.48]

1. `lui`로 주소의 비트 [31:12]를 임시 레지스터에 쓴다.
2. `jalr`이 주소의 하위 12비트를 그 레지스터에 더하고 그 합으로 점프한다.

```
// 0x12345678로 호출하기 (하위 12비트 0x678의 11번 비트 = 0)
lui  x5, 0x12345         // x5 = 0x12345000
jalr x1, 0x678(x5)       // PC = 0x12345678, x1 = 다음 명령어 주소
```

하위 12비트의 11번 비트가 1이면 lui 값 보정이 필요한 것은 addi 때와 같다(jalr의 즉시값도 부호 확장). PDF 외 보충

핵심

lui(U형식) = 20비트를 비트 [31:12]에, 하위 12비트 0. 0x003D0500 = `lui x19, 976` + `addi x19, x19, 1280`. SB = −4096 ~ 4094바이트, 짝수만. jal(UJ) = 20비트 즉시값. PC 상대 = 목표가 대부분 가까워서 PC를 기준으로, 분기 $\pm 2^{10}$ word, jal $\pm 2^{18}$ word. 먼 점프 = lui + jalr.

예제

c2-7의 while 반복문 코드가 주소 80000부터 놓여 있다: 80000 `slli`, 80004 `add`, 80008 `lw`, 80012 `bne x9, x24, Exit`, 80016 `addi`, 80020 `beq x0, x0, Loop`, 80024 `Exit:`. bne와 beq에 들어갈 오프셋(바이트)은? (Loop = 80000)

풀이 보기

bne: 목표 80024 − 자기 주소 80012 = **+12** (명령어 3개 앞).

beq: 목표 80000 − 자기 주소 80020 = **−20** (명령어 5개 뒤로). 둘 다 짝수이고 −4096 ~ 4094 범위 안이다. 교재 기준

함정

① SB 범위 −4096 ~ 4094는 **바이트**, p.48의 $\pm 2^{10}$은 **word**다. 단위를 확인하고 답할 것. ② 범위의 위쪽 끝은 4096이 아니고 4094다(짝수만, 부호 있는 13비트의 최댓값 4095에서 홀수 제외). ③ lui는 하위 12비트를 0으로 채운다. 기존 값이 남지 않는다. ④ PC 상대에서 기준은 분기 명령어 자신의 주소이고, 오프셋은 명령어 개수가 아닌 바이트 수다. ⑤ 슬라이드 p.47의 bne 칸 표에 적힌 opcode `1100111`은 오타다. 정답은 `1100011`(조건 분기 공통)이고, `1100111`은 jalr이다. [Ch2 p.47, 50]

시험 포인트

- lui의 동작(20비트 → 비트 [31:12], 하위 12비트 0)과 형식(U).
- 32비트 상수 만들기: lui + addi 값 계산.
- SB 형식 범위 −4096 ~ 4094, 짝수 주소만 분기 가능. jal = UJ 형식, 20비트.
- PC 상대 주소 지정의 정의와 PC를 기준으로 쓰는 이유(목표가 대부분 가까움, 주소를 직접 넣으면 프로그램이 2^{20}바이트로 제한).
- 먼 점프 = lui + jalr. bne opcode 1100011·funct3 001, jal opcode 1101111.

스스로 확인

1. lui를 실행하면 레지스터의 하위 12비트는 어떻게 되는가?

   답 보기

   0으로 채워진다. 20비트 상수는 비트 [31:12]에 들어간다.

2. x19에 0x003D0500을 넣는 명령어 두 개를 쓰시오(10진수 즉시값).

   답 보기

   `lui x19, 976`, `addi x19, x19, 1280`. 0x003D0 = 976, 0x500 = 1280.

3. SB 형식 조건 분기가 나타낼 수 있는 분기 주소 범위(바이트)와 제약은?

답 보기

−4096 ~ 4094, 2의 배수만(짝수 주소로만 분기).

4. jal이 쓰는 형식과 즉시값 비트 수는?

답 보기

UJ 형식, 20비트.

5. PC 상대 주소 지정이란? PC를 기준으로 쓰는 이유는?

답 보기

분기 주소 = PC + 명령어 안의 상수. 대부분의 분기 목표가 분기 명령어 근처(if, for, while)에 있기 때문.

6. 주소 1000의 `beq`가 주소 960으로 분기한다. 오프셋은?

답 보기

960 − 1000 = −40바이트(명령어 10개 뒤로).

7. O/X: `lui x5, 1` 다음 `addi x5, x5, 0x800`을 실행하면 x5 = 0x1800이다.

답 보기

X. 0x800은 12비트 부호 있는 수로 −2048이라 x5 = 0x0800. 0x1800을 만들려면 `lui x5, 2` + `addi x5, x5, -2048`.

8. 임의의 32비트 주소로 점프하는 두 명령어 조합은?

답 보기

lui(상위 20비트를 임시 레지스터에) + jalr(하위 12비트를 더해 점프).

Ch2 · 13 슬라이드 p.49–50

주소 지정 방식과 기계어 해독

이 단원을 마치면 RISC-V의 주소 지정 방식 4가지와 각 방식을 쓰는 명령어를 짝짓고, 16진수 기계어를 어셈블리 명령어로 바꿀 수 있다.

쉽게 말하면

친구에게 물건 위치를 알려 주는 방법은 여러 가지다. "이거야"라고 손에 쥐여 주기(즉시값), "3번 사물함에 있어"(레지스터), "3번 사물함에 적힌 주소에서 8칸 옆 창고"(베이스), "지금 서 있는 곳에서 12걸음 앞"(PC 상대). 주소 지정 방식은 명령어가 피연산자나 목표 주소를 찾는 이런 방법들이다.

비유가 안 맞는 지점 비유에서는 "물건"을 찾지만, PC 상대 방식은 데이터가 아닌 **다음에 실행할 명령어의 주소**를 정한다.

주소 지정 방식 4가지

피연산자나 주소를 정하는 여러 방법을 **주소 지정 방식(addressing mode)**이라 한다. [Ch2 p.49]

방식	피연산자(또는 주소)가 있는 곳	명령어 예
즉시값 주소 지정(immediate addressing)	명령어 안에 들어 있는 상수 자체	addi, xori, srai, …
레지스터 주소 지정(register addressing)	레지스터	add, sub, xor, …
베이스 또는 변위 주소 지정(base or displacement addressing)	메모리. 주소 = 레지스터 + 명령어 안의 상수	lw, sw, **jalr**, …
PC 상대 주소 지정(PC-relative addressing)	분기 주소 = PC + 명령어 안의 상수	beq, bne, jal, …

[Ch2 p.49]

하나의 연산이 여러 주소 지정 방식을 쓸 수 있다. 예를 들어 덧셈은 `add`(레지스터 방식)와 `addi`(즉시값 방식) 두 가지로 할 수 있다. [Ch2 p.49]

함정: jalr은 베이스 방식

jalr은 이름에 "jump"가 있어 PC 상대로 착각하기 쉽다. jalr의 목표 주소는 **레지스터 + 상수**(예: 0 + x1)라서 lw·sw와 같은 **베이스(변위) 방식**이다. 반대로 jal은 PC + 상수이므로 PC 상대 방식이다. [Ch2 p.49]

기계어 해독 4단계

예: 명령어 **0x00578833**을 어셈블리 언어로 바꿔 보자. [Ch2 p.50]

1. **2진수로 바꾸기**: 16진수 한 자리를 4비트로 바꾼다.
   `0 0 5 7 8 8 3 3` → `0000 0000 0101 0111 1000 1000 0011 0011`
2. **opcode와 형식 정하기**: 맨 오른쪽 7비트(비트 [6:0])가 opcode다. `0110011` → R형(add, sub, and, or 등 레지스터끼리의 산술·논리).
3. **형식에 맞춰 필드 나누기**: R형은 funct7(7) · rs2(5) · rs1(5) · funct3(3) · rd(5) · opcode(7).
4. **어셈블리로 쓰기**.

0000000	00101	01111	000	10000	0110011
funct7	rs2 = 5	rs1 = 15	funct3	rd = 16	opcode

funct7 = 0000000, funct3 = 000이고 R형이면 **add**다(sub는 funct7 = 0100000). 어셈블리는 `연산 rd, rs1, rs2` 순서로 쓰므로 결과는 `add x16, x15, x5`. [Ch2 p.50]

필드를 나눌 때는 2진수 32비트를 7 · 5 · 5 · 3 · 5 · 7로 다시 끊는다. 4비트 묶음 경계와 필드 경계가 다르므로 붙여 쓴 뒤 왼쪽부터 세는 것이 안전하다: `0000000 00101 01111 000 10000 0110011`.

opcode	형식	명령어
`0110011`	R	add, sub, and, or 등
`0010011`	I	addi 등 즉시값 산술·논리
`0000011`	I	lw 등 load
`1100111`	I	jalr
`0100011`	S	sw 등 store
`1100011`	SB	beq, bne 등 조건 분기
`0110111`	U	lui
`1101111`	UJ	jal

위 opcode는 모두 슬라이드 인코딩 표에 있다. [Ch2 p.50] 명령어별 funct3·funct7까지 담은 전체 표는 아래에 있다.

슬라이드 인코딩 표

슬라이드 p.50 오른쪽에 있는 표를 그대로 옮긴 것이다. 26-1 시험지에 준 encoding 표가 이런 형태일 가능성이 높으니, 이 표를 보고 형식·opcode·funct3·funct7을 빠르게 찾는 연습을 해 둔다. n.a.는 그 형식에 해당 칸이 없다는 뜻이다. [Ch2 p.50]

형식(Format)	명령어(Instruction)	opcode	funct3	funct7
R	`add`	`0110011`	`000`	`0000000`
R	`sub`	`0110011`	`000`	`0100000`
R	`sll`	`0110011`	`001`	`0000000`
R	`xor`	`0110011`	`100`	`0000000`
R	`srl`	`0110011`	`101`	`0000000`
R	`sra`	`0110011`	`101`	`0100000`
R	`or`	`0110011`	`110`	`0000000`
R	`and`	`0110011`	`111`	`0000000`
R	`lr.d`	`0110011`	`011`	`0001000`
R	`sc.d`	`0110011`	`011`	`0001100`
I	`lb`	`0000011`	`000`	n.a.
I	`lh`	`0000011`	`001`	n.a.
I	`lw`	`0000011`	`010`	n.a.
I	`lbu`	`0000011`	`100`	n.a.
I	`lhu`	`0000011`	`101`	n.a.
I	`addi`	`0010011`	`000`	n.a.
I	`slli`	`0010011`	`001`	`0000000`
I	`xori`	`0010011`	`100`	n.a.
I	`srli`	`0010011`	`101`	`0000000`
I	`srai`	`0010011`	`101`	`0100000`
I	`ori`	`0010011`	`110`	n.a.
I	`andi`	`0010011`	`111`	n.a.
I	`jalr`	`1100111`	`000`	n.a.
S	`sb`	`0100011`	`000`	n.a.
S	`sh`	`0100011`	`001`	n.a.
S	`sw`	`0100011`	`010`	n.a.
SB	`beq`	`1100011`	`000`	n.a.
SB	`bne`	`1100011`	`001`	n.a.
SB	`blt`	`1100011`	`100`	n.a.

형식(Format)	명령어(Instruction)	opcode	funct3	funct7
SB	`bge`	`1100011`	`101`	n.a.
SB	`bltu`	`1100011`	`110`	n.a.
SB	`bgeu`	`1100011`	`111`	n.a.
U	`lui`	`0110111`	n.a.	n.a.
UJ	`jal`	`1101111`	n.a.	n.a.

- R형 `lr.d`, `sc.d`는 표에 있지만 원자적(atomic) 메모리 접근용 명령어라 이 강의 범위 밖이다. 표에서 opcode가 R형과 같다는 것만 보고 넘어가면 된다. PDF 외 보충
- `lb`, `lh`, `lbu`, `lhu`, `sb`, `sh`의 동작은 Ch2 · 3 끝에 정리했다.
- 읽는 순서: opcode로 형식 범위를 좁히고 → funct3로 명령어를 고르고 → 그래도 둘 이상 남으면(add/sub, srl/sra, srli/srai) funct7을 본다.
- `slli` · `srli` · `srai`의 funct7 칸은 실제로는 I형 즉시값의 위 7비트 자리다(Ch2 · 6).

예제

기계어 **0x00412083**을 어셈블리로 바꾸시오.

풀이 보기

1) 2진수: `0000 0000 0100 0001 0010 0000 1000 0011`

2) opcode = `0000011` → I형 load.

3) I형 필드 imm[11:0](12) · rs1(5) · funct3(3) · rd(5) · opcode(7)로 나누면 `000000000100 00010 010 00001 0000011` → imm = 4, rs1 = 2, funct3 = 010(word 단위 load = lw), rd = 1.

4) `lw x1, 4(x2)` = `lw x1, 4(sp)`. fact에서 복귀 주소를 복원하는 바로 그 명령어다.

함정

① 어셈블리 순서는 `rd, rs1, rs2` 인데 기계어 필드 순서는 왼쪽부터 rs2, rs1, rd다. 거꾸로 읽어 `add x5, x15, x16` 으로 쓰기 쉽다. ② opcode는 맨 **오른쪽** 7비트다. ③ add와 sub는 opcode·funct3가 같고 funct7만 다르다. ④ "한 연산 = 한 주소 지정 방식"은 틀리다. 덧셈은 add·addi 둘 다 있다.

시험 포인트

- 주소 지정 방식 4가지 이름과 정의, 명령어 짝짓기(jalr = 베이스, jal = PC 상대).
- "한 연산이 여러 방식을 쓸 수 있다"의 예(add/addi).
- 0x00578833 → `add x16, x15, x5` 를 단계별로 풀기.
- 해독 4단계 순서(2진 변환 → opcode로 형식 결정 → 필드 분리 → 어셈블리).
- 슬라이드 p.50 인코딩 표(시험지에 주는 표와 같은 형태일 가능성이 높음)에서 opcode → funct3 → funct7 순으로 명령어 찾기.

스스로 확인

1. RISC-V 주소 지정 방식 4가지를 쓰시오.

   답 보기

   즉시값, 레지스터, 베이스(변위), PC 상대 주소 지정.

2. 다음 명령어의 주소 지정 방식은? (a) jalr (b) jal (c) srai (d) sub

   답 보기

   (a) 베이스(변위) (b) PC 상대 (c) 즉시값 (d) 레지스터.

3. 베이스(변위) 주소 지정에서 메모리 주소는 어떻게 정해지는가?

   답 보기

   레지스터 값 + 명령어 안의 상수.

4. 0x00578833을 어셈블리로 쓰시오.

   답 보기

   `add x16, x15, x5`. funct7 0, rs2 5, rs1 15, funct3 0, rd 16, opcode 0110011(R형).

5. 기계어 해독에서 형식을 결정하는 필드와 그 위치는?

   답 보기

   opcode, 맨 오른쪽 7비트(비트 [6:0]).

6. 0xFF810113을 어셈블리로 쓰시오. (힌트: opcode 0010011, funct3 000 = addi)

   답 보기

   `1111 1111 1000 0001 0000 0001 0001 0011` → imm = `111111111000` = −8, rs1 = 00010 = 2, funct3 = 000, rd = 00010 = 2. `addi x2, x2, -8` = `addi sp, sp, -8` (fact의 첫 줄).

Ch2 · 14 슬라이드 p.51

프로그램이 실행되기까지

이 단원을 마치면 C 코드가 실행되기까지의 4단계(컴파일러, 어셈블러, 링커, 로더)를 순서대로 쓰고 각 단계가 하는 일과 만들어 내는 파일을 말할 수 있다.

쉽게 말하면

한국어 요리책을 외국 주방에서 쓰려면 먼저 번역하고(컴파일러), 번역문을 주방에서 쓰는 약어 카드로 옮기고(어셈블러), 여러 사람이 만든 카드 묶음과 기본 소스 레시피를 하나로 철하고 쪽수를 다시 매기고(링커), 마지막으로 그 묶음을 조리대에 펼쳐 첫 쪽부터 시작하게 한다(로더).

비유가 안 맞는 지점 번역과 달리 컴파일러는 의미를 유지하는 범위에서 코드를 최적화해 바꿀 수 있고, 링커가 "쪽수를 다시 매기는" 일은 실제 메모리 주소를 고쳐 넣는 일이다.

슬라이드는 C 코드가 컴퓨터에서 실행되는 프로그램이 되기까지 **네 단계**가 있다고 하고, 흐름을 그림으로 보여 준다. C 프로그램 → 컴파일러 → 어셈블리 언어 프로그램 → 어셈블러 → 목적 파일(기계어 모듈) → 링커(라이브러리 루틴 목적 파일도 함께 받음) → 실행 파일(기계어 프로그램) → 로더 → 메모리. [Ch2 p.51] 각 단계가 하는 일의 세부 설명은 교재 기준이다. 교재 기준

단계	입력	하는 일	출력
컴파일러(compiler)	C 프로그램	고수준 언어를 어셈블리 언어로 번역	어셈블리 언어 프로그램
어셈블러(assembler)	어셈블리 언어 프로그램	어셈블리 명령어를 기계어(2진수)로 바꿈, 라벨 주소 기록	목적 파일(object file, 기계어 모듈)
링커(linker)	목적 파일들 + 라이브러리 루틴(기계어)	따로 번역된 여러 목적 파일을 하나로 합치고 주소를 확정	실행 파일(executable file)
로더(loader)	실행 파일	실행 파일을 메모리에 올리고 실행 시작	메모리에서 실행 중인 프로그램

컴파일러

C 같은 고수준 언어 프로그램을 어셈블리 언어 프로그램으로 바꾼다. 어셈블리 언어는 기계어를 사람이 읽을 수 있는 기호로 적은 것이다(예: `add x5, x6, x7`). 교재 기준

어셈블러

어셈블리 언어를 기계어로 바꿔 **목적 파일**을 만든다. 이를 위해 라벨(예: `L1`, `fact`)과 그 주소를 짝지은 **기호표(symbol table)**를 만든다. 교재 기준

어셈블러는 **의사 명령어(pseudoinstruction)**도 처리한다. 의사 명령어는 하드웨어에 실제로는 없지만 어셈블러가 진짜 명령어로 바꿔 주는 편의용 명령어다. 교재 기준

```
li   x9, 123      →  addi x9, x0, 123    // 상수 넣기
mv   x10, x11     →  addi x10, x11, 0    // 값 복사
j    Label        →  jal  x0, Label      // 그냥 점프(링크 버림)
```

목적 파일에는 보통 다음이 들어 있다. 교재 기준

헤더(object file header)	나머지 각 부분의 크기와 위치
텍스트 세그먼트(text segment)	기계어 코드
정적 데이터 세그먼트(static data segment)	프로그램 내내 쓰는 데이터
재배치 정보(relocation information)	프로그램이 메모리에 올라갈 때 절대 주소에 의존하는 명령어·데이터 목록
기호표(symbol table)	외부 참조처럼 아직 정해지지 않은 라벨
디버깅 정보	기계어와 원래 C 코드를 연결하는 정보

링커

프로그램을 여러 파일로 나눠 따로 컴파일·어셈블하면, 한 파일만 고쳐도 그 파일만 다시 번역하면 된다. 이렇게 따로 만든 목적 파일들과 라이브러리 루틴을 하나로 묶는 것이 **링커(link editor라고도 함)**다. 하는 일은 세 가지다. 교재 기준

1. 코드와 데이터 모듈을 메모리에 기호로 배치한다.
2. 데이터와 명령어 라벨의 주소를 정한다.
3. 내부·외부 참조를 고쳐 넣는다(patch).

결과물인 **실행 파일**은 목적 파일과 형식이 같지만, 풀리지 않은 참조가 없다.

로더

운영체제가 실행 파일을 디스크에서 읽어 메모리에 올리고 실행을 시작한다. 순서는 다음과 같다. 교재 기준

1. 실행 파일 헤더를 읽어 텍스트·데이터 크기를 확인한다.
2. 텍스트와 데이터가 들어갈 만큼의 주소 공간을 만든다.
3. 명령어와 데이터를 실행 파일에서 메모리로 복사한다.
4. main 프로그램에 넘길 인자를 스택에 복사한다.
5. 레지스터를 초기화하고 sp를 첫 번째 빈 칸으로 설정한다.
6. 시작 루틴으로 점프한다. 시작 루틴은 인자를 인자 레지스터에 넣고 main을 호출하며, main이 끝나면 exit 시스템 호출로 프로그램을 끝낸다.

> **핵심**
>
> C 프로그램 → **컴파일러** → 어셈블리 언어 → **어셈블러** → 목적 파일(기계어 모듈) → **링커**(+ 라이브러리) → 실행 파일 → **로더** → 메모리에서 실행.

예제

프로그램이 main.c와 util.c 두 파일로 되어 있고, main.c에서 util.c의 함수 `sum`을 부른다. main.o를 만든 시점에 `jal x1, sum`의 목표 주소는 정해져 있는가? 어느 단계에서 정해지는가?

풀이 보기

정해져 있지 않다. 어셈블러는 main.c만 보므로 sum의 위치를 모른다. 그래서 sum을 기호표에 "외부 참조"로, 이 jal을 재배치 정보에 적어 둔다.

링커가 main.o와 util.o를 합치면서 sum의 주소를 정하고 jal의 즉시값을 고쳐 넣는다. 교재 기준

함정

① 순서를 컴파일러 → 링커 → 어셈블러로 바꿔 쓰기 쉽다. 어셈블러가 먼저 목적 파일을 만들어야 링커가 합칠 수 있다. ② 목적 파일과 실행 파일을 구분할 것: 목적 파일은 어셈블러의 출력(외부 참조가 남아 있음), 실행 파일은 링커의 출력. ③ 메모리에 올리는 것은 링커가 아니고 로더다. ④ 의사 명령어는 하드웨어 명령어가 아니다. 어셈블러가 실제 명령어로 바꾼다.

시험 포인트

- 4단계 순서(컴파일러 → 어셈블러 → 링커 → 로더)와 각 단계의 입력·출력.
- 목적 파일과 실행 파일의 차이.
- 링커가 하는 일, 로더가 하는 일.
- 슬라이드는 그림뿐이라 세부 내용은 교재 기준으로 정리했다.

스스로 확인

1. C 코드가 실행되기까지의 4단계를 순서대로 쓰시오.

   답 보기

   컴파일러 → 어셈블러 → 링커 → 로더.

2. 어셈블러의 출력 파일 이름은?

   답 보기

   목적 파일(object file).

3. 여러 목적 파일과 라이브러리를 합쳐 실행 파일을 만드는 프로그램은?

   답 보기

   링커(linker).

4. 실행 파일을 메모리에 올리고 실행을 시작하는 것은?

   답 보기

   로더(loader). 운영체제의 일부다.

5. O/X: 의사 명령어 `mv x10, x11`은 하드웨어가 직접 실행하는 명령어다.

   답 보기

   X. 어셈블러가 `addi x10, x11, 0`으로 바꿔 준다.

Ch3 · 1 슬라이드 p.2–6

덧셈·뺄셈과 오버플로

이 단원을 마치면 2진수 덧셈·뺄셈을 손으로 하고, 결과의 부호만 보고 오버플로인지 판정할 수 있다.

쉽게 말하면

네 자리까지만 보이는 계산기를 떠올리자. 9999 + 1을 누르면 진짜 답 10000은 다섯 자리라서 화면에 담을 수 없다. 컴퓨터의 레지스터도 칸 수(32비트)가 정해져 있어서, 답이 그 칸에 안 들어가면 엉뚱한 값이 남는다. 이것이 오버플로다.

비유가 안 맞는 지점 2의 보수에서는 맨 왼쪽 밖으로 1이 넘쳐 나가는 일(자리올림이 밖으로 나감)이 곧 오버플로는 아니다. 7 − 6을 덧셈으로 계산하면 밖으로 1이 나가지만 답은 맞다. 부호 있는 수의 오버플로는 **결과의 부호가 틀렸는지**로 판정한다.

덧셈: 오른쪽부터 한 비트씩

2진수 덧셈은 십진수와 같다. 가장 오른쪽 비트부터 왼쪽으로 한 자리씩 더하고, 2가 되면 0을 쓰고 1을 왼쪽 자리로 넘긴다. 넘기는 1을 **자리올림(carry)**이라 한다. [Ch3 p.2]

```
자리올림        0000 ... 0000 1100
  7 =     0000 ... 0000 0111
+ 6 =     0000 ... 0000 0110
------------------------------
 13 =     0000 ... 0000 1101
```

비트 0: 1+0 = 1. 비트 1: 1+1 = 10_2 → 0 쓰고 1 올림. 비트 2: 1+1+올림 1 = 11_2 → 1 쓰고 1 올림. 비트 3: 0+0+올림 1 = 1. 결과 1101_2 = 13. 32비트 레지스터라서 앞의 28비트는 모두 0이다.

뺄셈: 부정해서 더한다

뺄셈은 덧셈으로 한다. 빼는 수(두 번째 피연산자)를 **부정(negate)**해서, 즉 2의 보수로 부호를 바꿔서 더한다. Ch2에서 배운 대로 −x는 모든 비트를 뒤집고 1을 더한 값(`~x + 1`)이다. [Ch3 p.3] [Ch2 p.20]

```
직접 빼기                    부정해서 더하기 (-6 = 1111 ... 1111 1010)
   0000 ... 0000 0111 (7)       0000 ... 0000 0111 (7)
 - 0000 ... 0000 0110 (6)     + 1111 ... 1111 1010 (-6)
 --------------------         --------------------
   0000 ... 0000 0001 (1)     1 0000 ... 0000 0001 (1, 맨 왼쪽 밖의 1은 버림)
```

두 방법의 답이 같다. 그래서 하드웨어는 덧셈기 하나로 뺄셈까지 처리한다. 덧셈 쪽에서 33번째 자리로 나간 1은 32비트 칸 밖이라 버려지고, 이것은 오류가 아니다.

오버플로의 정의

오버플로(overflow) 연산 결과를 주어진 하드웨어(정해진 비트 수)로 표현할 수 없는 상황. [Ch3 p.4]

부호 비트(sign bit) 2의 보수에서 가장 왼쪽 비트(MSB). 0이면 0 이상, 1이면 음수.

32비트 두 수를 더하거나 빼면 결과를 다 쓰는 데 33비트가 필요할 수 있다. 32비트 칸에는 33비트 중 아래 32비트만 남으므로, 부호 비트 자리에 원래 부호 대신 **결과값의 비트**가 들어간다. 그래서 오버플로가 나면 부호가 뒤집혀 보인다. [Ch3 p.5]

슬라이드 예: A = 0x00000001(양수 1), B = 0x7FFFFFFF(양수 $2^{31} - 1$, 32비트 최대 양수)를 더하면 S = 0x80000000이다. 부호 비트가 1이라 음수(-2^{31})로 읽힌다. 양수 + 양수가 음수가 됐으니 오버플로다. [Ch3 p.4]

```
A = 0000 0000 0000 0000 0000 0000 0000 0001   (> 0)
B = 0111 1111 1111 1111 1111 1111 1111 1111   (> 0)
S = 1000 0000 0000 0000 0000 0000 0000 0000   (< 0)  → 오버플로
```

오버플로가 생기지 않는 경우

- **부호가 다른 두 수의 덧셈**에서는 오버플로가 생기지 않는다. 합은 두 피연산자 사이에 있기 때문이다. 예: −10 + 4 = −6이고 −10 < −6 < 4. 두 수가 이미 표현 가능하니 그 사이 값도 표현 가능하다. [Ch3 p.4]
- **부호가 같은 두 수의 뺄셈**에서도 생기지 않는다. c − a = c + (−a)이고, a를 부정하면 부호가 바뀌어 "부호가 다른 두 수의 덧셈"이 되기 때문이다. [Ch3 p.4]

오버플로가 생기는 4가지 조건

연산	A	B	결과가 이러면 오버플로	기호로
A + B	양수(≥ 0)	양수(≥ 0)	음수(< 0)	+ + + = −
A + B	음수(< 0)	음수(< 0)	0 이상(≥ 0)	− + − = +
A − B	양수(≥ 0)	음수(< 0)	음수(< 0)	+ − − = −
A − B	음수(< 0)	양수(≥ 0)	0 이상(≥ 0)	− − + = +

이 넷 말고는 오버플로가 없다. 정리하면 덧셈은 부호가 같을 때, 뺄셈은 부호가 다를 때만 오버플로를 의심하고, 결과 부호가 "있어야 할 부호"와 반대면 오버플로다. [Ch3 p.5]

무부호 정수의 오버플로

무부호(unsigned) 정수는 음수가 없는 정수다. 주로 메모리 주소에 쓰고, 주소 계산의 오버플로는 보통 무시한다. 그래도 판정법은 알아야 한다. [Ch3 p.6]

- 덧셈: A + B = S일 때 **S < A 또는 S < B**면 오버플로. 더했는데 오히려 작아졌다는 뜻이다.
- 뺄셈: A − B = S일 때 **S > A**면 오버플로. 뺐는데 피감수(minuend, 빼임을 당하는 수 A)보다 커졌다는 뜻이다.

예(4비트 무부호): 1100_2(12) + 0110_2(6) = 1 0010_2 → 4비트에는 0010_2(2)만 남는다. 2 < 12이므로 오버플로. 0011_2(3) − 0101_2(5) = 1110_2(14)이고 14 > 3이므로 오버플로. PDF 외 보충

언어별 처리

오버플로를 어떻게 다룰지는 컴퓨터 설계자가 정해야 한다. C/C++와 Java는 정수 오버플로를 **무시**한다. Ada와 Fortran은 프로그램에 **알려 주도록(통보)** 요구한다. 결국 오버플로 처리는 프로그래머의 책임이다. [Ch3 p.6]

핵심

뺄셈 = 두 번째 피연산자를 부정해서 더하기. 오버플로는 "+ + + = −, − + − = +, + − − = −, − − + = +" 네 경우뿐이다. 부호가 다른 덧셈, 부호가 같은 뺄셈은 절대 오버플로가 아니다. 무부호는 덧셈 S < A 또는 S < B, 뺄셈 S > A.

예제

8비트 2의 보수에서 (1) 0110 0100(100) + 0011 0010(50), (2) 1000 0000(−128) − 0000 0001(1), (3) 0101 0000(80) − 0011 0000(48)을 계산하고 각각 오버플로인지 판정하라.

풀이 보기

(1) 0110 0100 + 0011 0010 = 1001 0110. 양수 + 양수인데 부호 비트가 1(음수, −106) → **오버플로**. 실제 답 150은 8비트 최대 127을 넘는다.

(2) −128 − 1 = −128 + (−1) = 1000 0000 + 1111 1111 = 1 0111 1111 → 0111 1111(127). 음수 − 양수인데 결과가 양수 → **오버플로** (− − + = +).

(3) 양수 − 양수, 부호가 같은 뺄셈이므로 **오버플로 불가**. 80 + (−48): 0101 0000 + 1101 0000 = 1 0010 0000 → 0010 0000 = 32. 밖으로 나간 1은 버리며 오류가 아니다.

함정

"맨 왼쪽에서 자리올림이 나가면 오버플로"는 틀렸다. (3)처럼 자리올림이 나가도 답이 맞을 수 있고, (1)처럼 자리올림이 안 나가도 오버플로일 수 있다. 부호 있는 수는 **피연산자 부호와 결과 부호**로 판정한다.

슬라이드의 7 + 6 = 1101_2도 32비트로 계산했기 때문에 13이다. 4비트 부호 있는 수로만 보면 1101_2은 −3이고, 이때는 + + + = −라서 오버플로다.

시험 포인트

- 오버플로 정의 한 줄: 결과를 사용 가능한 하드웨어로 표현할 수 없는 것.
- 오버플로가 생기는 4조건, 생기지 않는 2조건(부호 다른 덧셈, 부호 같은 뺄셈)과 그 이유.
- 0x00000001 + 0x7FFFFFFF = 0x80000000 → 양수 + 양수 = 음수.
- 무부호 판정식(S < A 또는 S < B / S > A), C·Java는 무시, Ada·Fortran은 통보.

스스로 확인

1. 컴퓨터는 뺄셈 A − B를 어떻게 덧셈으로 바꾸는가?

   답 보기

   B를 부정(2의 보수: 비트 반전 + 1)해서 A에 더한다. A − B = A + (−B).

2. O/X: 양수에서 양수를 빼면 오버플로가 생길 수 있다.

답 보기

X. 부호가 같은 뺄셈은 부정하면 부호가 다른 덧셈이 되어 결과가 두 값 사이에 있으므로 오버플로가 없다.

3. 뺄셈에서 오버플로가 생기는 두 경우를 부호 기호로 쓰라.

답 보기

+ − − = −(양수에서 음수를 뺐는데 음수), − − + = +(음수에서 양수를 뺐는데 양수).

4. 0x7FFFFFFF + 0x00000001의 32비트 결과와 오버플로 여부는?

답 보기

0x80000000, 오버플로. 양수 + 양수인데 부호 비트가 1이다.

5. 무부호 덧셈 A + B = S에서 오버플로 판정 조건은?

답 보기

S < A 또는 S < B (더했는데 피가수보다 작아짐).

6. 정수 오버플로를 무시하는 언어 두 개와 프로그램에 통보하도록 요구하는 언어 두 개는?

답 보기

무시: C/C++, Java. 통보: Ada, Fortran.

Ch3 · 2 슬라이드 p.7–11

곱셈 하드웨어

이 단원을 마치면 곱셈기 1판과 개선판의 구성을 설명하고, $0010_2 \times 0011_2$의 반복 표를 혼자 채울 수 있다.

쉽게 말하면

초등학교 세로 곱셈과 같다. 아래 수(승수)의 한 자리씩 보면서 위 수(피승수)를 한 칸씩 왼쪽으로 밀어 적고, 마지막에 모두 더한다. 2진수는 자리값이 0 아니면 1이라서 "피승수를 그대로 적을지, 0을 적을지"만 고르면 된다. 하드웨어는 이것을 "검사 → 더하기 → 밀기"의 반복으로 한다.

비유가 안 맞는 지점 사람은 줄을 다 적은 뒤 한꺼번에 더하지만, 곱셈기는 한 줄을 만들 때마다 바로 곱 레지스터에 더해 둔다(누적). 또 개선판은 피승수를 밀지 않고 곱 쪽을 오른쪽으로 민다.

용어와 손 곱셈

피승수(multiplicand)	곱해지는 수. 세로 곱셈의 위쪽 수.
승수(multiplier)	곱하는 수. 세로 곱셈의 아래쪽 수. 이 수의 각 비트가 "더할지 말지"를 정한다.
곱(product)	결과.
LSB(Least Significant Bit)	가장 오른쪽 비트(최하위 비트). 반대쪽 끝은 MSB(최상위 비트).

```
   1000      (피승수, 8)
 × 1001      (승수, 9)
 ------
   1000      ← 승수 비트 0 = 1 → 피승수 복사
  0000       ← 승수 비트 1 = 0 → 0
 0000        ← 승수 비트 2 = 0 → 0
1000         ← 승수 비트 3 = 1 → 피승수 복사 (한 칸씩 왼쪽으로 밀림)
-------
1001000      (곱, 72)
```

한 단계는 단순하다. ① 승수 비트가 1이면 피승수를 알맞은 자리에 복사하고 ② 0이면 그 자리에 0을 놓는다. 피승수는 단계마다 한 칸씩 왼쪽으로 밀린다. [Ch3 p.7]

```
곱의 길이 = 피승수 길이 + 승수 길이
```

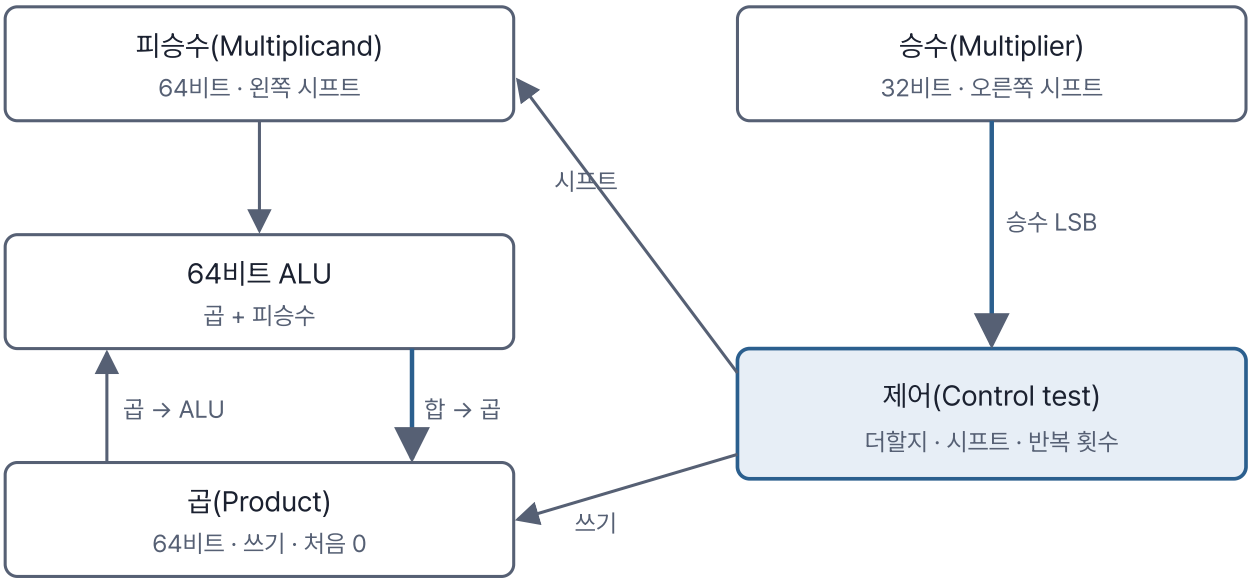
4비트 × 4비트면 곱은 최대 8비트, 32비트 × 32비트면 64비트가 필요하다. 그래서 곱셈기의 곱 레지스터는 64비트다. [Ch3 p.7]

곱셈기 1판: 구성

- **승수 레지스터 32비트**: 오른쪽으로 1비트씩 시프트. 매번 LSB만 검사한다.
- **피승수 레지스터 64비트**: 32비트 피승수를 **오른쪽 절반**에 넣고 시작(왼쪽 절반은 0). 왼쪽으로 1비트씩 시프트.
- **64비트 ALU**(산술 논리 장치, Arithmetic and Logic Unit): 곱과 피승수를 더한다.
- **곱 레지스터 64비트**: 처음엔 0. 더한 결과를 쓴다.

- **제어(Control test)**: 승수 LSB를 보고 더할지, 언제 시프트할지, 몇 번 반복했는지 정한다.

동작 방식을 Test-and-Add-and-Shift(검사-더하기-시프트)라 부른다. [Ch3 p.8]



곱셈기 1판. 승수의 LSB가 1이면 피승수를 곱에 더하고, 피승수는 왼쪽·승수는 오른쪽으로 민다. [Ch3 p.8]

1판 알고리즘

1. 승수의 LSB를 검사한다. 1이면 곱 = 곱 + 피승수. 0이면 아무것도 안 한다.
2. 피승수 레지스터를 왼쪽으로 1비트 시프트한다.
3. 승수 레지스터를 오른쪽으로 1비트 시프트한다.
4. 승수 비트 수만큼(32비트면 32번, 4비트 예에서는 4번) 반복한다.

[Ch3 p.9]

1판 반복 표: $0010_2 \times 0011_2$ (2 × 3)

4비트 예라서 피승수·곱 레지스터는 8비트, 승수는 4비트다. 피승수 0010_2은 8비트 레지스터의 오른쪽 절반에 들어가 0000 0010으로 시작한다. [Ch3 p.9]

반복	단계	승수(2진)	피승수(2진)	곱(2진)
0	초기값	0011	0000 0010	0000 0000
1	1: LSB 1 → 곱 = 곱 + 피승수	0011	0000 0010	**0000 0010**
	2: 피승수 왼쪽 시프트	0011	**0000 0100**	0000 0010
	3: 승수 오른쪽 시프트	**0001**	0000 0100	0000 0010
2	1: LSB 1 → 곱 = 곱 + 피승수	0001	0000 0100	**0000 0110**
	2: 피승수 왼쪽 시프트	0001	**0000 1000**	0000 0110
	3: 승수 오른쪽 시프트	**0000**	0000 1000	0000 0110
3	1: LSB 0 → 아무것도 안 함	0000	0000 1000	0000 0110
	2: 피승수 왼쪽 시프트	0000	**0001 0000**	0000 0110
	3: 승수 오른쪽 시프트	0000	0001 0000	0000 0110
4	1: LSB 0 → 아무것도 안 함	0000	0001 0000	0000 0110
	2: 피승수 왼쪽 시프트	0000	**0010 0000**	0000 0110
	3: 승수 오른쪽 시프트	0000	0010 0000	0000 0110

반복이 끝난 뒤의 값은 슬라이드 그림과 같다. 반복 1: 피승수 0000 0100, 승수 0001_2, 곱 0000 0010. 반복 2: 0000 1000, 0000_2, 0000 0110. 반복 3: 0001 0000, 0000_2, 0000 0110. 반복 4: 0010 0000, 0000_2, 0000 0110. 최종 곱 0000 0110 = 6. [Ch3 p.9]

곱셈기 개선판

1판은 64비트 피승수 레지스터와 64비트 ALU를 쓰지만, 실제로 더해지는 의미 있는 비트는 32비트뿐이라 낭비가 크다. 개선판은 이렇게 바꾼다. [Ch3 p.10]

부품	1판	개선판
피승수 레지스터	64비트, 왼쪽 시프트	**32비트**, 시프트 안 함
ALU	64비트	**32비트**
승수 레지스터	32비트	**없음**(곱 레지스터 오른쪽 절반에 넣음)
곱 레지스터	64비트, 처음 0	64비트, 왼쪽 절반 0 + 오른쪽 절반 승수
더하는 곳	곱 전체(64비트)	곱의 **왼쪽 절반**에 피승수를 더해 왼쪽 절반에 저장
시프트	피승수 왼쪽, 승수 오른쪽	**곱 레지스터를 오른쪽**으로 1비트

왜 되는가: 피승수를 왼쪽으로 미는 것과 곱을 오른쪽으로 미는 것은 둘의 상대 위치를 똑같이 바꾼다. 곱을 오른쪽으로 밀면 승수 비트는 오른쪽 끝에서 하나씩 빠져나가고(검사가 끝난 비트는 필요 없음), 그 빈자리를 곱의 비트가 왼쪽부터 채운다. 그래서 승수 레지스터를 따로 둘 필요가 없다. [Ch3 p.10]

개선판 한 번의 반복: ① 곱 레지스터의 LSB(= 아직 안 쓴 승수의 LSB)가 1이면 피승수를 곱의 왼쪽 절반에 더한다 ② 곱 레지스터 전체를 오른쪽으로 1비트 시프트한다. [Ch3 p.11]

개선판 반복 표: $0010_2 \times 0011_2$

반복	단계	피승수(2진)	곱 : 승수(2진)
0	초기값	0010	0000 0011
1	1: LSB 1 → 곱 = 곱 + 피승수(왼쪽 절반)	0010	**0010** 0011
	2: 곱 오른쪽 시프트	0010	0001 0001
2	1: LSB 1 → 곱 = 곱 + 피승수	0010	**0011** 0001
	2: 곱 오른쪽 시프트	0010	0001 1000
3	1: LSB 0 → 더하지 않음	0010	0001 1000
	2: 곱 오른쪽 시프트	0010	0000 1100
4	1: LSB 0 → 더하지 않음	0010	0000 1100
	2: 곱 오른쪽 시프트	0010	**0000 0110**

값은 슬라이드 p.11 표와 같다. 최종 곱 0000 0110 = 6. 슬라이드 표는 반복 3·4의 단계 이름도 "Prod = Prod+Mcand"로 적었지만 그때 LSB가 0이라 실제로는 더하지 않으며, 그래서 값이 바뀌지 않는다. [Ch3 p.11]

반복 2의 덧셈을 풀어 보면: 왼쪽 절반 0001_2 + 피승수 0010_2 = 0011_2, 오른쪽 절반 0001_2은 그대로 → 0011 0001.

핵심

1판 = 승수 32 · 피승수 64 · ALU 64 · 곱 64, Test-Add-Shift(피승수 왼쪽, 승수 오른쪽). 개선판 = 피승수 32 · ALU 32 · 승수 레지스터 없음(곱 오른쪽 절반), 곱 왼쪽 절반에 더하고 곱을 오른쪽 시프트. $0010_2 \times 0011_2$ = 0000 0110.

예제

개선판 곱셈기로 $0011_2 \times 0101_2$(3 × 5)을 계산하는 반복 표를 써라.

풀이 보기

초기: 피승수 0011_2, 곱 0000 0101.

반복 1: LSB 1 → 0000_2 + 0011_2 = 0011_2 → 0011 0101 → 시프트 0001 1010.

반복 2: LSB 0 → 그대로 → 시프트 0000 1101.

반복 3: LSB 1 → 0000_2 + 0011_2 = 0011_2 → 0011 1101 → 시프트 0001 1110.

반복 4: LSB 0 → 그대로 → 시프트 **0000 1111** = 15. 맞다.

함정

1판에서 시프트하는 쪽을 헷갈리기 쉽다. 피승수는 **왼쪽**, 승수는 **오른쪽**이다. 개선판에서 시프트하는 것은 피승수가 아니라 **곱 레지스터(오른쪽)**다.

개선판에서 피승수는 곱 레지스터 전체가 아니라 **왼쪽 절반**에 더한다. 오른쪽 절반(남은 승수)은 덧셈에서 건드리지 않는다.

시험 포인트

- 곱의 길이 = 피승수 길이 + 승수 길이 (32 × 32 → 64비트).
- 1판·개선판 각 레지스터와 ALU의 비트 폭, 시프트 방향.
- $0010_2 \times 0011_2$ 반복 표의 빈칸 채우기(두 판 모두).
- 개선판에서 승수 레지스터가 없어지는 이유.

스스로 확인

1. 16비트 수와 8비트 수를 곱하면 곱은 최대 몇 비트인가?

   답 보기

   24비트. 곱의 길이 = 두 피연산자 길이의 합.

2. 곱셈기 1판에서 피승수 레지스터와 ALU는 각각 몇 비트이고, 32비트 피승수는 처음에 어디에 놓이는가?

   답 보기

   둘 다 64비트. 피승수는 피승수 레지스터의 오른쪽 절반에 놓인다(왼쪽 절반 0).

3. 1판 $0010_2 \times 0011_2$에서 반복 2가 끝난 뒤 피승수·승수·곱 값은?

   답 보기

   피승수 0000 1000, 승수 0000_2, 곱 0000 0110.

4. 개선판 $0010_2 \times 0011_2$에서 반복 1이 끝난 뒤(시프트 후) 곱 레지스터 값은?

   답 보기

   0001 0001. 0000 0011 → 왼쪽 절반에 0010_2 더해 0010 0011 → 오른쪽 시프트.

5. 개선판에 승수 레지스터가 필요 없는 이유는?

답 보기

승수를 곱 레지스터의 오른쪽 절반에 넣고, 곱을 오른쪽으로 밀 때마다 검사 끝난 승수 비트가 빠지고 그 자리를 곱이 채우기 때문.

6. O/X: 개선판 곱셈기는 피승수를 매 반복 왼쪽으로 시프트한다.

답 보기

X. 피승수는 고정이고 곱 레지스터를 오른쪽으로 시프트한다.

Ch3 · 3 슬라이드 p.12–13

부호 있는 곱셈과 RISC-V 곱셈 명령

이 단원을 마치면 부호 있는 곱셈 절차를 말하고, mul·mulh·mulhu·mulhsu가 각각 무엇을 돌려주는지 3비트 예로 설명할 수 있다.

쉽게 말하면

(−3) × 5를 손으로 할 때 우리는 먼저 3 × 5 = 15를 구하고, 부호가 서로 다르니 앞에 −를 붙인다. 컴퓨터도 같은 방법을 쓸 수 있다. 크기만 곱하고 부호는 따로 정한다.

비유가 안 맞는 지점 같은 비트 묶음 101_2이 무부호로는 5, 부호 있는 수로는 −3이다. 그래서 같은 비트를 곱해도 "어떻게 읽느냐"에 따라 곱의 위쪽 절반이 달라지고, RISC-V는 이 차이 때문에 위쪽 절반을 구하는 명령을 세 개나 둔다.

부호 있는 곱셈

1. 승수와 피승수를 양수로 바꾼다(음수면 부정).
2. 곱셈 알고리즘을 **31번** 반복한다. 부호 비트는 계산에서 뺀다.
3. 원래 두 부호가 서로 다르면 곱을 부정한다.

[Ch3 p.12] 31번인 이유: 32비트 부호 있는 수에서 1비트는 부호이고 크기는 나머지 31비트이기 때문이다.

빠른 곱셈

덧셈기를 여러 개 쓰면 곱셈이 빨라진다. 승수의 각 비트에 대해 "피승수 또는 0"을 미리 만들어 두고, 이것들을 여러 덧셈기로 동시에 두 개씩 더해 나가는 **병렬 트리 덧셈기(parallel tree-based adder)**를 쓴다. 덧셈기를 많이 쓰니 비용이 늘고 대신 빨라진다. 즉 비용과 성능의 맞바꿈(tradeoff)이다. [Ch3 p.12]

32비트라면 덧셈 32번을 차례로 하는 대신, 트리 높이인 $\log_2 32 = 5$단계의 덧셈 시간이면 된다. 교재 기준

RISC-V 곱셈 명령 4개

32비트 × 32비트의 올바른 64비트 곱(부호 있음·무부호 모두)을 얻기 위해 RISC-V는 곱셈 명령 4개를 둔다. 레지스터는 32비트라 64비트 곱을 한 번에 못 담으므로, 아래 32비트와 위 32비트를 따로 구한다. [Ch3 p.13]

명령	이름	돌려주는 값
`mul`	multiply	64비트 곱의 **아래 32비트**(정수 32비트 곱)
`mulh`	mul high	위 32비트, 두 피연산자 모두 **부호 있음**
`mulhu`	mulh unsigned	위 32비트, 두 피연산자 모두 **무부호**
`mulhsu`	mulh signed-unsigned	위 32비트, 하나는 부호 있음·하나는 무부호

`mul`이 하나뿐인 이유: 아래 절반은 부호 해석과 상관없이 늘 같다. 위 절반만 해석에 따라 달라지므로 위 절반 명령만 셋이다. 64비트 곱 전체가 필요하면 `mul`과 `mulh` (또는 `mulhu`, `mulhsu`)를 함께 쓴다.

3비트 예: $101_2 \times 101_2 \rightarrow$ 6비트 곱

모드	A	B	A × B	위 3비트(2진)	아래 3비트(2진)
무부호	5 (101_2)	5 (101_2)	25 (011001_2)	011_2	001_2
부호 있음	−3 (101_2)	−3 (101_2)	9 (001001_2)	001_2	001_2
부호-무부호	−3 (101_2)	5 (101_2)	−15 (110001_2)	110_2	001_2

세 경우 모두 입력 비트는 101_2, 101_2로 같다. 아래 3비트(001_2)는 모두 같고 위 3비트만 다르다. [Ch3 p.13]

검산: 25 = 16+8+1 = 011001_2. 9 = 001001_2. −15는 6비트 2의 보수로 64 − 15 = 49 = 110001_2.

핵심

부호 있는 곱셈 = 양수로 바꿔 31회 → 부호가 다르면 부정. 빠른 곱셈 = 여러 덧셈기로 병렬 트리(비용 ↔ 성능). mul = 아래 32비트, mulh = 위(부호×부호), mulhu = 위(무부호×무부호), mulhsu = 위(부호×무부호).

예제

3비트 A = 111_2, B = 010_2일 때 무부호, 부호 있음, 부호-무부호(A 부호 있음, B 무부호) 각각의 6비트 곱과 위·아래 3비트를 구하라.

풀이 보기

무부호: 7 × 2 = 14 = 001110_2 → 위 001_2, 아래 110_2.

부호 있음: −1 × 2 = −2 = 111110_2 → 위 111_2, 아래 110_2.

부호-무부호: A = −1, B = 2 → −2 = 111110_2 → 위 111_2, 아래 110_2.

아래 3비트는 셋 다 110_2으로 같다.

함정

부호 있는 곱셈의 반복 횟수는 32가 아니라 **31**이다(부호 비트 제외). 무부호 32비트 곱셈은 32번이다.

"mulhu는 아래 32비트를 무부호로 구한다"는 틀렸다. mulh·mulhu·mulhsu는 모두 **위 32비트**다. 아래 32비트는 부호와 관계없이 mul 하나로 끝난다.

시험 포인트

- 부호 있는 곱셈 3단계와 반복 횟수 31.
- 빠른 곱셈 = 여러 덧셈기의 병렬 트리, 비용과 성능의 tradeoff.
- 네 명령의 이름과 결과, 3비트 표의 빈칸.

스스로 확인

1. 부호 있는 곱셈의 세 단계를 순서대로 쓰라.

   답 보기

   ① 두 수를 양수로 바꿈 ② 31회 곱셈(부호 제외) ③ 원래 부호가 다르면 곱을 부정.

2. 두 피연산자가 모두 무부호일 때 64비트 곱의 위 32비트를 구하는 RISC-V 명령은?

   답 보기

   `mulhu`.

3. RISC-V에서 아래 32비트를 구하는 명령이 하나뿐인 이유는?

   답 보기

   곱의 아래 절반은 부호 있음·무부호 해석과 상관없이 같기 때문.

4. 3비트 $101_2 \times 101_2$을 부호 있는 수로 곱한 6비트 결과는?

   답 보기

   $(-3) \times (-3) = 9 = 001001_2$.

5. 여러 덧셈기를 트리로 쓰는 빠른 곱셈의 장단점은?

   답 보기

   빠르지만 덧셈기가 많아 비용이 크다(비용과 성능의 tradeoff).

Ch3 · 4 슬라이드 p.14–18

나눗셈 하드웨어

이 단원을 마치면 나눗셈기 1판·개선판의 구성을 설명하고, 0000 0111 ÷ 0010_2의 반복 표를 혼자 채울 수 있다.

쉽게 말하면

세로 나눗셈에서 우리는 "이 자리에서 제수를 뺄 수 있나?"를 눈으로 보고 판단한다. 하드웨어는 눈이 없어서 **일단 빼 본다**. 결과가 음수가 되면 "못 빼는 자리였구나" 하고 다시 더해 원래대로 돌려놓고 몫에 0을 적는다. 음수가 아니면 몫에 1을 적는다.

비유가 안 맞는 지점 사람은 자리를 옮겨 가며 피제수 숫자를 "내려 쓰지만", 1판 하드웨어는 제수를 오른쪽으로 밀고, 개선판은 나머지 레지스터를 왼쪽으로 민다. 움직이는 쪽이 다르다.

용어와 손 나눗셈

피제수(dividend) 나뉘는 수.

제수(divisor) 나누는 수. 0이면 안 된다.

몫(quotient) 나눗셈의 주 결과.

나머지(remainder) 나누고 남은 수.

```
          1001        (몫, 9)
        --------
1000 ) 1001010        (피제수 74, 제수 8)
       -1000          ← 1001 ≥ 1000 → 몫 1, 뺌
       ------
          0010        ← 0 내림 → 0010 < 1000 → 몫 0
          0101        ← 1 내림 → 0101 < 1000 → 몫 0
          1010        ← 0 내림 → 1010 ≥ 1000 → 몫 1
         -1000
         ------
            10        (나머지, 2)
```

```
피제수 = 몫 × 제수 + 나머지
```

검산: 9 × 8 + 2 = 74. 2진 나눗셈은 시도할 때마다 제수를 빼 보며 몫을 한 자리씩 만든다. 0으로 나누기는 수학적으로 정의되지 않으므로 제수는 0이 아니어야 한다. [Ch3 p.14]

나눗셈기 1판: 구성

- 곱셈기 1판과 **같은 하드웨어**를 쓴다. 다른 점은 시프트 방향이 반대이고, 덧셈 대신 **뺄셈**을 한다는 것.
- **제수 레지스터 64비트**: 32비트 제수를 **왼쪽 절반**에 넣고 시작. 오른쪽으로 1비트씩 시프트.
- **64비트 ALU**: 나머지에서 제수를 뺀다(또는 복원할 때 더한다).
- **나머지 레지스터 64비트**: 피제수로 초기화한다.
- **몫 레지스터 32비트**: 왼쪽으로 1비트씩 시프트하며 새 몫 비트를 오른쪽 끝에 넣는다.

동작 방식을 Subtract-and-Test-and-Shift(빼기-검사-시프트)라 부른다. 32비트 나눗셈은 **33번** 반복한다(n비트면 n+1번). [Ch3 p.15]

1판 알고리즘

1. 나머지 = 나머지 − 제수.
2. 나머지의 MSB(부호 비트)를 검사한다.
   - MSB가 0(나머지 ≥ 0): 몫을 왼쪽으로 시프트하고 맨 오른쪽 비트를 1로.
   - MSB가 1(나머지 < 0): 제수를 다시 더해 **원래 값으로 복원**하고, 몫을 왼쪽으로 시프트하며 맨 오른쪽 비트는 0.
3. 제수를 오른쪽으로 1비트 시프트한다.
4. n+1번 반복한다(4비트 예는 5번).

[Ch3 p.16] 음수가 나오면 다시 더해 되돌리는 이 방식을 **복원(restoring) 나눗셈**이라 한다. 반복이 n+1번인 이유: 제수가 왼쪽 절반(n비트만큼 왼쪽으로 밀린 위치)에서 출발하므로, n칸 밀린 위치부터 0칸 위치까지 n+1개의 자리를 모두 시험해야 하기 때문이다.

1판 반복 표: 0000 0111 ÷ 0010_2 (7 ÷ 2)

4비트 예라서 제수·나머지 레지스터는 8비트, 몫은 4비트다. 제수 0010_2은 왼쪽 절반에 들어가 0010 0000으로 시작한다. [Ch3 p.16]

반복	단계	몫(2진)	제수(2진)	나머지(2진)
0	초기값	0000	0010 0000	0000 0111
1	1: 나머지 = 나머지 − 제수	0000	0010 0000	**1110 0111**
	2: 나머지 < 0 → 복원, 몫 sll, Q0 = 0	0000	0010 0000	0000 0111
	3: 제수 오른쪽 시프트	0000	0001 0000	0000 0111
2	1: 나머지 = 나머지 − 제수	0000	0001 0000	**1111 0111**
	2: 나머지 < 0 → 복원, 몫 sll, Q0 = 0	0000	0001 0000	0000 0111
	3: 제수 오른쪽 시프트	0000	0000 1000	0000 0111
3	1: 나머지 = 나머지 − 제수	0000	0000 1000	**1111 1111**
	2: 나머지 < 0 → 복원, 몫 sll, Q0 = 0	0000	0000 1000	0000 0111
	3: 제수 오른쪽 시프트	0000	0000 0100	0000 0111
4	1: 나머지 = 나머지 − 제수	0000	0000 0100	**0000 0011**
	2: 나머지 ≥ 0 → 몫 sll, Q0 = 1	**0001**	0000 0100	0000 0011
	3: 제수 오른쪽 시프트	0001	0000 0010	0000 0011
5	1: 나머지 = 나머지 − 제수	0001	0000 0010	**0000 0001**
	2: 나머지 ≥ 0 → 몫 sll, Q0 = 1	**0011**	0000 0010	0000 0001
	3: 제수 오른쪽 시프트	0011	0000 0001	0000 0001

최종: 몫 0011_2(3), 나머지 0000 0001(1). 7 = 3 × 2 + 1. 슬라이드 그림의 값(뺀 결과 1110 0111, 1111 0111, 1111 1111 → 복원, 그다음 0000 0011, 0000 0001)과 같다. 뺄셈 예: 0000 0111 − 0010 0000 = 7 − 32 = −25 = 1110 0111. [Ch3 p.16]

나눗셈기 개선판

- 개선된 곱셈기와 **같은 하드웨어**: 제수 32비트, ALU 32비트, 나머지 레지스터 64비트(피제수로 초기화).
- 몫 레지스터가 따로 없다. 몫은 나머지 레지스터의 **오른쪽 절반**에 쌓인다.
- 32비트 제수를 나머지 레지스터의 **왼쪽 절반**에서 뺀다.
- 나머지 레지스터를 왼쪽으로도, 오른쪽으로도 시프트한다(반복 중엔 왼쪽, 마지막에 왼쪽 절반만 오른쪽).

[Ch3 p.17]

1. 나머지 레지스터의 왼쪽 4비트(왼쪽 절반)에서 제수를 빼고 부호 비트를 검사한다.
2. 부호 비트가 1이면 복원한 뒤 나머지 레지스터를 왼쪽 시프트(맨 오른쪽 0). 0이면 왼쪽 시프트하고 맨 오른쪽을 1로.
3. 5번 반복한 뒤, 나머지 레지스터의 왼쪽 절반을 오른쪽으로 1비트 시프트한다.

[Ch3 p.18]

개선판 반복 표: 0000 0111 ÷ 0010_2

반복	단계	제수(2진)	나머지 레지스터(2진)
0	초기값	0010	0000 0111
1	1: 나머지 = 나머지 − 제수(왼쪽 절반)	0010	**1110** 0111
	2: 나머지 < 0 → +제수, sll, Q0 = 0	0010	0000 1110
2	1: 나머지 = 나머지 − 제수	0010	**1110** 1110
	2: 나머지 < 0 → +제수, sll, Q0 = 0	0010	0001 1100
3	1: 나머지 = 나머지 − 제수	0010	**1111** 1100
	2: 나머지 < 0 → +제수, sll, Q0 = 0	0010	0011 1000
4	1: 나머지 = 나머지 − 제수	0010	**0001** 1000
	2: 나머지 ≥ 0 → sll, Q0 = 1	0010	0011 0001
5	1: 나머지 = 나머지 − 제수	0010	**0001** 0001
	2: 나머지 ≥ 0 → sll, Q0 = 1	0010	0010 0011
	왼쪽 절반 오른쪽 시프트	0010	**0001 0011**

최종: 왼쪽 절반 0001_2 = 나머지 1, 오른쪽 절반 0011_2 = 몫 3. 슬라이드 p.18 표와 같다. 반복 1을 풀어 보면: 왼쪽 절반 $0000_2 - 0010_2 = 1110_2$(음수) → 복원해 0000 0111 → 왼쪽 시프트, 맨 오른쪽 0 → 0000 1110. [Ch3 p.18]

마지막 오른쪽 시프트가 필요한 이유: 시프트가 5번 일어나 나머지가 몫보다 한 번 더 왼쪽으로 밀렸기 때문이다. 왼쪽 절반만 한 칸 되돌린다.

핵심

피제수 = 몫 × 제수 + 나머지, 제수 ≠ 0. 1판: 제수 64비트(왼쪽 절반에서 시작, 오른쪽 시프트), ALU 64, 나머지 64(피제수로 초기화), 몫 32. 빼기 → 부호 검사 → 음수면 복원 + 몫 0, 아니면 몫 1 → 제수 오른쪽 시프트, n+1회(32비트 33회). 개선판: 개선 곱셈기와 같은 HW, 몫은 나머지 레지스터 오른쪽 절반, 끝에 왼쪽 절반을 오른쪽 시프트. 7 ÷ 2 → 몫 0011_2, 나머지 0001_2.

예제

개선판 나눗셈기로 0000 1011 ÷ 0011_2(11 ÷ 3)을 계산하라.

풀이 보기

초기: 0000 1011.

반복 1: $0000_2 - 0011_2 = 1101_2$(음수) → 복원 0000 1011 → sll → 0001 0110.

반복 2: $0001_2 - 0011_2 = 1110_2$(음수) → 복원 → sll → 0010 1100.

반복 3: $0010_2 - 0011_2 = 1111_2$(음수) → 복원 → sll → 0101 1000.

반복 4: $0101_2 - 0011_2 = 0010_2$(≥ 0) → 0010 1000 → sll, Q0 = 1 → 0101 0001.

반복 5: $0101_2 - 0011_2 = 0010_2$(≥ 0) → 0010 0001 → sll, Q0 = 1 → 0100 0011.

왼쪽 절반 오른쪽 시프트 → **0010 0011**: 나머지 0010_2(2), 몫 0011_2(3). 11 = 3 × 3 + 2.

함정

4비트 나눗셈은 4번이 아니라 **5번**(n+1번) 반복한다. 32비트는 33번이다. 곱셈은 n번(32번)인 것과 다르다.

나머지가 음수가 되면 몫 비트를 0으로 하는 것만으로 끝나지 않는다. 반드시 제수를 다시 더해 **복원**해야 다음 단계 값이 맞다.

시험 포인트

- 피제수 = 몫 × 제수 + 나머지 공식, 제수 ≠ 0.
- 1판 부품의 비트 폭과 초기화 위치(제수는 왼쪽 절반, 나머지는 피제수).
- Subtract-Test-Shift, 복원, 반복 횟수 n+1.
- 0000 0111 ÷ 0010_2 반복 표(1판·개선판) 빈칸 채우기.

스스로 확인

1. 나눗셈기 1판에서 32비트 제수는 64비트 제수 레지스터의 어디에 놓이고, 어느 방향으로 시프트되는가?

   답 보기

   왼쪽 절반에 놓이고, 오른쪽으로 시프트된다.

2. 뺄셈 결과 나머지가 음수일 때 1판은 무엇을 하는가?

답 보기

제수를 다시 더해 나머지를 복원하고, 몫을 왼쪽 시프트하며 새 비트를 0으로 한다.

3. 32비트 나눗셈에서 1판 알고리즘은 몇 번 반복하는가?

답 보기

33번(n+1번).

4. 1판 0000 0111 ÷ 0010_2에서 반복 4의 뺄셈 직후 나머지 값과 그 뒤 몫은?

답 보기

나머지 0000 0011(≥ 0), 몫 0001_2.

5. 개선판 나눗셈기에서 몫은 어디에 쌓이는가?

답 보기

나머지 레지스터의 오른쪽 절반.

6. 개선판 0000 0111 ÷ 0010_2에서 반복 5가 끝난 뒤 값과 최종 값은?

답 보기

0010 0011 → 왼쪽 절반을 오른쪽 시프트해 0001 0011(나머지 1, 몫 3).

Ch3 · 5 슬라이드 p.19–20

부호 있는 나눗셈과 빠른 나눗셈

이 단원을 마치면 음수가 섞인 나눗셈의 몫과 나머지 부호를 정하고, RISC-V 나눗셈 명령과 시프트 나눗셈의 주의점을 말할 수 있다.

쉽게 말하면

−7 ÷ 2의 답은 "몫 −3, 나머지 −1"로도, "몫 −4, 나머지 1"로도 공식 피제수 = 몫 × 제수 + 나머지를 만족한다. 답이 두 개면 곤란하니 규칙 하나를 정한다. **나머지의 부호는 피제수 부호를 따른다.**

비유가 안 맞는 지점 수학 시간에 배운 "나머지는 항상 0 이상"과 다르다. 컴퓨터(그리고 C의 %)는 피제수가 음수면 나머지도 음수가 될 수 있다.

부호 있는 나눗셈 규칙

가장 단순한 방법은 제수와 피제수의 부호를 기억해 두고, 두 부호가 다르면 몫을 부정하는 것이다. 까다로운 점은 나머지의 부호다. [Ch3 p.19]

−7 ÷ 2에서 두 후보를 보자.

- 몫 −3, 나머지 −1: −3 × 2 + (−1) = −7. 맞음. **이것을 쓴다.**
- 몫 −4, 나머지 1: −4 × 2 + 1 = −7. 식은 맞지만 **이것은 쓰지 않는다**(We do NOT use this).

규칙: 제수와 몫의 부호에 상관없이, **피제수와 나머지는 같은 부호**여야 한다. C의 나머지 연산(%)도 이 규칙을 따른다. [Ch3 p.19]

식	몫	나머지	검산 (몫 × 제수 + 나머지)
7 ÷ 2	3	1	3 × 2 + 1 = 7
−7 ÷ 2	−3	−1	−3 × 2 − 1 = −7
7 ÷ −2	−3	1	−3 × (−2) + 1 = 7
−7 ÷ −2	3	−1	3 × (−2) − 1 = −7

몫의 크기는 늘 3, 나머지의 크기는 늘 1이다. 몫의 부호는 두 부호가 다를 때 −, 나머지 부호는 피제수 부호. [Ch3 p.19]

왜 이 규칙인가: 이렇게 하면 부호와 상관없이 몫과 나머지의 **크기(절댓값)가 같다**. 몫 −4·나머지 1을 택하면 −7 ÷ 2의 몫 크기(4)가 7 ÷ 2의 몫 크기(3)와 달라져, −(x ÷ y) ≠ (−x) ÷ y가 되어 버린다. PDF 외 보충

부호 있는 나눗셈 절차(다시 정리)

1. 피연산자 부호를 무시하고 절댓값으로 몫과 나머지를 구한다.
2. 두 부호가 서로 다르면 몫을 부정한다.
3. 나머지의 부호를 피제수 부호에 맞춘다.

[Ch3 p.19]

빠른 나눗셈과 SRT

곱셈처럼 덧셈기를 여러 개 병렬로 쓰는 방법은 나눗셈에 쓸 수 없다. 나눗셈에서 다음 뺄셈을 할지 말지는 **앞 단계 나머지의 부호**에 달려 있어서(조건부), 단계를 동시에 진행할 수 없기 때문이다. [Ch3 p.20]

SRT 나눗셈은 한 단계에 몫 비트를 여러 개 **예측**한다. 피제수와 나머지의 위쪽 비트를 보고 표(table lookup)에서 몫 비트를 찾는다. [Ch3 p.20]

RISC-V 나눗셈 명령

명령	뜻	결과
`div`	divide	부호 있는 몫
`divu`	divide unsigned	무부호 몫
`rem`	remainder	부호 있는 나머지
`remu`	remainder unsigned	무부호 나머지

나눗셈 2개, 나머지 2개로 부호 있음·무부호를 모두 다룬다. [Ch3 p.20]

시프트로 곱하기·나누기

- 2의 거듭제곱을 곱할 때는 왼쪽 시프트: $x \times 2^k = x$를 왼쪽으로 k비트(`slli`).
- 2의 거듭제곱으로 나눌 때는 오른쪽 시프트. 음수는 **부호 확장**(빈자리를 부호 비트로 채움, 산술 시프트 `srai`)을 해야 한다.

[Ch3 p.20] 예: 5(0101_2)를 왼쪽으로 1비트 → 1010_2(10). −8(8비트 1111 1000)을 산술 오른쪽 1비트 → 1111 1100(−4). 0으로 채우면 0111 1100(124)이 되어 틀린다.

핵심

나머지 부호 = 피제수 부호. 몫은 부호가 다르면 음수. −7÷2 → −3, −1 / 7÷−2 → −3, 1 / −7÷−2 → 3, −1. 나눗셈은 뺄셈이 나머지 부호에 달려 있어 병렬화 불가 → SRT(표로 몫 여러 비트 예측). div·divu·rem·remu. 2^k 곱 = 왼쪽 시프트, 2^k 나눗셈 = 오른쪽 시프트(음수는 부호 확장).

예제

−13 ÷ 4와 13 ÷ −4의 몫과 나머지를 슬라이드 규칙으로 구하라.

풀이 보기

절댓값: 13 ÷ 4 = 몫 3, 나머지 1.

−13 ÷ 4: 부호가 다르므로 몫 −3, 나머지는 피제수(−) 부호 → −1. 검산 −3 × 4 − 1 = −13.

13 ÷ −4: 부호가 다르므로 몫 −3, 나머지는 피제수(+) 부호 → 1. 검산 −3 × (−4) + 1 = 13.

함정

−7 ÷ 2의 몫을 −4, 나머지를 1로 쓰면 틀린다. 식은 성립하지만 슬라이드가 명시적으로 쓰지 않는다고 한 답이다.

음수를 산술 오른쪽 시프트한 결과는 나눗셈 몫과 다를 수 있다. −7(1111 1001)을 오른쪽으로 1비트 산술 시프트하면 1111 1100 = **−4**인데, 위 규칙의 −7 ÷ 2 몫은 **−3**이다. 시프트는 −∞ 쪽으로 내림하기 때문이다. 나누어떨어지는 경우(−8 → −4)에는 같다.

PDF 외 보충

시험 포인트

- 나머지 부호 규칙과 4가지 부호 조합의 몫·나머지.
- 부호 있는 나눗셈 3단계.
- 나눗셈을 곱셈처럼 병렬화할 수 없는 이유, SRT의 아이디어.
- div, divu, rem, remu 이름과 뜻.

스스로 확인

1. −7 ÷ −2의 몫과 나머지는?

   답 보기

   몫 3, 나머지 −1. 나머지는 피제수(−7) 부호를 따른다.

2. 부호 있는 나눗셈에서 나머지의 부호를 정하는 규칙은?

   답 보기

   나머지는 피제수와 같은 부호(제수·몫 부호와 무관).

3. 나눗셈에 곱셈기처럼 병렬 하드웨어를 쓸 수 없는 이유는?

   답 보기

   각 단계의 뺄셈 여부가 앞 단계 나머지의 부호에 따라 정해지는 조건부 연산이기 때문.

4. 표를 보고 한 단계에 몫 비트 여러 개를 예측하는 나눗셈 기법 이름은?

   답 보기

   SRT 나눗셈.

5. 무부호 나머지를 구하는 RISC-V 명령은?

답 보기

`remu`.

6. 음수를 2의 거듭제곱으로 나누려고 오른쪽 시프트할 때 주의할 점은?

답 보기

빈자리를 부호 비트로 채우는 부호 확장(산술 시프트)을 해야 한다.

Ch3 · 6 슬라이드 p.21–26

실수와 부동소수점

이 단원을 마치면 정규화된 과학적 표기를 판별하고, 부동소수점의 부호·지수·가수와 정밀도·범위의 관계, FP 오버플로·언더플로를 설명할 수 있다.

쉽게 말하면

아주 큰 수(한 세기의 초 수)와 아주 작은 수(1나노초)를 같은 칸 수로 적으려면 "유효숫자 몇 개 × 10의 몇 제곱"으로 쓰면 된다. 3,155,760,000 대신 3.15576×10^9. 소수점이 고정된 자리에 있지 않고 지수에 따라 떠다닌다고 해서 **부동소수점(floating-point)**이다.

비유가 안 맞는 지점 컴퓨터는 10이 아니라 2의 거듭제곱을 쓰고, 칸 수가 정해져 있어서 모든 실수를 정확히 담지 못한다. 대부분의 실수는 가장 가까운 근삿값으로 저장된다.

부동소수점 하드웨어의 흐름

예전에는 부동소수점 연산 장치(FPU, Floating-Point Unit)가 주 프로세서와 분리된 **보조 프로세서(co-processor)**였다. 예: Intel 80386(주 프로세서)과 80387(부동소수점 보조 프로세서). 요즘은 부동소수점 연산을 빠르게 하는 것이 딥러닝 응용 성공의 열쇠다(예: NVIDIA P100, V100). [Ch3 p.21]

GPU에는 해마다 더 많은 부동소수점 코어가 들어간다. 부동소수점 장치는 정수 장치보다 복잡하다. [Ch3 p.22]

실수와 과학적 표기

프로그래밍 언어는 소수부가 있는 수를 지원하고, 수학에서는 이를 **실수(real)**라 한다. 예: 3.14159265…(π), 2.71828…(e), 0.0000000001 = 1.0×10^{-9}(1나노초의 초 수), 3,155,760,000 = 3.15576×10^9(한 세기의 초 수). [Ch3 p.23]

과학적 표기(scientific notation) 소수점 왼쪽에 숫자 한 자리만 두고 나머지는 지수로 나타내는 표기.

정규화(normalized) 수 과학적 표기에서 앞에 0이 없는 수. 보통 이렇게 쓴다. [Ch3 p.23]

이진 소수점(binary point) 2진수에서 소수점을 부르는 새 이름. 예: $1.0_2 \times 2^{-1}$. [Ch3 p.23]

표기	정규화?	이유
1.0×10^{-9}	O	소수점 왼쪽이 0이 아닌 한 자리
10.0×10^{-10}	X	소수점 왼쪽이 두 자리
0.1×10^{-8}	X	앞자리가 0(leading 0)

셋은 같은 값이지만 정규화된 것은 1.0×10^{-9} 하나다. [Ch3 p.23]

부동소수점 표기

부동소수점은 이진 소수점의 위치가 고정되지 않은 수 표현이다. C 언어는 이런 수를 `float`라 부른다. 2진수로는 다음 꼴이다. [Ch3 p.24]

$$1.\text{xxxxxxxx}_2 \times 2^{\text{yyyy}}$$

x 부분을 **가수(fraction, mantissa라고도 함)**, y 부분을 **지수(exponent)**라 한다. [Ch3 p.24]

정규화된 표준 표기를 쓰면 좋은 점 세 가지: [Ch3 p.24]

1. 부동소수점 수가 들어 있는 데이터를 주고받기(교환)가 단순해진다.
2. 수가 항상 이 형태라는 것을 알므로 부동소수점 알고리즘이 단순해진다.
3. 한 word에 저장되는 수의 정확도가 높아진다. 쓸모없는 앞자리 0 대신 소수점 오른쪽의 진짜 숫자를 담기 때문이다.

정밀도와 범위의 맞바꿈

부동소수점 수는 한 word(32비트)에 들어가야 하므로 가수 칸과 지수 칸의 크기를 타협해야 한다. 이것이 **정밀도(precision)와 범위(range)의 맞바꿈**이다. [Ch3 p.25]

- 가수 칸을 늘리면 → **정밀도**가 좋아진다(유효숫자가 많아짐).
- 지수 칸을 늘리면 → 표현할 수 있는 수의 **범위**가 넓어진다.

RISC-V 부동소수점 수의 구성: [Ch3 p.25]



- s: 부호 1비트. **부호-크기(sign and magnitude) 표현**(부호는 따로 1비트, 나머지는 크기)이라 0이면 양수, 1이면 음수.
- 지수 8비트(지수의 부호 포함), 가수 23비트.

$$(-1)^S \times F \times 2^E$$

일반적인 부동소수점 수는 이 꼴이다. [Ch3 p.25]

> **주의 (슬라이드 p.25)**
>
> p.25는 지수 칸을 "2의 보수(2's complement)"라고 적었다. 이것은 일반형을 설명한 것이고, 실제 IEEE 754 표준은 지수를 **bias(바이어스) 방식**으로 저장한다(다음 단원, p.28). 시험에서 IEEE 754의 지수 표현을 물으면 bias로 답한다.

부동소수점 오버플로·언더플로

RISC-V 부동소수점은 범위가 매우 넓다. 대략 $\pm 2.0 \times 10^{-38}$ ~ $\pm 2.0 \times 10^{38}$. 그래도 무한하지 않다. 그래서 정수처럼 부동소수점에서도 오버플로가 생기고, 0이 아닌 가수가 너무 작아서 표현할 수 없는 새 예외 상황인 **언더플로**도 생긴다. [Ch3 p.26]

FP 오버플로(overflow) (양의) 지수가 너무 커서 지수 칸에 담을 수 없는 상황.

FP 언더플로(underflow) 음의 지수가 너무 커서(절댓값이 너무 커서) 지수 칸에 담을 수 없는 상황. 즉 수가 0에 너무 가까움.

둘을 줄이려면 64비트 **배정도(double precision)** 형식(C의 `double`)을 쓴다. 범위는 대략 $\pm 2.0 \times 10^{-308}$ ~ $\pm 2.0 \times 10^{308}$. [Ch3 p.26]

핵심

정규화 = 소수점 왼쪽에 0이 아닌 한 자리. $1.xxx_2 \times 2^{yyy}$, x = 가수(fraction/mantissa), y = 지수. 장점 3: 데이터 교환·알고리즘 단순화, 정확도 증가. 가수↑ = 정밀도↑, 지수↑ = 범위↑. 부호 1 · 지수 8 · 가수 23. 오버플로 = 지수가 너무 큼, 언더플로 = 음의 지수가 너무 큼.

예제

(1) 0.00101_2를 정규화된 2진 과학적 표기로 쓰라. (2) 0.75를 2진 정규화 표기로 쓰라.

풀이 보기

(1) 이진 소수점을 오른쪽으로 3칸 옮기면 1.01_2이므로 $\mathbf{1.01_2 \times 2^{-3}}$.

(2) $0.75 = 1/2 + 1/4 = 0.11_2 = \mathbf{1.1_2 \times 2^{-1}}$.

함정

언더플로는 "음수가 너무 작다(−∞ 쪽으로 크다)"는 뜻이 아니다. 수의 크기가 0에 너무 가까워 **음의 지수**를 담을 수 없는 경우다. 매우 큰 음수는 지수가 크므로 오버플로 쪽이다.

"가수 칸을 늘리면 범위가 넓어진다"는 틀렸다. 가수는 정밀도, 지수는 범위다.

시험 포인트

- 정규화 O/X 판별(1.0×10^{-9} O, 10.0×10^{-10}·0.1×10^{-8} X), binary point 용어.
- 정규화 표기의 장점 3가지.
- 정밀도 vs 범위: 어느 칸을 늘리면 무엇이 좋아지나.
- FP 오버플로·언더플로의 정의, 줄이는 방법(double).
- 예전 FPU는 보조 프로세서(80386/80387).

스스로 확인

1. O/X: 0.1×10^{-8}은 정규화된 수다.

   답 보기

   X. 앞자리가 0이다. 정규화하면 1.0×10^{-9}.

2. 2진수에서 소수점을 부르는 이름은?

답 보기

이진 소수점(binary point).

3. 정규화된 표준 표기를 쓰는 장점 세 가지를 쓰라.

답 보기

① 데이터 교환 단순화 ② 부동소수점 알고리즘 단순화 ③ 한 word에 저장되는 수의 정확도 증가(앞자리 0 대신 유효숫자).

4. 지수 칸의 비트 수를 늘리면 무엇이 좋아지는가?

답 보기

표현 범위(range)가 넓어진다.

5. 부동소수점 언더플로의 정의는?

답 보기

음의 지수가 너무 커서 지수 칸에 담을 수 없는 것(0이 아닌 수가 너무 작아 표현 불가).

6. 예전에 Intel 80386과 함께 쓰던 부동소수점 보조 프로세서는?

답 보기

80387.

Ch3 · 7 슬라이드 p.27–31

IEEE 754 표준

이 단원을 마치면 십진수를 IEEE 754 단정도·배정도 비트로, 16진수 비트를 십진수로 양방향 변환하고, bias를 쓰는 이유를 설명할 수 있다.

쉽게 말하면

나라마다 콘센트 모양이 다르면 여행 때 어댑터가 필요하다. 1980년대 초에는 컴퓨터마다 부동소수점 저장 방식이 달라서 같은 프로그램이 기계마다 다른 답을 냈다. IEEE 754는 "모든 기계가 이 모양으로 저장하자"는 공통 규격이다.

비유가 안 맞는 지점 규격은 단순히 모양만 맞춘 것이 아니다. 숨은 1, bias 같은 장치로 같은 비트 수에 더 많은 정밀도를 담고, 정수 비교 회로로 크기 비교까지 되게 설계했다.

배경

부동소수점 표준은 **IEEE Standard 754-1985**로 정해졌다. 표현 방식이 제각각이라 과학 계산 코드의 이식성(portability, 다른 기계로 옮겨도 같은 결과)에 문제가 생겨서 만든 것이다. IEEE 754는 부동소수점 프로그램을 옮기기 쉽게 했고 연산 품질도 크게 높였다. 주요 CPU·GPU(Intel, AMD, NVIDIA 등)가 모두 지원한다. 주 설계자 **William Kahan**은 튜링상(Turing Award)을 받았다(슬라이드의 "Turning"은 오타). 교재 저자들도 2017년 튜링상을 받았다. [Ch3 p.27]

IEEE 754는 두 가지 표현을 둔다: **32비트 단정도(single precision)**와 **64비트 배정도(double precision)**. [Ch3 p.27]

비트 구성





$$값 = (-1)^{S} \times (1 + Fraction) \times 2^{(Exponent - Bias)}$$

[Ch3 p.28]

	부호	지수	가수(Fraction)	Bias	실제 지수 범위
단정도(32)	1	8	23 (+숨은 1 = 24비트 정밀도)	127	−126 ~ 127 (지수 칸 1 ~ 254)
배정도(64)	1	11	52	1023	−1022 ~ 1023 (지수 칸 1 ~ 2046)

배정도의 실제 지수 범위는 단정도와 같은 규칙으로 계산한 값이다. PDF 외 보충

숨은 1(hidden bit)

S는 부호 비트(0: 음이 아닌 수, 1: 음수). 정규화된 2진수는 이진 소수점 앞이 **항상 1**이다(2진수에서 0이 아닌 한 자리는 1뿐). 그래서 이 1은 저장하지 않고 **숨은 비트(hidden bit)**로 둔다. [Ch3 p.28]

유효숫자(significand) 1 + Fraction. 저장된 F 앞에 "1."을 붙인 값. 예: F = 1010 → 유효숫자 1.1010_2. 1.0 ≤ 유효숫자 < 2.0.

- 1비트를 아꼈으니 유효숫자에 비트를 하나 더 담는다. 단정도 유효숫자는 숨은 1 + 23비트 = **24비트**.
- 최소: F = 00…0 → 유효숫자 1.0. 최대: F = 11…1 → 유효숫자 $1.11\ldots1_2$ (2.0보다 아주 조금 작음, "2.0−").

[Ch3 p.28]

bias(바이어스) 지수

```
지수 칸 값(biased exponent) = 실제 지수 + Bias
```

Bias는 단정도 127, 배정도 1023이다. 이렇게 하면 지수 칸이 항상 무부호가 된다. 단정도에서 실제 지수 −126 ~ 127은 지수 칸 1 ~ 254로 저장된다. [Ch3 p.28]

bias를 쓰는 이유: IEEE 754 설계자는 부동소수점 수를 **정수 비교**만으로 처리(예: 정렬)할 수 있기를 원했다. [Ch3 p.29]

1. 부호가 MSB에 있어 부호 비교를 빨리 할 수 있다.
2. 지수를 가수보다 앞(왼쪽)에 두어 정렬이 단순해진다. 지수가 크면 비트 패턴도 크다.
3. 그런데 음수 지수를 2의 보수로 쓰면 비트 패턴이 커 보여 정렬이 꼬인다 → bias 지수로 해결한다.

수	지수를 2의 보수로 (비트 31–23)	bias 지수로 (비트 31–23)
$X = 1.0_2 \times 2^{-1}$	0 1111 1111 (−1)	0 0111 1110 (126)
$Y = 1.0_2 \times 2^{1}$	0 0000 0001 (1)	0 1000 0000 (128)
정수로 비교하면	X가 커 보임 (틀림)	Y가 커 보임 (맞음)

가수 칸은 둘 다 0이다. 실제로는 Y(2)가 X(0.5)보다 크므로 bias 방식이 맞다. [Ch3 p.29]

예약된 지수와 범위

지수 칸 00000000과 11111111(단정도 0과 255)은 **예약**되어 있다. 특이한 상황을 나타내는 특수 기호에 쓴다. 예: **NaN**(Not a Number, 수가 아님)은 0/0 같은 잘못된 연산의 결과. [Ch3 p.30]

단정도 지수 칸	단정도 Fraction	배정도 지수 칸	배정도 Fraction	뜻(Object represented)
0	0	0	0	0
0	0 아님(Nonzero)	0	0 아님	± 비정규화 수(denormalized number)
1 ~ 254	아무 값(Anything)	1 ~ 2046	아무 값	± 부동소수점 수(floating-point number)
255	0	2047	0	± 무한대(infinity)
255	0 아님	2047	0 아님	NaN(Not a Number)

[Ch3 p.30]

단정도 정규화 범위: [Ch3 p.30]

- 가장 작은 크기: 지수 칸 00000001(실제 지수 1 − 127 = −126), F = 000…0(유효숫자 1.0) → $\mathbf{\pm 1.0_2 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}}$.
- 가장 큰 크기: 지수 칸 11111110(실제 지수 254 − 127 = 127), F = 111…1(유효숫자 2.0−) → $\mathbf{\pm (2.0-) \times 2^{127} \approx \pm 3.4 \times 10^{38}}$.

p.26의 $\pm 2.0 \times 10^{\pm 38}$은 대략값이고, 정확한 값은 위와 같다. 배정도도 같은 규칙으로 $2^{-1022} \approx 2.2 \times 10^{-308}$에서 약 1.8×10^{308}까지다. PDF 외 보충

십진수 → IEEE 754: −0.75

1. 2진수로: $-0.75 = -3/4 = -3/2^2 = -11_2/2^2 = -0.11_2$.
2. 정규화: $-0.11_2 \times 2^0 = \mathbf{-1.1_2 \times 2^{-1}}$.
3. 부호: 음수 → S = 1.
4. Fraction: 유효숫자 1.1_2에서 앞의 1을 빼면 $.1_2$ → **1000 0000 0000 0000 0000 000**(23비트).
5. 지수 칸: 실제 지수 −1 + 127 = 126 = **0111 1110**. (배정도면 −1 + 1023 = 1022 = 011 1111 1110)
6. 이어 붙여 4비트씩 끊기: 1 01111110 1000…0 → 1011 1111 0100 0000 0000 0000 0000 0000 → **0xBF400000**.

1	01111110	10000000000000000000000
S	Exponent = 126	Fraction

검산: $(-1)^1 \times (1 + .1_2) \times 2^{(126 - 127)} = -1.5 \times 0.5 = -0.75$. [Ch3 p.31]

배정도로는 1 01111111110 1000…0 → 1011 1111 1110 1000 0…0 → 0xBFE8000000000000. PDF 외 보충

IEEE 754 → 십진수

거꾸로 갈 때는 ① 16진수를 2진수로 ② S(1비트), 지수(8비트), F(23비트)로 자르기 ③ 실제 지수 = 지수 칸 − 127 ④ $(-1)^S \times (1.F)_2 \times 2^{\text{실제 지수}}$ 계산. PDF 외 보충

예: 0xC0A00000 → 1100 0000 1010 0000 0000 … → S = 1, 지수 칸 1000 0001 = 129 → 실제 지수 2, F = 0100…0 → 유효숫자 $1.01_2 = 1.25$ → $-1.25 \times 2^2 = \mathbf{-5.0}$.

핵심

단정도 1/8/23, bias 127. 배정도 1/11/52, bias 1023. 값 = $(-1)^S \times (1 + F) \times 2^{(E - \text{Bias})}$, 숨은 1로 24비트 정밀도. 지수 칸 0·255는 예약(NaN = 0/0 등). bias를 쓰는 이유 = 정수 비교로 정렬. −0.75 = 0xBF400000, 0.0625 = 0x3D800000.

예제

(1) 0.0625를 단정도 16진수로. (2) 5.5를 단정도 16진수로. (3) 0x41200000을 십진수로.

풀이 보기

(1) $0.0625 = 1/16 = 1.0_2 \times 2^{-4}$. S = 0, 지수 칸 = −4 + 127 = 123 = 0111 1011, F = 0…0 → 0 01111011 000…0 → 0011 1101 1000 0000 … → **0x3D800000**.

(2) $5.5 = 101.1_2 = 1.011_2 \times 2^2$. S = 0, 지수 칸 = 2 + 127 = 129 = 1000 0001, F = 0110…0 → 0 10000001 0110…0 → 0100 0000 1011 0000 … → **0x40B00000**.

(3) 0100 0001 0010 0000 … → S = 0, 지수 칸 1000 0010 = 130 → 실제 지수 3, F = 010…0 → $1.01_2 = 1.25$ → 1.25×2^3 = **10.0**.

함정

Fraction 칸에 숨은 1을 넣으면 틀린다. −0.75의 Fraction은 1100…이 아니라 **1000…**이다(1.1_2에서 앞의 1을 뺌).

지수 칸에 실제 지수(−1)를 2의 보수로 넣으면(1111 1111) 틀린다. 반드시 bias를 더한 **126 = 0111 1110**을 넣는다.

4비트씩 끊을 때 부호 1비트 + 지수 8비트가 4의 배수로 안 떨어진다. 32비트를 다 이어 붙인 뒤 끊어야 한다.

시험 포인트

- 단정도·배정도 비트 폭과 bias(127/1023), 단정도 지수 칸 값 범위 1 ~ 254.
- 숨은 1의 의미와 24비트 정밀도.
- bias를 쓰는 이유 3가지(p.29 X·Y 예).
- 예약 지수(0, 255)와 NaN 예(0/0), 정규화 범위 ≈ $\pm 1.2 \times 10^{-38}$ ~ $\pm 3.4 \times 10^{38}$.
- −0.75 ↔ 0xBF400000 양방향 변환 (거의 그대로 출제 가능).
- IEEE 754-1985, Kahan(튜링상).

스스로 확인

1. IEEE 754 단정도의 부호·지수·가수 비트 수와 bias는?

   답 보기

   1 · 8 · 23, bias 127.

2. 단정도 유효숫자가 24비트 정밀도를 갖는 이유는?

답 보기

정규화된 수의 맨 앞 1을 저장하지 않는 숨은 비트로 두어 23비트 Fraction + 1비트가 되기 때문.

3. −0.75를 단정도로 저장할 때 지수 칸의 2진수는?

답 보기

0111 1110 (−1 + 127 = 126).

4. IEEE 754가 지수를 2의 보수 대신 bias로 저장하는 이유는?

답 보기

음수 지수가 2의 보수로는 큰 비트 패턴이 되어 정수 비교(정렬)가 틀어지므로, bias로 지수를 무부호로 만들어 정수 비교만으로 크기 비교·정렬이 되게 하려고.

5. 0xBF400000을 십진수로 바꾸라.

답 보기

S 1, 지수 칸 126 → 실제 지수 −1, F = 1000…0 → $-1.1_2 \times 2^{-1} = -0.75$.

6. 단정도에서 지수 칸 값 0과 255는 어떻게 쓰이는가?

답 보기

예약값. 0, ∞, NaN(0/0 같은 잘못된 연산) 등 특수한 경우를 나타낸다.

7. 1.0을 단정도 16진수로 쓰라.

답 보기

$1.0_2 \times 2^0$ → S 0, 지수 칸 0 + 127 = 127 = 0111 1111, F 0 → 0x3F800000.

8. IEEE 754의 주 설계자는 누구이고 어떤 상을 받았는가?

답 보기

William Kahan, 튜링상.

Ch3 · 8 슬라이드 p.32–36

부동소수점 덧셈과 곱셈

이 단원을 마치면 부동소수점 덧셈 4단계·곱셈 5단계를 순서대로 말하고, 0.5 + (−0.4375)와 0.5 × (−0.4375)를 2진수로 계산할 수 있다.

쉽게 말하면

3.2만 원과 450원을 더하려면 먼저 단위를 맞춰야 한다(3.2만 + 0.045만). 부동소수점 덧셈도 지수(단위)를 먼저 맞춘다. 곱셈은 단위를 맞출 필요 없이 숫자끼리 곱하고 지수끼리 더하면 된다($10^3 \times 10^2 = 10^5$).

비유가 안 맞는 지점 저장할 수 있는 자릿수가 정해져 있어서 맞추는 과정에서 작은 수의 끝자리가 잘려 나가고, 결과를 반올림해야 한다. 반올림 후 정규화가 깨지면 다시 정규화한다.

십진 덧셈 4단계: $9.999 \times 10^1 + 1.610 \times 10^{-1}$

가정: 지수는 2자리, 유효숫자는 4자리까지만 저장. [Ch3 p.32]

1. **소수점 맞추기**: 지수가 작은 쪽을 큰 쪽에 맞춘다. $1.610 \times 10^{-1} = 0.01610 \times 10^1$ → 4자리만 남겨 0.016×10^1.
2. **유효숫자 더하기**: $9.999 + 0.016 = 10.015$ → 10.015×10^1.
3. **정규화하고 오버플로·언더플로 검사**: 1.0015×10^2. 지수 2는 2자리 안에 들어감.
4. **반올림**: 유효숫자 4자리로 1.002×10^2.

반올림한 뒤 정규화가 깨지면 3단계를 다시 한다. 예: 9.9999×10^2를 반올림하면 10.000×10^2 → 다시 정규화해 1.000×10^3. [Ch3 p.32]

이진 덧셈: 0.5 + (−0.4375)

유효숫자 4비트로 계산한다. 먼저 2진수로 바꾼다. [Ch3 p.33]

- $0.5 = 1 \times 1/2 = 1 \times 2^{-1} = \mathbf{1.000_2 \times 2^{-1}}$
- $-0.4375 = -7 \times 1/16 = -111_2 \times 2^{-4} = \mathbf{-1.110_2 \times 2^{-2}}$

1. 작은 수를 오른쪽 시프트해 맞춤: $-1.110_2 \times 2^{-2} = -0.111_2 \times 2^{-1}$ ($1.110_2 >> 1 = 0.111_2$).
2. 유효숫자 더하기: $1.000_2 + (-0.111_2) = 0.001_2$ → $0.001_2 \times 2^{-1}$.
3. 정규화·검사: $0.001_2 \times 2^{-1} = 1.000_2 \times 2^{-4}$. $-126 \le -4 \le 127$이므로 오버플로·언더플로 없음.
4. 반올림: $1.000_2 \times 2^{-4}$, 반올림 필요 없음.

결과 $1.000_2 \times 2^{-4} = 1/16 = \mathbf{0.0625}$. 검산 0.5 − 0.4375 = 0.0625. 단정도 비트로는 0x3D800000(앞 단원). [Ch3 p.33]

부동소수점 덧셈기 하드웨어

부동소수점 덧셈기는 정수 덧셈기보다 훨씬 복잡하다. 위 예가 하드웨어를 지나가는 순서는 다음과 같다. [Ch3 p.34]

1. **지수 비교**: 작은 ALU가 두 지수 칸을 뺀다. 0.5는 지수 칸 126, −0.4375는 125 → 차이 1.

2. **정렬**: 차이만큼 지수가 작은 쪽의 유효숫자(1 + fraction)를 오른쪽 시프트한다. $1.110\ldots_2 \rightarrow 0.111\ldots_2$. 큰 쪽 지수 칸 126을 결과 지수 칸 후보로 고른다.
3. **더하기**: 큰 ALU가 유효숫자를 더한다. $1.000\ldots_2 + (-0.111\ldots_2) = 0.001\ldots_2$.
4. **정규화**: 결과를 왼쪽으로 3비트 시프트(<<3)해 $1.000\ldots_2$로 만들고, 지수 칸을 3 줄여 126 − 3 = 123.
5. **반올림**: 필요하면 반올림하고, 정규화가 깨지면 다시 정규화 단계로.

결과: 부호 0, 지수 칸 123, fraction 000…0 → $1.0_2 \times 2^{-4} = 0.0625$. [Ch3 p.34]

십진 곱셈 5단계: $1.110 \times 10^{10} \times 9.200 \times 10^{-5}$

같은 가정(지수 2자리, 유효숫자 4자리). [Ch3 p.35]

1. **지수 더하기**: 10 + (−5) = 5.
2. **유효숫자 곱하기**: 1.110 × 9.200 = 10.212000 → 10.212×10^5.
3. **정규화·검사**: 1.0212×10^6. 지수 6은 2자리 안.
4. **반올림**: 1.021×10^6.
5. **부호 결정**: 부호-크기 표현이라 부호를 따로 정한다. 두 수 모두 양수 → $+1.021 \times 10^6$.

이진 곱셈: 0.5 × (−0.4375)

$1.000_2 \times 2^{-1} \times (-1.110_2 \times 2^{-2})$. [Ch3 p.36]

1. 지수 더하기: −1 + (−2) = −3.
   - 지수 칸 값(bias 지수)끼리 그냥 더하면: (−1 + 127) + (−2 + 127) = −3 + 254 → bias가 두 번 들어가 **틀림**.
   - 그래서 bias를 한 번 뺀다: (−1 + 127) + (−2 + 127) − 127 = −3 + 127 = 124 → **맞음**.
2. 유효숫자 곱하기: $1.000_2 \times 1.110_2 = 1.110000_2$ → 4비트로 1.110_2.
3. 정규화·검사: $1.110_2 \times 2^{-3}$, 이미 정규화됨. −126 ≤ −3 ≤ 127이므로 오버플로·언더플로 없음.
4. 반올림: 필요 없음.
5. 부호 결정: 양수 × 음수 → 음수. $\mathbf{-1.110_2 \times 2^{-3}}$.

값: $-1.110_2 \times 2^{-3} = -0.001110_2 = -7/2^5 = -7/32 =$ **−0.21875**. 검산 0.5 × 0.4375 = 0.21875. [Ch3 p.36]

> **핵심**
>
> 덧셈 4단계: ① 작은 지수 쪽 유효숫자를 오른쪽 시프트해 정렬 ② 유효숫자 더하기 ③ 정규화·오버/언더플로 검사 ④ 반올림(깨지면 ③ 다시). 곱셈 5단계: ① 지수 더하기(bias 한 번 빼기) ② 유효숫자 곱 ③ 정규화·검사 ④ 반올림 ⑤ 부호 결정. $0.5 + (-0.4375) = 1.000_2 \times 2^{-4} = 0.0625$, $0.5 \times (-0.4375) = -1.110_2 \times 2^{-3} = -0.21875$.

예제

유효숫자 4비트로 (1) 0.75 + 0.375 (2) 0.75 × 0.375를 2진 부동소수점 단계대로 계산하라.

풀이 보기

$0.75 = 1.100_2 \times 2^{-1}$, $0.375 = 3/8 = 1.100_2 \times 2^{-2}$.

(1) ① $1.100_2 \times 2^{-2} = 0.110_2 \times 2^{-1}$ ② $1.100_2 + 0.110_2 = 10.010_2 \rightarrow 10.010_2 \times 2^{-1}$ ③ $1.0010_2 \times 2^0$ ④ 4비트로 $1.001_2 \times 2^0$ = **1.125**. 검산 0.75 + 0.375 = 1.125.

(2) ① −1 + (−2) = −3 (지수 칸 값으로는 126 + 125 − 127 = 124) ② $1.100_2 \times 1.100_2 = 10.010000_2$ ③ $1.0010_2 \times 2^{-2}$ ④ $1.001_2 \times 2^{-2}$ ⑤ 둘 다 양수 → + → $1.001_2 \times 2^{-2} = 0.01001_2 = 9/32$ = **0.28125**. 검산 0.75 × 0.375 = 0.28125.

함정

덧셈 정렬은 **지수가 작은 쪽**을 **오른쪽**으로 시프트한다. 큰 쪽을 왼쪽으로 밀면 앞자리가 넘쳐 잃는다.

곱셈에서 지수 칸 값끼리 더하기만 하면 bias가 두 번 들어간다(−3 + 254). 반드시 127을 한 번 빼야 한다.

정규화를 한 번 했어도 반올림 후 다시 깨질 수 있다(9.9999 → 10.000). 이때 3단계로 돌아간다.

시험 포인트

- FP 덧셈 4단계, 곱셈 5단계의 순서와 이름.
- $9.999 \times 10^1 + 1.610 \times 10^{-1} = 1.002 \times 10^2$, $1.110 \times 10^{10} \times 9.200 \times 10^{-5} = +1.021 \times 10^6$.
- 0.5 + (−0.4375), 0.5 × (−0.4375)의 단계별 값.
- 곱셈에서 bias를 한 번 빼는 이유.
- 덧셈기 하드웨어: 지수 차이만큼 시프트, 정규화 때 지수 증감.

스스로 확인

1. 부동소수점 덧셈의 4단계를 순서대로 쓰라.

   답 보기

   ① 지수가 작은 쪽을 정렬 ② 유효숫자 더하기 ③ 정규화·오버/언더플로 검사 ④ 반올림.

2. 0.5 + (−0.4375)에서 정렬 후 −0.4375는 어떻게 표현되는가?

   답 보기

   $-0.111_2 \times 2^{-1}$ ($-1.110_2 \times 2^{-2}$를 오른쪽으로 1비트).

3. 단정도 곱셈에서 두 수의 지수 칸이 126과 125일 때, 곱의 지수 칸 값은?

답 보기

126 + 125 − 127 = 124 (실제 지수 −3).

4. 9.9999×10^{2}를 유효숫자 4자리로 반올림하면 어떤 추가 처리가 필요한가?

답 보기

10.000×10^{2}로 정규화가 깨지므로 다시 정규화해 1.000×10^{3}.

5. 부동소수점 곱셈의 마지막(5번째) 단계는?

답 보기

곱의 부호 결정(부호-크기 표현, 두 부호가 같으면 +, 다르면 −).

6. 0.5 × (−0.4375)의 결과를 2진 정규화 표기와 십진수로 쓰라.

답 보기

$-1.110_{2} \times 2^{-3} = -0.21875$.

Ch3 · 9 슬라이드 p.37–40

FP 명령어와 정확한 연산

이 단원을 마치면 RISC-V 부동소수점 명령·레지스터를 구분하고, 4가지 반올림 모드와 guard·round·sticky 비트로 반올림 결과를 판정할 수 있다.

쉽게 말하면

계산기의 화면은 몇 자리뿐이지만, 계산 중에는 화면에 안 보이는 자리를 몇 개 더 들고 있다가 마지막에 반올림하면 더 정확하다. IEEE 754도 계산 중에 오른쪽에 여분 비트(guard, round, sticky)를 들고 있다가 반올림에 쓴다.

비유가 안 맞는 지점 기본 반올림은 학교에서 배운 사사오입과 조금 다르다. 정확히 절반일 때는 무조건 올리지 않고 **끝자리가 짝수(0)인 쪽**으로 보낸다.

RISC-V 부동소수점 명령

RISC-V는 IEEE 754 형식용 명령을 지원한다. `.s`는 단정도(single), `.d`는 배정도(double)다. [Ch3 p.37]

명령	설명
`fadd.s` / `fadd.d`, `fsub.s` / `fsub.d`	부동소수점 덧셈, 뺄셈
`fmul.s` / `fmul.d`, `fdiv.s` / `fdiv.d`, `fsqrt.s` / `fsqrt.d`	곱셈, 나눗셈, 제곱근
`feq.s` / `feq.d`, `flt.s` / `flt.d`, `fle.s` / `fle.d`	같음, 작음, 작거나 같음 비교
`flw` / `fsw`, `fld` / `fsd`	단정도·배정도 적재(load)·저장(store)

- 비교 명령은 결과를 **정수 레지스터**(예: `x5`)에 쓴다. 거짓이면 0, 참이면 1. 이 값으로 `beq`, `bne`를 써서 분기한다. [Ch3 p.37]
- RISC-V 설계자는 부동소수점 전용 레지스터를 따로 두었다: `f0` ~ `f31`. **`f0`은 상수 0으로 고정되어 있지 않다**(정수의 `x0`과 다름). [Ch3 p.37]
- 적재·저장도 따로 있다: `fld` / `fsd`(배정도), `flw` / `fsw`(단정도). [Ch3 p.37]

```
flt.s  x5, f0, f1      // f0 < f1 이면 x5 = 1, 아니면 x5 = 0
bne    x5, x0, Less    // x5가 0이 아니면(참이면) Less로 분기
```

슬라이드가 지정한 교재 pp.225–226 예제 중 하나는 화씨를 섭씨로 바꾸는 함수다. 인자 `fahr`는 `f10`으로 들어오고 결과도 `f10`으로 돌려준다. 상수는 전역 포인터 `x3` 기준으로 메모리에서 읽는다. 교재 기준

```
// float f2c(float fahr) { return ((5.0f/9.0f) * (fahr - 32.0f)); }
f2c:
  flw    f0, const5(x3)     // f0 = 5.0f
  flw    f1, const9(x3)     // f1 = 9.0f
  fdiv.s f0, f0, f1         // f0 = 5.0f / 9.0f
  flw    f1, const32(x3)    // f1 = 32.0f
  fsub.s f10, f10, f1       // f10 = fahr - 32.0f
  fmul.s f10, f0, f10       // f10 = (5.0f/9.0f) * (fahr - 32.0f)
  jalr   x0, 0(x1)          // 복귀
```

다른 예제(배정도 행렬 곱)도 같은 방식으로 `fld`, `fmul.d`, `fadd.d`, `fsd`를 쓴다.

부동소수점은 근삿값

정수와 달리 부동소수점 수는 대부분 정확히 나타낼 수 없는 수의 **근삿값**이다. 1과 2 사이에는 실수가 무한히 많지만, 배정도로 정확히 나타낼 수 있는 수는 슬라이드 표현으로 2^{53}개를 넘지 못한다. 그래서 실제 수에 최대한 가깝게 만들어야 한다. [Ch3 p.38] (정확히 세면 1 이상 2 미만의 배정도 수는 Fraction 52비트의 조합인 2^{52}개다. PDF 외 보충)

IEEE 754는 프로그래머가 원하는 근사 방법을 고를 수 있게 여러 **반올림 모드**를 둔다. [Ch3 p.38]

guard와 round 비트

IEEE 754는 중간 덧셈을 하는 동안 오른쪽에 여분 비트 2개를 항상 유지한다. 이름은 **guard(가드)**와 **round(라운드)**다. [Ch3 p.38]

예: 유효숫자 3자리, 가장 가까운 값으로 반올림하는 십진 계산에서 $2.56 \times 10^0 + 2.34 \times 10^2$. 2.56×10^0을 맞추면 0.0256×10^2이고, 5가 guard 자리, 6이 round 자리다. [Ch3 p.38]

guard·round 없이	guard·round 있이
2.34 + 0.02 = 2.36 → **2.36×10^2**	2.3400 + 0.0256 = 2.3656 → 반올림 **2.37×10^2**

진짜 답 236.56에 가까운 것은 2.37×10^2(237)이다. 여분 자리가 없으면 0.0256의 56이 미리 잘려 2.36이 된다.

4가지 반올림 모드

기본 모드는 **가장 가까운 짝수로 반올림(round to nearest even)**이다. [Ch3 p.39] 슬라이드의 수직선 예 −2.75와 3.8을 정수로 반올림하면 다음과 같다.

모드	뜻	−2.75	3.8
0 쪽으로(Round toward Zero)	버림(truncation), 0에 가까운 쪽	−2	3
+∞ 쪽으로(toward Positive Infinity)	수직선 오른쪽(큰 쪽)	−2	4
−∞ 쪽으로(toward Negative Infinity)	수직선 왼쪽(작은 쪽)	−3	3
가장 가까운 짝수(Nearest Even, 기본)	가까운 쪽, 정확히 절반이면 짝수 쪽	−3	4

−2.75와 3.8은 절반이 아니라서 nearest even은 그냥 가까운 쪽이다. 짝수 규칙은 정확히 절반일 때만 쓴다. 예: 2.5 → 2, 3.5 → 4.
PDF 외 보충

guard·round·sticky(GRS)

절반인 경우를 제대로 판정하려고 IEEE 754는 비트 3개를 유지한다. [Ch3 p.40]

Guard(G) 남길 fraction의 LSB 바로 다음 비트.

Round(R) guard 비트 바로 오른쪽 비트.

Sticky(S) guard·round 뒤에 오는 나머지 비트 전체를 OR한 값. 하나라도 1이면 1.

예: $1.0101001000010_2 \times 2^4$를 fraction 6비트로 남기는 경우. 남길 부분 1.010100_2, 그다음 G = 1, R = 0, 나머지 비트 00010을 OR하면 S = 1 → **$1.010100 \mid 101_2 \times 2^4$**(| 뒤는 GRS). [Ch3 p.40]

GRS	뜻	처리 (nearest even)
000 ~ 011	절반보다 작음	내림: fraction 그대로
100	정확히 절반	LSB가 0인 쪽(짝수)으로
101 ~ 111	절반보다 큼	올림: fraction + 1

위 예는 GRS = 101 → 올림 → $1.010101_2 \times 2^4$. sticky가 없으면 GR = 10만 보고 "정확히 절반"으로 오판하게 된다. sticky가 뒤에 1이 숨어 있음을 알려 준다.

- **Case 1**: $1.010100\mathbf{100}_2 \times 2^4$ → GRS 100(절반), LSB 0 → 그대로 → **$1.010100_2 \times 2^4$**. [Ch3 p.40]
- **Case 2**: $1.010101\mathbf{100}_2 \times 2^4$ → GRS 100(절반), LSB 1 → 짝수로 가려면 올림 → fraction 010101 + 1 = 010110 → **$1.010110_2 \times 2^4$**. [Ch3 p.40]

핵심

.s 단정도·.d 배정도. feq/flt/fle → 정수 레지스터에 0/1 → beq/bne. FP 레지스터 f0~f31(f0은 0 고정 아님), flw/fsw·fld/fsd. 반올림 모드 4: 0 쪽, +∞ 쪽, −∞ 쪽, 가장 가까운 짝수(기본). G = LSB 다음, R = 그다음, S = 나머지 OR. GRS 0xx → 내림, 101~111 → 올림, 100 → LSB 0 쪽.

예제

fraction을 4비트만 남길 때 nearest even으로 반올림하라. 수는 모두 2진 유효숫자다. (1) 1.0110 011 (2) 1.0111 100 (3) 1.0110 100 (4) 1.0110 1000001 (띄어 쓴 뒤 비트는 G, R, 그 뒤 나머지)

풀이 보기

(1) GRS 011 → 내림 → **1.0110_2**.

(2) GRS 100, LSB 1 → 올림 → fraction 0111 + 1 = 1000 → **1.1000_2**.

(3) GRS 100, LSB 0 → 그대로 **1.0110_2**.

(4) G = 1, R = 0, 나머지 비트 00001 → S = 1 → GRS 101 → 올림 → **1.0111_2**.

함정

`f0`은 `x0`처럼 항상 0이 아니다. 일반 레지스터처럼 값을 담는다.

FP 비교 결과는 FP 레지스터가 아니라 **정수 레지스터**에 0 또는 1로 들어간다.

"round toward zero"와 "round toward −∞"는 양수에선 같지만 음수에선 다르다. −2.75 → 0 쪽은 −2, −∞ 쪽은 −3.

GRS = 100일 때 무조건 올리지 않는다. LSB가 0이면 그대로, 1이면 올린다.

시험 포인트

- FP 비교 결과가 들어가는 곳(정수 레지스터)과 분기 방법(beq/bne).
- f0 ≠ 0 상수, flw/fsw/fld/fsd 구분.
- $2.56 \times 10^0 + 2.34 \times 10^2$: guard·round 없으면 2.36×10^2, 있으면 2.37×10^2.
- 4가지 반올림 모드 이름과 −2.75·3.8 결과, 기본 모드.
- GRS 정의와 판정, Case 1·2 결과.

스스로 확인

1. `flt.s`의 결과는 어디에 어떤 값으로 저장되는가?

   답 보기

   정수 레지스터에 참이면 1, 거짓이면 0.

2. O/X: RISC-V의 `f0`은 항상 0.0이다.

   답 보기

   X. f0은 상수 0으로 고정되어 있지 않다.

3. 배정도 값을 메모리에서 FP 레지스터로 읽는 명령은?

   답 보기

   `fld`.

4. 유효숫자 3자리로 $2.56 \times 10^0 + 2.34 \times 10^2$를 guard·round 비트 없이, 있이 계산한 결과는?

   답 보기

   없이 2.36×10^2, 있이 2.37×10^2.

5. IEEE 754의 기본 반올림 모드는?

   답 보기

   가장 가까운 짝수로 반올림(round to nearest even).

6. −2.75를 round toward +∞로 정수 반올림하면?

답 보기

−2 (수직선에서 오른쪽, 큰 쪽).

7. sticky 비트의 정의는?

답 보기

guard·round 비트 뒤에 있는 나머지 모든 비트를 논리 OR한 값.

8. $1.010101100_2 \times 2^4$(fraction 6비트 + GRS)를 nearest even으로 반올림하면?

답 보기

GRS 100이고 LSB 1 → 올림 → $1.010110_2 \times 2^4$.

Ch4 · 1 슬라이드 p.2–8

프로세서 구현 개요

이 단원을 마치면 성능 3요소를 누가 정하는지 말하고, 이 장에서 구현할 명령어 목록과 모든 명령어의 공통 두 단계를 쓰고, MUX와 제어 유닛이 왜 필요한지 설명할 수 있다.

쉽게 말하면

Ch2에서는 "어떤 명령어가 있는가"를 배웠다. 이 장에서는 그 명령어를 실제로 실행하는 기계를 부품부터 조립한다. 요리에 비유하면 Ch2가 레시피 목록이고, Ch4는 주방 설계다. 어느 레시피든 처음 두 동작(레시피 카드 꺼내기, 재료 꺼내기)은 같고, 그다음은 요리마다 다르다. 주방에는 조리대(계산 부품)와 냉장고(저장 부품)가 있고, 요리마다 어느 길로 재료를 옮길지 정하는 주방장(제어 유닛)이 필요하다.

비유가 안 맞는 지점 주방장은 판단하고 고민하지만 제어 유닛은 그렇지 않다. 명령어 비트를 입력으로 받아 정해진 신호를 내보내는 논리 회로일 뿐이다.

성능 3요소와 각각을 정하는 주체

Ch1에서 배운 대로 CPU 시간은 세 가지 요소의 곱이다. Ch4는 이 중 두 요소를 정하는 "CPU 하드웨어"를 다룬다. [Ch4 p.2]

```
CPU 시간 = 명령어 수(IC) × CPI × 클럭 주기
```

요소	정하는 주체
명령어 수(Instruction count)	ISA와 컴파일러
클럭 주기(Clock cycle time)	CPU 하드웨어
명령어당 클럭 사이클 수(CPI, Clock cycles Per Instruction)	CPU 하드웨어

여기서 "CPU 하드웨어"는 마이크로아키텍처(ISA를 실제 회로로 구현한 방식), 공정 기술 등을 말한다. 슬라이드의 예가 Intel과 AMD다. 둘 다 같은 x86 ISA를 쓰지만 하드웨어 구현이 달라서 클럭 주기와 CPI가 다르다. [Ch4 p.2]

두 가지 구현과 대상 명령어

이 장은 RISC-V를 두 가지로 구현한다. ① 단순한(순차적) 버전 ② 파이프라인 버전. 시험 범위는 ①인 **단일 사이클(single-cycle)** 구현까지다. [Ch4 p.3]

모든 명령어를 다 만들지 않고 핵심 부분집합(subset)만 구현한다. [Ch4 p.3]

종류	명령어
메모리 참조(memory-reference)	`lw`, `sw` (load word, store word)
산술·논리(arithmetic-logical)	`add`, `sub`, `and`, `or`
조건 분기(conditional branch)	`beq` (branch if equal)

상기: Ch2에서 `add` · `sub` · `and` · `or`는 R형, `lw`는 I형, `sw`는 S형, `beq`는 SB형이었다. 모든 명령어는 32비트다.

모든 명령어의 공통 두 단계

어떤 명령어든 처음 두 단계는 똑같다. [Ch4 p.3]

1. **명령어 인출(instruction fetch)**: 프로그램 카운터(PC, Program Counter)의 값을 명령어가 들어 있는 메모리에 보내고, 그 주소의 명령어를 가져온다.
2. **레지스터 읽기**: 명령어의 필드(rs1, rs2)로 읽을 레지스터를 골라 1개 또는 2개를 읽는다.

그다음부터는 명령어 종류(class)에 따라 다르다. 예를 들어 산술 명령어는 두 레지스터로 연산하고 결과를 레지스터에 저장한다. [Ch4 p.3] 슬라이드 p.4의 그림은 이 흐름을 PC, 가산기, ALU, 명령어 메모리, 데이터 메모리, 레지스터로 그린 단순화된 데이터패스다. 왼쪽이 공통 두 단계, 오른쪽이 이후 단계다. [Ch4 p.4]

프로그램 카운터(PC) — 지금 실행할 명령어의 메모리 주소를 담은 레지스터.

ALU(Arithmetic Logic Unit, 산술 논리 장치) — 덧셈·뺄셈·AND·OR 같은 연산을 하는 회로.

데이터패스(datapath) — 데이터를 옮기고 계산하고 저장하는 부품과 그 연결선 전체.

예: add와 lw의 실행 4단계

단계	`add x5, x6, x7` [Ch4 p.5]	`lw x5, 40(x6)` [Ch4 p.6]
1	명령어 인출	명령어 인출
2	레지스터 읽기: `x6`, `x7`	레지스터 읽기: `x6`
3	덧셈: `x6 + x7`	주소 계산 덧셈: `x6 + 40`
4	합을 `x5`에 쓰기	메모리 읽기 `Memory[x6+40]` 후 그 값을 `x5`에 쓰기

두 명령어 모두 3단계에서 ALU로 **덧셈**을 한다. 차이는 ALU의 두 번째 입력이 레지스터(`x7`)냐 명령어 안의 숫자(40)냐, 그리고 레지스터에 쓰는 값이 ALU 결과냐 메모리에서 읽은 값이냐다. 바로 이 차이 때문에 다음 절의 MUX가 필요하다.

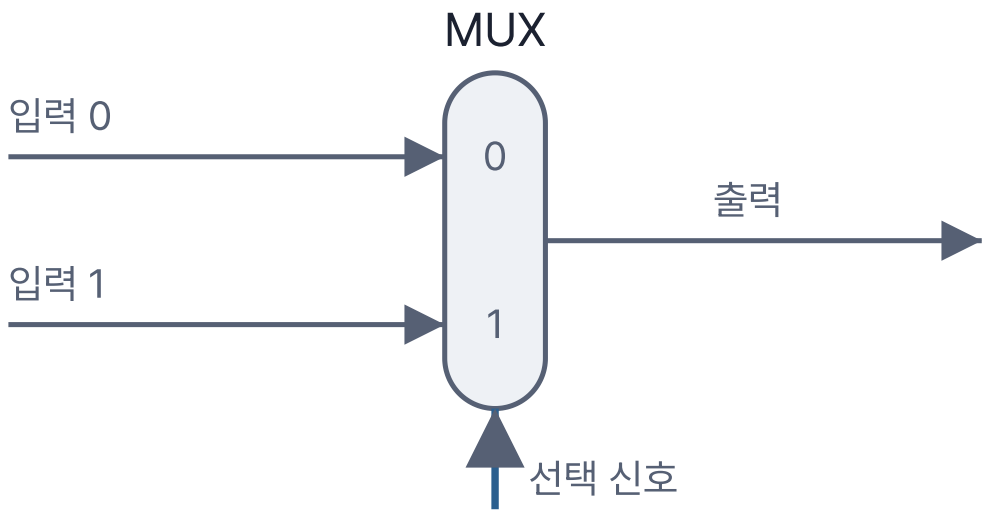
MUX와 제어 유닛이 필요한 이유

p.4의 단순 그림에는 중요한 것 두 가지가 빠져 있다. [Ch4 p.7]

1. **데이터 선을 그냥 이어 붙일 수 없다**(Cannot just join wires together) → 멀티플렉서가 필요하다. 예를 들어 레지스터에 쓸 값이 ALU 결과일 수도, 메모리 값일 수도 있는데, 두 선을 한 점에 묶으면 두 신호가 충돌한다.
2. **데이터패스를 제어하는 부분이 없다** → 제어 유닛이 필요하다. 명령어마다 레지스터에 쓸지, 메모리를 읽을지, ALU가 무엇을 할지 정해 줘야 한다.

멀티플렉서(MUX, multiplexer) — 여러 입력 중 하나를 **선택 신호**에 따라 골라 출력으로 내보내는 회로. 데이터 선택기(data selector)라고도 한다.

제어 유닛(control unit) — 명령어를 보고 데이터패스의 MUX 선택 신호, 쓰기 허가 신호, ALU 동작 등을 정하는 회로.



2입력 MUX. 선택 신호가 0이면 입력 0을, 1이면 입력 1을 내보낸다.

프로세서 = 데이터패스 + 제어

p.8의 기본 구현은 p.4 그림에 **MUX 3개와 제어 유닛**을 더한 것이다. 그래서 **프로세서 = 데이터패스 + 제어**다(Ch1의 5대 구성 요소와 같은 말). [Ch4 p.8] 세 MUX는 다음 자리에 있다. 교재 기준

1. **다음 PC 선택**: PC + 4 와 분기 목표 주소 중 하나.
2. **레지스터에 쓸 값 선택**: ALU 결과와 데이터 메모리에서 읽은 값 중 하나.
3. **ALU 두 번째 입력 선택**: 레지스터 값과 명령어의 즉시값(오프셋) 중 하나.

이 세 MUX의 이름은 c4-4에서 PCSrc, MemtoReg, ALUSrc로 붙는다.

핵심

IC = ISA·컴파일러, 클럭 주기·CPI = CPU 하드웨어. 구현 대상 = lw, sw, add, sub, and, or, beq. 공통 두 단계 = ① PC로 명령어 인출 ② 레지스터 1~2개 읽기. 선을 그냥 합칠 수 없어서 MUX, 데이터패스를 조종해야 해서 제어 유닛. 프로세서 = 데이터패스 + 제어.

예제

`sub x5, x6, x7`과 `lw x5, 40(x6)`에서 ALU의 두 번째 입력은 각각 무엇인가? 레지스터 `x5`에 들어가는 값은 각각 어디서 오는가?

풀이 보기

`sub` : ALU 두 번째 입력 = 레지스터 `x7`의 값, ALU가 뺄셈, `x5`에는 ALU 결과가 들어간다.

`lw` : ALU 두 번째 입력 = 명령어 안의 40, ALU가 덧셈해 주소를 만들고, `x5`에는 메모리에서 읽은 값이 들어간다.

같은 자리에 들어가는 값이 명령어마다 다르므로 그 자리마다 MUX가 필요하다.

함정

"CPI는 컴파일러가 정한다"는 틀리다. 슬라이드는 클럭 주기와 CPI를 둘 다 CPU 하드웨어가 정한다고 했다. 명령어 수만 ISA와 컴파일러다. 또 lw도 ALU를 쓴다는 점을 놓치기 쉽다. lw의 ALU는 주소 계산용 덧셈을 한다.

시험 포인트

- 성능 3요소와 결정 주체 짝짓기 [Ch4 p.2]
- 구현할 명령어 7개를 종류별로 쓰기, 공통 두 단계 쓰기 [Ch4 p.3]
- add·lw의 실행 단계 순서 [Ch4 p.5–6]
- MUX가 필요한 이유, 제어 유닛이 필요한 이유 [Ch4 p.7], 프로세서 = 데이터패스 + 제어 [Ch4 p.8]

스스로 확인

1. 성능의 세 요소 중 ISA와 컴파일러가 정하는 것은?

   답 보기

   명령어 수(IC). 클럭 주기와 CPI는 CPU 하드웨어가 정한다.

2. 모든 명령어에 공통인 처음 두 단계를 쓰시오.

   답 보기

   ① PC가 가리키는 주소에서 명령어 인출 ② 명령어 필드로 고른 레지스터 1개 또는 2개 읽기.

3. `lw x5, 40(x6)`의 3단계에서 ALU가 하는 연산과 그 목적은?

   답 보기

   덧셈 `x6 + 40`. 읽을 메모리 주소를 계산하기 위해서다.

4. 빈칸: 데이터 선은 그냥 합칠 수 없으므로 ( )가 필요하고, 데이터패스를 조종하기 위해 ( )가 필요하다.

   답 보기

   멀티플렉서(MUX), 제어 유닛.

5. 프로세서는 무엇과 무엇으로 이루어지는가?

   답 보기

   데이터패스와 제어.

6. O/X: 이 장에서 구현하는 부분집합에는 `addi`가 포함된다.

답 보기

X. 부분집합은 lw, sw, add, sub, and, or, beq다.

Ch4 · 2 슬라이드 p.9–10

논리 설계 규칙과 클로킹

이 단원을 마치면 데이터패스 부품을 조합 소자와 상태 소자로 나누고, 엣지 트리거 클로킹이 무엇이며 클럭 주기가 왜 가장 긴 지연으로 정해지는지 설명할 수 있다.

쉽게 말하면

계산기와 메모장을 떠올리면 된다. 계산기는 숫자를 넣는 즉시 답이 나오고 아무것도 기억하지 않는다(조합 소자). 메모장은 적어 둔 값을 계속 보여 주고, 정해진 순간에만 새 값으로 고쳐 쓴다(상태 소자). 클럭은 "지금 고쳐 써!"라고 일정 간격으로 울리는 종이다. 종이 울릴 때만 메모장이 바뀌고, 종과 종 사이에 계산기가 다음에 적을 값을 계산해 둔다.

비유가 안 맞는 지점 실제 조합 회로는 "즉시"가 아니라 신호가 통과하는 데 시간(지연)이 걸린다. 이 지연이 클럭 주기를 정한다는 것이 이 단원의 핵심이다.

두 종류의 논리 소자

데이터패스는 서로 다른 두 종류의 논리 소자로 이루어진다. [Ch4 p.9]

	조합 소자(combinational element)	상태 소자(state element)
하는 일	연산(operational)	정보 저장(memory)
예	AND 게이트, ALU, 가산기, MUX	레지스터, 메모리, PC
출력	지금 입력만의 함수. 같은 입력이면 항상 같은 출력	이전 클럭 사이클에 써 둔 값
입력	연산할 데이터	최소 2개: 쓸 데이터 값, 언제 쓸지 정하는 클럭

가산기·MUX는 슬라이드 p.9 예시에는 없지만 출력이 입력만의 함수이므로 조합 소자다. PC는 레지스터이므로 상태 소자다. PDF 외 보충

상태 소자는 입력이 최소 2개, 출력이 1개다. 입력은 **써 넣을 데이터 값**과 **클럭**(언제 쓸지 결정)이고, 출력은 **이전 클럭 사이클에 쓴 값**이다. [Ch4 p.9] 슬라이드 그림의 소자는 D 플립플롭 형태다.

D 플립플롭(D flip-flop) 1비트를 저장하는 가장 단순한 상태 소자. 입력 D(데이터)와 Clk(클럭), 출력 Q. 클럭 엣지 순간의 D 값이 저장되어 다음 엣지까지 Q로 나온다.

쓰기 제어(Write) 슬라이드 그림처럼 Write 입력이 붙은 상태 소자는 Write가 1이고 클럭 엣지가 올 때만 값을 바꾼다. Write가 0이면 엣지가 와도 이전 값을 유지한다. [Ch4 p.9]

활성(asserted) / 비활성(deasserted) 신호가 논리 1(참)이면 활성, 0이면 비활성이라 부른다. 다음 단원부터 제어 신호 설명에 쓴다. PDF 외 보충

클로킹 방식

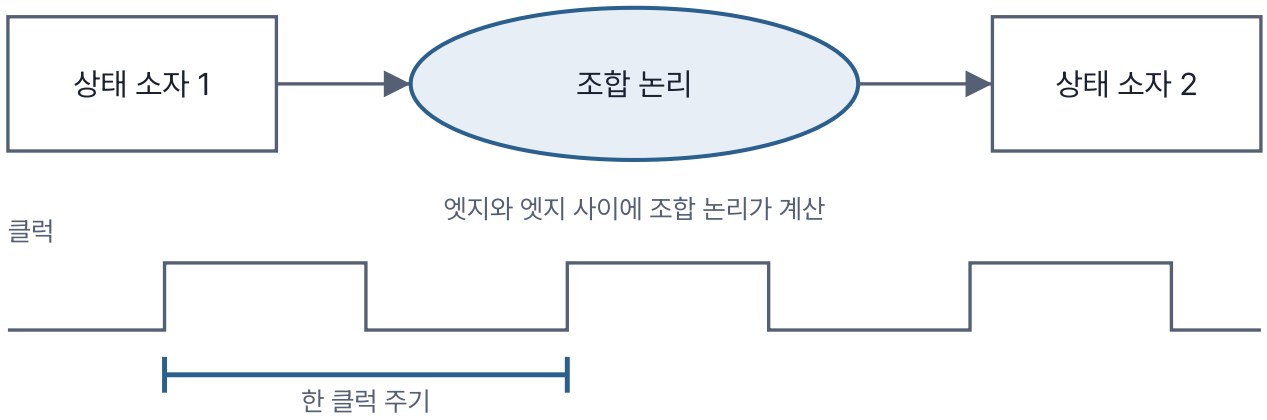
클로킹 방식(clocking methodology)은 신호를 **언제 읽을 수 있고 언제 쓸 수 있는지** 정하는 규칙이다. 하드웨어 동작을 예측 가능하게 만들려고 정한다. [Ch4 p.10]

이 장은 **엣지 트리거 클로킹(edge-triggered clocking)**을 가정한다. 순차 논리 소자(상태 소자)에 저장된 값은 **클럭 엣지에서만** 바뀐다는 뜻이다. [Ch4 p.10]

클럭 엣지(clock edge) 클럭 신호가 0에서 1로(상승 엣지) 또는 1에서 0으로(하강 엣지) 바뀌는 순간.

조합 논리는 클럭 사이클 동안 데이터를 변환한다. [Ch4 p.10]

- 그 변환은 **클럭 엣지와 엣지 사이**에 일어난다.
- 입력은 상태 소자에서 오고, 출력은 상태 소자로 간다.
- **가장 긴 지연이 클럭 주기를 정한다.**



상승 엣지에서 상태 소자 1의 값이 나가고, 조합 논리가 계산한 결과가 다음 상승 엣지에서 상태 소자 2에 저장된다. 조합 논리 지연이 한 주기 안에 끝나야 한다.

왜 가장 긴 지연이 주기를 정할까? 엣지에서 상태 소자가 값을 내보내면 조합 논리가 계산을 시작한다. 결과는 다음 엣지에서 다른 상태 소자에 저장된다. 그러므로 다음 엣지가 오기 전에 계산이 끝나야 한다. 경로가 여러 개면 가장 느린 경로까지 끝나야 하므로 **가장 긴 지연 ≤ 클럭 주기**가 되어야 한다.

엣지 트리거 방식에서는 한 상태 소자를 같은 사이클에 읽고 쓸 수도 있다. 예를 들어 PC는 사이클 내내 현재 값을 내보내고(읽기), 가산기가 만든 PC + 4는 다음 엣지에 저장된다(쓰기). 값은 엣지에서만 바뀌므로 읽는 도중에 값이 바뀌는 일이 없다. PDF 외 보충

핵심

조합 소자 = 출력이 입력만의 함수(AND, ALU). 상태 소자 = 정보 저장(레지스터, 메모리), 입력 ≥ 2(데이터, 클럭), 출력 = 이전 사이클에 쓴 값. 클로킹 방식 = 언제 읽고 쓰는지 정의. 엣지 트리거 = 상태 소자는 클럭 엣지에서만 갱신. 조합 논리는 엣지 사이에 동작, 가장 긴 지연이 클럭 주기를 결정.

예제

상태 소자 사이에 조합 논리 경로가 세 개 있고, 지연이 각각 300 ps, 650 ps, 500 ps다. 엣지 트리거 방식이라면 클럭 주기는 최소 얼마여야 하는가? 클럭 주파수는? (플립플롭 자체 지연은 무시) PDF 외 보충

풀이 보기

가장 긴 지연이 주기를 정하므로 최소 650 ps.

주파수 = 1 / 650 ps ≈ 1.54 GHz ($1 / (650 \times 10^{-12}) \approx 1.538 \times 10^{9}$ Hz).

함정

"메모리는 계산을 하지 않으니 조합 소자"라고 착각하기 쉽다. 분류 기준은 저장 여부다. 레지스터와 메모리는 정보를 저장하므로 상태 소자다. 또 상태 소자의 출력은 "지금 들어온 값"이 아니라 "이전 클럭 사이클에 쓴 값"이다.

시험 포인트

- 조합 소자 vs 상태 소자 정의와 예 분류 [Ch4 p.9]
- 상태 소자의 입력 2개와 출력 1개가 각각 무엇인지 [Ch4 p.9]
- 클로킹 방식 정의, 엣지 트리거 정의, 클럭 주기를 정하는 것 [Ch4 p.10]

스스로 확인

1. 다음을 조합 소자(C)와 상태 소자(S)로 분류: ALU, 레지스터 파일, AND 게이트, 데이터 메모리, PC.

   답 보기

   ALU = C, 레지스터 파일 = S, AND 게이트 = C, 데이터 메모리 = S, PC = S. 저장하면 상태 소자다.

2. 상태 소자에 필요한 두 입력은?

   답 보기

   써 넣을 데이터 값, 언제 쓸지 정하는 클럭.

3. 상태 소자의 출력은 어떤 값인가?

   답 보기

   이전 클럭 사이클에 써 둔 값.

4. 클로킹 방식(clocking methodology)을 정의하시오.

   답 보기

   신호를 언제 읽을 수 있고 언제 쓸 수 있는지 정하는 규칙. 하드웨어를 예측 가능하게 하려는 것이다.

5. 엣지 트리거 클로킹이란?

   답 보기

   상태 소자에 저장된 값이 클럭 엣지에서만 갱신되는 방식.

6. 빈칸: 조합 논리는 클럭 ( ) 사이에 동작하고, 가장 ( ) 지연이 클럭 주기를 결정한다.

답 보기

엣지, 긴.

Ch4 · 3 슬라이드 p.11–15

데이터패스 부품 만들기

이 단원을 마치면 명령어 인출, R형, load/store, 분기에 각각 필요한 부품과 연결을 말하고, 분기 목표 주소를 계산할 수 있다.

쉽게 말하면

전체 기계를 한 번에 만들지 않고, 명령어 종류별로 "이 명령어 하나만 실행하려면 무엇이 필요한가"를 따로 만든다. 레고를 조립하기 전에 바퀴 부분, 몸통 부분을 따로 만들어 두는 것과 같다. 다음 단원에서 이 조각들을 합친다.

비유가 안 맞는 지점 레고 조각은 그냥 붙이면 되지만, 데이터패스 조각은 같은 입력 자리에 서로 다른 선이 들어오므로 합칠 때 MUX가 추가로 필요하다.

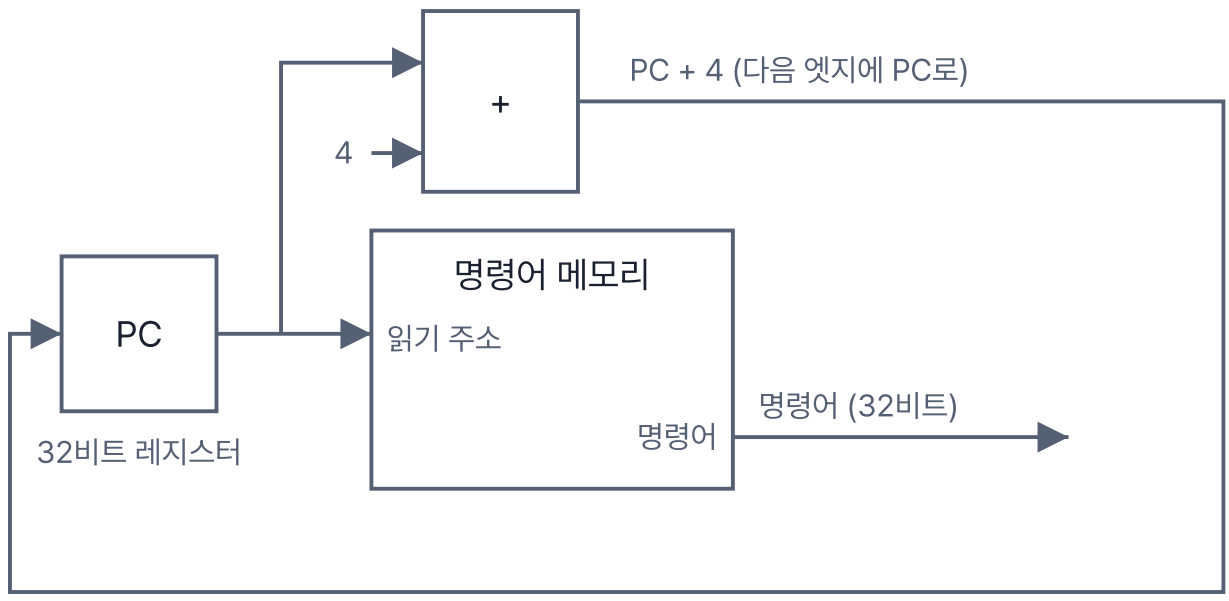
데이터패스 요소(datapath element)는 프로세서 안에서 데이터를 다루거나 담는 장치다. 명령어 메모리, 데이터 메모리, 레지스터 파일, ALU, 가산기가 여기 속한다. [Ch4 p.11]

1. 명령어 인출: 명령어 메모리, PC, 가산기

명령어를 실행하기 시작하려면 세 요소가 필요하다. [Ch4 p.11]

- **명령어 메모리(instruction memory)**: 명령어를 저장하고, 주소를 받으면 그 명령어를 내보낸다.
- **PC**: 현재 명령어의 주소를 담는 32비트 레지스터.
- **가산기(adder)**: PC를 다음 명령어 주소로 증가시킨다. 명령어가 4바이트이므로 **4를 더한다**.

어떤 명령어든 먼저 메모리에서 명령어를 인출하고, 다음 명령어를 준비하려고 PC를 증가시킨다. [Ch4 p.11]



명령어 인출 부분. 가산기는 항상 PC + 4를 계산하고, 그 값이 다음 클럭 엣지에 PC에 저장된다(분기는 뒤에서 추가).

2. R형 명령어: 레지스터 파일과 ALU

R형 명령어(`add` , `sub` , `and` , `or`)는 레지스터 두 개를 읽고, ALU 연산을 하고, 결과를 레지스터에 쓴다. 예: `add x1, x2, x3` . [Ch4 p.12]

레지스터 파일(register file) 프로세서의 범용 레지스터 32개를 모아 둔 구조. 레지스터 번호를 지정하면 어느 레지스터든 읽거나 쓸 수 있다. [Ch4 p.12]

R형 하나를 실행하려면 레지스터 파일에 다음이 필요하다. 교재 기준

- **읽기 포트 2개**: 입력 "읽기 레지스터 1·2"(5비트 번호, rs1·rs2), 출력 "읽기 데이터 1·2"(32비트 값).
- **쓰기 포트 1개**: 입력 "쓰기 레지스터"(5비트 번호, rd)와 "쓰기 데이터"(32비트 값).
- **RegWrite** 제어 신호: 1일 때만 클럭 엣지에서 쓰기가 일어난다. 읽기는 제어 신호 없이 번호만 주면 항상 나온다.

번호가 5비트인 이유는 레지스터가 32 = 2^5개이기 때문이다.

ALU는 32비트 입력 2개를 받아 32비트 결과를 낸다. 추가로 [Ch4 p.12]

- **Zero** 출력(1비트): 결과가 0이면 1이 된다.
- **ALU 동작 제어 신호(4비트)**: 어떤 연산을 할지 정한다(c4-5에서 자세히).

3. load/store: ImmGen과 데이터 메모리

load/store는 **베이스 레지스터**에 **12비트 부호 있는 오프셋**을 더해 메모리 주소를 계산한다. 예: `lw x1, offset(x2)` , `sw x1, offset(x2)` 에서 `x2` 가 베이스 레지스터다. [Ch4 p.13]

- store(`sw`): 레지스터 값(`x1`)을 메모리에 쓴다.
- load(`lw`): 메모리를 읽어 레지스터(`x1`)를 갱신한다.

ALU는 32비트끼리 더하므로 12비트 오프셋을 32비트로 늘려야 한다. 이때 Ch2에서 배운 **부호 확장(sign extension)**을 쓴다. 12비트의 맨 왼쪽 비트(부호 비트)를 위쪽 20비트에 복사하면 값이 그대로 유지된다. 이 일을 하는 장치가 **즉시값 생성기(ImmGen, Immediate generator)**다. 32비트 명령어 전체를 입력으로 받아 12비트 필드를 골라 32비트 부호 확장 값을 만든다. [Ch4 p.13]

ImmGen이 명령어 전체를 받는 이유는 형식마다 즉시값 위치가 다르기 때문이다. lw(I형)는 [31:20]에 12비트가 연속으로 있고, sw(S형)는 [31:25]와 [11:7]로 나뉘어 있다. ImmGen이 형식에 맞게 골라 붙인다. PDF 외 보충

데이터 메모리(data memory): 입력은 주소와 쓰기 데이터, 출력은 읽기 데이터. 제어 신호 **MemWrite**(쓰기)와 **MemRead**(읽기)가 있다. 교재 기준

4. 분기: 목표 주소 계산과 같음 검사

분기 명령어 `beq x1, x2, offset` 은 피연산자가 셋이다. 같은지 비교할 레지스터 2개, 분기 목표 주소를 계산할 12비트 오프셋 1개. [Ch4 p.14]

분기 목표 주소(branch target address)는 부호 확장한 오프셋을 PC에 더해서 구한다. 이때 오프셋을 **왼쪽으로 1비트 시프트**한다(반 word, 즉 2바이트 단위 변위). 그래서 분기는 **짝수 주소로만** 갈 수 있다. [Ch4 p.14]

```
분기 목표 주소 = PC + (부호 확장한 오프셋 필드 << 1)
```

상기: Ch2에서 SB형은 imm[12:1]만 저장하고 0번 비트는 항상 0이라 저장하지 않았다. 하드웨어의 "1비트 시프트"가 그 빠진 0을 붙이는 일이다. 범위는 −4096 ~ +4094바이트다. [Ch2 p.47]

여기서 PC는 **분기 명령어 자신의 주소**다(PC + 4가 아님). PDF 외 보충

분기 여부에 따라 PC를 갱신한다. [Ch4 p.14]

- 분기함(taken, 두 값이 같음): 분기 목표 주소가 새 PC.
- 분기 안 함(not taken): 증가된 PC(PC + 4)가 새 PC.

분기 데이터패스는 두 가지 일을 한다. [Ch4 p.15]

1. **목표 주소 계산**: ImmGen + 가산기. ImmGen이 부호 확장과 함께 분기용 1비트 왼쪽 시프트까지 해서(슬라이드 p.15) 분기 가산기로 보낸다. ALU와 별도인 가산기를 쓴다.
2. **레지스터 비교**: ALU가 두 레지스터를 **빼고** Zero 출력을 본다. 같으면 차가 0이므로 Zero = 1.

슬라이드 p.15 그림의 "Sign-bit wire replicated"는 부호 비트 선을 복제해 위쪽 비트를 채우는 것, 즉 부호 확장이다. "Shifted left by 1 bit"는 시프트 장치가 따로 계산하는 게 아니라 선을 한 칸 옆으로 옮겨 연결하는 것으로 구현할 수 있다는 뜻이다. 슬라이드에서는 두 주석이 모두 ImmGen을 가리킨다. 즉 시프트 상자가 따로 없고, ImmGen 출력이 분기 가산기(Add, Sum)로 바로 간다. [Ch4 p.15]

교재의 일부 그림에는 ImmGen과 분기 가산기 사이에 "Shift left 1" 상자가 따로 그려져 있다. 하는 일은 같으니 시험 그림에 그 상자가 있으면 "왼쪽 1비트 시프트"로 답하면 된다. PDF 외 보충

핵심

인출 = 명령어 메모리 + PC + 가산기(+4). R형 = 레지스터 파일(읽기 2, 쓰기 1, RegWrite) + ALU(32비트 입력 2, 4비트 제어, Zero). load/store = 베이스 + 12비트 부호 확장 오프셋(ImmGen) → 데이터 메모리. 분기 = 목표 PC + (오프셋 << 1)를 별도 가산기로, ALU 뺄셈의 Zero로 같음 검사. 짝수 주소만 가능.

예제

주소 1000번지(10진수)에 있는 `beq`의 12비트 오프셋 필드 값이 10진수 −6이다. 분기하면 다음 PC는? 분기하지 않으면? PDF 외 보충

풀이 보기

부호 확장한 −6을 1비트 왼쪽 시프트하면 −12. 목표 주소 = 1000 + (−12) = **988**.

분기하지 않으면 PC + 4 = **1004**.

988은 짝수다. 시프트 때문에 목표 주소는 항상 짝수가 된다.

함정

- 분기 목표 계산에 ALU를 쓴다고 착각하기 쉽다. ALU는 두 레지스터를 빼서 같은지 검사하는 데 쓰이고, 목표 주소는 별도 가산기가 계산한다.
- "오프셋을 2비트 시프트(×4)"는 MIPS 방식이다. RISC-V는 **1비트**(×2)다.
- 부호 확장은 위쪽을 0으로 채우는 것이 아니다. 부호 비트를 복사한다. 음수 오프셋이면 1로 채워진다.

시험 포인트

- 명령어 실행 시작에 필요한 세 요소 [Ch4 p.11]
- 레지스터 파일 정의, ALU의 입력·출력 폭, Zero, 4비트 제어 [Ch4 p.12]
- ImmGen이 하는 일(12비트 → 32비트 부호 확장) [Ch4 p.13]
- 분기 목표 주소 계산, 1비트 시프트 이유, 짝수 주소 [Ch4 p.14], 분기 데이터패스의 두 동작 [Ch4 p.15]

스스로 확인

1. 명령어를 실행하기 시작하는 데 필요한 세 요소는?

   답 보기

   명령어 메모리, PC, PC를 증가시키는 가산기.

2. PC에 4를 더하는 이유는?

   답 보기

   RISC-V 명령어가 32비트 = 4바이트이고 메모리가 바이트 주소이므로 다음 명령어는 4바이트 뒤에 있다.

3. ALU의 Zero 출력은 언제 1이 되는가?

   답 보기

   ALU 결과가 0일 때.

4. ImmGen의 입력과 출력은?

   답 보기

   입력 = 32비트 명령어 전체, 출력 = 12비트 필드를 부호 확장한 32비트 값.

5. 0x2000에 있는 `beq`의 오프셋 필드 값이 8이고 분기한다. 다음 PC는?

   답 보기

   0x2000 + (8 << 1) = 0x2000 + 16 = 0x2000 + 0x10 = **0x2010**.

6. 분기 데이터패스가 하는 두 동작은?

   답 보기

   ① ImmGen과 가산기로 분기 목표 주소 계산 ② ALU로 두 레지스터를 빼서 Zero로 같은지 검사.

7. O/X: RISC-V 분기는 홀수 주소로도 갈 수 있다.

답 보기

X. 오프셋을 1비트 시프트하므로 짝수 주소만 가능하다.

Ch4 · 4 슬라이드 p.16–17

단일 사이클 데이터패스 조립

이 단원을 마치면 단일 사이클 데이터패스에서 메모리가 둘인 이유를 말하고, MUX 세 개의 위치와 각 MUX가 고르는 두 입력을 쓰고, 전체 그림에서 신호의 흐름을 따라갈 수 있다.

쉽게 말하면

c4-3에서 만든 조각들을 한 판에 붙인다. 규칙은 하나다. **명령어 하나를 클럭 한 번에 끝낸다.** 한 사이클 안에서는 같은 부품을 두 번 쓸 수 없으니, 두 번 필요한 것은 부품을 두 개 둔다. 여러 조각이 같은 입력 자리를 원하면 그 자리에 MUX를 놓는다.

비유가 안 맞는 지점 "한 사이클에 한 번"은 실제로는 "한 사이클 동안 한 가지 용도로만"이다. 레지스터 파일처럼 읽기 포트가 둘인 부품은 한 사이클에 두 레지스터를 읽을 수 있다. 포트가 여러 개로 설계되어 있기 때문이다.

한 사이클에 자원은 한 번만

가장 단순한 데이터패스는 **모든 명령어를 클럭 한 사이클에 실행**한다. 이것이 **단일 사이클(single-cycle) 데이터패스**다. [Ch4 p.16]

- 명령어 하나당 어떤 데이터패스 자원도 두 번 이상 쓸 수 없다.
- 그러므로 **명령어 메모리와 데이터 메모리를 따로** 둔다. [Ch4 p.16] lw는 한 사이클 안에 명령어도 읽고(인출) 데이터도 읽어야 하는데, 메모리가 하나면 한 번밖에 쓸 수 없어서다.
- 같은 이유로 PC + 4용 가산기, 분기 목표용 가산기, ALU가 모두 따로 있다. 한 사이클에 덧셈이 세 번까지 필요하기 때문이다. PDF 외 보충

명령어마다 다른 데이터 출처를 쓰는 곳에는 **MUX**를 놓는다. [Ch4 p.16]

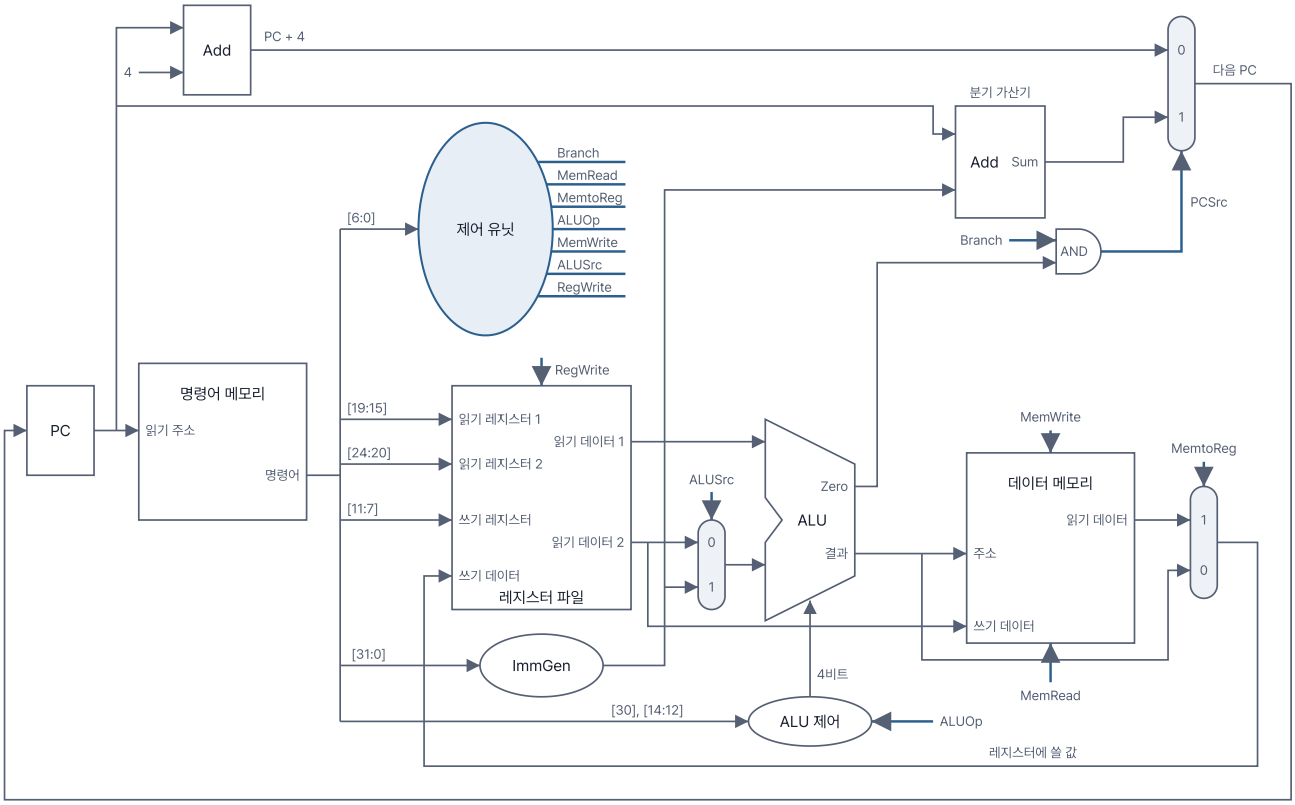
MUX 세 곳

MUX(선택 신호)	위치	0일 때	1일 때
ALUSrc	ALU 두 번째 입력 앞	레지스터 파일 읽기 데이터 2 (R형, beq)	ImmGen의 부호 확장 즉시값 (lw, sw)
MemtoReg	레지스터 파일 쓰기 데이터 앞	ALU 결과 (R형)	데이터 메모리 읽기 데이터 (lw)
PCSrc	PC 입력 앞	PC + 4	분기 목표 주소

[Ch4 p.16–17, p.24] 슬라이드 p.24 그림의 MUX 안 숫자는 PCSrc 위 0·아래 1, ALUSrc 위 0·아래 1, MemtoReg 위 1·아래 0이다. MemtoReg만 1이 위에 있으니 위치가 아니라 숫자로 읽는다.

전체 단일 사이클 데이터패스

R형, load/store, 분기를 모두 실행하는 단일 사이클 데이터패스다. 메모리가 명령어용과 데이터용 두 개다. [Ch4 p.17] 아래 그림은 슬라이드 p.17에 p.24의 제어 신호를 더하고, p.26–28처럼 제어 유닛과 AND 게이트를 넣은 형태다. p.17·p.24 그림에는 AND 게이트 없이 PCSrc 선만 있다. [Ch4 p.24, p.26–28]



단일 사이클 데이터패스(슬라이드 p.17·p.24 구성에 p.26–28의 제어 유닛과 AND 게이트를 더함). 제어 유닛 오른쪽의 강조 선 7개는 슬라이드 p.26–28과 같은 순서(Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc, RegWrite)이고, 선을 끝까지 그리면 겹쳐서 같은 이름의 짧은 강조 선으로 들어가는 곳을 표시했다. 분기용 1비트 시프트는 ImmGen이 함께 하므로 시프트 상자가 따로 없다(p.15). MUX 안 숫자는 슬라이드와 같다: PCSrc 위 0·아래 1, ALUSrc 위 0·아래 1, MemtoReg 위 1·아래 0. 선이 엇갈리기만 하는 곳은 연결이 아니고, T자로 갈라지는 곳만 연결이다. [ ] 안은 명령어의 비트 위치.

그림 읽는 법

1. **인출**: PC → 명령어 메모리 → 32비트 명령어. 동시에 위쪽 가산기가 PC + 4를 만든다.
2. **명령어 비트 나누기**: [6:0](opcode)은 제어 유닛으로(출력은 위에서부터 Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc, RegWrite), [19:15](rs1)·[24:20](rs2)는 읽기 레지스터 1·2로, [11:7](rd)은 쓰기 레지스터로, 명령어 전체는 ImmGen으로, [30]과 [14:12](funct7의 한 비트와 funct3)는 ALU 제어로 간다.
3. **ALU**: 첫 입력은 항상 읽기 데이터 1(rs1), 둘째 입력은 ALUSrc MUX가 고른다.
4. **데이터 메모리**: 주소 = ALU 결과, 쓰기 데이터 = 읽기 데이터 2(rs2, sw가 저장할 값).
5. **되돌림(write back)**: MemtoReg MUX가 ALU 결과나 메모리 값을 골라 레지스터 파일의 쓰기 데이터로 보낸다.
6. **다음 PC**: 분기 가산기가 PC + ImmGen 출력(ImmGen이 이미 1비트 시프트한 오프셋), 즉 PC + (즉시값 << 1)을 계산하고, Branch AND Zero가 PCSrc가 되어 PC + 4와 분기 목표 중 하나를 고른다.

ALU 제어에 funct7 전체가 아니라 [30] 한 비트만 가는 것은 이 부분집합에서 funct7 값 중 0000000(add)과 0100000(sub)을 가르는 비트가 30번뿐이기 때문이다. 교재 기준 PDF 외 보충

핵심

단일 사이클 = 모든 명령어를 한 클럭에. 한 명령어에서 자원은 한 번만 → 명령어 메모리와 데이터 메모리 분리. MUX 3개: ALUSrc(레지스터 / 즉시값), MemtoReg(ALU 결과 / 메모리), PCSrc(PC + 4 / 분기 목표).

예제

데이터 메모리와 명령어 메모리를 하나로 합친 단일 사이클 데이터패스를 만들면 어떤 명령어가 실행될 수 없는가? 이유는?

풀이 보기

`lw`와 `sw`. 이 둘은 같은 사이클 안에서 명령어를 인출하려고 메모리를 한 번 쓰고, 데이터를 읽거나 쓰려고 메모리를 또 한 번 써야 한다. 단일 사이클에서는 한 명령어가 자원을 두 번 쓸 수 없으므로 메모리가 두 개 필요하다. [Ch4 p.16]

함정

- "메모리가 두 개인 이유는 속도를 높이려고"는 핵심이 아니다. 슬라이드의 이유는 **한 사이클에 자원을 두 번 쓸 수 없어서**다.
- MemtoReg가 1이면 "메모리 → 레지스터", 즉 메모리 값을 고른다. 이름 그대로 읽으면 된다. ALUSrc가 1이면 즉시값이다(0이 레지스터).
- sw의 저장 값은 rs2(읽기 데이터 2)에서 바로 데이터 메모리로 간다. ALU를 거치지 않는다.

시험 포인트

- 단일 사이클 정의, 메모리를 분리하는 이유 [Ch4 p.16]
- MUX 세 곳과 각 입력 0·1이 무엇인지 [Ch4 p.16–17]
- 그림에서 부품 이름 채우기, 신호 흐름 순서 [Ch4 p.17]

스스로 확인

1. 단일 사이클 데이터패스에서 명령어 메모리와 데이터 메모리를 분리하는 이유는?

   답 보기

   모든 명령어를 한 사이클에 실행하므로 한 명령어가 같은 자원을 두 번 쓸 수 없다. lw·sw는 인출과 데이터 접근에 메모리를 두 번 쓴다.

2. ALU 두 번째 입력을 고르는 MUX의 이름과 두 입력은?

   답 보기

   ALUSrc. 0 = 레지스터 읽기 데이터 2, 1 = 부호 확장 즉시값.

3. 레지스터 파일 쓰기 데이터 앞 MUX의 이름과 두 입력은?

   답 보기

   MemtoReg. 0 = ALU 결과, 1 = 데이터 메모리 읽기 데이터.

4. PC 앞 MUX가 고르는 두 값은?

답 보기

PC + 4와 분기 목표 주소. 선택 신호는 PCSrc.

5. 읽기 레지스터 1, 읽기 레지스터 2, 쓰기 레지스터에 연결되는 명령어 비트 범위는?

답 보기

[19:15], [24:20], [11:7].

6. 데이터 메모리의 주소 입력과 쓰기 데이터 입력은 각각 어디서 오는가?

답 보기

주소 = ALU 결과, 쓰기 데이터 = 레지스터 파일 읽기 데이터 2.

Ch4 · 5 슬라이드 p.18–20

ALU 제어

이 단원을 마치면 ALU 제어 4비트 값과 연산을 짝짓고, ALUOp와 funct 필드로 ALU 제어 값을 만드는 진리표를 채우고, 다단 디코딩의 이점을 말할 수 있다.

쉽게 말하면

회사에서 사장(주 제어 유닛)은 "이건 주소 계산이야(더해)", "이건 비교야(빼)", "이건 R형이니 세부 사항은 팀장이 봐"까지만 정한다. 팀장(ALU 제어)은 그 지시와 명령어의 세부 필드(funct)를 보고 ALU에 정확한 연산을 지시한다. 일을 두 단계로 나누니 사장이 외울 것이 줄어든다.

비유가 안 맞는 지점 사장과 팀장은 순서대로 일하지만, 회로에서는 둘 다 같은 사이클 안에서 논리 게이트로 거의 동시에 계산된다.

ALU 제어 입력 4비트

RISC-V ALU는 4비트 제어 입력의 네 가지 조합을 쓴다. [Ch4 p.18]

ALU 제어 입력	연산
`0000`	AND
`0001`	OR
`0010`	add (덧셈)
`0110`	subtract (뺄셈)

add(`0010`)와 subtract(`0110`)는 한 비트만 다르다. 그 비트가 두 번째 피연산자를 부정(2의 보수)하라는 뜻이라, 뺄셈 = 부정해서 더하기(Ch3)가 그대로 하드웨어 신호로 나타난 것이다. PDF 외 보충

명령어 종류별 ALU 동작

명령어 종류에 따라 ALU는 위 네 가지 중 하나를 한다. [Ch4 p.18]

- **load/store**: 메모리 주소 계산 → **덧셈**.
- **분기(beq)**: 두 피연산자를 **빼서** 결과가 0인지 검사.
- **R형**: funct7과 funct3 필드 값에 따라 AND, OR, add, subtract 중 하나.

ALUOp와 다단 디코딩

4비트 ALU 제어 입력은 작은 제어 유닛(ALU 제어)이 만든다. 입력은 두 가지다. [Ch4 p.19]

- 명령어의 **funct7, funct3** 필드
- 주 제어 유닛이 만드는 2비트 제어 필드 **ALUOp**

ALUOp	뜻	쓰는 명령어
`00`	무조건 덧셈	lw, sw
`01`	무조건 뺄셈	beq
`10`	funct7·funct3를 보고 결정	R형

[Ch4 p.18–19] 교재 기준

이렇게 제어를 두 단계로 나누는 것을 **다단 디코딩(multi-level decoding)**이라 한다. [Ch4 p.19]

1. 주 제어 유닛이 ALUOp 비트를 만든다.
2. ALUOp를 입력으로 써서 ALU를 실제로 제어하는 신호를 만든다.

이점: **주 제어 유닛의 크기를 줄일 수 있다.** 또 작은 제어 유닛 여러 개를 쓰면 **제어 유닛의 지연도 줄 수 있다**(potentially). [Ch4 p.19] 주 제어 유닛은 opcode 7비트만 보면 되고, funct 10비트까지 볼 필요가 없어진다.

진리표와 don't care

ALU 제어의 입력은 ALUOp 2비트 + funct7 7비트 + funct3 3비트 = **12비트**다. 전체 진리표는 2^{12} = **4096**행이라 매우 크다. 그런데 실제로 나오는 조합은 몇 개뿐이고 나머지 입력 조합은 신경 쓰지 않아도 된다. [Ch4 p.20]

진리표(truth table) 입력의 모든 조합에 대해 출력을 적은 표.

don't care(X) 입력 열에 X가 있으면 출력이 그 열의 입력 값에 상관없다는 뜻. 표를 작게 만든다. [Ch4 p.20]

예: ALUOp가 00이면 funct 필드와 상관없이 ALU 제어를 `0010`으로 한다. [Ch4 p.20]

ALUOp1	ALUOp0	funct7	funct3	ALU 제어	해당
0	0	XXXXXXX	XXX	`0010`	lw, sw (add)
X	1	XXXXXXX	XXX	`0110`	beq (subtract)
1	X	0000000	000	`0010`	add
1	X	0100000	000	`0110`	sub
1	X	0000000	111	`0000`	and
1	X	0000000	110	`0001`	or

[Ch4 p.20]

ALUOp에도 X가 들어가는 이유: ALUOp = 11은 절대 나오지 않는다. 그래서 01을 알아보는 데는 ALUOp0 = 1만 보면 되고, 10을 알아보는 데는 ALUOp1 = 1만 보면 된다. 이렇게 표를 만든 뒤 최적화해서 논리 게이트로 바꾼다. [Ch4 p.20]

같은 내용을 명령어별로 정리하면 다음과 같다(감사 요약 "핵심 표 7"과 같다).

명령어	ALUOp	funct7	funct3	ALU 동작	ALU 제어
lw / sw	00	X	X	add (주소 계산)	`0010`
beq	01	X	X	subtract (같은지 검사)	`0110`
add	10	0000000	000	add	`0010`
sub	10	0100000	000	subtract	`0110`
and	10	0000000	111	AND	`0000`
or	10	0000000	110	OR	`0001`

핵심

ALU 제어: 0000 AND, 0001 OR, 0010 add, 0110 sub. lw·sw = 덧셈, beq = 뺄셈, R형 = funct7·funct3로 결정. ALUOp 00 / 01 / 10. 다단 디코딩: 주 제어 → ALUOp → ALU 제어, 주 제어 유닛이 작아지고 지연도 줄 수 있음. 입력 12비트 → 2^{12} = 4096 조합, don't care로 압축.

예제

명령어 `0100000 00111 00110 000 00101 0110011`이 실행될 때 ALUOp와 ALU 제어 입력은?

풀이 보기

---

opcode `0110011` = R형 → ALUOp = `10`.

funct7 = `0100000`, funct3 = `000` → sub → ALU 제어 = `0110`.

참고로 rs2 = 7, rs1 = 6, rd = 5이므로 `sub x5, x6, x7`이다.

함정

- ALUOp(2비트)와 ALU 제어(4비트)를 혼동하지 말 것. ALUOp는 주 제어 유닛의 출력, ALU 제어는 ALU 제어 유닛의 출력이다.
- beq는 R형이 아니지만 ALU가 뺄셈을 한다. funct3를 보는 것이 아니라 ALUOp = 01로 바로 뺄셈이 정해진다.
- 4096은 2^4가 아니라 입력 12비트(2 + 7 + 3)의 2^{12}이다.

시험 포인트

- ALU 제어 4비트 값과 연산 짝짓기 [Ch4 p.18]
- 명령어 종류별 ALU 동작 [Ch4 p.18]
- ALU 제어의 입력(ALUOp, funct7, funct3), 다단 디코딩 단계와 이점 [Ch4 p.19]
- 진리표 빈칸 채우기, 4096의 근거, don't care의 뜻 [Ch4 p.20]

스스로 확인

1. ALU 제어 입력 `0110`은 어떤 연산인가?

   답 보기

   subtract(뺄셈).

2. lw, beq, and 명령어의 ALUOp를 각각 쓰시오.

   답 보기

   lw = 00, beq = 01, and = 10(R형).

3. ALU 제어 유닛의 입력 세 가지는?

   답 보기

   ALUOp(2비트), funct7, funct3.

4. 다단 디코딩의 이점 두 가지는?

   답 보기

   주 제어 유닛의 크기 감소, 제어 유닛 지연 감소 가능성.

5. ALU 제어 진리표의 전체 입력 조합 수와 그 근거는?

   답 보기

   2^{12} = 4096. 입력이 ALUOp 2 + funct7 7 + funct3 3 = 12비트.

6. 진리표 입력 열의 X는 무슨 뜻인가?

   답 보기

   don't care. 출력이 그 열의 입력 값에 의존하지 않는다.

7. ALUOp = 10, funct7 = 0000000, funct3 = 110일 때 ALU 제어 값과 명령어는?

   답 보기

   `0001`, or.

Ch4 · 6 슬라이드 p.21–25

주 제어 유닛과 제어 신호

이 단원을 마치면 명령어 필드의 위치와 역할을 말하고, 제어 신호 각각이 0·1일 때 하는 일을 쓰고, 명령어별 제어 신호 값 표를 채우고, PCSrc가 왜 특별한지 설명할 수 있다.

쉽게 말하면

제어 유닛은 신호등 조작판이다. 명령어의 opcode를 보고 "이 MUX는 0, 레지스터 쓰기는 켜고, 메모리 쓰기는 끄고..." 하는 식으로 스위치들을 한 번에 정한다. 명령어 종류가 넷(R형, lw, sw, beq)이니 스위치 조합표도 네 줄이다.

비유가 안 맞는 지점 스위치 하나(PCSrc)는 조작판만으로 정할 수 없다. 계산 결과(ALU의 Zero)를 봐야 하기 때문이다.

명령어 필드 관찰

주 제어 유닛을 설계하려면 명령어의 필드와 제어 선을 확인해야 한다. 형식에서 관찰되는 사실은 다음과 같다. [Ch4 p.21]

funct7	rs2	rs1	funct3	rd	opcode
31–25	24–20	19–15	14–12	11–7	6–0

- **opcode**는 항상 [6:0]. opcode에 따라 funct3 [14:12]와 funct7 [31:25]가 **확장 opcode** 역할을 한다.
- **rs1** [19:15]: R형·분기의 첫 번째 레지스터 피연산자, load/store의 베이스 레지스터.
- **rs2** [24:20]: R형·분기의 두 번째 레지스터 피연산자, store에서 메모리로 복사될 레지스터.
- **rd** [11:7]: R형과 load의 목적지 레지스터.
- 다른 피연산자로 분기나 load/store의 **12비트 오프셋**이 올 수 있다.

[Ch4 p.21]

rs1·rs2·rd 위치가 그 필드를 가진 모든 형식에서 같으므로, 데이터패스는 해독이 끝나기 전에 [19:15]와 [24:20]을 레지스터 파일에 바로 연결해 읽기를 시작할 수 있다. (Ch2의 설계 원칙이 여기서 하드웨어를 단순하게 만든다.)

슬라이드 p.21 그림의 형식 예는 R형(add, sub), I형(addi, lw), S형(sw), SB형(beq)이다. [Ch4 p.21] 각 opcode 값은 다음과 같다(R형 `0110011` 은 슬라이드 p.18 표에도 있다). 교재 기준 PDF 외 보충

형식	명령어	opcode
R	add, sub, and, or	`0110011`
I	lw	`0000011`
S	sw	`0100011`
SB	beq	`1100011`

ImmGen 하드웨어 단순화: 즉시값 비트 뒤섞기

RISC-V 형식은 겉보기에 복잡하지만 실제로는 하드웨어를 단순하게 만든다. 일부 구현에서는 클럭 주기를 줄이기도 한다. 특히 SB와 UJ 형식은 **어셈블러에 일을 더 시키는 대신** 하드웨어를 단순하게 한다. [Ch4 p.22]

RISC-V에는 필드 크기가 모두 같은데 즉시값 비트 배치만 다른 형식 쌍이 둘 있다. **SB와 S**, **UJ와 U**. SB·UJ의 즉시값 비트는 뒤섞여(swirled) 있다. [Ch4 p.22]

형식	명령어 [31:25]	[11:7]
S (sw)	imm[11:5]	imm[4:0]
SB (beq)	imm[12 | 10:5]	imm[4:1 | 11]

왜 뒤섞을까? ImmGen은 즉시값의 각 비트 자리마다 "명령어의 몇 번 비트를 가져올지"를 형식별로 고르는 MUX를 둔다. 형식마다 같은 즉시값 비트가 명령어의 다른 자리에서 오면 그 MUX의 입력 수가 늘어난다. SB의 imm[10:5]와 imm[4:1]은 S의 같은 비트와 **명령어 안 위치가 같다**. UJ의 imm[19:12]도 U와 같은 위치([19:12])에 있다. 이렇게 맞추면 MUX가 작아진다. [Ch4 p.23]

- 즉시값 비트 19~12번의 MUX: **3:1 → 2:1**
- 즉시값 비트 10~1번의 MUX: **5:1·4:1 → 3:1·2:1**
- 이 하드웨어 효율의 유일한 대가는 **어셈블러**가 진다(비트를 뒤섞어 기계어로 만드는 일). [Ch4 p.23]

부호 비트(imm의 맨 위 비트)는 모든 형식에서 명령어의 31번 비트다. 그래서 부호 확장도 항상 31번 비트를 복사하면 된다. PDF 외 보충

제어 신호 각각의 의미

p.24는 데이터패스에 명령어 비트 표시와 필요한 MUX·제어 신호를 모두 더한 그림이다(c4-4 그림과 같은 구성). [Ch4 p.24]

신호	0(비활성)일 때	1(활성)일 때
RegWrite	아무 일 없음	쓰기 레지스터(rd)에 쓰기 데이터 입력 값이 기록됨
ALUSrc	ALU 둘째 입력 = 레지스터 파일 읽기 데이터 2	ALU 둘째 입력 = 명령어 12비트를 부호 확장한 즉시값
PCSrc	PC ← PC + 4	PC ← 분기 목표 주소
MemRead	아무 일 없음	주소 입력이 가리키는 데이터 메모리 내용이 읽기 데이터로 나옴
MemWrite	아무 일 없음	주소 입력이 가리키는 데이터 메모리 내용이 쓰기 데이터 값으로 바뀜
MemtoReg	레지스터 쓰기 데이터 = ALU 결과	레지스터 쓰기 데이터 = 데이터 메모리 읽기 데이터
Branch	분기 명령어 아님	분기 명령어(beq). Zero와 AND해서 PCSrc를 만듦

여기에 2비트 **ALUOp**(00 덧셈, 01 뺄셈, 10 funct로 결정)가 더해진다. 교재 기준 주 제어 유닛이 직접 내보내는 것은 Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc, RegWrite의 7개이고(슬라이드 p.25–28 그림의 위에서부터 순서), PCSrc는 그 바깥의 AND 게이트가 만든다(p.26–28 그림). [Ch4 p.25–28] 교재는 1비트 신호 6개(RegWrite, ALUSrc, PCSrc, MemRead, MemWrite, MemtoReg) + ALUOp로 세기도 하니, 문제에서 무엇을 세는지 확인할 것.

명령어별 제어 신호 값

명령어	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
R형	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

[Ch4 p.25] 같은 값이 p.26–28 동작 그림에도 쓰인다. [Ch4 p.26–28]

외우는 요령: 각 열을 "이 명령어에 그 기능이 필요한가"로 읽는다.

- ALUSrc = 1: 즉시값으로 주소를 계산하는 lw, sw.
- RegWrite = 1: 레지스터에 결과를 쓰는 R형, lw.
- MemtoReg = 1: 메모리 값을 레지스터에 쓰는 lw만.
- MemRead = 1: lw만, MemWrite = 1: sw만, Branch = 1: beq만.

sw와 beq의 MemtoReg가 X인 이유: 두 명령어는 RegWrite = 0이라 레지스터에 아무것도 쓰지 않는다. MemtoReg MUX가 무엇을 골라도 쓰이지 않으므로 상관없다(don't care). 교재 기준

반면 RegWrite, MemWrite, MemRead, Branch는 X로 둘 수 없다. 잘못 1이 되면 레지스터나 메모리 값이 망가지거나 엉뚱한 곳으로 분기하기 때문이다. PDF 외 보충

PCSrc: 유일한 예외

제어 유닛은 신호 하나를 빼고 모두 명령어의 opcode와 funct 필드**만으로** 정할 수 있다. 그 예외가 **PCSrc**다. PCSrc는 **명령어가 beq이고 ALU의 Zero 출력이 1일 때** 활성화된다. [Ch4 p.25]

```
PCSrc = Branch AND Zero
```

Branch는 opcode로 정해지지만 Zero는 실행 중 두 레지스터를 뺀 결과로 정해진다. 그래서 opcode만으로는 PCSrc를 알 수 없다. [Ch4 p.25] 교재 기준

핵심

opcode [6:0], funct3 [14:12], funct7 [31:25], rs1 [19:15], rs2 [24:20], rd [11:7]. 신호 표: R형 0 0 1 0 0 0 10 / lw 1 1 1 1 0 0 00 / sw 1 X 0 0 1 0 00 / beq 0 X 0 0 0 1 01 (ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, ALUOp 순). PCSrc만 opcode로 못 정함: beq이고 Zero = 1 (Branch AND Zero). SB·UJ 비트 뒤섞기 → ImmGen MUX 축소(19~12: 3:1 → 2:1, 10~1: 5/4:1 → 3/2:1), 대가는 어셈블러.

예제

sw의 제어 신호에서 실수로 RegWrite = 1이 되면 무슨 일이 생기는가? MemtoReg = X는 그대로라고 하자. PDF 외 보충

풀이 보기

sw는 S형이라 [11:7]이 rd가 아니라 imm[4:0]이다. 그런데 쓰기 레지스터 입력에는 [11:7]이 그대로 연결되어 있으므로, 오프셋의 아래 5비트를 번호로 하는 레지스터에 MemtoReg MUX가 고른 값(ALU 결과 또는 메모리 값)이 쓰여 값이 망가진다. 그래서 sw의 RegWrite는 반드시 0이어야 한다.

함정

- "PCSrc도 opcode만으로 정해진다"는 틀리다. beq라도 Zero가 0이면 PCSrc = 0이다.
- beq의 ALUSrc는 0이다. beq에도 즉시값(오프셋)이 있지만 그것은 분기 가산기로 가고, ALU는 두 레지스터를 뺀다.
- lw의 MemtoReg는 1, R형은 0. 반대로 외우기 쉬우니 "Mem to Reg = 메모리 값을 레지스터로"를 떠올릴 것.
- X(don't care)는 0과 같은 뜻이 아니다. 0이든 1이든 결과가 같다는 뜻이다.

시험 포인트

- 필드 위치와 rs1·rs2·rd 역할 [Ch4 p.21]
- SB·UJ 비트 뒤섞기의 이유와 대가, MUX 크기 변화 [Ch4 p.22–23]
- 명령어별 제어 신호 값 표 채우기, X의 이유 [Ch4 p.24–28]
- PCSrc의 조건 [Ch4 p.25]

스스로 확인

1. funct3와 funct7의 비트 위치는?

   답 보기

   funct3 = [14:12], funct7 = [31:25].

2. store 명령어에서 rs2 필드의 역할은?

   답 보기

   메모리로 복사될(저장할) 값을 담은 레지스터.

3. lw의 ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, ALUOp를 쓰시오.

   답 보기

   1, 1, 1, 1, 0, 0, 00.

4. beq의 MemtoReg가 X인 이유는?

답 보기

RegWrite = 0이라 레지스터에 쓰지 않으므로 MemtoReg MUX의 선택이 결과에 영향이 없다.

5. opcode와 funct만으로 정할 수 없는 제어 신호와 그 조건은?

답 보기

PCSrc. beq이고 ALU Zero = 1일 때 1 (Branch AND Zero).

6. SB·UJ 형식의 즉시값 비트 뒤섞기로 얻는 것과 그 대가를 지는 쪽은?

답 보기

ImmGen의 MUX가 작아져 하드웨어가 단순해진다(19~12번 3:1 → 2:1). 대가는 어셈블러가 진다.

7. ALUSrc가 1인 명령어를 이 부분집합에서 모두 쓰시오.

답 보기

lw, sw.

8. MemWrite = 1일 때 일어나는 일은?

답 보기

주소 입력이 가리키는 데이터 메모리 위치가 쓰기 데이터 입력 값으로 바뀐다.

Ch4 · 7 슬라이드 p.26–28

명령어별로 따라가기

이 단원을 마치면 add, lw, beq(그리고 sw)가 데이터패스의 어느 부품을 어떤 순서로 지나는지, 그때 제어 신호가 무엇인지 서술할 수 있다.

쉽게 말하면

같은 도로망(데이터패스)에서 차(데이터)가 명령어마다 다른 길로 간다. 제어 신호는 갈림길(MUX)마다 표지판을 바꾸고, 문(쓰기 허가)을 열고 닫는다. 명령어 하나를 골라 그 차가 지나는 길을 손가락으로 따라가 보는 것이 이 단원이다.

비유가 안 맞는 지점 실제로는 데이터가 모든 길로 동시에 흘러간다. 안 쓰는 부품도 무언가를 계산하고 있다. 다만 MUX가 고르지 않거나 쓰기 신호가 0이라 결과에 반영되지 않을 뿐이다.

아래 단계 구분은 교재가 쓰는 방식이다. 단일 사이클이므로 이 단계들은 모두 **한 클럭 사이클 안에서** 일어나고, 단계는 신호가 흐르는 순서를 설명하려고 나눈 것이다. 교재 기준

R형: add x1, x2, x3

1. 명령어 인출, 동시에 PC + 4 계산. [Ch4 p.26]
2. 레지스터 파일에서 `x2`(rs1 = 2)와 `x3`(rs2 = 3)를 읽는다. 동시에 주 제어 유닛이 opcode로 제어 신호를 만든다.
3. ALU가 두 값을 더한다(`x2 + x3`). ALU 제어는 funct7·funct3로 add(`0010`)를 고른다.
4. ALU 결과가 MemtoReg MUX(0)를 지나 `x1`(rd = 1)에 쓰인다.

슬라이드 p.26 그림의 숫자 2, 3, 1이 각각 읽기 레지스터 1, 읽기 레지스터 2, 쓰기 레지스터에 들어가는 번호이고, PC + 4가 다음 PC가 된다. [Ch4 p.26]

신호: ALUSrc = 0, MemtoReg = 0, RegWrite = 1, MemRead = 0, MemWrite = 0, Branch = 0, ALUOp = 10 → PCSrc = 0.

안 쓰이는 부품: 데이터 메모리, 분기 가산기의 결과(PCSrc가 고르지 않음), ImmGen 출력(ALUSrc가 고르지 않음).

load: lw x1, offset(x2)

1. 명령어 인출, PC + 4 계산. [Ch4 p.27]
2. 레지스터 파일에서 `x2`(rs1, 베이스)를 읽는다. ImmGen이 오프셋 12비트를 부호 확장한다.
3. ALU가 `x2 + offset`을 계산한다(ALUSrc = 1로 즉시값 선택, ALUOp = 00으로 덧셈).
4. 그 합을 주소로 데이터 메모리를 읽는다(MemRead = 1): `Mem[x2+offset]`.
5. 메모리 값이 MemtoReg MUX(1)를 지나 `x1`(rd)에 쓰인다.

[Ch4 p.27]

신호: ALUSrc = 1, MemtoReg = 1, RegWrite = 1, MemRead = 1, MemWrite = 0, Branch = 0, ALUOp = 00 → PCSrc = 0.

lw는 I형이라 [24:20] 자리가 rs2가 아니라 오프셋의 일부다. 레지스터 파일은 그 비트를 번호로 삼아 어떤 레지스터를 읽긴 하지만, ALUSrc = 1이라 ALU에는 쓰이지 않고 데이터 메모리에도 쓰지 않으므로(MemWrite = 0) 결과에 영향이 없다. PDF 외 보충

lw는 명령어 메모리, 레지스터 파일, ALU, 데이터 메모리, 레지스터 파일(쓰기)을 모두 차례로 지나므로 가장 긴 경로다. 이것이 c4-8의 출발점이다.

분기: beq x1, x2, offset

1. 명령어 인출, PC + 4 계산. [Ch4 p.28]
2. 레지스터 파일에서 `x1`(rs1)과 `x2`(rs2)를 읽는다.
3. ALU가 `x1 - x2`를 계산한다(ALUSrc = 0, ALUOp = 01). 동시에 분기 가산기가 PC + (offset << 1)을 계산한다.
4. ALU의 Zero로 다음 PC를 정한다. 같으면(Zero = 1) PCSrc = 1 → 분기 목표, 다르면 PC + 4.

슬라이드 p.28 그림은 목표를 "PC+offset"으로 적었다. 여기서 offset은 1비트 시프트까지 마친 바이트 단위 값으로 읽으면 된다. [Ch4 p.28]

신호: ALUSrc = 0, MemtoReg = X, RegWrite = 0, MemRead = 0, MemWrite = 0, Branch = 1, ALUOp = 01 → PCSrc = Zero.

안 쓰이는 부품: 데이터 메모리, 레지스터 쓰기(RegWrite = 0). ImmGen 출력은 ALU로는 안 가고(ALUSrc = 0), ImmGen이 부호 확장과 함께 분기용 1비트 왼쪽 시프트까지 해서(슬라이드 p.15) 분기 가산기로 보낸다.

store: sw x1, offset(x2)

PDF 외 보충 슬라이드에는 그림이 없지만 신호 표에 있으므로 같은 방식으로 따라가 본다.

1. 명령어 인출, PC + 4 계산.
2. 레지스터 파일에서 `x2`(rs1, 베이스)와 `x1`(rs2, 저장할 값)을 읽는다. ImmGen이 S형의 나뉜 오프셋을 모아 부호 확장한다.
3. ALU가 `x2 + offset`을 계산한다.
4. 그 주소에 `x1`의 값(읽기 데이터 2)을 쓴다(MemWrite = 1).

신호: ALUSrc = 1, MemtoReg = X, RegWrite = 0, MemRead = 0, MemWrite = 1, Branch = 0, ALUOp = 00 → PCSrc = 0.

주의: `sw x1, offset(x2)`에서 `x1`은 목적지가 아니라 **rs2**다. 레지스터 파일에 쓰는 것이 없다.

한눈에 비교

명령어	읽는 레지스터	ALU 입력 2	ALU 연산	데이터 메모리	레지스터 쓰기	다음 PC
add	rs1, rs2	rs2	add	안 씀	ALU 결과 → rd	PC + 4
lw	rs1	즉시값	add	읽기	메모리 값 → rd	PC + 4
sw	rs1, rs2	즉시값	add	쓰기(rs2 값)	안 씀	PC + 4
beq	rs1, rs2	rs2	subtract	안 씀	안 씀	Zero면 분기 목표, 아니면 PC + 4

핵심

add: 인출 → rs1·rs2 읽기 → ALU 덧셈 → rd에 ALU 결과. lw: 인출 → rs1 읽기 → rs1 + 오프셋 → 메모리 읽기 → rd에 메모리 값.
beq: 인출 → rs1·rs2 읽기 → ALU 뺄셈(+ 분기 가산기로 목표 계산) → Zero면 목표, 아니면 PC + 4. 모두 한 사이클 안.

예제

`x1 = 7`, `x2 = 7`이고 주소 0x100의 `beq x1, x2, offset`의 오프셋 필드가 20이다. ALU 결과, Zero, PCSrc, 다음 PC는?

풀이 보기

ALU 결과 = 7 − 7 = 0 → Zero = 1. Branch = 1이므로 PCSrc = 1 AND 1 = 1.

다음 PC = 0x100 + (20 << 1) = 0x100 + 40 = 0x100 + 0x28 = **0x128**.

함정

- "lw는 ALU를 안 쓴다"는 틀리다. 주소 계산에 덧셈으로 쓴다.
- beq에서 분기 목표 계산은 ALU가 아니라 분기 가산기가 한다. ALU는 뺄셈으로 비교만 한다.
- 단계가 여러 개라고 해서 여러 사이클이 걸리는 것이 아니다. 단일 사이클에서는 전부 한 사이클이다.
- sw의 x1은 쓰는 레지스터가 아니라 읽는 레지스터(rs2)다.

시험 포인트

- add, lw, beq의 단계별 경로 서술 [Ch4 p.26–28]
- 그 경로에서 각 제어 신호 값 [Ch4 p.24–28]
- 어떤 부품이 쓰이고 안 쓰이는지

스스로 확인

1. `add x1, x2, x3` 실행의 네 단계를 순서대로 쓰시오.

   답 보기

   ① 명령어 인출(PC + 4 계산) ② x2·x3 읽기 ③ ALU 덧셈 ④ 결과를 x1에 쓰기.

2. `lw x1, offset(x2)`에서 데이터 메모리의 주소 입력 값은?

   답 보기

   x2 + 부호 확장한 offset (ALU 결과).

3. R형 명령어 실행 때 결과에 쓰이지 않는 부품을 하나 쓰시오.

   답 보기

   데이터 메모리 (또는 분기 가산기 결과, ImmGen 출력).

4. `beq`에서 분기하지 않는 경우 다음 PC는?

   답 보기

   PC + 4. Zero = 0이라 PCSrc = 0.

5. `sw x5, 8(x6)`에서 rs1, rs2는 각각 어느 레지스터인가?

   답 보기

   rs1 = x6(베이스), rs2 = x5(저장할 값).

6. O/X: lw의 다섯 단계는 다섯 클럭 사이클에 걸쳐 실행된다(단일 사이클 구현).

   답 보기

   X. 단일 사이클 구현이므로 모두 한 클럭 사이클 안에서 일어난다.

단일 사이클의 한계

이 단원을 마치면 단일 사이클 구현이 오늘날 쓰이지 않는 이유를 서술하고, 부품 지연이 주어지면 클럭 주기를 계산할 수 있다.

쉽게 말하면

모든 손님에게 똑같이 1시간을 배정하는 미용실이 있다. 머리 감기만 하는 손님도, 파마하는 손님도 1시간이다. 그 1시간은 가장 오래 걸리는 파마에 맞춘 것이다. 대부분의 손님이 금방 끝나는 일인데도 모두 1시간을 기다려야 하니 비효율적이다.

비유가 안 맞는 지점 미용실은 손님마다 시간을 다르게 잡을 수 있지만, 클럭은 일정한 간격으로 뛰므로 명령어마다 주기를 바꾸는 것은 현실적으로 불가능하다(슬라이드의 "not feasible").

왜 단일 사이클을 쓰지 않는가

단일 사이클 설계는 올바르게 동작하지만, 현대 설계에 쓰기에는 너무 비효율적이다. [Ch4 p.29]

- 설계 안의 **모든 명령어가 같은 길이의 클럭 사이클**을 가져야 한다.
- 프로세서 안의 **가장 긴 경로가 클럭 주기를 정한다**(c4-2의 규칙).
- 그 경로는 거의 확실히 **load 명령어**다. 다섯 개의 장치를 **직렬로** 쓴다: **명령어 메모리 → 레지스터 파일 → ALU → 데이터 메모리 → 레지스터 파일**.
- 명령어마다 클럭 주기를 다르게 하는 것은 실현 불가능하다(not feasible).
- 이는 설계 원칙 **"자주 생기는 경우를 빠르게(Make the common case fast)"**를 어긴다. 짧은 명령어(자주 쓰임)도 가장 느린 명령어의 시간을 기다려야 하기 때문이다.

[Ch4 p.29]

Ch1의 공식으로 보면, 단일 사이클은 CPI = 1이지만 클럭 주기가 가장 긴 경로만큼 길다.

```
단일 사이클 CPU 시간 = IC × 1 × (가장 긴 경로 지연)
```

주기 계산 예

PDF 외 보충 부품 지연이 다음과 같다고 하자(교재 파이프라인 절에서 쓰는 예시 값과 같다). MUX, 가산기, 레지스터 자체 지연 등 나머지는 0으로 본다.

부품	지연
명령어 메모리(인출)	200 ps
레지스터 파일 읽기	100 ps
ALU	200 ps
데이터 메모리	200 ps
레지스터 파일 쓰기	100 ps

명령어	인출	레지스터 읽기	ALU	데이터 메모리	레지스터 쓰기	합계
lw	200	100	200	200	100	**800 ps**
sw	200	100	200	200		700 ps
R형(add)	200	100	200		100	600 ps
beq	200	100	200			500 ps

단일 사이클의 클럭 주기는 가장 긴 lw에 맞춰 **800 ps**다. beq는 500 ps면 끝나는데도 800 ps를 쓴다.

예제

PDF 외 보충 위 지연에서 프로그램의 명령어 비율이 lw 25%, sw 10%, R형 45%, beq 20%라고 하자. (가상으로) 명령어마다 필요한 만큼만 시간을 쓸 수 있다면 명령어당 평균 시간은? 단일 사이클 대비 몇 배 빠른가?

풀이 보기

평균 = 0.25 × 800 + 0.10 × 700 + 0.45 × 600 + 0.20 × 500 = 200 + 70 + 270 + 100 = **640 ps**.

단일 사이클은 모든 명령어가 800 ps이므로 800 / 640 = **1.25배**.

하지만 명령어마다 주기를 바꾸는 것은 실현 불가능하다는 것이 슬라이드의 결론이다. 그래서 다른 방법이 필요하다.

해결책 예고

슬라이드는 성능을 **파이프라이닝(pipelining)**으로 개선하자고 하며 마친다. [Ch4 p.29] 파이프라이닝 자체는 시험 범위 밖이다.

핵심

단일 사이클은 동작은 맞지만 비효율적. 모든 명령어의 클럭 주기가 같아야 하고, 가장 긴 경로(lw: 명령어 메모리 → 레지스터 파일 → ALU → 데이터 메모리 → 레지스터 파일, 5개 직렬)가 주기를 정함. 명령어마다 주기를 바꿀 수 없음 → "자주 생기는 경우를 빠르게" 위반 → 파이프라이닝으로 개선.

함정

- "단일 사이클은 CPI가 1이니 가장 빠르다"는 틀리다. CPI는 최소지만 클럭 주기가 가장 긴 명령어에 맞춰져 길어진다.
- 가장 긴 경로는 sw가 아니라 lw다. sw는 레지스터 쓰기 단계가 없다.
- 위반하는 원칙은 "자주 생기는 경우를 빠르게"다. 다른 8대 아이디어와 헷갈리지 말 것.

시험 포인트

- 단일 사이클을 오늘날 쓰지 않는 이유 서술 [Ch4 p.29]
- 가장 긴 경로의 명령어와 다섯 장치의 순서 [Ch4 p.29]
- 위반하는 설계 원칙 이름 [Ch4 p.29]
- 지연이 주어지면 단일 사이클 클럭 주기 계산

스스로 확인

1. 단일 사이클 구현에서 클럭 주기를 정하는 것은?

   답 보기

   프로세서 안의 가장 긴 경로(지연).

2. 가장 긴 경로를 갖는 명령어와 그 경로의 다섯 장치를 순서대로 쓰시오.

   답 보기

   load(lw). 명령어 메모리 → 레지스터 파일 → ALU → 데이터 메모리 → 레지스터 파일.

3. 단일 사이클 구현이 위반하는 설계 원칙은?

   답 보기

   자주 생기는 경우를 빠르게(Make the common case fast).

4. 명령어 메모리 250 ps, 레지스터 파일 읽기·쓰기 각 150 ps, ALU 200 ps, 데이터 메모리 300 ps일 때 단일 사이클의 클럭 주기는? (나머지 지연 무시)

   답 보기

   lw 경로 = 250 + 150 + 200 + 300 + 150 = **1050 ps**.

5. 같은 조건에서 R형 명령어에 실제로 필요한 시간은?

   답 보기

   250 + 150 + 200 + 150 = 750 ps. 그래도 단일 사이클에서는 1050 ps를 쓴다.

6. 슬라이드가 제시하는 성능 개선 방법은?

답 보기

파이프라이닝.

실전 · 1 관련 단원 Ch1 · 7~10

성능 계산: CPU 시간·MIPS·개선율

21-2 시험에는 "컴퓨터 성능을 결정하는 3가지"와 "CPI와 값을 주고 MIPS 계산"이, 26-1 시험에는 "실행 명령어들과 시간을 주고 일부가 개선될 때 개선율 계산"과 "단위가 헷갈리는 MIPS 계산"이 나왔다. 이 단원은 그 계산을 단위 환산부터 끝까지 손으로 푸는 연습이다.

푸는 순서

1. 주어진 값을 모두 **초(s)**와 **Hz**로 바꿔 적는다. GHz = 10^9, MHz = 10^6, ns = 10^{-9} 초, ps = 10^{-12} 초. 클럭 주기만 주면 클럭 속도 = 1 / 주기로 바꾼다.
2. 명령어 종류가 여러 개면 클럭 사이클 수 = $\Sigma(\text{CPI}_i \times \text{개수}_i)$를 먼저 구하고, 평균 CPI = 사이클 수 / 전체 명령어 수.
3. CPU 시간 = 명령어 수 × CPI / 클럭 속도.
4. MIPS = 클럭 속도 / (CPI × 10^6) 또는 명령어 수 / (실행 시간 × 10^6). 두 식으로 한 번씩 구해 같은지 검산한다.
5. "몇 배 빠른가"는 느린 쪽 시간 / 빠른 쪽 시간. 일부 개선은 암달: $T_{\text{개선 후}} = T_{\text{영향}} / n + T_{\text{비영향}}$, 속도 향상 = $T_{\text{전}} / T_{\text{후}}$.

쓰는 공식 모음

CPU 시간 = 명령어 수 × CPI × 클럭 주기 = 명령어 수 × CPI / 클럭 속도

클럭 사이클 수 = $\Sigma\ (\text{CPI}_i \times C_i)$, 평균 CPI = 클럭 사이클 수 / 명령어 수

MIPS = 명령어 수 / (실행 시간 × 10^6) = 클럭 속도 / (CPI × 10^6)

$T_{\text{개선 후}}$ = $T_{\text{영향 받는 부분}}$ / 개선 배수 + $T_{\text{영향 받지 않는 부분}}$

속도 향상(배) = $T_{\text{개선 전}}$ / $T_{\text{개선 후}}$

[Ch1 p.24, 28, 29, 33, 34]

시간 단위	초로 바꾸면	짝이 되는 주파수	Hz로 바꾸면
1 ms	10^{-3} 초	1 kHz	10^3 Hz
1 μs	10^{-6} 초	1 MHz	10^6 Hz
1 ns	10^{-9} 초	1 GHz	10^9 Hz
1 ps	10^{-12} 초	1 THz	10^{12} Hz

자주 나오는 짝: 250 ps ↔ 4 GHz, 500 ps = 0.5 ns ↔ 2 GHz, 1.25 ns ↔ 800 MHz, 2 ns ↔ 500 MHz. MIPS의 M은 백만(10^6)이다. 명령어 수가 "6억 개"로 주어지면 6 × 10^8으로 바꿔 쓴다.

완전 풀이 예제

클럭 속도 2 GHz인 컴퓨터에서 어떤 프로그램이 다음 명령어를 실행한다. (1) 평균 CPI (2) CPU 시간 (3) MIPS를 구하시오.

명령어 종류	CPI	실행 개수
A (산술)	1	6×10^6
B (분기)	2	3×10^6
C (load/store)	4	1×10^6

1단계 (단위) 클럭 속도 = 2 GHz = 2×10^9 Hz. 명령어 수 = 6 + 3 + 1 = $10 \times 10^6 = 10^7$개.

2단계 (사이클 수) $6 \times 10^6 \times 1 + 3 \times 10^6 \times 2 + 1 \times 10^6 \times 4 = (6 + 6 + 4) \times 10^6 = 16 \times 10^6$ 사이클.

3단계 (평균 CPI) $16 \times 10^6 / 10^7 =$ **1.6**. 가중 평균이므로 (1 + 2 + 4) / 3 ≈ 2.33처럼 단순 평균을 내면 틀린다.

4단계 (CPU 시간) $16 \times 10^6 / (2 \times 10^9) = 8 \times 10^{-3}$ 초 = **8 ms**.

5단계 (MIPS, 식 1) 클럭 속도 / (CPI × 10^6) = $2 \times 10^9 / (1.6 \times 10^6) =$ **1250**.

6단계 (MIPS, 식 2로 검산) 명령어 수 / (실행 시간 × 10^6) = $10^7 / (8 \times 10^{-3} \times 10^6) = 10^7 / 8000 = 1250$. 두 식이 같으므로 맞다.

함정

- MIPS 식 분모의 10^6을 빠뜨리면 답이 백만 배 커진다. 반대로 클럭을 MHz 숫자 그대로(500 MHz → 500) 넣으면서 10^6까지 나누면 백만 배 작아진다. 둘 중 한 가지로 일관되게 쓴다.
- 클럭 **주기**(ns, ps)가 주어지면 곱하고, 클럭 **속도**(GHz, MHz)가 주어지면 나눈다.
- 평균 CPI는 개수(또는 비율)로 가중한 평균이다. 단순 평균이 아니다.
- 암달에서 개선 배수 n은 영향 받는 부분에만 나눈다. 전체 시간을 n으로 나누지 않는다.
- "개선율"은 문제가 어떤 형태를 원하는지 확인한다. 속도 향상(배) = $T_{전} / T_{후}$, 성능 향상률 = (속도 향상 − 1) × 100%, 실행 시간 감소율 = $(1 - T_{후} / T_{전}) \times 100\%$. 예: 100초 → 80초면 1.25배, 성능 25% 향상, 시간 20% 감소. 지정이 없으면 배수로 쓰고 나머지를 괄호로 덧붙이면 안전하다. PDF 외 보충
- "MIPS가 높으면 항상 빠르다"는 X. ISA 차이, 명령어 복잡도 차이를 반영하지 못하고 프로그램마다 CPI가 다르다. [Ch1 p.34]

시험 포인트

- 성능 3요소: 명령어 수, CPI, 클럭 주기(클럭 속도)
- MIPS 두 식, 단위 환산(GHz·MHz·ns·ps, 10^6)
- 명령어 종류별 CPI 가중 평균, CPU 시간
- 실행 시간 표 + 일부 개선 → 암달로 개선 후 시간과 속도 향상, 불가능 판정

연습 문제

1. 21-2 기출 유형 프로그램의 CPU 실행 시간(성능)을 결정하는 3가지 요소를 쓰고, 각 요소를 주로 결정하는 것을 쓰시오.

   풀이 보기

   CPU 시간 = 명령어 수 × CPI × 클럭 주기이므로 3요소는 **명령어 수(instruction count), CPI(명령어당 클럭 사이클 수), 클럭 주기(clock cycle time, 또는 클럭 속도)**다. [Ch1 p.28]

   명령어 수는 프로그램, ISA, 컴파일러가 정한다. CPI는 CPU 하드웨어가 정하며 평균값이다. [Ch1 p.28] 성능 영향 표로 보면 알고리즘·프로그래밍 언어·컴파일러는 명령어 수와 CPI, ISA는 명령어 수·CPI·클럭 속도 셋 모두에 영향을 준다. [Ch1 p.29]

2. 21-2 기출 유형 클럭 속도가 500 MHz이고 평균 CPI가 2.5인 컴퓨터의 MIPS를 구하시오.

   풀이 보기

   500 MHz = 500×10^6 Hz = 5×10^8 Hz.

   MIPS = 클럭 속도 / (CPI × 10^6) = 5×10^8 / (2.5×10^6) = 2×10^2 = **200 MIPS**.

   검산: 1초에 5×10^8 사이클, 명령어 하나에 2.5 사이클 → 1초에 2×10^8개 = 200백만 개.

3. 26-1 기출 유형 클럭 주기가 0.5 ns이고 CPI가 1.25인 컴퓨터가 있다. (1) MIPS는? (2) 명령어 3.2×10^9개를 실행하는 데 걸리는 시간은?

   풀이 보기

   클럭 주기가 주어졌으므로 먼저 속도로 바꾼다. 클럭 속도 = 1 / (0.5×10^{-9}) = 2×10^9 Hz = 2 GHz.

   (1) MIPS = 2×10^9 / (1.25×10^6) = **1600**.

   (2) CPU 시간 = 명령어 수 × CPI × 클럭 주기 = $3.2 \times 10^9 \times 1.25 \times 0.5 \times 10^{-9}$ = 3.2×0.625 = **2.0초**.

   검산: MIPS = 3.2×10^9 / (2.0×10^6) = 1600. 일치.

4. 26-1 기출 유형 클럭 주기 250 ps인 컴퓨터에서 어떤 프로그램의 명령어 비율과 CPI가 다음과 같다. 산술 40% (CPI 1), load 30% (CPI 3), store 10% (CPI 3), 분기 20% (CPI 2). (1) 평균 CPI (2) MIPS (3) 명령어가 10^9개일 때 CPU 시간을 구하시오.

   풀이 보기

   (1) 비율이 주어지면 비율을 개수 대신 쓴다. $0.4 \times 1 + 0.3 \times 3 + 0.1 \times 3 + 0.2 \times 2 = 0.4 + 0.9 + 0.3 + 0.4$ = **2.0**.

   (2) 클럭 속도 = 1 / (250×10^{-12}) = 4×10^9 Hz. MIPS = 4×10^9 / (2.0×10^6) = **2000**.

   (3) $10^9 \times 2.0 \times 250 \times 10^{-12}$ = 500×10^{-3} = **0.5초**. 검산: 10^9 / (0.5×10^6) = 2000 MIPS.

5. 26-1 기출 유형 어떤 프로그램은 6억 개의 명령어를 실행하며 0.3초가 걸린다. 클럭 속도는 3 GHz이다. 이 프로그램의 MIPS와 평균 CPI를 구하시오.

풀이 보기

6억 = 6×10^8.

MIPS = 명령어 수 / (실행 시간 × 10^6) = 6×10^8 / (0.3×10^6) = **2000**.

CPI는 MIPS 식을 거꾸로 푼다. CPI = 클럭 속도 / (MIPS × 10^6) = 3×10^9 / (2000×10^6) = 3×10^9 / (2×10^9) = **1.5**.

검산: CPU 시간 = $6 \times 10^8 \times 1.5$ / (3×10^9) = 9×10^8 / (3×10^9) = 0.3초. 일치.

---

6. 예상 같은 ISA인 두 컴퓨터가 같은 프로그램을 실행한다. A는 클럭 2 GHz, CPI 2.0이고 B는 클럭 3 GHz, CPI 3.6이다. (1) 어느 쪽이 몇 배 빠른가? (2) 각 MIPS는?

풀이 보기

ISA가 같으므로 명령어 수 I는 같다.

(1) A의 시간 = I × 2.0 / (2×10^9) = I × 1.0 ns. B의 시간 = I × 3.6 / (3×10^9) = I × 1.2 ns. 성능$_A$ / 성능$_B$ = 시간$_B$ / 시간$_A$ = 1.2 / 1.0 → **A가 B보다 1.2배 빠르다**. 클럭이 높은 B가 진다.

(2) MIPS$_A$ = 2×10^9 / (2.0×10^6) = **1000**, MIPS$_B$ = 3×10^9 / (3.6×10^6) ≈ **833.3**. 같은 ISA·같은 명령어 수라서 이 경우엔 MIPS 비율(1000 / 833.3 = 1.2)이 성능 비율과 같다.

---

7. 26-1 기출 유형 어떤 프로그램의 실행 시간 100초가 다음처럼 나뉜다. 부동소수점 곱셈 40초, 부동소수점 덧셈 20초, 정수 연산 30초, load/store 10초. (1) 곱셈만 4배 빠르게 하면 전체 실행 시간과 속도 향상은? (2) load/store만 10배 빠르게 하면? (3) 곱셈만 개선해서 전체를 2배 빠르게 할 수 있는가? (4) 전체 1.25배를 얻으려면 곱셈을 몇 배 개선해야 하는가?

풀이 보기

(1) 영향 = 40초, 비영향 = 100 − 40 = 60초. $T_{후}$ = 40 / 4 + 60 = 10 + 60 = **70초**. 속도 향상 = 100 / 70 ≈ **1.43배**(성능 약 43% 향상, 시간 30% 감소).

(2) 영향 = 10초, 비영향 = 90초. $T_{후}$ = 10 / 10 + 90 = **91초**. 100 / 91 ≈ **1.10배**. 작은 부분은 10배 빨라져도 효과가 작다.

(3) 목표 = 100 / 2 = 50초. 50 = 40 / n + 60 → 40 / n = −10은 불가능하다. 곱셈을 0초로 만들어도 60초가 남아 최대 속도 향상은 100 / 60 ≈ 1.67배다. → **불가능**.

(4) 목표 = 100 / 1.25 = 80초. 80 = 40 / n + 60 → 40 / n = 20 → **n = 2**. 검산: 40 / 2 + 60 = 80, 100 / 80 = 1.25.

---

8. 26-1 기출 유형 클럭 1 GHz인 컴퓨터에서 프로그램이 산술 명령어 40 × 10^6개(CPI 1), load/store 30 × 10^6개(CPI 4), 분기 20 × 10^6개(CPI 2)를 실행한다. 캐시를 개선해 load/store의 CPI가 2로 줄었다. 개선 전후 CPU 시간과 속도 향상을 구하시오.

   풀이 보기

   개선 전 사이클 = 40 × 1 + 30 × 4 + 20 × 2 = 40 + 120 + 40 = 200 (× 10^6). 시간 = 200 × 10^6 / 10^9 = **0.2초**.

   개선 후 사이클 = 40 + 30 × 2 + 40 = 140 (× 10^6). 시간 = **0.14초**.

   속도 향상 = 0.2 / 0.14 ≈ **1.43배**(성능 약 43% 향상).

   암달로 검산: load/store가 차지하던 시간 0.12초가 2배 빨라짐 → 0.12 / 2 + 0.08 = 0.14초. 일치. 참고로 평균 CPI는 200 / 90 ≈ 2.22에서 140 / 90 ≈ 1.56으로 준다.

9. 예상 새 컴파일러를 쓰면 명령어 수는 20% 줄고 CPI는 10% 늘어난다. 클럭은 같다. 새 컴파일러의 결과는 몇 배 빠른가?

   풀이 보기

   CPU 시간은 명령어 수와 CPI의 곱에 비례한다. 새 시간 / 옛 시간 = 0.8 × 1.1 = 0.88.

   속도 향상 = 1 / 0.88 ≈ **1.14배**. 명령어 수와 CPI가 반대 방향으로 바뀌면 둘을 곱해서 판단한다(트레이드오프).

10. 예상 클럭 속도 800 MHz인 컴퓨터가 어떤 프로그램에서 400 MIPS를 낸다. 평균 CPI와 클럭 주기(ns)를 구하시오.

    풀이 보기

    CPI = 클럭 속도 / (MIPS × 10^6) = 800 × 10^6 / (400 × 10^6) = **2**.

    클럭 주기 = 1 / (800 × 10^6) = 1.25 × 10^{-9} 초 = **1.25 ns**.

11. 예상 O/X: ISA가 다른 두 컴퓨터 X(3000 MIPS)와 Y(2000 MIPS)가 있으면 X가 같은 프로그램을 더 빨리 실행한다.

    풀이 보기

    X. ISA가 다르면 같은 일을 하는 데 필요한 명령어 수가 달라 MIPS로 비교할 수 없다. MIPS는 ISA 차이와 명령어 복잡도 차이를 반영하지 못한다. 실행 시간(명령어 수 × CPI / 클럭 속도)으로 비교해야 한다. [Ch1 p.34]

실전 · 2 관련 단원 Ch2 · 3, 7, 8~10

C와 어셈블리 변환

26-1 시험에는 C 코드 ↔ 어셈블리 변환이 3문제(C → 어셈블리 2, 어셈블리 → C 1) 한 줄씩 나왔고, 21-2 시험에는 "레지스터 번호를 알려 주고 스택 할당 → 덧셈 → 원래 주소로 복귀"하는 짧은 코드 작성이 나왔다. 이 단원은 배열, if-else, 반복문, leaf 프로시저를 양방향으로 번역하는 연습이다.

푸는 순서

1. 변수 ↔ 레지스터 대응표를 문제에서 그대로 옮겨 적는다. 임시값은 x5~x7, x28~x31을 쓴다.
2. 배열 원소: 인덱스가 상수면 오프셋 = 인덱스 × 4 (`lw x9, 32(x22)`). 인덱스가 변수면 `slli` 로 × 4 → `add` 로 시작 주소를 더함 → `lw/sw ..., 0(주소)` .
3. if-else: 조건을 **뒤집어** Else로 분기 → then 부분 → `beq x0, x0, Exit` → `Else:` 부분 → `Exit:` .
4. 반복문: `Loop:` 에서 조건을 뒤집어 Exit로 분기 → 몸체 → 변수 증가 → `beq x0, x0, Loop` → `Exit:` .
5. leaf 프로시저: 저장할 레지스터 개수 k → 스택 포인터 sp(x2)를 `addi sp, sp, -4k` 로 내림 → `sw` 로 오프셋 4(k−1)부터 0까지 저장 → 계산 → 결과를 `x10` 에 → 같은 오프셋에서 `lw` 로 복원 → `addi sp, sp, 4k` → `jalr x0, 0(x1)` .
6. 어셈블리 → C는 거꾸로: 오프셋 ÷ 4 = 인덱스, 분기 조건을 다시 뒤집어 C 조건으로, 뒤로 가는 분기는 반복문.

완전 풀이 예제 (21-2 유형: 스택 할당 → 덧셈 → 복원 → 복귀)

다음 C 함수를 RISC-V 어셈블리로 쓰시오. 인자 a, b, c는 `x10`, `x11`, `x12`, 지역 변수 r은 `x19`, 중간값은 `x5`를 쓴다. 사용하는 `x5`와 `x19`는 스택에 저장했다가 복원하고, 결과는 `x10`으로 반환한다.

```
int func(int a, int b, int c)
{
    int r;
    r = a + b + c;
    return r;
}
```

1단계 (저장 개수) 저장할 레지스터는 x5, x19 두 개 → 2 × 4 = 8바이트. 스택은 높은 주소에서 낮은 주소로 자라므로 sp에서 **뺀다**: `addi sp, sp, -8`. [Ch2 p.37]

2단계 (저장) 확보한 8바이트에 오프셋 4와 0으로 넣는다: `sw x5, 4(sp)`, `sw x19, 0(sp)`.

3단계 (덧셈) a + b를 x5에, 거기에 c를 더해 r(x19)에: `add x5, x10, x11`, `add x19, x5, x12`.

4단계 (반환값) 반환값은 x10에 있어야 하므로 복사: `addi x10, x19, 0`.

5단계 (복원) 저장한 오프셋 그대로 읽는다: `lw x19, 0(sp)`, `lw x5, 4(sp)`. 그리고 자리 반납: `addi sp, sp, 8`.

6단계 (복귀) 복귀 주소는 x1에 있다: `jalr x0, 0(x1)`. [Ch2 p.38]

```
func:
    addi sp, sp, -8      // 스택에 2개 넣을 자리
    sw   x5, 4(sp)       // x5 저장
    sw   x19, 0(sp)      // x19 저장
    add  x5, x10, x11    // x5 = a + b
    add  x19, x5, x12    // r = (a + b) + c
    addi x10, x19, 0     // 반환값 x10 = r
    lw   x19, 0(sp)      // x19 복원
    lw   x5, 4(sp)       // x5 복원
    addi sp, sp, 8       // 스택에서 2개 지우기
    jalr x0, 0(x1)       // 호출한 곳으로 복귀
```

검산 (주소) 처음 sp = 0x7FFFFFF0이면 첫 줄 뒤 sp = 0x7FFFFFF0 − 8 = 0x7FFFFFE8. x5는 0x7FFFFFE8 + 4 = 0x7FFFFFEC, x19는 0x7FFFFFE8에 저장된다. 마지막 addi 뒤 sp = 0x7FFFFFF0으로 돌아온다.

참고: 호출 규약(p.39)만 따지면 x5는 caller-saved라 저장하지 않아도 되고 x19(callee-saved)만 저장하면 된다. 시험이 "사용하는 레지스터를 스택에 저장"하라고 하면 슬라이드 leaf_example처럼 쓰는 레지스터를 모두 저장한다. 문제의 지시를 따른다. [Ch2 p.38–39]

함정

- A[3]의 오프셋은 3이 아니라 12(= 3 × 4)다. 변수 인덱스 A[i]는 오프셋 자리에 i를 쓸 수 없으므로 `slli` 와 `add` 로 주소를 만든다. [Ch2 p.14, 30]
- if (a == b)는 `bne` 로, if (a != b)는 `beq` 로, if (a < b)는 `bge` 로 Else에 간다(조건 뒤집기). 어셈블리 → C에서는 다시 뒤집어 읽는다.
- `blt` , `bge` 는 레지스터끼리만 비교한다. i < 10처럼 상수와 비교하려면 먼저 `addi x6, x0, 10` 으로 상수를 레지스터에 넣는다.
- then 부분 뒤 `beq x0, x0, Exit` 를 빠뜨리면 else까지 실행된다.
- sp 조정은 **바이트** 수(개수 × 4)이고, 시작은 빼기(−), 끝은 더하기(+)다. 복귀는 `jalr x0, 0(x1)` . 반환값은 반드시 x10.
- `sw x9, 12(x22)` 에서 x9는 저장할 값이고 목적지는 메모리다. lw와 방향이 반대다.

시험 포인트

- 배열 lw/sw 오프셋(인덱스 × 4), 변수 인덱스는 slli 2 + add
- if-else 4줄 틀, while/for 틀(조건 뒤집기, 끝의 beq x0, x0, Loop)
- leaf 프로시저: `addi sp, sp, -4k` / sw / 계산 / addi x10 / lw / `addi sp, sp, 4k` / `jalr x0, 0(x1)`
- 어셈블리 → C: 오프셋 ÷ 4, 분기 조건 되뒤집기

연습 문제

1. 26-1 기출 유형 C → 어셈블리: `A[3] = A[1] + h;` h는 `x21` , 정수 배열 A의 시작 주소는 `x22` 이다. 임시 레지스터는 `x9` 를 쓴다.

   풀이 보기

   A[1]의 오프셋 = 1 × 4 = 4, A[3]의 오프셋 = 3 × 4 = 12.

   ```
   lw  x9, 4(x22)       // x9 = A[1]
   add x9, x9, x21      // x9 = A[1] + h
   sw  x9, 12(x22)      // A[3] = x9
   ```

2. 26-1 기출 유형 C → 어셈블리: `g = h - A[i];` g, h, i는 `x20` , `x21` , `x23` , A의 시작 주소는 `x22` 이다. 임시 레지스터는 `x5` , `x6` .

   풀이 보기

   i가 변수이므로 주소 = A + i × 4를 레지스터로 계산한다.

   ```
   slli x5, x23, 2      // x5 = i * 4
   add  x5, x5, x22     // x5 = A[i]의 주소
   lw   x6, 0(x5)       // x6 = A[i]
   sub  x20, x21, x6    // g = h - A[i]
   ```

   sub의 순서 주의: h − A[i]이므로 `sub x20, x21, x6` (첫 번째 소스가 h).

3. 26-1 기출 유형 C → 어셈블리: `if (a != b) c = a - b; else c = a + b;` a, b, c는 `x19`, `x20`, `x21`이다.

풀이 보기

조건 a != b를 뒤집으면 a == b → `beq`로 Else에 간다.

```
      beq x19, x20, Else   // a == b 이면 Else로
      sub x21, x19, x20    // c = a - b
      beq x0, x0, Exit     // 무조건 Exit로
Else: add x21, x19, x20    // c = a + b
Exit:
```

4. 예상 C → 어셈블리: `sum = 0; for (i = 0; i < 10; i++) sum = sum + A[i];` sum, i는 `x19`, `x20`, A의 시작 주소는 `x21`. 임시 레지스터는 `x5` ~ `x7`.

풀이 보기

for는 "초기화 → (조건 검사 → 몸체 → 증가) 반복"인 while과 같다. 조건 i < 10을 뒤집으면 i >= 10 → `bge`로 Exit. 10은 레지스터에 넣어 둔다.

```
      addi x19, x0, 0      // sum = 0
      addi x20, x0, 0      // i = 0
      addi x6, x0, 10      // x6 = 10 (비교용 상수)
Loop: bge  x20, x6, Exit   // i >= 10 이면 Exit로
      slli x5, x20, 2      // x5 = i * 4
      add  x5, x5, x21     // x5 = A[i]의 주소
      lw   x7, 0(x5)       // x7 = A[i]
      add  x19, x19, x7    // sum = sum + A[i]
      addi x20, x20, 1     // i++
      beq  x0, x0, Loop    // 처음으로
Exit:
```

5. 26-1 기출 유형 어셈블리 → C: `x10`은 정수 배열 A의 시작 주소이다. 다음 코드를 C 한 줄로 쓰시오.

```
lw  x5, 8(x10)
lw  x6, 12(x10)
add x5, x5, x6
sw  x5, 0(x10)
```

풀이 보기

오프셋 ÷ 4 = 인덱스: 8 → A[2], 12 → A[3], 0 → A[0].

x5 = A[2], x6 = A[3], x5 = A[2] + A[3], A[0]에 저장 → **`A[0] = A[2] + A[3];`**

6. 26-1 기출 유형 어셈블리 → C: a, b, m은 `x19`, `x20`, `x21`이다. C 코드로 쓰고 이 코드가 하는 일을 한 마디로 쓰시오.

```
      bge  x19, x20, Else
      addi x21, x20, 0
      beq  x0, x0, Exit
Else: addi x21, x19, 0
Exit:
```

풀이 보기

`bge x19, x20, Else`는 a >= b이면 Else로 간다. 컴파일러가 조건을 뒤집어 썼으므로 C 조건은 다시 뒤집은 a < b다.

then: `addi x21, x20, 0` → m = b. else: m = a.

`if (a < b) m = b; else m = a;` 두 수 중 큰 값을 m에 넣는다(최댓값).

---

7. 예상 어셈블리 → C: s, i, n은 `x19`, `x20`, `x21`이다. C 코드로 쓰고, n = 4일 때 끝난 뒤 s의 값을 쓰시오.

```
      addi x19, x0, 0
      addi x20, x0, 1
Loop: blt  x21, x20, Exit
      add  x19, x19, x20
      addi x20, x20, 1
      beq  x0, x0, Loop
Exit:
```

풀이 보기

s = 0, i = 1로 시작. `blt x21, x20, Exit`는 n < i이면 빠져나간다. 계속하는 조건은 그 반대인 i <= n이다. 뒤로 가는 `beq x0, x0, Loop`가 있으므로 반복문이다.

`s = 0; for (i = 1; i <= n; i++) s = s + i;` (while로 써도 된다: `s = 0; i = 1; while (i <= n) { s = s + i; i = i + 1; }`)

n = 4: s = 1 + 2 + 3 + 4 = **10**, 끝날 때 i = 5.

---

8. 예상 어셈블리 → C: i는 `x22`, 배열 A의 시작 주소는 `x25`이다.

```
slli x5, x22, 2
add  x5, x5, x25
lw   x6, 0(x5)
addi x6, x6, 1
sw   x6, 0(x5)
```

풀이 보기

앞 두 줄은 A[i]의 주소 계산, lw로 A[i]를 읽고 1을 더해 같은 주소에 다시 쓴다.

`A[i] = A[i] + 1;` (또는 `A[i]++;`)

---

9. 21-2 기출 유형 다음 함수를 RISC-V로 쓰시오. 인자 a, b, c, d는 `x10` ~ `x13`이다. `x5`에 a + b, `x6`에 c + d, `x7`에 두 합의 합을 계산하고, 사용하는 `x5`, `x6`, `x7`은 스택에 저장했다가 복원한 뒤 결과를 `x10`으로 반환하고 복귀한다. 처음 sp가 0x7FFFFFF0일 때 x5가 저장되는 주소도 쓰시오.

```
int add4(int a, int b, int c, int d)
{
    return a + b + c + d;
}
```

풀이 보기

저장할 레지스터 3개 → 12바이트. 오프셋 8, 4, 0.

```
add4:
    addi sp, sp, -12     // 3개 넣을 자리
    sw   x5, 8(sp)
    sw   x6, 4(sp)
    sw   x7, 0(sp)
    add  x5, x10, x11    // x5 = a + b
    add  x6, x12, x13    // x6 = c + d
    add  x7, x5, x6      // x7 = (a + b) + (c + d)
    addi x10, x7, 0      // 반환값
    lw   x7, 0(sp)
    lw   x6, 4(sp)
    lw   x5, 8(sp)
    addi sp, sp, 12      // 3개 지우기
    jalr x0, 0(x1)       // 복귀
```

주소: sp = 0x7FFFFFF0 − 12 = 0x7FFFFFE4(16진수로 12 = 0xC). x5는 8(sp) = 0x7FFFFFE4 + 8 = **0x7FFFFFEC**.

10. 21-2 기출 유형 빈칸 (가)~(마)를 채우시오. C: `int diff(int g, int h) { int f; f = g - h + 5; return f; }`, g·h는 `x10`·`x11`, f는 `x18`, 중간값은 `x9`, 사용한 두 레지스터를 스택에 저장한다.

```
diff:
    addi sp, sp, (가)
    sw   x9, (나)(sp)
    sw   x18, 0(sp)
    sub  x9, x10, x11
    addi x18, x9, 5
    (다)
    lw   x18, 0(sp)
    lw   x9, 4(sp)
    addi sp, sp, (라)
    (마)
```

풀이 보기

(가) `-8` : 2개 × 4바이트, 스택은 아래로 자라므로 뺀다.

(나) **4**: 복원하는 `lw x9, 4(sp)`와 같은 오프셋.

(다) `addi x10, x18, 0` : f를 반환값 레지스터 x10으로 옮긴다.

(라) **8**: 확보한 8바이트 반납.

(마) `jalr x0, 0(x1)` : x1에 든 복귀 주소로 돌아간다. x9, x18은 둘 다 callee-saved라 호출 규약으로도 저장이 필요하다.

[Ch2 p.39]

실전 · 3 관련 단원 Ch2 · 2~4, 6, 7

실행 결과 추적과 엔디언

26-1 시험에는 "코드를 실행한 뒤 레지스터에 저장된 결과값 쓰기"가 나왔고 little/big endian을 공부하라는 후기가 있었다. 이 단원은 짧은 코드를 한 줄씩 실행해 레지스터와 메모리 바이트를 쓰는 연습이다.

푸는 순서

1. 레지스터 표를 만든다(열: 줄, 명령어, 바뀐 레지스터, 10진수, 16진수 8자리). 한 줄 실행할 때마다 한 칸씩 채운다. `x0` 은 항상 0이다.
2. 즉시값(12비트)은 부호 확장된다. `addi x5, x0, -3` → 0xFFFFFFFD.
3. 음수 결과는 2의 보수 32비트로 쓴다: 양수 크기를 16진수로 쓰고 0x100000000에서 뺀다. 예: −15 = 0x100000000 − 0xF = 0xFFFFFFF1.
4. 시프트: slli = 오른쪽에 0, srli = 왼쪽에 0, srai = 왼쪽에 부호 비트. 32비트 밖으로 밀려난 비트는 버린다. 4칸 = 16진수 한 자리.
5. 반복문은 "몇 번째 반복에서 분기 조건이 참이 되는지" 표로 센다. 끝에서 검사하는 bne 반복은 조건이 깨지는 값은 몸체에서 처리되지 않는다.
6. 메모리: word 하나 = 주소 4개. 리틀 엔디언(RISC-V)은 **낮은 주소 = 최하위 바이트**이므로, lw 값은 **높은 주소부터** 바이트를 이어 붙여 읽는다. 빅 엔디언은 낮은 주소부터 그대로 읽는다. [Ch2 p.13]

완전 풀이 예제

다음 코드를 실행한 뒤 각 레지스터 값을 10진수와 16진수로 쓰시오. 그리고 `x10` = 0x00000100일 때 마지막 `sw` 뒤 주소 0x100 ~ 0x103의 바이트를 쓰시오(RISC-V).

```
addi x5, x0, 12
addi x6, x0, -3
add  x7, x5, x6
sub  x28, x6, x5
slli x29, x5, 2
srai x30, x28, 1
srli x31, x28, 28
sw   x28, 0(x10)
```

줄	계산	10진수	16진수
`addi x5, x0, 12`	0 + 12	x5 = 12	0x0000000C
`addi x6, x0, -3`	0 + (−3), 즉시값 부호 확장	x6 = −3	0xFFFFFFFD
`add x7, x5, x6`	12 + (−3)	x7 = 9	0x00000009
`sub x28, x6, x5`	(−3) − 12 (첫 소스에서 둘째 소스를 뺌)	x28 = −15	0xFFFFFFF1
`slli x29, x5, 2`	12×2^2	x29 = 48	0x00000030
`srai x30, x28, 1`	0xFFFFFFF1을 1칸 오른쪽, 왼쪽은 부호 비트 1	x30 = −8	0xFFFFFFF8
`srli x31, x28, 28`	0xFFFFFFF1을 28칸(16진 7자리) 오른쪽, 왼쪽은 0	x31 = 15	0x0000000F

자세히: −15를 32비트로 쓰면 15 = 0xF이므로 0x100000000 − 0xF = **0xFFFFFFF1**. 아래 8비트는 `1111 0001`이다. srai 1칸: `11 11 0001` → `1111 1000` (왼쪽에 1이 들어옴) → 0xFFFFFFF8 = −8. −15 ÷ 2 = −7.5인데 sra는 더 작은 쪽(−8)으로 내림이 된다는 점도 주의. srli 28칸: 16진수 8자리 F F F F F F F 1에서 오른쪽 7자리를 버리면 맨 앞 F만 남아 0x0000000F = 15.

메모리 x28 = 0xFFFFFFF1을 바이트로 나누면 FF FF FF F1(최상위 → 최하위). 리틀 엔디언이므로 가장 낮은 주소에 최하위 바이트 F1:

주소	0x100	0x101	0x102	0x103
리틀 엔디언 (RISC-V)	F1	FF	FF	FF
빅 엔디언이었다면	FF	FF	FF	F1

함정

- `sub rd, rs1, rs2`는 rs1 − rs2. 순서를 바꾸면 부호가 뒤집힌다.
- 음수를 16진수로 쓸 때 앞자리를 F로 채워 8자리를 다 쓴다(−3 = 0xFFFFFFFD, 0xFD로 쓰면 틀림).
- 음수에 srli를 하면 큰 양수가 된다(왼쪽에 0이 들어옴). 부호를 지키는 것은 srai.
- slli로 밀려 나간 위쪽 비트는 버려진다: 0x80000010 << 4 = 0x00000100.
- lui는 상위 20비트를 넣고 하위 12비트를 0으로 만든다. 뒤에 붙는 addi의 즉시값이 음수면 상위 쪽이 1 줄어든다. [Ch2 p.46]
- 리틀 엔디언에서 lw 값을 낮은 주소부터 그대로 이어 쓰면 빅 엔디언 답이 된다. 바이트 안의 두 16진 자리는 뒤집지 않는다(F1을 1F로 쓰지 않음).
- RISC-V는 정렬 제한이 없어 4의 배수가 아닌 주소에서도 lw가 된다. 그 경우도 주소부터 연속 4바이트를 같은 규칙으로 읽는다. [Ch2 p.13]

시험 포인트

- addi·add·sub·and·or·xor·시프트 결과를 16진수 8자리로
- 음수의 2의 보수 표현, 즉시값 부호 확장, srli와 srai 차이
- 반복 횟수 세기(bne·beq·blt 반복)
- 리틀/빅 엔디언 메모리 표 ↔ lw 결과, sw 후 바이트 배치

연습 문제

1. 26-1 기출 유형 처음 `x5` = 0x000000F0, `x6` = 0x0000003C이다. 다음을 각각 실행한 결과를 16진수로 쓰시오.

```
and  x7, x5, x6
or   x28, x5, x6
xor  x29, x5, x6
xori x30, x5, -1
andi x31, x6, 0x0F
```

<details>
<summary>풀이 보기</summary>

아래 8비트만 다르다. x5 = `1111 0000`, x6 = `0011 1100`.

and: `0011 0000` → x7 = **0x00000030**.

or: `1111 1100` → x28 = **0x000000FC**.

xor(다르면 1): `1100 1100` → x29 = **0x000000CC**.

xori −1: 즉시값 −1은 부호 확장되어 0xFFFFFFFF, 모든 비트 반전(NOT) → x30 = **0xFFFFFF0F**. [Ch2 p.27]

andi 0x0F: 0x3C의 아래 4비트만 남김(마스크) → x31 = **0x0000000C**.

</details>

2. 26-1 기출 유형 `x5` = 0x80000010일 때 다음 결과를 16진수로 쓰시오. (1) `slli x6, x5, 4` (2) `srli x7, x5, 4` (3) `srai x28, x5, 4` (4) `srai x29, x5, 31`

   풀이 보기

   4칸 = 16진수 한 자리. x5 = 8 0 0 0 0 0 1 0, 최상위 비트(부호)는 1(8 = `1000`).

   (1) 왼쪽으로 한 자리 밀고 맨 앞 8은 버림, 오른쪽에 0 → **0x00000100**.

   (2) 오른쪽으로 한 자리 밀고 맨 뒤 0은 버림, 왼쪽에 0 → **0x08000001**.

   (3) 왼쪽 4비트를 부호 비트 1로 채움 → `1111` = F → **0xF8000001**.

   (4) 31칸 밀면 부호 비트만 남고 왼쪽은 모두 1 → **0xFFFFFFFF**(−1). 부호 비트가 0이었다면 0이 된다.

---

3. 26-1 기출 유형 다음 코드 실행 후 `x7`, `x28`, `x29`, `x30`의 값을 16진수로 쓰시오.

```
addi x5, x0, 5
addi x6, x0, -8
add  x7, x5, x6
sub  x28, x5, x6
srai x29, x7, 1
srli x30, x7, 1
```

   풀이 보기

   x5 = 5, x6 = −8 = 0xFFFFFFF8.

   x7 = 5 + (−8) = −3 → **0xFFFFFFFD**.

   x28 = 5 − (−8) = 13 → **0x0000000D**.

   x29: 0xFFFFFFFD = ...1111 1101을 1칸 산술 시프트 → ...1111 1110 → **0xFFFFFFFE**(−2).

   x30: 같은 값을 1칸 논리 시프트 → 맨 앞에 0이 들어와 0111 1111 ... 1111 1110 → **0x7FFFFFFE**(큰 양수 2,147,483,646).

---

4. 예상 다음 실행 후 `x5`와 `x6`의 값을 16진수로 쓰시오.

```
lui  x5, 0x12345
addi x5, x5, 0x678
lui  x6, 0x12345
addi x6, x6, -1
```

   풀이 보기

   `lui x5, 0x12345` : 상위 20비트에 0x12345, 하위 12비트는 0 → 0x12345000. [Ch2 p.46]

   x5 = 0x12345000 + 0x678 = **0x12345678**.

   x6 = 0x12345000 + (−1) = **0x12344FFF**. 하위 12비트가 000에서 1을 빼니 위쪽에서 빌려 와 상위가 0x12344로 바뀐다.

---

5. 26-1 기출 유형 다음 코드 실행 후 `x5`와 `x6`의 값은? `add`는 몇 번 실행되는가?

```
      addi x5, x0, 0
      addi x6, x0, 1
      addi x7, x0, 5
Loop: add  x5, x5, x6
      addi x6, x6, 1
      bne  x6, x7, Loop
```

풀이 보기

반복	add 후 x5	addi 후 x6	bne x6 != 5?
1	0 + 1 = 1	2	참 → Loop
2	1 + 2 = 3	3	참 → Loop
3	3 + 3 = 6	4	참 → Loop
4	6 + 4 = 10	5	거짓 → 빠져나감

x5 = **10** (0x0000000A), x6 = **5**, add는 **4번**. x6 = 5가 되자마자 검사에서 끝나므로 5는 더해지지 않는다.

---

6. 예상 다음 코드 실행 후 `x5`, `x6`의 값과 `beq x6, x7, Exit`가 실행된 횟수를 쓰시오.

```
      addi x5, x0, 1
      addi x6, x0, 0
      addi x7, x0, 4
Loop: beq  x6, x7, Exit
      slli x5, x5, 1
      addi x6, x6, 1
      beq  x0, x0, Loop
Exit:
```

풀이 보기

x6 = 0, 1, 2, 3일 때는 분기하지 않고 x5를 두 배로 만든다: 1 → 2 → 4 → 8 → 16. x6 = 4일 때 Exit로 간다.

x5 = **16** (0x00000010) = 2^4, x6 = **4**. `beq x6, x7, Exit`는 x6 = 0, 1, 2, 3, 4일 때 한 번씩 → **5번**(몸체는 4번).

---

7. **26-1 기출 유형** RISC-V 메모리에 다음 바이트가 들어 있고 `x10` = 0x00000200이다. (1) `lw x5, 0(x10)` (2) `lw x6, 4(x10)` 실행 후 x5, x6의 값(16진수, x6는 10진수 부호 있는 값도)을 쓰시오. (3) 같은 바이트를 빅 엔디언 기계가 읽었다면 x5, x6는? (4) `lw x7, 2(x10)`의 결과는?

주소	0x200	0x201	0x202	0x203	0x204	0x205	0x206	0x207
바이트	78	56	34	12	EF	CD	AB	89

풀이 보기

(1) 주소 0x200 ~ 0x203. 리틀 엔디언은 높은 주소(0x203)부터 이어 붙인다: 12 34 56 78 → x5 = **0x12345678**.

(2) 주소 0x204 ~ 0x207: 89 AB CD EF → x6 = **0x89ABCDEF**. 최상위 비트가 1(8 = `1000`)이라 부호 있는 값은 음수: 0x89ABCDEF − 2^{32} = **−1,985,229,329**.

(3) 빅 엔디언은 낮은 주소부터 그대로: x5 = **0x78563412**, x6 = **0xEFCDAB89**.

(4) 주소 0x202 ~ 0x205: 바이트 34 12 EF CD → 높은 주소부터 CD EF 12 34 → x7 = **0xCDEF1234**. RISC-V는 정렬 제한이 없어 이런 접근도 허용된다. [Ch2 p.13]

---

8. **26-1 기출 유형** `x5` = 0xCAFE0123, `x10` = 0x00001000이다. 다음 실행 후 주소 0x1008 ~ 0x100F의 바이트를 RISC-V(리틀 엔디언)와 빅 엔디언으로 각각 쓰시오.

```
sw   x5, 8(x10)
addi x6, x0, -2
sw   x6, 12(x10)
```

풀이 보기

첫 sw의 주소 = 0x1000 + 8 = 0x1008. 0xCAFE0123 → 바이트 CA FE 01 23(최상위 → 최하위).

x6 = −2 = 0xFFFFFFFE → 바이트 FF FF FF FE. 둘째 sw의 주소 = 0x1000 + 12 = 0x100C.

주소	0x1008	0x1009	0x100A	0x100B	0x100C	0x100D	0x100E	0x100F
리틀 (RISC-V)	23	01	FE	CA	FE	FF	FF	FF
빅	CA	FE	01	23	FF	FF	FF	FE

---

9. 예상 RISC-V 메모리 0x300부터 다음 바이트가 있고 `x10` = 0x00000300이다. 코드 실행 후 `x5`, `x6`, `x10`의 값과 0x30C ~ 0x30F의 바이트를 쓰시오.

주소	0x300	0x301	0x302	0x303	0x304	0x305	0x306	0x307	0x308	0x309	0x30A	0x30B
바이트	05	00	00	00	FD	FF	FF	FF	10	00	00	00

```
      addi x5, x0, 0
      addi x6, x0, 3
Loop: lw   x7, 0(x10)
      add  x5, x5, x7
      addi x10, x10, 4
      addi x6, x6, -1
      bne  x6, x0, Loop
      sw   x5, 0(x10)
```

풀이 보기

세 word를 리틀 엔디언으로 읽으면 0x00000005 = 5, 0xFFFFFFFD = −3, 0x00000010 = 16.

반복 1: x7 = 5, x5 = 5, x10 = 0x304, x6 = 2 → 분기. 반복 2: x7 = −3, x5 = 2, x10 = 0x308, x6 = 1 → 분기. 반복 3: x7 = 16, x5 = 18, x10 = 0x30C, x6 = 0 → 빠져나감.

x5 = **18** (0x00000012), x6 = **0**, x10 = **0x0000030C**.

sw는 0x30C에 0x00000012를 쓴다 → 0x30C ~ 0x30F = **12 00 00 00**.

---

10. 예상 다음 실행 후 `x7`의 값은?

```
      addi x5, x0, -1
      addi x6, x0, 1
      addi x7, x0, 0
      blt  x5, x6, L1
      addi x7, x7, 1
L1:   bltu x5, x6, L2
      addi x7, x7, 2
L2:
```

풀이 보기

x5 = 0xFFFFFFFF. blt(부호): −1 < 1 참 → L1로 분기, `addi x7, x7, 1`은 건너뜀.

bltu(무부호): x5 = $2^{32} - 1$ < 1 거짓 → 분기 안 함, `addi x7, x7, 2` 실행.

x7 = **2**. [Ch2 p.31]

---

11. 예상 각 명령어의 주소가 왼쪽과 같다. 실행이 0x00400000에서 시작해 0x00400018(End)에 도착했을 때 `x1`, `x10`, `x11`의 값은?

```
0x00400000:      addi x10, x0, 7
0x00400004:      jal  x1, Dbl
0x00400008:      addi x11, x10, 0
0x0040000C:      beq  x0, x0, End
0x00400010: Dbl: add  x10, x10, x10
0x00400014:      jalr x0, 0(x1)
0x00400018: End:
```

풀이 보기

`jal x1, Dbl`은 다음 명령어 주소(PC + 4) = 0x00400008을 x1에 저장하고 Dbl로 간다. [Ch2 p.35]

Dbl: x10 = 7 + 7 = 14. `jalr x0, 0(x1)`로 0x00400008에 복귀.

x11 = 14, 그 뒤 End로 무조건 분기.

x1 = **0x00400008**, x10 = **14** (0x0000000E), x11 = **14**.

---

12. 예상 RISC-V(리틀 엔디언) 메모리의 주소 0x1000, 0x1001, 0x1002, 0x1003에 바이트 `8C`, `7F`, `00`, `A5`가 차례로 들어 있고 `x5` = 0x1000이다. 다음을 실행한 뒤 x6, x7, x8, x9의 값을 16진수 8자리로 쓰시오. (lb는 1바이트를 읽어 부호 확장, lbu는 0 확장)

```
lb   x6, 0(x5)
lbu  x7, 0(x5)
lb   x8, 1(x5)
lw   x9, 0(x5)
```

풀이 보기

x6: 주소 0x1000의 바이트 0x8C = 1000 1100, 맨 위 비트 1 → 부호 확장 → **0xFFFFFF8C** (−116).

x7: 같은 바이트를 0 확장 → **0x0000008C** (140).

x8: 주소 0x1001의 바이트 0x7F = 0111 1111, 맨 위 비트 0 → **0x0000007F** (lb여도 위쪽이 0).

x9: 리틀 엔디언이므로 높은 주소(0x1003)부터 이어 붙인다: A5 00 7F 8C → **0xA5007F8C**.

lb와 lbu는 바이트의 맨 위 비트가 1일 때만 결과가 다르다. [Ch2 p.13, 50]

---

13. 예상 주소 0x2000의 word는 처음에 0x11223344이고 `x5` = 0x2000이다. 다음을 실행한 뒤 x7, x8, x9의 값을 16진수 8자리로 쓰시오. (sb는 레지스터의 하위 1바이트만 저장)

```
addi x6, x0, -85
sb   x6, 2(x5)
lw   x7, 0(x5)
lbu  x8, 2(x5)
lb   x9, 2(x5)
```

풀이 보기

처음 바이트(리틀 엔디언): 0x2000 = 44, 0x2001 = 33, 0x2002 = 22, 0x2003 = 11.

`addi x6, x0, -85` : −85 = 0x100000000 − 0x55 = 0xFFFFFFAB. 하위 1바이트는 0xAB.

`sb x6, 2(x5)` : 주소 0x2002에 0xAB만 쓴다 → 바이트 44, 33, AB, 11. 다른 3바이트는 그대로다.

x7 = 높은 주소부터 11 AB 33 44 → **0x11AB3344**.

x8 = lbu로 0xAB를 0 확장 → **0x000000AB** (171).

x9 = lb로 0xAB(맨 위 비트 1)를 부호 확장 → **0xFFFFFFAB** (−85, x6과 같은 값으로 돌아온다). [Ch2 p.13, 50]

실전 · 4 관련 단원 Ch2 · 3, 5, 12, 13

기계어 인코딩과 리틀 엔디언 바이트

26-1 시험에 "어셈블리 코드를 기계어로 바꿔 little endian으로 쓰기"가 나왔고, encoding 표(opcode·funct 표)는 시험지에 주어졌다. 빈칸으로 형식 이름(R형·I형 등)과 branch의 offset도 나왔다. 이 단원은 표를 보고 어셈블리 → 기계어 → 메모리 바이트, 그리고 그 반대 방향을 손으로 끝까지 하는 연습이다.

푸는 순서 (어셈블리 → 메모리 바이트)

1. **표에서 명령어 찾기**: 형식(R/I/S/SB/U/UJ), opcode, funct3, funct7을 옮겨 적는다.
2. **피연산자를 필드로**: 레지스터 번호(x9 → 9)를 5비트로, 즉시값을 형식에 맞는 비트 수의 2의 보수로 바꾼다. 분기는 먼저 오프셋(바이트) = 목표 주소 − 분기 명령어 자신의 주소.
3. **형식 순서대로 이어 붙이기**: 왼쪽(31번)부터. 어셈블리의 `rd, rs1, rs2` 순서와 기계어 칸 순서가 다르다.
4. **32비트를 4비트씩 끊어 16진수** 8자리로.
5. **바이트로 나누어 메모리에**: 16진수 두 자리가 1바이트. 리틀 엔디언(RISC-V)은 **맨 오른쪽 바이트**(최하위)를 주소 +0에 놓는다. 빅 엔디언은 맨 왼쪽 바이트를 +0에.

반대 방향(메모리 바이트 → 어셈블리)은 5 → 1 순서로 거꾸로: 바이트를 엔디언에 맞게 다시 모아 32비트 → 맨 오른쪽 7비트(opcode)로 형식 결정 → 필드 자르기 → 어셈블리.

시험지에 주어질 encoding 표

슬라이드 p.50의 표를 그대로 옮겼다. 26-1 시험지에 준 encoding 표가 이런 형태일 가능성이 높다. 값을 외우기보다 표를 빠르게 읽고 칸 순서를 틀리지 않는 데 집중한다. n.a.는 해당 칸이 없다는 뜻이다. [Ch2 p.50]

형식(Format)	명령어(Instruction)	opcode	funct3	funct7
R	`add`	`0110011`	`000`	`0000000`
R	`sub`	`0110011`	`000`	`0100000`
R	`sll`	`0110011`	`001`	`0000000`
R	`xor`	`0110011`	`100`	`0000000`
R	`srl`	`0110011`	`101`	`0000000`
R	`sra`	`0110011`	`101`	`0100000`
R	`or`	`0110011`	`110`	`0000000`
R	`and`	`0110011`	`111`	`0000000`
R	`lr.d`	`0110011`	`011`	`0001000`
R	`sc.d`	`0110011`	`011`	`0001100`
I	`lb`	`0000011`	`000`	n.a.
I	`lh`	`0000011`	`001`	n.a.
I	`lw`	`0000011`	`010`	n.a.
I	`lbu`	`0000011`	`100`	n.a.
I	`lhu`	`0000011`	`101`	n.a.
I	`addi`	`0010011`	`000`	n.a.
I	`slli`	`0010011`	`001`	`0000000`
I	`xori`	`0010011`	`100`	n.a.
I	`srli`	`0010011`	`101`	`0000000`
I	`srai`	`0010011`	`101`	`0100000`
I	`ori`	`0010011`	`110`	n.a.
I	`andi`	`0010011`	`111`	n.a.
I	`jalr`	`1100111`	`000`	n.a.
S	`sb`	`0100011`	`000`	n.a.
S	`sh`	`0100011`	`001`	n.a.
S	`sw`	`0100011`	`010`	n.a.
SB	`beq`	`1100011`	`000`	n.a.
SB	`bne`	`1100011`	`001`	n.a.
SB	`blt`	`1100011`	`100`	n.a.

형식(Format)	명령어(Instruction)	opcode	funct3	funct7
SB	`bge`	`1100011`	`101`	n.a.
SB	`bltu`	`1100011`	`110`	n.a.
SB	`bgeu`	`1100011`	`111`	n.a.
U	`lui`	`0110111`	n.a.	n.a.
UJ	`jal`	`1101111`	n.a.	n.a.

`lr.d` · `sc.d` 는 강의 범위 밖의 원자적 명령어라 문제에 쓰지 않는다. `lb` · `lh` · `lbu` · `lhu` · `sb` · `sh` 의 동작은 Ch2 · 3 끝에 있다. slli·srli·srai의 funct7 칸은 실제로는 I형 12비트 즉시값의 위 7비트(imm[11:5]) 자리다. 시프트 양은 아래 5비트(imm[4:0])에 들어간다. srli와 srai는 이 위 7비트로만 구분된다. 교재 기준

형식별 칸 순서 (왼쪽 = 31번 비트)

형식	칸 순서(비트 수)	즉시값 메모
R	funct7(7) · rs2(5) · rs1(5) · funct3(3) · rd(5) · opcode(7)	즉시값 없음
I	imm[11:0](12) · rs1(5) · funct3(3) · rd(5) · opcode(7)	12비트 2의 보수, −2048 ~ 2047
S	imm[11:5](7) · rs2(5) · rs1(5) · funct3(3) · imm[4:0](5) · opcode(7)	12비트를 위 7·아래 5로 쪼갬
SB	imm[12](1) · imm[10:5](6) · rs2(5) · rs1(5) · funct3(3) · imm[4:1](4) · imm[11](1) · opcode(7)	13비트 오프셋, 0번 비트는 저장 안 함
U	imm[31:12](20) · rd(5) · opcode(7)	상위 20비트
UJ	imm[20](1) · imm[10:1](10) · imm[11](1) · imm[19:12](8) · rd(5) · opcode(7)	21비트 오프셋, 0번 비트는 저장 안 함

형식 비트 배치는 슬라이드 그림에 있다. [Ch2 p.22–24, p.47, p.50, Ch4 p.21] SB 범위 −4096 ~ 4094바이트는 [Ch2 p.47], 해독 4단계는 [Ch2 p.50], 리틀 엔디언은 [Ch2 p.13].

분기 오프셋(branch offset) 분기 목표 주소 − 분기 명령어 **자신의** 주소. 단위는 **바이트**다. 명령어 k개 뒤로 가면 −4k, 앞으로 가면 +4k. 어셈블리에 적는 숫자(예 `beq x5, x0, 8` 의 8)가 이 값이다.

imm[12:1] SB형이 실제로 저장하는 12비트. 13비트 오프셋 imm[12:0]에서 늘 0인 0번 비트를 뺀 것이라, 값으로는 오프셋 ÷ 2와 같다. 예: 오프셋 −12 → 13비트 `1111111110100` → imm[12:1] = `111111111010`.

완전 풀이 예제

다음 코드가 주소 0x00001000부터 놓여 있다. 앞의 두 명령어를 기계어(16진수)로 바꾸고, 주소 0x1000 ~ 0x1007에 리틀 엔디언으로 놓인 바이트를 쓰시오. 빅 엔디언이었다면 어떻게 다른가?

```
0x1000        sub  x5, x6, x7
0x1004        beq  x5, x0, Skip
0x1008        addi x10, x10, -1
0x100C  Skip: ...
```

① sub x5, x6, x7

1. 표: sub = R형, opcode 0110011, funct3 000, funct7 0100000.
2. 필드: `sub rd, rs1, rs2`이므로 rd = 5 = `00101`, rs1 = 6 = `00110`, rs2 = 7 = `00111`.
3. 이어 붙이기(R형 순서 funct7 · rs2 · rs1 · funct3 · rd · opcode):

0100000	00111	00110	000	00101	0110011
funct7	rs2 = 7	rs1 = 6	funct3	rd = 5	opcode

4. 4비트씩: `0100 0000 0111 0011 0000 0010 1011 0011` = **0x407302B3**.
5. 바이트: 40 · 73 · 02 · B3 (왼쪽이 최상위). 리틀 엔디언은 최하위 B3부터 → 0x1000 = B3, 0x1001 = 02, 0x1002 = 73, 0x1003 = 40.

② beq x5, x0, Skip

1. 표: beq = SB형, opcode 1100011, funct3 000.
2. 오프셋 = Skip − 자기 주소 = 0x100C − 0x1004 = **+8**바이트(명령어 2개 앞).
3. 8을 13비트로: `0 0000 0000 1000` → imm[12] = 0, imm[11] = 0, imm[10:5] = `000000`, imm[4:1] = `0100` (imm[0] = 0은 버림). imm[12:1] = `000000000100` (= 4 = 8 ÷ 2).
4. 레지스터: `beq rs1, rs2, 오프셋`이므로 rs1 = x5 = `00101`, rs2 = x0 = `00000`.

0 imm[12]	000000 imm[10:5]	00000 rs2 = 0	00101 rs1 = 5	000 funct3	0100 imm[4:1]	0 imm[11]	1100011 opcode

5. 이어 쓰면 `0000000 00000 00101 000 01000 1100011` → 4비트씩 `0000 0000 0000 0010 1000 0100 0110 0011` = **0x00028463**.
6. 바이트 00 · 02 · 84 · 63 → 리틀 엔디언 0x1004 = 63, 0x1005 = 84, 0x1006 = 02, 0x1007 = 00.

주소	0x1000	0x1001	0x1002	0x1003	0x1004	0x1005	0x1006	0x1007
리틀 엔디언 (RISC-V)	B3	02	73	40	63	84	02	00
빅 엔디언	40	73	02	B3	00	02	84	63

엔디언은 **word 안에서 바이트 순서**만 바꾼다. 명령어끼리의 순서(sub가 0x1000, beq가 0x1004)는 그대로이고, 한 바이트 안의 두 16진 자리(B3을 3B로 뒤집기 등)도 바뀌지 않는다.

검산: 0x1004 ~ 0x1007의 63 84 02 00을 거꾸로 모으면 0x00028463 → opcode `1100011`(SB), funct3 000(beq), rs1 5, rs2 0, imm[4:1] = 0100, 나머지 0 → 오프셋 8 → 목표 0x1004 + 8 = 0x100C = Skip. 일치한다.

함정

① 리틀 엔디언에서 16진수 문자를 한 글자씩 뒤집으면 안 된다. 0x407302B3 → B3 02 73 40 (바이트 단위)이고, 3B 20 37 04가 아니다.

② R형 칸은 rs2가 rs1보다 왼쪽이다. 분기도 `beq rs1, rs2`에서 rs2가 왼쪽 칸이다.

③ 분기 오프셋의 기준은 분기 명령어 **자신의** 주소다(다음 명령어 주소 아님). 단위는 바이트다.

④ SB형은 imm[11]이 7번 비트(맨 아래 쪽), imm[12]가 31번 비트에 간다. 13비트를 그대로 왼쪽 칸에 붙이면 틀린다. UJ형도 imm[11]이 20번 비트에 따로 간다.

⑤ 음수 즉시값은 칸 비트 수(I·S는 12, SB는 13, UJ는 21)의 2의 보수로 먼저 바꾼다.

⑥ 슬라이드 p.47의 bne 칸 표에는 opcode가 `1100111`로 적혀 있지만 위 문장은 "opcode: 1100011 for bne"다. 슬라이드 칸 표의 `1100111`은 오타이고 정답은 `1100011`이다(`1100111`은 jalr). 표에서 opcode를 옮길 때 이 둘을 헷갈리지 말 것. [Ch2 p.47, 50]

시험 포인트

- 어셈블리 → 16진 기계어 → 리틀 엔디언 바이트(주소 +0 = 최하위 바이트).
- 메모리 덤프(바이트) → 32비트 재조립 → opcode로 형식 → 어셈블리.
- 형식 이름 빈칸(R/I/S/SB/U/UJ), branch offset 빈칸(바이트), 분기 목표 = 분기 명령어 주소 + 오프셋.

연습 문제

모든 문제에서 위의 encoding 표를 쓴다. 메모리는 바이트 주소, 한 칸 = 1바이트.

1. 26-1 기출 유형 완전 풀이 예제 코드의 세 번째 명령어 `addi x10, x10, -1`(주소 0x1008)을 16진 기계어로 바꾸고, 0x1008 ~ 0x100B에 리틀 엔디언으로 놓인 바이트를 쓰시오.

   답 보기

   addi = I형, opcode 0010011, funct3 000. rd = rs1 = 10 = `01010`.

   −1을 12비트 2의 보수로: 1 = `000000000001` → 반전 `111111111110` → +1 → `111111111111`.

   imm · rs1 · funct3 · rd · opcode = `111111111111 01010 000 01010 0010011` → `1111 1111 1111 0101 0000 0101 0001 0011` = **0xFFF50513**.

   리틀 엔디언: 0x1008 = **13**, 0x1009 = **05**, 0x100A = **F5**, 0x100B = **FF**. (빅 엔디언이면 FF F5 05 13)

2. 26-1 기출 유형 `sw x9, 12(x2)`를 기계어로 바꾸어 주소 0x200부터 리틀 엔디언으로 쓰시오. 빅 엔디언과 비교하시오.

답 보기

sw = S형, opcode 0100011, funct3 010. `sw rs2, imm(rs1)`: 저장할 값 x9가 rs2 = `01001`, 베이스 x2가 rs1 = `00010`.

12 = `000000001100` → imm[11:5] = `0000000`, imm[4:0] = `01100`.

`0000000 01001 00010 010 01100 0100011` → `0000 0000 1001 0001 0010 0110 0010 0011` = **0x00912623**.

리틀 엔디언: 0x200 = 23, 0x201 = 26, 0x202 = 91, 0x203 = 00. 빅 엔디언: 0x200 = 00, 0x201 = 91, 0x202 = 26, 0x203 = 23.

---

3. 26-1 기출 유형 RISC-V 메모리 덤프가 다음과 같다. 주소 0x2000에서 시작하는 명령어를 어셈블리로 쓰시오.

주소	0x2000	0x2001	0x2002	0x2003
바이트	33	85	A4	00

답 보기

리틀 엔디언이므로 0x2003의 바이트가 최상위: 00 A4 85 33 → **0x00A48533**.

2진수 `0000 0000 1010 0100 1000 0101 0011 0011` → opcode(오른쪽 7비트) = `0110011` → R형.

7·5·5·3·5·7로 자르기: `0000000 01010 01001 000 01010 0110011` → funct7 0, rs2 10, rs1 9, funct3 000, rd 10 → add.

`add x10, x9, x10` (rd, rs1, rs2 순서).

---

4. 26-1 기출 유형 각 명령어의 인코딩 형식 이름을 쓰시오: (a) `jalr` (b) `slli` (c) `lui` (d) `jal` (e) `sra` (f) `sw` (g) `bge` (h) `lw`

답 보기

(a) I형 (b) I형 (c) U형 (d) UJ형 (e) R형 (f) S형 (g) SB형 (h) I형.

헷갈리는 것: jalr은 이름에 jump가 있지만 레지스터 + 12비트 상수라 I형(jal만 UJ형). slli는 시프트 양이 상수라 I형, sra는 시프트 양이 레지스터라 R형. lw는 load라 I형, sw는 store라 S형.

---

5. 26-1 기출 유형 다음 코드에서 `Loop` 자리의 branch offset(바이트), SB형에 저장되는 imm[12:1], 16진 기계어, 리틀 엔디언 바이트를 쓰시오.

```
0x0104  Loop: slli x6, x10, 2
0x0108        add  x6, x6, x12
0x010C        addi x10, x10, 1
0x0110        blt  x10, x11, Loop
```

답 보기

오프셋 = 0x0104 − 0x0110 = **−12**바이트(명령어 3개 뒤로).

12 = `0000000001100` (13비트) → 반전 `1111111110011` → +1 → `1111111110100` = −12. 0번 비트를 빼면 **imm[12:1] =** `1 11111111010`.

조각: imm[12] = 1, imm[11] = 1, imm[10:5] = `111111`, imm[4:1] = `1010`. blt = funct3 100, rs1 = x10 = `01010`, rs2 = x11 = `01011`.

`1 111111 01011 01010 100 1010 1 1100011` → `1111 1110 1011 0101 0100 1010 1110 0011` = **0xFEB54AE3**.

리틀 엔디언: 0x0110 = E3, 0x0111 = 4A, 0x0112 = B5, 0x0113 = FE.

---

6. 26-1 기출 유형 주소 0x3008 ~ 0x300B에 바이트 `E3 98 62 FE`가 순서대로 있다(RISC-V). 이 명령어를 어셈블리로 쓰고, 조건이 참일 때 분기 목표 주소를 구하시오.

답 보기

재조립: FE 62 98 E3 → 0xFE6298E3 → `1111 1110 0110 0010 1001 1000 1110 0011`.

opcode `1100011` → SB형. 자르기: `1 111111 00110 00101 001 1000 1 1100011` → imm[12] = 1, imm[10:5] = 111111, rs2 = 6, rs1 = 5, funct3 = 001(bne), imm[4:1] = 1000, imm[11] = 1.

13비트 오프셋 다시 모으기: imm[12] · imm[11] · imm[10:5] · imm[4:1] · 0 = `1 1 111111 1000 0` = `1111111110000` → 맨 앞이 1이라 음수, 반전 +1 → 10000_2 = 16 → **−16**.

명령어: `bne x5, x6, -16`. 목표 = 0x3008 − 16 = 0x3008 − 0x10 = **0x2FF8**.

---

7. 예상 `lui x5, 0x12345`의 기계어와, 주소 0x400부터 리틀 엔디언·빅 엔디언으로 놓인 바이트를 각각 쓰시오.

답 보기

---

lui = U형, opcode 0110111. imm[31:12] = 0x12345 = `0001 0010 0011 0100 0101`, rd = 5 = `00101`.

`00010010001101000101 00101 0110111` → `0001 0010 0011 0100 0101 0010 1011 0111` = **0x123452B7**.

리틀: 0x400 = B7, 0x401 = 52, 0x402 = 34, 0x403 = 12. 빅: 0x400 = 12, 0x401 = 34, 0x402 = 52, 0x403 = B7.

U형은 상위 20비트가 16진 5자리 그대로 앞에 나온다(0x12345...). rd·opcode 12비트가 뒤 3자리(2B7)를 만든다.

---

8. 예상 `srai x6, x7, 3`을 16진 기계어로 쓰시오. 같은 피연산자의 `srli`와 어디가 다른가?

   답 보기

   srai = I형, opcode 0010011, funct3 101, 위 7비트 0100000. 즉시값 12비트 = `0100000` + 시프트 양 3 = `00011` → `010000000011`.

   rs1 = 7 = `00111`, rd = 6 = `00110`. `010000000011 00111 101 00110 0010011` → `0100 0000 0011 0011 1101 0011 0001 0011` = **0x4033D313**.

   srli는 위 7비트가 0000000이라 **0x0033D313**. 30번 비트 하나(16진 맨 앞 자리 4 ↔ 0)만 다르다.

---

9. 예상 주소 0x1000의 `jal x1, 2048`은 어디로 점프하는가? 기계어와 리틀 엔디언 바이트를 쓰시오.

   답 보기

   목표 = 0x1000 + 2048 = 0x1000 + 0x800 = **0x1800**. x1에는 다음 명령어 주소 0x1004가 들어간다.

   2048 = 2^{11} → 21비트 imm[20:0]으로 쓰면 `0 00000000 1 0000000000 0` (imm[20] · imm[19:12] · imm[11] · imm[10:1] · imm[0] 순서)로 11번 비트만 1. 조각: imm[20] = 0, imm[10:1] = `0000000000`, imm[11] = **1**, imm[19:12] = `00000000`.

   UJ형 순서 imm[20] · imm[10:1] · imm[11] · imm[19:12] · rd · opcode = `0 0000000000 1 00000000 00001 1101111` → `0000 0000 0001 0000 0000 0000 1110 1111` = **0x001000EF**.

   리틀 엔디언: 0x1000 = EF, 0x1001 = 00, 0x1002 = 10, 0x1003 = 00. imm[11]이 20번 비트로 가서 16진 셋째 자리가 1이 된다.

---

10. 예상 이번에는 **빅 엔디언** 기계의 덤프다. 주소 0x400 ~ 0x403에 `FE 54 2C 23`이 있다. 같은 encoding 표를 쓴다고 할 때 명령어를 쓰시오.

    답 보기

    빅 엔디언은 +0이 최상위 바이트이므로 그대로 0xFE542C23 → `1111 1110 0101 0100 0010 1100 0010 0011`.

    opcode `0100011` → S형(sw). 자르기: `1111111 00101 01000 010 11000 0100011` → imm[11:5] = 1111111, rs2 = 5, rs1 = 8, funct3 = 010, imm[4:0] = 11000.

    12비트 즉시값 = `1111111 11000` = `111111111000` → 음수, 반전 +1 → 1000_2 = 8 → −8.

    `sw x5, -8(x8)`. 리틀 엔디언으로 읽었다면(23 2C 54 FE 순서로 착각) opcode가 엉뚱한 값이 되어 표에 없다. 덤프 문제는 엔디언부터 확인한다.

---

11. 26-1 기출 유형 다음 두 명령어가 주소 0x0200부터 놓여 있다. 0x0200 ~ 0x0207의 바이트를 리틀 엔디언으로 쓰시오.

```
lw   x8, -4(x2)
and  x7, x8, x9
```

답 보기

lw: I형 opcode 0000011, funct3 010. −4 = 12비트 `111111111100`, rs1 = 2, rd = 8 → `111111111100 00010 010 01000 0000011` → `1111 1111 1100 0001 0010 0100 0000 0011` = 0xFFC12403.

and: R형 funct7 0000000, funct3 111. rs2 = 9, rs1 = 8, rd = 7 → `0000000 01001 01000 111 00111 0110011` → `0000 0000 1001 0100 0111 0011 1011 0011` = 0x009473B3.

주소	0x200	0x201	0x202	0x203	0x204	0x205	0x206	0x207
바이트	03	24	C1	FF	B3	73	94	00

12. 예상 `bge x6, x0, 20`의 인코딩 칸을 채우시오. 빈칸 ①~④를 쓰고 16진수로 완성하시오.

0 imm[12]	① imm[10:5]	**00000** rs2	② rs1	③ funct3	④ imm[4:1]	**0** imm[11]	**1100011** opcode

답 보기

20 = 13비트 `0000000010100` → imm[10:5] = ① `000000`, imm[4:1] = ④ `1010`. rs1 = x6 = ② `00110`, bge funct3 = ③ `101`.

`0 000000 00000 00110 101 1010 0 1100011` → `0000 0000 0000 0011 0101 1010 0110 0011` = **0x00035A63**.

13. 예상 주소 0x0500 ~ 0x0503의 바이트가 `93 F2 F2 0F`다(RISC-V). 명령어를 쓰시오.

답 보기

재조립 0x0FF2F293 → `0000 1111 1111 0010 1111 0010 1001 0011`. opcode `0010011` → I형 산술·논리.

자르기 `000011111111 00101 111 00101 0010011` → imm = 255, rs1 = 5, funct3 = 111(andi), rd = 5.

`andi x5, x5, 255` (x5의 하위 8비트만 남기기).

실전 · 5 관련 단원 Ch3 · 2, 3

곱셈 upper/lower 빈칸

21-2 시험에 "부호 없는/부호 있는/부호 다른 곱셈" 표의 upper/lower 칸을 채우는 문제가 나왔다(Ch3 p.13 표). 같은 비트를 세 가지로 읽어 곱하고, 2n비트 결과를 위·아래로 나누는 연습이다. 숫자가 바뀌어도 같은 순서로 풀 수 있게 한다.

푸는 순서

1. **비트를 두 가지로 읽기**: n비트 수의 무부호 값, 부호 있는(2의 보수) 값. 맨 앞 비트가 1이면 부호 있는 값 = 무부호 값 − 2^n (3비트면 −8, 4비트면 −16).
2. **모드별 십진 곱**: 무부호 = 무부호 × 무부호, 부호 = 부호 × 부호, 부호-무부호 = A(부호) × B(무부호).
3. **곱을 2n비트 2진수로**: 양수는 그대로. 음수는 2^{2n} + 곱(6비트면 64 + 곱, 8비트면 256 + 곱)을 2진수로, 또는 크기를 2진수로 쓴 뒤 반전하고 1 더하기.
4. **반으로 자르기**: 왼쪽 n비트 = Upper, 오른쪽 n비트 = Lower.
5. **검산**: 세 모드의 Lower가 모두 같아야 한다(무부호 곱의 아래 n비트와 같음).

명령어와 연결

표의 칸	32비트에서 구하는 명령	피연산자 해석
Lower (세 모드 공통)	`mul`	상관없음
Upper, Signed	`mulh`	둘 다 부호 있음
Upper, Unsigned	`mulhu`	둘 다 무부호
Upper, Signed-Unsigned	`mulhsu`	하나 부호 있음, 하나 무부호

RISC-V는 32비트 × 32비트의 64비트 곱을 위 32비트와 아래 32비트로 따로 구한다. 아래 절반은 해석과 상관없이 같아서 `mul` 하나, 위 절반은 해석마다 달라서 세 개다. [Ch3 p.13] 3비트 표는 이것을 작게 줄인 것이다(3비트 × 3비트 → 6비트, 위 3 · 아래 3).

Lower가 늘 같은 이유: 2의 보수에서 음수 −k는 2^n − k와 같은 비트다. 곱을 2^n으로 나눈 나머지(아래 n비트)만 보면 2^n의 배수 차이는 사라지므로, 어떻게 읽든 아래 n비트는 같다. PDF 외 보충

`mulhsu`는 rs1을 부호 있는 수, rs2를 무부호로 읽는다. 표의 A가 rs1, B가 rs2에 해당한다. PDF 외 보충

완전 풀이 예제 (슬라이드 p.13 표)

3비트 A = 101, B = 101을 곱해 6비트 곱을 얻는다. 표를 채우시오.

Mode	A	B	A × B	Upper	Lower
Unsigned	(가)	(나)	(다)	(라)	(마)
Signed	(바)	(사)	(아)	(자)	(차)
Signed-Unsigned	(카)	(타)	(파)	(하)	(거)

1단계, 비트 읽기: 101의 무부호 값 = 4 + 1 = **5**. 맨 앞이 1이라 부호 있는 값 = 5 − 8 = **−3**. (확인: $3 = 011_2$ → 반전 100 → +1 → 101.)

2단계, 모드별 곱: Unsigned 5 × 5 = **25**. Signed (−3) × (−3) = **9**. Signed-Unsigned (−3) × 5 = **−15**.

3단계, 6비트 2진수

- 25 = 16 + 8 + 1 = `011001`
- 9 = 8 + 1 = `001001`
- −15: 15 = `001111` → 반전 `110000` → +1 → `110001`. 다른 방법: 64 − 15 = 49 = 32 + 16 + 1 = `110001`. 거꾸로 확인: 110001을 부호 있게 읽으면 49 − 64 = −15.

4단계, 자르기

Mode	A	B	A × B	Upper	Lower
Unsigned	5 (101)	5 (101)	25 (011001)	011	001
Signed	−3 (101)	−3 (101)	9 (001001)	001	001
Signed-Unsigned	−3 (101)	5 (101)	−15 (110001)	110	001

5단계, 검산: Lower가 셋 다 001. 슬라이드 문장 그대로 "아래 3비트는 같고 위 3비트는 곱셈 모드에 따라 다르다". [Ch3 p.13] 32비트로 옮기면 Upper 칸은 차례로 mulhu, mulh, mulhsu의 결과이고 Lower 칸은 mul의 결과다.

함정

① 음수 곱을 2n비트로 쓸 때 n비트(3비트)로 쓰면 틀린다. −15는 6비트 `110001` 이다. 부호 확장까지 포함해 2n비트 전체를 채운다.

② 결과가 양수면 Upper 맨 앞은 0, 음수면 1이다. Signed 결과 9인데 Upper가 1로 시작하면 계산이 틀린 것이다.

③ B의 맨 앞 비트가 0이면 B는 무부호로 읽든 부호 있게 읽든 같은 값이라 Signed와 Signed-Unsigned 줄이 같아진다. 실수가 아니다.

④ Lower를 구하는 명령은 mul 하나다. "mulhu = 무부호 아래 절반"은 틀린다.

연습 문제

1. 21-2 기출 유형 빈칸을 채우시오(3비트 곱셈, 6비트 곱).

Mode	A	B	A × B	Upper	Lower
Unsigned	5 (101)	5 (101)	(ㄱ)	011	(ㄴ)
Signed	(ㄷ)	−3 (101)	9 (001001)	(ㄹ)	001
Signed-Unsigned	−3 (101)	(ㅁ)	(ㅂ)	(ㅅ)	001

답 보기

(ㄱ) 25 (011001) (ㄴ) 001 (ㄷ) −3 (101) (ㄹ) 001 (ㅁ) 5 (101) (ㅂ) −15 (110001) (ㅅ) 110.

이유: 101은 무부호 5, 부호 −3. −15 = 64 − 15 = 49 = 110001_2. Lower는 모두 001.

---

2. 21-2 기출 유형 3비트 A = 110, B = 011로 같은 표(Unsigned, Signed, Signed-Unsigned)를 완성하시오.

답 보기

읽기: 110 = 무부호 6, 부호 6 − 8 = −2. 011 = 무부호 3, 부호 3 (맨 앞 0).

곱: Unsigned 6 × 3 = 18. Signed (−2) × 3 = −6. Signed-Unsigned (−2) × 3 = −6.

6비트: 18 = 16 + 2 = `010010`. −6 = 64 − 6 = 58 = `111010` (6 = 000110_2 → 반전 111001 → +1 → 111010).

Mode	A	B	A × B	Upper	Lower
Unsigned	6 (110)	3 (011)	18 (010010)	010	010
Signed	−2 (110)	3 (011)	−6 (111010)	111	010
Signed-Unsigned	−2 (110)	3 (011)	−6 (111010)	111	010

B의 맨 앞이 0이라 Signed와 Signed-Unsigned가 같다. Lower 010 공통.

---

3. 21-2 기출 유형 3비트 A = 111, B = 111의 세 모드 곱(6비트)과 Upper/Lower를 쓰시오.

답 보기

111 = 무부호 7, 부호 7 − 8 = −1.

Unsigned 7 × 7 = 49 = 32 + 16 + 1 = `110001` → Upper **110**, Lower **001**.

Signed (−1) × (−1) = 1 = `000001` → Upper **000**, Lower **001**.

Signed-Unsigned (−1) × 7 = −7 → 64 − 7 = 57 = `111001` → Upper **111**, Lower **001**.

Unsigned 결과는 맨 앞이 1이지만 무부호라 음수가 아니다(49).

---

4. 예상 4비트 A = 1011, B = 0110을 곱해 8비트 곱을 얻는다. 세 모드의 A × B(십진, 8비트)와 Upper 4비트, Lower 4비트를 쓰시오.

답 보기

읽기: 1011 = 무부호 11, 부호 11 − 16 = −5. 0110 = 6 (맨 앞 0이라 둘 다 6).

Unsigned 11 × 6 = 66 = 64 + 2 = `01000010` → Upper **0100**, Lower **0010**.

Signed (−5) × 6 = −30 → 8비트: 30 = `00011110` → 반전 `11100001` → +1 → `11100010` (= 256 − 30 = 226) → Upper **1110**, Lower **0010**.

Signed-Unsigned (−5) × 6 = −30 → Signed와 같음, Upper **1110**, Lower **0010**.

---

5. 예상 4비트 A = 1101, B = 1011. 세 모드의 표를 완성하시오.

답 보기

읽기: 1101 = 무부호 13, 부호 −3. 1011 = 무부호 11, 부호 −5.

Unsigned 13 × 11 = 143 = 128 + 8 + 4 + 2 + 1 = `10001111` → Upper **1000**, Lower **1111**.

Signed (−3) × (−5) = 15 = `00001111` → Upper **0000**, Lower **1111**.

Signed-Unsigned (−3) × 11 = −33 → 256 − 33 = 223 = 128 + 64 + 16 + 8 + 4 + 2 + 1 = `11011111` → Upper **1101**, Lower **1111**.

검산: Lower 1111 공통, 143을 16으로 나눈 나머지 15 = 1111_2.

---

6. 예상 4비트 A = 1000, B = 1000. Upper/Lower를 모드별로 쓰고, Unsigned와 Signed의 결과가 같은 이유를 쓰시오.

답 보기

1000 = 무부호 8, 부호 −8.

Unsigned 8 × 8 = 64 = `01000000` → Upper 0100, Lower 0000. Signed (−8) × (−8) = 64 → 같음, Upper 0100, Lower 0000. Signed-Unsigned (−8) × 8 = −64 → 256 − 64 = 192 = `11000000` → Upper 1100, Lower 0000.

이유: 두 수의 부호 있는 값이 무부호 값보다 각각 16씩 작지만, 곱 64가 우연히 같아서다. (−8)(−8) = 8 × 8. 비트가 같아도 Upper가 늘 다른 것은 아니다.

---

7. 예상 4비트 A = 0111, B = 1001에서 Signed-Unsigned 줄(A 부호 있음, B 무부호)과 Signed 줄을 구하고 차이를 설명하시오.

   답 보기

   0111 = 7(둘 다). 1001 = 무부호 9, 부호 9 − 16 = −7.

   Signed 7 × (−7) = −49 → 256 − 49 = 207 = `11001111` → Upper 1100, Lower 1111.

   Signed-Unsigned 7 × 9 = 63 = `00111111` → Upper 0011, Lower 1111. (Unsigned도 63으로 같다.)

   Signed-Unsigned에서 B를 무부호로 읽으므로 B = 9. A가 양수라 이 줄은 Unsigned와 같아진다. 어느 쪽이 부호 있는 수인지(rs1 = A)에 따라 결과가 달라진다.

8. 예상 32비트 RISC-V에서 x10 = 0xFFFFFFFF, x11 = 0x00000002일 때 다음 결과(16진수)를 쓰시오. (a) `mul x5, x10, x11` (b) `mulhu x6, x10, x11` (c) `mulh x7, x10, x11` (d) `mulhsu x28, x10, x11`

   답 보기

   0xFFFFFFFF = 무부호 $2^{32} - 1$, 부호 −1. 0x00000002 = 2.

   무부호 곱 = $2^{33} - 2$ = 0x00000001_FFFFFFFE. 부호 곱 = −2 = 0xFFFFFFFF_FFFFFFFE (64비트 2의 보수). 부호-무부호 = (−1) × 2 = −2 → 같은 64비트.

   (a) mul = 아래 32비트 = **0xFFFFFFFE** (셋 다 같음) (b) mulhu = **0x00000001** (c) mulh = **0xFFFFFFFF** (d) mulhsu = **0xFFFFFFFF**.

   3비트로 줄이면 $111_2 \times 010_2$ 표와 같은 구조다(Lower 110 공통, Upper 001 / 111 / 111). [Ch3 p.13]

9. 예상 3비트 Signed-Unsigned 모드에서 A = 111이고 결과의 Upper = 111, Lower = 001이다. B(3비트)를 구하시오.

   답 보기

   곱 `111001`을 부호 있게 읽으면 57 − 64 = −7. A는 부호 있는 −1이므로 (−1) × B = −7 → B = 7 = **111_2** (무부호로 읽음).

10. 21-2 기출 유형 O/X: (1) 곱셈 모드가 달라도 곱의 아래 절반은 같다. (2) `mulhu`는 무부호 곱의 아래 32비트를 준다. (3) 3비트 곱셈 표에서 Signed의 A × B가 음수이면 Upper의 맨 앞 비트는 1이다.

    답 보기

    (1) O. [Ch3 p.13] (2) X, mulhu는 **위** 32비트. 아래는 mul. (3) O, 2n비트 2의 보수에서 음수는 맨 앞 비트가 1이고 그 비트는 Upper에 있다.

실전 · 6 관련 단원 Ch3 · 6, 7, 8

부동소수점 계산과 16비트 표현

26-1 시험에 "십진수를 이진수로 바꿔 부동소수점 계산: A = −0.5, B = 2.625로 A+B, A×B"와 "16비트(부호 1 · 지수 5 · 가수 10)로 표현하기"가 나왔다. 십진 → 이진 → 정규화 → 계산 단계 → 32비트·16비트 16진수까지 한 줄로 이어서 푸는 연습이다.

푸는 순서

1. **십진 → 이진**: 정수부는 2진수로(2로 나누기), 소수부는 **×2를 반복**해 나온 정수 자리(0 또는 1)를 차례로 적는다. 소수부가 0이 되면 끝.
2. **정규화**: $1.xxx_2 \times 2^e$ 꼴로. 이진 소수점을 왼쪽으로 k칸 옮기면 e = +k, 오른쪽으로 k칸이면 e = −k.
3. **계산**: 덧셈 4단계(정렬 → 유효숫자 더하기 → 정규화·오버/언더플로 검사 → 반올림), 곱셈 5단계(지수 더하기 → 유효숫자 곱 → 정규화·검사 → 반올림 → 부호 결정). [Ch3 p.32–36]
4. **비트로**: S(음수 1), 지수 칸 = e + bias(32비트 127, 16비트 15), 가수 칸 = 1. 뒤의 xxx를 오른쪽에 0을 채워 23비트(또는 10비트)로.
5. **16진수**: S · 지수 · 가수를 모두 이어 붙인 뒤 왼쪽부터 4비트씩 끊는다.
6. **검산**: 거꾸로 $(-1)^S \times 1.\text{가수}_2 \times 2^{(\text{지수 칸} - \text{bias})}$를 계산해 원래 값과 비교.

16비트 형식 (1-5-10) PDF 외 보충

슬라이드에는 32비트(단정도)와 64비트(배정도)만 있다. [Ch3 p.27–28] 그런데 26-1 시험에 **16비트(부호 1 · 지수 5 · 가수 10)** 표현 문제가 나왔다. 규칙은 IEEE 754 그대로이고 비트 수와 bias만 다르다. 이 형식은 IEEE 754-2008에 반정밀도(half precision, binary16)로 정의되어 있다.




```
bias = 2^(k-1) - 1 (k = 지수 칸 비트 수)
```

```
값 = (-1)^S × (1 + Fraction) × 2^(Exponent - 15)
```

- **bias 유도**: 32비트 k = 8 → $2^7 - 1 = 127$, 64비트 k = 11 → $2^{10} - 1 = 1023$으로 슬라이드 값과 맞는다. [Ch3 p.28] 16비트 k = 5 → $2^4 - 1 = \mathbf{15}$.
- **예약 지수**: 지수 칸 `00000` (0)과 `11111` (31)은 32비트의 0과 255처럼 예약이다. 00000은 0과 비정규화 수, 11111은 ±∞와 NaN.
- **정규화 수의 지수 범위**: 지수 칸 1 ~ 30 → 실제 지수 1 − 15 = **−14** ~ 30 − 15 = **15**.
- **크기 범위**: 가장 작은 정규화 수 $1.0_2 \times 2^{-14} \approx 6.1 \times 10^{-5}$ (0x0400). 가장 큰 수 $(2 - 2^{-10}) \times 2^{15} = \mathbf{65504}$ (0x7BFF). 정밀도는 숨은 1 + 10 = 11비트.

	16비트	32비트 (단정도)
부호 · 지수 · 가수	1 · 5 · 10	1 · 8 · 23
bias	15	127
예약 지수 칸	00000, 11111	00000000, 11111111
정규화 지수 범위	−14 ~ 15	−126 ~ 127
16진 자릿수	4자리	8자리

소수부 ×2 방법

예: 2.625. 정수부 2 = `10`. 소수부 0.625를 2배씩 하면서 정수 자리를 떼어 낸다. PDF 외 보충

계산	결과	떼어 낸 비트	남은 소수부
0.625 × 2	1.25	1	0.25
0.25 × 2	0.5	0	0.5
0.5 × 2	1.0	1	0 (끝)

위에서 아래로 읽어 $0.625 = 0.101_2$. 그래서 $2.625 = \mathbf{10.101_2}$. 검산: 2 + 1/2 + 1/8 = 2.625.

완전 풀이 예제 (26-1 기출: A = −0.5, B = 2.625)

A + B와 A × B를 2진 부동소수점 단계대로 계산하고, A, B, 두 결과를 32비트 IEEE 754와 16비트(1-5-10) 16진수로 쓰시오.

0. 두 수를 2진 정규화

- A = −0.5: 0.5 × 2 = 1.0 → 0.1_2. -0.1_2 = **$-1.0_2 \times 2^{-1}$**.
- B = 2.625 = 10.101_2 → 소수점을 왼쪽으로 1칸 → **$1.0101_2 \times 2^1$**.

수	S	32비트 지수 칸	32비트	16비트 지수 칸	16비트
A = $-1.0_2 \times 2^{-1}$	1	−1 + 127 = 126 = 01111110_2	0xBF000000	−1 + 15 = 14 = 01110_2	0xB800
B = $1.0101_2 \times 2^1$	0	1 + 127 = 128 = 10000000_2	0x40280000	1 + 15 = 16 = 10000_2	0x4140

A 16비트: `1 01110 0000000000` → `1011 1000 0000 0000` = 0xB800. B 16비트: `0 10000 0101000000` → `0100 0001 0100 0000` = 0x4140. B 32비트: `0 10000000 01010000000000000000000` → `0100 0000 0010 1000 0000 …` = 0x40280000.

1. A + B (4단계)

1. **정렬**: 지수 −1과 1, 차이 2. 지수가 작은 A의 유효숫자를 오른쪽으로 2칸 시프트: $-1.0_2 \times 2^{-1}$ = **$-0.010_2 \times 2^1$**. (비트 칸으로 보면 32비트 128 − 126 = 2, 16비트 16 − 14 = 2)
2. **유효숫자 더하기**: $1.0101_2 + (-0.0100_2)$ = **1.0001_2** → $1.0001_2 \times 2^1$.

```
  1.0101
- 0.0100
--------
  1.0001
```

3. **정규화·검사**: 이미 1.xxxx 꼴이라 시프트 없음. 지수 1은 32비트 범위(−126 ~ 127)와 16비트 범위(−14 ~ 15) 안 → 오버플로·언더플로 없음.
4. **반올림**: 가수 0001은 10비트(16비트)·23비트(32비트) 칸에 다 들어가 반올림할 것이 없다.

결과 **$1.0001_2 \times 2^1 = 10.001_2 = 2.125$**. 검산: −0.5 + 2.625 = 2.125.

0	10000	0001000000
S	지수 16	Fraction

16비트: `0100 0000 0100 0000` = **0x4040**. 32비트: 지수 1 + 127 = 128 = 10000000_2 → `0 10000000 00010000000000000000000` → `0100 0000 0000 1000 0000 0000 0000 0000` = **0x40080000**.

되돌려 검산(16비트): 지수 칸 10000 = 16 → 16 − 15 = 1, 가수 0001000000 → 1.0001_2 = 1 + 1/16 = 1.0625 → 1.0625 × 2 = 2.125.

2. A × B (5단계)

1. **지수 더하기**: −1 + 1 = **0**. 비트 칸으로 하면 bias를 한 번 뺀다. 32비트 126 + 128 − 127 = 127, 16비트 14 + 16 − 15 = **15**. (빼지 않으면 bias가 두 번 들어간다.) [Ch3 p.36]
2. **유효숫자 곱**: $1.0000_2 \times 1.0101_2$ = **1.0101_2** (1.0_2을 곱하므로 그대로).

3. **정규화·검사**: $1.0101_2 \times 2^0$, 이미 정규화. 지수 0은 범위 안.
4. **반올림**: 가수 0101은 칸에 다 들어가 필요 없음.
5. **부호 결정**: (−) × (+) = (−). 부호 비트 1 XOR 0 = 1.

결과 **$-1.0101_2 \times 2^0 = -1.3125$**. 검산: −0.5 × 2.625 = −1.3125, $1.0101_2 = 1 + 1/4 + 1/16 = 1.3125$.

1	01111	0101000000
S	지수 15	Fraction

16비트: `1011 1101 0100 0000` = **0xBD40**. 32비트: 지수 0 + 127 = 127 = 01111111_2 → `1 01111111 01010000000000000000000` → `1011 1111 1010 1000 0000 0000 0000 0000` = **0xBFA80000**.

되돌려 검산(16비트): S = 1, 지수 칸 01111 = 15 → 0, 가수 0101000000 → $1.0101_2 = 1.3125$ → −1.3125.

값	2진 정규화	32비트	16비트
A = −0.5	$-1.0_2 \times 2^{-1}$	0xBF000000	0xB800
B = 2.625	$1.0101_2 \times 2^1$	0x40280000	0x4140
A + B = 2.125	$1.0001_2 \times 2^1$	0x40080000	0x4040
A × B = −1.3125	$-1.0101_2 \times 2^0$	0xBFA80000	0xBD40

함정

① 16비트에 bias 127을 쓰면 틀린다. 16비트는 15다($2^4 - 1$).

② 가수 칸에 숨은 1을 넣지 않는다. 1.0101_2의 가수 칸은 0101 뒤에 0을 채운 것이다.

③ 16비트도 S 1 + 지수 5 = 6비트라 4의 배수가 아니다. 16비트를 다 이어 붙인 뒤 4비트씩 끊는다. 0 10000 0001000000을 "0100 0010 ..."처럼 칸별로 끊으면 틀린다.

④ 덧셈 정렬은 지수가 **작은** 쪽을 **오른쪽**으로 민다. 곱셈은 정렬하지 않고 지수를 더한다.

⑤ 부동소수점은 부호-크기 표현이다. 음수라고 가수를 2의 보수로 바꾸지 않는다. 부호 비트만 1.

시험 포인트

- −0.5, 2.625의 2진 정규화와 A+B = 2.125, A×B = −1.3125 단계 풀이.
- 16비트 1-5-10, bias 15 = $2^{(5-1)} - 1$, 지수 범위 −14 ~ 15, 예약 00000·11111.
- 32비트 0x40080000 / 0xBFA80000, 16비트 0x4040 / 0xBD40.
- 덧셈 4단계·곱셈 5단계 이름과 순서, 곱셈에서 bias 한 번 빼기.

연습 문제

1. 26-1 기출 유형 A = −0.5, B = 2.625일 때 A − B를 부동소수점 덧셈 단계로 계산하고 32비트·16비트 16진수로 쓰시오.

답 보기

A − B = A + (−B) = $-1.0_2 \times 2^{-1} + (-1.0101_2 \times 2^1)$.

① 정렬: A → $-0.0100_2 \times 2^1$. ② 더하기: 둘 다 음수라 크기끼리 더하고 부호 −. $1.0101_2 + 0.0100_2 = 1.1001_2$ → $-1.1001_2 \times 2^1$. ③ 정규화 됨, 범위 안. ④ 반올림 없음.

값 $-11.001_2 = -3.125$. 검산 −0.5 − 2.625 = −3.125.

16비트: 1 · 10000(1 + 15) · 1001000000 → `1100 0010 0100 0000` = **0xC240**. 32비트: 1 · 10000000 · 1001000… → `1100 0000 0100 1000 0000 …` = **0xC0480000**.

---

2. 26-1 기출 유형 C = 1.5, D = −0.375일 때 C + D와 C × D를 단계대로 계산하고 16비트 16진수로 쓰시오.

답 보기

2진: $1.5 = 1.1_2 = 1.1_2 \times 2^0$. 0.375: ×2 → 0.75(0), ×2 → 1.5(1), 0.5 × 2 → 1.0(1) → 0.011_2 → $-1.1_2 \times 2^{-2}$.

C + D: ① 정렬 $-1.100_2 \times 2^{-2} = -0.011_2 \times 2^0$ ② $1.100_2 - 0.011_2 = 1.001_2$ ③ $1.001_2 \times 2^0$, 정규화 됨 ④ 반올림 없음 → **1.125**. 16비트 0 · 01111 · 0010000000 → `0011 1100 1000 0000` = **0x3C80** (32비트 0x3F900000).

C × D: ① 0 + (−2) = −2 (16비트 칸 15 + 13 − 15 = 13) ② $1.1_2 \times 1.1_2 = 10.01_2$ ③ 정규화: $10.01_2 \times 2^{-2} = 1.001_2 \times 2^{-1}$ (오른쪽으로 1칸, 지수 +1 → 칸 14) ④ 반올림 없음 ⑤ (+) × (−) = − → $\mathbf{-1.001_2 \times 2^{-1} = -0.5625}$. 검산 1.5 × 0.375 = 0.5625.

16비트 1 · 01110 · 0010000000 → `1011 1000 1000 0000` = **0xB880** (32비트 0xBF100000).

---

3. 예상 3.5 + 0.75를 덧셈 4단계로 계산하시오. 정규화 단계에서 지수가 어떻게 바뀌는가? 16비트 16진수도 쓰시오.

답 보기

$3.5 = 11.1_2 = 1.11_2 \times 2^1$. $0.75 = 0.11_2 = 1.1_2 \times 2^{-1}$.

① 정렬: 차이 2 → $1.100_2 \times 2^{-1} = 0.011_2 \times 2^1$. ② $1.110_2 + 0.011_2 = \mathbf{10.001_2}$. ③ 정규화: $10.001_2 \times 2^1 = 1.0001_2 \times 2^2$ (오른쪽으로 1칸 시프트, 지수 +1). 범위 안. ④ 반올림 없음.

결과 $1.0001_2 \times 2^2 = 4.25$. 16비트: 0 · 10001(2 + 15 = 17) · 0001000000 → `0100 0100 0100 0000` = **0x4440**. (32비트 0x40880000)

---

4. 예상 16비트(1-5-10, bias 15) 값 0xC500을 십진수로 바꾸시오.

답 보기

`1100 0101 0000 0000` → S = 1, 지수 칸 `10001` = 17, 가수 `0100000000`.

실제 지수 17 − 15 = 2. 유효숫자 $1.01_2 = 1.25$. 값 = -1.25×2^2 = **−5.0**. (32비트로는 0xC0A00000)

---

5. 예상 32비트 값 0x41A40000을 십진수로 바꾸시오.

답 보기

`0100 0001 1010 0100 0000 …` → S = 0, 지수 칸 `10000011` = 131, 가수 `0100100…`.

131 − 127 = 4. 1.01001_2 = 1 + 1/4 + 1/32 = 1.28125. 1.28125 × 16 = **20.5**. (확인: $20.5 = 10100.1_2 = 1.01001_2 \times 2^4$)

---

6. 예상 (1) 16비트 0x3E00을 32비트 IEEE 754로 바꾸시오. (2) 32비트 0xC1300000을 16비트로 바꾸시오.

답 보기

방법: 값(S, 실제 지수, 가수)은 그대로 두고 bias와 칸 폭만 바꾼다. 가수는 오른쪽에 0을 붙이거나 떼면 된다.

(1) `0011 1110 0000 0000` → S 0, 지수 칸 01111 = 15 → 실제 0, 가수 1000000000 → 1.1_2 = 1.5. 32비트: 지수 칸 0 + 127 = 127 = 01111111_2, 가수 1000…(23비트) → `0 01111111 1000…` → `0011 1111 1100 0000 …` = **0x3FC00000**.

(2) `1100 0001 0011 0000 …` → S 1, 지수 칸 10000010 = 130 → 실제 3, 가수 0110… → $-1.011_2 \times 2^3 = -1011_2 = -11$. 16비트: 지수 칸 3 + 15 = 18 = 10010_2, 가수 0110000000 → `1 10010 0110000000` → `1100 1001 1000 0000` = **0xC980**.

---

7. 26-1 기출 유형 빈칸: 16비트 형식이 부호 1비트, 지수 (ㄱ)비트, 가수 (ㄴ)비트이고 bias = $2^{(ㄷ)} - 1$ = (ㄹ)이다. 정규화된 수의 실제 지수 범위는 (ㅁ) ~ (ㅂ)이고, 지수 칸 (ㅅ)과 (ㅇ)은 예약되어 있다.

답 보기

(ㄱ) 5 (ㄴ) 10 (ㄷ) 4 (= 5 − 1) (ㄹ) 15 (ㅁ) −14 (ㅂ) 15 (ㅅ) 00000 (ㅇ) 11111.

이유: 예약을 빼면 지수 칸은 1 ~ 30이고, 15를 빼면 −14 ~ 15.

---

8. 예상 16비트 형식에서 두 수의 지수 칸이 `01110`, `10000`이다. 곱의 지수 칸(정규화 전)을 2진수로 쓰고, 그냥 더하면 왜 틀리는지 쓰시오.

답 보기

14 + 16 − 15 = 15 = `01111` (실제 지수 −1 + 1 = 0).

칸 값은 각각 실제 지수 + 15라서 그냥 더하면 실제 지수 합 + 30이 되어 bias가 두 번 들어간다(14 + 16 = 30 = 0 + 30). 한 번 빼야 실제 지수 + 15가 된다. 32비트에서 127을 한 번 빼는 것과 같다. [Ch3 p.36]

---

9. 예상 −6.75를 ×2 방법으로 2진수로 바꾸고 32비트와 16비트 16진수로 쓰시오.

답 보기

$6 = 110_2$. $0.75 \times 2 = 1.5 \to 1$, $0.5 \times 2 = 1.0 \to 1 \to 0.11_2$. $6.75 = 110.11_2 = 1.1011_2 \times 2^2$.

32비트: 1 · 10000001(2 + 127 = 129) · 1011000… → `1100 0000 1101 1000 0000 …` = **0xC0D80000**.

16비트: 1 · 10001(17) · 1011000000 → `1100 0110 1100 0000` = **0xC6C0**. 검산: $1.1011_2 = 1.6875$, × 4 = 6.75.

---

10. 예상 0.1을 16비트로 표현하시오. 정확히 표현되는가? 반올림은 어떻게 하는가?

답 보기

×2: 0.2(0), 0.4(0), 0.8(0), 1.6(1), 1.2(1), 0.4(0), 0.8(0), 1.6(1), 1.2(1), … → $0.0001100110011\ldots_2$ (0011이 끝없이 반복). 정확히 표현할 수 **없다**.

정규화: $1.100110011001100\ldots_2 \times 2^{-4}$. 가수 10비트 = `1001100110`, 그다음 비트가 0이라 버림(가까운 쪽으로 반올림하면 내림).

16비트: 0 · 01011(−4 + 15 = 11) · 1001100110 → `0010 1110 0110 0110` = **0x2E66**. 되돌리면 $1638/1024 \times 2^{-4} \approx 0.09998$로 0.1과 조금 다르다.

참고로 32비트는 가수 23비트 뒤의 비트가 1이라 올림이 일어나 0x3DCCCCCD가 된다. 그래서 반올림이 덧셈·곱셈의 마지막 단계에 따로 있다. PDF 외 보충

---

11. 예상 O/X: 16비트 형식으로 300 × 300을 계산하면 오버플로가 생긴다. 이유를 지수로 설명하시오.

답 보기

O. $300 = 100101100_2 = 1.00101100_2 \times 2^8$. 지수 더하기 8 + 8 = 16, 유효숫자 곱 $(1.00101100_2)^2 \approx 1.37$이라 정규화해도 지수 16. 16비트 최대 지수 15를 넘으므로 오버플로(값 90000 > 최대 65504). 32비트(최대 지수 127)에서는 문제없다.

실전 · 7 관련 단원 Ch4 · 1~7

데이터패스 빈칸과 제어 신호

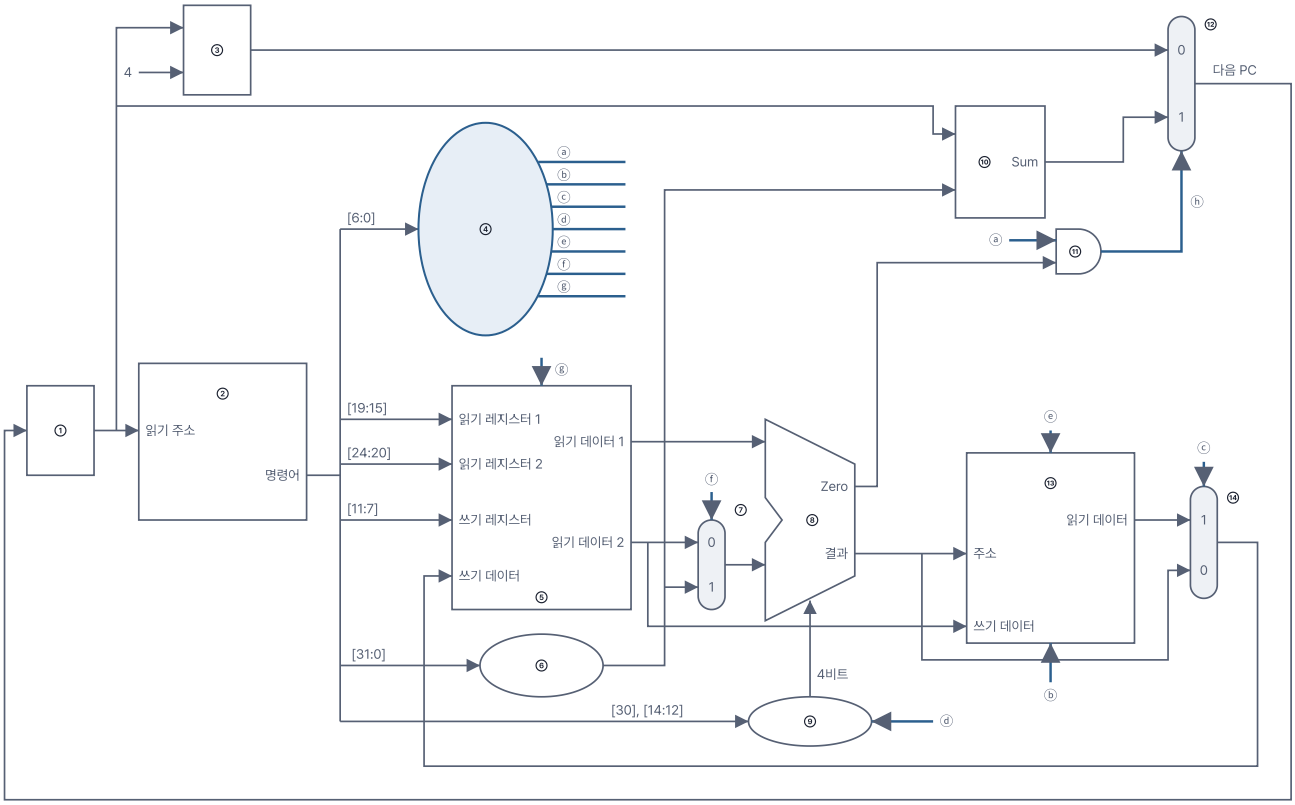
21-2 시험에는 단일 사이클 데이터패스 그림의 부품 이름 빈칸, ld·sd·beq·add의 제어 신호 0/1/X 빈칸, "프로세서를 구성하는 두 가지", "레지스터가 있는 하드웨어" 빈칸이 나왔다. 26-1 후기는 "datapath랑 control line 적기"와 "라인별로 참조하는 레지스터 번호(명령어 비트 범위 [19:15] 등)를 외울 것"을 꼽았다. 이 단원은 그 네 가지를 그림·표·구체 명령어로 연습한다.

푸는 순서

1. **그림 빈칸**: 왼쪽 끝에서 출발한다. PC → 명령어 메모리 → (위) PC + 4 가산기. 명령어에서 갈라지는 선을 따라 제어 유닛(위 타원), 레지스터 파일(큰 상자), ImmGen(아래 타원). 그다음 ALUSrc MUX → ALU → 데이터 메모리 → MemtoReg MUX. 위쪽 줄은 분기 가산기 → AND → PCSrc MUX. ImmGen 출력은 시프트 상자 없이 분기 가산기로 바로 간다(분기용 1비트 시프트는 ImmGen이 함께 한다, 슬라이드 p.15). 타원 셋은 제어 유닛, ImmGen, ALU 제어다.
2. **제어 신호 표**: 열마다 "이 명령어에 그 기능이 필요한가?"를 묻는다. 레지스터에 결과를 쓰나(RegWrite)? ALU 둘째 입력이 즉시값인가(ALUSrc)? 메모리를 읽나·쓰나(MemRead·MemWrite)? 레지스터에 쓸 값이 메모리 값인가(MemtoReg)? 분기인가(Branch)? ALU는 더하기 00 / 빼기 01 / funct로 결정 10(ALUOp).
3. **X 자리**: RegWrite = 0인 명령어(sw, beq)의 MemtoReg만 X다. 쓰기를 안 하니 무엇을 골라도 결과가 같다.
4. **라인별 비트**: [6:0] → 제어 유닛, [19:15] → 읽기 레지스터 1, [24:20] → 읽기 레지스터 2, [11:7] → 쓰기 레지스터, [31:0] → ImmGen, [30]·[14:12] → ALU 제어. 구체 명령어가 나오면 먼저 2진수로 필드를 나누고 각 필드 값을 해당 선에 적는다.

그림: 부품 이름 빈칸

c4-4의 전체 데이터패스에서 부품 이름을 ①~⑭로, 제어 신호 이름을 ⓐ~ⓗ로 비웠다. 제어 유닛 출력선 ⓐ~ⓖ는 슬라이드 p.26–28과 같은 순서이고, 같은 글자의 짧은 강조 선이 그 신호가 들어가는 곳이다. 포트 이름(읽기 레지스터 1 등)과 명령어 비트 범위는 남겨 두었다. 시험 그림도 포트 이름은 보통 적혀 있으니 그것을 단서로 쓴다. [Ch4 p.17, p.24, p.26]



빈칸 데이터패스. ①~⑭ 부품 이름, ⓐ~ⓗ 제어 신호 이름을 쓰시오. 강조 선은 제어 신호다. MUX 안 숫자는 슬라이드와 같다(PCSrc 위 0·아래 1, ALUSrc 위 0·아래 1, MemtoReg 위 1·아래 0). 답은 아래 연습 1·2번에 있다.

완전 풀이 예제: ld, sd, beq, add의 제어 신호 (21-2 기출 유형)

예제

21-2 기출 유형 다음 표의 빈칸에 0, 1, X(don't care) 중 하나를 쓰시오. `ld`는 load doubleword, `sd`는 store doubleword다.

명령어	RegWrite	ALUSrc	MemRead	MemWrite	MemtoReg	Branch	ALUOp
add							
ld							
sd							
beq							

0단계. ld·sd가 무엇인지 정리한다. 21-2 당시 교재는 64비트 RISC-V판이라 레지스터와 메모리 단위가 64비트(doubleword)였고, 그래서 load·store를 `ld`·`sd`로 썼다. 지금 슬라이드(32비트)의 `lw`·`sw`와 하는 일이 같다. 베이스 레지스터 + 오프셋으로 주소를 계산하고 메모리를 읽거나 쓴다. 그래서 **제어 신호 값은 lw·sw와 똑같다**. 시험지에 ld/sd가 나오면 lw/sw로 바꿔 읽으면 된다. 교재 기준 PDF 외 보충

1단계. RegWrite: 레지스터 rd에 결과를 쓰는가? add는 rd에 합을 쓴다 → 1. ld는 메모리 값을 rd에 쓴다 → 1. sd는 메모리에 쓸 뿐 레지스터에는 안 쓴다 → 0. beq는 비교만 한다 → 0. [Ch4 p.21, p.24]

2단계. ALUSrc: ALU 둘째 입력이 즉시값인가? ld·sd는 베이스 + 12비트 오프셋으로 주소를 계산하므로 즉시값 → 1. add는 두 레지스터를 더하므로 0. beq는 두 레지스터를 빼서 비교하므로 0이다. beq의 오프셋은 ALU가 아니라 분기 가산기로 간다. [Ch4 p.13–15]

3단계. MemRead·MemWrite: 데이터 메모리를 읽는가, 쓰는가? 읽는 것은 ld뿐 → MemRead는 ld만 1. 쓰는 것은 sd뿐 → MemWrite는 sd만 1. 나머지는 0이다. 이 둘은 X로 두면 안 된다. 잘못 1이 되면 메모리가 망가지거나 쓸데없는 읽기가 일어난다.

4단계. MemtoReg: 레지스터에 쓸 값이 메모리 값인가? ld → 1(메모리 값). add → 0(ALU 결과). sd와 beq는 RegWrite = 0이라 레지스터에 아무것도 쓰지 않는다. MemtoReg MUX가 무엇을 골라도 쓰이지 않으므로 **X**다.

5단계. Branch: 분기 명령어인가? beq만 1. 나머지는 0. Branch는 X가 될 수 없다. 1이 되면 Zero에 따라 엉뚱한 곳으로 분기할 수 있다. [Ch4 p.25]

6단계. ALUOp: ALU가 무엇을 해야 하나? ld·sd는 주소 계산 덧셈 → 00. beq는 뺄셈으로 같은지 검사 → 01. add는 R형이라 funct7·funct3를 봐야 하므로 → 10. [Ch4 p.18–19] 교재 기준

명령어	RegWrite	ALUSrc	MemRead	MemWrite	MemtoReg	Branch	ALUOp
add (R형)	1	0	0	0	0	0	10
ld (= lw)	1	1	1	0	1	0	00
sd (= sw)	0	1	0	1	X	0	00
beq	0	0	0	0	X	1	01

[Ch4 p.25] sub, and, or도 R형이라 add와 한 줄이 똑같다. 차이는 ALU 제어 유닛이 funct로 만드는 4비트 값(add 0010, sub 0110, and 0000, or 0001)에만 있다.

주의: 시험지마다 열 순서가 다르다(교재 표는 ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, ALUOp1, ALUOp0 순). 외운 숫자열을 그대로 옮기지 말고 열 이름을 보고 하나씩 채운다.

함정

- beq의 ALUSrc를 1로 쓰는 실수. beq에도 즉시값이 있지만 그것은 ImmGen이 1비트 시프트까지 해서 분기 가산기로 보낸다. ALU 둘째 입력은 rs2다.
- sw(sd)의 RegWrite를 X로 쓰는 실수. 1이 되면 [11:7](sw에서는 오프셋 아래 5비트)을 번호로 하는 레지스터가 망가진다. 반드시 0이다.
- X는 "0이어도 1이어도 결과가 같다"는 뜻이다. 0과 같은 뜻이 아니다.
- ALUOp(주 제어 유닛 출력 2비트)와 ALU 제어 입력(4비트)을 섞지 말 것.

연습 문제

1. 21-2 기출 유형 위 그림의 ①~⑭에 들어갈 부품 이름을 쓰시오.

답 보기

번호	이름	찾는 단서
①	PC(프로그램 카운터)	맨 왼쪽, 출력이 명령어 메모리의 읽기 주소로 감
②	명령어 메모리	포트 "읽기 주소", "명령어"
③	가산기(PC + 4)	입력이 PC와 상수 4
④	제어 유닛(주 제어)	입력이 [6:0] opcode
⑤	레지스터 파일	포트 "읽기 레지스터 1·2", "쓰기 레지스터", "쓰기 데이터"
⑥	ImmGen(즉시값 생성기, 부호 확장)	입력이 명령어 전체 [31:0]
⑦	ALUSrc MUX	위 0 = 읽기 데이터 2, 아래 1 = ImmGen 출력
⑧	ALU	출력 "Zero", "결과"
⑨	ALU 제어	입력이 [30], [14:12]와 ALUOp, 출력 4비트
⑩	분기 가산기(분기 목표 주소 계산)	입력이 PC와 ImmGen 출력(이미 1비트 시프트한 오프셋), 출력 "Sum"
⑪	AND 게이트	입력이 Branch와 Zero, 출력이 PCSrc
⑫	PCSrc MUX	위 0 = PC + 4, 아래 1 = 분기 목표(Sum), 출력 = 다음 PC
⑬	데이터 메모리	포트 "주소", "쓰기 데이터", "읽기 데이터"
⑭	MemtoReg MUX	위 1 = 메모리 읽기 데이터, 아래 0 = ALU 결과, 출력이 레지스터 쓰기 데이터로

타원 셋(④·⑥·⑨) 중 opcode를 받는 것이 제어 유닛, 명령어 전체를 받는 것이 ImmGen, funct 비트를 받고 ALU로 4비트를 보내는 것이 ALU 제어다. 가산기 둘(③·⑩)은 입력으로 가른다. 상수 4가 들어가면 PC + 4 가산기다. 슬라이드 그림에는 ImmGen과 분기 가산기 사이에 시프트 상자가 없다. 분기용 1비트 왼쪽 시프트는 ImmGen이 부호 확장과 함께 한다. [Ch4 p.11–17] 교재의 일부 그림에는 "Shift left 1" 상자가 따로 있으니, 시험 그림에 그 상자가 있으면 그 칸의 답은 "왼쪽 1비트 시프트"다. PDF 외 보충

---

2. 26-1 기출 유형 위 그림의 제어 선 ⓐ~ⓗ의 이름을 쓰시오.

답 보기

ⓐ Branch, ⓑ MemRead, ⓒ MemtoReg, ⓓ ALUOp(2비트, ALU 제어로), ⓔ MemWrite, ⓕ ALUSrc, ⓖ RegWrite(레지스터 파일 쓰기 허가), ⓗ PCSrc. ⓐ~ⓖ는 슬라이드 p.26–28 제어 유닛 출력의 위에서부터 순서 그대로다.

ⓑ·ⓔ는 둘 다 데이터 메모리로 들어가서 헷갈린다. 위 그림은 슬라이드 p.17·p.24처럼 위를 MemWrite, 아래를 MemRead로 그렸다. 시험 그림에서는 위치가 다를 수 있으니 "쓰기 데이터 포트 쪽 = 쓰기"처럼 근거를 붙여 답한다. ⓗ는 제어 유닛이 직접 내는 신호가 아니라 AND 게이트 출력이다(Branch AND Zero, p.26–28 그림). p.17·p.24 그림에는 AND 게이트 없이 PCSrc 선만 그려져 있다. [Ch4 p.24–28]

---

3. 예상 빈칸을 채우시오. (열 순서가 예제와 다르다.)

명령어	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
sub	(a)	0	(b)	0	0	0	(c)	0
lw	1	(d)	1	(e)	0	0	0	(f)
sw	(g)	(h)	0	0	(i)	0	0	0
beq	(j)	X	0	0	0	(k)	0	(l)

답 보기

(a) 0: R형은 레지스터 두 개 연산. (b) 1: rd에 결과를 쓴다. (c) 1: R형 ALUOp = 10.

(d) 1: 메모리 값을 레지스터로. (e) 1: 메모리를 읽는다. (f) 0: lw ALUOp = 00.

(g) 1: 주소 = rs1 + 즉시값. (h) X: RegWrite = 0이라 상관없음. (i) 1: 메모리에 쓴다.

(j) 0: ALU는 rs1 − rs2. (k) 1: 분기 명령어. (l) 1: beq ALUOp = 01.

---

4. 예상 역방향: 어떤 명령어의 제어 신호가 RegWrite = 0, ALUSrc = 1, MemWrite = 1, MemtoReg = X, ALUOp = 00이다. lw, sw, add, beq 중 무엇인가? 또 RegWrite = 1, ALUSrc = 0, MemtoReg = 0, ALUOp = 10인 명령어 종류는?

답 보기

첫째: 메모리에 쓰고(MemWrite = 1) 레지스터에 쓰지 않으며 주소를 즉시값으로 계산한다 → **sw**. 둘째: 레지스터 두 개로 계산해 ALU 결과를 레지스터에 쓰고 funct로 연산을 정한다 → **R형**(add, sub, and, or).

---

5. 21-2 기출 유형 제어 신호 표에서 X가 나오는 칸은 어디이며, 왜 X로 둘 수 있는가? 같은 sw에서 RegWrite는 왜 X가 아닌가?

답 보기

sw와 beq의 MemtoReg. 두 명령어는 RegWrite = 0이라 레지스터 파일에 쓰지 않는다. MemtoReg MUX는 "레지스터에 쓸 값"을 고르는 장치인데 그 값이 쓰이지 않으니 0이든 1이든 결과가 같다(don't care). 반면 RegWrite는 실제로 쓰기를 일으키는 신호라 1이 되면 레지스터 값이 망가진다. 그래서 반드시 0이다. 교재 기준

---

6. **26-1 기출 유형** 명령어 비트 범위와 그 선이 연결되는 곳을 짝지으시오. [6:0], [19:15], [24:20], [11:7], [31:0], [30]과 [14:12].

답 보기

비트	필드	연결되는 곳
[6:0]	opcode	제어 유닛
[19:15]	rs1	레지스터 파일 읽기 레지스터 1
[24:20]	rs2	레지스터 파일 읽기 레지스터 2
[11:7]	rd	레지스터 파일 쓰기 레지스터
[31:0]	명령어 전체	ImmGen
[30], [14:12]	funct7의 한 비트, funct3	ALU 제어

외우는 요령: 32비트를 왼쪽부터 7·5·5·3·5·7로 자른다(funct7 | rs2 | rs1 | funct3 | rd | opcode). 경계가 31·25 / 24·20 / 19·15 / 14·12 / 11·7 / 6·0이다. rs1·rs2·rd 위치는 그 필드가 있는 모든 형식에서 같아서 해독 전에 바로 레지스터 파일에 연결할 수 있다. [Ch4 p.21]

---

7. **26-1 기출 유형** `add x5, x6, x7`(기계어 `0x007302B3`)이 실행될 때, 아래 선에 실리는 값을 쓰시오. x6 = 10, x7 = 3이라 하자. (a) 제어 유닛 입력 (b) 읽기 레지스터 1 (c) 읽기 레지스터 2 (d) 쓰기 레지스터 (e) ALU 제어로 가는 [30]과 [14:12] (f) ALU 결과 (g) 레지스터 파일 쓰기 데이터

답 보기

먼저 2진수로 필드를 나눈다. `0x007302B3` = `0000 0000 0111 0011 0000 0010 1011 0011`

`0000000`	`00111`	`00110`	`000`	`00101`	`0110011`
funct7	rs2 = 7	rs1 = 6	funct3	rd = 5	opcode

(a) `0110011`(R형) (b) 6, 즉 `00110` → 읽기 데이터 1 = x6 = 10 (c) 7, 즉 `00111` → 읽기 데이터 2 = x7 = 3 (d) 5, 즉 `00101` (e) [30] = 0, [14:12] = `000` → add, ALU 제어 `0010` (f) 10 + 3 = 13, Zero = 0 (g) MemtoReg = 0이라 ALU 결과 13이 x5에 쓰인다.

ImmGen도 명령어 전체를 받아 무언가를 출력하지만 ALUSrc = 0이라 ALU에 쓰이지 않는다. [Ch4 p.5, p.26]

---

8. 26-1 기출 유형 `lw x9, 32(x22)` (기계어 `0x020B2483`)에서 읽기 레지스터 1, 읽기 레지스터 2, 쓰기 레지스터에 들어가는 번호와 ImmGen 출력, 데이터 메모리 주소를 쓰시오. x22 = 0x1000이다.

   답 보기

   `0x020B2483` = `0000 0010 0000 1011 0010 0100 1000 0011`. I형이므로 [31:20]이 12비트 즉시값이다.

000000100000	10110	010	01001	0000011
imm = 32	rs1 = 22	funct3	rd = 9	opcode

   읽기 레지스터 1 = [19:15] = **22**(베이스). 쓰기 레지스터 = [11:7] = **9**. ImmGen 출력 = 32(부호 확장). 주소 = ALU 결과 = 0x1000 + 32 = **0x1020**.

   읽기 레지스터 2 = [24:20]은 즉시값의 아래 5비트 `00000`이라 **0**(x0)이 들어간다. 선은 연결되어 있으니 무언가를 읽긴 하지만 ALUSrc = 1, MemWrite = 0이라 결과에 쓰이지 않는다. ALU 제어로 가는 [30] = 0, [14:12] = `010`도 ALUOp = 00이라 무시되고 덧셈(`0010`)이 된다. [Ch4 p.27] PDF 외 보충

9. 예상 슬라이드 예제 `lw x5, 40(x6)` (기계어 `0x02832283`)에서 읽기 레지스터 2 선에 실리는 번호는? 그 값이 결과에 영향을 주는가?

   답 보기

   즉시값 40 = `000000101000`. [24:20]은 그 아래 5비트 `01000` = **8**. 레지스터 파일은 x8을 읽어 읽기 데이터 2로 내보내지만, ALUSrc = 1이라 ALU는 즉시값 40을 쓰고, MemWrite = 0이라 데이터 메모리에도 쓰지 않는다. 영향 없음. 나머지: 읽기 레지스터 1 = 6, 쓰기 레지스터 = 5, 주소 = x6 + 40. [Ch4 p.6]

10. 26-1 기출 유형 `sw x9, 48(x22)` (기계어 `0x029B2823`)에서 읽기 레지스터 1, 읽기 레지스터 2, 쓰기 레지스터 선의 번호를 쓰고, 쓰기 레지스터 선의 값이 왜 문제가 되지 않는지 설명하시오. x22 = 0x2000이다.

    답 보기

    S형: 즉시값 48 = `0000001 10000`을 [31:25] = imm[11:5] = `0000001`, [11:7] = imm[4:0] = `10000`으로 나눠 넣는다.

0000001	01001	10110	010	10000	0100011
imm[11:5]	rs2 = 9	rs1 = 22	funct3	imm[4:0]	opcode

    읽기 레지스터 1 = **22**(베이스), 읽기 레지스터 2 = **9**(저장할 값, 읽기 데이터 2 → 데이터 메모리 쓰기 데이터), 쓰기 레지스터 = [11:7] = `10000` = **16**. 주소 = 0x2000 + 48 = 0x2030.

    쓰기 레지스터 선에는 16이 실리지만 RegWrite = 0이라 x16에 아무것도 쓰이지 않는다. sw의 RegWrite가 반드시 0이어야 하는 이유가 이것이다. [Ch4 p.21]

11. 26-1 기출 유형 주소 0x400에 있는 `beq x5, x6, 16`(바이트 오프셋 16, 기계어 `0x00628863`)에서 읽기 레지스터 1·2, [11:7], 분기 목표 주소를 쓰시오. x5 = x6 = 4이면 다음 PC는?

답 보기

바이트 오프셋 16 = `0 0 000000 1000 0`(imm[12] imm[11] imm[10:5] imm[4:1] imm[0]). SB형은 imm[0]을 저장하지 않고 imm[12|10:5]를 [31:25]에, imm[4:1|11]을 [11:7]에 넣는다.

0000000	00110	00101	000	10000	1100011
imm[12|10:5]	rs2 = 6	rs1 = 5	funct3	imm[4:1|11]	opcode

읽기 레지스터 1 = **5**, 읽기 레지스터 2 = **6**. [11:7] = `10000` = 16이 쓰기 레지스터로 가지만 RegWrite = 0이라 무시된다.

ImmGen은 분기 오프셋 필드(imm[12:1] = 8)를 부호 확장하면서 왼쪽으로 1비트 밀어 16을 만들고, 이 값이 바로 분기 가산기로 간다. 분기 가산기 출력 = 0x400 + 16 = **0x410**. [Ch4 p.15]

ALU: 4 − 4 = 0 → Zero = 1, Branch = 1 → PCSrc = 1 → 다음 PC = **0x410**. 같지 않았다면 0x404. [Ch4 p.14, p.25, p.28]

12. 예상 `sub x10, x18, x19`(기계어 `0x41390533`)에서 ALU 제어로 가는 [30]과 [14:12]의 값, ALUOp, ALU 제어 출력을 쓰시오. funct7 전체가 아니라 [30]만 보내도 되는 이유는?

답 보기

`0x41390533` = `0100 0001 0011 1001 0000 0101 0011 0011` → funct7 = `0100000`, rs2 = 19, rs1 = 18, funct3 = `000`, rd = 10, opcode = `0110011`.

[30] = 1(funct7의 둘째 비트), [14:12] = `000`, ALUOp = 10 → sub → ALU 제어 `0110`.

이 부분집합(add, sub, and, or)에서 funct7은 `0000000`과 `0100000` 둘뿐이고, 둘을 가르는 비트가 30번 하나다. [Ch4 p.18–20]

13. 21-2 기출 유형 프로세서를 구성하는 두 가지를 쓰시오.

답 보기

데이터패스(datapath)와 **제어(control)**. 프로세서 = 데이터패스 + 제어. 데이터패스는 연산을 하고, 제어는 데이터패스·메모리·입출력이 무엇을 할지 신호를 보낸다. [Ch1 p.15, Ch4 p.8]

14. 21-2 기출 유형 프로세서의 범용 레지스터 32개를 저장하고, 레지스터 번호를 지정해 읽거나 쓸 수 있는 구조의 이름은?

답 보기

레지스터 파일(register file). 파일 안의 어떤 레지스터든 번호로 지정해 읽고 쓸 수 있다. 읽기 포트 2개(읽기 레지스터 1·2)와 쓰기 포트 1개가 있다. [Ch4 p.12]

15. 예상 PCSrc가 1이 되는 조건을 쓰고, 다른 제어 신호와 무엇이 다른지 한 문장으로 쓰시오.

답 보기

명령어가 beq이고 ALU의 Zero 출력이 1일 때(PCSrc = Branch AND Zero). 다른 신호는 opcode·funct만으로 정해지지만 PCSrc는 실행 중 계산된 Zero에 따라 달라진다. [Ch4 p.25]

16. 예상 단일 사이클 데이터패스에 메모리가 두 개(명령어 메모리, 데이터 메모리)인 이유는?

답 보기

모든 명령어를 한 클럭 사이클에 실행하므로 한 명령어 안에서 어떤 자원도 두 번 쓸 수 없다. lw·sw는 같은 사이클에 명령어 인출과 데이터 접근을 둘 다 해야 하므로 메모리를 따로 둔다. [Ch4 p.16–17]

17. 예상 데이터패스의 MUX 세 개의 이름과, 각 MUX가 입력 0·1일 때 고르는 값을 쓰시오. 데이터 선에 MUX가 필요한 이유도 쓰시오.

답 보기

ALUSrc: 0 = 레지스터 읽기 데이터 2, 1 = 부호 확장 즉시값. MemtoReg: 0 = ALU 결과, 1 = 데이터 메모리 읽기 데이터. PCSrc: 0 = PC + 4, 1 = 분기 목표 주소. 슬라이드 그림에서는 MemtoReg MUX만 1이 위, 0이 아래로 적혀 있으니 위치가 아니라 숫자로 읽는다. [Ch4 p.24]

이유: 데이터 선은 그냥 묶을 수 없다(cannot just join wires together). 명령어마다 다른 출처의 값을 같은 입력에 넣으려면 MUX로 하나를 골라야 한다. [Ch4 p.7–8, p.16]

18. 예상 ImmGen의 입력과 출력은 각각 몇 비트이며, 무엇을 하는가?

답 보기

입력 32비트 명령어 전체, 출력 32비트. 명령어에서 12비트 필드를 골라 32비트 부호 있는 값으로 부호 확장한다. [Ch4 p.13]

실전 · 8 관련 단원 전 범위

T/F와 빈칸 모음

26-1 후기: "T/F는 교수님이 수업 시간에 말씀하신 것", "빈칸: 캐시 메모리, branch의 offset, encoding 이름". 21-2 후기: "T/F로 Ch1 임베디드 내용(한두 줄짜리)". 슬라이드의 정의 문장, NOT 강조 문장, 비교 문장을 그대로 또는 한 곳만 바꿔 T/F로 만들고, 핵심 용어를 빈칸으로 만들었다.

푸는 순서

1. 문장에서 **바뀌기 쉬운 곳**을 찾는다. 비교 짝(빠르다/느리다, 크다/작다, 높은/낮은 주소), 주체 짝(입력/출력, 호출자/피호출자, 하드웨어/어셈블러), 부정(NOT, 없다), 숫자(127, 4096, 18~24개월).
2. 슬라이드의 NOT 문장을 떠올린다. 예: 사용자는 컴퓨터 가까이 있을 필요 NOT, 실리콘은 전기를 잘 통하지 NOT, CPU 시간은 I/O 시간을 포함 NOT, MIPS는 ISA 차이를 반영 NOT, RISC-V는 정렬을 요구 NOT, 산술 명령은 메모리에 직접 접근 NOT, f0은 0 고정 NOT.
3. F라고 판단하면 **틀린 단어 하나**를 맞는 단어로 고쳐 쓴다. 시험이 "틀린 곳을 고치시오"로 나와도 바로 답할 수 있게 연습한다.
4. 빈칸은 영어 이름과 한국어 이름을 둘 다 떠올린다. 슬라이드가 영어라 답을 영어로 써도 되는지 시험지 지시를 확인한다.

예제

다음 문장이 맞으면 T, 틀리면 F를 쓰고 틀린 곳을 고치시오.

(1) 캐시는 DRAM으로 만들며, DRAM은 SRAM보다 빠르지만 밀도가 낮아 비싸다.

(2) 지난 20년 동안 클럭 속도는 1000배 늘었지만 전력은 30배만 늘었는데, 전압이 5 V에서 1 V로 내려갔기 때문이다.

(3) 네트워크로 연결된 컴퓨터의 사용자는 사용하는 컴퓨터 가까이에 있어야 한다.

(1) 풀이. 바뀌기 쉬운 곳은 "DRAM으로", "DRAM은 SRAM보다"다. 슬라이드: 캐시는 DRAM의 버퍼 역할을 하는 작고 빠른 메모리이고, **SRAM**으로 만든다. SRAM은 DRAM보다 빠르지만 밀도가 낮아 더 비싸다. 두 곳이 바뀌었으므로 **F**. 고쳐 쓰기: "캐시는 **SRAM**으로 만들며, **SRAM은 DRAM보다** 빠르지만 밀도가 낮아 비싸다." [Ch1 p.17]

(2) 풀이. 숫자 1000, 30, 5 V, 1 V를 확인한다. 슬라이드 문장과 같다. 전력 $\propto$ 용량성 부하 $\times$ 전압^2 $\times$ 주파수이므로 전압이 낮아지면 전력 증가가 억제된다. **T**. [Ch1 p.31]

(3) 풀이. 슬라이드의 NOT 문장 "Users need NOT be near the computer"(원격 접근, nonlocal access)를 뒤집었다. **F**. 고쳐 쓰기: "가까이에 있을 필요가 없다." [Ch1 p.19]

Ch1 컴퓨터 추상화와 기술

T/F

1. 21-2 기출 유형 T/F: 임베디드 컴퓨터는 다른 장치 안에 들어 있는 컴퓨터로, 미리 정해진 응용 프로그램을 실행한다.

   답 보기

   T. A computer inside another device used for running predetermined applications. [Ch1 p.3]

2. 21-2 기출 유형 T/F: 임베디드 컴퓨터는 전력·성능·비용 제약이 느슨한 편이다.

답 보기

F. 제약이 **엄격하다**(stringent power, performance, and cost constraints). [Ch1 p.3]

3. 21-2 기출 유형 T/F: ECU(Electronic Control Unit)는 자동차 전자 장치 안에서 차량의 전기 시스템 하나 이상을 제어하는 임베디드 시스템이다.

답 보기

T. [Ch1 p.7]

4. 26-1 기출 유형 T/F: 개인용 컴퓨터(PC)는 미리 정해진 응용만 실행하는 컴퓨터다.

답 보기

F. PC는 **범용(general purpose)**이고 다양한 소프트웨어를 실행하며 비용과 성능의 절충을 따른다. 미리 정해진 응용을 실행하는 것은 임베디드 컴퓨터다. [Ch1 p.3]

5. 26-1 기출 유형 T/F: 서버는 보통 네트워크를 통해 접근하며, 높은 용량·성능·신뢰성을 가진다.

답 보기

T. [Ch1 p.3]

6. 26-1 기출 유형 T/F: 포스트 PC 시대에 기존 서버를 대체하는 것은 클라우드 컴퓨팅이며, 이는 WSC(warehouse scale computer)로 알려져 있다.

답 보기

T. PC를 대체하는 것은 PMD(개인 휴대 기기), 서버를 대체하는 것은 클라우드 컴퓨팅이다. [Ch1 p.4]

7. 26-1 기출 유형 T/F: 입력 장치는 메모리에서 데이터를 읽고, 출력 장치는 메모리에 데이터를 쓴다.

답 보기

F. 반대다. 입력은 메모리에 데이터를 **쓰고**(writes), 출력은 메모리에서 데이터를 **읽는다**(reads). [Ch1 p.15]

8. 26-1 기출 유형 T/F: 5대 구성 요소는 데스크톱과 서버에만 해당하고 임베디드 컴퓨터는 구성이 다르다.

답 보기

F. 데스크톱, 서버, 임베디드 등 **모든 종류의 컴퓨터**가 같은 구성 요소를 가진다. [Ch1 p.15]

9. 26-1 기출 유형 T/F: ABI(Application Binary Interface)는 ISA와 운영체제 인터페이스를 합친 것이다.

답 보기

T. ABI = ISA + OS 인터페이스. [Ch1 p.17]

---

10. 26-1 기출 유형 T/F: 휘발성 메모리는 전원이 꺼져도 명령어와 데이터를 유지하며, DRAM이 그 예다.

답 보기

F. 휘발성 메모리는 전원이 꺼지면 내용을 **잃는다**. DRAM·SRAM이 휘발성(주기억)이고, 전원이 꺼져도 유지하는 것은 비휘발성(보조 기억: 하드 디스크, 플래시, 광 디스크)이다. [Ch1 p.18]

---

11. 26-1 기출 유형 T/F: LAN의 예는 인터넷이고, WAN의 예는 이더넷이다.

답 보기

F. 반대다. LAN(한 건물 정도의 좁은 지역) 예 = **이더넷**, WAN(수백 km, 대륙 규모) 예 = **인터넷**. [Ch1 p.19]

---

12. 26-1 기출 유형 T/F: 실리콘은 전기를 잘 통하기 때문에 반도체라고 부른다.

답 보기

F. 전기를 **잘 통하지 않기** 때문에(does NOT conduct electricity well) 반도체라 부른다. 특수한 화학 공정으로 도체, 절연체, 조건에 따라 통하거나 막는 영역(스위치)을 만들 수 있다. [Ch1 p.21]

---

13. 26-1 기출 유형 T/F: CPU 실행 시간(CPU 시간)에는 I/O나 다른 프로그램이 쓴 시간이 포함된다.

답 보기

F. CPU 시간은 CPU가 그 작업을 계산하는 데 쓴 시간이며, I/O나 다른 프로그램이 쓴 시간을 포함하지 **않는다**. 모든 것을 포함하는 것은 경과 시간(wall clock time, response time, elapsed time)이다. [Ch1 p.25]

---

14. 26-1 기출 유형 T/F: 프로세서를 더 빠른 것으로 바꾸면 응답 시간과 처리량이 모두 좋아지지만, 프로세서를 추가하면 처리량만 좋아진다.

답 보기

T. [Ch1 p.24]

---

15. 26-1 기출 유형 T/F: 프로그램의 명령어 수는 CPU 하드웨어가 정하고, CPI는 프로그램·ISA·컴파일러가 정한다.

답 보기

F. 반대다. 명령어 수 = 프로그램, ISA, 컴파일러가 결정. CPI = CPU 하드웨어가 결정. [Ch1 p.28, Ch4 p.2]

---

16. 26-1 기출 유형 T/F: 전력 장벽(power wall)이란 누설 때문에 전압을 더 낮출 수 없고, 냉각 비용 때문에 열을 더 제거할 수 없는 상황이다.

답 보기

T. We can NOT reduce the voltage further(leakage), We can NOT remove more heat(cooling cost). [Ch1 p.31]

17. 26-1 기출 유형 T/F: 멀티코어 마이크로프로세서의 이점은 처리량보다 응답 시간 쪽에 더 크다.

답 보기

F. 이점은 **응답 시간보다 처리량**에 더 크다(benefit is more on throughput than response time). [Ch1 p.32]

18. 26-1 기출 유형 T/F: MIPS는 컴퓨터 사이의 ISA 차이와 명령어 사이의 복잡도 차이를 반영한다.

답 보기

F. MIPS는 둘 다 반영하지 **못한다**(does NOT account for). 또 같은 CPU에서도 CPI가 프로그램마다 다르다. [Ch1 p.34]

빈칸

1. 26-1 기출 유형 (①)는 DRAM의 버퍼 역할을 하는 작고 빠른 메모리이며, (②)으로 만든다. (②)은 DRAM보다 빠르지만 밀도가 낮아 더 비싸다.

답 보기

① 캐시 메모리(cache memory) ② SRAM(static random access memory). 주기억장치는 DRAM 칩으로 만든다. SRAM과 DRAM은 메모리 계층의 두 층이다. [Ch1 p.17]

2. 예상 하드웨어와 최하위 소프트웨어 사이 인터페이스의 추상화를 (①)라 하고, (①)에 운영체제 인터페이스를 더한 것을 (②)라 한다. 아키텍처 추상화를 따르는 하드웨어를 (③)이라 한다.

답 보기

① 명령어 집합 구조(ISA, instruction set architecture), 간단히 아키텍처 ② ABI(application binary interface) ③ 구현(implementation). [Ch1 p.17]

3. 21-2 기출 유형 컴퓨터의 5대 고전적 구성 요소는 입력, 출력, (①), (②), (③)이며, 이 중 (②)와 (③)을 합친 것이 프로세서다.

답 보기

① 메모리 ② 데이터패스 ③ 제어(②·③ 순서 바뀌어도 됨). [Ch1 p.15]

4. 예상 컴퓨터 구조의 8대 아이디어 중, 드문 경우를 최적화하는 것보다 성능 향상이 큰 것은 (①), 여분 부품을 넣어 시스템을 믿을 만하게 만드는 것은 (②), 병렬성의 특정한 한 형태는 (③), 집적회로 자원이 18~24개월마다 2배가 되는 것을 고려하는 것은 (④)이다.

   답 보기

   ① 자주 생기는 경우를 빠르게(Make the common case fast) ② 중복을 통한 신뢰성(Dependability via redundancy) ③ 파이프라이닝을 통한 성능(Performance via pipelining) ④ 무어의 법칙을 고려한 설계(Design for Moore's Law). [Ch1 p.12]

5. 예상 지리적으로 좁은 지역(보통 한 건물) 안에서 데이터를 나르는 네트워크는 (①), 수백 km 이상 대륙에 걸치는 네트워크는 (②)다.

   답 보기

   ① LAN(local area network, 예: 이더넷) ② WAN(wide area network, 예: 인터넷). [Ch1 p.19]

6. 예상 개인 사용자는 (①)을 줄이는 데 관심이 있고, 데이터센터 관리자는 (②)을 늘리는 데 관심이 있다.

   답 보기

   ① 응답 시간(response time, 실행 시간) ② 처리량(throughput, 대역폭). [Ch1 p.24]

7. 21-2 기출 유형 컴퓨터 성능을 결정하는 세 가지: CPU 시간 = (①) × (②) × (③).

   답 보기

   ① 명령어 수(instruction count) ② CPI(명령어당 클럭 사이클 수) ③ 클럭 주기(clock cycle time). ③ 대신 "÷ 클럭 속도"로 써도 같다. [Ch1 p.28–29, Ch4 p.2]

Ch2 명령어: 컴퓨터의 언어

T/F

1. 26-1 기출 유형 T/F: 프로세서마다 ISA가 다르지만, 비슷한 하드웨어 기술과 모든 컴퓨터가 제공해야 하는 기본 연산 때문에 ISA끼리 서로 비슷하다.

   답 보기

   T. [Ch2 p.3]

2. 26-1 기출 유형 T/F: CISC는 CPU 설계가 쉽고, RISC는 어셈블리 프로그래밍이 쉽다.

답 보기

F. CISC의 단점이 **CPU 설계가 훨씬 어렵다**는 것이고 장점이 어셈블리 프로그래밍이 쉽다는 것이다. RISC의 장점이 CPU 설계가 쉽다는 것이다. [Ch2 p.6–7]

---

3. 26-1 기출 유형 T/F: 현대 x86(CISC) 프로세서의 대부분은 RISC 기법으로 구현된다.

답 보기

T. [Ch2 p.7]

---

4. 26-1 기출 유형 T/F: RISC-V에서 64비트 데이터를 워드(word)라 부른다.

답 보기

F. 32비트가 **워드**, 64비트는 **더블워드**(doubleword)다. [Ch2 p.10]

---

5. 26-1 기출 유형 T/F: RISC-V에서는 산술 명령어가 메모리의 데이터에 직접 접근할 수 있다.

답 보기

F. 직접 접근할 수 **없다**(can NOT). 메모리 데이터를 다루려면 load로 레지스터에 가져오고 store로 되돌려야 한다. [Ch2 p.15]

---

6. 26-1 기출 유형 T/F: RISC-V는 빅 엔디언이며, 워드가 4의 배수 주소에서 시작해야 한다.

답 보기

F. 두 곳이 틀렸다. RISC-V는 **리틀 엔디언**이고, 워드 정렬을 요구하지 **않는다**(no alignment restriction). [Ch2 p.13]

---

7. 26-1 기출 유형 T/F: 즉시값 피연산자를 쓰는 명령어는 상수를 메모리에서 읽어 오는 것보다 훨씬 빠르고 에너지도 적게 든다.

답 보기

T. [Ch2 p.16]

---

8. 26-1 기출 유형 T/F: sra/srai는 오른쪽으로 시프트하고 빈 자리를 0으로 채운다.

답 보기

F. 빈 자리를 **부호 비트**로 채운다(예 1100 0011 → 1110 0001). 0으로 채우는 것은 srl이다. [Ch2 p.25]

---

9. 26-1 기출 유형 T/F: AND 명령어는 일부 비트를 1로 설정하는 데 쓰고, OR 명령어는 일부 비트만 골라 나머지를 0으로 지우는 마스크에 쓴다.

답 보기

F. 반대다. **AND** = 마스크(일부 비트를 고르고 나머지를 0으로), **OR** = 일부 비트를 1로 설정하고 나머지는 그대로. [Ch2 p.26]

10. 26-1 기출 유형 T/F: RISC-V에서 NOT 연산은 모든 비트가 1인 값과 XOR해서 얻을 수 있다.

답 보기

T. [Ch2 p.27]

11. 26-1 기출 유형 T/F: rs1 = 0xFFFFFFFF, rs2 = 1일 때 blt는 분기하지만 bltu는 분기하지 않는다.

답 보기

T. 부호 있는 비교로는 −1 < 1(참), 부호 없는 비교로는 $2^{32} - 1 < 1$(거짓). [Ch2 p.31]

12. 26-1 기출 유형 T/F: 스택은 낮은 주소에서 높은 주소로 자라며, RISC-V에는 push·pop 명령어가 따로 있다.

답 보기

F. 두 곳이 틀렸다. 스택은 **높은 주소에서 낮은 주소로** 자라고, 명시적 push·pop 명령어가 **없다**(load·store로 접근). [Ch2 p.37]

13. 26-1 기출 유형 T/F: x5~x7과 x28~x31은 피호출자(callee)가 보존해야 하는 레지스터다.

답 보기

F. x5~x7, x28~x31은 **호출자 저장(caller saved)** 임시 레지스터로, 피호출자가 보존하지 **않는다**. 피호출자가 보존하는 것은 x8~x9, x18~x27이다. [Ch2 p.39]

14. 26-1 기출 유형 T/F: 스택과 힙은 저장 공간을 최대한 쓰도록 서로를 향해 자란다.

답 보기

T. [Ch2 p.45]

15. 26-1 기출 유형 T/F: SB형 분기 명령어는 −4096 ~ 4094 범위의 2의 배수 주소로만 분기할 수 있어, 짝수 주소로만 분기한다.

답 보기

T. [Ch2 p.47]

16. 26-1 기출 유형 T/F: 하나의 연산은 한 가지 주소 지정 방식만 쓸 수 있다.

답 보기

F. 한 연산이 여러 주소 지정 방식을 쓸 수 있다. 예: 덧셈 = add(레지스터) / addi(즉시값). [Ch2 p.49]

17. 26-1 기출 유형 T/F: S형에서 12비트 즉시값을 두 필드로 나누는 것은 rs1·rs2 필드를 모든 형식에서 같은 자리에 두기 위해서다.

답 보기

T. opcode와 funct3도 모든 형식에서 크기·위치가 같다. [Ch2 p.24]

빈칸

1. 26-1 기출 유형 명령어의 배치를 명령어 형식이라 한다. 대부분의 산술·논리 명령어(즉시값 제외)는 (①), load와 즉시값 산술은 (②), store는 (③), 조건 분기는 (④), jal은 (⑤), lui는 (⑥)를 쓴다.

답 보기

① R형 ② I형 ③ S형 ④ SB형 ⑤ UJ형 ⑥ U형. [Ch2 p.21, p.46–47]

2. 26-1 기출 유형 분기 명령어는 (①)형 형식을 쓰며, 분기 목표 주소 = (②) + 분기 오프셋이다. 이런 방식을 (③) 주소 지정이라 한다. 오프셋은 (④)비트 즉시값이고 하드웨어에서 왼쪽으로 (⑤)비트 시프트해 쓴다.

답 보기

① SB ② PC ③ PC 상대(PC-relative) ④ 12 ⑤ 1(하프워드 변위). 범위는 −4096 ~ 4094 바이트, word로 $\pm 2^{10}$.
[Ch2 p.47–48, Ch4 p.14]

3. 예상 주소 지정 방식: `addi`는 (①) 주소 지정, `add`는 (②) 주소 지정, `lw`·`sw`는 (③) 주소 지정, `beq`·`jal`은 (④) 주소 지정을 쓴다.

답 보기

① 즉시값(immediate) ② 레지스터(register) ③ 베이스 또는 변위(base or displacement) ④ PC 상대(PC-relative). [Ch2 p.49]

4. 예상 설계 원칙 1: (①). 설계 원칙 2: (②). 설계 원칙 3: (③).

답 보기

① 단순성은 규칙성을 선호한다(Simplicity favors regularity) ② 작을수록 빠르다(Smaller is faster) ③ 좋은 설계에는 좋은 타협이 필요하다(Good design demands good compromises). [Ch2 p.9, p.10, p.23]

5. 예상 RISC-V에서 스택 포인터는 (①), 복귀 주소 레지스터는 (②), 프레임 포인터는 (③), 상수 0에 고정된 레지스터는 (④)이다. 인자 레지스터는 (⑤)이고 그중 반환값은 (⑥)에 둔다.

   답 보기

   ① x2(sp) ② x1 ③ x8(fp) ④ x0 ⑤ x10~x17 ⑥ x10, x11. [Ch2 p.16, p.34, p.37, p.44]

6. 예상 레지스터 수가 부족해 레지스터 값을 메모리로 옮겨 두는 것을 (①)이라 하고, 그 이상적인 자료 구조는 (②)이다. 프로시저의 저장 레지스터와 지역 변수를 담은 스택 구간을 (③) 또는 활성 레코드라 한다.

   답 보기

   ① 레지스터 스필링(register spilling) ② 스택(stack, LIFO) ③ 프로시저 프레임(procedure frame). [Ch2 p.37, p.44]

7. 예상 2의 보수에서 −x = (①). 비트 수를 늘릴 때 값을 보존하려고 부호 비트를 왼쪽으로 복제하는 것을 (②)이라 한다.

   답 보기

   ① ~x + 1(모든 비트 반전 후 1 더하기) ② 부호 확장(sign extension). [Ch2 p.20]

Ch3 컴퓨터 산술

T/F

1. 26-1 기출 유형 T/F: 부호가 서로 다른 두 수를 더할 때는 오버플로가 생기지 않는다.

   답 보기

   T. 합이 두 피연산자 중 하나보다 클 수 없기 때문이다(예 −10 < −6 < 4). [Ch3 p.4]

2. 26-1 기출 유형 T/F: 부호가 같은 두 수를 뺄 때 오버플로가 생길 수 있다.

   답 보기

   F. 부호가 같은 뺄셈은 부호가 다른 덧셈과 같으므로 오버플로가 생기지 **않는다**. [Ch3 p.4]

3. 26-1 기출 유형 T/F: C/C++와 Java는 정수 오버플로를 무시하고, Ada와 Fortran은 프로그램에 알리도록 요구한다.

   답 보기

   T. [Ch3 p.6]

4. 26-1 기출 유형 T/F: 3비트 곱셈 예에서 부호 없는·부호 있는·부호 다른 모드에 따라 아래 3비트가 달라지고 위 3비트는 같다.

   답 보기

   F. 반대다. **아래(lower) 비트는 같고 위(upper) 비트가 모드마다 다르다**. 그래서 mul 하나와 mulh·mulhu·mulhsu 셋이 있다. [Ch3 p.13]

5. 26-1 기출 유형 T/F: 나눗셈은 곱셈처럼 병렬 하드웨어로 빠르게 만들 수 있다.

   답 보기

   F. 나눗셈의 뺄셈은 나머지 부호에 따라 조건부라 곱셈처럼 병렬 하드웨어를 쓸 수 **없다**. 대신 SRT 나눗셈이 표를 보고 몫 여러 비트를 예측한다. [Ch3 p.20]

6. 26-1 기출 유형 T/F: 부호 있는 나눗셈에서 나머지의 부호는 제수의 부호와 같게 한다.

   답 보기

   F. 나머지 부호는 **피제수**의 부호와 같다. 예 −7 ÷ 2 → 몫 −3, 나머지 −1. [Ch3 p.19]

7. 26-1 기출 유형 T/F: 부동소수점에서 가수(fraction) 크기를 늘리면 표현 범위가 넓어지고, 지수 크기를 늘리면 정밀도가 높아진다.

   답 보기

   F. 반대다. 가수를 늘리면 **정밀도**가, 지수를 늘리면 **범위**가 커진다. [Ch3 p.25]

8. 26-1 기출 유형 T/F: IEEE 754 단정도에서 지수 필드 00000000과 11111111은 예약되어 있다.

   답 보기

   T. 그래서 정규화 수의 지수 칸 값(바이어스된 지수)은 1 ~ 254(실제 지수 −126 ~ 127)다. [Ch3 p.28, p.30]

9. 26-1 기출 유형 T/F: 부동소수점 곱셈에서 두 수의 바이어스된 지수를 그대로 더하면 곱의 바이어스된 지수가 된다.

   답 보기

   F. 그대로 더하면 bias가 두 번 들어간다. 더한 뒤 **bias를 한 번 빼야** 한다: (−1 + 127) + (−2 + 127) − 127 = −3 + 127. [Ch3 p.36]

10. 26-1 기출 유형 T/F: IEEE 754의 기본 반올림 모드는 0 쪽으로 버리기(truncation)다.

    답 보기

    F. 기본 모드는 **가장 가까운 짝수로 반올림**(round to nearest even)이다. 네 모드: 0 쪽, +∞ 쪽, −∞ 쪽, 가장 가까운 짝수. [Ch3 p.39]

11. 26-1 기출 유형 T/F: RISC-V의 부동소수점 레지스터 f0은 x0처럼 상수 0에 고정되어 있다.

답 보기

F. f0은 상수 0에 고정되어 있지 **않다**(f0 is NOT hardwired to constant 0). [Ch3 p.37]

12. 26-1 기출 유형 T/F: feq·flt·fle 같은 부동소수점 비교 명령어는 결과(0 또는 1)를 정수 레지스터에 쓰고, 그 결과로 beq·bne를 쓸 수 있다.

답 보기

T. [Ch3 p.37]

빈칸

1. 예상 IEEE 754 값 = $(-1)^{S} \times (1 + (\ ①\ )) \times 2^{(E - (\ ②\ ))}$. (②)는 단정도 (③), 배정도 (④)이다. 16비트(부호 1, 지수 5, 가수 10) 형식이면 (②) = (⑤)이다.

답 보기

① Fraction(가수) ② Bias ③ 127 ④ 1023 ⑤ $2^{5-1} - 1 = 15$. 지수를 2의 보수로 두지 않고 bias를 더한 부호 없는 값으로 두는 이유는 정수 비교로 정렬하기 쉽게 하려는 것이다. [Ch3 p.28–29]

2. 예상 IEEE 754는 반올림을 위해 세 비트를 둔다. 가수 LSB 바로 다음 비트는 (①), 그다음 비트는 (②), 그 뒤 나머지 모든 비트를 OR한 것은 (③) 비트다.

답 보기

① 가드(guard) ② 라운드(round) ③ 스티키(sticky). GRS가 000~011이면 내림, 101~111이면 올림, 100(정확히 절반)이면 LSB가 0인 쪽으로. [Ch3 p.40]

3. 예상 지수가 너무 커서 지수 필드에 담을 수 없는 것은 (①), 음의 지수가 너무 커서 담을 수 없는 것은 (②)다. 이를 줄이려면 C의 double에 해당하는 64비트 (③) 형식을 쓴다.

답 보기

① 오버플로(overflow) ② 언더플로(underflow) ③ 배정도(double precision). [Ch3 p.26]

4. 21-2 기출 유형 RISC-V 곱셈 명령: 64비트 곱의 아래 32비트는 (①), 두 피연산자가 모두 부호 있을 때 위 32비트는 (②), 모두 부호 없을 때 위 32비트는 (③), 하나는 부호 있고 하나는 부호 없을 때 위 32비트는 (④).

답 보기

① mul ② mulh ③ mulhu ④ mulhsu. [Ch3 p.13]

5. 예상 나눗셈의 관계식: 피제수 = (①) × (②) + (③). (②)는 0이 아니어야 한다.

답 보기

① 몫(quotient) ② 제수(divisor) ③ 나머지(remainder). [Ch3 p.14]

Ch4 프로세서 (단일 사이클까지)

T/F

1. 26-1 기출 유형 T/F: 명령어 수는 ISA와 컴파일러가, 클럭 주기와 CPI는 CPU 하드웨어가 결정한다.

답 보기

T. 여기서 CPU 하드웨어는 마이크로아키텍처, 공정 기술 등을 말한다. [Ch4 p.2]

2. 26-1 기출 유형 T/F: 모든 명령어의 처음 두 단계는 같다. PC를 명령어 메모리로 보내 명령어를 인출하고, 명령어 필드로 레지스터 한두 개를 읽는다.

답 보기

T. 그다음 동작은 명령어 종류에 따라 다르다. [Ch4 p.3]

3. 26-1 기출 유형 T/F: 데이터 선은 그냥 묶어서 연결할 수 있으므로 데이터패스에 MUX가 필요 없다.

답 보기

F. 데이터 선은 묶을 수 **없으므로** MUX가 필요하다. 또 데이터패스를 제어할 제어 유닛도 필요하다. [Ch4 p.7]

4. 26-1 기출 유형 T/F: 레지스터나 메모리처럼 정보를 저장하는 소자를 조합 소자라 한다.

답 보기

F. 저장하는 소자는 **상태 소자**(state element)다. 조합 소자(combinational element)는 AND 게이트·ALU처럼 출력이 입력만의 함수인 연산 소자다. [Ch4 p.9]

5. 26-1 기출 유형 T/F: 상태 소자는 적어도 입력 두 개(쓸 데이터 값, 클럭)와 출력 하나를 가지며, 출력은 이전 클럭 사이클에 쓴 값이다.

답 보기

T. [Ch4 p.9]

6. 26-1 기출 유형 T/F: 클럭 주기는 조합 논리의 가장 짧은 지연으로 정해진다.

   답 보기

   F. **가장 긴** 지연이 클럭 주기를 정한다(the longest delay determines clock period). [Ch4 p.10]

7. 26-1 기출 유형 T/F: 단일 사이클 데이터패스에서는 한 명령어가 같은 자원을 두 번 쓸 수 있으므로 메모리 하나로 충분하다.

   답 보기

   F. 어떤 자원도 명령어당 두 번 이상 쓸 수 **없으므로** 명령어 메모리와 데이터 메모리를 따로 둔다. [Ch4 p.16]

8. 26-1 기출 유형 T/F: 다단 디코딩은 주 제어 유닛의 크기를 줄일 수 있고, 작은 제어 유닛 여러 개를 쓰면 제어 유닛의 지연도 줄일 수 있다.

   답 보기

   T. [Ch4 p.19]

9. 26-1 기출 유형 T/F: 제어 유닛은 모든 제어 신호를 opcode와 funct 필드만으로 정할 수 있다.

   답 보기

   F. 하나를 뺀 모두(all but one)를 정할 수 있다. 예외는 **PCSrc**로, beq이고 ALU Zero가 1일 때 활성화된다. [Ch4 p.25]

10. 26-1 기출 유형 T/F: SB·UJ 형식의 즉시값 비트 뒤섞기는 하드웨어를 단순하게 하며, 그 대가는 어셈블러만 진다.

    답 보기

    T. 즉시값 19~12번 비트 MUX가 3:1 → 2:1, 10~1번이 5/4:1 → 3/2:1로 준다. [Ch4 p.22–23]

11. 26-1 기출 유형 T/F: 단일 사이클 구현은 명령어마다 클럭 주기 길이를 다르게 할 수 있어 효율적이다.

    답 보기

    F. 모든 명령어에 클럭 주기 길이가 **같아야** 하고, 가장 긴 경로(load: 명령어 메모리 → 레지스터 파일 → ALU → 데이터 메모리 → 레지스터 파일)가 주기를 정한다. 그래서 비효율적이며 "자주 생기는 경우를 빠르게" 원칙에 어긋난다. [Ch4 p.29]

12. 26-1 기출 유형 T/F: rd 필드는 R형과 store 명령어의 목적지 레지스터다.

    답 보기

    F. rd는 R형과 **load**의 목적지다. store에서 rs2는 메모리로 복사될 레지스터다. [Ch4 p.21]

빈칸

1. 예상 데이터패스의 논리 소자는 두 종류다. 출력이 입력의 함수인 (①) 소자(예: AND 게이트, ALU)와 정보를 저장하는 (②) 소자(예: 레지스터, 메모리).

   답 보기

   ① 조합(combinational) ② 상태(state). [Ch4 p.9]

2. 예상 순차 논리 소자에 저장된 값이 클럭 엣지에서만 갱신되는 클로킹 방식을 (①) 클로킹이라 한다. 클로킹 방식은 신호를 언제 읽고 언제 쓸 수 있는지 정해 하드웨어를 (②) 가능하게 만든다.

   답 보기

   ① 엣지 트리거(edge-triggered) ② 예측(predictable). [Ch4 p.10]

3. 예상 명령어 실행을 시작하는 데 필요한 세 요소: 명령어를 저장하는 (①), 현재 명령어 주소를 담는 (②), 그 주소를 다음 명령어 주소로 늘리는 (③).

   답 보기

   ① 명령어 메모리 ② PC(프로그램 카운터) ③ 가산기(adder, PC + 4). [Ch4 p.11]

4. 예상 ALU 제어 4비트 입력은 명령어의 (①)·(②) 필드와 주 제어 유닛이 만드는 2비트 (③)로 만든다. 입력 조합이 2^{12} = (④)개라 진리표에서 출력에 상관없는 입력을 X, 즉 (⑤)로 표시한다.

   답 보기

   ① funct7 ② funct3 ③ ALUOp ④ 4096 ⑤ don't care. [Ch4 p.19–20]

5. 예상 분기 데이터패스는 두 일을 한다. ImmGen과 (①)로 분기 목표 주소를 계산하고, ALU로 두 레지스터를 (②)해 (③) 출력을 검사한다.

   답 보기

   ① 가산기(분기 가산기) ② 뺄셈(subtract) ③ Zero. [Ch4 p.15]

함정

- Ch3 p.25 슬라이드는 지수를 "2's complement"로 적었지만 IEEE 754(p.28)는 bias를 쓴다. "IEEE 754 지수 표현" 문제에는 bias로 답한다.
- "반대로 바꾼" 문장(입력/출력, AND/OR, LAN/WAN, 명령어 수/CPI, 가수/지수)이 가장 흔한 F 유형이다. 짝을 지어 외운다.
- T/F에 "모든", "항상", "만"이 들어 있으면 슬라이드에 같은 강도의 표현이 있는지 확인한다. 예: PCSrc는 "all but one"이라 "모든 신호"는 F.

시험 포인트

- Ch1 임베디드·PC·서버 정의, PMD·클라우드 [Ch1 p.3–4, p.7]
- 슬라이드 NOT 문장들: Ch1 p.19·21·25·31·34, Ch2 p.13·15·37·39, Ch3 p.37, Ch4 p.7
- 빈칸 단골: 캐시 = SRAM 버퍼, 형식 이름(R·I·S·SB·UJ·U), 분기 오프셋 = PC 상대·1비트 시프트

모의고사 · 1회 90분 · 100점

모의고사 1회

타이머를 90분에 맞추고 노트를 덮은 채 푼다. 계산 문제는 풀이 과정도 칸에 쓴다(실제 시험도 과정 점수가 있다). 10문항에 모두 답을 쓴 뒤 맨 아래 버튼을 누르면 정답과 채점 기준이 열린다. 채점 기준으로 스스로 점수를 매기고, 깎인 문항의 개념은 '불안한 개념 목록'에 적는다.

1. **12점** 다음 문장이 맞으면 O, 틀리면 X를 쓰고, X인 문장은 틀린 곳을 바르게 고치시오. (각 2점)

(1) 칩 하나에 수십~수백 개의 트랜지스터를 담은 것을 VLSI라 하고, 수백~수백만 개를 담은 것을 집적 회로(IC)라 한다.

(2) 지난 20년 동안 클럭 속도는 약 1000배 늘었지만 전력은 약 30배만 늘었다. 전압이 5 V에서 1 V로 낮아졌기 때문이다.

(3) `lui`는 20비트 상수를 레지스터의 0번~19번 비트에 넣고, 나머지 12비트는 0으로 채운다.

(4) `jalr x0, 0(x1)`은 `x1`에 저장된 주소로 분기하며, 목적지 레지스터로 `x0`을 써서 복귀 주소를 버린다.

(5) IEEE 754 단정도에서 bias는 127이고, 지수 칸 `00000000`과 `11111111`은 예약되어 있어 정규화 수의 실제 지수 범위는 −126 ~ 127이다.

(6) 조합 소자(combinational element)는 클럭 엣지에서만 값을 바꾸고, 상태 소자(state element)의 출력은 현재 입력만의 함수이다.

정답과 채점 기준

(1) **X**. 둘이 바뀌었다. 집적 회로(IC)가 수십~수백 개, VLSI(Very Large-Scale IC)가 수백~수백만 개다.

(2) **O**. 동적 전력은 전압의 제곱에 비례하므로 전압을 낮춘 덕에 전력 증가가 30배에 그쳤다.

(3) **X**. `lui`는 20비트 상수를 **12번~31번 비트**(상위 20비트)에 넣고, 오른쪽(하위) 12비트를 0으로 채운다.

(4) **O**.

(5) **O**. 지수 칸 1 ~ 254에서 127을 빼면 −126 ~ 127이다.

(6) **X**. 설명이 서로 바뀌었다. 조합 소자(AND 게이트, ALU)의 출력은 입력의 함수이고, 상태 소자(레지스터, 메모리)는 에지 트리거 방식에서 클럭 엣지에서만 저장값이 바뀐다.

- **2점** O인 문장 (2)(4)(5): O만 맞게 쓰면 2점.
- **2점** X인 문장 (1)(3)(6): X 1점 + 바르게 고친 내용 1점. 고친 내용이 "틀렸다"만이면 고침 점수 없음.
- **0점** O/X가 틀리면 그 문장은 0점.

(3)을 O로 고르는 경우가 많다. "20비트"라는 숫자가 맞아서 위치까지 맞다고 착각하기 쉽다. lui의 U는 upper다. (6)은 "출력이 입력의 함수"라는 표현이 어느 소자 설명인지 거꾸로 외우면 틀린다. 상태 소자는 이전 클럭에 써 둔 값을 내보낸다.

추가 점수를 받으려면: (2)에 동적 전력 $\propto$ 용량성 부하 $\times$ 전압^2 $\times$ 주파수 식을 함께 쓰고, (3)에 "큰 상수는 lui로 상위 20비트를 넣은 뒤 addi로 하위 12비트를 더한다"를 덧붙인다.

근거 [Ch1 p.20, p.31] [Ch2 p.36, p.46] [Ch3 p.28, p.30] [Ch4 p.9–10]

2. **10점** 빈칸 ①~⑤를 채우시오. (각 2점)

(1) 캐시 메모리는 DRAM의 버퍼 역할을 하는 작고 빠른 메모리이며 (①)으로 만든다. (①)과 DRAM은 메모리 (②)의 두 층이다.

(2) `beq x1, x2, offset`에서 offset은 명령어 안의 (③)비트 필드이다. 분기 목표 주소를 만들 때 이 값을 부호 확장하고 왼쪽으로 (④)비트 시프트한 뒤 PC에 더한다.

(3) `jal`이 쓰는 명령어 형식의 이름은 (⑤)형이다.

정답과 채점 기준

① **SRAM**(static random access memory) ② **계층**(memory hierarchy) ③ **12** ④ **1** ⑤ **UJ**

- **2점** 빈칸 하나당. ①은 "SRAM" 또는 "정적 RAM". ②는 "계층"이나 "hierarchy".
- **0점** ④에 "2"를 쓴 경우(word 단위로 착각).
- **1점** ⑤에 "J형"이라고 쓴 경우(강의 표기는 UJ).

①에 DRAM을 쓰는 실수가 많다. 캐시가 DRAM의 버퍼이므로 다른 재료다. SRAM은 DRAM보다 빠르지만 밀도가 낮아 비싸다. ④를 2로 쓰는 것은 MIPS(옛 ISA)처럼 4바이트 단위로 생각한 것이다. RISC-V 분기는 halfword 단위라 1비트만 시프트하고, 그래서 짝수 주소로만 분기한다.

추가 점수를 받으려면: ③④에 이어 "그래서 분기 범위는 −4096 ~ 4094바이트, 2의 배수"를 쓰고, ⑤에 "jal은 20비트 즉시값이라 분기보다 범위가 넓다($\pm 2^{18}$ words)"를 덧붙인다.

근거 [Ch1 p.17] [Ch4 p.14] [Ch2 p.47–48]

3. **10점** 클럭 속도 2.5 GHz인 컴퓨터에서 어떤 프로그램이 명령어 2×10^9개를 실행한다. 명령어 비율과 CPI는 다음과 같다.

종류	ALU 연산	load	store	branch
비율	50%	20%	10%	20%
CPI	1	4	3	2

(1) 평균 CPI (2점) (2) CPU 시간(초) (2점) (3) MIPS (2점) (4) 캐시를 개선해 load의 CPI만 4에서 2로 줄이면 몇 배 빨라지는가? (2점) (5) load에 걸리는 시간을 0에 가깝게 줄여도 전체 성능은 최대 몇 배까지만 좋아지는가? (2점)

정답과 채점 기준

(1) 평균 CPI = 0.5 × 1 + 0.2 × 4 + 0.1 × 3 + 0.2 × 2 = 0.5 + 0.8 + 0.3 + 0.4 = **2.0**

(2) CPU 시간 = 명령어 수 × CPI ÷ 클럭 속도 = $2 \times 10^9 \times 2.0 \div (2.5 \times 10^9)$ = **1.6초**

(3) MIPS = 클럭 속도 ÷ (CPI × 10^6) = $2.5 \times 10^9 \div (2.0 \times 10^6)$ = **1250**. 검산: 명령어 수 ÷ (실행 시간 × 10^6) = $2 \times 10^9 \div (1.6 \times 10^6)$ = 1250.

(4) 새 CPI = 0.5 + 0.2 × 2 + 0.3 + 0.4 = 1.6. 새 시간 = $2 \times 10^9 \times 1.6 \div (2.5 \times 10^9)$ = 1.28초. 1.6 ÷ 1.28 = **1.25배**. (명령어 수와 클럭이 같으므로 CPI 비 2.0 ÷ 1.6으로 바로 구해도 된다.)

(5) load가 차지하는 사이클 비율 = 0.8 ÷ 2.0 = 40%. 암달의 법칙으로 개선 후 시간 = 0.4T ÷ n + 0.6T. n이 아무리 커져도 0.6T보다 작아지지 않으므로 최대 1 ÷ 0.6 ≈ **1.67배**.

- **2점** 소문항마다 식 1점 + 값 1점.
- **주의** (3)에서 1.25×10^9이나 "1250 MIPS/s"처럼 단위를 섞어 쓰면 값 점수 없음. MIPS는 이미 "초당 백만 명령어"라 숫자만 쓴다.
- **1점** (5)에서 비율을 명령어 비율 20%로 잡아 1.25배라고 쓰면 식 점수만.

(5)에서 가장 흔한 오답은 명령어 비율 20%를 그대로 써서 1 ÷ 0.8 = 1.25배라고 하는 것이다. 암달의 법칙은 **시간**(사이클)의 비율로 계산해야 한다. load는 명령어의 20%지만 CPI가 커서 사이클의 40%를 차지한다. (3)에서 10^6으로 나누는 것을 빼먹으면 1.25×10^9이 나온다.

추가 점수를 받으려면: (4)의 1.25배와 (5)의 최대 1.67배를 비교해 "흔한 경우를 빠르게(make the common case fast)"와 연결하고, MIPS는 ISA 차이와 명령어 복잡도 차이를 반영하지 못한다는 한계를 한 줄 덧붙인다.

근거 [Ch1 p.28–29, p.33–34]

4. **10점** 다음 C 코드를 RISC-V 어셈블리로 바꾸시오. `cnt`, `i`, `n`, `k`는 각각 `x19`, `x20`, `x21`, `x22`에 있고, int 배열 `A`의 시작 주소는 `x25`에 있다. 임시 레지스터는 `x5`, `x6`을 쓴다.

```
cnt = 0;
for (i = 0; i < n; i = i + 1)
    if (A[i] < k)
        cnt = cnt + 1;
```

(1) 어셈블리 코드 (6점) (2) n = 4, A = {3, −2, 7, 0}, k = 3일 때 끝난 뒤 `cnt`의 값 (2점) (3) 작성한 코드에서 A[i]와 k를 비교하는 분기를 부호 없는 비교 명령어(`bgeu` 또는 `bltu`)로 바꾸면 (2)의 결과는 어떻게 되는가? (2점)

정답과 채점 기준

```
      addi x19, x0, 0       // cnt = 0
      addi x20, x0, 0       // i = 0
Loop: bge  x20, x21, Exit   // i >= n 이면 반복 끝
      slli x5, x20, 2       // x5 = i * 4 (int는 4바이트)
      add  x5, x5, x25      // x5 = A[i]의 주소
      lw   x6, 0(x5)        // x6 = A[i]
      bge  x6, x22, Next    // A[i] >= k 이면 cnt 증가를 건너뜀
      addi x19, x19, 1      // cnt = cnt + 1
Next: addi x20, x20, 1      // i = i + 1
      beq  x0, x0, Loop     // 무조건 Loop로
Exit:
```

(2) A[i] < 3인 원소는 −2와 0 두 개 → **cnt = 2**.

(3) `bgeu x6, x22, Next` 로 바꾸면 −2(0xFFFFFFFE)를 부호 없는 수 4294967294로 본다. 4294967294 ≥ 3이므로 건너뛴다. 0만 세어져 **cnt = 1**로 바뀐다.

- **1점** cnt, i 초기화(`addi x?, x0, 0`).
- **2점** 주소 계산: i × 4(`slli ..., 2`)와 시작 주소 더하기. ×4를 빼먹으면 0점.
- **1점** `lw` 로 A[i] 읽기.
- **1점** 조건 분기: C의 조건(A[i] < k)을 뒤집어 `bge` 로 건너뛰거나, `blt` 로 증가 쪽으로 가는 구조가 맞을 것.
- **1점** i 증가 후 Loop로 돌아가기, 반복 시작에서 i >= n 검사.
- **2점** (2) 값. **2점** (3) 값과 "−2를 큰 양수로 본다"는 이유.

자주 나오는 오답: (a) `lw x6, i(x25)` 처럼 오프셋 자리에 레지스터를 넣는다. 오프셋은 상수만 된다. (b) `slli` 없이 `add x5, x20, x25` 로 i를 바이트 주소로 착각한다. (c) 조건을 뒤집지 않고 `blt x6, x22, Next` 로 써서 A[i] < k일 때 오히려 건너뛴다.

추가 점수를 받으려면: 반복 끝 검사를 아래로 옮겨 반복마다 분기 수를 줄이는 방법(조건을 처음 한 번 검사 후 끝에서 `blt x20, x21, Loop`)을 함께 쓰고, (3)에서 Ch2 p.31 예(rs1 = 0xFFFFFFFF, rs2 = 1일 때 blt 참, bltu 거짓)를 언급한다.

근거 **[Ch2 p.14, p.28–31]**

5. **10점** RISC-V(리틀 엔디언) 메모리에 다음 바이트가 있고 `x10` = 0x00000100이다.

주소	0x100	0x101	0x102	0x103	0x104	0x105	0x106	0x107
바이트	9C	FF	00	80	21	43	65	87

```
lw   x5, 0(x10)
lw   x6, 4(x10)
srai x7, x5, 8
add  x28, x5, x5
sw   x7, 8(x10)
```

(1) `x5`, `x6`의 값(16진수 8자리) (2점) (2) `x7`의 값 (2점) (3) `x28`의 값과, 부호 있는 수로 볼 때 이 덧셈에서 오버플로가 생겼는지와 그 판단 근거 (3점) (4) 실행 후 주소 0x108 ~ 0x10B의 바이트 (2점) (5) 같은 바이트를 빅 엔디언 기계가 `lw x5, 0(x10)`으로 읽었다면 `x5`는? (1점)

정답과 채점 기준

(1) 리틀 엔디언은 가장 낮은 주소에 최하위 바이트가 있으므로 주소가 큰 쪽부터 왼쪽에 쓴다. x5 = **0x8000FF9C**, x6 = **0x87654321**.

(2) `srai`는 오른쪽 8비트(16진수 두 자리) 시프트, 빈자리는 부호 비트로 채운다. x5의 부호 비트는 1(16진 숫자 8 = 1000_2)이므로 FF가 들어온다. x7 = **0xFF8000FF**.

(3) 0x8000FF9C + 0x8000FF9C = 0x1_0001_FF38. 33번째 비트는 버려져 x28 = **0x0001FF38**. 음수 + 음수(두 부호 비트 1)인데 결과의 부호 비트가 0(양수)이므로 **오버플로가 생겼다**.

(4) x7 = 0xFF8000FF를 리틀 엔디언으로: 0x108 = **FF**, 0x109 = **00**, 0x10A = **80**, 0x10B = **FF**.

(5) 빅 엔디언은 가장 낮은 주소가 최상위 바이트: x5 = **0x9CFF0080**.

- **2점** (1) 각 1점. 바이트 순서를 거꾸로(0x9CFF0080) 쓰면 0점.
- **2점** (2) 부호 비트로 채운 값이어야 2점. 0으로 채운 0x008000FF는 0점.
- **3점** (3) 값 1점, 오버플로 "생김" 1점, 근거(같은 부호를 더했는데 결과 부호가 다름) 1점.
- **2점** (4) 네 바이트 순서까지 맞아야 2점, 순서만 거꾸로면 1점.
- **1점** (5).

(2)에서 srai와 srli를 혼동하는 경우가 많다. srli라면 0x008000FF다. (3)에서 "부호가 다른 수를 더하면 오버플로가 없다"는 규칙만 떠올리고 판단을 거꾸로 하는 실수가 있다. 이 문제는 같은 부호(둘 다 음수)다. (4)는 레지스터 값을 그대로 FF 80 00 FF로 적는 실수가 흔하다. 그것은 빅 엔디언 순서다.

추가 점수를 받으려면: (3)에 "부호 없는 수로 보면 합 0x0001FF38이 더한 수 0x8000FF9C보다 작으므로 부호 없는 오버플로도 생겼다"를 덧붙이고, C/Java는 정수 오버플로를 무시한다는 점을 쓴다.

근거 [Ch2 p.13, p.25] [Ch3 p.4–6]

6. **10점** 다음 코드가 주소 0x00001000부터 놓여 있다. 아래 encoding 표를 보고 답하시오.

```
0x00001000: Top: sw   x9, -12(x18)
0x00001004:      srai x5, x6, 3
0x00001008:      bne  x5, x0, Top
```

형식(Format)	명령어(Instruction)	opcode	funct3	funct7
R	`add`	`0110011`	`000`	`0000000`
R	`sub`	`0110011`	`000`	`0100000`
R	`sll`	`0110011`	`001`	`0000000`
R	`xor`	`0110011`	`100`	`0000000`
R	`srl`	`0110011`	`101`	`0000000`
R	`sra`	`0110011`	`101`	`0100000`
R	`or`	`0110011`	`110`	`0000000`
R	`and`	`0110011`	`111`	`0000000`
R	`lr.d`	`0110011`	`011`	`0001000`
R	`sc.d`	`0110011`	`011`	`0001100`
I	`lb`	`0000011`	`000`	n.a.
I	`lh`	`0000011`	`001`	n.a.
I	`lw`	`0000011`	`010`	n.a.
I	`lbu`	`0000011`	`100`	n.a.
I	`lhu`	`0000011`	`101`	n.a.
I	`addi`	`0010011`	`000`	n.a.
I	`slli`	`0010011`	`001`	`0000000`
I	`xori`	`0010011`	`100`	n.a.
I	`srli`	`0010011`	`101`	`0000000`
I	`srai`	`0010011`	`101`	`0100000`
I	`ori`	`0010011`	`110`	n.a.
I	`andi`	`0010011`	`111`	n.a.
I	`jalr`	`1100111`	`000`	n.a.
S	`sb`	`0100011`	`000`	n.a.
S	`sh`	`0100011`	`001`	n.a.
S	`sw`	`0100011`	`010`	n.a.
SB	`beq`	`1100011`	`000`	n.a.
SB	`bne`	`1100011`	`001`	n.a.
SB	`blt`	`1100011`	`100`	n.a.

형식(Format)	명령어(Instruction)	opcode	funct3	funct7
SB	`bge`	`1100011`	`101`	n.a.
SB	`bltu`	`1100011`	`110`	n.a.
SB	`bgeu`	`1100011`	`111`	n.a.
U	`lui`	`0110111`	n.a.	n.a.
UJ	`jal`	`1101111`	n.a.	n.a.

Name	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
R-type	funct7	rs2	rs1	funct3	rd	opcode
I-type	immediate[11:0]		rs1	funct3	rd	opcode
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
U-type	immediate[31:12]				rd	opcode

SB형의 칸 배치는 표에 없으므로 외워서 쓴다.

(1) 세 명령어의 형식 이름 (1점) (2) 세 명령어의 32비트 기계어를 16진수로 (각 2점, 6점) (3) 0x1000 ~ 0x100B의 12바이트를 주소 순서대로 (3점)

정답과 채점 기준

(1) sw = **S형**, srai = **I형**, bne = **SB형**.

(2) `sw x9, -12(x18)` : −12를 12비트 2의 보수로 = `1111 1111 0100` . imm[11:5] = `1111111` , imm[4:0] = `10100` . rs2 = x9 = `01001` (저장할 값), rs1 = x18 = `10010` (기준 주소).

1111111	01001	10010	010	10100	0100011
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode

4비트씩: 1111 1110 1001 1001 0010 1010 0010 0011 = **0xFE992A23**

`srai x5, x6, 3` : 위 7비트 `0100000` (표의 funct7 칸), 시프트 양 3 = `00011` , rs1 = 6, funct3 = `101` , rd = 5.

0100000	00011	00110	101	00101	0010011
funct7	shamt	rs1	funct3	rd	opcode

0100 0000 0011 0011 0101 0010 1001 0011 = **0x40335293**

`bne x5, x0, Top` : 오프셋 = 0x1000 − 0x1008 = −8. 13비트 2의 보수 = `1 1111 1111 1000` . imm[12] = 1, imm[11] = 1, imm[10:5] = `111111` , imm[4:1] = `1100` . rs1 = x5, rs2 = x0, funct3 = `001` .

1 [12]	111111 imm[10:5]	00000 rs2	00101 rs1	001 funct3	1100 imm[4:1]	1 [11]	1100011 opcode

1111 1110 0000 0010 1001 1100 1110 0011 = **0xFE029CE3**

(3) 각 word의 최하위 바이트가 낮은 주소로:

주소	+0	+1	+2	+3
0x1000	23	2A	99	FE
0x1004	93	52	33	40
0x1008	E3	9C	02	FE

- **1점** (1) 셋 다 맞아야 1점.
- **2점** (2) 명령어마다. 필드 배치는 맞고 16진 변환 한 자리만 틀리면 1점. 어셈블리 순서대로(rd, rs1, rs2) 칸을 채우면 0점.
- **2점** bne: 오프셋 −8을 바르게 구하면 1점, imm 비트 흩뿌리기까지 맞으면 1점.
- **3점** (3) word마다 1점. (2)가 틀렸어도 자기 답을 바르게 뒤집었으면 인정.

sw에서 rs1과 rs2를 바꿔 넣는 실수가 가장 많다. 저장할 값 x9가 rs2, 괄호 안 기준 레지스터 x18이 rs1이다. bne는 오프셋을 −2(명령어 개수)로 쓰거나, 목표 − (분기 주소 + 4)로 계산하는 실수가 있다. RISC-V 오프셋은 분기 명령어 자신의 주소 기준, 바이트 단위다. 13비트 오프셋에서 0번 비트는 저장하지 않는다.

추가 점수를 받으려면: S형에서 즉시값을 둘로 나누는 이유(rs1·rs2·opcode·funct3를 모든 형식에서 같은 자리에 두기 위해)와, SB형의 imm 비트 배치가 ImmGen의 MUX를 줄이려는 것이라는 점을 한 줄씩 쓴다.

근거 [Ch2 p.13, p.24, p.47, p.50] [Ch4 p.22–23]

7. **10점** 4비트 A = `1010`, B = `1101`을 곱해 8비트 곱을 얻는다.

Mode	A (십진)	B (십진)	A × B (십진, 8비트)	Upper 4비트	Lower 4비트
Unsigned	(가)	(나)	(다)	(라)	(마)
Signed	(바)	(사)	(아)	(자)	(차)
Signed-Unsigned (A 부호 있음, B 부호 없음)	(카)	(타)	(파)	(하)	(거)

(1) 표를 완성하시오. (6점) (2) 32비트 RISC-V에서 세 모드의 Upper 32비트를 얻는 명령어와, Lower 32비트를 얻는 명령어를 각각 쓰시오. (2점) (3) 세 모드에서 Lower가 모두 같은 이유를 쓰시오. (2점)

정답과 채점 기준

Mode	A	B	A × B	Upper	Lower
Unsigned	10	13	130 (`10000010`)	`1000`	`0010`
Signed	−6	−3	18 (`00010010`)	`0001`	`0010`
Signed-Unsigned	−6	13	−78 (`10110010`)	`1011`	`0010`

풀이: 1010을 부호 있는 수로 보면 $-(0101_2 + 1) = -0110_2 = -6$. 1101은 $-(0010_2 + 1) = -0011_2 = -3$. −78의 8비트 2의 보수 = 256 − 78 = 178 = `10110010`.

(2) Upper: Unsigned = `mulhu`, Signed = `mulh`, Signed-Unsigned = `mulhsu`. Lower는 세 모드 모두 `mul` 하나.

(3) n비트 수의 곱을 2^n으로 나눈 나머지(아래 n비트)는 입력을 부호 있게 읽든 없게 읽든 같다. 부호 있는 값과 부호 없는 값은 2^n의 배수만큼 차이 나고, 그 차이에 다른 수를 곱해도 2^n의 배수라 아래 n비트에 영향이 없다. 그래서 하위 절반은 `mul` 하나로 충분하다.

- **6점** (1) 행마다 2점: 십진 해석(A, B) 1점 + 곱·Upper·Lower 1점.
- **2점** (2) Upper 셋 1점, mul 1점.
- **2점** (3) "하위 비트는 모드와 무관"만 쓰면 1점, 2^n 배수 차이로 설명하면 2점.

Signed-Unsigned 줄에서 B까지 −3으로 읽어 18을 쓰는 실수가 많다. 이 모드는 A만 부호 있게 읽는다. 또 −78을 8비트로 바꿀 때 78의 2진수(01001110)를 그대로 쓰는 실수가 있다. 음수는 반드시 2의 보수로 바꾼다.

추가 점수를 받으려면: 슬라이드 3비트 예($101_2 \times 101_2$: Upper 011/001/110, Lower 001 공통)를 같은 원리로 언급하고, 부호 있는 곱셈은 절댓값으로 곱한 뒤 부호가 다르면 곱을 부정하는 방법도 있다는 점(31번 반복)을 덧붙인다.

근거 [Ch3 p.12–13]

8. **10점** A = 2.75, B = −0.625이다.

(1) A, B를 2진수로 바꾸고 정규화된 형태($1.xxx_2 \times 2^y$)로 쓰시오. (2점)

(2) A + B를 부동소수점 덧셈 단계(정렬 → 유효숫자 덧셈 → 정규화 → 반올림)로 계산하시오. (3점)

(3) A × B를 부동소수점 곱셈 단계로 계산하시오. 지수는 32비트 형식의 biased exponent로 계산하는 과정도 보이시오. (3점)

(4) A + B와 A × B를 IEEE 754 32비트와, 부호 1·지수 5·가수 10비트(bias 15)의 16비트 형식으로 각각 16진수로 쓰시오. (2점)

정답과 채점 기준

(1) 2.75 = 2 + 0.75. 0.75 × 2 = 1.5 → 1, 0.5 × 2 = 1.0 → 1이므로 $0.75 = 0.11_2$. A = 10.11_2 = **$1.011_2 \times 2^1$**. 0.625 × 2 = 1.25 → 1, 0.25 × 2 = 0.5 → 0, 0.5 × 2 = 1.0 → 1이므로 $0.625 = 0.101_2$. B = **$-1.01_2 \times 2^{-1}$**.

(2) ① 정렬: 지수가 작은 B를 지수 1에 맞춘다. 2칸 오른쪽 시프트: $-1.01_2 \times 2^{-1} = -0.0101_2 \times 2^1$. ② 유효숫자 덧셈: $1.0110_2 - 0.0101_2 = 1.0001_2$. ③ 정규화: $1.0001_2 \times 2^1$, 이미 정규화됨, 지수 1은 −126 ~ 127 안이라 오버플로·언더플로 없음. ④ 반올림: 필요 없음. 결과 **$1.0001_2 \times 2^1 = 2.125$** (검산 2.75 − 0.625 = 2.125).

(3) ① 지수: 1 + (−1) = 0. 지수 칸(biased)으로는 (1 + 127) + (−1 + 127) − 127 = 127 (bias를 한 번 빼야 함). ② 유효숫자 곱: $1.011_2 \times 1.01_2 = 1.011_2 + 0.01011_2 = 1.10111_2$. ③ 정규화: 이미 1.xxx 꼴, 지수 0은 범위 안. ④ 반올림: 필요 없음. ⑤ 부호: 양 × 음 = 음. 결과 **$-1.10111_2 \times 2^0 = -1.71875$** (검산 2.75 × (−0.625) = −1.71875).

(4) A + B: 32비트 = 0 / 10000000 / 0001000…0 → **0x40080000**. 16비트 = 0 / 10000(16) / 0001000000 → **0x4040**.

A × B: 32비트 = 1 / 01111111 / 1011100…0 → **0xBFDC0000**. 16비트 = 1 / 01111(15) / 1011100000 → **0xBEE0**.

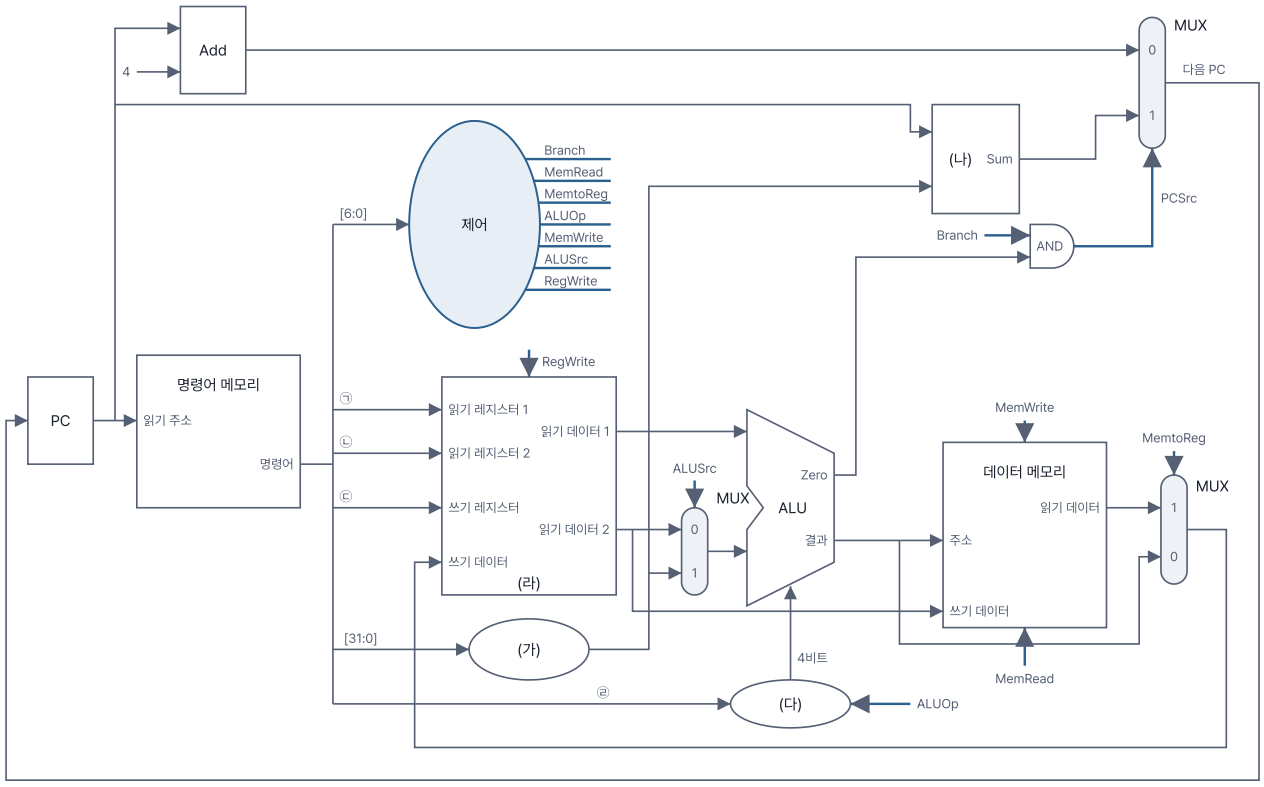
- **2점** (1) 각 1점. 정규화 형태까지 써야 함.
- **3점** (2) 정렬(작은 쪽을 오른쪽 시프트) 1점, 유효숫자 계산 1점, 정규화 확인과 결과 1점.
- **3점** (3) 지수 칸(biased) 합에서 127을 빼는 과정 1점, 유효숫자 곱 1점, 부호와 결과 1점.
- **2점** (4) 네 값 중 셋 이상 맞으면 2점, 둘이면 1점.

(3)에서 두 수의 지수 칸 값을 그대로 더해 128 + 126 = 254를 답으로 쓰는 실수(bias 127이 두 번 들어감)가 대표적이다. 올바른 지수 칸은 254 − 127 = 127(실제 지수 0)이다. (4)에서 숨은 비트(맨 앞 1)까지 가수 칸에 넣는 실수, 16비트 지수 칸을 구할 때 127을 더하는 실수가 많다. 16비트 bias = $2^{5-1} - 1 = 15$다.

추가 점수를 받으려면: 정렬할 때 지수가 **작은** 쪽을 오른쪽으로 시프트하는 이유(큰 수의 상위 비트를 잃지 않으려고)와, 반올림 뒤 다시 정규화가 깨지면 3단계로 돌아간다는 점을 쓴다.

근거 [Ch3 p.28, p.31–33, p.36]

9. **8점** 다음은 단일 사이클 데이터패스이다. 부품 넷을 (가)~(라)로, 명령어 비트 범위 넷을 ㉠~㉣로 비웠다. 강조 선은 제어 신호다.



단일 사이클 데이터패스 빈칸. MUX 안 숫자는 슬라이드와 같다(PCSrc 위 0·아래 1, ALUSrc 위 0·아래 1, MemtoReg 위 1·아래 0).

(1) (가)~(라)의 이름을 쓰시오. (4점)

(2) 명령어 `add x7, x12, x30`이 실행될 때, 선 ㉠~㉣에 실리는 값을 2진수로 쓰고 각각 명령어의 몇 번 비트인지 쓰시오. (4점)

정답과 채점 기준

(1) (가) **ImmGen**(즉시값 생성기, 부호 확장) (나) **분기 목표 주소를 더하는 가산기**(Add, PC + 오프셋) (다) **ALU 제어**(ALU control) (라) **레지스터 파일**(Registers)

(2) add는 R형: funct7 `0000000` , rs2 = 30, rs1 = 12, funct3 `000` , rd = 7.

- ㉠ = [19:15] = rs1 = `01100` (12) → 읽기 레지스터 1
- ㉡ = [24:20] = rs2 = `11110` (30) → 읽기 레지스터 2
- ㉢ = [11:7] = rd = `00111` (7) → 쓰기 레지스터
- ㉣ = [30]과 [14:12] = `0` 과 `000` → ALU 제어 (add이므로 ALU 제어 출력은 0010)

- **4점** (1) 하나당 1점. (나)를 "PC + 4 가산기"라고 쓰면 0점(그것은 위쪽 Add).
- **4점** (2) 선 하나당 1점: 비트 범위와 값이 둘 다 맞아야 1점.

(가)를 "부호 확장기"라고만 써도 뜻은 통하지만 슬라이드 이름은 ImmGen이다. 교재 옛 판 그림처럼 (가)와 (나) 사이에 "Shift left 1" 상자를 그리는 답이 있는데, 강의 슬라이드에는 그 상자가 없고 ImmGen 출력이 분기 가산기로 바로 간다. (2)에서 rs1과 rs2의 비트 범위를 바꾸는 실수가 잦다. rs1이 [19:15]로 오른쪽(낮은 비트)에 더 가깝다.

추가 점수를 받으려면: ㉣에서 [30]이 funct7의 한 비트라 add(0)와 sub(1)를 가르는 데 쓰인다는 점, 나머지 두 선([6:0] → 제어 유닛, [31:0] → ImmGen)까지 정리해 쓴다.

근거 **[Ch4 p.12–15, p.17, p.21, p.24]**

10. **10점** (1) 단일 사이클 구현을 오늘날 쓰지 않는 이유를 클럭 주기, 가장 긴 경로, 설계 원칙과 연결해 서술하시오. (6점)

(2) 각 부품의 지연이 명령어 메모리 200 ps, 레지스터 파일 읽기 100 ps, ALU 150 ps, 데이터 메모리 200 ps, 레지스터 파일 쓰기 100 ps라고 하자(MUX·가산기·제어 지연은 무시). 단일 사이클 구현의 클럭 주기는 얼마이며, `add` 명령어는 한 사이클 중 몇 ps를 쓰지 않고 기다리는가? (4점) PDF 외 보충

정답과 채점 기준

(1) 단일 사이클 구현은 모든 명령어를 한 클럭에 끝내므로 클럭 주기가 모든 명령어에 대해 같아야 한다. 그래서 가장 긴 경로가 클럭 주기를 정한다. 가장 긴 경로는 대개 load로, 다섯 단위를 차례로 쓴다: 명령어 메모리 → 레지스터 파일 → ALU → 데이터 메모리 → 레지스터 파일. 명령어마다 클럭 주기를 바꾸는 것은 현실적으로 불가능하므로, 짧게 끝나는 흔한 명령어(add 등)도 load 길이만큼 기다린다. 이것은 "흔한 경우를 빠르게(Making the common case fast)" 원칙을 어긴다. 그래서 파이프라이닝으로 성능을 높인다.

(2) lw = 200 + 100 + 150 + 200 + 100 = **750 ps**가 가장 길어 클럭 주기 750 ps. add = 200 + 100 + 150 + 100 = 550 ps(데이터 메모리를 쓰지 않음)이므로 **200 ps**를 기다린다.

- **2점** 모든 명령어가 같은 클럭 주기(한 사이클)를 써야 한다.
- **2점** 가장 긴 경로가 클럭을 결정하고, 그것이 load이며 다섯 단위 순서가 맞음(순서가 틀리면 1점).
- **1점** 명령어별로 클럭 주기를 바꿀 수 없음.
- **1점** "Making the common case fast" 원칙 위반을 명시.
- **2점** (2) 클럭 주기 750 ps(lw 경로 합).
- **2점** (2) add 경로 550 ps와 대기 200 ps.

"단일 사이클은 CPI가 1이라 느리다"는 답은 방향이 틀렸다. CPI는 1로 가장 작지만 클럭 주기가 길어져서 문제다. load의 다섯 단위에서 마지막 "레지스터 파일(쓰기)"을 빠뜨리는 경우가 많다. (2)에서 add에 데이터 메모리 지연을 넣으면 750 ps가 되어 차이가 사라진다.

추가 점수를 받으려면: CPU 시간 = 명령어 수 × CPI × 클럭 주기 식으로 "CPI = 1이지만 클럭 주기가 커서 CPU 시간이 크다"를 설명하고, 같은 하드웨어를 한 명령어에서 두 번 쓸 수 없어 명령어 메모리와 데이터 메모리가 따로 있다는 단일 사이클의 또 다른 특징을 덧붙인다.

근거 [Ch4 p.16, p.29] [Ch1 p.12, p.28]

모의고사 · 2회 90분 · 100점

모의고사 2회

1회와 같은 유형이지만 숫자, 방향, 빈칸 위치를 바꿨다. 타이머 90분, 노트를 덮고, 계산 과정까지 칸에 쓴다. 10문항에 모두 답을 쓴 뒤 맨 아래 버튼을 누른다. 1회에서 틀렸던 유형은 이번에 시간을 따로 재 본다.

1. **12점** 다음 문장이 맞으면 O, 틀리면 X를 쓰고, X인 문장은 틀린 곳을 바르게 고치시오. (각 2점)

(1) Apple iPhone XS Max를 열어 보면 5대 구성 요소 중 데이터패스·제어·메모리가 대부분을 차지하고, I/O는 작은 부분이다.

(2) 비휘발성 메모리는 2차 메모리(secondary memory)로, 주 메모리보다 크고 느리며 하드 디스크와 플래시 메모리가 그 예다.

(3) RISC-V는 32비트 레지스터 64개로 된 레지스터 파일을 가지며, 레지스터 수가 많을수록 클럭 사이클이 짧아진다.

(4) C 문장 `a = b + c + d + e;`를 RISC-V 산술 명령어로 옮기려면 명령어가 3개 필요하다.

(5) 부동소수점 형식에서 가수(fraction) 칸을 늘리면 표현 범위가 넓어지고, 지수 칸을 늘리면 정밀도가 높아진다.

(6) 제어 유닛은 PCSrc를 제외한 모든 제어 신호를 명령어의 opcode와 funct 필드만으로 정할 수 있다.

정답과 채점 기준

(1) **X**. 반대다. iPhone XS Max는 **I/O**(터치 LCD, 카메라, 마이크, 스피커, Wi-Fi 등)가 대부분이고, 데이터패스·제어·메모리는 작은 부분이다.

(2) **O**.

(3) **X**. 레지스터 파일은 **32개**(x0~x31)의 32비트 레지스터다. 레지스터가 많으면 클럭 사이클이 **길어질** 수 있다(설계 원칙 2: Smaller is faster).

(4) **O**. RISC-V 산술 명령어는 소스 둘, 목적지 하나뿐이라 네 수를 더하려면 3개가 필요하다.

(5) **X**. 반대다. 가수 칸을 늘리면 **정밀도**가, 지수 칸을 늘리면 **범위**가 커진다.

(6) **O**. PCSrc는 beq이면서 ALU의 Zero 출력이 1일 때 켜지므로 실행 결과가 있어야 정해진다.

- **2점** O인 문장 (2)(4)(6): O만 맞게 쓰면 2점.
- **2점** X인 문장 (1)(3)(5): X 1점 + 고친 내용 1점. (3)은 개수(32)와 방향(길어짐) 둘 다 고쳐야 고침 점수.

(3)은 틀린 곳이 두 군데라 하나만 고치고 넘어가기 쉽다. 64는 "64비트 = doubleword"와 섞인 숫자다. (6)을 X로 고르는 경우는 "모든"이라는 말에 걸린 것인데, 문장에 "PCSrc를 제외한"이 있어 슬라이드와 같다.

추가 점수를 받으려면: (1)에 "A12 프로세서는 큰 코어 2개와 작은 코어 4개, 2.5 GHz"처럼 슬라이드 세부를, (6)에 PCSrc = Branch AND Zero 식을 덧붙인다.

근거 [Ch1 p.16, p.18] [Ch2 p.9–10] [Ch3 p.25] [Ch4 p.25]

2. **12점** 빈칸 ①~⑥을 채우시오. (각 2점)

(1) CPU 실행 시간 = 명령어 수 × CPI × 클럭 주기이다. 명령어 수는 (①), ISA, 컴파일러가 정하고, CPI와 클럭 주기는 (②)가 정한다.

(2) 프로세서 = (③) + 제어(control)이다.

(3) R형 명령어는 (④)에서 레지스터 두 개를 읽어 ALU 연산을 한 뒤, 결과를 다시 (④)의 한 레지스터에 쓴다. (④)는 프로세서의 범용 레지스터 32개를 담고, 레지스터 번호로 읽고 쓴다.

(4) `sw`가 쓰는 명령어 형식은 (⑤)형이고, `bne`가 쓰는 형식은 (⑥)형이다.

정답과 채점 기준

① **프로그램**(알고리즘) ② **CPU 하드웨어**(마이크로아키텍처, 공정 기술) ③ **데이터패스**(datapath) ④ **레지스터 파일**(register file) ⑤ **S** ⑥ **SB**

- **2점** 빈칸 하나당.
- **1점** ②에 "하드웨어 설계자"처럼 뜻이 통하지만 용어가 다르면 1점.
- **0점** ④에 "레지스터", "메모리"만 쓰면 0점(구조의 이름은 레지스터 파일).

①에 "CPU 하드웨어"를 쓰는 오답이 많다. 명령어 수는 소프트웨어 쪽(프로그램, ISA, 컴파일러)이 정한다. ⑥을 S로 쓰면 분기의 13비트 오프셋을 어디에 놓는지 구분하지 못한 것이다. 분기는 SB형이다.

추가 점수를 받으려면: ②에 Ch4 p.2의 "CPU 하드웨어 = 마이크로아키텍처, 공정 기술 등(Intel vs AMD)"을, ③에 5대 구성 요소(입력, 출력, 메모리, 데이터패스, 제어)를 덧붙인다.

근거 [Ch1 p.15, p.28] [Ch4 p.2, p.8, p.12] [Ch2 p.21, p.47]

3. **10점** (1) 클럭 주기가 400 ps인 컴퓨터가 명령어 5×10^8개짜리 프로그램을 0.25초에 실행한다. 평균 CPI와 MIPS를 구하시오. (4점)

(2) 어떤 프로그램의 실행 시간 60초는 부동소수점 연산 30초, 메모리 접근 18초, 나머지 12초로 나뉜다.

(a) 부동소수점 연산만 개선해서 전체를 2배 빠르게(30초) 할 수 있는가? 식으로 답하시오. (2점)

(b) 부동소수점 연산만 개선해서 전체를 1.5배 빠르게 하려면 부동소수점 연산을 몇 배 빠르게 해야 하는가? (2점)

(c) 부동소수점 연산과 메모리 접근을 모두 3배 빠르게 하면 전체는 몇 배 빨라지는가? (소수 둘째 자리까지) (2점)

정답과 채점 기준

(1) 클럭 속도 = $1 \div (400 \times 10^{-12}\ \text{s}) = 2.5 \times 10^9$ Hz = 2.5 GHz. 사이클 수 = 실행 시간 × 클럭 속도 = $0.25 \times 2.5 \times 10^9 = 6.25 \times 10^8$. CPI = $6.25 \times 10^8 \div 5 \times 10^8$ = **1.25**. MIPS = 명령어 수 ÷ (실행 시간 × 10^6) = $5 \times 10^8 \div (0.25 \times 10^6)$ = **2000**. 검산: 2500 MHz ÷ 1.25 = 2000.

(2)(a) 개선 후 시간 = 30 ÷ n + 30. 이것이 30이 되려면 30 ÷ n = 0이어야 하므로 **불가능**하다. n을 아무리 키워도 2배에 가까워질 뿐 도달하지 못한다.

(b) 목표 60 ÷ 1.5 = 40초. 30 ÷ n + 30 = 40 → 30 ÷ n = 10 → **n = 3배**.

(c) 30 ÷ 3 + 18 ÷ 3 + 12 = 10 + 6 + 12 = 28초. 60 ÷ 28 ≈ **2.14배**.

- **4점** (1) 클럭 속도 또는 사이클 수 계산 1점, CPI 1점, MIPS 식 1점, 값 1점.
- **2점** (a) 암달 식 1점, "불가능"과 이유 1점.
- **2점** (b) 식 1점, 값 1점.
- **2점** (c) 식 1점, 값 1점.

(1)에서 400 ps를 클럭 속도처럼 곱하는 실수(0.25 × 400)가 있다. 주기와 속도는 역수다. MIPS를 2×10^9처럼 쓰면 10^6으로 나누는 것을 잊은 것이다. (a)에서 "60 ÷ 2 = 30이니까 n = 2배"라고 답하는 경우가 있는데, 개선되지 않는 30초는 그대로 남는다.

추가 점수를 받으려면: (a)의 상한 60 ÷ 30 = 2배를 "도달 불가능한 상한"이라고 명시하고, 슬라이드 예(100초 중 곱셈 80초, 2.5배 → n = 4, 곱셈 외만 개선 시 최대 1.25배)와 같은 구조라고 연결한다.

근거 [Ch1 p.26–28, p.33–34]

4. **10점** 다음 C 함수를 RISC-V 어셈블리로 바꾸시오. 인자 a, b, c는 `x10`, `x11`, `x12`로 들어온다. a − b는 `x19`, c << 2는 `x20`, t는 `x9`에 두고, 이 세 레지스터는 스택에 저장했다가 복원한다. 결과는 `x10`으로 돌려주고 호출한 곳으로 돌아간다.

```
int mix(int a, int b, int c)
{
    int t;
    t = (a - b) + (c << 2);
    return t;
}
```

(1) 어셈블리 코드 (7점) (2) `x19`, `x20`, `x9` 대신 `x5`, `x6`, `x7`을 썼다면 스택 저장이 꼭 필요한가? 이유와 함께 쓰시오. (1점) (3) 함수 시작 때 sp(x2) = 0x7FFFFFF0이었다. 세 레지스터가 저장되는 주소를 각각 쓰고, a = 7, b = 3, c = 5일 때 반환값을 쓰시오. (2점)

정답과 채점 기준

```
mix:
    addi sp, sp, -12      // 3개(12바이트) 자리 확보
    sw   x9, 8(sp)        // 피호출자 보존 레지스터 저장
    sw   x19, 4(sp)
    sw   x20, 0(sp)
    sub  x19, x10, x11    // x19 = a - b
    slli x20, x12, 2      // x20 = c << 2
    add  x9, x19, x20     // t = x19 + x20
    addi x10, x9, 0       // 반환값 x10 = t
    lw   x20, 0(sp)       // 복원
    lw   x19, 4(sp)
    lw   x9, 8(sp)
    addi sp, sp, 12       // 스택 되돌리기
    jalr x0, 0(x1)        // 호출한 곳으로 복귀
```

(2) **필요 없다**. x5~x7은 임시(호출자 보존, caller-saved) 레지스터라 피호출자가 보존하지 않아도 된다. x9, x19, x20은 피호출자 보존(callee-saved, x8~x9, x18~x27)이라 쓰면 반드시 저장·복원해야 한다.

(3) 확보 후 sp = 0x7FFFFFF0 − 12 = 0x7FFFFFE4. x9 → **0x7FFFFFEC**, x19 → **0x7FFFFFE8**, x20 → **0x7FFFFFE4**. 반환값 = (7 − 3) + (5 × 4) = **24**.

- **2점** 프롤로그: sp를 −12, 세 레지스터를 서로 다른 오프셋에 sw.
- **2점** 본체: sub, slli 2, add가 맞는 레지스터로.
- **1점** 반환값을 x10에(`addi x10, x9, 0` 또는 같은 효과).
- **2점** 에필로그: 같은 오프셋에서 lw로 복원, sp +12, `jalr x0, 0(x1)` .
- **1점** (2). **2점** (3) 주소 1점, 반환값 1점.

자주 나오는 오답: (a) sp를 +12로 먼저 늘린다. 스택은 높은 주소에서 낮은 주소로 자라므로 확보는 빼기다. (b) 반환값을 복원 뒤에 옮긴다. x9를 먼저 복원하면 t가 사라진다. (c) 복귀를 `jal x1, ...` 이나 `jalr x1, 0(x1)` 로 쓴다. 복귀는 목적지 x0으로 복귀 주소를 버리는 jalr이다. (d) leaf 함수인데 x1까지 저장한다. 틀린 것은 아니지만 필요 없다.

추가 점수를 받으려면: 슬라이드 leaf_example에서 x5·x6 저장은 필요 없고 x20만 필요하다는 설명(p.39)을 인용하고, 프로시저 실행 6단계 중 어느 줄이 3) 저장 공간 확보, 5) 결과 배치, 6) 복귀인지 표시한다.

근거 [Ch2 p.33–39]

---

5. **10점** `x10` = 0x00002000(int 배열 A의 시작 주소), `x11` = 4(n), `x12` = 0x00000102(key)이다. 메모리 0x2000 ~ 0x200F의 바이트는 다음과 같다.

주소	+0	+1	+2	+3
0x2000	01	02	00	00
0x2004	02	01	00	00
0x2008	00	00	01	02
0x200C	00	00	00	00

```
       addi x13, x0, -1
       addi x5, x0, 0
Loop:  bge  x5, x11, Exit
       slli x6, x5, 2
       add  x6, x6, x10
       lw   x7, 0(x6)
       beq  x7, x12, Found
       addi x5, x5, 1
       beq  x0, x0, Loop
Found: addi x13, x5, 0
Exit:
```

(1) 이 코드를 C로 쓰시오. 변수 이름은 i(`x5`), idx(`x13`), A, n, key를 쓴다. (4점)

(2) RISC-V(리틀 엔디언)에서 실행이 끝난 뒤 `x13` 의 값과 `lw` 가 실행된 횟수 (3점)

(3) 같은 메모리 바이트를 빅 엔디언 기계가 실행했다면 A[0]~A[2]의 값(16진수)과 `x13` 의 값은? (3점)

정답과 채점 기준

```
idx = -1;
for (i = 0; i < n; i = i + 1) {
    if (A[i] == key) {
        idx = i;
        break;
    }
}
```

(배열 A에서 key를 처음 찾은 위치를 idx에 넣고, 없으면 −1. while 문으로 써도 같은 동작이면 정답.)

(2) 리틀 엔디언: A[0] = 0x00000201, A[1] = 0x00000102. A[1] = key이므로 i = 1에서 Found. **x13 = 1**, lw는 i = 0, 1에서 **2번**.

(3) 빅 엔디언(낮은 주소가 최상위 바이트): A[0] = **0x01020000**, A[1] = **0x02010000**, A[2] = **0x00000102**. A[2] = key이므로 **x13 = 2**(lw 3번).

- **1점** (1) idx = −1 초기화.
- **1점** (1) i = 0부터 i < n 동안 반복, i 증가.
- **1점** (1) A[i] == key 비교(slli 2 + add가 A[i]의 주소라는 것을 읽음).
- **1점** (1) 찾으면 idx = i 후 반복 종료(break 또는 동등한 구조).
- **3점** (2) A[0], A[1] 값 바르게 읽기 1점, x13 = 1 1점, lw 2번 1점.
- **3점** (3) A[0]~A[2] 값 2점(하나 틀리면 1점), x13 = 2 1점.

(1)에서 break를 빠뜨리면 마지막으로 찾은 위치가 되어 동작이 달라진다. Found 다음이 바로 Exit라 반복으로 돌아가지 않는다. (2)에서 바이트를 주소 순서대로 이어 읽어 A[0] = 0x01020000으로 쓰는 것은 빅 엔디언 방식이다. 리틀 엔디언은 주소가 큰 바이트부터 왼쪽에 쓴다.

추가 점수를 받으려면: 슬라이드 예(0x00112233 저장 시 빅 엔디언 00 11 22 33, 리틀 엔디언 33 22 11 00)로 원리를 한 줄 설명하고, i × 4를 slli로 계산하는 이유(int가 4바이트, 시프트가 곱셈보다 빠름)를 덧붙인다.

근거 [Ch2 p.12–14, p.28–30]

6. **10점** 다음 코드가 주소 0x00400010부터 놓여 있다. 아래 encoding 표를 보고 답하시오.

```
0x00400010:       lw   x8, 20(x2)
0x00400014:       sub  x9, x8, x10
0x00400018:       blt  x9, x0, Neg
0x0040001C:       ...
0x00400020:       ...
0x00400024: Neg:  (아래 (3)의 바이트)
```

형식(Format)	명령어(Instruction)	opcode	funct3	funct7
R	`add`	`0110011`	`000`	`0000000`
R	`sub`	`0110011`	`000`	`0100000`
R	`sll`	`0110011`	`001`	`0000000`
R	`xor`	`0110011`	`100`	`0000000`
R	`srl`	`0110011`	`101`	`0000000`
R	`sra`	`0110011`	`101`	`0100000`
R	`or`	`0110011`	`110`	`0000000`
R	`and`	`0110011`	`111`	`0000000`
R	`lr.d`	`0110011`	`011`	`0001000`
R	`sc.d`	`0110011`	`011`	`0001100`
I	`lb`	`0000011`	`000`	n.a.
I	`lh`	`0000011`	`001`	n.a.
I	`lw`	`0000011`	`010`	n.a.
I	`lbu`	`0000011`	`100`	n.a.
I	`lhu`	`0000011`	`101`	n.a.
I	`addi`	`0010011`	`000`	n.a.
I	`slli`	`0010011`	`001`	`0000000`
I	`xori`	`0010011`	`100`	n.a.
I	`srli`	`0010011`	`101`	`0000000`
I	`srai`	`0010011`	`101`	`0100000`
I	`ori`	`0010011`	`110`	n.a.
I	`andi`	`0010011`	`111`	n.a.
I	`jalr`	`1100111`	`000`	n.a.
S	`sb`	`0100011`	`000`	n.a.
S	`sh`	`0100011`	`001`	n.a.
S	`sw`	`0100011`	`010`	n.a.
SB	`beq`	`1100011`	`000`	n.a.
SB	`bne`	`1100011`	`001`	n.a.
SB	`blt`	`1100011`	`100`	n.a.

형식(Format)	명령어(Instruction)	opcode	funct3	funct7
SB	`bge`	`1100011`	`101`	n.a.
SB	`bltu`	`1100011`	`110`	n.a.
SB	`bgeu`	`1100011`	`111`	n.a.
U	`lui`	`0110111`	n.a.	n.a.
UJ	`jal`	`1101111`	n.a.	n.a.

Name	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
R-type	funct7	rs2	rs1	funct3	rd	opcode
I-type	immediate[11:0]		rs1	funct3	rd	opcode
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
U-type	immediate[31:12]				rd	opcode

SB형의 칸 배치는 표에 없으므로 외워서 쓴다.

(1) `lw x8, 20(x2)`와 `blt x9, x0, Neg`의 형식 이름과 32비트 기계어(16진수) (각 3점, 6점)

(2) `lw`의 기계어가 놓인 0x00400010 ~ 0x00400013의 바이트를 주소 순서대로 (2점)

(3) 주소 0x00400024 ~ 0x00400027의 바이트가 차례로 `33 C5 A5 00`이다. 분기가 일어나면 실행될 이 명령어를 어셈블리로 쓰시오. (2점)

정답과 채점 기준

(1) `lw x8, 20(x2)` : **I형**. imm = 20 = `000000010100`, rs1 = x2 = `00010`, funct3 = `010`, rd = x8 = `01000`, opcode = `0000011`.

000000010100	00010	010	01000	0000011
imm[11:0]	rs1	funct3	rd	opcode

0000 0001 0100 0001 0010 0100 0000 0011 = **0x01412403**

`blt x9, x0, Neg` : **SB형**. 오프셋 = 0x00400024 − 0x00400018 = 0xC = 12. 13비트 = `0 0000 0000 1100`. imm[12] = 0, imm[11] = 0, imm[10:5] = `000000`, imm[4:1] = `0110`. rs2 = x0, rs1 = x9 = `01001`, funct3 = `100`.

0	000000	00000	01001	100	0110	0	1100011
[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	[11]	opcode

0000 0000 0000 0100 1100 0110 0110 0011 = **0x0004C663**

(2) 0x00400010 = **03**, 0x00400011 = **24**, 0x00400012 = **41**, 0x00400013 = **01**.

(3) 리틀 엔디언이므로 거꾸로 모아 0x00A5C533 = `0000000 01010 01011 100 01010 0110011` . opcode 0110011 → R형, funct3 100·funct7 0000000 → xor. rd = 01010 = x10, rs1 = 01011 = x11, rs2 = 01010 = x10. → `xor x10, x11, x10`

- **3점** lw: 형식 1점, 필드 배치 1점, 16진수 1점.
- **3점** blt: 형식과 오프셋 12 1점, imm 비트 배치 1점, 16진수 1점.
- **2점** (2) 순서까지 맞아야 2점. 01 41 24 03(빅 엔디언 순서)은 0점.
- **2점** (3) 바이트 순서 뒤집기 1점, 명령어와 레지스터 1점.

blt에서 오프셋을 명령어 개수(3)로 쓰거나 0x00400024 − 0x0040001C로 계산하는 실수가 있다. 기준은 분기 명령어 자신의 주소다. (3)에서 바이트를 그대로 이어 0x33C5A500으로 읽으면 opcode가 0000000이 되어 표에 없는 명령어가 나온다. 표에 없는 opcode가 나오면 바이트 순서를 의심한다.

추가 점수를 받으려면: 해독 4단계(2진수로 → opcode로 형식 결정 → 형식대로 필드 나누기 → 어셈블리)를 명시하고, lw의 rs1(기준 레지스터)과 rd(목적지)가 어셈블리 표기 순서와 다르게 놓인다는 점을 쓴다.

근거 [Ch2 p.13, p.23, p.47, p.50]

7. 10점 (1) 8비트 2의 보수 정수 연산이다. 각 결과(16진수 2자리)를 쓰고 부호 있는 오버플로 여부를 판단하시오. (4점)

(a) 0x70 + 0x30 (b) 0x90 − 0x20 (c) 0xF0 + 0x20 (d) 0x50 − 0x30

(2) (c)를 부호 없는 수의 덧셈으로 보면 오버플로인가? 판단 규칙과 함께 쓰시오. (1점)

(3) 부호 있는 나눗셈의 몫(Q)과 나머지(R)를 쓰시오. (4점) (a) −23 ÷ 4 (b) 23 ÷ (−4) (c) (−23) ÷ (−4). 또 −23 ÷ 4에서 Q = −6, R = 1로 하지 않는 이유는?

(4) 나눗셈은 곱셈처럼 여러 덧셈기를 병렬로 써서 빠르게 만들 수 없다. 이유를 쓰시오. (1점)

정답과 채점 기준

(1) (a) 112 + 48 = 160은 8비트 범위(−128 ~ 127)를 넘는다. 결과 **0xA0**(−96), 양 + 양 = 음이므로 **오버플로**.

(b) −112 − 32 = −144. 결과 **0x70**(+112), 음 − 양 = 양이므로 **오버플로**.

(c) −16 + 32 = 16. 결과 **0x10**, 부호가 다른 두 수의 덧셈이라 **오버플로 없음**.

(d) 80 − 48 = 32. 결과 **0x20**, 부호가 같은 두 수의 뺄셈이라 **오버플로 없음**.

(2) 부호 없는 덧셈은 합이 더한 수보다 작으면 오버플로다. 240 + 32 = 272인데 결과 0x10 = 16 < 240이므로 **부호 없는 오버플로**.

(3) 규칙: 부호를 떼고 계산 → 피제수와 제수의 부호가 다르면 몫을 부정 → 나머지 부호는 피제수와 같게. 23 ÷ 4 = 5 나머지 3.

- (a) **Q = −5, R = −3** (검산 −5 × 4 − 3 = −23)
- (b) **Q = −5, R = 3** (검산 (−5) × (−4) + 3 = 23)
- (c) **Q = 5, R = −3** (검산 5 × (−4) − 3 = −23)

Q = −6, R = 1도 −6 × 4 + 1 = −23으로 식은 맞지만, **나머지의 부호가 피제수와 같아야 한다**는 규칙에 어긋난다. 이 규칙 덕분에 부호와 상관없이 몫의 절댓값이 같다(C의 % 연산도 이 규칙).

(4) 나눗셈의 각 단계 뺄셈은 앞 단계 나머지의 부호에 따라 달라지는(조건부) 연산이라 병렬로 미리 계산할 수 없다. (대신 SRT 나눗셈은 표를 보고 몫 비트 여러 개를 예측한다.)

- **4점** (1) 항목마다 결과와 판단이 모두 맞아야 1점.
- **1점** (2) 판단과 규칙(S < A).
- **3점** (3) (a)(b)(c) 각 1점.
- **1점** (3) Q = −6, R = 1을 쓰지 않는 이유(나머지 부호 = 피제수 부호).
- **1점** (4) "뺄셈이 나머지 부호에 따라 조건부"를 쓸 것.

(d)를 "뺄셈이니 오버플로 가능"이라고 답하는 실수가 많다. 뺄셈 c − a = c + (−a)이므로 부호가 같은 두 수의 뺄셈은 부호가 다른 두 수의 덧셈과 같아 오버플로가 없다. (3)(b)에서 나머지를 −3으로 쓰는 실수는 제수의 부호를 따른 것이다. 피제수 23이 양수이므로 나머지도 양수다.

추가 점수를 받으려면: (1)에서 오버플로 판단을 "결과를 다 담으려면 한 비트(32비트 연산이면 33번째, 여기서는 9번째)가 더 필요한데, 그 자리를 잃어 부호 비트에 값이 들어간다"는 원리로 설명하고, (2)에 "부호 없는 수는 주로 메모리 주소에 쓰여 오버플로를 무시한다"를 덧붙인다.

근거 [Ch3 p.4–6, p.19–20]

8. **10점** A = −3.0, B = 1.75이다.

(1) A, B를 정규화된 2진수($1.xxx_2 \times 2^y$)로 쓰시오. (2점)

(2) A + B를 덧셈 4단계로 계산하시오. 정규화 단계에서 지수가 어떻게 바뀌는지 보이시오. (3점)

(3) A × B를 곱셈 5단계로 계산하시오. 16비트(1-5-10) 형식의 biased exponent로 지수 계산 과정을 보이시오. (3점)

(4) 16비트 형식의 bias를 구하는 식을 쓰고, A + B와 A × B를 32비트와 16비트 16진수로 쓰시오. (2점)

정답과 채점 기준

(1) $3 = 11_2 = 1.1_2 \times 2^1$이므로 A = $\mathbf{-1.1_2 \times 2^1}$. $0.75 = 0.11_2$이므로 $1.75 = 1.11_2$, B = $\mathbf{1.11_2 \times 2^0}$.

(2) ① 정렬: 지수가 작은 B를 1칸 오른쪽 시프트: $1.11_2 \times 2^0 = 0.111_2 \times 2^1$. ② 유효숫자 덧셈: $-1.100_2 + 0.111_2 = -0.101_2$. ③ 정규화: $-0.101_2 \times 2^1$은 1.xxx 꼴이 아니므로 유효숫자를 왼쪽으로 1칸, 지수 1 감소 → $-1.01_2 \times 2^0$. 범위 안이라 오버플로·언더플로 없음. ④ 반올림: 필요 없음. 결과 $\mathbf{-1.01_2 \times 2^0 = -1.25}$ (검산 −3 + 1.75 = −1.25).

(3) ① 지수: 1 + 0 = 1. 16비트 지수 칸(biased): (1 + 15) + (0 + 15) − 15 = 16. ② 유효숫자 곱: $1.1_2 \times 1.11_2 = 1.11_2 + 0.111_2 = 10.101_2$. ③ 정규화: $10.101_2 = 1.0101_2 \times 2^1$이므로 지수 1 증가 → 실제 지수 2, 지수 칸 17 = `10001`. ④ 반올림: 필요 없음. ⑤ 부호: 음 × 양 = 음. 결과 $\mathbf{-1.0101_2 \times 2^2 = -5.25}$ (검산 −3 × 1.75 = −5.25).

(4) bias = $2^{5-1} - 1 = \mathbf{15}$.

A + B = −1.25: 32비트 = 1 / 01111111 / 0100…0 → **0xBFA00000**. 16비트 = 1 / 01111 / 0100000000 → **0xBD00**.

A × B = −5.25: 32비트 = 1 / 10000001 / 0101000…0 → **0xC0A80000**. 16비트 = 1 / 10001 / 0101000000 → **0xC540**.

- **2점** (1) 각 1점.
- **3점** (2) 정렬 1점, 유효숫자 계산 1점, 정규화(왼쪽 시프트·지수 감소)와 결과 1점.
- **3점** (3) 지수 칸(biased) 합에서 bias 한 번 빼기 1점, 유효숫자 곱 10.101_2 1점, 정규화(지수 증가)와 부호·결과 1점.
- **2점** (4) bias 식 포함 다섯 값 중 넷 이상 2점, 둘~셋 1점.

(3)에서 유효숫자 곱 10.101_2을 정규화하지 않고 그대로 가수 칸에 "0101"이 아닌 "10101"을 넣거나, 지수를 그대로 1로 두는 실수가 흔하다. 곱이 2 이상이면 오른쪽으로 1칸, 지수 +1이다. (2)는 반대로 결과가 1보다 작아져 왼쪽 시프트·지수 −1이 필요하다. 덧셈과 곱셈의 정규화 방향을 함께 확인한다.

추가 점수를 받으려면: 16비트에서 지수 칸 00000과 11111이 예약되어 실제 지수 범위가 −14 ~ 15라는 점, 정렬 시 밀려난 비트를 위해 IEEE 754가 guard·round 비트를 둔다는 점을 덧붙인다.

근거 [Ch3 p.28, p.30, p.33, p.36, p.38]

9. **8점** (1) 다음 제어 신호 표를 0, 1, X로 채우시오. 열 순서는 제어 유닛 출력 순서이다. (4점)

명령어	Branch	MemRead	MemtoReg	ALUOp1	ALUOp0	MemWrite	ALUSrc	RegWrite
sw								
sub								
beq								
lw								

(2) `beq x5, x6, L`에서 x5 = x6일 때와 x5 ≠ x6일 때 ALU 제어 입력, Zero, PCSrc의 값과 다음 PC를 각각 쓰시오. (2점)

(3) 고장으로 ALUSrc가 항상 0이 되었다. `lw x5, 8(x6)`을 실행하면 데이터 메모리의 어느 주소를 읽게 되는가? 명령어 비트로 이유를 쓰시오. (2점)

정답과 채점 기준

명령어	Branch	MemRead	MemtoReg	ALUOp1	ALUOp0	MemWrite	ALUSrc	RegWrite
sw	0	0	X	0	0	1	1	0
sub	0	0	0	1	0	0	0	1
beq	1	0	X	0	1	0	0	0
lw	0	1	1	0	0	0	1	1

(2) 둘 다 ALU 제어 = `0110` (뺄셈), Branch = 1. x5 = x6이면 x5 − x6 = 0 → Zero = 1 → PCSrc = 1 → 다음 PC = **PC + 오프셋** (L의 주소). x5 ≠ x6이면 Zero = 0 → PCSrc = 0 → 다음 PC = **PC + 4**.

(3) ALUSrc = 0이면 ALU 둘째 입력이 즉시값 대신 레지스터 파일의 읽기 데이터 2가 된다. 읽기 레지스터 2에는 명령어 [24:20]이 연결되어 있다. lw는 I형이라 [24:20]은 즉시값의 아래 5비트 imm[4:0] = 8 = `01000` 이다. 그래서 **x6 + x8**(레지스터 x8의 값) 주소를 읽는다.

- **4점** (1) 행마다 1점(한 칸이라도 틀리면 그 행 0점). sub 행은 슬라이드의 R-format 행과 같다.
- **2점** (2) 두 경우 각 1점(Zero, PCSrc, 다음 PC가 모두 맞아야).
- **2점** (3) 둘째 입력이 읽기 데이터 2로 바뀜 1점, [24:20] = 01000 → x8이라 주소 x6 + x8 1점.

sw의 MemtoReg에 0이나 1을 쓰면 슬라이드 표와 다르다. 슬라이드 표는 X다. 레지스터에 쓰지 않으므로(RegWrite = 0) 무엇을 골라도 상관없다. beq의 ALUOp를 10으로 쓰는 실수가 있다. beq는 funct를 보지 않고 ALUOp = 01로 바로 뺄셈이다. (3)에서 "주소 x6 + 0"이나 "x6 + 8"이라고 답하면 MUX가 무엇을 고르는지 따라가지 않은 것이다.

추가 점수를 받으려면: PCSrc는 제어 유닛이 opcode만으로 정할 수 없는 유일한 신호(Branch AND Zero)라는 점, ALUOp 00/01/10이 각각 덧셈/뺄셈/funct로 결정이라는 점을 덧붙인다.

근거 [Ch4 p.18–21, p.25, p.27–28]

---

10. **8점** RISC-V의 설계 원칙 세 가지를 쓰고, 각 원칙이 RISC-V 명령어 집합에서 어떻게 드러나는지 예를 하나씩 들어 설명하시오. 세 번째 원칙은 S형 명령어 형식과 연결해 설명하시오.

정답과 채점 기준

원칙 1. 단순함은 규칙성을 좋아한다(Simplicity favors regularity). 모든 산술 명령어가 피연산자 3개(소스 둘, 목적지 하나)라는 같은 꼴을 갖는다. 그래서 a = b + c + d + e는 명령어 3개가 된다. 모든 명령어가 32비트로 같은 길이인 것도 규칙성이다. 규칙성은 구현을 단순하게 하고, 단순함은 낮은 비용으로 높은 성능을 낸다.

원칙 2. 작을수록 빠르다(Smaller is faster). 레지스터를 32개(x0~x31)로 제한한다. 레지스터가 많으면 클럭 사이클이 길어질 수 있어 개수와 성능 사이의 절충이다.

원칙 3. 좋은 설계에는 적절한 절충이 필요하다(Good design demands good compromises). 형식이 여러 개(R, I, S, SB, U, UJ)면 해독이 복잡해지지만, 모든 명령어를 32비트로 통일할 수 있다. 대신 형식을 최대한 비슷하게 유지한다. S형은 12비트 즉시값을 imm[11:5]와 imm[4:0]로 나누어, rs1·rs2 필드를 모든 형식에서 같은 자리에 두고 opcode·funct3도 같은 크기·같은 자리에 둔다.

- **3점** 원칙 이름 각 1점(영어나 한국어 중 하나로 정확하면 인정).
- **3점** 원칙마다 RISC-V 예 1점(원칙 1: 피연산자 3개 또는 32비트 고정, 원칙 2: 레지스터 32개와 클럭, 원칙 3: 형식 여러 개지만 32비트 통일).
- **2점** S형 즉시값을 둘로 나누는 이유(rs1·rs2 위치 고정) 1점, opcode·funct3도 같은 자리라는 점 1점.

"원칙 3 = 흔한 경우를 빠르게"처럼 8대 아이디어와 설계 원칙을 섞는 실수가 있다. 8대 아이디어는 Ch1, 설계 원칙 세 가지는 Ch2에서 나온다. 원칙 2를 "레지스터가 많을수록 빠르다"로 거꾸로 쓰는 실수도 있다.

추가 점수를 받으려면: SB·UJ형의 비트 흩뿌리기가 어셈블러에 일을 더 주는 대신 ImmGen의 MUX를 줄여 하드웨어를 단순하게 한다는 점(Ch4 p.22–23)을 원칙 1·3의 예로 덧붙이고, x0을 상수 0으로 고정해 sub x1, x0, x1로 부호를 바꾸는 예를 든다.

근거 [Ch2 p.9–10, p.16, p.21, p.23–24] [Ch4 p.22–23]

회상 점검

답을 쓴 뒤 채점 기준으로 스스로 채점하고, 6점 미만인 주제는 '불안한 개념 목록'에 적는다. Claude 세션에 답을 붙여 넣으면 맞은 점·틀린 점·빠진 디테일을 짚어 준다.

노트와 슬라이드를 덮고, 친구에게 말로 설명하듯 문장으로 쓴다. 키워드만 나열하면 시험장에서 문장이 나오지 않는다. 숫자(쪽수에 나온 예제 수치)는 정확히 쓴다.

Ch1 컴퓨터 추상화와 기술

CPU 실행 시간 공식을 세 요소로 쓰고, 각 요소를 정하는 것이 무엇인지 설명하라. 하드웨어 설계자가 성능을 높이는 두 가지 방법과 그 사이의 절충도 설명하라.

10점 채점 기준 보기

- **3점** CPU 시간 = 명령어 수 × CPI × 클럭 주기 (= 명령어 수 × CPI ÷ 클럭 속도). 초/프로그램 = 명령어/프로그램 × 사이클/명령어 × 초/사이클로 단위까지.
- **3점** 명령어 수 = 프로그램, ISA, 컴파일러. CPI = CPU 하드웨어(명령어마다 사이클이 달라 평균). 클럭 주기 = CPU 하드웨어.
- **2점** 설계자의 두 방법: 프로그램에 필요한 클럭 사이클 수 줄이기, 클럭 주기 줄이기(클럭 속도 올리기).
- **2점** 둘 사이에 절충(trade-off)이 있다: 클럭을 빠르게 하면 사이클 수가 늘 수 있다.

자주 빠뜨리는 디테일: CPI가 "평균"이라는 점(같은 CPU에서도 프로그램마다 다름), Ch4 p.2의 "CPU 하드웨어 = 마이크로아키텍처, 공정 기술". 교재 예제(Ch1 p.27·28에서 지정한 교재 쪽)의 풀이 흐름도 말로 설명할 수 있어야 한다.

응답 시간과 처리량을 정의하고, 누가 어느 쪽에 관심이 있는지 말하라. 프로세서를 더 빠른 것으로 바꿀 때와 프로세서를 추가할 때 각각 무엇이 좋아지는지 설명하라. CPU 시간과 경과 시간(wall clock time)의 차이도 설명하라.

10점 채점 기준 보기

- **2점** 응답 시간(실행 시간) = 작업 시작부터 끝까지 시간, 개인 사용자가 줄이고 싶어 함.
- **2점** 처리량 = 정해진 시간에 한 일의 총량(대역폭), 데이터센터 관리자가 늘리고 싶어 함.
- **2점** 빠른 프로세서로 교체 → 둘 다 좋아짐. 프로세서 추가 → 처리량만.

- **2점** 성능 = 1 ÷ 실행 시간, "X가 Y보다 n배 빠르다" = 실행 시간$_Y$ ÷ 실행 시간$_X$ = n.
- **2점** 경과 시간은 디스크·메모리 접근, I/O, OS 오버헤드까지 전부 포함. CPU 시간은 CPU가 그 작업에 쓴 시간만(I/O·다른 프로그램 시간 제외), 사용자 CPU 시간과 시스템 CPU 시간으로 나뉨.

자주 빠뜨리는 디테일: 성능 비를 실행 시간 비로 바꿀 때 분자·분모가 뒤집힌다는 점, "멀티코어의 이점은 응답 시간보다 처리량 쪽"(p.32)과의 연결.

전력 장벽(power wall)을 설명하라. 동적 전력 식, 지난 20년 동안 클럭과 전력이 늘어난 배수와 그 이유, 장벽의 두 원인, 그 결과 생긴 설계 변화를 포함하라.

10점 채점 기준 보기

- **2점** CMOS 동적 에너지 ∝ 용량성 부하 × 전압^2, 동적 전력 ∝ 용량성 부하 × 전압^2 × 주파수.
- **3점** 클럭 약 1000배, 전력 약 30배. 전압이 5 V → 1 V로 낮아졌기 때문(세대마다 약 15%씩).
- **3점** 전압을 더 낮출 수 없음(누설, leakage), 열을 더 제거할 수 없음(냉각 비용).
- **2점** 결과: 2006년 무렵부터 칩당 여러 코어(멀티코어). 이점은 응답 시간보다 처리량 쪽에 더 큼.

자주 빠뜨리는 디테일: 전압이 "제곱"으로 들어간다는 점, 포스트 PC 시대에 가장 귀한 자원이 에너지라는 슬라이드 문장(p.30).

암달의 법칙을 식으로 쓰고, 슬라이드 예제(100초 프로그램, 곱셈 80초)를 처음부터 끝까지 풀어 설명하라. 이 법칙이 주는 교훈과 MIPS가 성능 지표로 부족한 이유도 말하라.

10점 채점 기준 보기

- **2점** 개선 후 시간 = 영향받는 시간 ÷ 개선 배수 + 영향받지 않는 시간.
- **3점** 2.5배(40초) 목표: 40 = 80 ÷ n + 20 → n = 4.
- **2점** 곱셈 외(20초)만 개선: 40 = 80 ÷ n + 20은 불가능, 최대 100 ÷ 80 = 1.25배.
- **1점** 교훈: 흔한 경우를 빠르게(make the common case fast).
- **2점** MIPS = 명령어 수 ÷ (실행 시간 × 10^6) = 클럭 속도 ÷ (CPI × 10^6). ISA 차이와 명령어 복잡도 차이를 반영하지 못하고, 같은 CPU에서도 CPI가 프로그램마다 달라 MIPS가 달라진다.

자주 빠뜨리는 디테일: 비율은 명령어 개수 비율이 아니라 시간 비율로 넣는다는 점, MIPS 식에서 10^6.

컴퓨터의 5대 고전적 구성 요소를 쓰고, 각 요소와 메모리 사이에서 데이터가 어느 방향으로 흐르는지 설명하라. 캐시, SRAM, DRAM, 휘발성·비휘발성 메모리의 관계와 ISA·ABI·구현의 뜻도 함께 설명하라.

10점 채점 기준 보기

- **2점** 입력, 출력, 메모리, 데이터패스, 제어. 프로세서 = 데이터패스 + 제어. 데스크톱·서버·임베디드 모두 같은 구성.
- **2점** 입력은 메모리에 쓰고, 출력은 메모리에서 읽는다. 프로세서는 메모리에서 명령어와 데이터를 가져온다. 제어는 데이터패스·메모리·입출력의 동작을 정하는 신호를 보낸다.
- **2점** 캐시 = DRAM의 버퍼인 작고 빠른 메모리, SRAM으로 만듦. SRAM은 DRAM보다 빠르지만 밀도가 낮아 비쌈. 둘은 메모리 계층의 두 층.
- **2점** 휘발성(주 메모리, DRAM·SRAM)은 전원이 꺼지면 내용을 잃음. 비휘발성(2차 메모리, 디스크·플래시·광디스크)은 유지, 더 크고 느림.
- **2점** ISA = 하드웨어와 최하위 소프트웨어 사이 인터페이스의 추상화. ABI = ISA + OS 인터페이스. 구현 = 그 추상화를 따르는 하드웨어.

자주 빠뜨리는 디테일: 입력·출력의 방향을 거꾸로 말하기, iPhone XS Max에서 I/O가 대부분이라는 예(p.16).

Ch2 명령어: 컴퓨터의 언어

RISC-V에서 메모리 주소가 어떻게 매겨지는지 설명하라. 바이트 주소, 연속한 word 주소 차이, 엔디언(0x00112233을 저장하는 예로), 정렬 제약을 모두 포함하라. 배열 원소 A[8]을 읽는 lw의 오프셋이 왜 32인지도 설명하라.

10점 채점 기준 보기

- **2점** 메모리는 0부터 시작하는 1차원 큰 배열, 주소가 인덱스. 거의 모든 구조가 바이트 단위로 주소를 매김.
- **2점** word(32비트)의 주소는 그 안 4바이트 중 하나의 주소. 연속한 word의 주소는 4씩 차이.
- **3점** 빅 엔디언: 최상위 바이트가 가장 낮은 주소(00 11 22 33). 리틀 엔디언: 최하위 바이트가 가장 낮은 주소(33 22 11 00). RISC-V는 리틀 엔디언.
- **1점** RISC-V는 정렬 제약이 없다(word가 4의 배수 주소에서 시작하지 않아도 됨).
- **2점** int는 4바이트라 A[8]의 바이트 오프셋 = 8 × 4 = 32, 기준 레지스터에 시작 주소.

자주 빠뜨리는 디테일: 산술 명령어는 메모리에 직접 접근하지 못해 load/store가 필요하다는 점(p.15), 자주 안 쓰는 변수만 메모리로 spill.

프로시저 실행 6단계를 순서대로 말하고, 각 단계에서 RISC-V가 쓰는 레지스터·명령어(x10~x17, x1, jal, jalr, sp)를 연결하라.

10점 채점 기준 보기

- **3점** 6단계: ① 인자를 프로시저가 접근할 곳에 둔다 ② 제어를 프로시저로 넘긴다 ③ 필요한 저장 공간 확보 ④ 작업 수행 ⑤ 결과를 호출자가 접근할 곳에 둔다 ⑥ 호출한 지점으로 제어를 돌려준다.
- **2점** 인자·반환값 레지스터 x10~x17(8개), 반환값은 x10·x11. 복귀 주소 레지스터 x1.
- **2점** jal(jump-and-link): 목표로 분기하면서 다음 명령어 주소(PC + 4)를 rd(x1)에 저장. jump = 제어 넘기기, link = 돌아올 연결.
- **2점** jalr x0, 0(x1): x1의 주소로 간접 분기, 목적지 x0이라 복귀 주소를 버림.
- **1점** ③의 저장 공간은 스택(sp = x2를 빼서 확보, 높은 주소 → 낮은 주소로 자람).

자주 빠뜨리는 디테일: 인자가 8개를 넘거나 반환값이 2개를 넘으면 메모리(스택)를 쓴다는 점, RISC-V에는 push·pop이 따로 없다는 점.

레지스터 보존 규약을 설명하라. 호출자 보존과 피호출자 보존 레지스터의 번호 범위, 각각 누가 언제 저장하는지, 슬라이드 leaf_example에서 x20만 저장하면 되는 이유, 중첩 호출에서 x1과 x10에 생기는 문제를 포함하라.

10점 채점 기준 보기

- **2점** 호출자 보존(caller-saved, 임시): x5~x7, x28~x31. 피호출자가 보존하지 않으므로 호출 뒤 바뀔 수 있음, 필요하면 호출자가 호출 전에 저장.
- **2점** 피호출자 보존(callee-saved): x8~x9, x18~x27. 피호출자가 쓰려면 쓰기 전에 저장, 돌아가기 전에 복원.
- **2점** leaf_example: x5·x6은 임시라 저장 불필요, x20은 보존 레지스터라 저장 필요.
- **2점** 중첩 호출: A가 B를 jal로 부르면 x1(복귀 주소)과 x10(인자)이 덮어써짐 → 호출 전에 스택에 저장, 호출 후 복원.
- **2점** p.43 정리: 호출자는 x10~x17, x5~x7, x28~x31을, 피호출자는 x1, x8~x9, x18~x27을 저장.

자주 빠뜨리는 디테일: p.39의 "19개 레지스터를 두 그룹"과 p.43의 확장 규약(호출자 쪽에 x10~x17, 피호출자 쪽에 x1)이 다르게 보인다는 점. 재귀 fact에서 x1과 x10(n)을 저장하는 이유.

RISC-V 명령어 형식 R, I, S, SB, U, UJ의 필드 배치를 왼쪽(31번 비트)부터 비트 수와 함께 말하고, 각 형식을 쓰는 명령어를 하나씩 들어라. S형이 즉시값을 둘로 나누는 이유도 설명하라.

10점 채점 기준 보기

- **2점** R: funct7(7) rs2(5) rs1(5) funct3(3) rd(5) opcode(7). 예 add, sub.
- **2점** I: imm[11:0](12) rs1 funct3 rd opcode. 예 lw, addi, jalr. S: imm[11:5](7) rs2 rs1 funct3 imm[4:0](5) opcode. 예 sw.
- **2점** SB: imm[12] imm[10:5] rs2 rs1 funct3 imm[4:1] imm[11] opcode. 예 beq, bne. 0번 비트는 저장 안 함.
- **2점** U: imm[31:12](20) rd opcode, 예 lui. UJ: imm[20|10:1|11|19:12] rd opcode, 예 jal.
- **2점** S형 분할 이유: rs1·rs2 필드를 모든 형식에서 같은 자리에, opcode·funct3도 같은 크기·자리에 두기 위해(설계 원칙 3, 해독을 쉽게).

자주 빠뜨리는 디테일: 모든 명령어가 32비트(규칙성), 시프트 즉시값 명령어(slli 등)는 I형이지만 위 7비트가 funct7 역할을 한다는 점.

PC 상대 주소 지정을 설명하라. 왜 PC를 기준 레지스터로 쓰는지, 분기와 jal이 닿을 수 있는 범위(바이트와 word 단위), 아주 먼 주소로 점프하는 방법을 포함하라. RISC-V의 주소 지정 방식 네 가지와 각각의 예 명령어도 말하라.

10점 채점 기준 보기

- **2점** 주소를 20비트에 통째로 넣으면 프로그램이 2^{20}바이트(1 MiB)를 넘지 못해 비현실적. 그래서 레지스터 + 분기 오프셋 방식.
- **2점** 대부분의 분기 목표는 가까이(if/for/while)에 있으므로 PC가 이상적인 기준 레지스터. 목표 = PC + 오프셋.
- **2점** SB형 분기: −4096 ~ 4094바이트(2의 배수, 짝수 주소만), word로는 $\pm 2^{10}$. jal: $\pm 2^{18}$ words.
- **2점** 먼 점프: lui로 주소의 12~31번 비트를 임시 레지스터에, jalr이 하위 12비트를 더해 그 주소로 점프.
- **2점** 즉시(addi), 레지스터(add), 기준/변위(lw, sw, jalr), PC 상대(beq, jal). 한 연산이 여러 방식을 쓸 수 있음(add/addi).

자주 빠뜨리는 디테일: p.47 범위는 바이트, p.48 범위는 word 단위라는 차이. "한 연산은 한 방식만"은 틀린 말이다.

CISC와 RISC를 비교하라. 각각의 특징, 장점, 단점, 예를 들고, 현대 x86 프로세서가 어떻게 구현되는지와 ISA와 마이크로아키텍처의 차이도 설명하라.

10점 채점 기준 보기

- **2점** CISC: 오래된 설계, 복잡한 가변 길이 명령어, 강력한 명령어 다수, 예 x86. 장점: 어셈블리 프로그래밍·컴파일러가 쉬움, 명령어 메모리 사용 적음. 단점: CPU 설계가 훨씬 어려움.
- **2점** RISC: 단순하고 표준화된 명령어, 작은 명령어 집합(CISC 연산은 RISC 연산의 사슬), 예 ARM, MIPS, SPARC, RISC-V.
- **2점** RISC 장점: CPU 설계가 쉬움, 명령어 집합이 작아 클럭 속도가 높음. 단점: 어셈블리가 길어짐(컴파일러 문제), 메모리를 많이 씀.
- **2점** 현대 x86(CISC) 대부분은 RISC 기법으로 구현(명령어를 마이크로명령어로 변환).
- **2점** 마이크로아키텍처 = 특정 설계 제약·목표 아래 ISA의 구현. ISA를 만족하면 여러 구현 가능(286, 386, Pentium, Core i). 파이프라이닝·추측 실행처럼 소프트웨어에 드러나지 않는 하드웨어 기법.

자주 빠뜨리는 디테일: RISC-V는 UC Berkeley의 다섯 번째 RISC ISA이고 완전 개방 표준이라는 점, ISA끼리 비슷한 두 가지 이유(비슷한 하드웨어 기술, 모든 컴퓨터가 제공해야 하는 기본 연산).

Ch3 컴퓨터 연산

정수 덧셈·뺄셈의 오버플로를 설명하라. 부호 있는 수에서 오버플로가 생길 수 있는 경우와 생길 수 없는 경우, 판정 규칙, 부호 없는 수의 판정, 언어별 처리를 포함하라.

10점 채점 기준 보기

- **2점** 정의: 연산 결과를 주어진 하드웨어(비트 수)로 표현할 수 없을 때. 32비트 덧셈 결과가 33비트가 필요할 때 부호 비트 자리에 값이 들어간다.
- **2점** 부호가 다른 두 수의 덧셈, 부호가 같은 두 수의 뺄셈은 오버플로가 생기지 않는다(합이 두 피연산자 사이, 뺄셈 = 부정해서 더하기).
- **2점** 덧셈: + + + = −, − + − = +. 뺄셈: + − (−) = −, − − (+) = +.
- **2점** 부호 없는 수: 덧셈 S < A 또는 S < B이면 오버플로, 뺄셈 S > A이면 오버플로. 메모리 주소에 쓰여 보통 무시.
- **2점** C/C++·Java는 정수 오버플로를 무시, Ada·Fortran은 프로그램에 알리도록 요구. 처리 책임은 프로그래머.

자주 빠뜨리는 디테일: 슬라이드 예(0x00000001 + 0x7FFFFFFF = 음수), "−10 + 4 = −6"으로 부호가 다른 덧셈이 안전한 이유를 보이는 예.

개선된(refined) 순차 곱셈 하드웨어의 구성과 동작을 설명하고, $0010_2 \times 0011_2$을 반복마다 곱 레지스터 값이 어떻게 바뀌는지 따라가라. 처음 버전과 무엇이 달라졌는지도 말하라.

10점 채점 기준 보기

- **2점** 처음 버전: 64비트 피승수 레지스터·64비트 ALU·32비트 승수 레지스터·64비트 곱 레지스터, 검사-더하기-시프트(피승수 왼쪽, 승수 오른쪽).
- **2점** 개선 버전: 32비트 피승수, 32비트 ALU, 승수 레지스터 없음, 64비트 곱 레지스터 오른쪽 절반에 승수.
- **2점** 동작: 곱 레지스터 LSB가 1이면 피승수를 왼쪽 절반에 더함, 그다음 곱 레지스터 전체를 오른쪽으로 1비트 시프트. 피승수 대신 곱을 시프트.
- **3점** 0000 0011 → (더함) 0010 0011 → (시프트) 0001 0001 → (더함) 0011 0001 → 0001 1000 → (더하지 않음) 0000 1100 → 0000 0110. 결과 6.
- **1점** 곱의 길이 = 피승수 길이 + 승수 길이.

자주 빠뜨리는 디테일: 반복 횟수 = 비트 수(4비트면 4번), 여러 덧셈기를 트리로 쓰면 빨라진다(비용-성능 절충).

RISC-V 곱셈 명령어 네 개를 설명하고, 슬라이드 3비트 예($101_2 \times 101_2$)의 세 모드 결과를 Upper·Lower까지 말하라. 아래 절반이 같은 이유도 설명하라.

10점 채점 기준 보기

- **3점** mul = 64비트 곱의 하위 32비트. mulh = 둘 다 부호 있을 때 상위 32비트. mulhu = 둘 다 부호 없을 때 상위. mulhsu = 하나는 부호 있고 하나는 없을 때 상위.
- **4점** Unsigned 5 × 5 = 25 (011001) → 011 / 001. Signed (−3) × (−3) = 9 (001001) → 001 / 001. Signed-Unsigned (−3) × 5 = −15 (110001) → 110 / 001.
- **3점** Lower는 모드와 상관없이 같고 Upper만 다르다. 부호 있게 읽은 값과 없게 읽은 값은 2^n의 배수만큼 차이 나서 아래 n비트에 영향이 없기 때문.

자주 빠뜨리는 디테일: 부호 있는 곱셈의 다른 방법(양수로 바꿔 31번 반복 후 부호가 다르면 부정), −15를 6비트 2의 보수 110001로 바꾸는 과정.

부호 있는 나눗셈 규칙 세 단계를 말하고, −7 ÷ 2, 7 ÷ (−2), (−7) ÷ (−2)의 몫과 나머지를 이유와 함께 설명하라. RISC-V 나눗셈 명령어와, 나눗셈을 빠르게 하기 어려운 이유도 말하라.

10점 채점 기준 보기

- **3점** ① 부호를 무시하고 절댓값으로 몫·나머지 계산 ② 부호가 다르면 몫을 부정 ③ 나머지 부호를 피제수와 같게.

- **3점** −7 ÷ 2: Q = −3, R = −1. 7 ÷ (−2): Q = −3, R = 1. (−7) ÷ (−2): Q = 3, R = −1. Q = −4, R = 1은 식은 맞지만 쓰지 않음.
- **2점** div, divu, rem, remu(부호 있는·없는 몫과 나머지).
- **2점** 각 단계 뺄셈이 나머지 부호에 따라 조건부라 곱셈처럼 병렬화 불가. SRT 나눗셈은 피제수·나머지 상위 비트로 표를 찾아 몫 비트 여러 개를 예측.

자주 빠뜨리는 디테일: 피제수 = 몫 × 제수 + 나머지, 0으로 나누기는 수학적으로 불가, 2의 거듭제곱 나눗셈은 오른쪽 시프트(음수는 부호 확장).

IEEE 754 단정도 표현을 설명하라. 칸 구성, 값 계산식, 숨은 비트, bias를 쓰는 이유, 예약된 지수, 표현 범위를 포함하고, −0.75를 32비트로 바꾸는 과정을 말로 보여라.

10점 채점 기준 보기

- **2점** 부호 1, 지수 8, 가수 23비트(배정도 1·11·52). 값 = $(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$.
- **2점** 숨은 비트: 정규화된 수는 항상 1.xxx라 앞의 1을 저장하지 않음, 유효숫자 24비트. 1.0 ≤ significand < 2.0.
- **2점** bias 127(배정도 1023). 정수 비교로 정렬하기 쉽게: 부호가 MSB, 지수가 가수 앞, 음수 지수도 부호 없는 수가 되도록.
- **2점** 지수 칸 00000000과 11111111은 예약. 실제 지수 −126 ~ 127. 범위 약 $\pm 1.2 \times 10^{-38}$ ~ $\pm 3.4 \times 10^{38}$.
- **2점** $-0.75 = -0.11_2 = -1.1_2 \times 2^{-1}$. S = 1, 지수 칸 −1 + 127 = 126 = 01111110_2, 가수 100…0 → 0xBF400000.

자주 빠뜨리는 디테일: Ch3 p.25는 지수를 "2의 보수"라고 적었지만 IEEE 754는 bias 방식(p.28)으로 답한다. 오버플로(지수가 너무 큼)와 언더플로(음의 지수가 너무 큼)의 정의, 0/0 같은 무효 연산은 NaN.

부동소수점 덧셈 4단계와 곱셈 5단계를 슬라이드 예 0.5 + (−0.4375)와 0.5 × (−0.4375)로 설명하라. 곱셈에서 biased 지수를 더할 때 주의할 점, guard·round·sticky 비트와 짝수 쪽 반올림 규칙도 말하라.

10점 채점 기준 보기

- **3점** 덧셈: ① 지수가 작은 수를 오른쪽 시프트해 정렬($-1.110_2 \times 2^{-2} \rightarrow -0.111_2 \times 2^{-1}$) ② 유효숫자 덧셈($1.000_2 - 0.111_2 = 0.001_2$) ③ 정규화·오버플로/언더플로 검사($1.000_2 \times 2^{-4}$) ④ 반올림. 결과 0.0625.
- **3점** 곱셈: ① 지수 더하기(−1 + (−2) = −3) ② 유효숫자 곱($1.000_2 \times 1.110_2 = 1.110_2$) ③ 정규화·검사 ④ 반올림 ⑤ 부호 결정. 결과 $-1.110_2 \times 2^{-3} = -0.21875$.
- **2점** biased 지수끼리 더하면 bias가 두 번 들어가므로 한 번 뺀다: (−1 + 127) + (−2 + 127) − 127.

- **2점** guard = 가수 LSB 바로 다음 비트, round = 그다음, sticky = 나머지 비트의 OR. GRS 000~011 버림, 101~111 올림, 100은 LSB가 0인 쪽으로(짝수 쪽 반올림, 기본 모드).

자주 빠뜨리는 디테일: 반올림 뒤 정규화가 깨지면 3단계를 다시 함($9.9999 \times 10^2 \rightarrow 10.000 \times 10^2 \rightarrow 1.000 \times 10^3$), 반올림 모드 4가지(0 쪽, $+\infty$ 쪽, $-\infty$ 쪽, 가까운 짝수).

Ch4 프로세서(단일 사이클까지)

단일 사이클 데이터패스에서 `lw x1, offset(x2)`가 지나가는 부품을 순서대로 설명하라. 각 부품에서 어떤 값이 만들어지고, 명령어의 어느 비트가 어디로 가는지, 이때의 제어 신호 값까지 말하라.

10점 채점 기준 보기

- **2점** PC → 명령어 메모리에서 명령어 인출, 동시에 PC + 4 가산기.
- **2점** [19:15](rs1 = x2) → 레지스터 파일 읽기 레지스터 1 → x2 값. [31:0] → ImmGen → 12비트 offset을 32비트로 부호 확장.
- **2점** ALUSrc MUX가 즉시값 선택 → ALU가 x2 + offset(덧셈, ALUOp 00) → 데이터 메모리 주소.
- **2점** 데이터 메모리 읽기(MemRead = 1) → MemtoReg MUX가 메모리 값 선택(MemtoReg = 1) → [11:7](rd = x1)의 쓰기 레지스터에 기록(RegWrite = 1).
- **2점** 제어 신호 전체: ALUSrc 1, MemtoReg 1, RegWrite 1, MemRead 1, MemWrite 0, Branch 0, ALUOp 00. PCSrc = 0이라 다음 PC = PC + 4.

자주 빠뜨리는 디테일: 처음 두 단계(인출, 레지스터 읽기)는 모든 명령어가 같다는 점, 이 경로가 다섯 단위를 거쳐 가장 길어 클럭 주기를 정한다는 점.

기본 데이터패스에 MUX와 제어 유닛이 필요한 이유를 말하고, 세 MUX 각각이 고르는 두 입력과 그 선택 신호, 어떤 명령어가 어느 쪽을 고르는지 설명하라. 메모리가 두 개인 이유도 말하라.

10점 채점 기준 보기

- **2점** 데이터 선은 그냥 묶을 수 없다 → 여러 출처 중 고르는 MUX 필요. 데이터패스를 조종할 제어가 없다 → 제어 유닛 필요. 프로세서 = 데이터패스 + 제어.
- **2점** ALUSrc MUX: 레지스터 읽기 데이터 2(0, R형·beq) vs 부호 확장 즉시값(1, lw·sw).
- **2점** MemtoReg MUX: ALU 결과(0, R형) vs 데이터 메모리 읽은 값(1, lw).

- **2점** PCSrc MUX: PC + 4(0) vs 분기 목표 PC + 오프셋(1, beq이고 Zero = 1).
- **2점** 단일 사이클에서는 한 명령어 동안 어떤 자원도 두 번 쓸 수 없으므로 명령어 메모리와 데이터 메모리를 분리.

자주 빠뜨리는 디테일: MemtoReg MUX는 슬라이드 그림에서 위가 1, 아래가 0이라는 점(다른 MUX와 반대), 분기 오프셋의 1비트 시프트는 ImmGen 쪽에서 처리되고 별도 시프트 상자는 없다는 점.

제어 신호 7개(RegWrite, ALUSrc, MemRead, MemWrite, MemtoReg, Branch, ALUOp)와 PCSrc가 각각 0과 1일 때 하는 일을 설명하고, R형·lw·sw·beq의 값을 표로 써라. X가 나오는 자리와 이유도 말하라.

10점 채점 기준 보기

- **3점** 각 신호의 의미: RegWrite = 쓰기 레지스터에 쓰기 데이터 기록, ALUSrc = 둘째 ALU 입력이 즉시값, MemRead/MemWrite = 데이터 메모리 읽기/쓰기, MemtoReg = 레지스터에 쓸 값이 메모리 값, PCSrc = PC를 분기 목표로, ALUOp = ALU 연산 종류(2비트).
- **4점** 표(ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, ALUOp1, ALUOp0): R형 0 0 1 0 0 0 1 0 / lw 1 1 1 1 0 0 0 0 / sw 1 X 0 0 1 0 0 0 / beq 0 X 0 0 0 1 0 1. 행마다 1점.
- **2점** X는 sw·beq의 MemtoReg: RegWrite = 0이라 레지스터에 아무것도 쓰지 않으므로 MUX가 무엇을 골라도 결과가 같다.
- **1점** PCSrc만 opcode·funct로 정할 수 없고 beq AND Zero로 정해진다.

자주 빠뜨리는 디테일: 제어 유닛 입력은 Instruction[6:0](opcode), sw의 RegWrite는 X가 될 수 없다(1이면 엉뚱한 레지스터를 덮어씀).

ALU 제어를 설명하라. ALU 제어 4비트 값과 연산, 명령어 종류별로 ALU가 해야 하는 일, ALUOp의 뜻, ALU 제어 유닛의 입력, 다단 디코딩의 이점, 진리표에 don't care를 쓰는 이유를 포함하라.

10점 채점 기준 보기

- **2점** 0000 AND, 0001 OR, 0010 add, 0110 subtract.
- **2점** load·store = 주소 계산 덧셈, branch = 뺄셈 후 0인지 검사, R형 = funct7·funct3에 따라 AND/OR/add/sub.
- **2점** ALUOp(주 제어 유닛의 2비트 출력): 00 덧셈, 01 뺄셈, 10 funct로 결정. ALU 제어 입력 = ALUOp + funct7 + funct3.
- **2점** 다단 디코딩: 주 제어가 ALUOp를 만들고, ALUOp로 실제 ALU 제어 신호를 만든다. 주 제어 유닛 크기를 줄이고 제어 지연도 줄일 수 있다.
- **2점** 입력 12비트라 전체 진리표는 2^{12} = 4096행. 실제 나오는 조합만 쓰고 나머지 입력은 X(don't care)로 둬 표를 압축, 그다음 최적화해 게이트로.

자주 빠뜨리는 디테일: ALUOp가 00이면 funct와 상관없이 0010, 데이터패스에서 ALU 제어로 가는 비트는 [30]과 [14:12].

조합 소자와 상태 소자를 구별하고, 에지 트리거 클로킹 방식이 무엇인지, 클럭 주기가 무엇으로 정해지는지 설명하라. 그리고 단일 사이클 구현을 오늘날 쓰지 않는 이유로 이어서 설명하라.

10점 채점 기준 보기

- **2점** 조합 소자: AND 게이트, ALU처럼 출력이 입력의 함수. 상태 소자: 레지스터, 메모리처럼 정보를 저장.
- **2점** 상태 소자는 입력이 최소 둘(쓸 데이터, 쓰는 때를 정하는 클럭), 출력은 이전 클럭에 쓴 값.
- **2점** 클로킹 방식 = 신호를 언제 읽고 쓰는지 정해 하드웨어를 예측 가능하게. 에지 트리거: 순차 소자의 값은 클럭 엣지에서만 갱신. 조합 논리는 엣지 사이에 상태 소자 출력을 받아 상태 소자 입력으로.
- **2점** 가장 긴 지연이 클럭 주기를 결정.
- **2점** 단일 사이클: 모든 명령어가 같은 클럭 길이, 가장 긴 load 경로(명령어 메모리 → 레지스터 파일 → ALU → 데이터 메모리 → 레지스터 파일)가 클럭을 정함, 흔한 경우를 빠르게 원칙 위반 → 파이프라이닝.

자주 빠뜨리는 디테일: 레지스터 파일과 메모리처럼 "Write" 제어 입력이 있는 상태 소자는 그 신호가 켜졌을 때만 엣지에서 쓴다는 점.

4단계 · 빈틈 모으기

아직 불안한 개념 목록

모의고사·실전 연습·회상 점검에서 틀리거나 6점 미만이었던 주제를 여기 적습니다. 다음 공부는 이 목록 맨 위부터 시작합니다. 적은 내용은 이 기기에 자동 저장됩니다.

쓰는 법

1. 한 줄에 하나씩: **주제 · 어디서 틀렸나 · 다시 볼 단원**. 예) `lw 오프셋을 인덱스 그대로 씀 · 실전 2-3번 · Ch2 · 3`
2. 다시 풀어서 맞히면 줄 앞에 `[해결]`을 붙이고, 이틀 뒤 한 번 더 맞히면 지웁니다.
3. Claude 세션에 이 목록을 붙여 넣으면, 목록 순서대로 설명과 확인 질문을 이어 갑니다.

예) IEEE 754 지수 칸을 구할 때 bias를 안 더함 · 모의고사 1회 8번 · Ch3 · 7

처음 공부하는 사람이 자주 올리는 항목

- 10진수 ↔ 2진수 ↔ 16진수 변환이 느리다 → 기초 · 2
- MIPS·CPU 시간 계산에서 단위(GHz, ns, 10^6)를 놓친다 → 기초 · 5, 실전 1
- lw/sw 오프셋에 ×4를 빼먹는다 → Ch2 · 3
- caller-saved와 callee-saved가 헷갈린다 → Ch2 · 10
- IEEE 754 지수 칸을 구할 때 실제 지수에 bias를 더하지 않거나, 맨 앞 1(숨은 1)을 가수 칸에 적는다 → Ch3 · 7
- sw·beq의 MemtoReg가 왜 X인지 설명하지 못한다 → Ch4 · 6

4단계 · 틀린 문제 모아 보기

오답 노트

스스로 확인·실전 연습 문항 아래의 **[틀림]**을 누른 문제가 단원별로 여기에 모입니다. 다시 풀어 맞히면 **[다시 풀어 맞음]**을 눌러 목록에서 빼세요. 틀린 문제는 **불안한 개념 목록**에도 한 줄씩 자동으로 들어갑니다.

시험 직전용

한눈에 정리

시험 직전에 다시 볼 표 모음입니다. 자세한 설명은 각 단원에 있습니다. 정답은 먼저 풀어 본 뒤 펼치세요.

---

1. 성능 요소 영향 표 [Ch1 p.28–29]

요소	명령어 수(IC)	CPI	클럭	이유
알고리즘	○	○		실행할 명령어 수와 종류(CPI가 다른 명령어 비중)를 정함
프로그래밍 언어	○	○		문장이 명령어로 바뀌는 방식
컴파일러	○	○		어떤 명령어를 몇 개 고르는지
ISA	○	○	○	명령어 종류·사이클·구현 모두에 영향
기술(Technology)			○	공정이 회로 속도를 정함

Technology 행은 교재 표에 없고 슬라이드에서 추가된 행으로, 클럭 칸에만 표시가 있습니다 [Ch1 p.29]. "CPI는 CPU 하드웨어가 정한다"[Ch1 p.28]와 표가 다르게 보이는 이유: 명령어별 사이클은 하드웨어가 정하지만, 그 명령어들을 어떤 비율로 섞는지는 SW가 정하기 때문.

단위 형태: 초/프로그램 = 명령어/프로그램 × 사이클/명령어 × 초/사이클 [Ch1 p.29]

2. 교재 지정 예제 (슬라이드가 쪽을 직접 지정) 교재 기준

1. A: 2 GHz에서 10초. B로 6초를 원하는데 사이클이 1.2배 필요 → B의 클럭은? [Ch1 p.27 → 교재 pp.34–35]
2. A: 클럭 주기 250 ps, CPI 2.0 / B: 500 ps, CPI 1.2 (같은 ISA) → 어느 쪽이 몇 배 빠른가? [Ch1 p.28 → 교재 pp.35–36]
3. 명령어 클래스 A·B·C의 CPI 1·2·3. 코드 1 = (2,1,2)개, 코드 2 = (4,1,1)개 → 명령어 수, 사이클 수, CPI 비교 [교재 pp.36–37]
4. 새 프로세서: 용량성 부하 85%, 전압·주파수 각각 15% 감소 → 전력은 몇 배? [Ch1 p.31 → 교재 pp.41–42]
5. 100초 중 곱셈 80초. 전체 2.5배 빠르게 하려면 곱셈을 몇 배? 곱셈 외 부분만 개선하면? [Ch1 p.33]

정답 (먼저 풀고 열기)

1. B 사이클 = $1.2 \times 20 \times 10^9 = 24 \times 10^9$ → 24×10^9 / 6초 = **4 GHz**
2. A: $I \times 2.0 \times 250$ ps = $500 \times I$ ps, B: $I \times 1.2 \times 500$ ps = $600 \times I$ ps → **A가 1.2배 빠름**
3. 코드 1: 5개, 2+2+6 = 10사이클, CPI 2.0 / 코드 2: 6개, 4+2+3 = 9사이클, CPI 1.5 → **명령어가 더 많은 코드 2가 더 빠름**
4. $0.85 \times 0.85^2 \times 0.85 = 0.85^4 \approx$ **0.52배**
5. $40 = 80/n + 20$ → **n = 4**. 곱셈 외만 개선 → 80초가 남으므로 **불가능(최대 1.25배)**

3. 명령어 형식 비트 폭 [Ch2 p.22–24, p.47, Ch4 p.21]

형식	31 ..... 0 (괄호 = 비트 수)	용도
R	`funct7(7) · rs2(5) · rs1(5) · funct3(3) · rd(5) · opcode(7)`	즉시값 없는 산술·논리 (add, sub, and, or)
I	`imm[11:0](12) · rs1(5) · funct3(3) · rd(5) · opcode(7)`	load, 즉시값 산술 (lw, addi), jalr
S	`imm[11:5](7) · rs2(5) · rs1(5) · funct3(3) · imm[4:0](5) · opcode(7)`	store (sw)
SB	`imm[12|10:5](7) · rs2 · rs1 · funct3 · imm[4:1|11](5) · opcode`	조건 분기 (beq, bne)
U	`imm[31:12](20) · rd(5) · opcode(7)`	lui
UJ	`imm[20|10:1|11|19:12](20) · rd(5) · opcode(7)`	jal

rs1·rs2·rd·opcode·funct3는 그 필드가 있는 모든 형식에서 같은 위치 → 해독 전에 레지스터를 읽을 수 있음 [Ch2 p.24]. opcode: R형 0110011, 분기 1100011(bne funct3 = 001), jal 1101111 [Ch2 p.47, 50]. p.47 bne 칸 표의 opcode 1100111은 오타(1100111은 jalr).
전체 인코딩 표(명령어별 opcode·funct3·funct7)는 Ch2 · 13 [Ch2 p.50].

4. 레지스터 규약 [Ch2 p.16, 34, 37, 39, 41, 43, 44]

레지스터	역할	호출할 때 누가 저장?
x0	항상 0 (하드와이어드)	해당 없음
x1	복귀 주소 (jal이 저장)	callee p.41·43
x2 (sp)	스택 포인터, 높은 주소 → 낮은 주소로 자람	해당 없음
x8 (fp)	프레임 포인터 (프로시저 안에서 안 변하는 기준)	callee
x10~x17	인자 8개, 그중 x10·x11 = 반환값	caller p.41·43
x5~x7, x28~x31	임시(temporary) 7개	caller
x8~x9, x18~x27	저장(saved) 12개	callee

주의: p.39의 정의는 "19개를 두 그룹"(임시 7 + 저장 12)이고 x10~17·x1은 들어 있지 않음. p.41·43의 호출 규약에서 caller 쪽에 x10~17, callee 쪽에 x1이 추가됨. "19개"를 물으면 p.39 기준으로 답할 것. 프로시저용 레지스터는 9개 = x10~x17 + x1 [Ch2 p.34].

5. 주소 지정 방식 ↔ 명령어 [Ch2 p.49]

방식	피연산자 위치	예
즉시값	명령어 안의 상수	addi, xori, srai
레지스터	레지스터	add, sub, xor
베이스(변위)	메모리[레지스터 + 상수]	lw, sw, **jalr** ← 함정
PC 상대	PC + 상수	beq, bne, jal

6. IEEE 754 요약 [Ch3 p.27–31]

	부호	지수	가수(Fraction)	Bias	실제 지수 범위
단정도(32)	1	8	23 (+숨은 1 = 24비트 정밀도)	127	−126 ~ 127 (지수 칸 1 ~ 254)
배정도(64)	1	11	52	1023	−1022 ~ 1023 (지수 칸 1 ~ 2046)

값 = $(-1)^S \times (1 + F) \times 2^{(E - Bias)}$, $1.0 \le 1 + F < 2.0$. 지수 칸 전부 0·전부 1(단정도 0·255)은 예약(NaN 등). 단정도 정규화 범위 $\pm 1.0_2 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$ ~ $\pm(2 - 2^{-23}) \times 2^{127} \approx \pm 3.4 \times 10^{38}$ [Ch3 p.30] (p.26의 $\pm 2.0 \times 10^{\pm 38}$은 대략값). 배정도 행의 범위는 단정도와 같은 규칙으로 계산한 값 (PDF 외 보충).

7. ALU 제어 표 [Ch4 p.18, 20]

명령어	ALUOp	funct7	funct3	ALU 동작	ALU 제어(4비트)
lw / sw	00	X	X	add (주소 계산)	`0010`
beq	01	X	X	subtract (같은지 검사)	`0110`
add	10	0000000	000	add	`0010`
sub	10	0100000	000	subtract	`0110`
and	10	0000000	111	AND	`0000`
or	10	0000000	110	OR	`0001`

입력 전체 조합 $2^{12} = 4096$ → don't care(X)로 압축 [Ch4 p.20]. 다단 디코딩(주 제어 → ALUOp → ALU 제어) 이점: 주 제어 유닛이 작아지고 지연도 줄 수 있음 [Ch4 p.19].

8. 주 제어 신호 표 [Ch4 p.25]

명령어	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R형	0	0	1	0	0	0	10
lw	1	1	1	1	0	0	00
sw	1	X	0	0	1	0	00
beq	0	X	0	0	0	1	01

- ALUSrc: 0 = 레지스터(rs2), 1 = 부호 확장 즉시값 / MemtoReg: 0 = ALU 결과, 1 = 메모리 데이터를 레지스터에 씀
- RegWrite: 레지스터 파일 쓰기 / MemRead·MemWrite: 데이터 메모리 읽기·쓰기 / Branch: 분기 명령어 표시
- PCSrc는 opcode만으로 정할 수 없는 유일한 신호: "beq이고 ALU의 Zero가 1"일 때 1 → 교재 식으로 PCSrc = Branch AND Zero [Ch4 p.25]
- sw·beq의 MemtoReg가 X인 이유: 레지스터에 쓰지 않으므로(RegWrite = 0) 무엇을 고르든 상관없음

9. 손 계산 정답 모음

1. 0x00578833을 어셈블리로 [Ch2 p.50]
2. 슬라이드 p.21의 2진수 `0000000 10101 10100 000 01001 0110011`은 어떤 명령어? [Ch2 p.21–22]
3. 0x003D0500을 x19에 넣기 [Ch2 p.46]
4. x10 = 0x00000DC0, x11 = 0x00003C00일 때 and, or, 그리고 x10 xor 0xFFFFFFFF [Ch2 p.26–27]
5. rs1 = 0xFFFFFFFF, rs2 = 1일 때 blt와 bltu의 분기 여부 [Ch2 p.31]
6. −0.75와 0.0625를 IEEE 754 단정도 16진수로 [Ch3 p.31, 33]

정답 (먼저 풀고 열기)

1. `0000000 00101 01111 000 10000 0110011` → funct7 0, rs2 5, rs1 15, funct3 0, rd 16, R형 → **add x16, x15, x5**
2. rd = 01001 = 9 → **add x9, x20, x21** (슬라이드 텍스트의 "add x0"은 원문 표기 오류, 2진수는 x9)
3. **lui x19, 976** (0x003D0 → 31~12비트) → **addi x19, x19, 1280** (0x500). 하위 12비트 최상위 비트가 1이면 addi가 부호 확장하므로 lui 값을 1 늘려야 함 (PDF 외 보충)
4. and = **0x00000C00**, or = **0x00003DC0**, xor = **0xFFFFF23F** (RISC-V엔 NOT이 없어 xor로 대신)
5. blt: −1 < 1 → **분기함** / bltu: $2^{32} - 1 < 1$ → **분기 안 함**
6. $-0.75 = -1.1_2 \times 2^{-1}$ → S 1, 지수 칸 126(= −1 + 127), 가수 100… → **0xBF400000** / $0.0625 = 1.0_2 \times 2^{-4}$ → 지수 칸 123(= −4 + 127) → **0x3D800000**